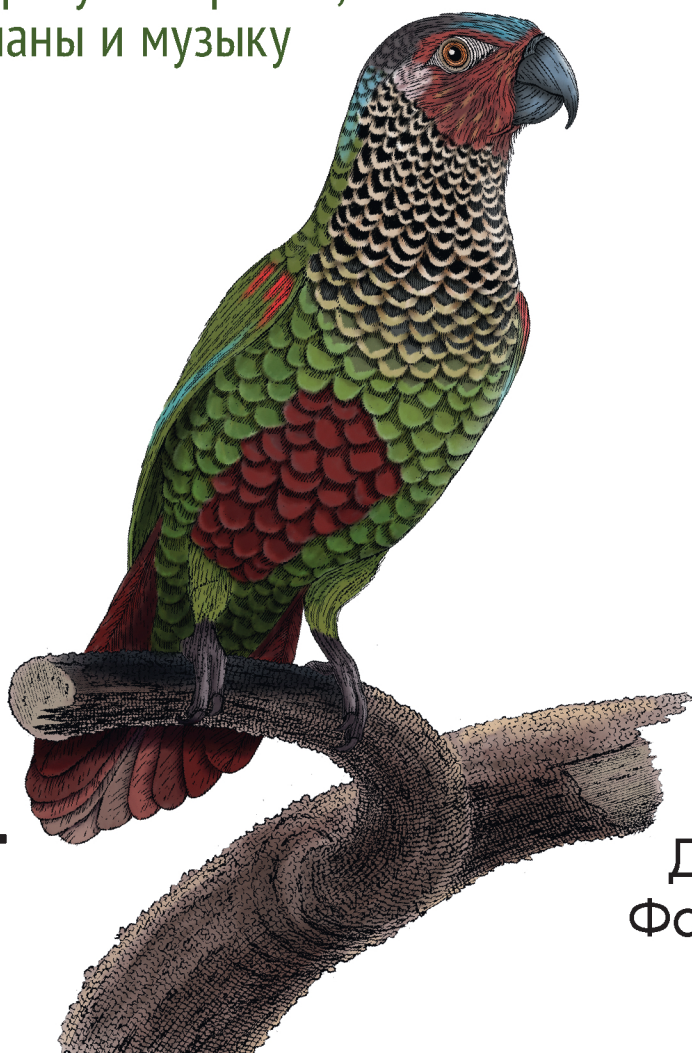


O'REILLY®

Второе
издание

Генеративное глубокое обучение

Как не мы рисуем картины,
пишем романы и музыку



SPRINT
book

Дэвид
Фостер

SECOND EDITION

Generative Deep Learning

*Teaching Machines to Paint, Write,
Compose, and Play*

David Foster

Foreword by Karl Friston

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Генеративное глубокое обучение

Как не мы рисуем картины, пишем романы и музыку

ВТОРОЕ ИЗДАНИЕ

Дэвид Фостер

SPRINT 2024
book

ББК Ф81
УДК 004.8
Ф81

Фостер Дэвид

Ф81 Генеративное глубокое обучение. Как не мы рисуем картины, пишем романы и музыку. 2-е межд изд. — Астана: «Спринт Бук», 2024. — 448 с.: ил.
ISBN 978-601-08-3729-4

Генеративное моделирование — одна из самых обсуждаемых тем в области искусственного интеллекта. Машины можно научить рисовать, писать и сочинять музыку. Вы сами можете посадить искусственный интеллект за парту или мольберт, для этого достаточно познакомиться с самыми актуальными примерами генеративных моделей глубокого обучения: вариационными автокодировщиками, генеративно-состязательными сетями, моделями типа кодер-декодер и многим другим.

Дэвид Фостер делает понятными и доступными архитектуру и методы генеративного моделирования, его советы и подсказки сделают ваши модели более творческими и эффективными в обучении. Вы начнете с основ глубокого обучения на базе Keras, а затем перейдете к самым передовым алгоритмам.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК Ф81
УДК 004.8

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1098134181 англ.

Authorized Russian translation of the English edition of Generative Deep Learning: Teaching Machines To Paint, Write, Compose, and Play 2nd Edition
ISBN 978-1098134181 © 2023 Applied Data Science Partners Ltd.

ISBN 978-601-08-3729-4

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.
© Перевод на русский язык ТОО «Спринт Бук», 2024
© Издание на русском языке, оформление ТОО «Спринт Бук», 2024

Отзывы о книге

В книге «Генеративное глубокое обучение» доступным языком описывается набор инструментов глубокого обучения для генеративного моделирования. Если вы творческая личность, любите возиться с кодом и желаете применить глубокое обучение в своей работе, то эта книга для вас.

*Дэвид Ха (David Ha), руководитель отдела
стратегии, Stability AI*

Отличная книга, в которой подробно рассматриваются основные методы, лежащие в основе современных технологий генеративного глубокого обучения. В ней вы найдете ясные объяснения и остроумные аналогии, подкрепленные дидактическими легко читаемыми примерами кода. Захватывающее исследование одной из самых увлекательных областей искусственного интеллекта!

Франсуа Шолле (François Chollet), создатель Keras

Дэвид Фостер кратко и доходчиво объясняет сложные понятия, дополняя объяснения наглядными иллюстрациями, примерами кода и упражнениями. Отличный источник знаний для студентов и практиков!

*Сюзанна Илич (Suzana Ilić), главный
менеджер программы ответственного ИИ,
Microsoft Azure OpenAI*

Генеративный ИИ — это следующий революционный шаг в технологии ИИ, который окажет огромное влияние на мир, и данная книга отлично описывает эту область, ее невероятный потенциал и возможные риски.

*Коннор Лихи (Connor Leahy), генеральный
директор Conjecture и соучредитель EleutherAI*

Предсказать мир означает понять мир — во всех его проявлениях. В этом смысле генеративный ИИ раскрывает самую суть интеллекта.

*Йонас Андрулис (Jonas Andrulis), основатель
и генеральный директор Aleph Alpha*

Генеративный ИИ реорганизует многие и многие отрасли и создает новое поколение инструментов для творчества. Книга дает уникальную возможность освоить генеративное моделирование и начать творить самостоятельно с помощью этой революционной технологии.

*Эд Ньютон-Рекс (Ed Newton-Rex), вице-президент
по звуку в Stability AI и композитор*

Дэвид научил меня всему, что я знаю о машинном обучении, и умело объяснил основные концепции. «Генеративное глубокое обучение» — моя любимая книга о генеративном ИИ. Она стоит на полке над моим рабочим столом среди других любимых технических книг.

*Зак Заутт (Zack Thoutt), директор контроля
производства в AutoSalesVelocity*

Генеративный ИИ почти наверняка сильно повлияет на общество. В книге эта область описана доступным языком и с техническими подробностями.

*Раза Хабиб (Raza Habib), соучредитель
Humanloop*

Когда люди спрашивают, как начать работу с генеративным ИИ, я всегда рекомендую книгу Дэвида. Второе издание великолепно, потому что оно охватывает самые мощные модели: диффузионные и трансформеры. Настоятельно рекомендую всем, кто интересуется моделированием творчества!

*Доктор Тристан Беренс (Dr. Tristan Behrens),
эксперт по ИИ и музыкальному творчеству ИИ,
резидент KI Salon Heilbronn*

Как источник глубоких технических сведений эта книга — первая, к которой я обращаюсь, когда возникают идеи относительно генеративного ИИ. Она должна стоять на книжной полке каждого специалиста по данным.

*Мартин Мусиол (Martin Musiol),
основатель generativeAI.net*

В издании очень подробно описана полная классификация генеративных моделей. Но самая замечательная черта книги — она охватывает теоретические основы моделей, а также помогает читателю закрепить полученные знания на практических примерах. Я должен отметить, что глава о генеративно-сопоставительных сетях (GAN) — одна из лучших из встречавшихся мне по этой теме. Она четко и ясно описывает приемы точной настройки моделей. В книге рассматривается широкий спектр видов генеративного ИИ, в том числе для обработки текста, изображений и музыки. Отличный ресурс для тех, кто начинает работать с генеративным ИИ.

*Айшвария Шринивасан (Aishwarya Srinivasan),
специалист по данным, Google Cloud*

Краткое содержание

Предисловие.....	18
Вступление.....	19

ЧАСТЬ I. ВВЕДЕНИЕ В ГЕНЕРАТИВНОЕ ГЛУБОКОЕ ОБУЧЕНИЕ

Глава 1. Генеративное моделирование.....	31
Глава 2. Глубокое обучение	51

ЧАСТЬ II. МЕТОДЫ

Глава 3. Вариационные автокодировщики.....	87
Глава 4. Генеративно-состязательные сети.....	123
Глава 5. Модели авторегрессии	158
Глава 6. Модели нормализующих потоков.....	194
Глава 7. Модели на основе энергии.....	215
Глава 8. Модели диффузии	231

ЧАСТЬ III. ПРАКТИЧЕСКОЕ ПРИМЕНЕНИЕ

Глава 9. Трансформеры.....	261
Глава 10. Продвинутые GAN	291
Глава 11. Генерирование музыки.....	320
Глава 12. Модели мира	353
Глава 13. Мультимодальные модели.....	379
Глава 14. Заключение	411
Ссылки.....	437
Об авторе	443
Иллюстрация на обложке.....	444

Оглавление

Предисловие	18
Вступление	19
Цели и подходы.....	20
Предварительные условия	21
Структура издания.....	21
Изменения во втором издании	23
Прочие ресурсы.....	24
Условные обозначения.....	25
Примеры кода	25
Использование программного кода примеров.....	26
Благодарности	27
От издательства	29

ЧАСТЬ I. ВВЕДЕНИЕ В ГЕНЕРАТИВНОЕ ГЛУБОКОЕ ОБУЧЕНИЕ

Глава 1. Генеративное моделирование	31
Что такое генеративное моделирование.....	31
Генеративное и дискриминативное моделирование.....	33
Появление генеративного моделирования	35
Генеративное моделирование и ИИ	36
Наша первая генеративная модель.....	37
Привет, мир!.....	37
Базовые принципы генеративного моделирования.....	39
Обучение представлению.....	41
Основы теории вероятностей.....	44
Классификация генеративных моделей	47
Код примеров генеративного глубокого обучения.....	48
Клонирование репозитория	49
Использование Docker	49
Применение графического процессора	49
Резюме.....	49

Глава 2. Глубокое обучение	51
Данные для глубокого обучения	51
Глубокие нейронные сети	53
Что такое нейронная сеть.....	53
Выявление высокоуровневых признаков.....	55
TensorFlow и Keras.....	56
Многослойный перцептрон	56
Подготовка данных	57
Конструирование модели.....	59
Компиляция модели	64
Обучение модели	65
Оценка модели	67
Сверточные нейронные сети (CNN).....	69
Сверточные слои	69
Пакетная нормализация	75
Слои прореживания.....	78
Построение CNN	80
Обучение и оценка CNN	83
Резюме.....	84

ЧАСТЬ II. МЕТОДЫ

Глава 3. Вариационные автокодировщики.....	87
Введение	88
Автокодировщики	89
Набор данных Fashion-MNIST.....	90
Архитектура автокодировщика.....	91
Автокодировщик	92
Декодировщик	93
Объединение кодировщика и декодировщика	96
Реконструкция изображений	97
Визуализация скрытого пространства	98
Генерирование новых изображений	99
Вариационный автокодировщик	102
Кодировщик.....	103
Функция потерь	109

Обучение вариационного автокодировщика	110
Анализ вариационного автокодировщика	112
Исследование скрытого пространства	113
Набор данных CelebA	114
Обучение вариационного автокодировщика	115
Анализ вариационного автокодировщика	117
Генерирование новых лиц	118
Арифметика скрытого пространства	119
Преобразование одного лица в другое.....	121
Резюме.....	122
Глава 4. Генеративно-сопоставительные сети.....	123
Введение	124
Глубокая сверточная GAN	126
Набор данных Bricks.....	126
Дискриминатор.....	127
Генератор.....	130
Обучение DCGAN.....	133
Анализ DCGAN	138
Обучение GAN: советы и рекомендации.....	139
Дискриминатор получает подавляющее преимущество перед генератором.....	139
Генератор получает подавляющее преимущество перед дискриминатором	140
Неинформативные потери.....	141
Гиперпараметры	142
Решение проблем генеративно-сопоставительных сетей	142
Генеративно-сопоставительные сети Вассерштейна со штрафом за градиент	142
Функция потерь Вассерштейна	143
Ограничение Липшица	144
Реализация ограничения Липшица.....	145
Функция потерь со штрафом за градиент	146
Обучение WGAN-GP.....	148
Анализ WGAN-GP.....	150

Условные генеративно-сопоставительные сети	151
Архитектура CGAN.....	152
Обучение CGAN.....	154
Анализ CGAN.....	155
Резюме.....	156
Глава 5. Модели авторегрессии	158
Введение	159
Сети с долгой краткосрочной памятью	160
Набор данных Recipes	161
Работа с текстовыми данными	162
Лексемизация.....	164
Создание набора обучающих данных	166
Архитектура модели LSTM.....	167
Слой Embedding	168
Слой LSTM	169
Ячейка LSTM.....	171
Обучение LSTM	173
Анализ LSTM.....	175
Расширения RNN	179
Многослойные рекуррентные сети.....	179
Управляемые рекуррентные блоки.....	180
Двунаправленные ячейки	182
PixelCNN	182
Маскированные сверточные слои.....	183
Остаточные блоки.....	185
Обучение PixelCNN	186
Анализ PixelCNN	188
Смесь распределений	190
Резюме.....	193
Глава 6. Модели нормализующих потоков.....	194
Введение	195
Нормализующие потоки.....	197
Замена переменных.....	197

Определитель якобиана.....	199
Уравнение замены переменных	200
RealNVP.....	201
Набор данных Two Moons.....	202
Слои связи.....	202
Обучение модели RealNVP	207
Анализ модели RealNVP	210
Другие модели нормализующего потока	212
GLOW	212
FFJORD.....	213
Резюме.....	214
Глава 7. Модели на основе энергии.....	215
Введение	215
Модели на основе энергии	217
Набор данных MNIST.....	218
Функция энергии.....	219
Выборка с использованием динамики Ланжевена.....	220
Обучение с контрастивной дивергенцией.....	223
Анализ модели на основе энергии.....	227
Другие модели на основе энергии.....	229
Резюме.....	230
Глава 8. Модели диффузии	231
Введение	232
Модели удаления шума	234
Набор данных Flowers.....	234
Процесс прямой диффузии	236
Трюк с перепараметризацией.....	237
Режимы диффузии.....	237
Процесс обратной диффузии.....	240
Модель удаления шума U-Net.....	243
Обучение диффузионной модели	250
Выборка из диффузионной модели удаления шума	251
Анализ модели	254
Резюме.....	257

ЧАСТЬ III. ПРАКТИЧЕСКОЕ ПРИМЕНЕНИЕ

Глава 9. Трансформеры	261
Введение	262
GPT.....	262
Набор данных Wine Reviews.....	263
Внимание	264
Запросы, ключи и значения	265
Многоголовое внимание.....	267
Причинно-следственная маскировка	268
Блок трансформера	271
Позиционное представление	274
Обучение GPT	276
Анализ GPT	277
Другие трансформеры	280
T5	281
GPT-3 и GPT-4.....	284
ChatGPT.....	286
Резюме.....	290
Глава 10. Продвинутые GAN	291
Введение	292
ProGAN	292
Прогрессивное обучение	293
Выходы	299
StyleGAN	300
Сеть отображения.....	302
Синтезирующая сеть	302
Вывод сети StyleGAN	303
StyleGAN2.....	305
Модуляция и демодуляция весов	306
Регуляризация длины пути	307
Вывод сети StyleGAN2.....	309
Другие важные генеративно-состязательные сети	310
Self-Attention GAN	310
BigGAN	312

VQ-GAN	313
ViT VQ-GAN	317
Резюме	318
Глава 11. Генерирование музыки	320
Введение	321
Генерирование музыки с помощью модели трансформера	322
Набор данных JS Bach Cello Suite	323
Парсинг MIDI-файлов	324
Кодирование	325
Создание обучающего набора	327
Синусоидальное позиционное кодирование	328
Несколько входов и выходов	329
Анализ трансформера генерирования музыки	331
Генерирование полифонической музыки	335
MuseGAN	339
Набор данных Bach Chorale	339
Генератор MuseGAN	342
Критик MuseGAN	348
Анализ сети MuseGAN	349
Резюме	351
Глава 12. Модели мира	353
Введение	353
Обучение с подкреплением	354
CarRacing	356
Обзор модели мира	358
Архитектура	358
Обучение	360
Сбор данных в ходе случайных прогонов	361
Обучение VAE	363
Архитектура VAE	363
Анализ VAE	365
Сбор данных для обучения RNN	367
Обучение сети MDN-RNN	367
Архитектура сети MDN-RNN	368
Выборка следующего состояния и вознаграждения из MDN-RNN	369

Обучение контроллера	370
Архитектура контроллера.....	370
CMA-ES.....	371
Параллельное выполнение алгоритма CMA-ES.....	373
Обучение в мнимом окружении.....	375
Резюме.....	378
Глава 13. Мультимодальные модели.....	379
Введение	380
DALL·E 2	381
Архитектура.....	381
Кодировщик.....	382
CLIP.....	382
Модель выборки.....	387
Декодировщик	389
Примеры изображений, сгенерированных моделью DALL·E 2	393
Imagen	396
Архитектура.....	397
DrawBench.....	398
Примеры изображений, созданных Imagen	399
Stable Diffusion	399
Архитектура.....	400
Примеры изображений, созданных Stable Diffusion	401
Flamingo.....	401
Архитектура.....	402
Vision Encoder.....	402
Perceiver Resampler.....	403
Языковая модель.....	405
Примеры использования Flamingo.....	408
Резюме.....	409
Глава 14. Заключение.....	411
Хронология генеративного ИИ.....	412
2014–2017 годы — эпоха VAE и GAN.....	412
2018–2019 годы — эпоха трансформеров	414
2020–2022 годы — эпоха больших моделей.....	415

Текущее состояние генеративного ИИ	416
Большие языковые модели.....	416
Модели преобразования текста в программный код.....	420
Модели преобразования текста в изображение	422
Другие приложения	426
Будущее генеративного ИИ	428
Генеративный ИИ в повседневной жизни.....	428
Генеративный ИИ на рабочем месте	430
Генеративный ИИ в образовании.....	431
Практические и этические проблемы генеративного ИИ.....	432
Заключительные комментарии	435
Ссылки	437
Об авторе	443
Иллюстрация на обложке	444

*Посвящается Алине, самому
очаровательному шумовому вектору из всех.*

Предисловие

Эта книга стала частью моей жизни. Найдя ее экземпляр в своей гостиной, я спросил сына: «Где ты ее достал?» Он, ошеломленный моей забывчивостью, ответил: «Ты сам мне ее дал». Читая вместе с сыном разные главы, я пришел к выводу, что «Генеративное глубокое обучение» — это «Анатомия Грея»¹ для генеративного искусственного интеллекта (ИИ).

Автор анализирует анатомию генеративного ИИ авторитетно и с невероятной ясностью. Он предлагает поистине замечательный отчет о быстро меняющейся области, подкрепленный практичными примерами, увлекательными повествованиями и ссылками, которые настолько актуальны, что кажутся живой историей.

На протяжении всей книги автор не перестает удивляться и восхищаться потенциалом генеративного ИИ, что особенно заметно в убедительной развязке. Раскрыв технологию, он напоминает, что мы находимся на заре новой эпохи интеллекта — эпохи, в которой генеративный ИИ является зеркалом нашего языка, искусства, творчества, отражая не только то, что мы создали, но и то, что могли бы создать — что можем создать, — ограниченные только «собственным воображением».

Центральная тема генеративных моделей в искусственном интеллекте глубоко резонирует со мной, потому что я вижу точно такие же течения в естественных науках, а именно взгляд на себя как на генеративную модель нашего живого мира. Я подозреваю, что в следующем издании этой книги мы прочтем о слиянии искусственного и естественного интеллектов. До этого времени я буду хранить свой экземпляр на книжной полке рядом с томиком «Анатомии Грея» и другими сокровищами.

*Карл Фристон (Karl Friston), член Королевского общества, профессор нейробиологии
Университетского колледжа в Лондоне*

¹ «Анатомия Грея» — признанный классическим учебник анатомии человека (Анатомия Грея. Анатомические структуры с оригинальной и современной терминологией на английском, латинском и русском языках / Под ред. Г. Л. Билича, Е. Ю. Зигаловой. — М.: Эксмо, 2021). — *Примеч. пер.*

Вступление

Чего не могу воссоздать, того не понимаю.

Ричард Фейнман (Richard Feynman)

Генеративный ИИ — одна из самых революционных технологий нынешнего времени, меняющая способ нашего взаимодействия с машинами. Его возможность внести революционные изменения в нашу жизнь, работу и игры была предметом бесчисленных разговоров, дебатов и прогнозов. А если бы эта мощная технология имела еще больший потенциал? Если возможности генеративного ИИ выйдут за рамки того, что мы можем вообразить сегодня? Будущее генеративного ИИ может оказаться более захватывающим, чем можно было бы представить...

Еще на заре своего существования, живя в пещерах, человек искал возможность создавать оригинальные и красивые творения. Творчество первобытного человека выражалось в наскальных рисунках, которые изображали диких животных и абстрактные узоры и были созданы с помощью пигментов, аккуратно и методично нанесенных на камень. Эпоха романтизма подарила нам чудо симфоний Чайковского с их способностью вызывать чувство восхищения и грусти благодаря звуковым волнам, сплетенным в прекрасные мелодии и гармонии. И в наши дни мы, бывает, выскакиваем в полночь из дому, чтобы забежать в книжный магазин и купить продолжение истории о вымышленном волшебнике, потому что комбинация букв образует захватывающее повествование, заставляющее нас переворачивать страницу за страницей, чтобы узнать, какие еще приключения ждут нашего героя.

Поэтому неудивительно, что человечество задалось главным вопросом: можем ли мы создать что-то, что было бы творческим само по себе?

Поиск ответа на этот вопрос является целью генеративного моделирования. Благодаря последним достижениям науки и техники мы можем создавать машины, способные рисовать оригинальные картины в определенном стиле, писать абзацы связного текста с хорошо прослеживаемой структурой, сочинять музыку, которую приятно слушать, и разрабатывать выигрышные стратегии для сложных игр, генерируя сценарии возможного развития событий. И это только начало генеративной революции, не оставляющей нам другого выбора, кроме отыскания ответов на некоторые из самых важных вопросов о механике творчества и в конечном счете о том, что значит быть человеком.

Иными словами, сейчас самое время заняться изучением генеративного моделирования, так давайте приступим!

Цели и подходы

Эта книга не предполагает наличия у читателя каких-либо знаний о генеративном ИИ. Мы познакомимся со всеми ключевыми идеями по ходу, поэтому не волнуйтесь, если у вас нет опыта работы с генеративным ИИ. Все необходимое вы найдете здесь!

Книга не только описывает современные методы, но и служит полным руководством по генеративному моделированию, охватывающему широкий спектр семейств моделей. Не существует ни одного метода, который был бы объективно лучше или хуже любого другого, — на самом деле во многих современных моделях смешиваются идеи из самых разных подходов к генеративному моделированию. Поэтому важно быть в курсе событий во всех областях генеративного ИИ, а не сосредотачиваться на одной конкретной технике. Одно можно сказать наверняка: область генеративного ИИ развивается очень быстро и невозможно предугадать, откуда придет следующая революционная идея.

Учитывая все это, я попробую показать вам, как обучать свои генеративные модели на собственных данных, не полагаясь на готовые и предварительно обученные модели. В настоящее время существует множество впечатляющих генеративных моделей с открытым исходным кодом, которые можно загрузить и запустить, написав несколько строк кода, но цель этой книги иная — изучить их архитектуру и приемы проектирования, учитывающие основные принципы, чтобы вы хорошо разобрались в особенностях их работы и могли запрограммировать любой метод, используя Python и Keras.

Таким образом, эту книгу можно рассматривать как карту современного ландшафта генеративного ИИ, охватывающую не только теорию, но и реальное применение и включающую практические примеры ключевых моделей. Мы шаг за шагом рассмотрим код каждого метода и отметим те или иные особенности реализации теории, лежащей в его основе. Книгу можно читать последовательно, от начала до конца, или использовать как справочник, выбирая только интересующие вас темы. Но в любом случае я надеюсь, что вы найдете ее полезной и интересной!



На протяжении всей книги вам будут встречаться короткие поучительные истории, помогающие объяснить механику некоторых моделей. Пожалуй, один из лучших способов изучения новой абстрактной теории — сначала преобразовать ее во что-то менее абстрактное, например в рассказ, и только потом погружаться в техническое описание. Рассказы и технические описания моделей повторяют друг друга, объясняя одно и то же с разных точек зрения, поэтому иногда может быть полезно вернуться к соответствующему рассказу в процессе изучения технических деталей каждой модели.

Предварительные условия

Предполагается, что у читателя есть опыт программирования на Python. Если вы не знакомы с Python, начните его изучение с сайта LearningPython.org (<https://www.learnpython.org/>). В Интернете есть много бесплатных ресурсов, позволяющих приобрести достаточный объем знаний о Python для работы с примерами, приведенными в этой книге.

Кроме того, некоторые модели описаны с использованием математических обозначений, поэтому будет полезно иметь представление о линейной алгебре (например, как выполняется умножение матриц и т. д.) и общей теории вероятностей. Хорошим источником этих знаний является книга Дайзенрота (Deisenroth) и других авторов *Mathematics for Machine Learning*¹ (Cambridge University Press).

Эта книга не предполагает наличия у читателя каких-либо знаний в области генеративного моделирования (мы рассмотрим ключевые концепции в главе 1) или TensorFlow и Keras (эти библиотеки будут представлены в главе 2).

Структура издания

Книга разделена на три части.

Часть I — это введение в генеративное моделирование и глубокое обучение. Здесь мы исследуем главные идеи, лежащие в основе всех методов, описанных в последующих частях книги.

- В главе 1 «Генеративное моделирование» дается определение генеративного моделирования и рассматривается небольшой пример, который поможет уяснить некоторые ключевые понятия, важные для всех генеративных моделей. Здесь также приводится классификация семейств генеративных моделей, которую будем изучать в части II.
- В главе 2 «Глубокое обучение» вы начнете знакомиться с глубоким обучением и нейронными сетями, создав первый пример многослойного перцептрона (multilayer perceptron, MLP) с помощью Keras. Затем мы усовершенствуем его, включив сверточные слои и внеся другие улучшения, чтобы увидеть разницу в качестве работы.

В части II рассматриваются шесть ключевых методов, которые мы будем использовать для построения генеративных моделей, и приводятся практические примеры реализации каждого из них.

- В главе 3 «Вариационные автокодировщики» мы рассмотрим вариационный автокодировщик (variational autoencoder, VAE) и возможность его

¹ Дайзенрот М. П., Фейзал А. А., Он Ч. С. Математика в машинном обучении. — СПб.: Питер, 2023.

применения для генерации изображений лиц и плавных переходов между лицами в скрытом пространстве модели.

- В главе 4 «Генеративно-сопоставительные сети» исследуем генеративно-сопоставительные сети (Generative Adversarial Networks, GAN), применяемые для генерации изображений, включая глубокие сверточные GAN, условные GAN, а также некоторые усовершенствованные модели, такие как GAN Вассерштейна, отличающиеся более высокой стабильностью обучения.
- В главе 5 «Модели авторегрессии» мы начнем с введения в рекуррентные нейронные сети, такие как сети с долгой краткосрочной памятью (Long Short-Term Memory, LSTM), используемые для генерации текста, и PixelCNN, применяемые для генерации изображений.
- В главе 6 «Модели нормализующих потоков» сосредоточимся на нормализующих потоках: исследуем теорию, лежащую в основе метода, и рассмотрим практический пример построения модели RealNVP для генерации изображений.
- В главе 7 «Модели на основе энергии» мы рассмотрим модели на основе энергии (energy-based models), включая такие важные методы, как обучение с использованием контрастной дивергенции и выборка с применением динамики Ланжевена.
- В главе 8 «Модели диффузии» погрузимся в практическое руководство по построению моделей диффузии, используемых во многих современных моделях генерации изображений, таких как DALL·E 2 и Stable Diffusion.

Наконец, в части III мы, опираясь на материал из предыдущих глав, приступим к изучению внутренних особенностей работы современных моделей для генерации изображений, составления текстов, сочинения музыки и обучения с подкреплением на основе моделей.

- В главе 9 «Трансформеры» исследуем происхождение и технические детали моделей StyleGAN, а также других современных GAN, применяемых для генерации изображений, таких как VQ-GAN.
- В главе 10 «Продвинутые GAN» рассмотрим архитектуру Transformer и познакомимся с практическими рекомендациями по созданию своей версии GPT для генерации текста.
- В главе 11 «Генерирование музыки» обратим внимание на особенности работы с музыкальными данными и применение таких методов, как Transformers и MuseGAN.
- В главе 12 «Модели мира» вы увидите, как можно использовать генеративные модели в контексте обучения с подкреплением с применением моделей мира и методов, основанных на трансформерах.

- В главе 13 «Мультимодальные модели» вы познакомитесь с особенностями работы четырех современных мультимодальных моделей, способных обрабатывать разнотипные данные, включая DALL·E 2, Imagen и Stable Diffusion, которые используются для преобразования текста в изображение, и Flamingo — модель визуального языка.
- В главе 14 «Заключение» мы подведем итоги основных этапов развития генеративного ИИ на сегодняшний день и обсудим изменения, которые он может внести в нашу повседневную жизнь в ближайшие годы.

Изменения во втором издании

Спасибо всем, кто прочитал первое издание этой книги. Мне очень приятно, что многие из вас нашли ее полезной и высказали свое мнение о том, что хотели бы видеть во втором издании. Область глубокого генеративного обучения значительно продвинулась в развитии с 2019 года — момента публикации первого издания, поэтому я не только обновил содержимое книги, но и добавил несколько новых глав, чтобы привести материал в соответствие с современным состоянием дел.

Далее кратко изложено, в чем заключаются основные обновления отдельных глав и общее улучшение книги.

- Глава 1 теперь включает раздел, посвященный различным семействам генеративных моделей, их классификации и взаимосвязям между ними.
- Глава 2 содержит улучшенные диаграммы и более подробные объяснения ключевых понятий.
- Глава 3 дополнена новым проработанным примером и сопровождающими его пояснениями.
- Глава 4 теперь включает объяснение условных архитектур GAN.
- В главу 5 включен раздел, посвященный моделям авторегрессии для изображений, например PixelCNN.
- Глава 6 — совершенно новая, в ней описывается модель RealNVP.
- Глава 7 тоже новая, в ней рассматриваются такие методы, как динамика Ланжевена и контрастивная дивергенция.
- Глава 8 написана недавно и посвящена диффузионным моделям удаления шума, лежащим в основе многих современных приложений.
- Глава 9 — это расширение материала, представленного в заключении первого издания. В ней более глубоко рассматривается архитектура различных моделей StyleGAN и представлен новый материал о VQ-GAN.

- Глава 10 — новая, в ней подробно говорится об архитектуре Transformer.
- Глава 11 включает современные архитектуры Transformer, заменяющие модели LSTM из первого издания.
- Глава 12 включает обновленные диаграммы и описания, а также раздел о влиянии этого подхода на современное обучение с подкреплением.
- Глава 13 — новая, в ней подробно объясняется, как работают такие потрясающие модели, как DALL·E 2, Imagen, Stable Diffusion и Flamingo.
- Глава 14 обновлена, чтобы отразить выдающийся прогресс в этой области, произошедший со времени выхода в свет первого издания, и дать более полное и подробное представление о том, куда движется генеративный ИИ.
- Были учтены все комментарии к первому изданию и исправлены выявленные опечатки.
- В начало каждой главы добавлен список ее целей, чтобы вы могли увидеть, какие ключевые темы в ней затронуты.
- Некоторые аллегорические истории были переписаны, чтобы сделать их более краткими и ясными. Я рад тому, что, судя по отзывам, многим читателям эти истории помогли лучше понять ключевые понятия!
- Заголовки и подзаголовки всех глав реорганизованы так, чтобы было понятно, какие их части посвящены объяснениям, а какие — построению собственных моделей.

Прочие ресурсы

В качестве общего введения в машинное и глубокое обучение рекомендуются две книги:

- *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* Опельена Жерона (Aurelien Geron), вышедшая в издательстве O'Reilly ¹ (<https://learning.oreilly.com/library/view/hands-on-machine-learning/9781098125967/>);
- *Deep Learning with Python* Франсуа Шолле (Francois Chollet), вышедшая в издательстве Manning².

Большинство статей, упоминаемых в этой книге, получены из arXiv (<https://arxiv.org/>) — бесплатного репозитория научных статей. Сейчас многие авторы публикуют свои статьи в arXiv до их рецензирования. Обзор последних по-

¹ Жерон О. Прикладное машинное обучение с помощью Scikit-Learn и TensorFlow. Концепции, инструменты и техники для создания интеллектуальных систем.

² Шолле Ф. Глубокое обучение на Python. 2-е межд. издание. — СПб.: Питер, 2023.

ступлений в репозиторий — отличный способ быть в курсе самых передовых разработок в области машинного обучения.

Я также настоятельно рекомендую сайт Papers with Code (<https://paperswithcode.com/>), где можно найти самые свежие решения различных задач машинного обучения, ссылки на статьи и официальные репозитории GitHub. Это отличный ресурс для любого желающего выяснить, какие современные методы помогают достичь самых высоких результатов, и он, безусловно, помог мне решить, какие методы включить в эту книгу.

Условные обозначения

В книге приняты следующие шрифтовые соглашения.

Курсив

Используется для обозначения новых терминов и важных моментов.

Моноширинный шрифт

Применяется для оформления листингов программ, имен файлов и их расширений, программных элементов внутри обычного текста: имен переменных и функций, баз данных, типов данных, переменных окружения, инструкций, ключевых слов.



Так оформляются подсказки или предложения.



Так оформляются общие примечания.



Так оформляются предупреждения или предостережения.

Примеры кода

Примеры кода, приведенные в этой книге, имеются в репозитории GitHub (https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition). Я постарался сделать так, чтобы ни одна из моделей не требовала для обучения непомерно большого количества вычислительных ресурсов и вы могли начать обучение собственных моделей, не тратя много времени и денег на дорогостоящее

оборудование. В репозитории вы найдете подробное руководство, помогающее начать работу с Docker и при необходимости настроить облачные ресурсы с графическими процессорами в Google Cloud.

Со времени выхода первого издания в код примеров были внесены следующие изменения.

- Все примеры теперь можно запускать из одного блокнота, а не импортировать часть кода из модулей всей кодовой базы. Это сделано для того, чтобы вы могли выполнять все примеры по порядку, ячейка за ячейкой и шаг за шагом вникать, как именно строится каждая модель.
- Разделы каждого блокнота теперь в целом согласованы между примерами.
- Во многих примерах используются фрагменты кода из замечательного репозитория Keras с открытым исходным кодом (<https://oreil.ly/1UTwa>). Это сделано для того, чтобы не создавать отдельный репозиторий с исходным кодом примеров генеративного ИИ, когда уже существуют отличные реализации, доступные на сайте Keras. В этой книге и в репозитории я привел ссылки с сайта Keras на авторов используемого кода.
- Я добавил новые источники данных и улучшил процесс сбора данных — теперь есть сценарий, который можно запустить для сбора данных из необходимых источников и для обучения примерам из книги с помощью таких инструментов, как Kaggle API (<https://oreil.ly/8ibPw>).

Использование программного кода примеров

Вспомогательные материалы (примеры кода, упражнения и т. д.) доступны для загрузки по адресу https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition.

Если у вас есть вопросы технического характера или возникла проблема с использованием примеров кода, отправьте электронное письмо по адресу bookquestions@oreilly.com.

В общем случае все примеры кода из книги вы можете использовать в своих программах и в документации. Вам не нужно обращаться в издательство за разрешением, если вы не собираетесь воспроизводить существенные части программного кода. Если вы разрабатываете программу и используете в ней несколько фрагментов кода из книги, вам не нужно обращаться за разрешением. Но для продажи или распространения примеров из книги вам потребуется разрешение от издательства O'Reilly. Вы можете отвечать на вопросы, цитируя данную книгу или примеры из нее, но для включения существенных объемов программного кода из книги в документацию вашего продукта потребуется разрешение.

Мы рекомендуем, но не требуем добавлять ссылку на первоисточник при цитировании. Под ссылкой на первоисточник мы подразумеваем указание авторов, издательства и ISBN.

За получением разрешения на использование значительных объемов программного кода из книги обращайтесь по адресу permissions@oreilly.com.

Благодарности

Позвольте поблагодарить всех, кто помогал мне в работе над этой книгой.

Прежде всего искреннее спасибо тем, кто нашел время для научного редактирования, в частности: Вишвеше Рави Шримали (Vishwesh Ravi Shrimali), Липи Дипаакши Патнаику (Lipi Deepaakshi Patnaik), Любе Эллиотт (Luba Elliott) и Лорне Барклай (Lorna Barclay). Большое спасибо также Самиру Бико (Samir Bico) за помощь в рецензировании и тестировании кодовой базы, прилагаемой к книге. Ваш вклад был неоценим.

Кроме того, выражаю огромную благодарность коллегам из Applied Data Science Partners (<https://adsp.ai/>): Росс Витешчаку (Ross Witeszczak), Эми Булл (Amy Bull), Али Паранде (Ali Parandeh), Зину Эддину (Zine Eddine), Джо Роу (Joe Rowe), Герте Салиллари (Gerta Salillari), Алишии Паркс (Aleshia Parkes), Эвелине Кирейлите (Evelina Kireilyte), Риккардо Толли (Riccardo Tolli), Мэй До (Mai Do), Халилу Саеду (Khaleel Syed) и Уиллу Холмсу (Will Holmes). Спасибо за вашу благосклонность в течение всего времени работы над книгой, с нетерпением жду новых проектов машинного обучения, реализацией которых в будущем мы будем заниматься вместе! Особое спасибо Россу за веру в меня как в делового партнера — если бы мы не решили начать совместный бизнес, то эта книга, возможно, никогда бы не появилась!

Я также хочу поблагодарить всех, кто когда-либо учил меня математике, — мне очень повезло, что в школе у меня были потрясающие преподаватели, которые развили мой интерес к этому предмету и призывали меня продолжать заниматься им в университете. Благодарю вас за самоотдачу и за то, что щедро делились со мной своими знаниями.

Огромное спасибо сотрудникам издательства O'Reilly за то, что помогали мне в работе над этой книгой. Особую благодарность хочу выразить Мишель Кронин (Michele Cronin), помогавшей на всех этапах полезной обратной связью и посылавшей мне дружеские напоминания о необходимости продолжать писать главы! Спасибо также Николь Баттерфилд (Nicole Butterfield), Кристоферу Фошеру (Christopher Faucher), Чарльзу Румелиотису (Charles Roumeliotis), Сюзанне Хьюстон (Suzanne Huston) за подготовку книги к печати и Майку

Лоукидесу (Mike Loukides), который первым обратился ко мне с предложением написать книгу. Вы все оказали огромную поддержку этому проекту, и я хочу поблагодарить вас за то, что вы предоставили мне возможность написать книгу о том, что я люблю.

Все время, пока я писал книгу, моя семья оказывала мне всяческую поддержку. Огромное спасибо моей маме Джиллиан Фостер (Gillian Foster) за проверку каждой строки текста на наличие опечаток и особенно за то, что научила меня считать! Твое внимание к деталям чрезвычайно помогло при редактировании этой книги, и я очень благодарен за все возможности, которые предоставили мне ты и папа. Мой папа Клайв Фостер (Clive Foster) научил меня программированию — эта книга полна практических примеров, которые я сумел создать во многом благодаря его терпению в те годы, когда изучал Бейсик и пытался писать футбольные игры. Мой брат Роб Фостер (Rob Foster) — самый скромный гений из всех гениев, особенно в области лингвистики, беседы с ним об искусственном интеллекте и будущем машинного обучения на основе текстов оказались на удивление плодотворными. Наконец, я хотел бы поблагодарить свою сестру Нану (Nana) — неиссякаемый источник вдохновения и радости. Ее любовь к литературе — одна из причин, по которой я решил, что написать книгу было бы захватывающим занятием.

Наконец, я хотел бы поблагодарить свою жену Лорну Барклай (Lorna Barclay). Она не только прочла каждое слово в этой книге, но и оказывала мне бесконечную поддержку на протяжении всего процесса, заваривала чай и приносила разные вкусняшки. Я едва ли смог бы завершить этот проект без тебя. Спасибо за то, что всегда была рядом и сделала это путешествие намного приятнее. Обещаю, что не буду говорить о генеративном ИИ за обеденным столом по крайней мере несколько дней после выхода книги.

Наконец, я хочу поблагодарить нашу прекрасную маленькую дочку Алину за непередаваемую радость, которую я испытывал, видя ее после долгих ночей работы над книгой. Твой очаровательный смех стал идеальной фоновой музыкой для меня. Спасибо за то, что вдохновляешь меня и всегда держишь в тонусе.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу
`comp@sprintbook.kz` (издательство «SprintBook», компьютерная редакция).

Мы будем рады узнать ваше мнение!

Часть I

Введение в генеративное глубокое обучение

Часть I содержит общую информацию о генеративном моделировании и глубоком обучении.

В **главе 1** вы познакомитесь с определением генеративного моделирования, мы рассмотрим небольшой пример, который поможет разобраться в некоторых ключевых понятиях, важных для всех генеративных моделей. Здесь также будет представлена классификация семейств генеративных моделей, которые вы начнете изучать в части II.

Глава 2 содержит руководство по инструментам и методам глубокого обучения, овладеть которыми необходимо, чтобы начать создавать более сложные генеративные модели. В частности, здесь вы построите свою первую глубокую нейронную сеть — многослойный перцептрон (multilayer perceptron, MLP) — с помощью Keras. Затем мы добавим в нее сверточные слои и другие улучшения, чтобы увидеть разницу в производительности.

К концу части I вы получите хорошее представление об основных идеях, лежащих в основе всех техник, описанных в последующих частях книги.

Генеративное моделирование

В этой главе:

- ключевые различия между генеративными и дискриминативными моделями;
- основные свойства генеративных моделей на простом примере;
- основные понятия теории вероятностей, лежащие в основе генеративных моделей;
- различные семейства генеративных моделей;
- приемы клонирования кодовой базы, прилагаемой к книге (вы сможете приступить к созданию генеративных моделей!).

Эта глава является общим введением в генеративное моделирование. Сначала мы посмотрим, что подразумевается под названием *«генеративная модель»* и чем генеративное моделирование отличается от более широко известного *дискриминативного моделирования*. После этого определим базовые принципы, описывающие желаемые свойства, которыми должна обладать хорошая генеративная модель, а также ознакомимся с основными положениями теории вероятностей, которые важно знать, чтобы в полной мере оценить, как различные подходы решают задачу генеративного моделирования.

Это естественным образом приведет нас к предпоследнему разделу, где перечислены шесть обширных семейств генеративных моделей, занимающих доминирующее положение в этой области в настоящее время. В последнем разделе я объясню, как начать работу с кодовой базой, прилагаемой к книге.

Что такое генеративное моделирование

В общих чертах генеративную модель можно определить так:

Генеративная модель описывает, как генерируется набор данных, с точки зрения вероятностной модели. Используя эту модель, можно генерировать новые данные.

Что это означает на практике? Допустим, у нас есть коллекция изображений лошадей. С ее помощью можно *обучить* генеративную модель, которая усвоит основные правила, управляющие сложными отношениями между пикселями на изображениях. Затем мы сможем *использовать* эту модель для создания новых реалистичных изображений лошадей, отсутствующих в исходной коллекции. Этот процесс показан на рис. 1.1.

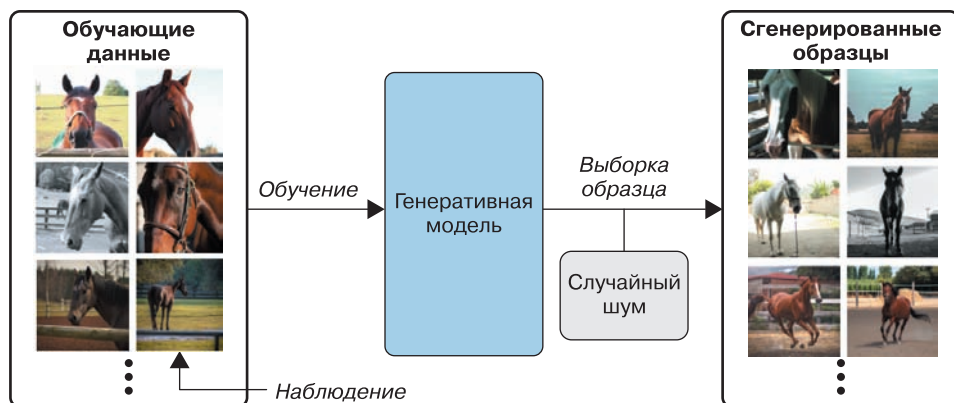


Рис. 1.1. Генеративная модель обучается генерировать реалистичные изображения лошадей

Чтобы построить генеративную модель, необходим набор данных, состоящий из множества образцов сущности, которую нужно сгенерировать. Этот набор данных называется *обучающим набором*, а один образец данных в нем — *наблюдением*.

Каждое наблюдение состоит из множества *признаков* — в задачах генерации изображений их роль обычно играют отдельные пиксели, в задачах генерации текста признаками могут служить отдельные слова или группы букв. Наша цель — создать модель, способную генерировать новые наборы признаков, которые выглядят так, будто созданы с использованием тех же правил, что и исходные данные. Концептуально генерация изображений — невероятно сложная задача, учитывая огромное количество способов выбора значений для отдельных пикселей и относительно крошечное число вариантов такого их расположения, когда получается изображение, похожее на моделируемый объект.

Генеративная модель должна быть *вероятностной*, а не *детерминированной*, поскольку нам нужна возможность получать на выходе различные изображения, а не один и тот же результат. Если модель просто представляет фиксированные вычисления, например выбирает среднее значение каждого пикселя в наборе данных, то она не будет генеративной. Генеративная модель

должна включать случайный элемент, который влияет на отдельные образцы, генерируемые моделью.

Другими словами, мы можем представить, что существует какое-то неизвестное вероятностное распределение, объясняющее, почему одни изображения могли бы присутствовать в обучающем наборе, а другие — нет. Наша задача — создать модель, максимально точно имитирующую это распределение, а затем произвести выборку из нее, чтобы сгенерировать новые наблюдения, которые выглядят так, будто могли бы иметься в исходном, обучающем наборе.

Генеративное и дискриминативное моделирование

Чтобы по-настоящему понять цель и важность генеративного моделирования, полезно сравнить его с аналогом — *дискриминативным моделированием*. Знакомые с машинным обучением знают, что большинство задач, с которыми вы столкнетесь, скорее всего, носят дискриминативный характер. Рассмотрим пример, чтобы понять разницу.

Предположим, у нас есть набор данных с коллекцией картин, часть которых написаны Ван Гогом, а часть — другими художниками. Имея достаточный объем данных, мы сможем обучить дискриминативную модель, способную предсказать, была ли данная картина написана Ван Гогом. Наша модель может выучить, какие цвета, формы и текстуры с большей вероятностью будут указывать на принадлежность картины кисти голландского мастера, и в соответствии с этими характеристиками оценивать свой прогноз. На рис. 1.2 показан процесс дискриминативного моделирования — обратите внимание на то, как он отличается от процесса генеративного моделирования, изображенного на рис. 1.1.

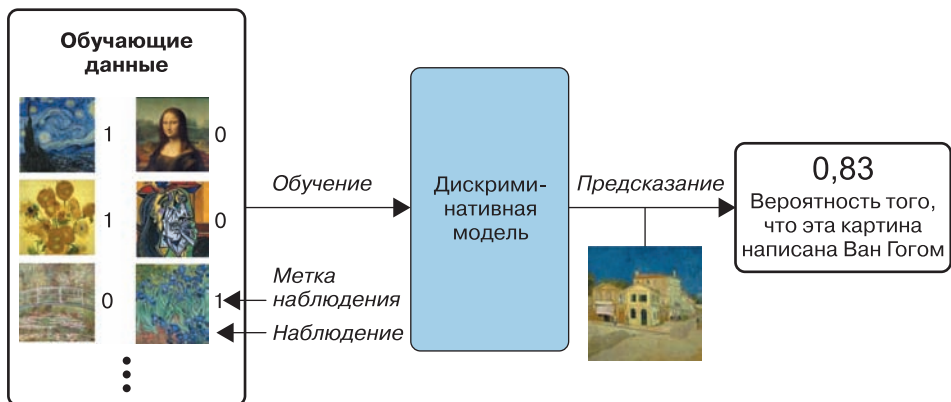


Рис. 1.2. Дискриминативная модель обучается предсказывать, принадлежит ли данная картина кисти Ван Гога

При выполнении дискриминативного моделирования каждое наблюдение в обучающих данных имеет *метку*. Для задачи бинарной классификации, такой как определение принадлежности холста, картины Ван Гога будут помечены меткой 1, а картины других художников — меткой 0. Исследовав этот набор, наша модель научится различать эти две группы и выведет вероятность того, что новое наблюдение имеет метку 1, то есть что эта картина написана Ван Гогом.

В генеративном моделировании, напротив, не требуется, чтобы наборы данных содержали метки, поскольку его задача — создавать совершенно новые изображения, а не пытаться предсказать метку существующего.

Определим эти виды моделирования более формально, используя математические обозначения.

Дискриминативная модель оценивает $p(y | \mathbf{x})$.

То есть цель дискриминативной модели — оценить вероятность метки y для данного наблюдения $\mathbf{x}$.

Генеративная модель оценивает $p(\mathbf{x})$.

То есть цель генеративной модели — смоделировать вероятностное распределение наблюдений $\mathbf{x}$. Выборка из этого распределения позволяет генерировать новые наблюдения.



Условные генеративные модели

Обратите внимание: точно так же можно построить генеративную модель для моделирования условной вероятности $p(\mathbf{x} | y)$ увидеть наблюдение с определенной меткой y .

Например, если предположить, что набор данных содержит разные типы фруктов, то мы могли бы указать своей генеративной модели, чтобы она генерировала изображения яблок.

Учтите, что, даже если бы мы смогли построить идеальную дискриминативную модель для идентификации картин Ван Гога, она все равно не знала бы, как создать картину, похожую на картину Ван Гога. Она сможет выводить только вероятности для существующих изображений, потому что именно этому была обучена. Наша задача совсем другая — обучить генеративную модель, которая сможет генерировать наборы пикселей, с высокой вероятностью принадлежащие исходному набору обучающих данных.

Появление генеративного моделирования

До недавнего времени основные успехи в машинном обучении были обусловлены достижениями в дискриминативном моделировании. Это связано с тем, что решить задачу генеративного моделирования гораздо сложнее, чем любую дискриминативную. Например, гораздо проще научить модель предсказывать, написана ли картина Ван Гогом, чем научить ее создавать совершенно новые картины в стиле Ван Гога.

Точно так же гораздо проще обучить модель прогнозировать, была ли страница текста написана Чарльзом Диккенсом, чем построить модель, генерирующую последовательность абзацев в стиле Диккенса. До недавнего времени большинство генеративных задач были просто недостижимы, и многие сомневались, что их вообще удастся решить. Творчество считалось чисто человеческой способностью, с которой не мог конкурировать ИИ.

Однако по мере развития технологий машинного обучения этот скептицизм постепенно ослабевает. За последние десять лет многие из наиболее интересных результатов в этой области появились благодаря применению новейших достижений в машинном обучении для решения задач генеративного моделирования. Например, на рис. 1.3 показан поразительный прогресс в области создания фотореалистичных изображений лица, достигнутый с 2014 года.

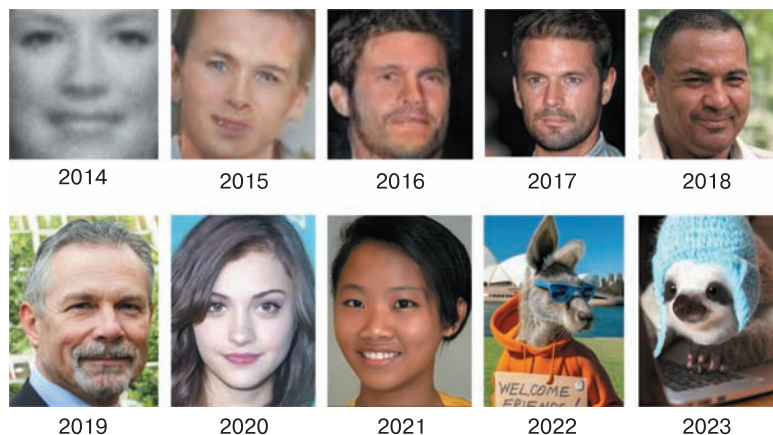


Рис. 1.3. Качество изображений лиц, сгенерированных генеративными моделями, значительно улучшилось за последние десять лет (источник: [Brundage et al., 2018], https://www.eff.org/files/2018/02/20/malicious_ai_report_final.pdf)

Дискриминативное моделирование не только проще, чем генеративное, но и исторически более применимо для решения практических задач в различных

отраслях. Например, врачу полезнее знать вероятность наличия признаков глаукомы на снимке сетчатки глаза, чем возможность генерировать новые изображения, похожие на снимки задней стенки глазного яблока.

Однако ситуация начала меняться с увеличением числа компаний, предлагающих услуги генеративных моделей, специально предназначенных для решения конкретных бизнес-задач. Например, теперь можно получить доступ к API, которые генерируют оригинальные сообщения в блогах по определенной теме, создают изображения вашего продукта в любых окружениях или пишут текст для социальных сетей и рекламные объявления, соответствующие вашему бренду. Есть также явные положительные примеры применения генеративного ИИ в таких отраслях, как разработка компьютерных игр и кинематография, и здесь же наверняка найдут практическое применение усовершенствованные модели автоматического создания видео и музыки.

Генеративное моделирование и ИИ

Помимо практического применения (многие варианты которого пока не найдены), существует еще три причины, по которым генеративное моделирование можно считать ключом к гораздо более сложным формам искусственного интеллекта, недоступным для дискриминативного моделирования.

Во-первых, с теоретической точки зрения мы не должны довольствоваться успехами в области классификации данных, но должны стремиться к более полному пониманию того, как эти данные были созданы. Это, несомненно, более трудная задача из-за высокой размерности пространства возможных выходных данных и относительно небольшого числа творений, которые мы классифицировали бы как принадлежащие набору данных. Однако, как станет ясно в дальнейшем, многие из методов, которые привели к развитию дискриминативного моделирования, такие как глубокое обучение, могут использоваться и в генеративных моделях.

Во-вторых, как мы увидим в главе 12, генеративное моделирование сейчас существенно влияет на прогресс в других областях ИИ, таких как обучение с подкреплением, когда обучаемый агент получает возможность оптимизировать цели методом проб и ошибок. Например, обучение с подкреплением можно использовать, чтобы обучить робота ходить по какой-то местности. Традиционный подход заключается в проведении множества экспериментов, в ходе которых агент пробует разные стратегии. Со временем он выявит наиболее успешные из них и, следовательно, будет постепенно совершенствоваться. Типичная проблема этого подхода заключается в его малой гибкости, потому что он предназначен для оптимизации решения одной конкретной задачи. Альтернативный подход, который недавно получил распространение, состоит в том, чтобы обучить аген-

та изучать свое окружение с помощью генеративной модели, независимой от конкретной задачи. В результате агент сможет быстро адаптироваться к новым задачам, тестируя стратегии в собственной модели мира, а не в реальной среде, что зачастую более эффективно в вычислительном отношении и не требует переобучения с нуля для каждой новой задачи.

И в-третьих, если мы действительно зададимся целью создать машину, обладающую интеллектом, сравнимым с человеческим, то нам определенно потребуются использовать приемы генеративного моделирования. Один из ярких примеров генеративной модели в природе — человек, читающий эту книгу. Отвлекитесь на минутку и проведите мысленный эксперимент, который поможет вам понять, насколько невероятной генеративной моделью вы являетесь. Вы можете закрыть глаза и представить, как будет выглядеть слон под любыми возможными углами зрения. Вы можете вообразить ряд возможных концовок вашего любимого телешоу и спланировать свою неделю, прорабатывая разные варианты будущего в воображении. Современная нейробиологическая теория предполагает, что наше восприятие реальности — это не сложная дискриминативная модель, получающая информацию от органов чувств и делающая предсказания на ее основе, а генеративная модель, которая с рождения обучается моделированию нашего окружения, точно соответствующего будущему. Некоторые теории даже предполагают, что результатом этой генеративной модели является непосредственное восприятие нами реальности. Очевидно, что глубокое понимание того, как создавать машины, обладающие этими способностями, будет иметь ключевое значение для дальнейшего понимания работы мозга в частности и искусственного интеллекта в целом.

Наша первая генеративная модель

А теперь, кратко ознакомившись с историей и перспективами, начнем путешествие в захватывающий мир генеративного моделирования. Для начала рассмотрим простейший пример генеративной модели и некоторые идеи, которые помогут перейти к более сложным архитектурам, с которыми мы столкнемся позже в книге.

Привет, мир!

Для начала поиграем в генеративное моделирование в двух измерениях. Я выбрал правило, согласно которому был сгенерирован набор точек $\mathbf{X}$, изображенных на рис. 1.4. Назовем это правило p_{data} . Ваша задача состоит в том, чтобы выбрать другую точку $\mathbf{x} = (x_1, x_2)$ в пространстве, которая выглядит так, будто она сгенерирована тем же правилом.

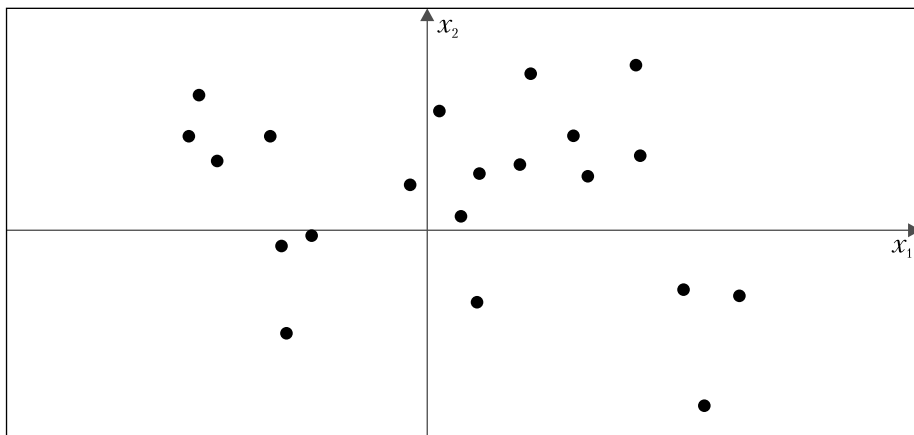


Рис. 1.4. Набор точек на двумерной плоскости, сгенерированный с использованием неизвестного правила p_{data}

Выбрали? Вы, вероятно, применили свои знания о существующих точках, чтобы построить ментальную модель p_{model} и определить, где в пространстве должна находиться эта точка. В этом отношении p_{model} является *оценкой* p_{data} . Возможно, вы решили, что p_{model} должна выглядеть так, как показано на рис. 1.5: прямоугольное поле, в котором могут находиться точки, и область за пределами прямоугольника, где их быть не должно.

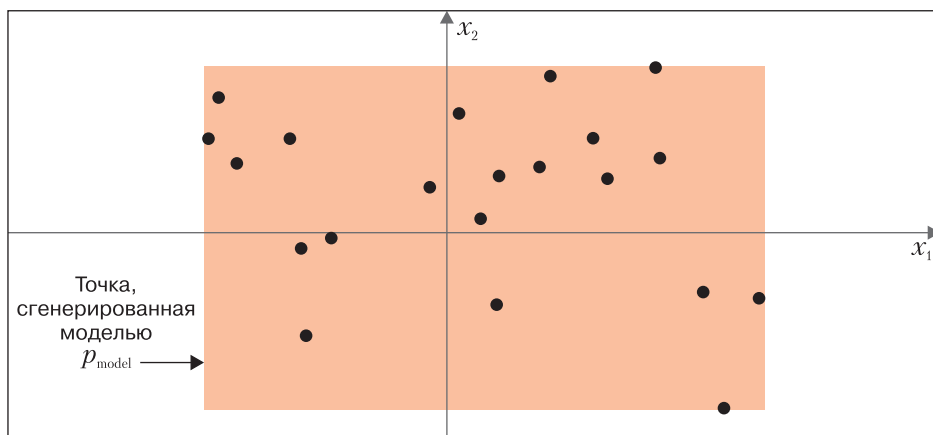


Рис. 1.5. Оранжевый прямоугольник p_{model} — это оценка фактического распределения p_{data}

Чтобы сгенерировать новое наблюдение, можно просто выбрать случайную точку внутри поля, или, выражаясь более формальным языком, *выбрать образец* из распределения p_{model} . Поздравляю, вы только что разработали свою первую генератив-

ную модель! Вы использовали обучающие данные (черные точки) для построения модели (оранжевая область), из которой можно выбрать образцы для создания других точек, которые выглядят принадлежащими к обучающему набору.

Теперь формализуем эти рассуждения, придав им вид базовых принципов, которые помогут понять, чего пытаются достичь генеративное моделирование.

Базовые принципы генеративного моделирования

Мы можем выразить цели и мотивы построения генеративной модели в виде следующих принципов.

БАЗОВЫЕ ПРИНЦИПЫ ГЕНЕРАТИВНОГО МОДЕЛИРОВАНИЯ

- У нас есть набор данных с наблюдениями $\mathbf{X}$.
- Предполагается, что наблюдения сгенерированы в соответствии с некоторым неизвестным распределением p_{data} .
- Генеративная модель p_{model} пытается имитировать p_{data} . Правильно подобрав p_{model} , мы сможем с ее помощью генерировать наблюдения, которые выглядят так, будто были получены из p_{data} .
- Соответственно, желаемыми свойствами p_{model} являются следующие.

Точность

Сгенерированные образцы с высокой оценкой p_{model} должны выглядеть так, будто получены из p_{data} . Сгенерированные образцы с низкой оценкой p_{model} не должны выглядеть так, будто получены из p_{data} .

Способность генерировать

Должна существовать возможность получать новые образцы из p_{model} .

Представительность

Должна существовать возможность понять, как различные высокоуровневые признаки данных представлены в p_{model} .

Теперь раскроем истинное распределение данных p_{data} и посмотрим, как базовые принципы применяются к этому примеру. Как показано на рис. 1.6, правило, на основе которого получены данные, — это просто равномерное распределение точек по земной суше.

Очевидно, что p_{model} — это упрощенное представление p_{data} . Точки А, В и С соответствуют трем наблюдениям, сгенерированным моделью p_{model} с разной степенью успеха.

- Точка А — это образец, сгенерированный нашей моделью, но совершенно очевидно, что она не принадлежит распределению p_{data} , поскольку находится посреди моря.

- Точка В никогда не могла быть сгенерирована моделью p_{model} , поскольку находится за пределами оранжевого прямоугольника. Отсюда следует, что наша модель имеет некоторые пробелы в своей способности генерировать наблюдения по всему спектру потенциальных возможностей.
- Точка С — это наблюдение, которое вполне могло быть получено из обоих распределений, p_{model} и p_{data} .

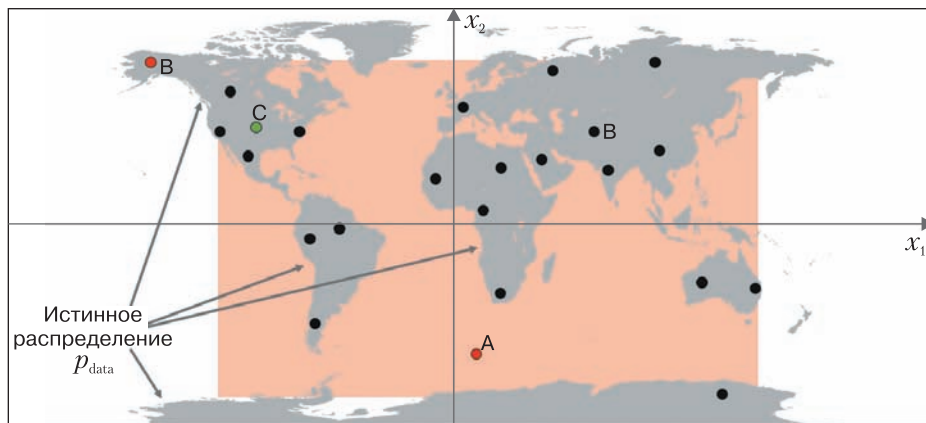


Рис. 1.6. Оранжевые прямоугольники p_{model} — это оценка истинного распределения p_{data} (серые области)

Несмотря на недостатки, эта модель позволяет с легкостью выбирать новые образцы, поскольку представляет равномерное распределение по оранжевой прямоугольной области. Мы можем просто выбрать случайную точку внутри этого прямоугольника, чтобы получить новый образец.

Кроме того, мы с уверенностью можем сказать, что наша модель — это простое представление базового сложного распределения, отражающая некоторые из основных свойств высокого уровня. Истинное распределение разделено на большие области суши (континенты) и воды (моря и океаны). Это свойство справедливо и для нашей модели, за исключением того, что в ней предполагается один большой континент, а не множество.

Этот пример продемонстрировал фундаментальные принципы генеративного моделирования. Задачи, решаемые в этой книге, будут гораздо более сложными и многомерными, но основные подходы к их решению останутся неизменными.

Обучение представлению

Давайте углубимся в вопросы обучения *представлению* многомерных данных, поскольку эта тема будет повторяться на протяжении всей книги.

Предположим, вы решили описать свою внешность незнакомому человеку, который будет искать вас в толпе людей. Вы едва ли начнете с описания цвета пиксела 1 на вашей фотографии, затем пиксела 2, пиксела 3 и т. д. Вместо этого вы наверняка разумно предположите, что ищущий вас имеет общее представление о том, как выглядит средний человек, и просто дополните общую картину признаками, описывающими группы пикселов, например: «у меня светлые волосы» или «я ношу очки». Получив не более десяти таких признаков, человек сможет преобразовать описание обратно в пикселы, чтобы создать в уме ваш образ. Изображение не будет идеальным, но оно будет достаточно близко к вашей реальной внешности, чтобы вас можно было найти среди сотен других людей, даже если вас никогда раньше не видели.

Основная идея обучения представлению — в том, чтобы вместо моделирования многомерного выборочного пространства непосредственно попытаться описать каждое наблюдение из обучающего набора с использованием некоторого *скрытого пространства* меньшей размерности, а затем определить функцию отображения, которая может взять точку из скрытого пространства и отобразить ее в точку в исходном пространстве. Другими словами, каждая точка в скрытом пространстве является *представлением* некоторого многомерного изображения.

Что это означает с практической точки зрения? Предположим, у нас есть обучающий набор, состоящий из черно-белых изображений форм тортов (рис. 1.7).

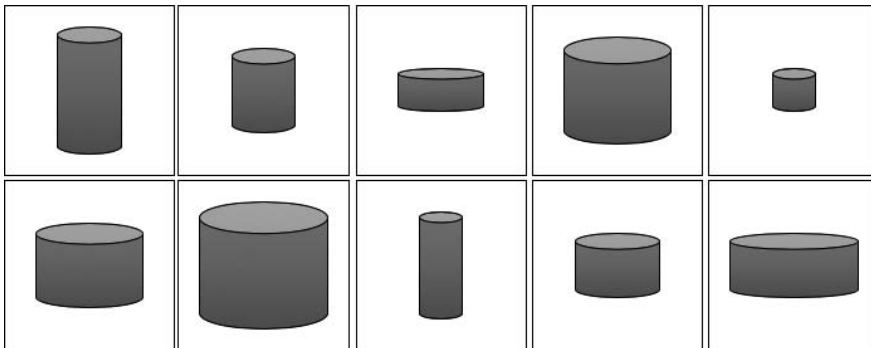


Рис. 1.7. Набор изображений форм тортов

Совершенно очевидно, что уникально идентифицировать формы тортов можно по двум признакам: по высоте и ширине. То есть каждое изображение формы торта можно преобразовать в точку в скрытом пространстве всего двух измерений, даже притом что обучающий набор изображений предоставлен в многомерном пространстве пикселей. Это также означает, что мы можем создавать изображения форм тортов, которых нет в обучающем наборе, применяя подходящую функцию отображения f к новой точке в скрытом пространстве (рис. 1.8).

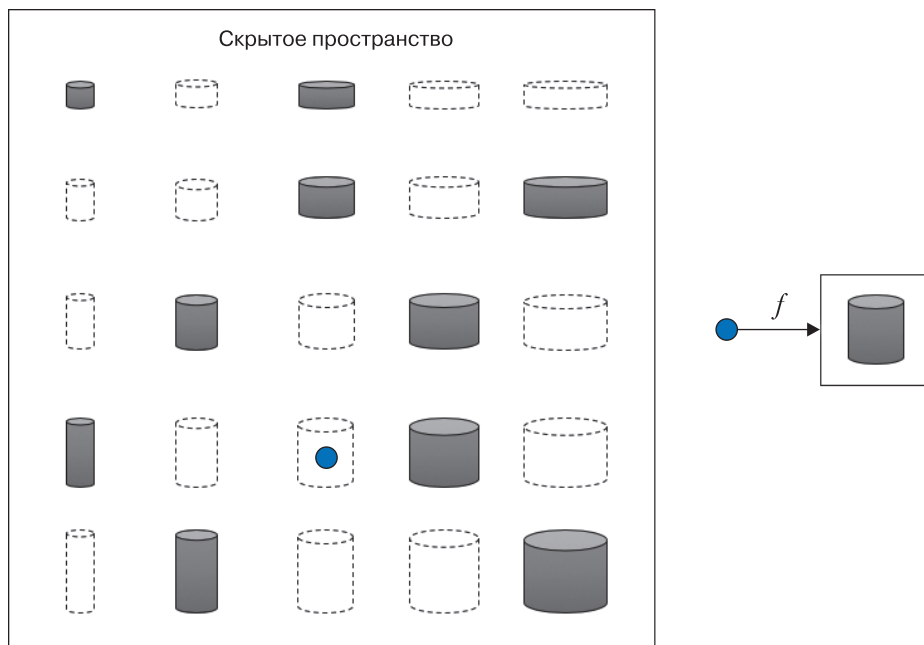


Рис. 1.8. Скрытое пространство свойств форм тортов и функция f , отображающая точку из скрытого пространства в пространство исходных изображений

Однако понять, что исходный набор данных можно описать с помощью более простого скрытого пространства, — непростая задача для компьютера. Сначала он должен выяснить, что высота и ширина — это два измерения скрытого пространства, которые наилучшим образом описывают этот набор данных, а затем сконструировать функцию отображения f , которая сможет выбрать точку в этом пространстве и привести ее в соответствие с черно-белым изображением формы. Машинное обучение (и глубокое обучение в частности) дает нам возможность обучать машины находить такие сложные функции взаимосвязи без помощи человека.

Одно из преимуществ обучения представлению заключается в возможности выполнять в лучше управляемом скрытом пространстве операции, влияющие на высокоуровневые свойства изображений. В исходном пространстве изображений неясно, как сгенерировать каждый пиксел, чтобы получить *более высокое* изображение торта. Однако в скрытом пространстве для этого достаточно увеличить на 1 скрытое измерение, отвечающее за *высоту*, а затем применить функцию отображения для перехода обратно в пространство изображений. Пример такого подхода мы рассмотрим в следующей главе, но применительно не к формам тортов, а к граням.

Как будет показано в последующих главах, идея кодирования набора обучающих данных в скрытое пространство, чтобы выбрать точку из него и декодировать ее обратно в исходное пространство, является общей для многих методов генеративного моделирования. С математической точки зрения методы *кодирования* — *декодирования* пытаются преобразовать высоконелинейное *многообразие*, на котором лежат данные (например, в пространстве пикселей), в более простое скрытое пространство, каждую точку которого с большой вероятностью можно представить правильно сформированным изображением (рис. 1.9).

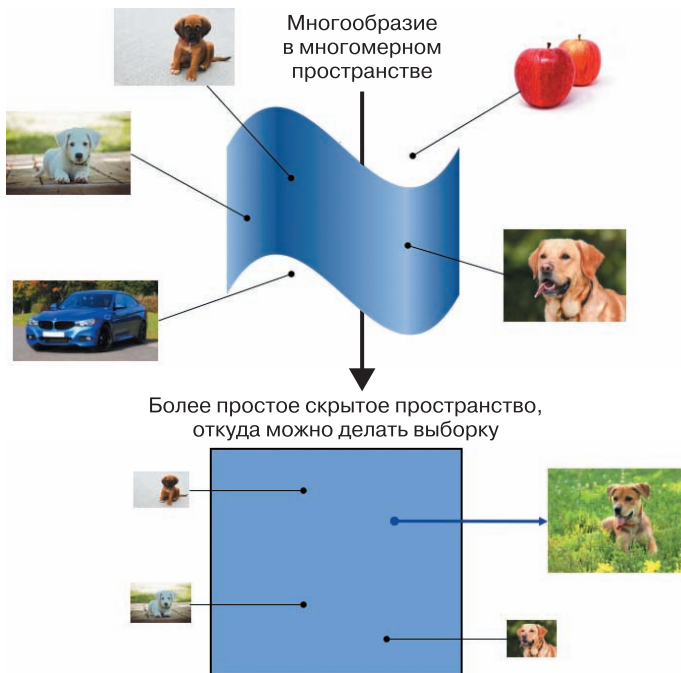


Рис. 1.9. Многообразие собак в многомерном пространстве изображений отображается в более простое скрытое пространство, откуда можно делать выборку

Основы теории вероятностей

Мы уже видели, что генеративное моделирование тесно связано со статистическим моделированием распределения вероятностей. Поэтому теперь самое время представить базовые понятия математической статистики теории вероятностей, которые будут использоваться на протяжении всей книги для объяснения теоретической основы каждой модели.

Сразу хочу успокоить тех, кто никогда не изучал теорию вероятностей. Чтобы построить и опробовать большинство моделей глубокого обучения, которые мы увидим далее в этой книге, не обязательно иметь детальное понимание математической статистики. Но, чтобы получить полное представление о задаче, которую мы пытаемся решить, стоит постараться хорошо разобраться в основах теории вероятностей. Это поможет заложить основы понимания различных семейств генеративных моделей, которые будут представлены далее в этой главе.

Первым делом определим пять ключевых терминов, связав каждый из них с примером генеративной модели, представляющей карту мира в двух измерениях.

Выборочное пространство

Это полное множество значений, которые может принимать наблюдение $\mathbf{x}$.



В предыдущем примере выборочное пространство включает все точки на карте мира, характеризующиеся широтой и долготой $\mathbf{x} = (x_1, x_2)$. Например, $\mathbf{x} = (40,7306, -73,9352)$ — это точка в выборочном пространстве (New York City), принадлежащая истинному распределению, генерирующему данные. $\mathbf{x} = (11,3493, 142,1996)$ — это точка в выборочном пространстве, не принадлежащая истинному распределению, генерирующему данные (она находится в море).

Функция плотности вероятности

(или просто *функция плотности*) $p(\mathbf{x})$ отображает точку $\mathbf{x}$ из выборочного пространства в число от 0 до 1. Интеграл функции плотности по всем точкам в выборочном пространстве должен быть равен 1, то есть это четко определенное распределение вероятностей.



В примере с картой мира функция плотности нашей генеративной модели равна 0 за пределами оранжевого прямоугольника и постоянна внутри него, соответственно, интеграл функции плотности по всему выборочному пространству равен 1.

Существует только одна истинная функция плотности $p_{\text{data}}(\mathbf{x})$, которая, как предполагается, сгенерировала наблюдаемый набор данных, и бесконечное число функций плотности $p_{\text{model}}(\mathbf{x})$, которые можно использовать для оценки $p_{\text{data}}(\mathbf{x})$.

Параметрическое моделирование

Это метод, который можно применять для структурирования подхода к поиску подходящей функции плотности $p_{\text{model}}(\mathbf{x})$. *Параметрическая модель* — это семейство функций плотности $p_{\theta}(\mathbf{x})$, которое можно описать с использованием конечного числа параметров θ .



Если предположить равномерное распределение нашего семейства моделей, то множество всех возможных прямоугольников, которые можно было бы изобразить на рис. 1.5, является примером параметрической модели. В данном случае мы имеем четыре параметра: координаты левого нижнего (θ_1, θ_2) и правого верхнего (θ_3, θ_4) углов прямоугольника. Таким образом, каждая функция плотности $p_{\theta}(\mathbf{x})$ в этой параметрической модели (то есть каждый прямоугольник) может быть уникально представлена четырьмя числами: $\theta = (\theta_1, \theta_2, \theta_3, \theta_4)$.

Правдоподобие

Правдоподобие $\mathcal{L}(\theta | \mathbf{x})$ набора параметров θ является функцией, которая измеряет правдоподобие θ с учетом некоторой наблюдаемой точки $\mathbf{x}$. Определяется она как:

$$\mathcal{L}(\theta | \mathbf{x}) = p_{\theta}(\mathbf{x}),$$

то есть правдоподобие θ некоторой наблюдаемой точки $\mathbf{x}$ определяется как значение функции плотности, параметризованной θ , в точке $\mathbf{x}$. При наличии полного набора данных $\mathbf{X}$ независимых наблюдений мы могли бы написать:

$$\mathcal{L}(\theta | \mathbf{X}) = \prod_{\mathbf{x} \in \mathbf{X}} p_{\theta}(\mathbf{x}).$$



В примере с картой мира оранжевый прямоугольник, покрывающий только левую половину карты, имел бы правдоподобие 0 — он не сможет сгенерировать набор данных, потому что мы наблюдаем точки и в правой половине карты. Оранжевый прямоугольник на рис. 1.5 имеет положительное правдоподобие, так как функция плотности положительна для всех точек данных в этой модели.

Поскольку с вычислительной точки зрения найти это произведение довольно сложно, вместо него часто используется *логарифм правдоподобия* ℓ :

$$\ell(\theta | \mathbf{X}) = \sum_{\mathbf{x} \in \mathbf{X}} \log p_{\theta}(\mathbf{x}).$$

Есть вполне конкретные статистические причины, объясняющие, почему правдоподобие определяется именно так, но нам достаточно будет интуитивного понимания. Мы просто определяем правдоподобие того, что набор параметров θ равен вероятности увидеть данные в рамках модели, параметризованной θ .



Обратите внимание, что правдоподобие является функцией параметров, а не данных. Его не следует интерпретировать как вероятность правильности данного набора параметров. Другими словами, это не распределение вероятностей по пространству параметров, то есть вероятности не суммируются/интегрируются до 1 по отношению к параметрам.

Поэтому интуитивно понятно, что цель параметрического моделирования — поиск оптимального значения набора параметров $\hat{\theta}$, которое максимизирует вероятность наблюдения набора данных $\mathbf{X}$.

Оценка максимального правдоподобия

Это метод, позволяющий оценить $\hat{\theta}$ — набор параметров θ функции плотности $p_{\theta}(\mathbf{x})$, которые наиболее вероятно объясняют некоторые наблюдаемые данные $\mathbf{X}$. Более формально:

$$\hat{\theta} = \arg \max_{\theta} \ell(\theta | \mathbf{X}).$$

$\hat{\theta}$ также называют *оценкой максимального правдоподобия* (maximum likelihood estimate, MLE).



В примере с картой мира оценка MLE — это наименьший прямоугольник, который все еще содержит все точки из обучающего набора.

Нейронные сети обычно *минимизируют* функцию потерь, поэтому можно также говорить о поиске набора параметров, которые *минимизируют отрицательную логарифмическую вероятность*:

$$\hat{\theta} = \arg \min_{\theta} (-\ell(\theta | \mathbf{X})) = \arg \min_{\theta} (-\log p_{\theta}(\mathbf{X})).$$

Генеративное моделирование можно рассматривать как форму оценки максимального правдоподобия, где параметры θ — это веса нейронных сетей, содержащихся в модели. Мы пытаемся найти значения этих параметров, которые максимизируют вероятность наблюдения заданных данных, или, что то же самое, минимизируют отрицательное логарифмическое правдоподобие.

Однако для задач большой размерности обычно невозможно напрямую вычислить $p_{\theta}(\mathbf{x})$ — эти задачи *трудноразрешимы*. Как мы увидим в следующем разделе, разные семейства генеративных моделей используют разные подходы к решению этой задачи.

Классификация генеративных моделей

Все типы генеративных моделей так или иначе направлены на решение одной и той же задачи, но используют разные подходы к моделированию функции плотности $p\theta(\mathbf{x})$. В целом возможны три подхода.

1. Явное моделирование функции плотности, но с некоторыми ограничениями, чтобы эта функция была управляемой, то есть ее можно было вычислить.
2. Явное моделирование удобного приближения функции плотности.
3. Неявное моделирование функции плотности с помощью стохастического (случайного) процесса, который напрямую генерирует данные.

Они показаны на рис. 1.10 в виде классификации с шестью семействами генеративных моделей, которые мы рассмотрим в части II. Обратите внимание на то, что эти семейства не являются взаимоисключающими — существует множество примеров моделей, объединяющих два подхода. Семейства лучше рассматривать как различные общие подходы к генеративному моделированию, а не как явные архитектуры моделей.

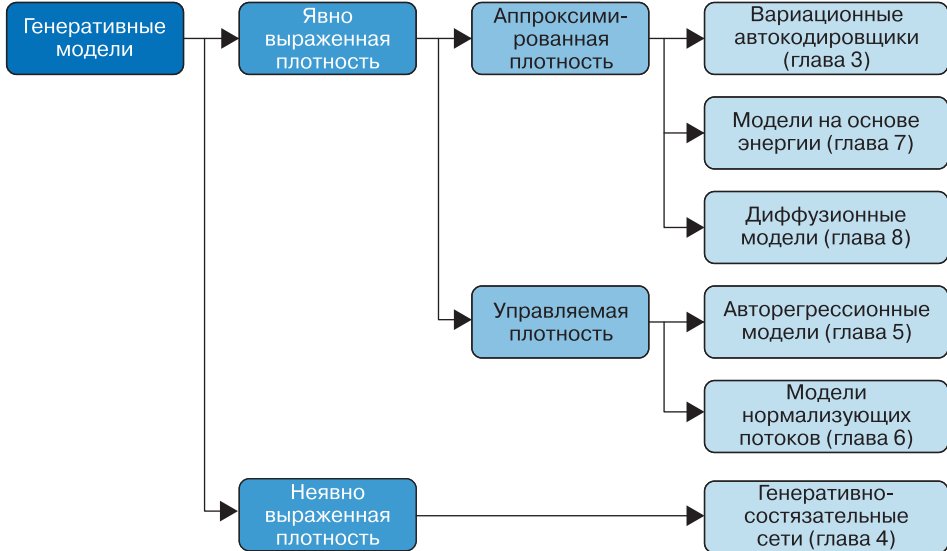


Рис. 1.10. Классификация подходов к созданию генеративных моделей

Первое возможное разделение — разделение моделей, в которых функция плотности вероятности $p(\mathbf{x})$ моделируется *явно* и в которых она моделируется *неявно*.

Модели с неявно выраженной плотностью не нацелены на оценку плотности вероятности, а сосредоточены исключительно на создании стохастического процесса, который непосредственно генерирует данные. Самый известный пример генеративной модели с неявно выраженной плотностью — *генеративно-состязательная сеть* (Generative Adversarial Network, GAN). Далее *модели с явно выраженной плотностью* можно разделить на оптимизирующие непосредственно функцию плотности (управляемые модели) и оптимизирующие только ее аппроксимацию.

Управляемые модели накладывают ограничения на архитектуру модели, поэтому функция плотности имеет форму, упрощающую вычисления. Например, *авто-регрессионные модели* накладывают ограничение на порядок входных объектов, поэтому выходные данные могут генерироваться последовательно — слово за словом или пиксел за пикселем. *Модели нормализующих потоков* применяют ряд простых обратимых функций к простому распределению для создания более сложных распределений.

К числу *моделей на основе аппроксимации плотности* относятся вариационные автокодировщики, которые вводят скрытую переменную и оптимизируют аппроксимацию функции совместной плотности. В *моделях на основе энергии* тоже используются приближенные методы, но для этого применяется выборка цепей Маркова, а не вариационные подходы. *Диффузионные модели* аппроксимируют функцию плотности, обучая модель постепенному подавлению шума в ранее поврежденном изображении.

Общей чертой, объединяющей все семейства генеративных моделей, является *глубокое обучение*. Почти все сложные генеративные модели имеют в своей основе глубокую нейронную сеть, поскольку они способны с нуля изучить сложные взаимосвязи, управляющие структурой данных. Мы рассмотрим глубокое обучение в главе 2, также там будут показаны практические примеры создания собственных глубоких нейронных сетей.

Код примеров генеративного глубокого обучения

Последний раздел этой главы поможет вам приступить к созданию генеративных моделей глубокого обучения, познакомив с базой кода, сопровождающей книгу.



Многие примеры, приводимые здесь, заимствованы из замечательных реализаций с открытым исходным кодом, доступных на веб-сайте Keras (<https://oreil.ly/1UTwa>). Я настоятельно рекомендую посетить этот ресурс, так как на нем постоянно появляются новые модели и примеры.

Клонирование репозитория

Чтобы получить доступ к примерам, вы должны клонировать репозиторий Git, созданный для книги. *Git* — это система с открытым исходным кодом для управления версиями, которая позволит скопировать код на свой компьютер, чтобы запускать его локально или в облачном окружении. Возможно, у вас эта система уже установлена. Если нет, то следуйте инструкциям для своей операционной системы (<https://oreil.ly/tFOdN>).

Чтобы клонировать репозиторий с примерами, в окне терминала перейдите в папку, куда вы хотели бы сохранить файлы, и введите следующую команду:

```
git clone https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition.git
```

Теперь в этой папке на вашем компьютере будут находиться файлы, скопированные из репозитория.

Использование Docker

Код примеров для книги предназначен для применения с *Docker* — бесплатной технологией контейнеризации, которая чрезвычайно упрощает начало работы с новым кодом независимо от аппаратной архитектуры или операционной системы. Если вы никогда не использовали Docker, не волнуйтесь — в репозитории книги вы найдете файл README, где описывается, как начать работу.

Применение графического процессора

Если у вас нет доступа к собственному графическому процессору (GPU), это тоже не проблема! Все примеры в книге будут обучаться на обычном процессоре, хотя это займет больше времени, чем при использовании машины с графическим процессором. В README также есть раздел, посвященный настройке среды Google Cloud, которая предоставляет доступ к графическому процессору с оплатой по факту использования.

Резюме

В этой главе вы познакомились с генеративным моделированием — важным разделом машинного обучения, дополняющим хорошо изученную область дискриминативного моделирования. Вы узнали, что генеративное моделирование в настоящее время является одной из наиболее активных и захватывающих сфер исследований в области ИИ, которая может похвастаться множеством последних достижений как в теории, так и на практике.

Мы начали с простого примера, и вы увидели, что генеративное моделирование в конечном итоге фокусируется на моделировании распределения исходных данных. Это создает множество сложных и интересных проблем, которые мы обобщили, определив основу для понимания желаемых свойств любой генеративной модели.

Затем мы рассмотрели ключевые понятия теории вероятностей, знание которых необходимо для понимания теоретических основ всех подходов к генеративному моделированию, и выделили шесть семейств генеративных моделей, о которых поговорим в части II этой книги. Также вы увидели, как начать работу с примерами генеративного глубокого обучения, клонировав репозиторий.

Еще вы поняли, как эта модель может потерпеть неудачу с увеличением сложности генеративной задачи, и проанализировали общие проблемы генеративного моделирования. Наконец, мы бросили первый взгляд на обучение представлению — важную идею, лежащую в основе многих генеративных моделей.

В следующей главе приступим к исследованию глубокого обучения и вы увидите, как использовать библиотеку Keras для построения моделей, способных решать задачи дискриминативного моделирования. Это даст вам основу, необходимую для изучения генеративного глубокого обучения в последующих главах.

Глубокое обучение

В этой главе вы:

- познакомитесь с различными типами неструктурированных данных, которые можно смоделировать с помощью глубокого обучения;
- определите глубокую нейронную сеть и увидите, как ее использовать для моделирования сложных наборов данных;
- создадите многослойный перцептрон для прогнозирования содержимого изображения;
- улучшите качество работы модели с помощью сверточных слоев, слоев прореживания и пакетной нормализации.

Начнем с упрощенного определения глубокого обучения:

«Глубокое обучение — это класс алгоритмов машинного обучения, использующих *несколько составных слоев с обрабатывающими узлами* для выявления высокоуровневых представлений в *неструктурированных* данных».

Чтобы уяснить суть глубокого обучения, взглянем на определение немного шире. Прежде всего уточним, что подразумевается под неструктурированными данными, для моделирования которых можно использовать глубокое обучение, а затем углубимся в механику построения многослойной сети для решения задач классификации. Это послужит основой для изучения материала будущих глав, в которых мы сосредоточимся на глубоком обучении для генеративных задач.

Данные для глубокого обучения

Многие алгоритмы машинного обучения требуют передачи им *структурированных* данных в табличной форме, где каждое наблюдение-строка описывается признаками-столбцами. Например, возраст человека, его доход и количество посещений сайта за последний месяц — все это признаки, которые могут помочь предсказать, подпишется ли человек на определенную онлайн-услугу

в следующем месяце. Структурированную таблицу с этими признаками можно было бы использовать для обучения логистической регрессии, случайного леса или модели XGBoost для предсказания ответа на вопрос: подпишется этот человек (1) или нет (0)? Здесь каждый отдельный признак содержит крупную информацию о наблюдении, и, анализируя их, модель сможет узнать, как все эти признаки в совокупности влияют на ответ.

Под *неструктурированными* данными подразумеваются любые данные, не сгруппированные естественным образом в столбцы признаков, например изображения, аудиозаписи и текст. Конечно, изображения имеют пространственную структуру, аудиозаписи — временную, видеозаписи — и пространственную, и временную, но, поскольку данные не распределены по столбцам признаков, они считаются неструктурированными (рис. 2.1).

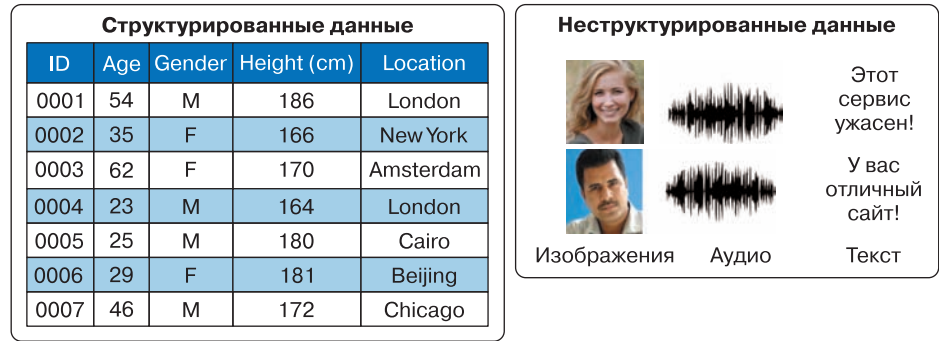


Рис. 2.1. Разница между структурированными и неструктурированными данными

Когда данные не структурированы, отдельные пикселы, частоты или символы не несут почти никакой информации. Например, знание того, что 234-й пиксел в изображении имеет темно-коричневый цвет, никак не помогает определить, что изображено — дом или собака. Точно так же знание того, что 24-й символ в предложении — это буква «е», никак не позволяет судить, о чем говорится в тексте — о футболе или о политике.

Пикселы (символы) — это лишь мазки на холсте, из которых собраны информативные признаки более высокого уровня: изображение дымохода или слово «*нападающий*». Если изобразить дымоход на другой стороне дома, то изображение все равно будет содержать дымоход, но та же информация будет передаваться совершенно другими пикселями. Если слово «*нападающий*» появится в тексте чуть раньше или чуть позже, то текст все равно будет повествовать о футболе, но ту же информацию будут передавать символы в другой позиции. Детальность данных в сочетании с высокой степенью пространственной зависимости не позволяет рассматривать пиксел (символ) как отдельный информационный признак.

Если обучить алгоритм логистической регрессии, случайного леса или XGBoost на исходных значениях пикселей, то обученная модель будет работать плохо почти во всех случаях, кроме самых простых задач классификации. Эти модели полагаются на информативность входных признаков и их пространственную зависимость. Модели глубокого обучения, напротив, способны сами выделять высокоуровневые информативные признаки непосредственно из неструктурированных данных.

Глубокое обучение можно применить и к структурированным данным, но его настоящая мощь, особенно в генеративном моделировании, заключается в способности работать с неструктурированными данными. Нам чаще требуется генерировать неструктурированные данные, такие как новые изображения или строки текста, именно поэтому глубокое обучение оказало такое большое влияние на сферу генеративного моделирования.

Глубокие нейронные сети

Большинство систем глубокого обучения представляют собой *искусственные нейронные сети* (Artificial Neural Networks, ANN), или просто *нейронные сети*, с несколькими скрытыми слоями, наложенными друг на друга. По этой причине термин «*глубокое обучение*» сейчас стал почти синонимом термина «*глубокие нейронные сети*». Отмечу, однако, что любая система, использующая несколько слоев для выявления высокоуровневых представлений во входных данных, тоже является формой глубокого обучения, например сети глубокого доверия (deep belief networks).

Давайте для начала определим, что именно подразумевается под нейронными сетями, а затем посмотрим, как их использовать для выявления высокоуровневых признаков в неструктурированных данных.

Что такое нейронная сеть

Нейронная сеть состоит из последовательности *слоев*, наложенных друг на друга. Каждый слой содержит *узлы* (нейроны), связанные с узлами предыдущего слоя посредством набора *весов* (весовых коэффициентов). Как мы увидим далее, существует много разных типов слоев, но на практике чаще всего используются *полносвязные* (dense) слои, соединяющие каждый свой узел со всеми узлами предыдущего слоя.

Нейронные сети, в которых соседние уровни полностью связаны, называются *многослойными перцептронами* (multilayer perceptron, MLP). Это первый тип нейронной сети, который мы будем изучать. Пример MLP показан на рис. 2.2.

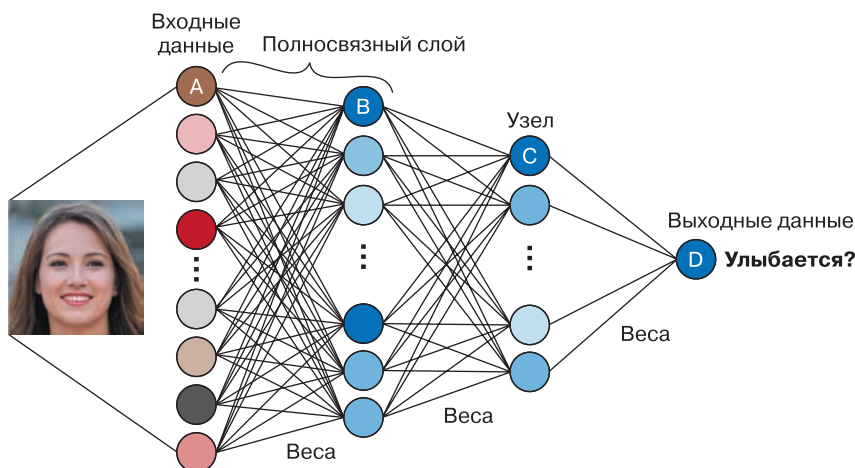


Рис. 2.2. Пример многослойного перцептрона, предсказывающего, какое лицо изображено на фотографии — улыбающееся или нет

Входные данные (например, изображение) преобразуются каждым слоем по очереди в процессе так называемого прямого прохода через сеть, пока не достигнут выходного слоя. В частности, каждый узел применяет нелинейное преобразование к взвешенной сумме своих входных данных и передает результаты (выходные данные) следующему уровню. Кульминацией является последний выходной слой, отдельный узел которого выводит вероятность принадлежности исходного входного сигнала определенной категории (например, улыбка).

Магия глубоких нейронных сетей заключается в поиске набора весов для каждого слоя, которые в совокупности приводят к наиболее точным прогнозам. Процесс поиска этих весов называется *обучением сети*.

В ходе обучения сети передаются пакеты изображений и полученные результаты сравниваются с истинными. Например, сеть может выдать вероятность 80 % для изображения человека, который действительно улыбается, и вероятность 23 % для изображения человека, который на самом деле не улыбается. Идеальный прогноз для этих примеров 100 и 0 %, поэтому ошибка незначительна. Затем ошибка в прогнозировании распространяется обратно по сети и корректирует каждый набор весов на небольшую величину в направлении, улучшающем прогноз. Этот процесс так и называется — *обратное распространение*. Постепенно каждый узел становится квалифицированным в определении конкретного признака, что в конечном итоге помогает сети делать более точные прогнозы.

Выявление высокоуровневых признаков

Важнейшее свойство нейронных сетей, делающее их такими мощными, — способность изучать признаки входных данных без вмешательства человека. Другими словами, нам не нужно заниматься какой-либо разработкой признаков, поэтому нейронные сети так полезны! Мы можем позволить модели решать, как распределить свои веса, руководствуясь только желанием минимизировать ошибку в прогнозах.

Например, пройдемся по сети, показанной на рис. 2.2, предположив, что она уже обучена точно предсказывать, есть ли на фотографии улыбающееся лицо.

1. Узел А получает значение отдельного канала входного пиксела.
2. Узел В объединяет свои входные значения так, чтобы выдать максимальный результат при наличии определенного низкоуровневого признака, такого как граница.
3. Узел С объединяет низкоуровневые признаки так, чтобы он активировался сильнее всего, когда на изображении виден признак более высокого уровня, например *зубы*.
4. Узел D объединяет признаки высокого уровня и активируется сильнее всего, когда человек на исходном изображении улыбается.

Узлы в каждом последующем слое могут представлять все более сложные аспекты исходных данных путем объединения признаков более низкого уровня из предыдущего слоя. Удивительно, но это происходит естественным образом в процессе обучения — нам не нужно *говорить* каждому узлу, что искать и какие признаки — низкоуровневые или высокоуровневые.

Слои между входным и выходным слоями называются *скрытыми*. В нашем примере имеется только два скрытых слоя, но вообще глубокие нейронные сети могут иметь гораздо больше таких слоев. Объединение большого количества слоев позволяет нейронной сети постепенно выявлять признаки все более высокого уровня, накапливая информацию из признаков более низкого уровня в предыдущих слоях. Например, сеть ResNet [He et al., 2015], предназначенная для распознавания изображений, содержит 152 слоя.

А теперь перейдем к практической стороне глубокого обучения и настроим Keras и TensorFlow — две библиотеки, которые позволят вам начать создавать собственные генеративные глубокие нейронные сети.

TensorFlow и Keras

TensorFlow (<https://www.tensorflow.org/>) — это библиотека для Python с открытым исходным кодом, предназначенная для машинного обучения и разработанная в Google (рис. 2.3). Сейчас это одна из библиотек, наиболее широко используемых в решениях машинного обучения, причем с особым упором на манипулирование тензорами (отсюда и такое название). Она включает низкоуровневые функции, необходимые для обучения нейронных сетей, например вычисление градиента произвольных дифференцируемых выражений и эффективное выполнение тензорных операций.

Keras (<https://keras.io/>) — это высокоуровневая библиотека для Python, предназначенная для конструирования нейронных сетей с использованием TensorFlow. Это чрезвычайно гибкая библиотека с очень удобным API, что делает ее идеальным выбором для начального знакомства с глубоким обучением. Кроме того, Keras предлагает множество строительных блоков, объединяя которые можно создавать очень сложные архитектуры глубокого обучения.



Рис. 2.3. TensorFlow и Keras — отличные инструменты для создания решений глубокого обучения

Если вы только начинаете знакомство с глубоким обучением, я настоятельно рекомендую выбрать Keras с низкоуровневой библиотекой TensorFlow. Эти две библиотеки образуют мощную комбинацию, с помощью которой вы построите любую сеть, какую только сможете представить, и предлагают простой в освоении API, что очень важно для быстрого воплощения новых идей и понятий. А теперь посмотрим, как легко построить многослойный перцептрон (MLP) с помощью Keras.

Многослойный перцептрон

В этом разделе мы научим MLP классифицировать заданное изображение с помощью *обучения с учителем*. Так называется тип машинного обучения, в котором компьютер обучается на маркированном наборе данных. Другими словами, набор данных, используемый для обучения, включает входные данные с соответствующими выходными метками. Цель алгоритма — изучить взаимосвязь входных данных и выходных меток, чтобы делать прогнозы для новых, прежде не встречавшихся данных.

MLP — это дискриминативная (а не генеративная) модель, но обучение с учителем по-прежнему будет играть важную роль во многих типах генеративных моделей, которые мы рассмотрим в следующих главах, поэтому данный пример послужит хорошей отправной точкой.



Запуск кода примера

Код для примера можно найти в блокноте Jupyter, доступном по адресу notebooks/02_deeplearning/01_mlp/mlp.ipynb в репозитории книги.

Подготовка данных

В этом примере мы используем набор данных CIFAR-10 (<https://oreil.ly/cNbFG>) — коллекцию из 60 000 цветных изображений 32×32 пиксела. Он входит в состав дистрибутива библиотеки Keras. Каждое изображение принадлежит одному из десяти классов (рис. 2.4).



Рис. 2.4. Примеры изображений из набора данных CIFAR-10 [Krizhevsky, 2009]

Исходные данные представлены массивом пикселей, в котором каждый канал цвета каждого пикселя представлен целым числом от 0 до 255. Сначала нам нужно подготовить изображения, масштабировав эти значения в диапазон от 0 до 1, поскольку нейронные сети работают лучше, когда входные значения меньше 1.

Также нужно изменить целочисленные метки изображений, превратив их в векторы прямого кодирования, потому что на выходе нейронная сеть будет выдавать вероятность принадлежности изображения к каждому классу. Если целочисленная метка, обозначающая класс изображения, имеет значение i , то соответствующий ей вектор прямого кодирования имеет длину 10 (число классов) с нулями во всех элементах, кроме i -го, который равен 1. Эти шаги показаны в примере 2.1.

Пример 2.1. Предварительная обработка набора данных CIFAR-10

```
import numpy as np
from tensorflow.keras import datasets, utils

(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data() ❶

NUM_CLASSES = 10

x_train = x_train.astype('float32') / 255.0 ❷
x_test = x_test.astype('float32') / 255.0

y_train = utils.to_categorical(y_train, NUM_CLASSES) ❸
y_test = utils.to_categorical(y_test, NUM_CLASSES)
```

❶ Загрузка набора данных CIFAR-10. `x_train` и `x_test` — это массивы `numpy` с формой `[50000, 32, 32, 3]` и `[10000, 32, 32, 3]` соответственно. `y_train` и `y_test` — это массивы `numpy` с формой `[50000, 1]` и `[10000, 1]` соответственно, содержащие целочисленные метки от 0 до 9, которые определяют класс каждого изображения.

❷ Исходные данные масштабируются так, чтобы значения всех каналов всех пикселей находились в диапазоне от 0 до 1.

❸ Прямое кодирование меток — `y_train` и `y_test` получают новые формы: `[50000, 10]` и `[10000, 10]` соответственно.

Как видите, обучающие данные (`x_train`) хранятся в *тензоре* с формой `[50000, 32, 32, 3]`. В этом наборе данных нет *столбцов*, ни *строк*, но есть *тензор* с четырьмя измерениями. Тензор — это просто многомерный массив, естественное расширение матрицы до более чем двух измерений. Первое измерение этого массива определяет индекс изображения в наборе данных, второе и третье —

размеры изображения¹, а последнее — индекс канала, то есть красный, зеленый или синий, потому что изображения хранятся в формате RGB.

В примере 2.2 показано, как найти значение канала конкретного пиксела в определенном изображении.

Пример 2.2. Зеленый канал (1) значения пиксела в позиции (12, 13) изображения с номером 54

```
x_train[54, 12, 13, 1]
# 0.36862746
```

Конструирование модели

Keras предлагает два способа определения структуры нейронной сети: в виде модели `Sequential` или с помощью функционального API.

Модель `Sequential` удобно использовать, когда нужно быстро определить линейную последовательность слоев, то есть когда один слой непосредственно следует за предыдущим без какого-либо ветвления. Мы можем определить модель MLP с помощью класса `Sequential`, как показано в примере 2.3.

Пример 2.3. Реализация MLP с использованием модели `Sequential`

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Flatten(input_shape=(32, 32, 3)),
    layers.Dense(200, activation = 'relu'),
    layers.Dense(150, activation = 'relu'),
    layers.Dense(10, activation = 'softmax'),
])
```

Однако многие модели, представленные в книге, требуют, чтобы слои передавали свои выходные данные сразу нескольким слоям, находящимся за ними, или, наоборот, чтобы слой получал входные данные от нескольких слоев перед ним. Класс `Sequential` не подходит для создания таких моделей, в этих случаях можно использовать более гибкий функциональный API.



Я советую с самого начала задействовать функциональный API библиотеки Keras, даже для создания линейных моделей, потому что это умение пригодится вам потом, когда ваши нейронные сети станут сложнее в архитектурном отношении. Функциональный API дает полную свободу в выборе архитектуры глубокой нейронной сети.

¹ Индексы строк и столбцов в пиксельной матрице изображения. — *Примеч. пер.*

В примере 2.4 показан тот же MLP, реализованный с помощью функционального API. При использовании функционального API мы конструируем экземпляры класса `Model`, определяя в них общие входные и выходные слои.

Пример 2.4. Реализация MLP с помощью функционального API

```
from tensorflow.keras import layers, models

input_layer = layers.Input(shape=(32,32, 3))
x = layers.Flatten()(input_layer)
x = layers.Dense(units=200, activation = 'relu')(x)
x = layers.Dense(units=150, activation = 'relu')(x)
output_layer = layers.Dense(units=10, activation = 'softmax')(x)
model = models.Model(input_layer, output_layer)
```

Оба метода дают идентичные модели — схема получившейся архитектуры показана на рис. 2.5.

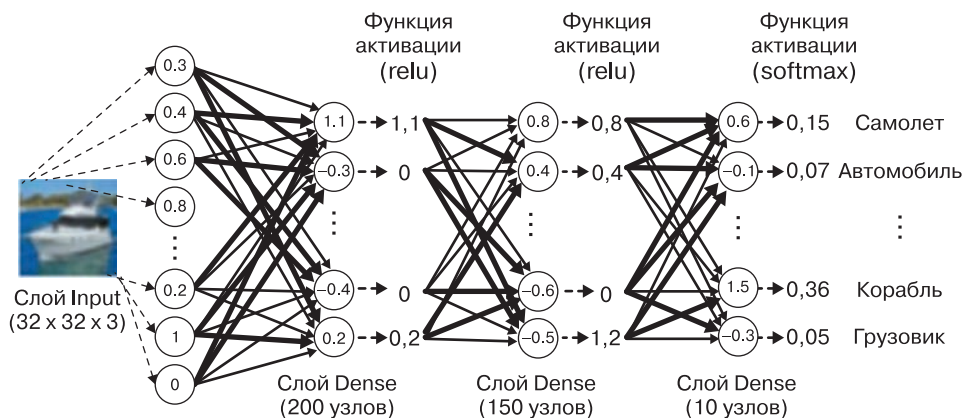


Рис. 2.5. Архитектура MLP

Теперь подробно рассмотрим различные слои и функции активации, используемые в MLP.

Слой

Для построения MLP мы задействовали слои трех типов: `Input`, `Flatten` и `Dense`.

Слой `Input` — это точка входа в сеть. С его помощью мы сообщаем сети, что каждый элемент входных данных имеет форму кортежа. Обратите внимание: мы не указываем размер пакета — в этом нет необходимости, потому что в слой

`Input` можно передать любое количество изображений. Нам не нужно явно указывать размер пакета в определении входного слоя.

Затем мы «сплющиваем» входные данные, преобразуя их в вектор с помощью слоя `Flatten`. В результате получается вектор длиной $3072 = (32 \times 32 \times 3)$. Это делается по той причине, что последующий слой `Dense` требует, чтобы на его вход подавались данные в виде плоского вектора, а не многомерного массива. Как мы увидим далее, другие типы слоев принимают многомерные массивы, поэтому, выбирая слой того или иного типа, вы должны знать форму его входных и выходных данных, чтобы понять, когда следует использовать `Flatten`.

Слой `Dense` является, пожалуй, преобладающим типом слоя в любой нейронной сети. Он содержит определенное количество узлов, полностью связанных с предыдущим слоем, то есть каждый узел этого слоя связан с каждым узлом предыдущего слоя единственной связью, имеющей положительный или отрицательный вес. На выходе каждый узел возвращает взвешенную сумму входных данных, полученных от предыдущего слоя, которая затем передается следующему слою через нелинейную *функцию активации*. Функция активации играет важнейшую роль в способности нейронной сети обучаться — выявлять сложные признаки, а не просто выводить линейную комбинацию входных данных.

Функции активации

Существует много видов функций активации, но наиболее важными из них являются `ReLU`, `sigmoid` и `softmax`.

Функция активации *ReLU* (*rectified linear unit* — блок линейной ректификации) возвращает 0, если входное значение отрицательное, и само входное значение — в остальных случаях. Функция активации *LeakyReLU* очень похожа на `ReLU`, но, в отличие от нее, для отрицательных входных значений возвращает не 0, а небольшое отрицательное число, пропорциональное входному значению. Узлы с функцией активации `ReLU` могут иногда *умирать*, если всегда возвращают 0 из-за большого смещения предварительной активации в сторону отрицательных значений. В этом случае градиент получается равным нулю и через этот узел не происходит обратного распространения ошибки. Функция активации `LeakyReLU` решает проблему, гарантируя, что градиент не будет равен нулю. Сейчас считается, что функции, основанные на `ReLU`, — это наиболее надежные инструменты для передачи информации между слоями глубокой сети и стабильности обучения.

Функция активации *sigmoid* может пригодиться, когда требуется, чтобы выходные значения масштабировались в диапазоне между 0 и 1, как, например, в задачах бинарной классификации с одним выходным узлом или в задачах классификации с несколькими метками, когда каждое наблюдение может одновременно

принадлежать нескольким классам. Для сравнения на рис. 2.6 показаны графики функций активации ReLU, LeakyReLU и sigmoid.

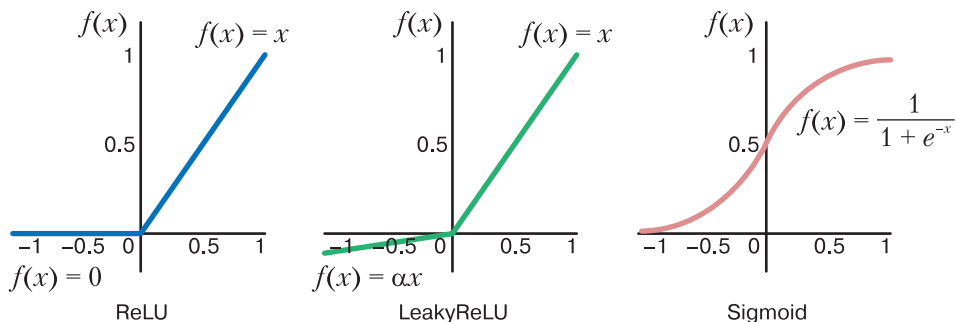


Рис. 2.6. Функции активации ReLU, LeakyReLU и sigmoid

Функция активации *softmax* может пригодиться, когда требуется, чтобы общая сумма выходных значений слоя была равна 1, как, например, в задачах мультиклассовой классификации, когда каждое наблюдение может принадлежать только одному классу. Она определяется как:

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^J e^{x_j}}.$$

Здесь J — общее число узлов в слое. В своей нейронной сети мы используем активацию softmax в последнем слое для получения на выходе набора из десяти вероятностей, сумма которых равна 1. Эти вероятности можно интерпретировать как вероятности принадлежности изображения к каждому классу.

В Keras функции активации можно также определять внутри слоя (пример 2.5) или в отдельном слое (пример 2.6).

Пример 2.5. Функция активации ReLU определена как часть слоя Dense

```
x = layers.Dense(units=200, activation = 'relu')(x)
```

Пример 2.6. Функция активации ReLU определена в отдельном слое

```
x = layers.Dense(units=200)(x)
x = layers.Activation('relu')(x)
```

В своем примере мы пропускаем входные данные через два полносвязных скрытых слоя, первый — с 200 узлами, второй — со 150 и оба — с функциями активации ReLU.

Исследование модели

С помощью метода `model.summary()` можно проверить форму каждого слоя сети (табл. 2.1).

Таблица 2.1. Вывод метода `model.summary()`

Слой (тип)	Выходная форма	Число параметров
Input	(None, 32, 32, 3)	0
Flatten	(None, 3072)	0
Dense	(None, 200)	614 600
Dense	(None, 150)	30 150
Dense	(None, 10)	1510

Всего параметров 646 260

Обучаемых параметров 646 260

Необучаемых параметров 0

Обратите внимание на то, что форма входного слоя `Input` соответствует форме `x_train`, а форма выходного слоя `Dense` — форме `y_train`. Keras использует маркер `None` для первого измерения, чтобы показать, что количество наблюдений, которые будут переданы в сеть, пока неизвестно. На самом деле это число и не нужно: мы можем передать в сеть и одно наблюдение, и 1000, потому что тензорные операции проводятся по всем наблюдениям одновременно с использованием алгоритмов линейной алгебры, которые реализует библиотека TensorFlow. Этим же обусловлено и увеличение производительности, когда обучение глубоких нейронных сетей производится на графических процессорах вместо обычных: графические процессоры оптимизированы для умножения больших тензоров, потому что эти вычисления необходимы и для сложных манипуляций с графикой.

Метод `summary` выводит также количество параметров (весов), которые будут обучаться в каждом слое. Если вы обнаружите, что модель обучается слишком медленно, то выведите сводную информацию, чтобы увидеть, есть ли в сети какие-нибудь слои с огромным количеством весов. Если такие слои имеются, подумайте над тем, можно ли уменьшить число узлов в слое, чтобы ускорить обучение.



Убедитесь, что понимаете, как рассчитывается количество параметров в каждом слое! Важно помнить, что по умолчанию каждый узел в данном слое также подключен к одному дополнительному узлу *сдвига*, который всегда выводит 1. Это гарантирует, что выходной сигнал модуля может быть ненулевым, даже если все входные значения, полученные из предыдущего слоя, будут равны 0. Соответственно, количество параметров в слое Dense из 200 узлов составляет $200 \times (3072 + 1) = 614\,600$.

Компиляция модели

На этом этапе производится компиляция модели с заданными оптимизатором и функцией потерь, как показано в примере 2.7.

Пример 2.7. Определение оптимизатора и функции потерь

```
from tensorflow.keras import optimizers

opt = optimizers.Adam(learning_rate=0.0005)
model.compile(loss='categorical_crossentropy', optimizer=opt,
              metrics=['accuracy'])
```

Теперь познакомимся с понятиями функции потерь и оптимизатора.

Функции потерь

Функция потерь используется нейронной сетью для сравнения ее прогноза с истиной. Она возвращает одно число для каждого наблюдения; чем оно больше, тем хуже сеть справилась с прогнозированием этого наблюдения.

Keras предлагает множество встроенных функций потерь, а также дает вам возможность создавать собственные. На практике чаще всего используются три функции: средняя квадратическая ошибка, многозначная перекрестная энтропия и бинарная перекрестная энтропия. Важно понимать, когда лучше применять каждую из них.

Если нейронная сеть предназначена для решения задачи регрессии (то есть выходные данные представляют значения из непрерывной области), то можно использовать функцию потерь — *среднюю квадратическую ошибку*. Это среднее значение суммы квадратов разностей между истинным значением y_i и прогнозируемым p_i в каждом выходном узле, где среднее берется по всем n выходным узлам:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - p_i)^2.$$

Если нейронная сеть предназначена для решения задачи классификации, где каждое наблюдение может относиться только к одному классу, то правильнее будет выбрать функцию потерь, известную как *многозначная перекрестная энтропия*. Она определяется следующим образом:

$$-\sum_{i=1}^n y_i \log(p_i).$$

Наконец, если нейронная сеть предназначена для решения задачи бинарной классификации с одним выходным узлом или классификации с несколькими метками, когда каждое наблюдение может одновременно принадлежать нескольким классам, то следует использовать *бинарную перекрестную энтропию*:

$$-\frac{1}{n} \sum_{i=1}^n (y_i \log(p_i) + (1 - y_i) \log(1 - p_i)).$$

Оптимизаторы

Оптимизатор — это алгоритм, применяемый для обновления весов в нейронной сети на основе градиента функции потерь. Одним из наиболее часто используемых и стабильных оптимизаторов является *Adam* [Kingma, Ba, 2014]. В большинстве случаев вам не придется настраивать параметры по умолчанию оптимизатора Adam, за исключением *скорости обучения*. Чем выше скорость обучения, тем сильнее будут меняться веса на каждом шаге обучения. При более высокой скорости обучение изначально происходит быстрее, но протекает менее стабильно и может не найти минимум функции потерь. Этот параметр можно настраивать и корректировать во время обучения.

Другой распространенный оптимизатор, с которым вы можете столкнуться, — *RMSPprop*. Работая с ним, вам также не придется заниматься кропотливой настройкой параметров, но я советую прочитать документацию Keras (<https://keras.io/optimizers>), чтобы понять роль каждого из них.

В вызов метода модели `compile` передаются функция потерь, оптимизатор и параметр `metrics`, в котором можно передать любые дополнительные показатели, которые хотелось бы задать для процесса обучения, например точность.

Обучение модели

До сих пор мы не передали в модель никаких данных, а только настроили архитектуру и скомпилировали модель с функцией потерь и оптимизатором.

Чтобы обучить модель, достаточно вызвать метод `fit`, как показано в примере 2.8.

Пример 2.8. Вызов метода `fit` для обучения модели

```
model.fit(x_train ①
        , y_train ②
        , batch_size = 32 ③
        , epochs = 10 ④
        , shuffle = True ⑤
        )
```

- ① Исходные данные — массив с изображениями.
- ② Метки классов в формате прямого кодирования.
- ③ Параметр `batch_size` определяет, сколько наблюдений будет передаваться в сеть на каждом шаге обучения.
- ④ Параметр `epochs` определяет, сколько раз сеть будет просматривать полный комплект обучающих данных.
- ⑤ Если `shuffle = True`, то пакеты обучающих данных будут перемешиваться случайным образом перед каждым шагом обучения.

Он запускает обучение глубокой нейронной сети предсказанию категории изображения из набора данных CIFAR-10. Далее описывается, как протекает процесс обучения.

Сначала веса сети инициализируются небольшими случайными значениями. Затем сеть выполняет серию шагов обучения. На каждом шаге в нее передается один *пакет* изображений и в ходе обратного распространения ошибки корректируются веса. Параметр `batch_size` определяет количество изображений в каждом пакете. Чем больше его размер, тем стабильнее результат вычисления градиента, но каждый шаг обучения выполняется медленнее.



Если для вычисления градиента на каждом шаге обучения использовать весь набор данных, то потребуется слишком много времени и вычислительных ресурсов, поэтому размер пакета обычно выбирается в диапазоне от 32 до 256. Рекомендуется постепенно увеличивать его в процессе обучения [Smith et al., 2017].

Так продолжается, пока сеть не увидит все наблюдения, имеющиеся в наборе данных. Этим завершается первая *эпоха*. Затем начинается вторая эпоха, и данные вновь передаются в сеть пакетами. Этот процесс повторяется до тех пор, пока не будет достигнуто заданное число эпох.

Во время обучения Keras информирует о ходе выполнения (рис. 2.7). Как видите, в этом примере обучающий набор данных был разделен на 1563 пакета (по 32 изображения в каждом) и обработан сетью десять раз (то есть выполнено

десять эпох) со скоростью примерно 2 мс на пакет. Потеря многозначной перекрестной энтропии снизилась с 1,8377 до 1,3696, что привело к увеличению точности с 33,69 % после первой эпохи до 51,67 % после десятой.

```
model.fit(x_train, y_train, batch_size=32, epochs=10, shuffle=True)
Epoch 1/10
1563/1563 [=====] - 3s 2ms/step - loss: 1.8377 - accuracy: 0.3369
Epoch 2/10
1563/1563 [=====] - 3s 2ms/step - loss: 1.6552 - accuracy: 0.4076
Epoch 3/10
1563/1563 [=====] - 3s 2ms/step - loss: 1.5743 - accuracy: 0.4396
Epoch 4/10
1563/1563 [=====] - 3s 2ms/step - loss: 1.5288 - accuracy: 0.4549
Epoch 5/10
1563/1563 [=====] - 3s 2ms/step - loss: 1.4888 - accuracy: 0.4706
Epoch 6/10
1563/1563 [=====] - 2s 2ms/step - loss: 1.4542 - accuracy: 0.4851
Epoch 7/10
1563/1563 [=====] - 3s 2ms/step - loss: 1.4332 - accuracy: 0.4908
Epoch 8/10
1563/1563 [=====] - 2s 2ms/step - loss: 1.4094 - accuracy: 0.4992
Epoch 9/10
1563/1563 [=====] - 2s 2ms/step - loss: 1.3896 - accuracy: 0.5045
Epoch 10/10
1563/1563 [=====] - 3s 2ms/step - loss: 1.3696 - accuracy: 0.5167
```

Рис. 2.7. Вывод метода fit

Оценка модели

Мы знаем, что модель достигла точности 51,67 % на обучающем наборе, но хорошо ли она справится с данными, которых никогда не видела прежде?

Чтобы ответить на этот вопрос, можно использовать метод `evaluate` из библиотеки Keras, как показано в примере 2.9.

Пример 2.9. Оценка качества модели на контрольном наборе

```
model.evaluate(x_test, y_test)
```

На рис. 2.8 показано, что вывел этот метод.

```
10000/10000 [=====] - 1s 55us/step
[1.4358007415771485, 0.4896]
```

Рис. 2.8. Вывод метода evaluate

Он выводит список отслеживаемых показателей: многозначной перекрестной энтропией и точностью. Точность модели составляет 49,0 % на изображениях,

которых она никогда раньше не видела. Обратите внимание: если бы модель угадывала случайным образом, то ее точность составляла бы около 10 % (так как имеется десять классов), поэтому 49,0 % — хороший результат, учитывая, что использована очень простая нейронная сеть.

Мы можем просмотреть некоторые прогнозы для изображений из контрольного набора с помощью метода `predict`, как показано в примере 2.10.

Пример 2.10. Просмотр прогнозов, полученных для изображений из контрольного набора с помощью метода `predict`

```
CLASSES = np.array(['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
                    'frog', 'horse', 'ship', 'truck'])
```

```
preds = model.predict(x_test) ❶
preds_single = CLASSES[np.argmax(preds, axis = -1)] ❷
actual_single = CLASSES[np.argmax(y_test, axis = -1)]
```

❶ `preds` — это массив с формой `[10000, 10]`, то есть вектор с десятью вероятностями принадлежности к разным классам для каждого наблюдения.

❷ Этот массив вероятностей преобразуется в единственное предсказание с использованием функции `argmax` из `numpy`. Здесь `axis = -1` подразумевает функцию свертки массива по последнему измерению (измерению классов), то есть `preds_single` имеет форму `[10000, 1]`.

Мы можем просмотреть некоторые изображения вместе с их метками и прогнозами, выполнив код из примера 2.11. Как и ожидалось, точность прогнозирования составляет примерно 50 %.

Пример 2.11. Вывод прогнозов MLP вместе с фактическими метками

```
import matplotlib.pyplot as plt

n_to_show = 10
indices = np.random.choice(range(len(x_test)), n_to_show)

fig = plt.figure(figsize=(15, 3))
fig.subplots_adjust(hspace=0.4, wspace=0.4)

for i, idx in enumerate(indices):
    img = x_test[idx]
    ax = fig.add_subplot(1, n_to_show, i+1)
    ax.axis('off')
    ax.text(0.5, -0.35, 'pred = ' + str(preds_single[idx]), fontsize=10,
           ha='center', transform=ax.transAxes)
    ax.text(0.5, -0.7, 'act = ' + str(actual_single[idx]), fontsize=10,
           ha='center', transform=ax.transAxes)
    ax.imshow(img)
```


На рис. 2.9 показаны некоторые случайно выбранные изображения с прогнозами модели и истинными метками.



Рис. 2.9. Некоторые прогнозы, сделанные моделью, и фактические метки

Поздравляю! Вы только что сконструировали свою первую глубокую нейронную сеть с помощью Keras и использовали ее для прогнозирования новых данных. Несмотря на то что эта задача является задачей обучения с учителем, в последующих главах (при рассмотрении вопросов создания генеративных моделей) многие из основных понятий (функции потерь, функции активации, формы слоев и др.) не утратят своей важности. Теперь посмотрим на возможные пути улучшения этой модели введением нескольких слоев нового типа.

Сверточные нейронные сети (CNN)

Одна из причин того, почему наша сеть работает не так хорошо, как могла бы, заключается в том, что в ней нет ничего, что учитывало бы пространственную структуру входных изображений. Фактически на первом шаге мы «сплющиваем» изображения в вектор, чтобы передать их первому слою Dense.

Решить проблему можно, используя *сверточный слой*.

Сверточные слои

Что подразумевается под *сверткой* в контексте глубокого обучения?

На рис. 2.10 показано, как выполняется свертка фрагмента $3 \times 3 \times 1$ черно-белого изображения с использованием *фильтра* (или *ядра*) $3 \times 3 \times 1$. Свертка осуществляется поэлементным умножением фильтра на фрагмент изображения

и последующим суммированием результатов. Чем лучше фрагмент изображения соответствует фильтру, тем больше результат, и наоборот, чем хуже фрагмент соответствует фильтру, тем меньше результат. Верхний пример хорошо резонирует с фильтром, поэтому он дает большое положительное значение. Нижний пример плохо соответствует фильтру, поэтому дает значение, близкое к нулю.

Фрагмент изображения 3×3			Фильтр			
0.6	0.4	0.6	1	1	1	= 2.8
0.1	-0.2	-0.3	0	0	0	
-0.5	-0.4	-0.3	-1	-1	-1	
-0.7	0.6	0.2	1	1	1	= -0.1
0.1	0.5	-0.3	0	0	0	
-0.3	-0.4	0.5	-1	-1	-1	

Рис. 2.10. Применение сверточного фильтра 3×3 к двум фрагментам черно-белого изображения

Перемещая фильтр по всему изображению слева направо и сверху вниз и записывая результаты свертки, мы получим новый массив, в котором будут резко выделяться конкретные особенности входного изображения в зависимости от значений в фильтре. Например, на рис. 2.11 показаны два фильтра, выделяющие горизонтальные и вертикальные границы.



Запуск кода для этого примера

Увидеть наглядно, как выполняется этот сверточный процесс, можно в блокноте Jupyter, доступном по адресу Notes/02_deeplearning/02_cnn/convolutions.ipynb в репозитории книги.

Сверточный слой — это просто набор фильтров, а значения в фильтрах — это веса, которые определяются нейронной сетью в процессе обучения. Первоначально они выбираются случайным образом, но постепенно адаптируются, чтобы начать выделять интересные признаки, такие как края или определенные цветовые комбинации.

Слой Conv2D в Keras применяет свертки к входному тензору с двумя пространственными измерениями (изображение как раз является таким тензором).

В примере 2.12 показан код, использующий Keras, чтобы сконструировать сверточный слой с двумя фильтрами, соответствующий примеру на рис. 2.11.

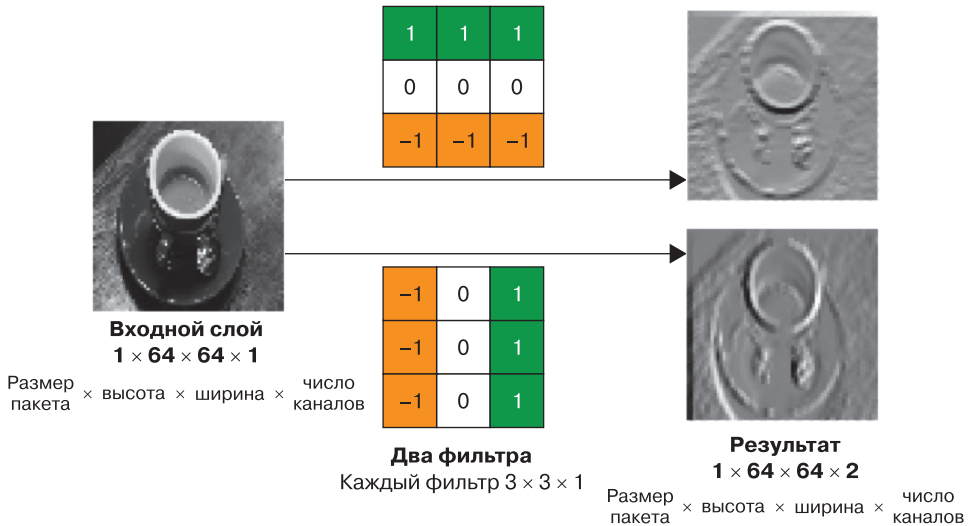


Рис. 2.11. Два сверточных фильтра применяются к черно-белому изображению

Пример 2.12. Слой Conv2D, применяемый к черно-белым входным изображениям

```
from tensorflow.keras import layers

input_layer = layers.Input(shape=(64,64,1))
conv_layer_1 = Conv2D(
    filters = 2
    , kernel_size = (3,3)
    , strides = 1
    , padding = "same"
)(input_layer)
```

Давайте более подробно рассмотрим параметры `strides` и `padding` слоя Conv2D.

Параметр `strides`

Параметр `strides` определяет величину шага перемещения фильтров по входным данным. Чем больше шаг, тем меньше размер выходного тензора. Например, при `strides = 2` высота и ширина выходного тензора будут вдвое меньше высоты и ширины входного. Это важно для уменьшения пространственных размеров тензора при прохождении через сеть при одновременном увеличении количества каналов.

Параметр padding

Если задан параметр `padding = "same"`, то входные данные будут дополняться нулями, чтобы размер выходного тензора в точности совпадал с размером входного при `strides = 1`.

На рис. 2.12 показано ядро 3×3 , которое перемещается по входному изображению 5×5 , с параметрами `padding = "same"` и `strides = 1`. На выходе этого сверточного слоя получится тензор с тем же размером 5×5 , так как дополнение позволяет ядру расширить границы изображения на краях, чтобы его можно было применить пять раз в обоих направлениях. Без дополнения ядро было бы применено только три раза вдоль каждого направления, и в результате получился бы тензор 3×3 .

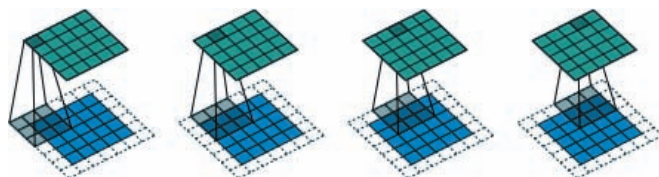


Рис. 2.12. Ядро $3 \times 3 \times 1$ (серое) перемещается по входному изображению $5 \times 5 \times 1$ (синее) с параметрами `padding="same"` и `strides=1`, в результате получается выходной тензор $5 \times 5 \times 1$ (зеленый). Изображение взято из статьи <https://arxiv.org/abs/1603.07285> [Dumoulin, Visin, 2018]

Параметр `padding = "same"` позволяет контролировать размер тензора по мере прохождения через множество сверточных слоев. Тензор, возвращаемый сверточным слоем с параметром `padding = "same"`, получает следующую форму:

$$\left(\frac{\text{высота входного изображения}}{\text{шаг}}, \frac{\text{ширина входного изображения}}{\text{шаг}}, \text{число фильтров} \right).$$

Добавление сверточных слоев

Результатом слоя `Conv2D` является другой четырехмерный тензор, имеющий форму (*размер пакета, высота, ширина, число фильтров*), поэтому слои `Conv2D` можно помещать друг за другом и тем самым увеличивать глубину нейронной сети. Для наглядности представим, что мы применяем слои `Conv2D` к набору данных CIFAR-10. На этот раз вместо одного канала, определяющего оттенок серого, их будет три: красный, зеленый и синий.

В примере 2.13 показано, как построить простую сверточную нейронную сеть, которую можно обучить, чтобы добиться большего успеха в решении нашей задачи.

Пример 2.13. Создание модели сверточной нейронной сети с помощью Keras

```
from tensorflow.keras import layers, models

input_layer = layers.Input(shape=(32,32,3))
conv_layer_1 = layers.Conv2D(
    filters = 10
    , kernel_size = (4,4)
    , strides = 2
    , padding = 'same'
)(input_layer)
conv_layer_2 = layers.Conv2D(
    filters = 20
    , kernel_size = (3,3)
    , strides = 2
    , padding = 'same'
)(conv_layer_1)
flatten_layer = layers.Flatten()(conv_layer_2)
output_layer = layers.Dense(units=10, activation = 'softmax')(flatten_layer)
model = models.Model(input_layer, output_layer)
```

Получившаяся архитектура сети показана на рис. 2.13.

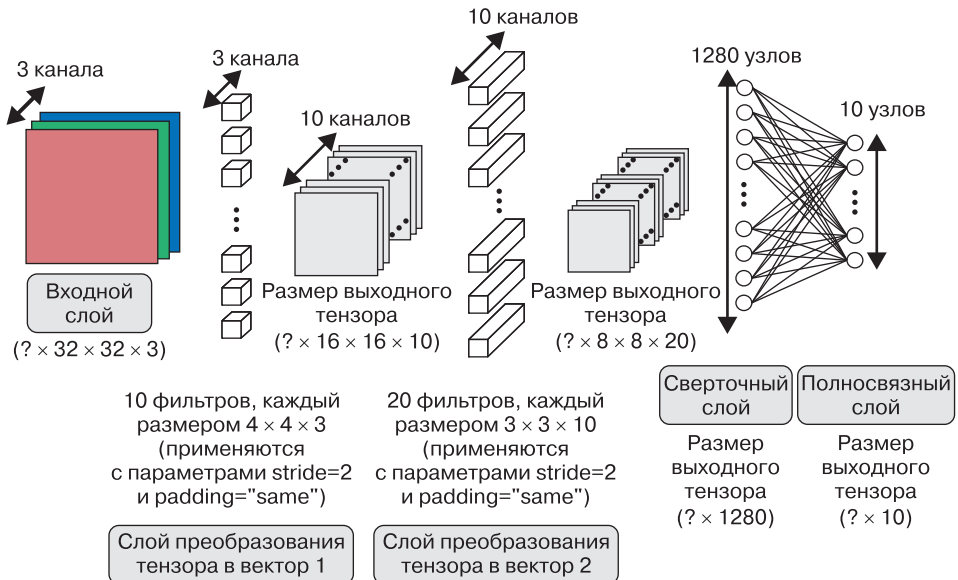


Рис. 2.13. Схема сверточной нейронной сети

Обратите внимание: теперь, когда обрабатываются цветные изображения, каждый фильтр в первом сверточном слое имеет глубину 3, а не 1 (то есть каждый фильтр имеет форму $4 \times 4 \times 3$, а не $4 \times 4 \times 1$). Это необходимо для обработки трех каналов (красного, зеленого, синего) входного изображения. Та же идея применима к фильтрам во втором сверточном слое, имеющим глубину 10, чтобы соответствовать десяти каналам на выходе первого сверточного слоя.



В общем случае глубина фильтров в слое всегда равна количеству каналов на выходе предыдущего слоя.

Исследование модели

Будет весьма познавательно посмотреть, как меняется форма тензора по мере прохождения данных от одного сверточного слоя к другому. Это можно сделать с помощью метода `model.summary()` (табл. 2.2).

Таблица 2.2. Сводная информация о сверточной нейронной сети

Слой (тип)	Выходная форма	Число параметров
Input	(None, 32, 32, 3)	0
Conv2D	(None, 16, 16, 10)	490
Conv2D	(None, 8, 8, 20)	1820
Flatten	(None, 1280)	0
Dense	(None, 10)	12 810

Всего параметров 15 120

Обучаемых параметров 15 120

Необучаемых параметров 0

Проанализируем эту сеть от входа до выхода, отмечая формы тензоров.

1. Входные данные имеют форму (None, 32, 32, 3) — Keras использует None, чтобы показать, что мы можем передать в сеть сразу любое количество изображений. Поскольку сеть просто выполняет операции тензорной алгебры, мы можем передавать изображения в сеть *пакетами*, а не по отдельности.
2. В первом сверточном слое все десять фильтров имеют форму $4 \times 4 \times 3$, так как мы выбрали фильтр с высотой и шириной 4 (`kernel_size = (4, 4)`) и в предыдущем слое данные имеют три канала: красный, зеленый и синий. Следовательно-

но, число параметров (или весов) в слое составляет $(4 \times 4 \times 3 + 1) \times 10 = 490$, где слагаемое 1 обусловлено включением члена смещения, присоединенного к каждому из фильтров. Как и прежде, результатом применения каждого фильтра к каждому фрагменту $4 \times 4 \times 3$ входного изображения является сумма поэлементных произведений весов фильтра на значения каналов в фрагменте изображения, который покрывает этот фильтр. Так как заданы параметры `strides = 2` и `padding = "same"`, ширина и высота выходного тензора в два раза меньше ширины и высоты входного тензора и равны 16, а поскольку имеется десять фильтров, выход первого слоя представляет собой пакет тензоров, каждый из которых имеет форму `[16, 16, 10]`.

3. Во втором сверточном слое мы выбрали форму 3×3 для фильтров, и теперь они имеют глубину 10 согласно количеству каналов в предыдущем слое. Поскольку в этом слое у нас 20 фильтров, общее количество параметров (весов) получается равным $(3 \times 3 \times 10 + 1) \times 20 = 1820$. И снова мы используем параметры слоя `strides = 2` и `padding = "same"`, из-за чего ширина и высота выходного тензора уменьшаются вдвое. Таким образом, выходной тензор имеет форму `(None, 8, 8, 20)`.
4. Далее многомерный тензор преобразуется в вектор с помощью слоя `Flatten`. В результате получается набор из $8 \times 8 \times 20 = 1280$ узлов. Обратите внимание на то, что в слое `Flatten` нет обучаемых параметров, потому что эта операция просто реструктурирует тензор.
5. Наконец, полученные узлы передаются в заключительный слой `Dense` с десятью узлами и функцией активации `softmax`, представляющими вероятность принадлежности изображения к каждой категории в задаче классификации с десятью категориями. Этот слой создает дополнительные $1280 \times 10 = 12\,810$ обучаемых параметров (весов).

Этот пример наглядно демонстрирует, как из нескольких сверточных слоев создать сверточную нейронную сеть. Но прежде чем показать, насколько эта сеть эффективнее распознает изображения по сравнению с полносвязной нейронной сетью, взглянем еще на два типа слоев, также позволяющих увеличить качество прогнозирования: `BatchNormalization` и `Dropout`.

Пакетная нормализация

Одна из проблем, часто возникающих при обучении глубоких нейронных сетей, заключается в сохранении значений весов в разумных пределах — если они становятся слишком большими, это признак того, что сеть страдает от так называемого *взрыва градиента*. Поскольку ошибки распространяются через сеть в обратном направлении, градиент, вычисляемый в ранних слоях, может иногда расти в экспоненциальной прогрессии, вызывая гигантские колебания значений веса.



Если функция потерь начинает возвращать NaN (not a number — «не число»), то, скорее всего, веса выросли настолько, что стали вызывать ошибку переполнения.

Подобное может происходить не сразу после начала обучения сети. Иногда сеть может благополучно обучаться часами, и вдруг функция потерь возвращает NaN и сеть взрывается. Это особенно неприятно, когда сеть, казалось бы, успешно обучается в течение длительного времени. Чтобы этого не случилось, нужно понять причину взрыва градиента.

Ковариантный сдвиг

Одной из причин масштабирования входных данных является стремление обеспечить стабильное начало обучения в течение первых нескольких итераций. Поскольку веса сети изначально выбираются случайно, немасштабированные входные данные потенциально могут привести к получению огромных значений активации, которые немедленно вызывают взрыв градиента. Например, вместо передачи фактических значений пикселей, изменяющихся в диапазоне от 0 до 255, мы обычно масштабируем эти значения и приводим к диапазону от -1 до 1 .

Поскольку входные данные масштабируются, естественно ожидать, что активации во всех последующих слоях тоже будут довольно хорошо масштабироваться. На первых итерациях это действительно может быть так, но по мере обучения сети веса будут отодвигаться все дальше от своих случайных начальных значений, и в результате предположение может перестать соответствовать действительности. Это явление известно как *ковариантный сдвиг* (covariate shift).



Аналогия ковариантного сдвига

Представьте, что вы несете высокую стопку книг и вдруг налетает ветер. Вы наклоняете ее навстречу ветру, чтобы компенсировать его воздействие, но при этом некоторые книги сдвигаются, так что вся стопка становится менее устойчивой. В первое время вы продолжаете нести стопку, не испытывая особых проблем, но с каждым порывом она становится все более и более неустойчивой и наконец падает. Это и есть ковариантный сдвиг.

Вернемся к нейронным сетям: каждый их слой подобен книге в стопке. Чтобы не потерять стабильность, когда сеть обновляет веса, каждый слой неявно предполагает, что распределение входных данных, получаемых от предыдущего слоя, приблизительно одинаково на всех итерациях. Но поскольку ничто не может предотвратить значительное смещение распределения какой-либо из активаций в определенном направлении, иногда это может приводить к неконтролируемым изменениям значений весов и к общему коллапсу сети.

Обучение с использованием пакетной нормализации

Пакетная нормализация значительно смягчает эту проблему. Решение выглядит на удивление простым. Слой пакетной нормализации вычисляет среднее значение и стандартное отклонение для каждого из входных каналов в пакете и нормализует эти показатели, вычитая среднее значение и деля разность на стандартное отклонение. Для каждого канала есть два обучаемых параметра: масштаб (γ) и сдвиг (β). Выход слоя — это всего лишь нормализованный вход, масштабированный по величине γ и сдвинутый на величину β (рис. 2.14).

Вход: значения x в пакете: $\mathcal{B}\{x_{1..m}\}$;

Параметры для обучения: γ, β

Выход: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{Среднее по пакету}$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad \text{Стандартное отклонение по пакету}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad \text{Нормализация}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad \text{Масштабирование и сдвиг}$$

Алгоритм 1: Batch Normalizing Transform (Преобразование пакетной нормализации), применяется к активации x в пакете

Рис. 2.14. Процесс пакетной нормализации

(источник: [Ioffe, Szegedy, 2015], <https://arxiv.org/abs/1502.03167>)

Слой пакетной нормализации можно поместить после полносвязных или сверточных слоев, чтобы нормализовать их выход.



Это похоже на связывание книг в стопку, гарантирующее, что с течением времени в их положении не произойдут огромные сдвиги.

Прогнозирование с использованием пакетной нормализации

Как этот слой работает во время тестирования? На этапе прогнозирования может понадобиться спрогнозировать только одно наблюдение, но в этот момент не существует *пакета*, для которого можно вычислить средние значения. Чтобы обойти эту проблему, во время обучения слой пакетной нормализации также вычисляет скользящее среднее от среднего значения и стандартного отклонения каждого канала и сохраняет его в слое для использования во время тестирования.

Сколько параметров содержится в слое пакетной нормализации? Для каждого канала в предыдущем слое определяются два веса: масштаб (γ) и сдвиг (β). Это *обучаемые* параметры. Скользящее среднее и стандартное отклонение вычисляются для каждого канала на основе данных, проходящих через слой, а не на основе ошибок, распространяющихся в обратном направлении, и поэтому называются *необучаемыми* параметрами. Таким образом, из четырех параметров для каждого канала в предыдущем слое два являются обучаемыми, а два — необучаемыми.

В Keras пакетную нормализацию реализует слой `BatchNormalization`, как показано в примере 2.14.

Пример 2.14. Слой `BatchNormalization` в Keras

```
from tensorflow.keras import layers
layers.BatchNormalization(momentum = 0.9)
```

Параметр `momentum` — вес, придаваемый предыдущему значению при вычислении скользящего среднего и стандартного отклонения.

Слои прореживания

При подготовке к экзамену, чтобы упрочить свои знания, студенты обычно используют конспекты и примерные вопросы. Некоторые пытаются запомнить ответы на них, но затем испытывают трудности на экзамене, потому что не понимают сути другого вопроса. Более успешные студенты читают учебники и решают практические задачи, чтобы углубить свое понимание предмета и суметь правильно ответить на вопросы, которых им прежде не задавали.

Тот же принцип действует и в машинном обучении. Любой успешный алгоритм машинного обучения должен уметь обобщать полученные знания на новые, прежде не виденные им данные, а не просто *запоминать* набор обучающих данных. Если алгоритм показывает хорошие результаты на наборе обучающих данных, но часто ошибается на контрольном наборе данных, то очевидно, что

он страдает от *переобучения*. Для смягчения этой проблемы используются методы *регуляризации*, гарантирующие, что модель будет оштрафована, если начнет переобучаться.

Есть много способов регуляризации алгоритма машинного обучения, но в глубоком обучении чаще всего применяется прием, основанный на применении слоев *прореживания*. Эта идея была представлена Джеффри Хинтоном (Geoffrey Hinton) в 2012 году [Hinton et al., 2012], а двумя годами позже описана в статье Шриваставы (Srivastava) и его соавторов [Srivastava et al., 2014].

Слои прореживания устроены очень просто. Во время обучения каждый прореживающий слой выбирает случайный набор узлов из предыдущего слоя и устанавливает их выходные значения равными нулю (рис. 2.15).

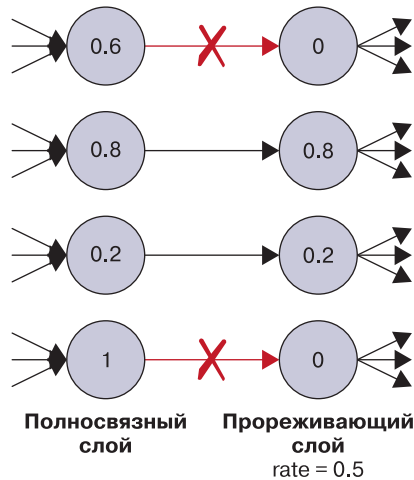


Рис. 2.15. Прореживающий слой

Эта простая манипуляция значительно снижает вероятность переобучения, гарантируя, что сеть не станет чрезмерно зависимой от определенных узлов или их групп, которые, по сути, просто запоминают наблюдения из обучающего набора. При добавлении прореживающих слоев сеть больше не может слишком полагаться на какой-то один узел, поэтому знания распределяются по ней более равномерно.

Так модель обретает способность лучше обобщать прежде не встречавшиеся ей данные, поскольку была обучена давать точные прогнозы даже в незнакомых условиях, вызванных сбросом значений случайных узлов. В прореживающем слое нет весов для обучения — отбрасываемые узлы определяются стохастически

(случайно). На этапе прогнозирования прореживающий слой ничего не отбрасывает, поэтому для прогнозирования используется вся сеть.



Аналогия прореживания

Прореживание напоминает то, как студент-математик читает конспекты, случайно выбирая ключевые формулы, отсутствующие в справочнике (учебнике). Так он учится отвечать на вопросы, применяя основные принципы, а не отыскивая готовые решения. В результате на экзамене ему будет гораздо проще отвечать на вопросы, которые прежде не задавали, опираясь на свои способности делать обобщения, выходя за пределы учебников.

Слой `Dropout` в Keras реализует эту функцию с помощью параметра `rate`, определяющего долю отбрасываемых узлов из предыдущего слоя, как показано в примере 2.15.

Пример 2.15. Слой `Dropout` в Keras

```
from tensorflow.keras import layers
layers.Dropout(rate = 0.25)
```

Обычно слои `Dropout` используются после слоев `Dense`, как наиболее склонных к переобучению из-за большого количества весов. Но их можно применять и после сверточных слоев.



Пакетная нормализация, представленная ранее, тоже уменьшает вероятность переобучения, поэтому многие современные архитектуры глубокого обучения не используют прореживание, а применяют для регуляризации только пакетную нормализацию. Как и для большинства принципов глубокого обучения, не существует золотого правила, применимого в любой ситуации. Единственный способ точно узнать, что оптимальнее, — протестировать разные архитектуры и определить, какие лучше всего работают с контрольным набором данных.

Построение CNN

Итак, вы познакомились с тремя новыми типами слоев в Keras: `Conv2D`, `BatchNormalization` и `Dropout`. Объединим их в модель CNN и посмотрим, хорошо ли она справится с задачей классификации набора данных CIFAR-10.



Запуск кода этого примера

Для этого можно воспользоваться блокнотом `notebooks/02_deeplearning/02_cnn/cnn.ipynb` для Jupyter Notebook из репозитория с примерами для книги.

Архитектура нашей модели показана в примере 2.16.

Пример 2.16. Код, конструирующий модель CNN с помощью Keras

```
from tensorflow.keras import layers, models

input_layer = layers.Input((32,32,3))

x = layers.Conv2D(filters = 32, kernel_size = 3
                  , strides = 1, padding = 'same')(input_layer)
x = layers.BatchNormalization()(x)
x = layers.LeakyReLU()(x)

x = layers.Conv2D(filters = 32, kernel_size = 3, strides = 2, padding = 'same')(x)
x = layers.BatchNormalization()(x)
x = layers.LeakyReLU()(x)

x = layers.Conv2D(filters = 64, kernel_size = 3, strides = 1, padding = 'same')(x)
x = layers.BatchNormalization()(x)
x = layers.LeakyReLU()(x)

x = layers.Conv2D(filters = 64, kernel_size = 3, strides = 2, padding = 'same')(x)
x = layers.BatchNormalization()(x)
x = layers.LeakyReLU()(x)

x = layers.Flatten()(x)

x = layers.Dense(128)(x)
x = layers.BatchNormalization()(x)
x = layers.LeakyReLU()(x)
x = layers.Dropout(rate = 0.5)(x)

output_layer = layers.Dense(10, activation = 'softmax')(x)

model = models.Model(input_layer, output_layer)
```

Здесь используются четыре слоя `Conv2D`, за каждым из которых следуют слои `BatchNormalization` и `LeakyReLU`. После преобразования полученного тензора в вектор мы пропускаем данные через слой `Dense` размером 128, за которым снова следуют `BatchNormalization` и `LeakyReLU`. Далее следует слой `Dropout` для регуляризации, а завершается сеть выходным слоем `Dense` размером 10.



Порядок следования слоев пакетной нормализации и активации — это во многом вопрос личных предпочтений. Обычно слой пакетной нормализации помещается перед слоем активации, но в некоторых успешных архитектурах эти слои применяются в обратном порядке. Если вы решите использовать пакетную нормализацию перед активацией, то запомнить порядок вам поможет мнемоническое сокращение **BAD** (batch normalization, activation, then dropout — «пакетная нормализация, активация и затем прореживание»)!

В табл. 2.3 показана сводная информация о модели.

Таблица 2.3. Сводная информация о модели CNN для классификации CIFAR-10

Слой (тип)	Выходная форма	Число параметров
Input	(None, 32, 32, 3)	0
Conv2D	(None, 32, 32, 32)	896
BatchNormalization	(None, 32, 32, 32)	128
LeakyReLU	(None, 32, 32, 32)	0
Conv2D	(None, 16, 16, 32)	9248
BatchNormalization	(None, 16, 16, 32)	128
LeakyReLU	(None, 16, 16, 32)	0
Conv2D	(None, 16, 16, 64)	18 496
BatchNormalization	(None, 16, 16, 64)	256
LeakyReLU	(None, 16, 16, 64)	0
Conv2D	(None, 8, 8, 64)	36 928
BatchNormalization	(None, 8, 8, 64)	256
LeakyReLU	(None, 8, 8, 64)	0
Flatten	(None, 4096)	0
Dense	(None, 128)	524 416
BatchNormalization	(None, 128)	512
LeakyReLU	(None, 128)	0
Dropout	(None, 128)	0
Dense	(None, 10)	1290

Всего параметров 592 554

Обучаемых параметров 591 914

Необучаемых параметров 640



Прежде чем двигаться дальше, проверьте, сможете ли вы вручную вычислить форму выходного тензора и количество параметров для каждого слоя. Это хороший способ доказать себе, что вы полностью разобрались в том, как устроен каждый слой и как он связан с предыдущим слоем! Не забудьте про веса смещений, которые включены в состав слоев Conv2D и Dense.

Обучение и оценка CNN

Скомпилируем и обучим модель, а затем вызовем метод `evaluate`, чтобы определить ее точность на контрольном наборе (рис. 2.16).

```
model.evaluate(x_test, y_test, batch_size=1000)

10000/10000 [=====] - 15s 1ms/step

[0.8423407137393951, 0.7155999958515167]
```

Рис. 2.16. Результаты классификации с использованием CNN

Как видите, этой модели удалось достичь точности 71,5 %, что можно считать большим успехом по сравнению с точностью 49,0 % предыдущей модели. На рис. 2.17 показаны некоторые прогнозы, полученные с помощью новой сверточной модели.



Рис. 2.17. Прогнозы, сделанные сетью CNN

Столь заметное улучшение достигнуто простым изменением архитектуры модели, включением в нее сверточных слоев, пакетной нормализации и прореживания. Обратите внимание: количество параметров в новой модели меньше, чем в предыдущей, хотя количество слоев и увеличилось. Это доказывает, насколько важно экспериментировать с архитектурой модели и понимать, как в своих интересах использовать разные типы слоев. При конструировании генеративных моделей еще важнее понимать внутреннюю работу модели, потому что именно слои, расположенные в середине сети, выявляют высокоуровневые признаки, которые нас интересуют больше всего.

Резюме

Итак, мы рассмотрели основные идеи глубокого обучения, которые понадобятся вам при создании своих первых глубоких генеративных моделей. Мы начали с создания многослойного перцептрона (MLP) с использованием Keras и на наборе данных CIFAR-10 обучили модель прогнозировать категорию этого изображения. Затем улучшили архитектуру модели, добавив сверточные слои, слои пакетной нормализации и слои прореживания, и создали сверточную нейронную сеть (CNN).

Самый важный урок этой главы: глубокие нейронные сети по своей природе очень гибкие и не существует правил, жестко регламентирующих создание архитектуры модели, что позволяет смело экспериментировать со слоями и их порядком. Конечно, существуют определенные рекомендации и практики, но это не означает, что вы не можете экспериментировать со слоями и порядком их размещения. Важно помнить об этом, чтобы не чувствовать себя обязанными использовать только архитектуры, заимствованные из книг, — рассматривайте их как примеры, демонстрирующие возможность сочетания слоев разных типов. Архитектура создаваемой нейронной сети ограничена только вашим воображением.

В следующей главе мы обсудим, как задействовать эти строительные блоки для конструирования сети, способной генерировать изображения.

Часть II

Методы

Здесь мы исследуем шесть семейств генеративных моделей, включая теорию их работы и практические примеры построения.

В **главе 3** рассмотрим свою первую генеративную модель глубокого обучения — *вариационный автокодировщик*. Эти модели позволяют не только создавать реалистичные лица, но и изменять существующие изображения, например добавлять улыбку или изменять цвет волос.

В **главе 4** рассматривается один из самых успешных методов генеративного моделирования последних лет — *генеративно-сопоставительная сеть* (Generative Adversarial Network, GAN). Мы увидим, как обучение GAN было доработано и адаптировано для постоянного расширения возможностей генеративного моделирования.

В **главе 5** углубимся в несколько примеров *авторегрессионных моделей*, включая LSTM и PixelCNN. Это семейство моделей рассматривает процесс генерации как задачу прогнозирования последовательности — она лежит в основе современных моделей генерации текста, а также может использоваться для генерации изображений.

В **главе 6** мы рассмотрим семейство *моделей нормализации потока*, включая RealNVP. Эта модель основана на формуле замены переменных, которая позволяет преобразовать простое распределение, такое как распределение Гаусса, в более сложное распределение, чтобы сохранить управляемость.

В **главе 7** представлено семейство *моделей на основе энергии*, или *энергетических моделей*. Они обучают скалярную энергетическую функцию для оценки достоверности заданных входных данных. Мы изучим метод обучения энергетических моделей, называемый контрастивной дивергенцией, и метод выборки новых наблюдений, называемый динамикой Ланжевена.

Наконец, в **главе 8** рассмотрим семейство *диффузионных моделей*. Этот метод моделирования основан на итеративном добавлении шума к изображению и последующем обучении модели удалять его, что дает возможность преобразовывать чистый шум в реалистичные образцы.

К концу **части II** вы создадите практические примеры генеративных моделей из всех шести семейств и сможете объяснить, как каждая из них работает с теоретической точки зрения.

Вариационные автокодировщики

В этой главе вы:

- узнаете, как архитектурные особенности автокодировщиков делают их идеально подходящими для генеративного моделирования;
- создадите и обучите автокодировщик с нуля, используя Keras;
- попробуете применить автокодировщики для создания новых изображений и узнаете об ограничениях этого подхода;
- познакомитесь с архитектурой вариационного автокодировщика и узнаете, как эта модель решает многие проблемы, характерные для стандартных автокодировщиков;
- создадите вариационный автокодировщик с нуля с помощью Keras;
- попробуете использовать вариационные автокодировщики для создания новых изображений;
- попробуете применить вариационные автокодировщики для манипулирования сгенерированными изображениями с помощью арифметики скрытого пространства.

В 2013 году Дидерик П. Кингма (Diederik P. Kingma) и Макс Веллинг (Max Welling) опубликовали статью, заложившую основы типа нейронной сети, известной как вариационный автокодировщик (variational autoencoder, VAE) [Kingma, Welling, 2013]. Теперь это одна из самых фундаментальных и хорошо известных архитектур глубокого обучения для генеративного моделирования и отличная отправная точка в пути к генеративному глубокому обучению.

Эту главу мы и начнем с создания стандартного автокодировщика, а затем расширим его и попытаемся получить вариационный автокодировщик. Попутно детально рассмотрим оба типа моделей, чтобы понять, как они работают. К концу главы вы получите полное представление о том, как создавать модели на основе автокодировщика и манипулировать ими, в частности, как разработать вариационный автокодировщик с нуля для генерации изображений на основе вашего собственного обучающего набора.

Введение

Начнем с простой истории, которая поможет объяснить фундаментальную проблему, которую пытается решить автокодировщик.

БРАЙАН, СТИЧ И ПЛЯТЯНОЙ ШКАФ

Представьте, что на полу перед вами лежит куча вашей одежды: брюки, футболки, обувь и пальто разных моделей. Ваш стилист Брайан расстраивается из-за того, что ему приходится тратить много времени, чтобы найти в этой куче нужные вещи, поэтому он разрабатывает хитрый план. Он советует вам убрать одежду в платяной шкаф, бесконечно высокий и широкий (рис. 3.1). Если вам понадобится конкретная вещь, то вы просто должны сообщить Брайану о ее местонахождении, и он сошьет ее с нуля. Вскоре становится очевидно, что лучше размещать похожие предметы рядом друг с другом, чтобы Брайан мог точно воссоздать каждый предмет, учитывая только его местоположение.



Рис. 3.1. Человек, стоящий перед безразмерным платяным шкафом
(создано с помощью Midjourney, <https://midjourney.com/>)

После нескольких недель практики вы с Брайаном приспособились к планировке платяного шкафа, и теперь вы можете сообщить ему о местонахождении любого предмета одежды, а он — сшить точную его копию!

У вас возникает идея проверить, что получится, если указать Брайану на пустое место в шкафу. К своему изумлению, вы обнаруживаете, что Брайан способен создавать совершенно новые предметы одежды, которых раньше не существовало! Процесс неидеален, но теперь у вас есть безграничные возможности для создания новой одежды — достаточно указать на пустующее место в безразмерном шкафу и позволить Брайану сотворить чудо на швейной машине.

А теперь посмотрим, как эта история связана с созданием автокодировщиков.

Автокодировщики

На рис. 3.2 изображена схема процесса, описываемого этой историей. Вы играете роль *кодировщика*, перемещая каждый предмет одежды в нужное место в шкафу. Это называется *кодированием*, а Брайан играет роль *декодировщика*, отыскивая место в шкафу и пытаясь воссоздать предмет. Этот процесс называется *декодированием*.

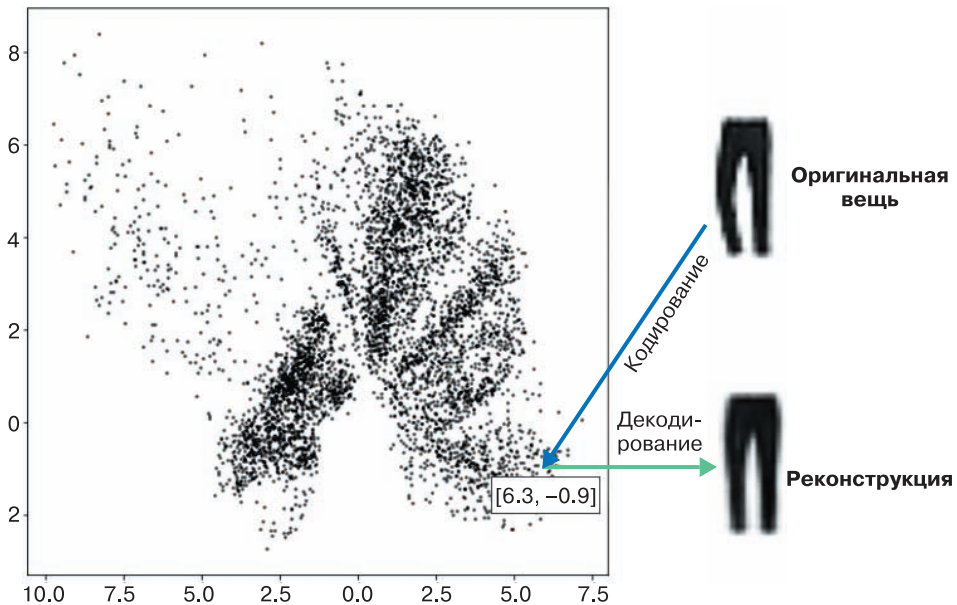


Рис. 3.2. Одежда в безразмерном шкафу — каждая черная точка представляет предмет одежды

Каждое место в гардеробе представлено двумя числами, то есть двумерным вектором. Например, брюки на рис. 3.2 закодированы точкой $[6.3, -0.9]$. Этот вектор также известен как *векторное представление* (embedding), поскольку кодер пытается представить с его помощью как можно больше информации, чтобы декодировщик мог воспроизвести точную реконструкцию.

Автокодировщик — это просто нейронная сеть, обученная выполнять задачу кодирования и декодирования элемента так, чтобы выходные данные этого процесса оказались как можно ближе к исходному элементу. Важно отметить, что ее можно использовать в качестве генеративной модели, потому что с ее помощью

можно декодировать любые точки в двумерном пространстве (даже те, которые не являются представлением оригинальных предметов), чтобы создать новый предмет одежды.

Теперь посмотрим, как создать автокодировщик с помощью Keras и применить его к реальному набору данных.



Запуск кода этого примера

Код для этого примера находится в блокноте Jupyter, доступном по адресу notebooks/03_vae/01_autoencoder/autoencoder.ipynb в репозитории книги.

Набор данных Fashion-MNIST

В этом примере будем использовать набор данных Fashion-MNIST (<https://oreil.ly/DS4-4>) — коллекцию черно-белых изображений предметов одежды, каждое размером 28×28 пикселей. Некоторые примеры изображений из этого набора данных показаны на рис. 3.3.

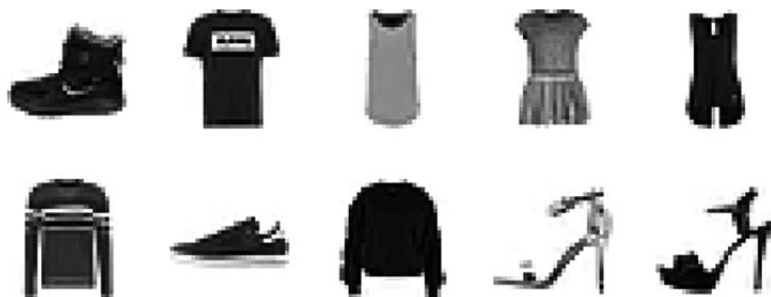


Рис. 3.3. Примеры изображений из набора данных Fashion-MNIST

Набор данных распространяется в виде архива вместе с библиотекой TensorFlow, поэтому его можно загрузить, как показано в примере 3.1.

Пример 3.1. Загрузка набора данных Fashion-MNIST

```
from tensorflow.keras import datasets
(x_train, y_train), (x_test, y_test) = datasets.fashion_mnist.load_data()
```

Это черно-белые изображения размером 28×28 (пиксели имеют значения от 0 до 255), которые необходимо предварительно обработать, чтобы привести значения пикселей в диапазон от 0 до 1. Мы также увеличим размеры каждого изображения до 32×32 , чтобы упростить манипулирование формой тензора при его прохождении через сеть, как показано в примере 3.2.

Пример 3.2. Предварительная обработка данных

```
def preprocess(imgs):  
    imgs = imgs.astype("float32") / 255.0  
    imgs = np.pad(imgs, ((0, 0), (2, 2), (2, 2)), constant_values=0.0)  
    imgs = np.expand_dims(imgs, -1)  
    return imgs  
  
x_train = preprocess(x_train)  
x_test = preprocess(x_test)
```

Теперь познакомимся с общей структурой автокодировщика, чтобы понять, как его создать с помощью TensorFlow и Keras.

Архитектура автокодировщика

Автокодировщик — это нейронная сеть, состоящая из двух частей:

- сеть *кодировщика* сжимает многомерные входные данные, такие как изображение, в векторное представление меньшей размерности;
- сеть *декодировщика* распаковывает заданное векторное представление в исходное состояние, например в изображение.

Схема архитектуры сети показана на рис. 3.4. Входное изображение кодируется в скрытое векторное представление z , которое затем декодируется в исходное пространство пикселей.

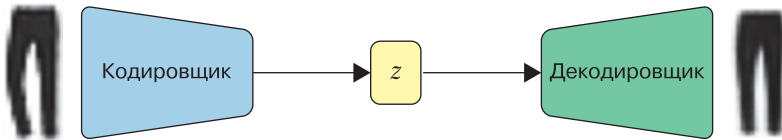


Рис. 3.4. Архитектура автокодировщика

Автокодировщик обучен восстанавливать изображение после того, как оно прошло через кодировщик и обратно через декодировщик. На первый взгляд кажется странным: зачем восстанавливать набор изображений, которые уже есть? Однако, как мы увидим далее, именно пространство представления, также называемое *скрытым* пространством, — это наиболее интересная часть автокодировщика, потому что выборка из него позволяет генерировать новые изображения.

Сначала определим, что подразумевается под представлением. Представление (z) — это сжатая форма исходного изображения в скрытом пространстве меньшей размерности. Суть в том, что можно генерировать новые изображения,

выбирая произвольные точки в скрытом пространстве и пропуская их через декодировщик, обученный преобразовывать такие точки в полноценные изображения.

В своем примере мы создаем представления изображений в двумерном скрытом пространстве. Это поможет визуализировать скрытое пространство, так как мы легко справляемся с построением точек на плоскости. На практике скрытое пространство автокодировщика часто имеет больше двух измерений, чтобы можно было уловить больше нюансов изображений.



Автокодировщики как модели удаления шума

Автокодировщики можно использовать и для очистки искаженных изображений от шума, так как в процессе обучения кодировщик узнает, что бессмысленно пытаться фиксировать случайный шум в скрытом пространстве. В подобных задачах двумерное скрытое пространство слишком мало для представления релевантной информации, имеющейся во входных данных. Однако, как мы увидим далее, увеличение размерности скрытого пространства быстро приводит к проблемам, если использовать автокодировщик в качестве генеративной модели.

Теперь посмотрим, как построить кодировщик и декодировщик.

Автокодировщик

Задача кодировщика в автокодировщике состоит в том, чтобы взять входное изображение и отобразить его в векторное представление в скрытом пространстве. Архитектура кодировщика, который мы будем строить, показана в табл. 3.1.

Таблица 3.1. Сводная информация о кодировщике

Слой (тип)	Выходная форма	Число параметров
Input	(None, 32, 32, 1)	0
Conv2D	(None, 16, 16, 32)	320
Conv2D	(None, 8, 8, 64)	18 496
Conv2D	(None, 4, 4, 128)	73 856
Flatten	(None, 2048)	0
Dense	(None, 2)	4098

Всего параметров 96 770

Обучаемых параметров 96 770

Необучаемых параметров 0

Чтобы получить нужную архитектуру, создадим слой `Input` для приема изображений, последовательно пропустим их через три слоя `Conv2D`, каждый из которых выявляет все более высокоуровневые признаки. Используем размер шага 2 на выходе каждого слоя, чтобы вдвое уменьшить размер выходных данных, и одновременно увеличим число каналов. Результат последнего сверточного слоя преобразуется в вектор и передается в слой `Dense` размером 2, представляющий двумерное скрытое пространство.

В примере 3.3 показано, как построить эту модель с помощью Keras.

Пример 3.3. Кодировщик

```
encoder_input = layers.Input(
    shape=(32, 32, 1), name="encoder_input"
) ❶
x = layers.Conv2D(32, (3, 3), strides = 2, activation = 'relu', padding="same")(
    encoder_input
) ❷
x = layers.Conv2D(64, (3, 3), strides = 2, activation = 'relu', padding="same")(x)
x = layers.Conv2D(128, (3, 3), strides = 2, activation = 'relu', padding="same")(x)
shape_before_flattening = K.int_shape(x)[1:]

x = layers.Flatten()(x) ❸
encoder_output= layers.Dense(2, name='encoder_output')(x) ❹

encoder = models.Model(encoder_input, encoder_output) ❺
```

- ❶ Определение входного слоя кодировщика (принимающего изображение).
- ❷ Стопка сверточных слоев, наложенных друг на друга.
- ❸ Преобразование результата последнего сверточного слоя в плоский вектор.
- ❹ Слой `Dense`, преобразующий вектор в двумерное скрытое пространство.
- ❺ Модель Keras, определяющая кодировщик, — модель, которая принимает исходное изображение и кодирует его в точку в двумерном пространстве.



Обязательно экспериментируйте с параметрами моделей, представленных в книге, чтобы понять, как архитектура влияет на количество весов в каждом слое, качество и время выполнения модели.

Декодировщик

Декодировщик является зеркальным отражением кодировщика, но вместо сверточных слоев в нем используются слои *обратной свертки*, как показано в табл. 3.2.

Таблица 3.2. Сводная информация о декодировщике

Слой (тип)	Выходная форма	Число параметров
Input	(None, 2)	0
Dense	(None, 2048)	6144
Reshape	(None, 4, 4, 128)	0
Conv2DTranspose	(None, 8, 8, 128)	147 584
Conv2DTranspose	(None, 16, 16, 64)	73 792
Conv2DTranspose	(None, 32, 32, 32)	18 464
Conv2D	(None, 32, 32, 1)	289

Всего параметров	246 273
Обучаемых параметров	246 273
Необучаемых параметров	0

СЛОИ ОБРАТНОЙ СВЕРТКИ

Обычный сверточный слой позволяет вдвое уменьшить высоту и ширину входного тензора, если установить параметр `strides = 2`.

Слой обратной свертки использует тот же принцип, что и обычный сверточный (применяет фильтр к изображению), с той лишь разницей, что параметр `strides = 2` удваивает высоту и ширину входного тензора.

В слое обратной свертки параметр `strides` определяет размер внутреннего дополнения для добавления нулевых значений между пикселями изображения (рис. 3.5). Здесь фильтр $3 \times 3 \times 1$ (серый) последовательно накладывается на изображение $3 \times 3 \times 1$ (синее) с параметром `strides = 2`, и в результате получается выходной тензор $6 \times 6 \times 1$ (зеленый).

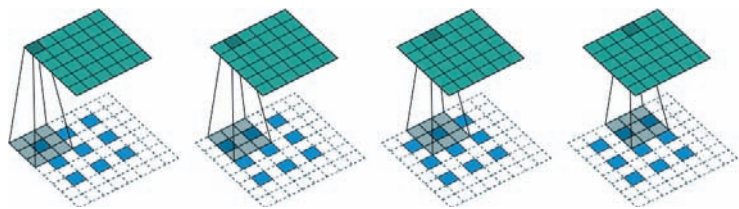


Рис. 3.5. Принцип работы слоя обратной свертки
(источник: [Dumoulin, Visin, 2018], <https://arxiv.org/abs/1603.07285>)

Слой `Conv2DTranspose` в Keras позволяет применять к тензорам операцию, обратную свертке. Добавляя такие слои, можно постепенно с помощью параметра `strides = 2` увеличивать размер выходных данных до размеров, соответствующих размерам оригинального изображения 32×32 .

Пример 3.4 демонстрирует реализацию декодировщика с использованием Keras.

Пример 3.4. Декодировщик

```
decoder_input = layers.Input(shape=(2,), name='decoder_input') ❶
x = layers.Dense(np.prod(shape_before_flattening))(decoder_input) ❷
x = layers.Reshape(shape_before_flattening)(x) ❸
x = layers.Conv2DTranspose(
    128, (3, 3), strides=2, activation = 'relu', padding="same"
)(x) ❹
x = layers.Conv2DTranspose(
    64, (3, 3), strides=2, activation = 'relu', padding="same"
)(x)
x = layers.Conv2DTranspose(
    32, (3, 3), strides=2, activation = 'relu', padding="same"
)(x)
decoder_output = layers.Conv2D(
    1,
    (3, 3),
    strides = 1,
    activation="sigmoid",
    padding="same",
    name="decoder_output"
)(x)

decoder = models.Model(decoder_input, decoder_output) ❺
```

❶ Определение входного слоя декодировщика (принимающего точку в скрытом пространстве).

❷ Входные данные передаются в слой `Dense`.

❸ Полученный вектор преобразуется в тензор в форме, в которой его можно передать первому слою обратной свертки.

❹ Стопка слоев обратной свертки, наложенных друг на друга.

❺ Модель Keras, определяющая декодировщик, — принимает точку в скрытом пространстве и декодирует ее в изображение в исходном пространстве.

Объединение кодировщика и декодировщика

Чтобы обучить кодировщик и декодировщик одновременно, нужно определить модель, которая будет направлять поток изображений через кодировщик и обратно через декодировщик. К счастью, Keras упрощает эту задачу (пример 3.5). Обратите внимание на то, что выходные данные автокодировщика — это просто выходные данные кодировщика, обработанные декодировщиком.

Пример 3.5. Полный автокодировщик

```
autoencoder = Model(encoder_input, decoder(encoder_output)) ❶
```

❶ Модель Keras, определяющая полный автокодировщик, — модель, которая принимает изображение, пропускает его через кодировщик и затем обратно через декодировщик, в результате чего получается реконструированное изображение.

Теперь, когда у нас есть модель, осталось только скомпилировать ее с функцией потерь и оптимизатором (пример 3.6). На роль функции потерь обычно выбирается либо средняя квадратическая ошибка (root mean squared error, RMSE), либо бинарная перекрестная энтропия между отдельными пикселями исходного изображения и его реконструкции.

Пример 3.6. Компиляция автокодировщика

```
# Компиляция автокодировщика  
autoencoder.compile(optimizer="adam", loss="binary_crossentropy")
```

ВЫБОР ФУНКЦИИ ПОТЕРЬ

При использовании оптимизации для RMSE сгенерированные выходные данные будут симметрично распределены вокруг средних значений пикселей, потому что завышение наказывается так же, как и занижение.

Бинарная перекрестная энтропия, напротив, асимметрична — она накладывает более жесткие штрафы на предсказания, сильно отличающиеся от истины, что способствует сдвигу предсказаний в середину диапазона. Например, если истинное значение пикселя велико (скажем, 0,7), то создание пикселя со значением 0,8 наказывается более строго, чем пикселя со значением 0,6. Если истинное значение пикселя низкое (скажем, 0,3), то создание пикселя со значением 0,2 наказывается более строго, чем пикселя со значением 0,4.

В результате изображения, сгенерированные с помощью бинарной перекрестной энтропии, получаются более размытыми, чем при использовании RMSE (поскольку она имеет тенденцию приближать прогнозы к 0,5), но иногда это желательно, так как RMSE может дать явно пикселизированные края.

Не существует правильного или неправильного выбора — попробуйте все варианты и выберите тот, который лучше всего подходит для вас.

Теперь обучим модель автокодировщика, передав исходные изображения на вход и выход (пример 3.7).

Пример 3.7. Обучение модели автокодировщика

```
autoencoder.fit(
    x_train,
    y = x_train,
    epochs=5,
    batch_size = 100,
    shuffle = True,
    validation_data=(x_test, x_test),
)
```

Обучив автокодировщик, проверим, способен ли он реконструировать входные изображения.

Реконструкция изображений

Для проверки способности реконструировать изображения можно пропустить изображения из контрольного набора через автокодировщик и сравнить результаты с исходными изображениями. Код реализации этого эксперимента показан в примере 3.8.

Пример 3.8. Реконструкция изображений с помощью автокодировщика

```
example_images = x_test[:5000]
predictions = autoencoder.predict(example_images)
```

На рис. 3.6 приведены несколько примеров исходных изображений (*верхний ряд*), двумерные векторы после кодирования и реконструированные изображения после декодирования (*нижний ряд*).

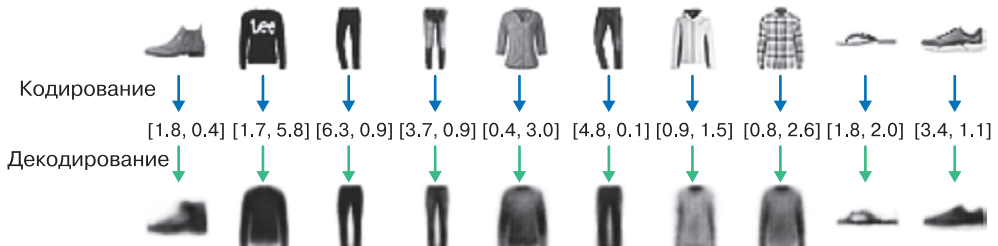


Рис. 3.6. Изображения предметов одежды до кодирования и после декодирования

Обратите внимание на то, что реконструированные изображения далеки от идеала — на исходных изображениях есть детали, которые не восстанавливаются

в процессе декодирования, например логотипы. Это связано с тем, что, сжимая каждое изображение всего до двух чисел, мы, естественно, теряем часть информации.

Теперь посмотрим, как кодировщик представляет изображения в скрытом пространстве.

Визуализация скрытого пространства

Чтобы увидеть, как изображения представляются в скрытом пространстве, можно передать контрольные изображения кодировщику и построить полученные векторные представления, как показано в примере 3.9.

Пример 3.9. Получение векторных представлений изображений с помощью кодировщика

```
embeddings = encoder.predict(example_images)

plt.figure(figsize=(8, 8))
plt.scatter(embeddings[:, 0], embeddings[:, 1], c="black", alpha=0.5, s=3)
plt.show()
```

Этот код выводит диаграмму рассеяния, показанную на рис. 3.2: каждая черная точка представляет изображение, отображенное в скрытое пространство.

Чтобы лучше понять, как структурировано скрытое пространство, можно использовать поставляемые с набором данных Fashion-MNIST метки, описывающие тип предмета на каждом изображении. Всего существует десять групп предметов (табл. 3.3).

Таблица 3.3. Метки в наборе Fashion-MNIST

ID	Метка
0	T-shirt/top (футболка)
1	Trouser (брюки)
2	Pullover (пуловер)
3	Dress (платье)
4	Coat (пальто)
5	Sandal (сандалии)
6	Shirt (рубашка)
7	Sneaker (кроссовки)
8	Bag (сумка)
9	Ankle boot (ботильоны)

Мы можем раскрасить каждую точку, присвоив каждой метке свой цвет, и получить диаграмму, показанную на рис. 3.7. Теперь структура становится более понятной! Несмотря на то что во время обучения модель не видела ни одной метки, автокодировщик естественным образом сгруппировал похожие предметы в близкие области скрытого пространства. Например, темно-синее облако точек в правом нижнем углу скрытого пространства — это разные изображения брюк, а красное облако ближе к центру — это все ботильоны.

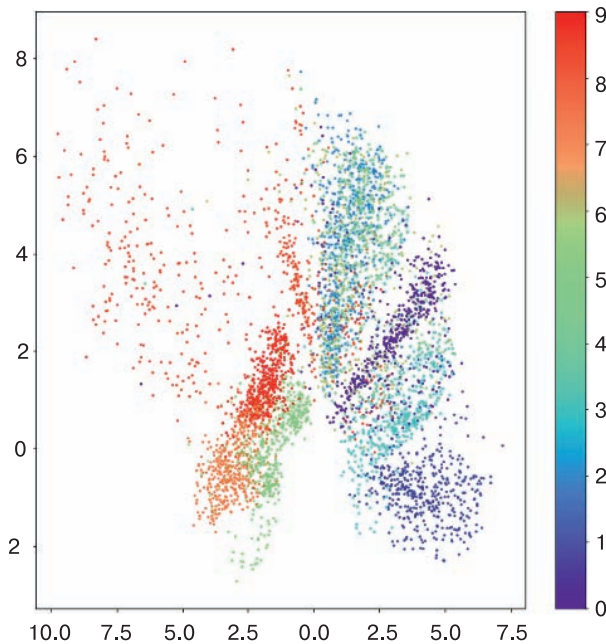


Рис. 3.7. Скрытое пространство, в котором точки, соответствующие разным предметам одежды, окрашены в разные цвета

Генерирование новых изображений

Мы можем сгенерировать новые изображения, выбирая точки в скрытом пространстве и используя декодировщик для их преобразования обратно в пространство пикселей, как показано в примере 3.10.

Пример 3.10. Генерирование новых изображений с помощью декодировщика

```
mins, maxs = np.min(embeddings, axis=0), np.max(embeddings, axis=0)
sample = np.random.uniform(mins, maxs, size=(18, 2))
reconstructions = decoder.predict(sample)
```

На рис. 3.8 показаны некоторые примеры сгенерированных изображений вместе с их представлениями в скрытом пространстве.

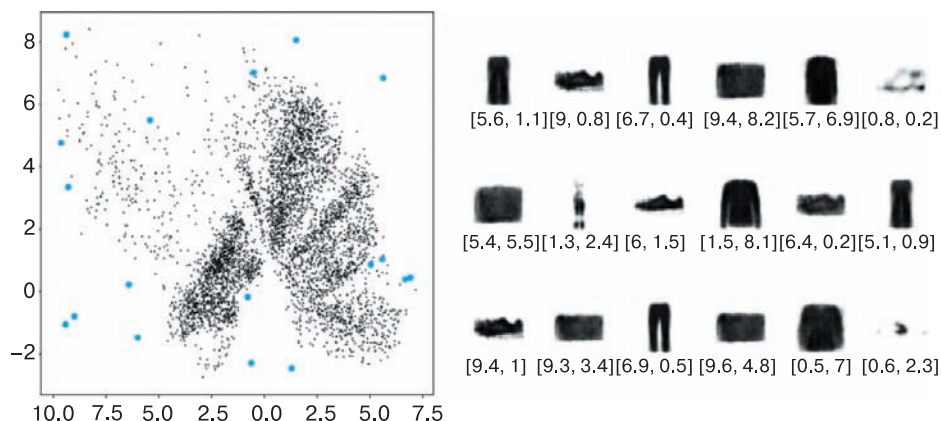


Рис. 3.8. Сгенерированные изображения предметов одежды

Каждая синяя точка соответствует одному из изображений, показанных справа на диаграмме вместе с векторным представлением под изображением. Обратите внимание на то, что некоторые из созданных элементов более реалистичны, чем другие. Почему так получилось?

Чтобы ответить на этот вопрос, отметим несколько наблюдений, касающихся общего распределения точек в скрытом пространстве (см. рис. 3.7).

- Одни предметы одежды представлены на очень маленькой площади, а другие — на более обширной.
- Распределение не симметрично относительно точки $(0, 0)$ и не ограничено. Например, точек с положительными значениями y гораздо больше, чем с отрицательными, а некоторые точки доходят даже до значения $y > 8$.
- Между точками разных цветов имеются почти пустые пространства, содержащие мало точек.

Эти наблюдения делают выборку из скрытого пространства весьма сложной задачей. Если наложить на скрытое пространство изображения декодированных точек в виде сетки (рис. 3.9), то станет понятно, почему декодировщик не всегда в состоянии сгенерировать более или менее удовлетворительные изображения.

Во-первых, мы видим, что если наугад выбирать точки в ограниченном пространстве, которое мы определили, то с большей вероятностью мы выберем

что-то, что декодируется как сумка (ID 8), а не как ботиллон (ID 9), потому что область скрытого пространства, занимаемого сумками (оранжевые точки), больше области ботиллонов (красные точки).

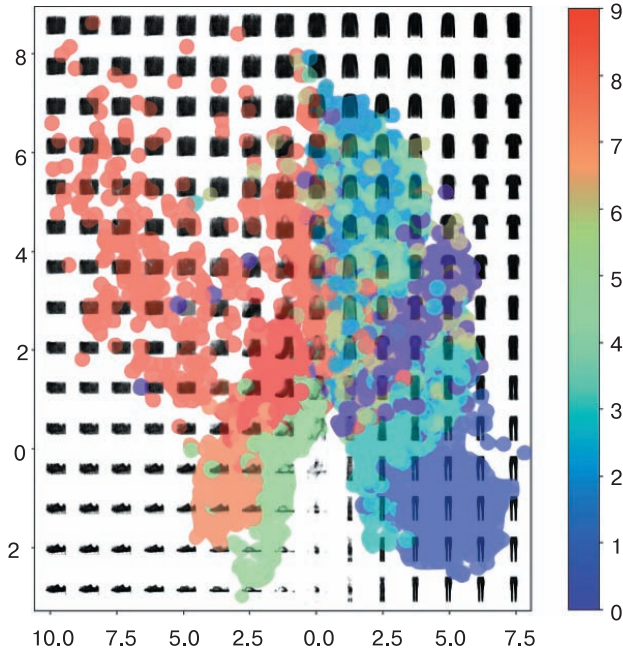


Рис. 3.9. Сетка с декодированными представлениями поверх скрытого пространства точек, полученных на основе изображений из оригинального набора данных и окрашенных в разные цвета по типам предметов одежды

Во-вторых, непонятно, как следует выбирать *случайную* точку в скрытом пространстве, поскольку распределение этих точек неизвестно. Технически мы вольны выбрать любую точку на плоскости, причем не гарантируется, что точки будут сосредоточены вокруг $(0, 0)$, что делает выбор представительного образца из скрытого пространства чрезвычайно проблематичной задачей.

И в-третьих, мы можем видеть в скрытом пространстве пустые области, в которые не попадает ни одно из исходных изображений. Например, по краям имеются обширные пустые пространства — у автокодировщика нет оснований гарантировать декодирование этих точек в узнаваемые предметы одежды, поскольку в эти области кодируется очень мало изображений.

Даже точки в центре могут не декодироваться в четкие изображения. Это связано с тем, что автокодировщик не обязан обеспечивать *непрерывность* пространства.

Например, точка $(-1, -1)$ может декодироваться во вполне удовлетворительное изображение сандалии, а уже точка $(-1, 1, -1, 1)$ может не давать столь же удовлетворительного ее изображения из-за отсутствия необходимых для этого механизмов.

При использовании двумерного скрытого пространства эта проблема мало заметна — автокодировщик ограничен небольшим количеством измерений и вынужден втискивать группы предметов одежды в узкие рамки. В результате пространство между этими группами оказывается довольно невелико. Но с увеличением числа измерений для создания более сложных изображений, в том числе таких, как лица, проблема становится более очевидной. Если дать автокодировщику полную свободу действий при использовании скрытого пространства для кодирования изображений, то мы получим огромные промежутки между группами похожих точек, не позволяющие получить правильно сформированные изображения.

Чтобы решить эти проблемы, следует преобразовать наш автокодировщик *вариационный автокодировщик*.

Вариационный автокодировщик

Начнем обсуждение с того, что вернемся к безразмерному шкафу и внесем несколько изменений...

СНОВА О БЕЗРАЗМЕРНОМ ПЛАТЯНОМ ШКАФЕ

Предположим, что теперь вместо размещения каждого предмета одежды в строго определенном месте в шкафу вы решили выделить для каждого типа предметов общую область, где они будут найдены с большей вероятностью. Вы полагаете, что такой менее строгий подход поможет решить проблему локальных разрывов.

Кроме того, чтобы, придерживаясь новой системы размещения, не допустить беспорядка, вы договариваетесь с Брайаном, что будете размещать центр области, выделенной для каждого типа одежды, как можно ближе к середине шкафа, а сами предметы постараетесь класть на расстоянии 1 м от центра области (не ближе и не дальше). Чем больше отклонение от этого расстояния, тем больше вам придется платить Брайану — вашему стилисту.

После нескольких месяцев следования этим двум новым правилам вы делаете шаг назад и восхищаетесь новой компоновкой шкафа, а также некоторыми примерами новых предметов одежды, которые создал Брайан. Так намного лучше! Создаваемые предметы очень разнообразны, и на этот раз нет образцов некачественной одежды. Кажется, эти два правила в корне изменили ситуацию!

Теперь попробуем разобраться, что нужно сделать с автокодировщиком, чтобы преобразовать его в вариационный автокодировщик и превратить в настоящую генеративную модель.

В общем-то, изменить нужно только две части: кодировщик и функцию потерь.

Кодировщик

В автокодировщике каждое изображение отображается непосредственно в точку в скрытом пространстве. В вариационном автокодировщике каждое изображение отображается в многомерное нормальное распределение вокруг точки в скрытом пространстве (рис. 3.10).

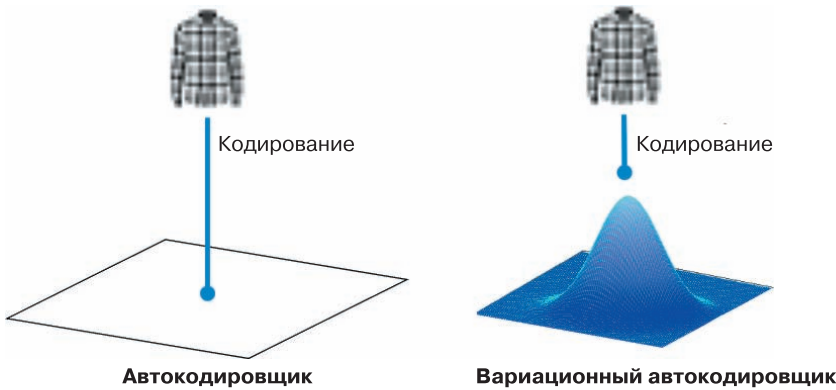


Рис. 3.10. Различия между кодировщиком в обычном и вариационном автокодировщике

НОРМАЛЬНОЕ РАСПРЕДЕЛЕНИЕ

Нормальное (или гауссово) распределение $\mathcal{N}(\mu, \sigma)$ — это распределение вероятностей, характеризующееся специфической колоколообразной формой кривой, определяемой двумя переменными: *средним* (μ) и *дисперсией* (σ^2). *Стандартное отклонение* (σ) — это квадратный корень из дисперсии.

Вот функция плотности вероятности одномерного нормального распределения:

$$f(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

На рис. 3.11 показано несколько одномерных нормальных распределений с разными значениями среднего и дисперсии. Красная кривая соответствует *стандартному* (или *единичному*) *нормальному распределению* $\mathcal{N}(0, 1)$ — нормальному распределению со средним значением 0 и дисперсией 1.

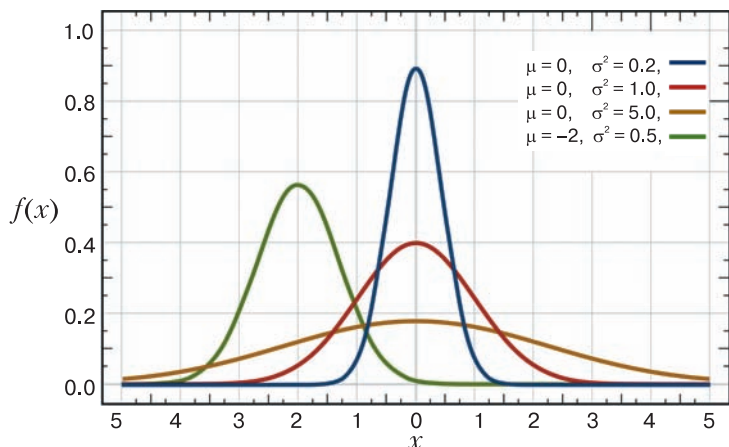


Рис. 3.11. Одномерное нормальное распределение
(источник: Википедия, <https://oreil.ly/gWwKV>)

Выбрать точку z из нормального распределения со средним μ и стандартным отклонением σ можно с помощью уравнения:

$$z = \mu + \sigma \varepsilon,$$

где ε — значение, выбранное из стандартного нормального распределения.

Нормальное распределение может быть не только одномерным — функция плотности вероятности для обобщенного *многомерного нормального распределения* (или *многомерного гауссова распределения*) $\mathcal{N}(\mu, \Sigma)$ в k -мерном пространстве с вектором средних μ и ковариационной матрицей Σ выглядит так:

$$f(x_1, \dots, x_k) = \frac{\exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right)}{\sqrt{(2\pi)^k |\Sigma|}}.$$

В книге мы будем использовать в основном *изотропные* многомерные нормальные распределения, для которых ковариационная матрица является диагональной. Это означает, что распределение независимо в каждом измерении (то есть мы можем выбрать вектор, в котором каждый элемент распределяется с независимым средним значением и дисперсией). Такое же изотропное многомерное нормальное распределение мы будем использовать в нашем вариационном автокодировщике.

Многомерное стандартное нормальное распределение $\mathcal{N}(0, I)$ — это многомерное распределение с вектором средних, равных нулю, и единичной ковариационной матрицей.



Нормальное или гауссово?

В этой книге термины «нормальное» и «гауссово» используются как взаимозаменяемые и обычно подразумевают изотропный и многомерный характер распределения. Например, фразу «мы выбираем из гауссова распределения» можно интерпретировать как «мы выбираем из изотропного многомерного гауссова распределения».

Кодировщик должен лишь отобразить каждый вход в вектор средних и вектор дисперсии, не беспокоясь о ковариации между измерениями. Вариационные автокодировщики предполагают, что между измерениями в скрытом пространстве нет корреляции.

Часто также выбирается отображение в *логарифм* дисперсии, потому что он может принимать любые действительные значения в диапазоне $(-\infty, \infty)$, что соответствует естественному диапазону выходных значений узла нейронной сети, тогда как значения дисперсии всегда положительны. Таким образом, мы можем использовать нейронную сеть в качестве кодировщика для преобразования входного изображения в векторы средних и логарифмов дисперсий.

В итоге кодировщик будет преобразовывать каждое входное изображение в два вектора, средних и логарифмов дисперсии, которые вместе определяют многомерное нормальное распределение в скрытом пространстве:

- `z_mean` — средняя точка распределения;
- `z_log_var` — логарифм дисперсии каждого измерения.

Выбрать точку z из распределения, определяемого этими значениями, можно с помощью следующего уравнения:

```
z = z_mean + z_sigma * epsilon
```

где

```
z_sigma = exp(z_log_var * 0.5)
epsilon ~ N(0, I)
```



Вот как связаны между собой `z_sigma` (σ) и `z_log_var` ($\log(\sigma^2)$):

$$\sigma = \exp(\log(\sigma)) = \exp(2 \log(\sigma) / 2) = \exp(\log(\sigma^2) / 2).$$

Декодировщик вариационного автокодировщика идентичен декодировщику простого автокодировщика, что дает общую архитектуру, показанную на рис. 3.12.

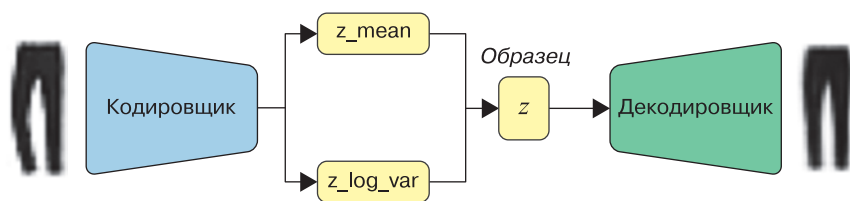


Рис. 3.12. Архитектура вариационного автокодировщика (VAE)

Почему такое небольшое изменение в кодировщике дает положительный эффект?

Прежде к непрерывному пространству не предъявлялось требования непрерывности — даже если точка $(-2, 2)$ декодировалась в правильно сформированное изображение сандалии, это не означало, что изображение для точки $(-2, 1, 2, 1)$ должно быть похожим. Теперь же, поскольку мы выбираем случайную точку из области вокруг z_mean , декодировщик должен декодировать все соседние точки в очень похожие изображения, то есть потери при реконструкции должны оставаться небольшими. Это свойство гарантирует, что при выборе точки в скрытом пространстве, которую декодировщик прежде никогда не видел, она с большой вероятностью будет декодирована в хорошо сформированное изображение.

Построение кодировщика VAE

Посмотрим, как сконструировать эту версию кодировщика в Keras.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/03_vae/02_vae_fashion/vae_fashion.ipynb` в репозитории книги. Он был заимствован (и адаптирован) из отличного руководства по VAE (<https://oreil.ly/A7yqJ>), написанного Франсуа Шолле (Francois Chollet) и доступного на сайте Keras.

Сначала создадим новый тип слоя `Sampling`, который позволит делать выборку из распределения, определяемого переменными `z_mean` и `z_log_var`, как показано в примере 3.11.

Пример 3.11. Слой `Sampling`

```
class Sampling(layers.Layer): ❶
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = K.random_normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon ❷
```

- ❶ Новый класс слоев создается путем наследования базового класса `Layer` из Keras (см. врезку «Наследование класса `Layer`»).
- ❷ Используется трюк с перепараметризацией (см. врезку «Трюк с перепараметризацией»), чтобы создать выборку из нормального распределения, параметризованного значениями `z_mean` и `z_log_var`.

НАСЛЕДОВАНИЕ КЛАССА `LAYER`

Библиотека Keras позволяет создавать новые классы слоев, предлагая для этого абстрактный класс `Layer`. Вам нужно лишь создать свой подкласс этого класса и определить метод `call`, описывающий преобразование тензора слоем.

Например, для нужд вариационного автокодировщика можно создать слой `Sampling`, способный выбирать образцы `z` из нормального распределения с параметрами `z_mean` и `z_log_var`. Этот прием может пригодиться, когда требуется применить к тензору преобразование, не включенное ни в один из готовых типов слоев в Keras.

ТРЮК С ПЕРЕПАРАМЕТРИЗАЦИЕЙ

Вместо выборки непосредственно из нормального распределения с параметрами `z_mean` и `z_log_var` можно выбрать `epsilon` из стандартного нормального распределения, а затем вручную скорректировать результат, чтобы получить правильное среднее значение и дисперсию.

Этот прием известен как *трюк с перепараметризацией*. Он важен, потому что означает, что градиенты могут свободно распространяться обратно по слою. Сохраняя всю случайность слоя, содержащуюся в переменной `epsilon`, можно показать, что частная производная выхода слоя по отношению к его входу является детерминированной, то есть независимой от случайного значения `epsilon`, что важно для обратного распространения ошибки через слой.

Полный код кодировщика, включая слой `Sampling`, показан в примере 3.12.

Пример 3.12. Кодировщик

```
encoder_input = layers.Input(
    shape=(32, 32, 1), name="encoder_input"
)
x = layers.Conv2D(32, (3, 3), strides=2, activation="relu", padding="same")(
    encoder_input
)
x = layers.Conv2D(64, (3, 3), strides=2, activation="relu", padding="same")(x)
```

```

x = layers.Conv2D(128, (3, 3), strides=2, activation="relu", padding="same")(x)
shape_before_flattening = K.int_shape(x)[1:]

x = layers.Flatten()(x)
z_mean = layers.Dense(2, name="z_mean")(x) ❶
z_log_var = layers.Dense(2, name="z_log_var")(x)
z = Sampling()([z_mean, z_log_var]) ❷

encoder = models.Model(encoder_input, [z_mean, z_log_var, z], name="encoder") ❸

```

❶ Вместо прямого подключения слоя `Flatten` к скрытому двумерному пространству мы подключаем его к слоям `z_mean` и `z_log_var`.

❷ Слой `Sampling` выбирает точку `z` в скрытом пространстве из нормального распределения, определяемого параметрами `z_mean` и `z_log_var`.

❸ Модель Keras, определяющая кодировщик, — модель, которая принимает входное изображение и выводит `z_mean`, `z_log_var` и образец `z`, выбранный из нормального распределения, определяемого этими параметрами.

Сводная информация о кодировщике приведена в табл. 3.4.

Таблица 3.4. Сводная информация о кодировщике VAE

Слой (тип)	Выходная форма	Число параметров	Связан с
Input (input)	(None, 32, 32, 1)	0	[]
Conv2D (conv2d_1)	(None, 16, 16, 32)	320	[input]
Conv2D (conv2d_2)	(None, 8, 8, 64)	18 496	[conv2d_1]
Conv2D (conv2d_3)	(None, 4, 4, 128)	73 856	[conv2d_2]
Flatten (flatten)	(None, 2048)	0	[conv2d_3]
Dense (z_mean)	(None, 2)	4098	[flatten]
Dense (z_log_var)	(None, 2)	4098	[flatten]
Sampling (z)	(None, 2)	0	[z_mean, z_log_var]

Всего параметров	100 868
Обучаемых параметров	100 868
Необучаемых параметров	0

Единственное, что еще мы должны изменить, — это функция потерь.

Функция потерь

Прежняя наша функция потерь оценивала только среднюю квадратическую ошибку между изображениями и их реконструкциями после прохождения через кодировщик и декодировщик. Эта потеря при реконструкции имеется также в вариационном автокодировщике, но нам необходим дополнительный компонент — *расстояние Кульбака — Лейблера* (Kullback — Leibler divergence).

Расстояние Кульбака — Лейблера измеряет, насколько одно распределение вероятности отличается от другого. В вариационном автокодировщике требуется знать, насколько наше нормальное распределение с параметрами `z_mean` и `z_log_var` отличается от стандартного нормального распределения. Для данного случая расстояние Кульбака — Лейблера имеет замкнутую форму:

```
kl_loss = -0.5 * sum(1 + z_log_var - z_mean ^ 2 - exp(z_log_var))
```

или в математической записи:

$$D_{KL}[N(\mu, \sigma) \| N(0, 1)] = -\frac{1}{2} \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2).$$

Сумма вычисляется по всем измерениям в скрытом пространстве. `kl_loss` минимизируется до 0, когда для всех измерений `z_mean = 0` и `z_log_var = 0`. Когда эти два члена начинают отличаться от 0, `kl_loss` увеличивается.

Таким образом, член с расстоянием Кульбака — Лейблера наказывает сеть за представление наблюдений в виде значений `z_mean` и `z_log_var`, значительно отличающихся от параметров стандартного нормального распределения, а именно `z_mean = 0` и `z_log_var = 0`.

Чем же помогает такая добавка в функцию потерь?

Во-первых, имеется четко определенное распределение, пригодное для выбора точек в скрытом пространстве, — стандартное нормальное распределение. Во-вторых, поскольку этот член пытается привести все закодированные распределения к стандартному нормальному распределению, уменьшается вероятность возникновения больших пустых пространств между кластерами точек. Вместо этого кодировщик попытается использовать пространство вокруг источника максимально симметрично и эффективно.

В исходной статье с описанием VAE функция потерь была представлена простым сложением потерь на реконструкцию и потерь на расстояние KL. Наш вариант (β -VAE) включает коэффициент, который взвешивает расстояние KL, чтобы гарантировать его сбалансированность с потерями при реконструкции.

Если потери на реконструкцию получат слишком большой вес, то потери KL не будут иметь желаемого регулирующего эффекта и мы увидим те же проблемы, с которыми столкнулись при использовании простого автокодировщика. Если слишком большой вес получит член с расстоянием KL, то потери KL будут доминировать и восстановленные изображения станут получаться некачественными. Этот весовой коэффициент является одним из параметров, которые необходимо настроить при обучении VAE.

Обучение вариационного автокодировщика

В примере 3.13 показано, как сконструировать модель VAE целиком в виде подкласса абстрактного класса `Model`. Такой подход позволяет включить вычисление расстояния KL в функции потерь в собственный метод `train_step`.

Пример 3.13. Обучение VAE

```
class VAE(models.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super(VAE, self).__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.total_loss_tracker = metrics.Mean(name="total_loss")
        self.reconstruction_loss_tracker = metrics.Mean(
            name="reconstruction_loss"
        )
        self.kl_loss_tracker = metrics.Mean(name="kl_loss")

    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.reconstruction_loss_tracker,
            self.kl_loss_tracker,
        ]

    def call(self, inputs): ❶
        z_mean, z_log_var, z = encoder(inputs)
        reconstruction = decoder(z)
        return z_mean, z_log_var, reconstruction

    def train_step(self, data): ❷
        with tf.GradientTape() as tape:
            z_mean, z_log_var, reconstruction = self(data)
            reconstruction_loss = tf.reduce_mean(
                500
                * losses.binary_crossentropy(
                    data, reconstruction, axis=(1, 2, 3)
                )
            ) ❸
```

```

        kl_loss = tf.reduce_mean(
            tf.reduce_sum(
                -0.5
                * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var)),
                axis = 1,
            )
        )
        total_loss = reconstruction_loss + kl_loss ❹

    grads = tape.gradient(total_loss, self.trainable_weights)
    self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

    self.total_loss_tracker.update_state(total_loss)
    self.reconstruction_loss_tracker.update_state(reconstruction_loss)
    self.kl_loss_tracker.update_state(kl_loss)

    return {m.name: m.result() for m in self.metrics}

vae = VAE(encoder, decoder)
vae.compile(optimizer="adam")
vae.fit(
    train,
    epochs=5,
    batch_size=100
)

```

- ❶ Эта функция описывает, что нужно сделать при передаче вариационному автокодировщику конкретного входного изображения.
- ❷ Эта функция описывает один шаг обучения VAE, включая вычисление функции потерь.
- ❸ Для потерь при реконструкции используется значение `beta`, равное 500.
- ❹ Общая величина потерь вычисляется как сумма потерь на реконструкцию и потерь KL.



Градиентная лента

Градиентная лента (gradient tape) в TensorFlow — это механизм, позволяющий вычислять градиенты операций, выполняемых в модели во время прямого прохода. Чтобы использовать его, необходимо заключить код, выполняющий операции, которые нужно дифференцировать, в контекст `tf.GradientTape()`. После выполнения операций вы сможете вычислить градиент функции потерь по отношению к некоторым переменным, вызвав `tape.gradient()`. Затем градиенты можно применять для обновления переменных с помощью оптимизатора. Этот механизм может пригодиться для расчета градиента пользовательских функций потерь (как сделано здесь), а также для определения своих циклов обучения, как мы увидим в главе 4.

Анализ вариационного автокодировщика

Теперь, обучив VAE, мы можем применять кодировщик для кодирования изображений из контрольного набора и построения значений z_mean в скрытом пространстве. Также можем сделать выборку из стандартного нормального распределения, чтобы сгенерировать точки в скрытом пространстве, и использовать декодировщик для декодирования этих точек обратно в пространство пикселей, чтобы увидеть, как работает VAE.

Структура нового скрытого пространства, некоторые выбранные точки и соответствующие им декодированные изображения показаны на рис. 3.13. Сразу можно заметить несколько изменений в организации скрытого пространства.

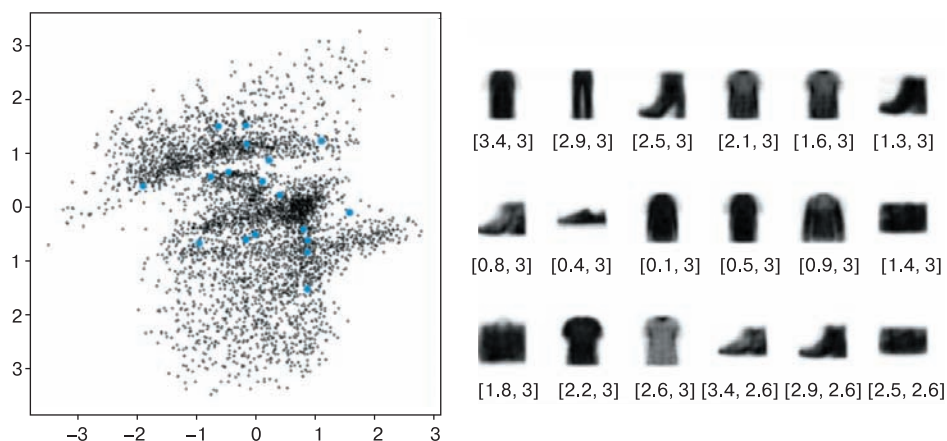


Рис. 3.13. Новое скрытое пространство: черные точки соответствуют значениям z_mean закодированных изображений, а синие — точкам, выбранным из скрытого пространства (соответствующим декодированным изображениям справа)

Во-первых, член, определяющий расстояние Кульбака — Лейблера, гарантирует, что значения z_mean и z_log_var не будут сильно отклоняться от стандартного нормального распределения. Во-вторых, у нас получилось не так уж много изображений, сформированных неправильно, поскольку скрытое пространство локально непрерывно из-за того, что кодировщик теперь стохастический, а не детерминированный.

Наконец, окрасив точки в скрытом пространстве в разные цвета для разных типов предметов одежды (рис. 3.14), можно увидеть, что никакие точки не пользуются никакими предпочтениями. График справа показывает пространство,

преобразованное в p -значения, и, как видим, все цвета представлены примерно одинаково, причем метки во время обучения не применялись — VAE самостоятельно изучил разные формы одежды, стремясь минимизировать потери при реконструкции.

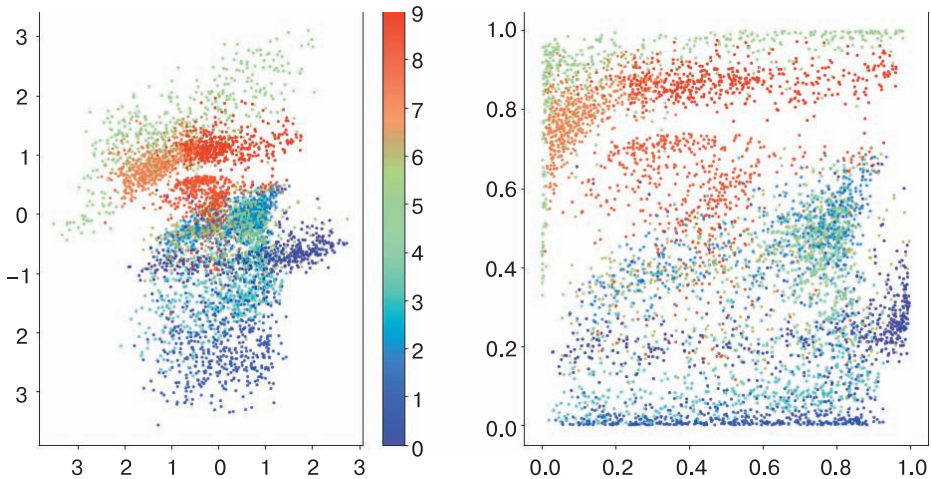


Рис. 3.14. Скрытое пространство вариационного автокодировщика, в котором точки, соответствующие разным типам предметов одежды, окрашены в разные цвета

Исследование скрытого пространства

До сих пор, работая над обычным и вариационным автокодировщиками, мы ограничивались двумерным скрытым пространством. Это помогло наглядно показать внутреннюю работу VAE и понять, почему небольшие изменения в архитектуре автокодировщика помогли преобразовать его в более мощный класс сети, пригодной для генеративного моделирования.

Теперь обратим внимание на более сложный набор данных и посмотрим, каких удивительных результатов могут достичь вариационные автокодировщики при увеличении размерности скрытого пространства.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/03_vae/03_faces/vae_faces.ipynb` в репозитории книги.

Набор данных CelebA

В этом разделе мы используем набор данных CelebFaces Attributes (CelebA, <https://oreil.ly/tEUUh>) для обучения следующего вариационного автокодировщика. Эта коллекция содержит более 200 000 цветных изображений лиц знаменитостей, снабженных различными подписями, например: «в шляпе», «с улыбкой» и т. д. (рис. 3.15). Конечно, нам не нужны метки для обучения VAE, но они пригодятся потом, когда мы начнем исследовать, как эти признаки фиксируются в многомерном скрытом пространстве. После обучения VAE мы сможем выбирать точки из скрытого пространства и генерировать новые изображения лиц знаменитостей.



Рис. 3.15. Примеры изображений из набора CelebA (источник [Liu et al., 2015])

Набор данных CelebA имеется на сайте Kaggle, откуда вы сможете загрузить его, запустив сценарий загрузки, включенный в репозиторий книги, как показано в примере 3.14. Изображения и сопутствующие метаданные будут сохранены в локальной папке `/data`.

Пример 3.14. Загрузка набора данных CelebA

```
bash scripts/download_kaggle_data.sh jessicali9530 celeba-dataset
```

Для создания набора данных TensorFlow, ссылающегося на каталог, где хранятся изображения, мы будем использовать функцию `image_dataset_from_directory` из библиотеки Keras, как показано в примере 3.15. Такое решение позволит читать пакеты изображений в память только тогда, когда это необходимо (например, во время обучения), и мы сможем работать с большими наборами данных, не беспокоясь о нехватке памяти. Эта функция также изменяет размеры изображений до 64×64 , осуществляя интерполяцию между значениями пикселей.

Пример 3.15. Предварительная обработка набора данных CelebA

```
train_data = utils.image_dataset_from_directory(
    "/app/data/celeba-dataset/img_align_celeba/img_align_celeba",
    labels=None,
    color_mode="rgb",
    image_size=(64, 64),
    batch_size=128,
    shuffle=True,
    seed=42,
    interpolation="bilinear",
)
```

Исходные данные, содержащие значения в диапазоне $[0, 255]$ и обозначающие интенсивность окраски пикселей, масштабируются в диапазон $[0, 1]$, как показано в примере 3.16.

Пример 3.16. Предварительная обработка набора данных CelebA

```
def preprocess(img):
    img = tf.cast(img, "float32") / 255.0
    return img

train = train_data.map(lambda x: preprocess(x))
```

Обучение вариационного автокодировщика

Архитектура сети для моделирования лиц похожа на архитектуру сети из примера моделирования изображений Fashion-MNIST, есть лишь несколько небольших отличий.

- Данные теперь имеют три входных канала (RGB) вместо одного (оттенки серого). То есть нужно изменить количество каналов на 3 в конечном слое обратной свертки декодера.
- Мы будем использовать скрытое пространство с двумя сотнями измерений вместо двух. Поскольку изображения лиц намного сложнее изображений в Fashion-MNIST, увеличим размерность скрытого пространства, чтобы сеть смогла закодировать минимально необходимое количество деталей.

- После каждого сверточного слоя добавлены слои пакетной нормализации для ускорения обучения. Несмотря на то что обработка каждого пакета занимает больше времени, для достижения той же величины потери требуется намного меньше пакетов.
- Весовой коэффициент β при расстоянии KL увеличен до 2000. Этот параметр требует настройки, но для нашего набора данных и нашей архитектуры это значение дает хорошие результаты.

Архитектуры кодировщика и декодировщика показаны в табл. 3.5 и 3.6 соответственно.

Таблица 3.5. Сводная информация о кодировщике VAE для изображений лиц

Слой (тип)	Выходная форма	Число параметров	Связан с
Input (input)	(None, 32, 32, 3)	0	[]
Conv2D (conv2d_1)	(None, 16, 16, 128)	3584	[input]
BatchNormalization (bn_1)	(None, 16, 16, 128)	18 496	[conv2d_1]
LeakyReLU (lr_1)	(None, 16, 16, 128)	0	[bn_1]
Conv2D (conv2d_2)	(None, 8, 8, 128)	147 584	[lr_1]
BatchNormalization (bn_2)	(None, 8, 8, 128)	512	[conv2d_2]
LeakyReLU (lr_2)	(None, 8, 8, 128)	0	[bn_2]
Conv2D (conv2d_3)	(None, 4, 4, 128)	147 584	[lr_2]
BatchNormalization (bn_3)	(None, 4, 4, 128)	512	[conv2d_3]
LeakyReLU (lr_3)	(None, 4, 4, 128)	0	[bn_3]
Conv2D (conv2d_4)	(None, 2, 2, 128)	147 584	[lr_3]
BatchNormalization (bn_4)	(None, 2, 2, 128)	512	[conv2d_4]
LeakyReLU (lr_4)	(None, 2, 2, 128)	0	[bn_4]
Flatten (flatten)	(None, 512)	0	[lr_4]
Dense (z_mean)	(None, 200)	102 600	[flatten]
Dense (z_log_var)	(None, 200)	102 600	[flatten]
Sampling (z)	(None, 200)	0	[z_mean, z_log_var]

Всего параметров 653 584

Обучаемых параметров 652 560

Необучаемых параметров 1024

Таблица 3.6. Сводная информация о декодировщике VAE для изображений лиц

Слой (тип)	Выходная форма	Число параметров
Input	(None, 200)	0
Dense	(None, 512)	102 912
BatchNormalization	(None, 512)	2048
LeakyReLU	(None, 512)	0
Reshape	(None, 2, 2, 128)	0
Conv2DTranspose	(None, 4, 4, 128)	147 584
BatchNormalization	(None, 4, 4, 128)	512
LeakyReLU	(None, 4, 4, 128)	0
Conv2DTranspose	(None, 8, 8, 128)	147 584
BatchNormalization	(None, 8, 8, 128)	512
LeakyReLU	(None, 8, 8, 128)	0
Conv2DTranspose	(None, 16, 16, 128)	147 584
BatchNormalization	(None, 16, 16, 128)	512
LeakyReLU	(None, 16, 16, 128)	0
Conv2DTranspose	(None, 32, 32, 128)	147 584
BatchNormalization	(None, 32, 32, 128)	512
LeakyReLU	(None, 32, 32, 128)	0
Conv2DTranspose	(None, 32, 32, 3)	3459

Всего параметров 700 803

Обучаемых параметров 698 755

Необучаемых параметров 2048

После пяти эпох обучения VAE должен быть готов генерировать новые изображения лиц знаменитостей!

Анализ вариационного автокодировщика

Для начала рассмотрим примеры реконструированных лиц. В верхнем ряду на рис. 3.16 показаны исходные изображения, а в нижнем ряду — их реконструкции, полученные после прохождения через кодировщик и декодировщик.

Примеры исходных изображений лиц



Реконструкции



Рис. 3.16. Реконструированные лица, полученные после прохождения через кодировщик и декодировщик

VAE благополучно уловил ключевые черты каждого лица — угол наклона головы, причёску, выражение лица и т. д. Некоторые мелкие детали отсутствуют, но не забывайте, что цель создания вариационного автокодировщика состоит не в том, чтобы добиться идеальной точности при реконструкции. Наша конечная цель — создание новых лиц путем выборки случайных точек из скрытого пространства. Для этого требуется гарантировать близость распределения точек в скрытом пространстве к многомерному стандартному нормальному распределению. Если обнаружатся какие-либо измерения, значительно отличающиеся от стандартного нормального распределения, то, вероятно, следует уменьшить весовой коэффициент потерь при реконструкции, поскольку член, определяющий расстояние Кульбака — Лейблера, влияет недостаточно.

На рис. 3.17 показаны первые 50 измерений в скрытом пространстве. Как видите, все распределения мало отличаются от стандартного нормального, поэтому можно перейти к созданию некоторых лиц.

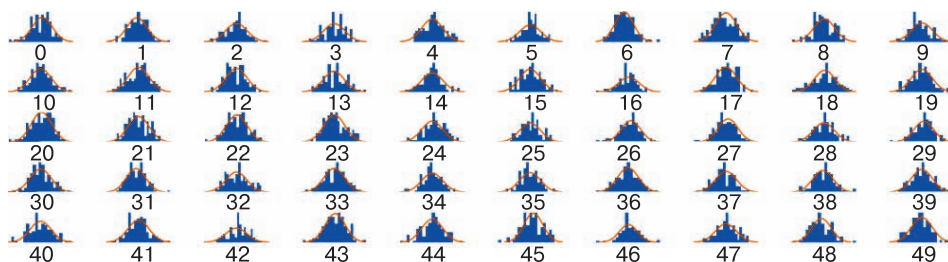


Рис. 3.17. Распределение точек для первых 50 измерений скрытого пространства

Генерирование новых лиц

Сгенерировать новые лица можно с помощью кода, представленного в примере 3.17.

Пример 3.17. Генерирование новых лиц по точкам в скрытом пространстве

```
grid_width, grid_height = (10,3)
z_sample = np.random.normal(size=(grid_width * grid_height, 200)) ❶

reconstructions = decoder.predict(z_sample) ❷

fig = plt.figure(figsize=(18, 5))
fig.subplots_adjust(hspace=0.4, wspace=0.4)
for i in range(grid_width * grid_height):
    ax = fig.add_subplot(grid_height, grid_width, i+1)
    ax.axis("off")
    ax.imshow(reconstructions[i, :, :, :]) ❸
```

❶ Выбираем 30 точек из 200-мерного стандартного нормального распределения.

❷ Декодируем их.

❸ Выводим получившиеся изображения!

Результат показан на рис. 3.18.

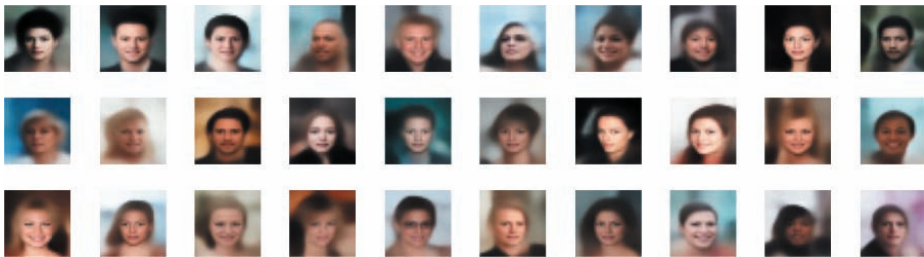


Рис. 3.18. Новые сгенерированные лица

Удивительно, но VAE смог преобразовать каждую выбранную точку в довольно реалистичное изображение лица человека. Это наша первая демонстрация истинной мощи генеративных моделей!

Теперь посмотрим, сможем ли мы использовать скрытое пространство для выполнения некоторых интересных операций над сгенерированными изображениями.

Арифметика скрытого пространства

Одним из преимуществ отображения изображений в пространство меньшего размера является возможность выполнения арифметических операций с векторами в скрытом пространстве, результаты которых можно преобразовать в визуальные аналоги путем декодирования в пространство исходных изображений. Предположим, что мы решили преобразовать изображение грустного

лица в улыбающееся. Для этого сначала нужно найти вектор в скрытом пространстве, который указывает в направлении увеличения признака «улыбка». Прибавив этот вектор к закодированному исходному изображению в скрытом пространстве, мы получим новую точку, которая при декодировании должна дать улыбающуюся версию исходного изображения.

Но как найти вектор, соответствующий признаку «улыбка»? Каждое изображение в наборе данных CelebA отмечено атрибутами, один из которых — «с улыбкой». Если взять среднюю позицию закодированных изображений в скрытом пространстве с атрибутом «с улыбкой» и вычесть среднюю позицию закодированных изображений без него, то мы получим вектор, который указывает в направлении увеличения признака «улыбка».

Фактически мы выполним следующую арифметическую операцию с векторами в скрытом пространстве, где α — коэффициент, определяющий степень увеличения или уменьшения признака, соответствующего вектору:

$$z_{\text{new}} = z + \alpha * \text{feature_vector}$$

Посмотрим, как это работает. На рис. 3.19 показаны несколько изображений, закодированных в скрытое пространство. Мы добавляем или вычитаем вектор (например, улыбка, блондинка, мужчина, очки), умноженный на коэффициент, чтобы получить разные версии изображения, в которых меняется только требуемый признак.



Рис. 3.19. Добавление признаков к изображениям лиц и их вычитание

Весьма примечательно, что даже при значительном перемещении точки в скрытом пространстве основная часть изображения практически не меняется, кроме признака, которым мы пытаемся манипулировать. Это демонстрирует мощную способность вариационных автокодировщиков выявлять и корректировать высокоуровневые признаки в изображениях.

Преобразование одного лица в другое

Эту же идею можно использовать и для преобразования одного лица в другое. Представьте две точки в скрытом пространстве, A и B, соответствующие двум изображениям. Если пройти из точки A в точку B по прямой, декодируя каждую точку линии по мере движения, то можно увидеть постепенное превращение начального лица в конечное.

Математически эту прямую линию можно описать следующим уравнением:

$$z_{\text{new}} = z_A * (1 - \alpha) + z_B * \alpha$$

Здесь α — это число между 0 и 1, определяющее расстояние от начальной точки A.

На рис. 3.20 показан этот процесс в действии. Мы берем два изображения, кодируем их в скрытое пространство, а затем декодируем точки, лежащие на прямой линии между ними через равные промежутки.

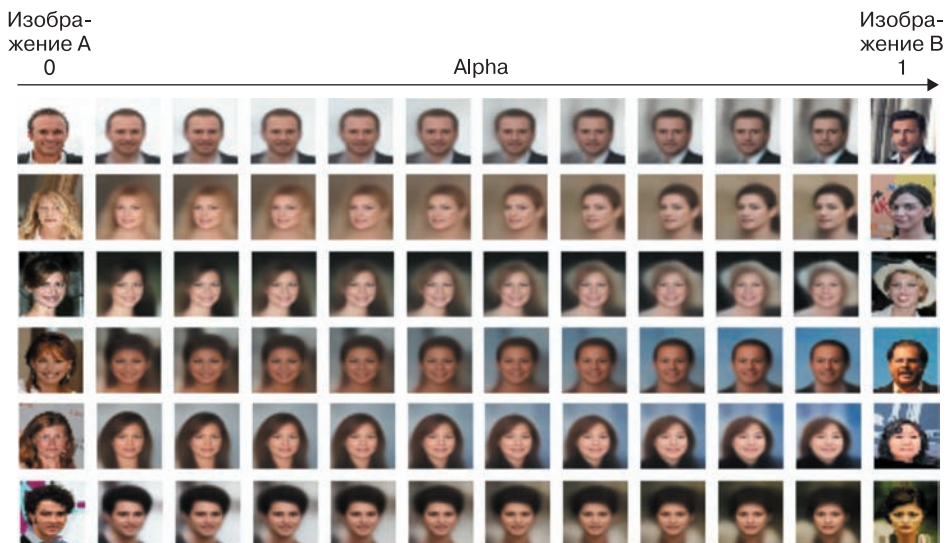


Рис. 3.20. Преобразование одного лица в другое

Обратите внимание на то, насколько плавным выглядит переход даже при изменении нескольких признаков одновременно: удалении очков, изменении цвета волос, пола и др. Такая плавность еще раз доказывает, что скрытое пространство вариационного автокодировщика действительно непрерывно, его можно обойти и исследовать, создав множество разных человеческих лиц.

Резюме

В этой главе вы узнали, что вариационные автокодировщики являются мощным инструментом в арсенале генеративного моделирования. Сначала мы посмотрели, как можно использовать простые автокодировщики для отображения многомерных изображений в скрытое пространство с небольшим числом измерений, чтобы из отдельных неинформативных пикселей извлечь высокоуровневые признаки. Однако быстро выяснили, что простые автокодировщики из-за своих недостатков малоприспособлены на роль генеративной модели — выбор представительных точек из полученного скрытого пространства по ряду причин сопряжен с некоторыми проблемами.

Вариационные автокодировщики решают эти проблемы, вводя в модель элемент случайности и ограничивая распределение точек в скрытом пространстве. Внеся небольшие изменения, обычный автокодировщик можно преобразовать в вариационный, что позволяет использовать его в качестве генеративной модели.

Наконец, мы применили новую технику для генерации лиц и увидели, что для получения новых лиц можно просто выбирать точки из стандартного нормального распределения. Более того, выполняя арифметические операции с векторами в скрытом пространстве, можно достичь удивительных эффектов, таких как трансформация лица и манипулирование признаками.

В следующей главе мы рассмотрим тип генеративной модели, привлекающей еще большее внимание, — генеративно-состязательную сеть.

Генеративно-сопоставительные сети

В этой главе вы:

- познакомитесь с архитектурой генеративно-сопоставительной сети (GAN);
- создадите и обучите глубокую сверточную GAN (DCGAN) с помощью Keras;
- используете DCGAN для создания новых изображений;
- изучите некоторые распространенные проблемы, возникающие при обучении DCGAN;
- узнаете, как архитектура Wasserstein GAN (WGAN) решает эти проблемы;
- познакомитесь с дополнительными способами улучшения WGAN, такими как включение в функцию потерь со штрафом за градиент (GP);
- создадите WGAN-GP с нуля с помощью Keras;
- используете WGAN-GP для создания изображений лиц;
- узнаете, как условная GAN (CGAN) позволяет обуславливать выходные данные, генерируемые для заданной метки;
- создадите и обучите CGAN с помощью Keras и используете ее для манипуляций со сгенерированными изображениями.

В 2014 году на конференции Neural Information Processing Systems (NIPS) в Монреале Ян Гудфеллоу (Ian Goodfellow) с коллегами представили статью *Generative Adversarial Networks* [Goodfellow et al., 2014]. Введение в генеративно-сопоставительные сети теперь рассматривается как один из поворотных моментов в развитии генеративного моделирования, поскольку основные идеи, представленные в этой статье, привели к появлению множества успешных и впечатляющих генеративных моделей.

В этой главе вы сначала познакомитесь с теоретической основой GAN. Затем узнаете, как с помощью библиотеки Keras создавать генеративно-сопоставительные сети.

Введение

Начнем с короткой истории, иллюстрирующей некоторые фундаментальные концепции, используемые в процессе обучения GAN.

СТРОИТЕЛЬНЫЕ БЛОКИ BRICKKI И ФАЛЬСИФИКАТОРЫ

Это ваш первый день на новой работе в роли руководителя отдела контроля качества в компании Brickki, специализирующейся на производстве высококачественных строительных блоков всех форм и размеров (рис. 4.1).



Рис. 4.1. Производственная линия по изготовлению строительных блоков разных форм и размеров (создано с помощью Midjourney, <https://midjourney.com/>)

Вас сразу предупредили о проблеме с некоторыми изделиями, сходящими с конвейера. Конкурент начал изготавливать поддельные копии строительных блоков Brickki и нашел способ подменять ими блоки, отправляемые вашей компанией покупателям. Вы решаете научиться отличать поддельные блоки от настоящих, чтобы отбраковывать их еще до того, как они будут упакованы и отправлены покупателям. Со временем, прислушиваясь к отзывам клиентов, вы все увереннее начинаете отличать подделки.

Мошенникам это не нравится — они реагируют на обретенные вами навыки, внося некоторые изменения в процесс подделки, чтобы вам было сложнее заметить разницу между настоящими и поддельными блоками. Но вы не сдаётесь и продолжаете учиться распознавать более сложные подделки, стараясь быть на шаг впереди фальсификаторов. Этот процесс не останавливается ни на минуту: фальсификаторы продолжают совершенствовать свои технологии создания блоков, а вы продолжаете учиться выявлять их подделки.

С каждой неделей вам все труднее отличить настоящие блоки Brickki от поддельных. Кажется, что этой простой игры в догонялки достаточно для того, чтобы добиться значительного улучшения как качества подделки, так и качества ее выявления.

История о компании Brickki и фальсификаторах описывает процесс обучения генеративно-состязательной сети.

Генеративно-состязательная сеть — это битва двух противоборствующих сторон — *генератора* и *дискриминатора*. Генератор пытается преобразовать случайный шум в наблюдения, которые выглядят так, будто были выбраны из исходного набора данных, а дискриминатор старается определить, исходит ли наблюдение из оригинального набора данных или является одной из подделок генератора (рис. 4.2).

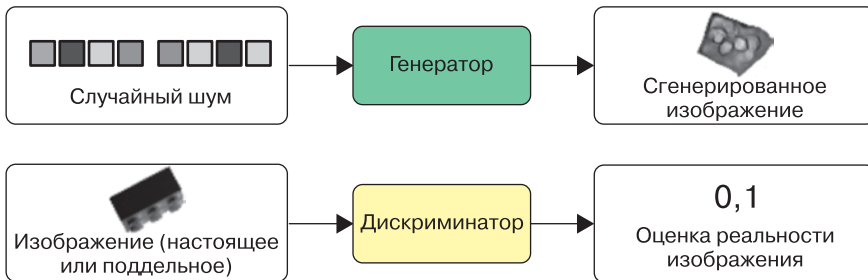


Рис. 4.2. Входы и выходы двух подсетей в генеративно-состязательной сети

В начале процесса генератор выводит искаженные изображения, а дискриминатор оценивает их случайным образом. Главная особенность генеративно-состязательной сети — попеременное обучение двух подсетей: генератор постепенно совершенствуется в обмане дискриминатора, а тот, адаптируясь, учится правильно определять фальшивые наблюдения, что заставляет генератор искать новые способы обмана дискриминатора и т. д.

Глубокая сверточная GAN

Чтобы увидеть процесс в действии, построим с помощью Keras свою первую генеративно-состязательную сеть, формирующую фотографии строительных блоков.

Мы будем как можно точнее выполнять рекомендации, приведенные в одной из первых крупных статей по GAN *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks* [Radford et al., 2015]. Авторы этой статьи, опубликованной в 2015 году, показали, как построить глубокую сверточную GAN (DCGAN) для генерации реалистичных изображений из различных наборов данных. Они также предложили несколько изменений, значительно улучшающих качество создаваемых изображений.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/04_gan/01_dcgan/dcgan.ipynb` в репозитории книги.

Набор данных Bricks

Прежде всего вам нужно загрузить обучающие данные. Мы будем использовать набор данных Images of LEGO Bricks (<https://oreil.ly/3vp9f>), доступный на сайте Kaggle. Это коллекция 40 000 фотографий 50 различных игрушечных кубиков, снятых с разных ракурсов. Некоторые примеры изображений продуктов Brickki показаны на рис. 4.3.



Рис. 4.3. Примеры изображений из набора данных Bricks

Чтобы загрузить набор данных, можно воспользоваться сценарием загрузки с сайта Kaggle, доступным в репозитории книги, как показано в примере 4.1. Изображения и сопутствующие метаданные будут сохранены в локальной папке `/data`.

Пример 4.1. Загрузка набора данных Bricks

```
bash scripts/download_kaggle_data.sh joosthazelzet lego-brick-images
```

Для создания набора данных TensorFlow, ссылающегося на каталог, где хранятся изображения, мы будем использовать функцию `image_dataset_from_directory` из библиотеки Keras, как показано в примере 4.2. Такое решение позволит читать пакеты изображений в память только тогда, когда это будет необходимо (например, во время обучения), и мы сможем работать с большими наборами данных, не беспокоясь о нехватке памяти. Эта функция также изменяет размеры изображений до 64×64 , осуществляя интерполяцию между значениями пикселей.

Пример 4.2. Создание набора данных TensorFlow из файлов изображений в каталоге

```
train_data = utils.image_dataset_from_directory(
    "/app/data/lego-brick-images/dataset/",
    labels=None,
    color_mode="grayscale",
    image_size=(64, 64),
    batch_size=128,
    shuffle=True,
    seed=42,
    interpolation="bilinear",
)
```

Исходные данные содержат значения в диапазоне $[0, 255]$ и обозначают интенсивность окраски пикселей. Перед обучением GAN мы масштабируем данные в диапазон $[-1, 1]$, чтобы иметь возможность использовать функцию активации `tanh` в последнем слое генератора, которая дает более сильные градиенты, чем функция `sigmoid` (пример 4.3).

Пример 4.3. Предварительная обработка набора данных Bricks

```
def preprocess(img):
    img = (tf.cast(img, "float32") - 127.5) / 127.5
    return img

train = train_data.map(lambda x: preprocess(x))
```

Давайте посмотрим, как сконструировать дискриминатор.

Дискриминатор

Цель дискриминатора — определить реальность изображения. Это задача классификации изображений с учителем, поэтому мы можем использовать ту же сетевую архитектуру, что и в главе 2, — последовательность сверточных слоев, за которыми следует выходной полносвязный слой.

Полная архитектура создаваемого дискриминатора показана в табл. 4.1.

Таблица 4.1. Сводная информация о дискриминаторе

Слой (тип)	Выходная форма	Число параметров
Input	(None, 64, 64, 1)	0
Conv2D	(None, 32, 32, 64)	1024
LeakyReLU	(None, 32, 32, 64)	0
Dropout	(None, 32, 32, 64)	0
Conv2D	(None, 16, 16, 128)	131 072
BatchNormalization	(None, 16, 16, 128)	512
LeakyReLU	(None, 16, 16, 128)	0
Dropout	(None, 16, 16, 128)	0
Conv2D	(None, 8, 8, 256)	524 288
BatchNormalization	(None, 8, 8, 256)	1024
LeakyReLU	(None, 8, 8, 256)	0
Dropout	(None, 8, 8, 256)	0
Conv2D	(None, 4, 4, 512)	2 097 152
BatchNormalization	(None, 4, 4, 512)	2048
LeakyReLU	(None, 4, 4, 512)	0
Dropout	(None, 4, 4, 512)	0
Conv2D	(None, 1, 1, 1)	8192
Flatten	(None, 1)	0

Всего параметров 2 765 312

Обучаемых параметров 2 763 520

Необучаемых параметров 1792

В примере 4.4 представлен код, использующий Keras и конструирующий дискриминатор.

Пример 4.4. Дискриминатор

```
discriminator_input = layers.Input(shape=(64, 64, 1)) ❶
x = layers.Conv2D(64, kernel_size=4, strides=2, padding="same", use_bias = False)(
    discriminator_input
) ❷
```

```

x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    128, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    256, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    512, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    1,
    kernel_size=4,
    strides=1,
    padding="valid",
    use_bias = False,
    activation = 'sigmoid'
)(x)
discriminator_output = layers.Flatten()(x) ❸
discriminator = models.Model(discriminator_input, discriminator_output) ❹

```

❶ Определение входного слоя Input дискриминатора (для приема изображений).

❷ Набор сверточных слоев Conv2D, следующих друг за другом и перемежаемых слоями BatchNormalization, LeakyReLU и Dropout.

❸ Преобразование выхода последнего сверточного слоя в вектор. К этому моменту тензор имеет форму $1 \times 1 \times 1$, поэтому слой Dense нам не нужен.

❹ Модель Keras, определяющая дискриминатор, — модель, которая принимает исходное изображение и выводит единственное число между 0 и 1.

Отмечу, что в некоторых сверточных слоях используется шаг величиной 2, позволяющий уменьшить размер тензора при прохождении через сеть (с 64 до 32, 16, 8, 4 и, наконец, 1), при этом мы увеличиваем количество каналов (от 1 в исходном черно-белом изображении до 64, затем 128, 256 и, наконец, 512) перед свертыванием до одного прогноза.

Функция активации sigmoid в последнем слое Conv2D обеспечивает масштабирование выходного значения в диапазон от 0 до 1.

В примере 4.5 представлен код, конструирующий генератор.

Пример 4.5. Генератор

```
generator_input = layers.Input(shape=(100,)) ❶
x = layers.Reshape((1, 1, 100))(generator_input) ❷
x = layers.Conv2DTranspose(
    512, kernel_size=4, strides=1, padding="valid", use_bias = False
)(x) ❸
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    256, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    128, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    64, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
generator_output = layers.Conv2DTranspose(
    1,
    kernel_size=4,
    strides=2,
    padding="same",
    use_bias = False,
    activation = 'tanh'
)(x) ❹
generator = models.Model(generator_input, generator_output) ❺
```

- ❶ Определение входного слоя Input генератора — вектор длиной 100.
- ❷ За ним следует слой Reshape, возвращающий тензор $1 \times 1 \times 100$, чтобы можно было начать применять операции обратной свертки.
- ❸ Полученный тензор пропускается через четыре слоя Conv2DTranspose, между которыми расположены слои BatchNormalization и LeakyReLU.
- ❹ Последний слой Conv2DTranspose использует функцию активации tanh, чтобы преобразовать выходные значения в диапазон $[-1, 1]$ в соответствии с областью исходных изображений.
- ❺ Модель Keras, определяющая генератор, — модель, которая принимает вектор длиной 100 и возвращает тензор с формой [64, 64, 1].

Обратите внимание на то, что в некоторых слоях Conv2DTranspose используется шаг величиной 2, позволяющий увеличить размер тензора при прохождении

через сеть (с 1 до 4, 8, 16, 32 и, наконец, 64). При этом мы уменьшаем количество каналов (с 512 до 256, 128, 64 и, наконец, 1), чтобы привести его в соответствие с числом каналов в черно-белом изображении на выходе.

УВЕЛИЧЕНИЕ РАЗРЕШЕНИЯ И CONV2DTRANSPOSE

Альтернативой использованию слоев *Conv2DTranspose* является применение слоя *UpSampling2D* со следующим за ним обычным слоем *Conv2D* с шагом 1, как показано в примере 4.6.

Пример 4.6. Пример увеличения разрешения

```
x = layers.UpSampling2D(size = 2)(x)
x = layers.Conv2D(256, kernel_size=4, strides=1, padding="same")(x)
```

Слой *UpSampling2D* просто дублирует каждую строку и каждый столбец во входных данных. За ним следует обычный сверточный слой *Conv2D* с размером шага 1, выполняющий операцию свертки. Суть та же, что и при обратной свертке, но вместо заполнения нулями промежутков между пикселями операция увеличения разрешения просто повторяет значения существующих пикселей.

По опыту, метод *Conv2DTranspose* может привести к появлению в выходном изображении *артефактов* или эффекта шахматной доски (рис. 4.4), снижающих качество результата. Тем не менее эти слои все еще используются во многих весьма впечатляющих генеративно-сопоставительных сетях, описываемых в литературе, и доказали, что являются мощным инструментом в арсенале практиков глубокого обучения.



Рис. 4.4. Артефакты, появляющиеся при использовании слоев *Conv2DTranspose* (источник: [Odena et al., 2016], <https://distill.pub/2016/deconv-checkerboard>)

Оба метода — `Upsampling2D + Conv2D` и `Conv2DTranspose` — с успехом можно использовать для преобразования точки из скрытого пространства обратно в область исходных изображений. Это как раз тот случай, когда желательно попробовать оба метода и посмотреть, какой позволяет получить лучшие результаты.

Обучение DCGAN

Генератор и дискриминатор в DCGAN имеют очень простую архитектуру, не сильно отличающуюся от архитектуры моделей VAE, которые мы рассматривали в главе 3. Ключом к пониманию GAN является понимание процесса обучения генератора и дискриминатора.

Для обучения дискриминатора нужно создать обучающий набор, в котором часть изображений являются случайно выбранными *реальными* наблюдениями, а часть — результатом работы генератора. Для реальных изображений ответ должен быть равным 1, а для сгенерированных — 0. Эту задачу следует рассматривать как задачу обучения с учителем, где реальные изображения имеют метки 1, а поддельные — 0 и используется функция потерь бинарной перекрестной энтропии.

Как обучить генератор? Нам нужен способ оценки каждого сгенерированного изображения, чтобы его можно было применить для получения изображений с высокой оценкой. К счастью, у нас есть дискриминатор, который делает именно это! Мы можем сгенерировать пакет изображений, пропустить их через дискриминатор и получить оценку для каждого изображения. В таком случае на роль функции потерь для генератора прекрасно подойдет бинарная перекрестная энтропия между этими вероятностями и вектором единиц, потому что наша цель — обучить генератор создавать изображения, которые дискриминатор считает реальными.

Очень важно чередовать обучение этих двух сетей, следя за тем, чтобы за раз обновлялись веса только одной из них. Например, в цикле обучения генератора обновляются только веса генератора. Если позволить параллельно изменяться весам дискриминатора, то он просто обучится с большой вероятностью предсказывать реальность сгенерированных изображений, а это не то, что нам нужно. Мы хотим, чтобы сгенерированные изображения получали оценку, близкую к 1 (которая присваивается реальным изображениям), потому что генератор достиг определенного совершенства, а не потому, что дискриминатор плохо отличает реальные изображения от сгенерированных.

Схема процесса обучения дискриминатора и генератора представлена на рис. 4.5.

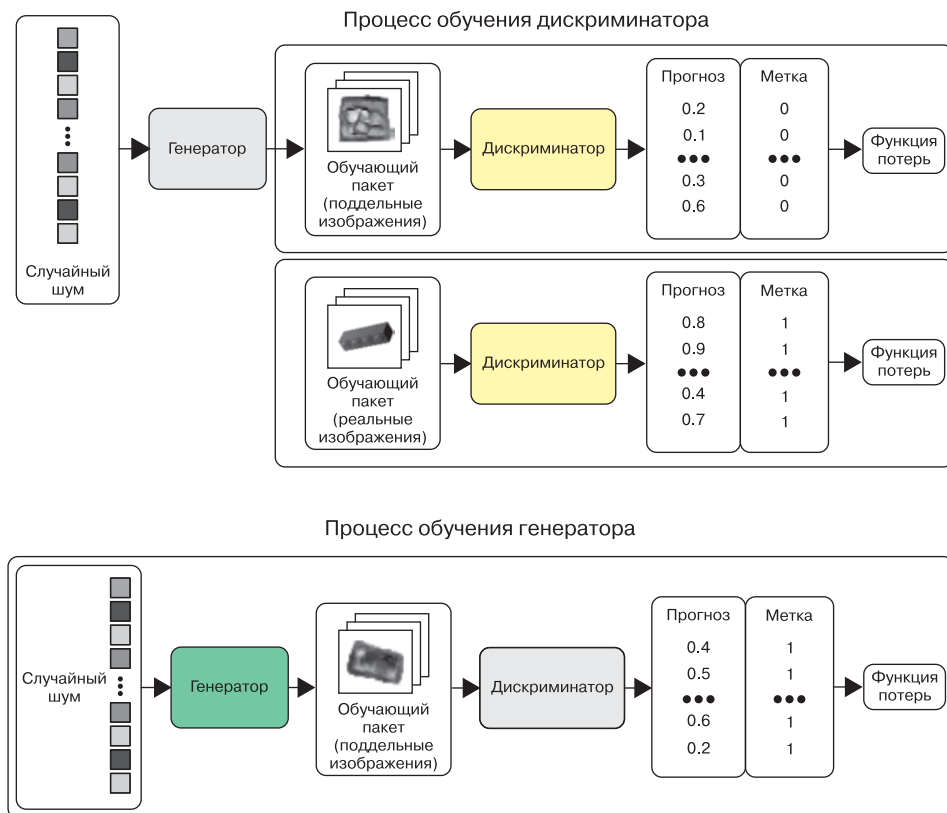


Рис. 4.5. Обучение DCGAN — серыми прямоугольниками обозначены сети, веса которых фиксируются на том или ином этапе обучения

Keras предоставляет возможность определить свою функцию `train_step` для реализации этой логики. В примере 4.7 показан полный класс DCGAN модели.

Пример 4.7. Компиляция DCGAN

```
class DCGAN(models.Model):
    def __init__(self, discriminator, generator, latent_dim):
        super(DCGAN, self).__init__()
        self.discriminator = discriminator
        self.generator = generator
        self.latent_dim = latent_dim
```

```

def compile(self, d_optimizer, g_optimizer):
    super(DCGAN, self).compile()
    self.loss_fn = losses.BinaryCrossentropy() ❶
    self.d_optimizer = d_optimizer
    self.g_optimizer = g_optimizer
    self.d_loss_metric = metrics.Mean(name="d_loss")
    self.g_loss_metric = metrics.Mean(name="g_loss")

@property
def metrics(self):
    return [self.d_loss_metric, self.g_loss_metric]

def train_step(self, real_images):
    batch_size = tf.shape(real_images)[0]
    random_latent_vectors = tf.random.normal(
        shape=(batch_size, self.latent_dim)
    ) ❷

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = self.generator(
            random_latent_vectors, training = True
        ) ❸
        real_predictions = self.discriminator(real_images, training = True) ❹
        fake_predictions = self.discriminator(
            generated_images, training = True
        ) ❺

        real_labels = tf.ones_like(real_predictions)
        real_noisy_labels = real_labels + 0.1 * tf.random.uniform(
            tf.shape(real_predictions)
        )
        fake_labels = tf.zeros_like(fake_predictions)
        fake_noisy_labels = fake_labels - 0.1 * tf.random.uniform(
            tf.shape(fake_predictions)
        )

        d_real_loss = self.loss_fn(real_noisy_labels, real_predictions)
        d_fake_loss = self.loss_fn(fake_noisy_labels, fake_predictions)
        d_loss = (d_real_loss + d_fake_loss) / 2.0 ❻

        g_loss = self.loss_fn(real_labels, fake_predictions) ❼

    gradients_of_discriminator = disc_tape.gradient(
        d_loss, self.discriminator.trainable_variables
    )
    gradients_of_generator = gen_tape.gradient(
        g_loss, self.generator.trainable_variables
    )

```

```

        self.d_optimizer.apply_gradients(
            zip(gradients_of_discriminator, discriminator.trainable_variables)
        )
        self.g_optimizer.apply_gradients(
            zip(gradients_of_generator, generator.trainable_variables)
        )

        self.d_loss_metric.update_state(d_loss)
        self.g_loss_metric.update_state(g_loss)

    return {m.name: m.result() for m in self.metrics}

dcgan = DCGAN(
    discriminator=discriminator, generator=generator, latent_dim=100
)

dcgan.compile(
    d_optimizer=optimizers.Adam(
        learning_rate=0.0002, beta_1 = 0.5, beta_2 = 0.999
    ),
    g_optimizer=optimizers.Adam(
        learning_rate=0.0002, beta_1 = 0.5, beta_2 = 0.999
    ),
)

dcgan.fit(train, epochs=300)

```

- ❶ Функция потерь для генератора и дискриминатора — `BinaryCrossentropy`.
- ❷ Для обучения сети сначала выбирается пакет векторов из многомерного стандартного нормального распределения.
- ❸ Далее этот пакет пропускается через генератор, создающий пакет сгенерированных изображений.
- ❹ После этого дискриминатору предлагается предсказать реальность пакета реальных изображений...
- ❺ ...И пакета сгенерированных изображений.
- ❻ Величина потерь дискриминатора определяется как среднее бинарной перекрестной энтропии для реальных (с меткой 1) и поддельных (с меткой 0) изображений.
- ❼ Величина потерь генератора определяется как бинарная перекрестная энтропия между предсказаниями дискриминатора для сгенерированных изображений и меткой 1.

Обновление весов дискриминатора и генератора по отдельности.

Дискриминатор и генератор постоянно борются друг с другом, что может сделать ход обучения DCGAN нестабильным. В идеале процесс обучения достигнет устойчивого баланса, что позволит генератору получить значимую информацию от дискриминатора и начать улучшать качество изображений. После достаточного количества эпох дискриминатор имеет все шансы занять доминирующее положение (рис. 4.6), но к этому моменту генератор может научиться создавать изображения довольно высокого качества.

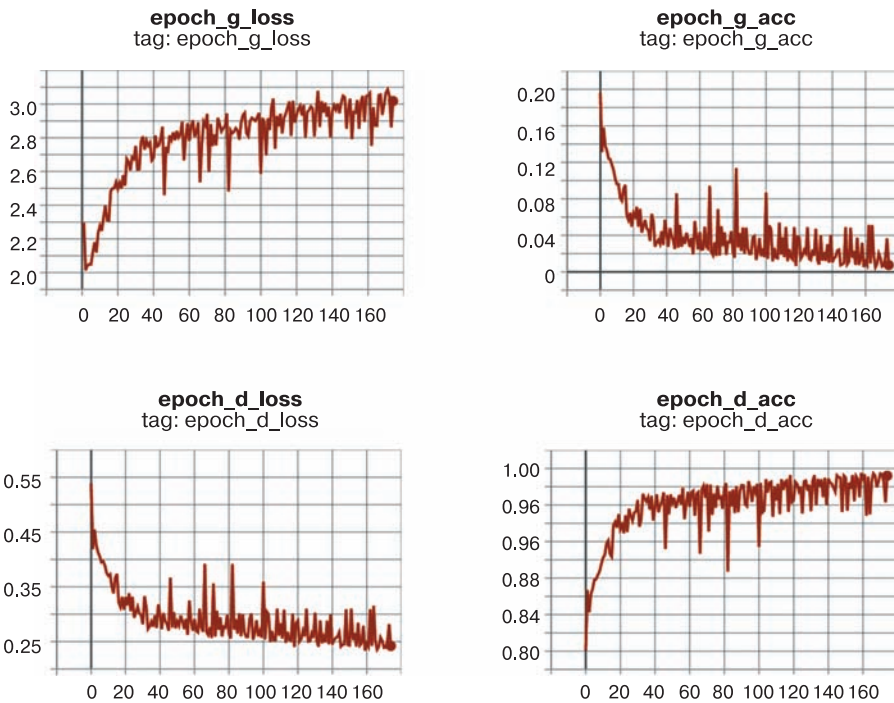


Рис. 4.6. Изменение потери точности дискриминатора и генератора в ходе обучения



Добавление шума к меткам

Полезный трюк при обучении GAN — добавить в обучающие метки небольшое количество случайного шума. Это помогает повысить стабильность процесса обучения и четкость генерируемых изображений. Такое искажение меток действует как средство укрощения дискриминатора, усложняющее ему задачу и избавляющее генератор от перегрузки.

Анализ DCGAN

Наблюдая за изображениями, созданными генератором в разные эпохи обучения (рис. 4.7), легко заметить, что генератор постепенно набирается опыта в создании изображений, похожих на изображения из обучающего набора.

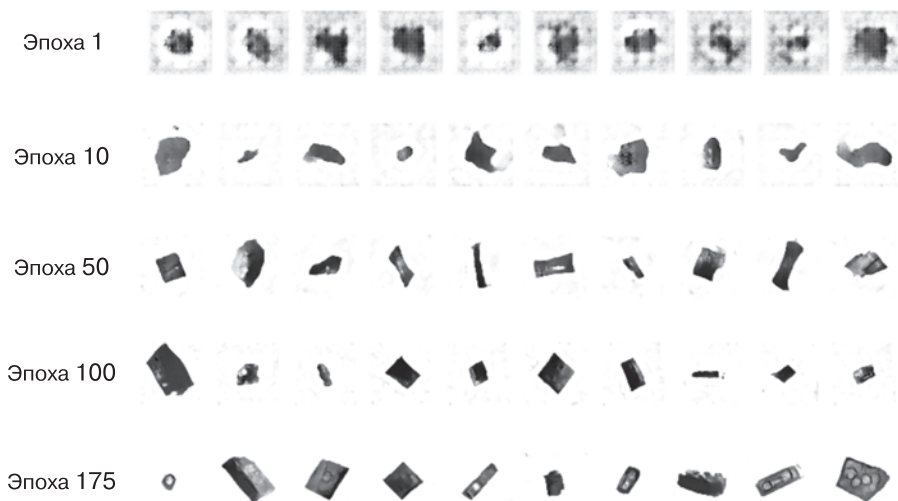


Рис. 4.7. Изображения, созданные генератором в разные эпохи обучения

Немного удивительно то, что нейронная сеть способна преобразовать случайный шум во что-то значимое. Мы не передали модели никаких дополнительных признаков, кроме простых пикселей, поэтому она должна самостоятельно выявить высокоуровневые признаки, такие как тени, кубические формы или окружности.

Еще одно требование к успешной генеративной модели — она не должна воспроизводить изображения из обучающего набора. Чтобы убедиться, что наша модель соблюдает его, можно найти изображение из обучающего набора, наиболее близкое к конкретному сгенерированному примеру, и определить расстояние между ними. В качестве меры расстояния можно использовать *расстояние L1*, определяемое так:

```
def compare_images(img1, img2):  
    return np.mean(np.abs(img1 - img2))
```

На рис. 4.8 показаны самые близкие друг к другу сгенерированные и обучающие изображения. Как видите, несмотря на определенное сходство, они не идентичны. Это доказывает, что генератор выявил соответствующие высокоуровневые признаки и смог сгенерировать примеры, отличные от тех, которые он уже видел.

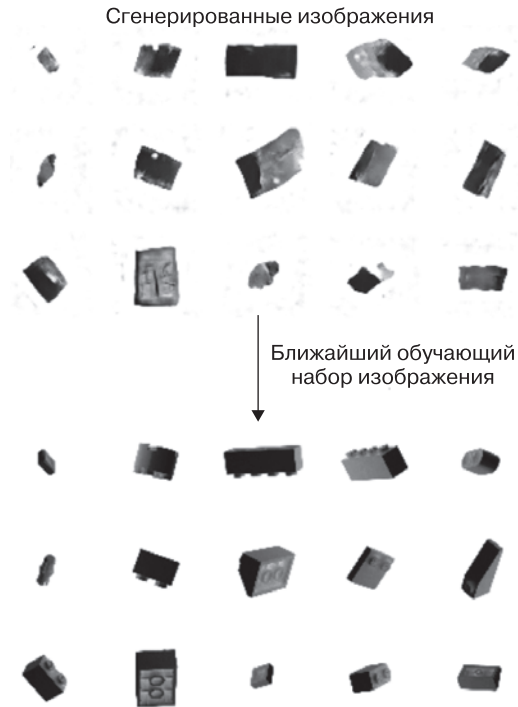


Рис. 4.8. Самые близкие друг к другу сгенерированные и обучающие изображения

Обучение GAN: советы и рекомендации

Появление генеративно-сопоставительных сетей стало крупным шагом в развитии генеративного моделирования, однако обучать их очень сложно. Рассмотрим наиболее типичные проблемы, возникающие при обучении GAN, и потенциальные решения, а затем исследуем некоторые изменения в архитектуре GAN, помогающие устранить многие из этих проблем.

Дискриминатор получает подавляющее преимущество перед генератором

Если дискриминатор становится слишком мощным, то сигнал функции потерь делается слишком слабым, чтобы обеспечить значимые улучшения в генераторе. В худшем случае дискриминатор с успехом обучается отличать реальные изображения от поддельных, и градиенты полностью исчезают, что приводит к остановке обучения генератора (рис. 4.9).

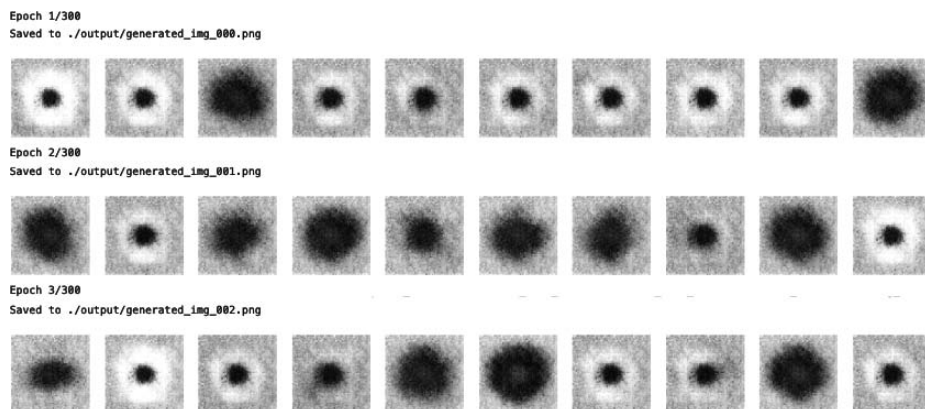


Рис. 4.9. Пример вывода, когда дискриминатор обретает подавляющее преимущество перед генератором

Обнаружив коллапс функции потерь дискриминатора, постарайтесь найти способы его ослабления, например:

- увеличьте параметр `rate` слоев `Dropout` в дискриминаторе, чтобы уменьшить объем информации, проходящей через сеть;
- уменьшите скорость обучения дискриминатора;
- уменьшите количество сверточных фильтров в дискриминаторе;
- обучая дискриминатор, добавьте шум к меткам;
- при обучении дискриминатора случайным образом изменяйте метки некоторых изображений на противоположные.

Генератор получает подавляющее преимущество перед дискриминатором

Если дискриминатор окажется недостаточно мощным, генератор найдет небольшое количество образцов, обманывающих его. Это состояние известно как *коллапс формы*.

Предположим, что мы обучаем генератор на нескольких пакетах, не обновляя дискриминатор между ними. В этом случае генератор будет стремиться найти единственное наблюдение, также известное как *форма* (*mode*), которое всегда обманывает дискриминатор и начинает отображать каждую точку в скрытом пространстве в это наблюдение. В результате градиент функции потерь падает почти до 0, и модель оказывается неспособной выйти из этого состояния.

Даже если затем попытаться возобновить обучение дискриминатора, чтобы прекратить его одурачивание этой единственной точкой, то генератор просто найдет другую форму, обманывающую дискриминатор, поскольку он уже стал нечувствителен к его вкладу и, следовательно, не имеет стимулов для увеличения разнообразия своих результатов.

Эффект коллапса формы показан на рис. 4.10.

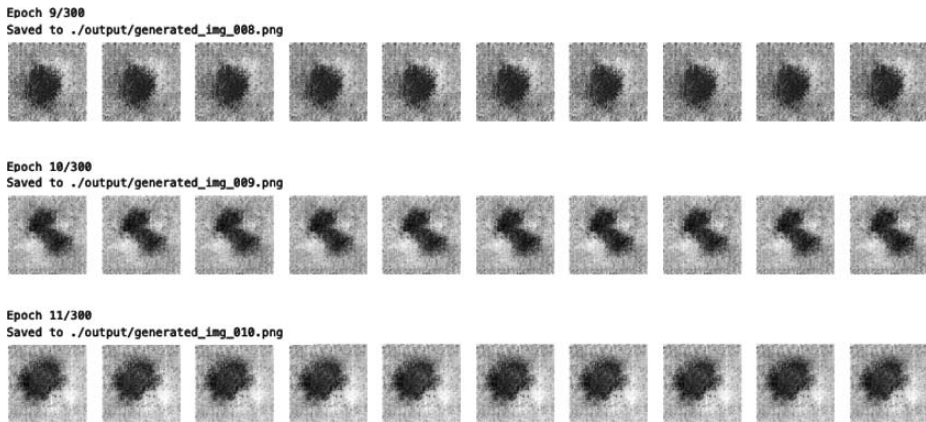


Рис. 4.10. Пример коллапса формы, когда генератор обретает подавляющее преимущество перед дискриминатором

Если вы обнаружите, что ваш генератор страдает от коллапса формы, то попробуйте усилить дискриминатор, используя меры, противоположные перечисленным в предыдущем разделе. Также можно попытаться снизить скорость обучения обеих сетей и увеличить размер пакета.

Неинформативные потери

Поскольку целью модели глубокого обучения является минимизация функции потерь, естественно полагать, что чем меньшее значение дает функция потерь генератора, тем выше качество получаемых им изображений. Но потери генератора оцениваются по действующему дискриминатору, который постоянно совершенствуется, поэтому нельзя сравнивать потери, полученные в разные моменты в ходе обучения. Действительно, потери генератора на рис. 4.6 со временем увеличиваются, хотя качество изображений явно улучшается. Отсутствие корреляции между потерями генератора и качеством изображений иногда затрудняет контроль над процессом обучения GAN.

Гиперпараметры

Даже в простых генеративно-сопоставительных сетях имеется большое количество гиперпараметров, доступных для настройки. Наряду с общей архитектурой дискриминатора и генератора необходимо учитывать параметры, управляющие пакетной нормализацией, прореживанием, скоростью обучения, слоями активации, сверточными фильтрами, размером ядра, величиной шага, размером пакета и размером скрытого пространства. Генеративно-сопоставительные сети очень чувствительны даже к самым незначительным изменениям этих параметров, и нередко поиск подходящего их набора выполняется методом проб и ошибок, а не в соответствии с установленным набором рекомендаций.

Вот почему так важно понимать особенности работы генеративно-сопоставительных сетей и знать, как интерпретировать функцию потерь, — чтобы осмысленно корректировать гиперпараметры, которые могут улучшить стабильность модели.

Решение проблем генеративно-сопоставительных сетей

За последние годы было предложено несколько ключевых улучшений, позволивших значительно повысить общую стабильность моделей GAN и уменьшить вероятность возникновения некоторых проблем, перечисленных ранее, таких как коллапс формы.

В оставшейся части этой главы мы рассмотрим подход Wasserstein GAN — Gradient Penalty (WGAN-GP), который заключается во внесении в исследованную ранее инфраструктуру GAN некоторых важных изменений для увеличения стабильности модели.

Генеративно-сопоставительные сети Вассерштейна со штрафом за градиент

В этом разделе мы создадим WGAN-GP для генерации лиц из набора данных CelebA, который использовали в главе 3.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/04_gan/02_wgan_gp/wgan_gp.ipynb` в репозитории книги. Он был заимствован (и адаптирован) из превосходного руководства по WGAN-GP (<https://oreil.ly/dHYbC>), написанного Аakashем Кумаром Нэйном (Aakash Kumar Nain) и доступного на сайте Keras.

Появление генеративно-состязательных сетей Wasserstein GAN (WGAN), представленных в 2017 году в статье Мартина Аржовски (Martin Arjovsky) [Arjovsky et al., 2017], стало одним из первых больших шагов к стабилизации обучения генеративно-состязательных сетей. Добавив несколько изменений, авторам этого подхода удалось показать, как получить генеративно-состязательную сеть, у которой имеются следующие два свойства (цитата из статьи):

- «осмысленные значения потерь, коррелирующие со сходимостью генератора и качеством образцов;
- повышенная стабильность процесса оптимизации».

В частности, в статье представлена новая функция потерь Вассерштейна для дискриминатора и генератора. Ее использование вместо бинарной перекрестной энтропии дает более стабильную сходимость GAN.

В этом разделе мы определим функцию потерь Вассерштейна, а затем посмотрим, какие еще изменения нужно внести в архитектуру модели и процесс обучения, чтобы задействовать эту новую функцию потерь.

Полное определение класса модели можно найти в блокноте Jupyter, доступном по адресу `chapter05/wgan-gp/faces/train.ipynb` в репозитории книги.

Функция потерь Вассерштейна

Для начала вспомним, как выглядит бинарная перекрестная энтропия — функция, которую мы использовали для обучения дискриминатора и генератора генеративно-состязательной сети:

$$-\frac{1}{n} \sum_{i=1}^n (y_i \log(p_i) + (1 - y_i) \log(1 - p_i)).$$

В процессе обучения дискриминатора D мы вычисляем потери, сравнивая оценки для реальных изображений $p_i = D(x_i)$ с ответом $y_i = 1$ и оценки для сгенерированных изображений $p_i = D(G(z_i))$ с ответом $y_i = 0$. Минимизацию функции потерь для дискриминатора GAN можно записать так:

$$\min_D \left(\mathbb{E}_{x \sim p_X} [\log D(x)] + \mathbb{E}_{z \sim p_Z} [\log(1 - D(G(z)))] \right).$$

В процессе обучения генератора G мы вычисляем потери, сравнивая оценки для сгенерированных изображений $p_i = D(G(z_i))$ с ответом $y_i = 1$. Соответственно, минимизацию функции потерь для генератора GAN можно записать в виде функции потерь Вассерштейна, как показано далее:

$$\min_G \left(\mathbb{E}_{z \sim p_Z} [\log D(G(z))] \right).$$

Во-первых, функция потерь Вассерштейна требует использовать метки $y_i = 1$ и $y_i = -1$ вместо 1 и 0. Из последнего слоя дискриминатора нужно убрать функцию активации sigmoid с тем, чтобы оценки p_i больше не ограничивались диапазоном $[0, 1]$, и теперь они могут принимать любые значения в диапазоне $[-\infty, \infty]$. По этой причине дискриминатор в WGAN обычно называют *критиком*, который выводит *оценку*, а не вероятность.

Функция потерь Вассерштейна определяется следующим образом:

$$-\frac{1}{n} \sum_{i=1}^n (y_i p_i).$$

В процессе обучения критика D мы вычисляем потери, сравнивая оценки для реальных изображений $p_i = D(x_i)$ с ответом $y_i = 1$ и оценки для сгенерированных изображений $p_i = D(G(z_i))$ с ответом $y_i = -1$. То есть минимизацию функции потерь для критика WGAN можно записать так:

$$\min_D - \left(\mathbb{E}_{x \sim p_x} [D(x)] - \mathbb{E}_{z \sim p_z} [D(G(z))] \right).$$

Иными словами, критик WGAN пытается максимизировать разницу между оценками для реальных и сгенерированных изображений, при этом реальные изображения получают более высокую оценку.

В процессе обучения генератора WGAN мы вычисляем потери, сравнивая оценки для сгенерированных изображений $p_i = D(G(z_i))$ с ответом $y_i = 1$. Значит, минимизацию функции потерь для генератора WGAN можно записать так:

$$\min_G - \left(\mathbb{E}_{z \sim p_z} [D(G(z))] \right).$$

Иначе говоря, генератор WGAN пытается создать изображения, которые получают как можно более высокую оценку критика (то есть стремится обмануть критика и заставить его думать, что изображения реальны).

Ограничение Липшица

Возможно, вас удивило, что вместо применения функции активации sigmoid, ограничивающей выходное значение обычным диапазоном $[0, 1]$, мы позволили критику возвращать любые оценки в диапазоне $[-\infty, \infty]$. По этой причине величина потери Вассерштейна может получиться очень большой, что вызывает беспокойство — обычно следует избегать появления слишком больших чисел в нейронных сетях!

Фактически авторы статьи о WGAN утверждают, что при использовании функции потерь Вассерштейна также необходимо наложить дополнительное ограничение на критика. В частности, критик должен быть *одномерной липшицевой непрерывной функцией*. Остановимся на этом подробнее.

Критик — это функция D , преобразующая изображение в оценку реальности. Мы говорим, что это одномерная липшицева функция, если для любых двух входных изображений x_1 и x_2 она удовлетворяет следующему неравенству:

$$\frac{|D(x_1) - D(x_2)|}{|x_1 - x_2|} \leq 1.$$

Здесь $|x_1 - x_2|$ — это средняя попиксельная абсолютная разница между двумя изображениями, а $|D(x_1) - D(x_2)|$ — абсолютная разница между оценками критика. По сути, мы ограничиваем скорость, с которой могут меняться оценки критика между двумя изображениями, то есть абсолютное значение градиента нигде не должно превышать 1. Это верно для одномерной непрерывной функции Липшица (рис. 4.11) — ни в одной точке линия не попадает в область конуса, где бы вы ни поместили конус на линии. Иначе говоря, скорость, с которой линия может подниматься или опускаться в любой точке, ограничена.

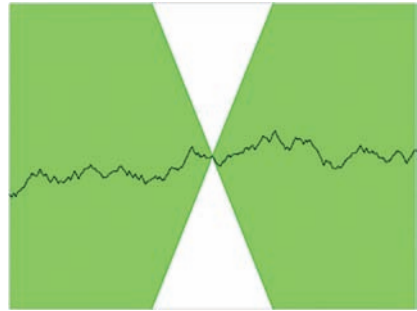


Рис. 4.11. Непрерывная функция Липшица (источник: Википедия)



Желающим глубже вникнуть в математическое обоснование того, почему функция потерь Вассерштейна может дать положительные результаты, только когда применяется это ограничение, Джонатан Хуэй (Jonathan Hui) предлагает отличное объяснение в своей статье (<https://oreil.ly/devy5>).

Реализация ограничения Липшица

В оригинальной статье о WGAN авторы показывают, как реализовать ограничение Липшица усечением весов критика до диапазона $[-0,01, 0,01]$ после каждого обучающего пакета. Один из недостатков этого подхода заключается в значительном снижении способности критика к обучению из-за уменьшения его весов. Фактически даже в оригинальной статье о WGAN авторы пишут: «Усечение веса — ужасный способ реализации ограничения Липшица». Сильный критик имеет решающее значение для успеха WGAN, поскольку без

точных градиентов генератор не сможет научиться адаптировать свои веса для получения более качественных подделок.

Другие исследователи продолжили поиск альтернативных способов реализации ограничения Липшица и улучшения способности WGAN изучать сложные признаки. Одним из таких способов стала генеративно-состязательная сеть Вассерштейна со штрафом за градиент (Wasserstein GAN with Gradient Penalty).

В статье, представляющей этот вариант [Gulrajani et al., 2017], авторы показывают, как можно напрямую реализовать ограничение Липшица путем включения в функцию потерь критика штрафного члена за градиент, который наказывает модель, если норма градиента отклоняется от 1. Это обеспечивает гораздо более устойчивый процесс обучения.

В следующем разделе мы увидим, как встроить этот дополнительный член в функцию потерь критика.

Функция потерь со штрафом за градиент

На рис. 4.12 показана диаграмма процесса обучения критика в сети WGAN-GP. Если сравнить ее с диаграммой процесса обучения дискриминатора на рис. 4.5, то можно увидеть, что ключевым дополнением в общую функцию потерь, кроме функции потерь Вассерштейна, является штраф за градиент между реальными и поддельными изображениями.

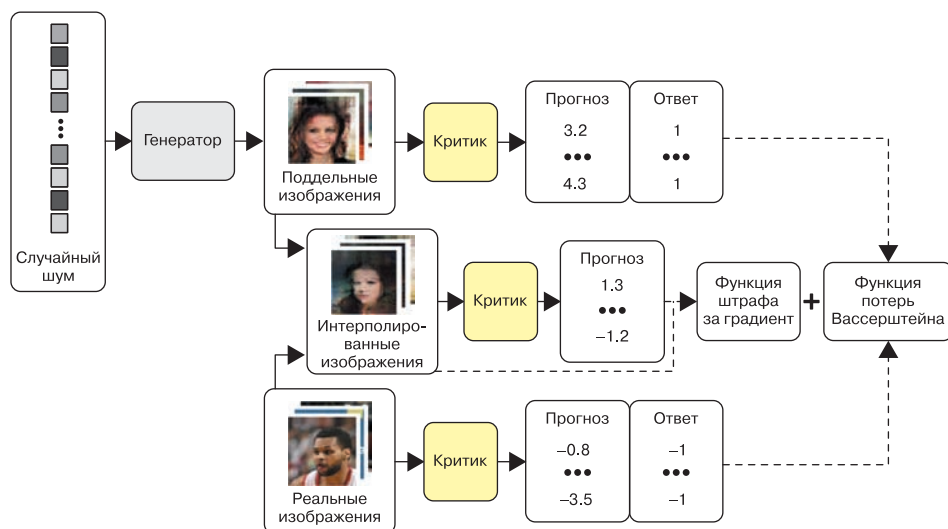


Рис. 4.12. Процесс обучения критика WGAN-GP

Функция потерь со штрафом за градиент измеряет квадрат разности между нормой градиента прогнозов для входных изображений и 1. Естественно, модель будет стремиться найти веса, гарантирующие минимальный штраф за градиент, стараясь тем самым соответствовать ограничению Липшица.

Однако было бы очень сложно вычислять этот градиент повсюду в процессе обучения, поэтому в WGAN-GP он оценивается лишь по нескольким точкам. Для сохранения баланса мы используем набор интерполированных изображений, лежащих в случайно выбранных точках вдоль линий, соединяющих пары реальных и поддельных изображений в пакете (рис. 4.13).

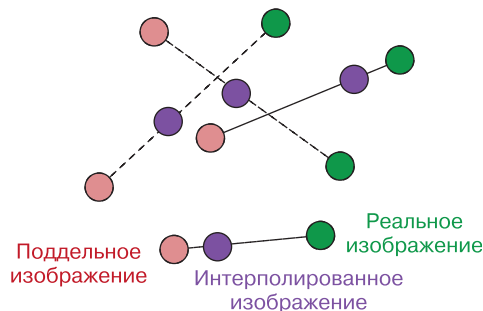


Рис. 4.13. Интерполяция изображений

В примере 4.8 показано, как выглядит код, вычисляющий штраф за градиент.

Пример 4.8. Функция потерь со штрафом за градиент

```
def gradient_penalty(self, batch_size, real_images, fake_images):
    alpha = tf.random.normal([batch_size, 1, 1, 1], 0.0, 1.0) ❶
    diff = fake_images - real_images
    interpolated = real_images + alpha * diff ❷

    with tf.GradientTape() as gp_tape:
        gp_tape.watch(interpolated)
        pred = self.critic(interpolated, training=True) ❸

    grads = gp_tape.gradient(pred, [interpolated])[0] ❹
    norm = tf.sqrt(tf.reduce_sum(tf.square(grads), axis=[1, 2, 3])) ❺
    gp = tf.reduce_mean((norm - 1.0) ** 2) ❻
    return gp
```

❶ Каждому изображению, сохраняемому как вектор `alpha`, в пакете присваивается случайное число в диапазоне от 0 до 1.

❷ Вычисляется множество интерполированных изображений.

❸ Критику предлагается оценить каждое из этих интерполированных изображений.

- ❹ С учетом входных изображений вычисляется градиент прогнозов.
- ❺ Вычисляется норма L2 этого вектора.
- ❻ Функция возвращает средний квадрат расстояния между нормой L2 и 1.

Обучение WGAN-GP

Ключевое преимущество применения функции потерь Вассерштейна — отсутствие необходимости беспокоиться о балансировании обучения критика и генератора. Фактически при использовании функции потерь Вассерштейна критик должен быть обучен сходимости до обновления генератора, чтобы убедиться в точности градиентов обновления весов генератора. Это основное отличие WGAN-GP от стандартной GAN, где важно не позволить дискриминатору стать слишком сильным.

Таким образом, между циклами обучения генератора в Wasserstein GAN можно выполнить несколько циклов обучения критика, чтобы обеспечить его сходимость. На практике обычно проводится от трех до пяти циклов обучения критика на один цикл обучения генератора.

Теперь мы познакомились с обеими ключевыми концепциями, лежащими в основе WGAN-GP: функцией потерь Вассерштейна и штрафным членом за градиент в функции потерь критика. Этап обучения модели WGAN, включающий их, показан в примере 4.9.

Пример 4.9. Обучение WGAN-GP

```
def train_step(self, real_images):
    batch_size = tf.shape(real_images)[0]

    for i in range(3): ❶
        random_latent_vectors = tf.random.normal(
            shape=(batch_size, self.latent_dim)
        )

        with tf.GradientTape() as tape:
            fake_images = self.generator(
                random_latent_vectors, training = True
            )
            fake_predictions = self.critic(fake_images, training = True)
            real_predictions = self.critic(real_images, training = True)

            c_wass_loss = tf.reduce_mean(fake_predictions) - tf.reduce_mean(
                real_predictions
            ) ❷
            c_gp = self.gradient_penalty(
                batch_size, real_images, fake_images
```



```

    ) ❸
    c_loss = c_wass_loss + c_gp * self.gp_weight ❹

    c_gradient = tape.gradient(c_loss, self.critic.trainable_variables)
    self.c_optimizer.apply_gradients(
        zip(c_gradient, self.critic.trainable_variables)
    ) ❺

    random_latent_vectors = tf.random.normal(
        shape=(batch_size, self.latent_dim)
    )
    with tf.GradientTape() as tape:
        fake_images = self.generator(random_latent_vectors, training=True)
        fake_predictions = self.critic(fake_images, training=True)
        g_loss = -tf.reduce_mean(fake_predictions) ❻

    gen_gradient = tape.gradient(g_loss, self.generator.trainable_variables)
    self.g_optimizer.apply_gradients(
        zip(gen_gradient, self.generator.trainable_variables)
    ) ❼

    self.c_loss_metric.update_state(c_loss)
    self.c_wass_loss_metric.update_state(c_wass_loss)
    self.c_gp_metric.update_state(c_gp)
    self.g_loss_metric.update_state(g_loss)

    return {m.name: m.result() for m in self.metrics}

```

- ❶ Выполнение трех обновлений критика.
- ❷ Вычисление потери Вассерштейна для критика — разности между средним прогнозом для поддельных и реальных изображений.
- ❸ Вычисление штрафа за градиент (см. пример 4.8).
- ❹ Функция потерь критика вычисляется как взвешенная сумма потерь Вассерштейна и штрафа за градиент.
- ❺ Обновление весов генератора.
- ❻ Обновление весов критика.
- ❼ Вычисление потери Вассерштейна для генератора.



Пакетная нормализация в WGAN-GP

Прежде чем перейти к обучению WGAN-GP, важно отметить, что в критике не должна использоваться пакетная нормализация. Причина этого в том, что пакетная нормализация создает корреляцию между изображениями в одном пакете, а это снижает эффективность штрафа за градиент. Как показали эксперименты, сети WGAN-GP способны давать превосходные результаты даже без пакетной нормализации в критике.

Теперь, рассмотрев все ключевые различия между стандартной GAN и WGAN-CP, подведем итоги.

- WGAN-GP использует потери Вассерштейна.
- WGAN-GP обучается с применением меток 1 для реальных и -1 — для поддельных объектов.
- В последнем слое критика не используется функция активации sigmoid.
- В функцию потерь для критика включен штраф за градиент.
- Между двумя циклами обучения генератора выполняется несколько циклов обучения критика.
- В критике нет слоев пакетной нормализации.

Анализ WGAN-GP

Рассмотрим несколько случайно выбранных образцов, созданных генератором после 25 эпох обучения (рис. 4.14).



Рис. 4.14. Примеры изображений, сгенерированных сетью WGAN-GP

Как видите, модель сумела определить существенные высокоуровневые признаки лиц и не проявила признаков коллапса формы.

Обратите внимание и на то, как изменяются потери модели с течением времени (рис. 4.15) — функции потерь обоих компонентов, критика и генератора, остаются стабильными и постепенно сходятся.

Если сравнить вывод сети WGAN-GP с выводом вариационного автокодировщика, то можно увидеть, что генеративно-сопоставительная сеть создает, как правило, более четкие изображения, особенно это касается границы между волосами и фоном. Вариационные автокодировщики склонны создавать более мягкие изображения с размытыми границами цвета, тогда как генеративно-сопоставительные сети порождают более четкие и контрастные изображения.

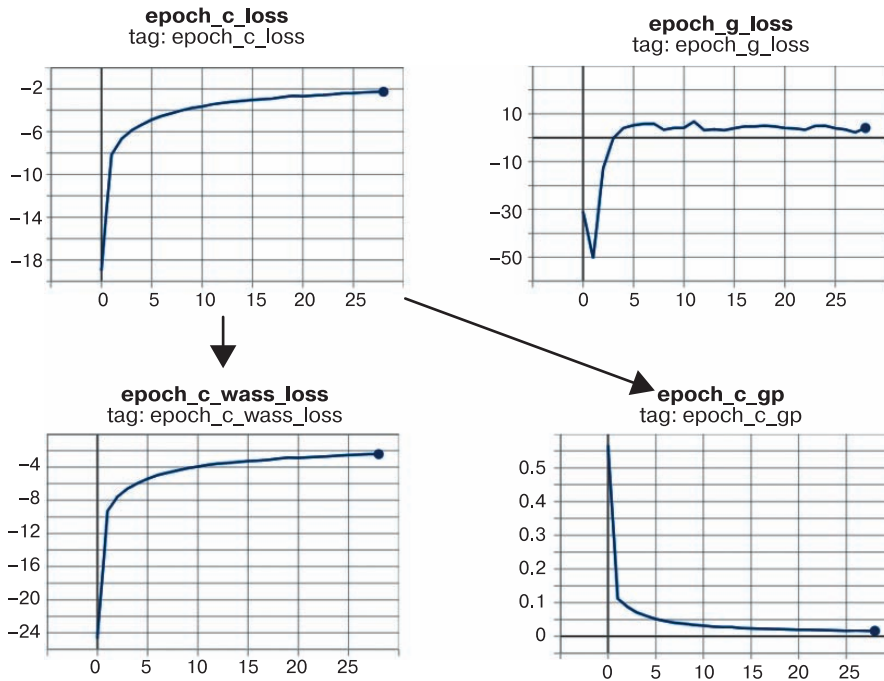


Рис. 4.15. Кривые изменения потерь в WGAN-GP: потери критика (`epoch_c_loss`) разбиваются на потери Вассерштейна (`epoch_c_wass`) и потери штрафа за градиент (`epoch_c_gp`)

Столь же верно и то, что генеративно-состязательные сети, как правило, труднее поддаются обучению, чем вариационные автокодировщики, и для достижения ими удовлетворительного качества требуется больше времени. Тем не менее большинство современных генеративных моделей основано на генеративно-состязательных сетях, потому что обучение крупномасштабных GAN на графических процессорах в течение длительного времени дает более впечатляющие результаты.

Условные генеративно-состязательные сети

К настоящему моменту мы создали несколько генеративно-состязательных сетей, способных генерировать реалистичные изображения на основе заданного обучающего набора. Но ни одна из них не дает возможности управлять типом создаваемого изображения, например, мы не сможем по желанию получить мужское или женское лицо, большой или маленький строительный блок. Мы можем выбрать случайную точку в скрытом пространстве, но не способны предсказать, какой тип изображения будет при этом создан.

В заключительной части этой главы создадим GAN, позволяющую управлять выходными данными, — так называемую *условную GAN* (conditional GAN, CGAN). Эта идея впервые была представлена Мирзой (Mirza) и Осиндеро (Osindero) в 2014 году в статье *Conditional Generative Adversarial Nets* [Mirza, Osindero, 2014] и заключается в относительно простом расширении архитектуры GAN.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/04_gan/03_cgan/cgan.ipynb` в репозитории книги. Он был заимствован (и адаптирован) из превосходного руководства по CGAN (<https://oreil.ly/Ey11I>), написанного Саяком Полом (Sayak Paul) и доступного на сайте Keras.

Архитектура CGAN

В этом примере мы заставим свою CGAN оценивать признак «блонд» в выходных изображениях. То есть сможем явно указать, какое изображение лица хотим получить — со светлыми волосами или нет. Этот признак включен в набор данных CelebA как метка.

Высокоуровневая архитектура CGAN показана на рис. 4.16.

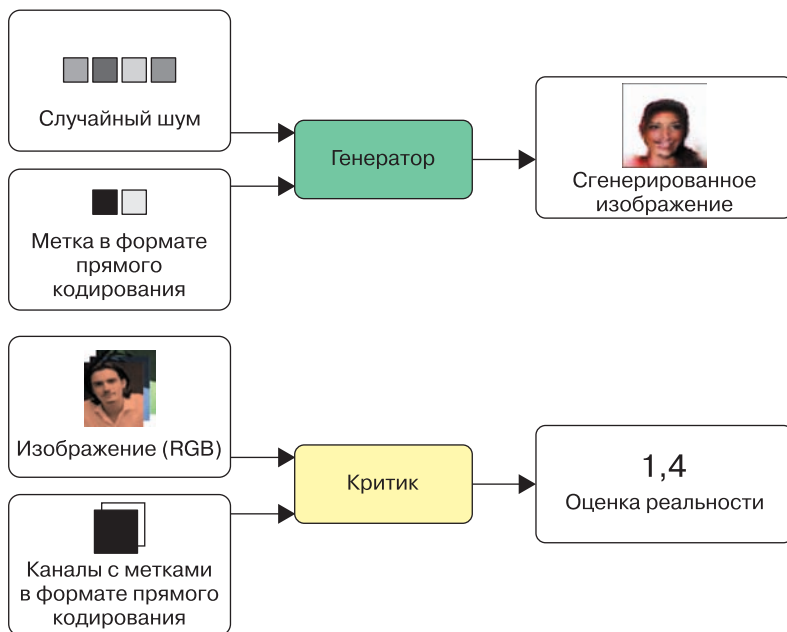


Рис. 4.16. Входы и выходы генератора и критика в CGAN

Ключевое отличие CGAN от обычной GAN состоит в том, что в CGAN генератору и критику передается дополнительная информация, касающаяся метки. На входе в генератор она просто добавляется в точку скрытого пространства как вектор прямого кодирования. На входе в критика информация о метке добавляется как дополнительные каналы к изображению RGB. Для этого вектор прямого кодирования повторяется столько раз, сколько необходимо, чтобы получить форму, соответствующую форме входных изображений.

В результате критик в CGAN получает доступ к дополнительной информации о содержимом изображения, поэтому, чтобы обмануть его, генератор должен генерировать изображения, соответствующие заданной метке. Даже если генератор создаст идеальное изображение, но оно не будет соответствовать метке, критик сможет забраковать его просто потому, что изображение и метка не совпали.



В нашем примере вектор прямого кодирования метки будет иметь длину 2, потому что согласно постановке задачи мы имеем только два класса: «блонд» и «не блонд». Однако при желании вы можете задать столько меток, сколько пожелаете, — например, можно обучить CGAN на наборе данных Fashion-MNIST для вывода одного из десяти различных предметов одежды, подав вектор меток в формате прямого кодирования длиной 10 на вход генератора и десять дополнительных каналов меток в формате прямого кодирования — на вход критика.

Единственное изменение, которое нужно внести в архитектуру, — объединить информацию о метках с существующими входными данными генератора и критика, как показано в примере 4.10.

Пример 4.10. Входные слои в CGAN

```
critic_input = layers.Input(shape=(64, 64, 3)) ❶
label_input = layers.Input(shape=(64, 64, 2))
x = layers.Concatenate(axis = -1)([critic_input, label_input])
...
generator_input = layers.Input(shape=(32,)) ❷
label_input = layers.Input(shape=(2,))
x = layers.Concatenate(axis = -1)([generator_input, label_input])
x = layers.Reshape((1, 1, 34))(x)
...
```

❶ Каналы изображений и каналы меток передаются критику по отдельности и затем объединяются.

❷ Скрытый вектор и классы меток передаются генератору по отдельности и объединяются перед изменением формы.

Обучение CGAN

Нужно также внести некоторые изменения в метод `train_step` сети CGAN, чтобы привести его в соответствие с новыми форматами входных данных генератора и критика, как показано в примере 4.11.

Пример 4.11. Метод `train_step` сети CGAN

```
def train_step(self, data):
    real_images, one_hot_labels = data ❶

    image_one_hot_labels = one_hot_labels[:, None, None, :] ❷
    image_one_hot_labels = tf.repeat(
        image_one_hot_labels, repeats=64, axis = 1
    )
    image_one_hot_labels = tf.repeat(
        image_one_hot_labels, repeats=64, axis = 2
    )

    batch_size = tf.shape(real_images)[0]

    for i in range(self.critic_steps):
        random_latent_vectors = tf.random.normal(
            shape=(batch_size, self.latent_dim)
        )

        with tf.GradientTape() as tape:
            fake_images = self.generator(
                [random_latent_vectors, one_hot_labels], training = True
            ) ❸

            fake_predictions = self.critic(
                [fake_images, image_one_hot_labels], training = True
            ) ❹
            real_predictions = self.critic(
                [real_images, image_one_hot_labels], training = True
            )

            c_wass_loss = tf.reduce_mean(fake_predictions) - tf.reduce_mean(
                real_predictions
            )
            c_gp = self.gradient_penalty(
                batch_size, real_images, fake_images, image_one_hot_labels
            ) ❺
            c_loss = c_wass_loss + c_gp * self.gp_weight

        c_gradient = tape.gradient(c_loss, self.critic.trainable_variables)
        self.c_optimizer.apply_gradients(
            zip(c_gradient, self.critic.trainable_variables)
        )
```

```

random_latent_vectors = tf.random.normal(
    shape=(batch_size, self.latent_dim)
)

with tf.GradientTape() as tape:
    fake_images = self.generator(
        [random_latent_vectors, one_hot_labels], training=True
    ) ❹
    fake_predictions = self.critic(
        [fake_images, image_one_hot_labels], training=True
    )
    g_loss = -tf.reduce_mean(fake_predictions)

gen_gradient = tape.gradient(g_loss, self.generator.trainable_variables)
self.g_optimizer.apply_gradients(
    zip(gen_gradient, self.generator.trainable_variables)
)

```

- ❶ Изображения и метки извлекаются из входных данных.
- ❷ Векторы прямого кодирования расширяются до формата прямого кодирования изображений, имеющего тот же пространственный размер, что и входные изображения (64×64).
- ❸ Генератор получает список с двумя компонентами входных данных — случайным вектором в скрытом пространстве и вектором меток прямого кодирования.
- ❹ Критик получает список с двумя компонентами входных данных — поддельным/реальным изображением и каналами меток в формате прямого кодирования.
- ❺ Функция штрафа за градиент тоже должна получить каналы меток, потому что для получения оценки реалистичности она использует критика.
- ❻ Изменения в этапе обучения критика вносятся и в этап обучения генератора.

Анализ CGAN

Мы можем управлять выводом CGAN, передавая на вход генератора конкретную метку в формате прямого кодирования. Например, чтобы сгенерировать изображение лица с темными (то есть не светлыми) волосами, нужно передать вектор $[1, 0]$. Чтобы сгенерировать изображение лица со светлыми волосами, следует передать вектор $[0, 1]$.

На рис. 4.17 приведены несколько примеров изображений, сгенерированных сетью CGAN. Здесь на вход сети подаются четыре вектора, случайно выбранных в скрытом пространстве. Каждый подается два раза с разными векторами условной метки. Как видите, CGAN научилась использовать вектор меток для

управления только атрибутом цвета волос на изображениях. Особенно впечатляет то, что остальная часть изображения практически не меняется — это доказывает способность GAN организовывать точки в скрытом пространстве так, что отдельные объекты можно отделить друг от друга.

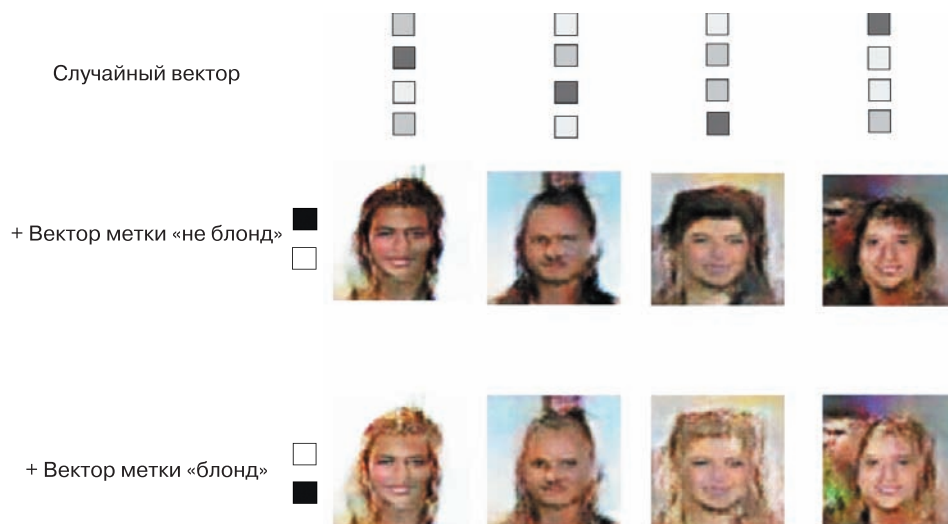


Рис. 4.17. Выходные данные CGAN при добавлении векторов в скрытое пространство к латентной выборке векторов «блонд» и «не блонд»



Если в наборе исходных данных доступны метки, то обычно рекомендуется их тоже включать во входные данные GAN, даже если нет необходимости накладывать какие-то условия на генерируемые выходные данные, потому что наличие дополнительной информации способствует улучшению качества генерируемых изображений. Метки можно рассматривать как высокоинформативное расширение пиксельной информации.

Резюме

В этой главе мы рассмотрели три разновидности генеративно-сопоставительных сетей: глубокие сверточные GAN (DCGAN), более сложные Wasserstein GAN со штрафом за градиент (WGAN-GP) и условные GAN (CGAN).

Все генеративно-сопоставительные сети имеют архитектуру «генератор против дискриминатора (или критика)», где дискриминатор пытается заметить разницу между реальными и поддельными изображениями, а генератор стремится обмануть его. При сбалансированном обучении этих двух соперников генератор

GAN может постепенно научиться производить наблюдения, похожие на наблюдения из обучающего набора.

Сначала мы обучили сеть DCGAN генерировать изображения игрушечных кубиков. Она смогла научиться реалистично представлять трехмерные объекты в виде изображений, довольно точно передавая тени, формы и текстуры. Мы также исследовали различные причины, по которым обучение GAN может потерпеть неудачу, включая коллапс формы и затухание градиента.

Затем мы увидели, как применение функции потерь Вассерштейна избавляет от многих из этих проблем, делая обучение WGAN более предсказуемым и надежным. Естественное развитие WGAN — модель WGAN-GP, включающая в процесс обучения одномерную непрерывную функцию Липшица в виде члена в функции потерь, чтобы подтянуть норму градиента к 1.

Применив WGAN-GP к задаче генерации лиц, мы увидели, как, выбирая точки из стандартного нормального распределения, можно получать новые лица. Процесс выборки очень похож на использовавшийся в вариационном автокодировщике, но лица, созданные сетью GAN, существенно отличаются — они часто более контрастные и имеют более четкие границы между различными частями.

Наконец, мы создали условную сеть CGAN, позволяющую управлять типом генерируемого изображения. С этой целью на вход критика и генератора передали метки, несущие дополнительную информацию, которая необходима, чтобы генератор мог соблюсти поставленное условие.

В целом, как видим, генеративно-состязательные сети обладают огромной гибкостью и с успехом могут применяться во многих практических направлениях. В частности, GAN обеспечили значительный прогресс в области генерации изображений благодаря множеству интересных расширений базовой структуры, с которыми вы познакомитесь в главе 10.

В следующей главе рассмотрим еще одно семейство генеративных моделей, идеально подходящих для моделирования последовательных данных, — модели авторегрессии.

ГЛАВА 5

Модели авторегрессии

В этой главе вы:

- узнаете, почему модели авторегрессии хорошо подходят для генерирования последовательных данных, таких как текст;
- узнаете, как обрабатывать и лексемизировать текстовые данные;
- познакомитесь с архитектурой рекуррентных нейронных сетей (Recurrent Neural Network, RNN);
- создадите и обучите сеть с долгой краткосрочной памятью (Long Short-Term Memory, LSTM) с помощью Keras;
- попыдаете использовать LSTM для генерирования нового текста;
- познакомитесь с другими вариантами RNN, включая управляемые рекуррентные блоки (Gated Recurrent Unit, GRU) и двунаправленные ячейки;
- исследуете приемы, позволяющие рассматривать изображения как последовательности пикселей;
- познакомитесь с архитектурой PixelCNN;
- создадите модель PixelCNN с помощью Keras;
- попыдаете генерировать изображения, используя PixelCNN.

К настоящему моменту вы познакомились с двумя разными семействами генеративных моделей, в которых задействованы скрытые переменные: вариационными автокодировщиками (variational autoencoders, VAE) и генеративно-состязательными сетями (generative adversarial networks, GAN). Обе модели вводят новую переменную с распределением, из которого легко выполнить выборку, и модель учится *декодировать* эту переменную обратно в исходную предметную область.

Теперь мы обратим внимание на *модели авторегрессии* — семейство моделей, которые упрощают задачу генеративного моделирования, рассматривая ее как последовательный процесс. Модели авторегрессии вычисляют прогнозы, опираясь на предыдущие значения в последовательности, а не на скрытую случайную величину. Поэтому они пытаются явно смоделировать распределение, генерирующее данные, а не аппроксимировать его (как в случае с VAE).

В этой главе мы рассмотрим две разные модели авторегрессии: сети с долгой краткосрочной памятью и PixelCNN. Первую мы применим к текстовым данным, а вторую — к изображениям. В главе 9 мы подробно рассмотрим еще одну весьма успешную модель авторегрессии — трансформер (Transformer).

Введение

Чтобы понять, как работает LSTM, сначала посетим странную тюрьму, заключенные которой организовали литературное общество...

ЛИТЕРАТУРНОЕ ОБЩЕСТВО ДЛЯ ПРОБЛЕМНЫХ ПРАВОНАРУШИТЕЛЕЙ

Тюремный надзиратель Эдвард Сопп ненавидел свою работу. Он проводил дни, наблюдая за заключенными и почти не имея свободного времени, чтобы предаться своей истинной страсти — написанию коротких рассказов. В конце концов вдохновение покинуло Соппа, и ему как-то нужно было восстановить способность к сочинительству.

Однажды ему в голову пришла блестящая идея, воплотив которую он смог бы создавать новые художественные произведения в своем стиле и в то же время занять заключенных. Идея состояла в том, чтобы заставить заключенных писать рассказы за него! Он основал новое общество LSTM (Literary Society for Troublesome Miscreants — литературное общество для проблемных правонарушителей) (рис. 5.1).



Рис. 5.1. Большая камера с заключенными, читающими книги (создано с помощью Midjourney)

¹ Здесь игра слов: аббревиатура LSTM в мире машинного обучения расшифровывается как Long Short-Term Memory — «долгая краткосрочная память». — *Примеч. пер.*

Тюрьма, где работает Эдвард, необычная, потому что в ней имеется только одна большая камера, где содержатся 256 заключенных. У каждого заключенного есть свое мнение о том, как следует продолжить очередной рассказ Эдварда. Каждый день надзиратель сообщает заключенным последнее слово из своего рассказа, а те должны скорректировать свое представление о его текущем состоянии, опираясь на новое слово и вчерашние мнения заключенных.

Каждый заключенный корректирует свое мнение, сопоставляя новое слово и мнения других заключенных с собственным мнением в прошлом. Прежде всего каждый решает, что из вчерашнего мнения он должен забыть, опираясь на новое слово и мнения других заключенных в камере. Затем все заключенные формируют новые мысли и решают, какие из них стоит добавить к прежним мнениям, которые решено перенести из предыдущего дня. После этого каждый формирует свое новое мнение на текущий день.

Однако заключенные скрытны и не всегда делятся своими мыслями с сокамерниками. Они также индивидуально используют последнее выбранное слово и мнения других заключенных, чтобы решить, какую часть своего мнения раскрыть.

Когда Эдвард хочет получить следующее слово, он просит заключенных сообщить свои мнения охраннику, который объединяет полученную информацию и выбирает, какое слово будет добавлено в конец рассказа. Это новое слово затем сообщается заключенным в камере, и процесс продолжается, пока рассказ не будет завершен.

Чтобы обучить заключенных и охранника, Эдвард передает написанные собственноручно короткие последовательности слов в камеру и проверяет, правильно ли заключенные выбрали следующее слово. Он сообщает им, насколько правильный выбор они сделали, и постепенно заключенные обучаются писать рассказы в собственном уникальном стиле.

После множества итераций обучения Эдвард обнаружил, что система достигла достаточно высокого уровня совершенства в создании реалистичного текста. Правда, текстам не хватает семантической структуры, но они, безусловно, демонстрируют характеристики, свойственные его предыдущим рассказам. Наконец, удовлетворенный результатами Сопп публикует сборник рассказов под названием «Басни Э. Соппа».

История мистера Соппа и его басен, написанных коллективом заключенных, весьма напоминает одну из наиболее успешных и широко используемых методик глубокого обучения для создания таких последовательных данных, как текст, — сеть с долгой краткосрочной памятью (Long Short-Term Memory, LSTM).

Сети с долгой краткосрочной памятью

Сеть LSTM — это разновидность *рекуррентной нейронной сети* (Recurrent Neural Network, RNN). Сети RNN содержат рекуррентный слой (или *ячейку*), способный обрабатывать последовательные данные и генерировать новые данные для определенного временного шага, формируя часть входных данных для следующего временного шага.

Когда сети RNN только появились, рекуррентные слои были очень простыми и состояли исключительно из оператора `tanh`, который гарантировал масштабирование информации, передаваемой между временными шагами, в диапазон от -1 до 1 . Но позднее выяснилось, что это решение страдает проблемой затухания градиента и плохо подходит для создания длинных последовательностей.

Впервые ячейки LSTM упомянуты в 1997 году в статье Зеппа Хохрайтера (Sepp Hochreiter) и Юргена Шмидхубера (Jürgen Schmidhuber) [Hochreiter, Schmidhuber, 1997]. В этой статье авторы описывают, почему LSTM не страдают проблемой затухания градиента, характерной для обычных RNN, и могут обучаться на последовательностях длиной в сотни временных шагов. С тех пор архитектура LSTM не раз адаптировалась и улучшалась. В настоящее время ее вариации, такие как *управляемые рекуррентные блоки* (Gated Recurrent Unit, GRU), широко используются на практике и доступны в виде слоев в Keras.

LSTM применялись для решения широкого спектра задач, связанных с последовательными данными, включая прогнозирование временных рядов, анализ эмоциональной окраски и классификацию аудио. В этой главе мы будем применять LSTM для решения задачи генерации текста.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/05_autoregressive/01_lstm/lstm.ipynb` в репозитории книги.

Набор данных Recipes

В качестве основы используем набор данных Epicurious Recipes (<https://oreil.ly/laNut>), доступный на сайте Kaggle. Он содержит более 20 000 рецептов с сопроводительными метаданными, такими как информация о пищевой ценности и списки ингредиентов.

Загрузить этот набор данных можно с помощью сценария, доступного в репозитории книги, как показано в примере 5.1. Рецепты и сопутствующие метаданные будут сохранены в локальной папке `/data`.

Пример 5.1. Загрузка набора данных Epicurious Recipe

```
bash scripts/download_kaggle_data.sh hugodarwood epirecipes
```

В примере 5.2 показано, как прочитать и отфильтровать данные, чтобы остались только рецепты с заголовками и описанием. Пример текстовой строки рецепта показан в примере 5.3.

Пример 5.2. Чтение и очистка данных

```
with open('/app/data/epirecipes/full_format_recipes.json') as json_data:
    recipe_data = json.load(json_data)

filtered_data = [
    'Recipe for ' + x['title'] + ' | ' + ' '.join(x['directions'])
    for x in recipe_data
    if 'title' in x
    and x['title'] is not None
    and 'directions' in x
    and x['directions'] is not None
]
```

Пример 5.3. Текстовая строка из набора данных Recipes

Recipe for Ham Persillade with Mustard Potato Salad and Mashed Peas | Chop enough parsley leaves to measure 1 tablespoon; reserve. Chop remaining leaves and stems and simmer with broth and garlic in a small saucepan, covered, 5 minutes. Meanwhile, sprinkle gelatin over water in a medium bowl and let soften 1 minute. Strain broth through a fine-mesh sieve into bowl with gelatin and stir to dissolve. Season with salt and pepper. Set bowl in an ice bath and cool to room temperature, stirring. Toss ham with reserved parsley and divide among jars. Pour gelatin on top and chill until set, at least 1 hour. Whisk together mayonnaise, mustard, vinegar, 1/4 teaspoon salt, and 1/4 teaspoon pepper in a large bowl. Stir in celery, cornichons, and potatoes. Pulse peas with marjoram, oil, 1/2 teaspoon pepper, and 1/4 teaspoon salt in a food processor to a coarse mash. Layer peas, then potato salad, over ham.

Прежде чем знакомиться с особенностями конструирования сетей LSTM с помощью Keras, необходимо сделать небольшое отступление, чтобы исследовать структуру текстовых данных и понять, чем они отличаются от изображений, которые мы видели ранее в этой книге.

Работа с текстовыми данными

Многие методы, прекрасно работающие с изображениями, плохо подходят для работы с текстовыми данными, потому что текстовые данные и изображения имеют несколько ключевых различий.

- Текстовые данные состоят из дискретных фрагментов (символов или слов), тогда как пиксели на изображении являются точками в непрерывном цветовом спектре. Мы легко можем сделать зеленый пиксел синим, но превратить слово «кошка» в слово «собака» намного сложнее. Из этого следует, что мы легко можем применить прием обратного распространения к изображениям, вычисляя градиент функции потерь в отношении отдельных пикселей и определяя направление, в котором цвета пикселей должны изменяться для

минимизации потери. Применить обратное распространение к дискретным текстовым данным обычным способом не получится, поэтому нужно найти способ обойти эту проблему.

- Данные в виде текста имеют временно́е измерение, но не имеют пространственного, тогда как изображения имеют два пространственных измерения, но не имеют временно́го. Порядок слов в текстовых данных очень важен, и, к примеру, слова, расположенные в обратном порядке, теряют смысл, тогда как изображения обычно могут переворачиваться без потери смыслового наполнения. Кроме того, в последовательностях слов нередко существуют протяженные зависимости, которые модель должна уметь выявлять, например, ответ на вопрос или перенос вперед контекста местоимения. В случае с изображениями все пиксели могут обрабатываться одновременно.
- Текстовые данные очень чувствительны к небольшим изменениям в отдельных единицах (словах или символах). Изображения, как правило, менее чувствительны к изменениям в отдельных пиксельных единицах — в изображении дома все равно можно узнать дом, даже изменив некоторые пиксели, формирующие его изображение. Однако изменение даже одного слова в тексте может радикально изменить его смысл или полностью лишить смысла. Это очень затрудняет обучение модели созданию связного текста, так как каждое слово жизненно важно для общего значения отрывка.
- Данные в виде текста имеют структуру, основанную на грамматических правилах, тогда как изображения не следуют каким-то правилам, регламентирующим, как должны назначаться значения пикселям. Например, в любом контексте было бы грамматически безграмотно написать: «Кошка сидит на обладающий». Существуют также семантические правила, которые чрезвычайно трудно смоделировать: например, выражение «я в пляже» семантически построено неправильно, хотя грамматически в нем нет ничего плохого.



Достижения в области текстового генеративного глубокого обучения

До недавнего времени большинство самых сложных моделей генеративного глубокого обучения были сосредоточены на анализе изображений, поскольку многие из задач, представленных в списке, были вне досягаемости даже самых передовых методов. Однако за последние пять лет был достигнут значительный прогресс в области генеративного глубокого обучения на текстовых данных благодаря внедрению архитектуры Transformer, которую мы рассмотрим в главе 9.

А теперь, памятуя об этом, посмотрим, какие шаги необходимо предпринять, чтобы привести текстовые данные в форму, пригодную для обучения сети LSTM.

Лексемизация

Первый шаг: очистка и лексемизация текста. *Лексемизация* — это процесс разделения текста на отдельные единицы — лексемы, такие как слова или символы.

Порядок лексемизации текста зависит от целей, стоящих перед моделью. Деление текста на слова и символы имеет свои плюсы и минусы, и выбор, который мы сделаем, будет влиять на то, как следует очистить текст перед моделированием и что получится на выходе модели.

При лексемизации по словам:

- весь текст можно преобразовать в нижний регистр, чтобы гарантировать одинаковую лексемизацию слов с прописными буквами в начале предложений и тех же слов в их середине. Однако иногда это может быть нежелательно: например, сохранение прописных букв в некоторых именах собственных (имена людей, географические названия) может дать определенные выгоды, потому что они будут лексемизироваться независимо;
- текстовый *словарь* (набор отдельных слов в обучающем наборе) может получиться очень большим, при этом некоторые слова будут использоваться в тексте очень редко или даже только один раз. Такие уникальные слова может быть целесообразно заменить специальной лексемой, обозначающей *неизвестное слово*, а не включать их как отдельные лексемы, чтобы уменьшить количество весов в нейронной сети;
- слова можно *упростить*, сведя их к базовой форме, чтобы глаголы в разном времени лексемизировались в одну лексему. Например, *просмотр*, *просмотреть*, *просмотрел* и *просмотрели* — все эти слова будут преобразованы в лексему *просмотр*;
- знаки препинания тоже нужно лексемизировать или вообще отбросить.

Применение лексемизации слов означает, что модель никогда не сможет сгенерировать слово, не вошедшее в учебный словарь.

Если выбрать лексемизацию по символам:

- то модель сможет генерировать последовательности символов, образующие новые слова, не вошедшие в учебный словарь. Это может быть желательно в одних контекстах, но нежелательно в других;
- прописные буквы могут преобразовываться в строчные или сохраняться в виде отдельных лексем;
- словарь обычно получается куда менее объемным. Это способствует увеличению скорости обучения модели, так как в конечном выходном слое содержится меньше весов.

В этом примере мы используем лексемизацию по словам с преобразованием символов в нижний регистр и без упрощения слов. Мы также будем лексемизировать знаки препинания, чтобы модель смогла предсказывать, когда, например, должны заканчиваться предложения или применяться запятые.

Код примера 5.4 очищает текст и выполняет лексемизацию.

Пример 5.4. Лексемизация

```
def pad_punctuation(s):
    s = re.sub(f"([{{string.punctuation}}])", r' \1 ', s)
    s = re.sub(' +', ' ', s)
    return s

text_data = [pad_punctuation(x) for x in filtered_data] ❶

text_ds = tf.data.Dataset.from_tensor_slices(text_data).batch(32).shuffle(1000) ❷

vectorize_layer = layers.TextVectorization( ❸
    standardize = 'lower',
    max_tokens = 10000,
    output_mode = "int",
    output_sequence_length = 200 + 1,
)

vectorize_layer.adapt(text_ds) ❹
vocab = vectorize_layer.get_vocabulary() ❺
```

❶ Добавление пробелов вокруг знаков препинания, чтобы они воспринимались как отдельные слова.

❷ Преобразование в набор данных TensorFlow.

❸ Создание слоя `TextVectorization` для преобразования текста в нижний регистр, присваивания наиболее распространенным 10 000 слов соответствующих целочисленных индексов и усечения длины последовательности до 201 лексем.

❹ Применение слоя `TextVectorization` к обучающим данным.

❺ Переменная `vocab` хранит список лексем слов.

В примере 5.5 показан результат лексемизации рецепта из примера 5.3. Длина последовательности, используемой для обучения модели, — это параметр процесса обучения. В примере мы решили ограничить последовательность длиной 200, поэтому дополняем или обрезаем рецепт до этой длины, чтобы создать целевую переменную (подробнее об этом рассказывается в следующем разделе). При необходимости, чтобы достичь желаемой длины, в конец вектора добавляют нули.



Стоп-лексемы

Лексема с индексом 0 известна как стоп-лексема и обозначает конец текстовой строки.

Пример 5.5. Рецепт из примера 5.3 после лексемизации и замены лексем индексами

```
[ 26 16 557 1 8 298 335 189 4 1054 494 27 332 228
 235 262 5 594 11 133 22 311 2 332 45 262 4 671
 4 70 8 171 4 81 6 9 65 80 3 121 3 59
 12 2 299 3 88 650 20 39 6 9 29 21 4 67
 529 11 164 2 320 171 102 9 374 13 643 306 25 21
 8 650 4 42 5 931 2 63 8 24 4 33 2 114
 21 6 178 181 1245 4 60 5 140 112 3 48 2 117
 557 8 285 235 4 200 292 980 2 107 650 28 72 4
 108 10 114 3 57 204 11 172 2 73 110 482 3 298
 3 190 3 11 23 32 142 24 3 4 11 23 32 142
 33 6 9 30 21 2 42 6 353 3 3224 3 4 150
 2 437 494 8 1281 3 37 3 11 23 15 142 33 3
 4 11 23 32 142 24 6 9 291 188 5 9 412 572
 2 230 494 3 46 335 189 3 20 557 2 0 0 0
 0 0 0 0 0]
```

В примере 5.6 показана часть словаря лексем с соответствующими им индексами. Слой резервирует индекс 0 для заполнения (то есть для стоп-лексемы) и индекс 1 для неизвестных слов, не входящих в 10 000 наиболее общеупотребимых слов (например, индекс 1 присваивается слову *persillade*). Остальные слова получают индексы в зависимости от частоты их встречаемости в текстах. Количество слов, включаемых в словарь, тоже является параметром обучения. Чем их больше в словаре, тем меньше неизвестных лексем встретится в тексте, однако модель должна быть больше, чтобы вместить больший размер словаря.

Пример 5.6. Словарь для слоя TextVectorization

```
0:
1: [UNK]
2: .
3: ,
4: and
5: to
6: in
7: the
8: with
9: a
```

Создание набора обучающих данных

Наша сеть LSTM будет обучаться предсказывать следующее слово по имеющейся последовательности с учетом слов, предшествующих текущей позиции. Например, мы можем передать в модель текст «курица гриль с отварным» и ожи-

дать, что она выведет подходящее следующее слово, например, «картофелем», а не «бананом».

Поэтому мы можем просто сдвинуть всю последовательность на одну лексему, чтобы создать целевую переменную.

Код, представленный в примере 5.7, создает обучающий набор данных.

Пример 5.7. Создание обучающего набора данных

```
def prepare_inputs(text):
    text = tf.expand_dims(text, -1)
    tokenized_sentences = vectorize_layer(text)
    x = tokenized_sentences[:, :-1]
    y = tokenized_sentences[:, 1:]
    return x, y
```

```
train_ds = text_ds.map(prepare_inputs) ❶
```

❶ Создает обучающий набор, состоящий из вектора индексов лексем рецептов (входные данные) и того же самого вектора, смещенного на одну лексему (цель).

Архитектура модели LSTM

В табл. 5.1 показана общая архитектура модели LSTM. На вход модели подается последовательность целочисленных индексов лексем, а на выходе для каждого слова в словаре возвращается вероятность того, что именно это слово должно быть очередным в последовательности. Чтобы понять, как работает модель, нужно познакомиться с двумя новыми типами слоев: *Embedding* и *LSTM*.

Таблица 5.1. Сводная информация о модели LSTM

Слой (тип)	Выходная форма	Число параметров
Input	(None, None)	0
Embedding	(None, None, 100)	1 000 000
LSTM	(None, None, 128)	117 248
Dense	(None, None, 10000)	1 290 000

Всего параметров 2 407 248

Обучаемых параметров 2 407 248

Необучаемых параметров 0



Входной слой в модели LSTM

Обратите внимание на то, что входной слой не требует заранее указывать длину последовательности. И размер пакета, и длина последовательности могут свободно меняться от случая к случаю, отсюда и форма (None, None). Это объясняется тем, что все нижестоящие слои не зависят от длины передаваемой последовательности.

Слой Embedding

Слой Embedding (создание векторного представления) — это, по сути, таблица соответствий, преобразующая каждую лексему в вектор длиной `embedding_size` (рис. 5.2). Вектор соответствий определяется моделью в процессе обучения подобно *весам*. То есть количество весов в этом слое равно размеру словаря, умноженному на `embedding_size` — размер векторного представления (например, $10\,000 \times 100 = 1\,000\,000$).

Размер словаря (10 000)	Лексема	Векторное представление				
	0	-0.13	0.45	...	0.13	-0.04
	1	0.22	0.56	...	0.24	-0.63
	...	...	...	...	...	...
	9998	0.16	-0.70	...	-0.35	1.02
	9999	-0.98	-0.45	...	-0.15	-0.52
		Размер векторного представления лексемы (100)				

Рис. 5.2. Слой Embedding — это таблица преобразования целочисленных представлений лексем в векторы

Целочисленный индекс каждой лексемы преобразуется в непрерывный вектор, позволяя модели выучить векторное представление каждого слова, которое может обновляться на этапе обратного распространения. Мы также могли бы использовать метод прямого кодирования для каждой входной лексемы, но подход на основе векторного представления с применением слоя Embedding предпочтительнее, поскольку делает это векторное представление обучаемым, что увеличивает гибкость модели при принятии решения о том, какое представление должна иметь каждая лексема, чтобы повысить качество модели.

Таким образом, слой Input передает тензор целочисленных представлений лексем с формой `[batch_size, seq_length]` в слой Embedding, который возвращает

тензор с формой `[batch_size, seq_length, embedding_size]`, затем передаваемый в слой LSTM (рис. 5.3).

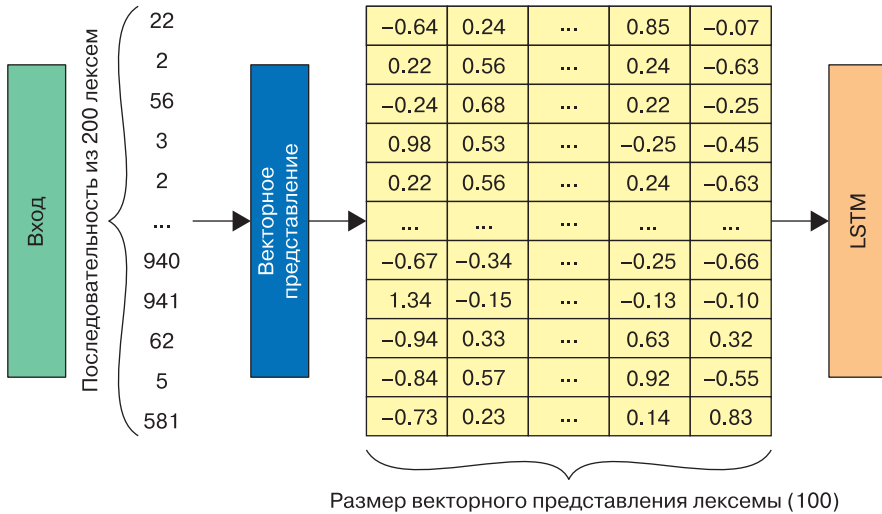


Рис. 5.3. Как единственная последовательность пересекает слой Embedding

Слой LSTM

Чтобы понять, как работает слой LSTM, сначала познакомимся с работой рекуррентного слоя.

Рекуррентный слой обладает особой возможностью обрабатывать последовательные входные данные $x_1 \dots x_n$. Он состоит из ячейки, обновляющей свое *скрытое состояние* h_t по мере прохождения каждого элемента последовательности x_t .

Скрытое состояние — вектор, длина которого равна количеству *узлов* в ячейке, — можно рассматривать как текущее понимание последовательности ячейкой. На временном шаге t ячейка использует предыдущее значение скрытого состояния h_{t-1} с данными из текущего временного шага x_t , чтобы получить обновленный вектор скрытого состояния h_t . Этот рекуррентный процесс продолжается до конца последовательности. Когда последовательность закончится, слой возвращает окончательное скрытое состояние ячейки h_n , которое передается в следующий слой сети. Этот процесс показан на рис. 5.4.

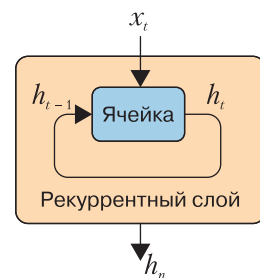


Рис. 5.4. Схема работы простого рекуррентного слоя

Чтобы объяснить работу слоя более подробно, развернем процесс и посмотрим, как одиночная последовательность пересекает слой (рис. 5.5).

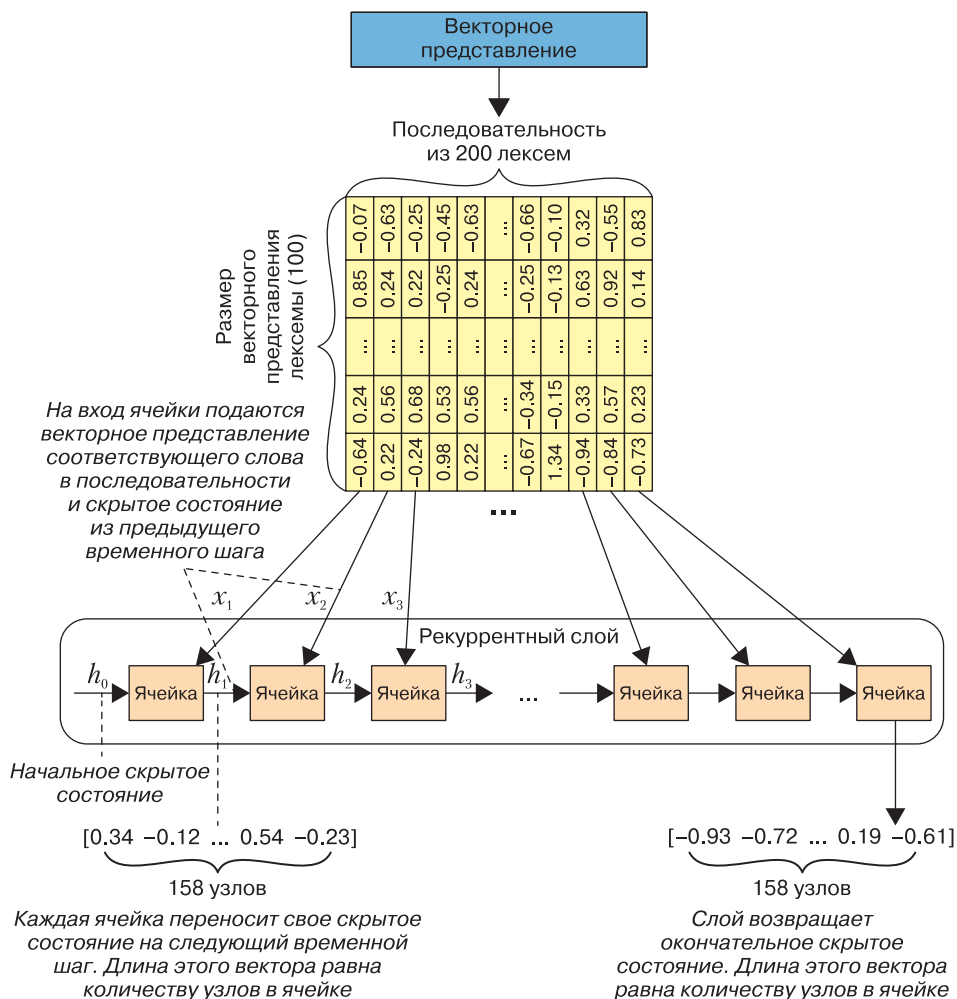


Рис. 5.5. Как одиночная последовательность пересекает рекуррентный слой



Веса ячеек

Важно помнить, что все ячейки на этой диаграмме имеют одинаковый вес (потому что на самом деле все они совершенно одинаковые). Между этой диаграммой и диаграммой на рис. 5.4 нет никакой разницы; это всего лишь другой способ представления механики рекуррентного слоя.

Эта диаграмма рекуррентного процесса, где каждая ячейка представляет отдельный временной шаг, наглядно показывает, как скрытое состояние постоянно обновляется при прохождении через ячейки. Здесь ясно видно, что предыдущее скрытое состояние смешивается с текущей точкой данных в последовательности (то есть с векторным представлением текущего слова) и создается следующее скрытое состояние. Результатом слоя является достигнутое скрытое состояние ячейки после обработки всех слов входной последовательности.



Выбор названия «скрытое состояние» для результата, получаемого ячейкой, — это следствие неудачного соглашения. На самом деле оно не скрыто, и вы не должны считать его таковым. В действительности последнее скрытое состояние становится результатом всего слоя, кроме того, далее в этой главе не раз будет показана возможность получения доступа к скрытому состоянию на каждом отдельном временном шаге.

Ячейка LSTM

Теперь, увидев, как работает простой рекуррентный слой, рассмотрим отдельную ячейку LSTM.

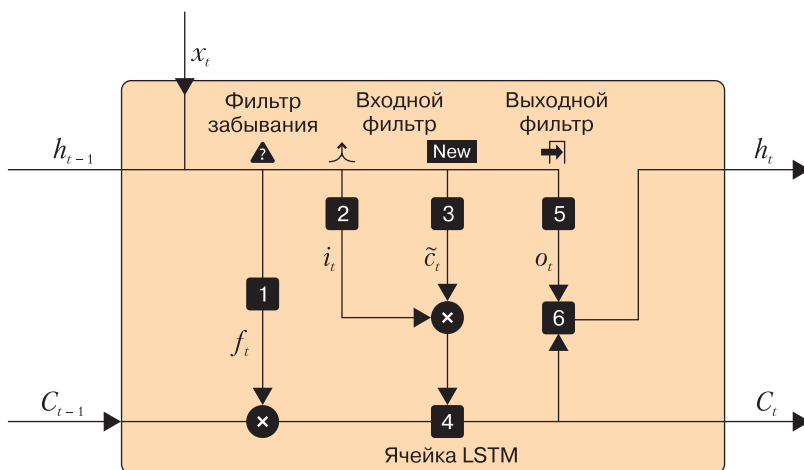
Задача ячейки LSTM — вернуть новое скрытое состояние h_t , опираясь на предыдущее скрытое состояние h_{t-1} и векторное представление текущего слова x_t . Напомню: длина h_t равна количеству узлов в LSTM. Это параметр, устанавливаемый при определении слоя и не имеющий никакого отношения к длине последовательности.



Не путайте понятия «ячейка» (cell) и «узел» (unit). В слое LSTM есть только одна ячейка с множеством содержащихся в ней узлов (так же как в тюремной камере из нашей предыстории содержалось много заключенных). Мы часто изображаем рекуррентный слой в виде развернутой цепочки ячеек, поскольку это помогает наглядно представить, как происходит обновление скрытого состояния на каждом временном шаге.

Ячейка LSTM поддерживает состояние C_t , которое можно рассматривать как ее внутреннее убеждение в том, каково текущее состояние последовательности. Это состояние отличается от скрытого состояния h_t , которое в конечном итоге возвращается ячейкой после последнего временного шага. Состояние C_t имеет ту же длину, что и скрытое состояние, — она равна количеству узлов в ячейке.

Рассмотрим подробнее отдельную ячейку и то, как происходит обновление скрытого состояния (рис. 5.6).



$$1 \quad f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$2 \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$3 \quad \tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$$4 \quad C_t = f_t C_{t-1} + i_t \tilde{C}_t$$

$$5 \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$6 \quad h_t = o_t \tanh(C_t)$$

Рис. 5.6. Ячейка LSTM

Скрытое состояние обновляется в шесть приемов.

1. Скрытое состояние h_{t-1} , полученное на предыдущем временном шаге, и векторное представление текущего слова x_t объединяются и передаются через *фильтр забывания*, являющийся обычным полносвязным слоем с матрицей весов W_f , смещением b_f и функцией активации sigmoid. Вектор f_t на выходе фильтра имеет длину, равную количеству узлов в ячейке, и содержит значения в диапазоне от 0 до 1, определяющие, какая доля предыдущего состояния ячейки C_{t-1} должна сохраниться.
2. Объединенный вектор передается через *входной фильтр*, который, подобно фильтру забывания, является полносвязным слоем с матрицей весов W_i , смещением b_i и функцией активации sigmoid. Выходной вектор i_t этого фильтра имеет длину, равную числу узлов в ячейке, и содержит значения в диапазоне от 0 до 1, определяющие, какая доля новой информации будет добавлена в предыдущее состояние ячейки C_{t-1} .

3. Объединенный вектор передается через полносвязный слой с матрицей весов W_c , смещением b_c и функцией активации $\tanh$, чтобы получить вектор $\tilde{C}_t$, содержащий новую информацию, которую ячейка решила сохранить. Он содержит значения в диапазоне от -1 до 1 , и его длина тоже равна количеству узлов в ячейке.
4. Векторы f_t и C_{t-1} умножаются поэлементно и складываются с поэлементным произведением i_t и $\tilde{C}_t$. Это равноценно забыванию части предыдущего состояния ячейки и добавлению новой релевантной информации для получения обновленного состояния ячейки C_t .
5. Исходный объединенный вектор также передается *черезвыходной фильтр* — полносвязный слой с матрицей весов W_o , смещением b_o и активацией sigmoid . Вектор результата o_t имеет длину, равную количеству узлов в ячейке, и хранит значения в диапазоне от 0 до 1 , определяющие, какая доля обновленного состояния C_t может покинуть ячейку.
6. o_t умножаются поэлементно на обновленное состояние ячейки C_t , а затем применяется активация $\tanh$, чтобы создать новое скрытое состояние h_t .



Слой LSTM в Keras

В Keras все эти сложности заключены в типе слоя LSTM, поэтому вам не придется беспокоиться об их реализации самостоятельно!

Обучение LSTM

В примере 5.8 показан код, конструирующий сеть LSTM.

Пример 5.8. Конструирование, компиляция и обучение сети LSTM

```
inputs = layers.Input(shape=(None,), dtype="int32") ❶
x = layers.Embedding(10000, 100)(inputs) ❷
x = layers.LSTM(128, return_sequences=True)(x) ❸
outputs = layers.Dense(10000, activation = 'softmax')(x) ❹
lstm = models.Model(inputs, outputs) ❺

loss_fn = losses.SparseCategoricalCrossentropy()
lstm.compile("adam", loss_fn) ❻
lstm.fit(train_ds, epochs=25) ❼
```

❶ Входной слой Input не требует заранее указывать длину последовательности (она может быть любой), поэтому мы используем `None` как заполнитель.

❷ Слою Embedding требуется два параметра: размер словаря (10 000 лексем) и размерность векторного представления (100).

❸ Слои LSTM требуют указывать размерность скрытого вектора (128). Мы также решили возвращать полную последовательность скрытых состояний, а не только скрытое состояние, полученное на последнем шаге.

❹ Слой Dense преобразует скрытые состояния на каждом временном шаге в вектор вероятностей следующей лексемы.

❺ Экземпляр класса `Model` прогнозирует следующую лексему, исходя из входной последовательности лексем. Это делается для каждой лексемы в последовательности.

❻ Модель компилируется с функцией потерь `SparseCategoricalCrossentropy` — это та же самая многозначная перекрестная энтропия, но она применяется, когда метки являются целыми числами, а не векторами прямого кодирования.

❼ Модель обучается на наборе обучающих данных.

На рис. 5.7 можно увидеть первые несколько эпох процесса обучения LSTM — обратите внимание на то, как выходные данные примера становятся более осмысленными по мере уменьшения величины потерь. На рис. 5.8 показано уменьшение меры перекрестной энтропии на протяжении всего процесса обучения.

```
Epoch 1/25
628/629 [=====>.] - ETA: 0s - loss: 4.4536
generated text:
recipe for mold salad are high 8 pickled to fold cook the dish into and warm in baking reduced but halves beans
and cut

629/629 [=====] - 29s 43ms/step - loss: 4.4527
Epoch 2/25
628/629 [=====>.] - ETA: 0s - loss: 3.2339
generated text:
recipe for racks - up-don with herb fizz | serve checking thighs onto sanding butter and baking surface in a hea
vy heavy large saucepan over blender ; stand overnight . [UNK] over moderate heat until very blended , garlic ,
about 8 minutes . cook airtight until cooked are soft seeds , about 1 45 minutes . sugar , until s is brown , 5
to sliced , parmesan , until browned and add extract . wooden crumb to outside of out sheets . flatten and prehe
ated return to the paste . add in pecans oval and let transfer day .

629/629 [=====] - 30s 48ms/step - loss: 3.2336
Epoch 3/25
629/629 [=====] - ETA: 0s - loss: 2.6229
generated text:
recipe for grilled chicken | preheat oven to 400*f . cook in large 8 - caramel grinder or until desired are firm
, about 6 minutes

629/629 [=====] - 27s 42ms/step - loss: 2.6229
Epoch 4/25
629/629 [=====] - ETA: 0s - loss: 2.3426
generated text:
recipe for pizza salad with sweet red pepper and star fruit | combine all ingredients except lowest ingredients
in a large skillet . working with batches and deglaze , cook until just cooked through , about 1 minute . meanwh
ile , boil potatoes and paprika in a little oil over medium - high heat , stirring it just until crisp , about 3
minutes . stir in bell pepper , onion and cooked paste and jalapeño until clams well after most - reggiano , abo
ut 5 minutes . transfer warm 2 tablespoons flesh of eggplants to medium bowl . serve .
```

Рис. 5.7. Первые несколько эпох обучения LSTM

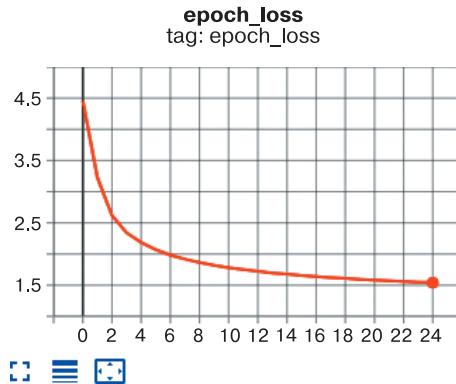


Рис. 5.8. График изменения меры перекрестной энтропии в процессе обучения LSTM

Анализ LSTM

Теперь, когда сеть LSTM скомпилирована и обучена, можно начать использовать ее для генерации длинных строк текста. Для этого нужно выполнить следующие действия.

1. Передать в сеть имеющуюся последовательность слов и попросить ее предсказать следующее за ней слово.
2. Добавить это слово в конец последовательности и повторить.

Сеть будет выводить набор, включающий вероятности для всех слов в словаре, опираясь на которые мы можем делать выбор. Таким образом, генерировать текст можно не только детерминированно, но и стохастически. Кроме того, можно ввести в процесс выбора *температурный* параметр, чтобы указать, насколько детерминированным он должен быть.



Температурный параметр

Температура, близкая к 0, делает выборку более детерминированной (то есть чаще будет выбираться слово с наибольшей вероятностью), тогда как температура 1 означает, что каждое слово выбирается с учетом вероятности, выдаваемой моделью.

Эта возможность реализуется с помощью кода из примера 5.9, определяющего функцию обратного вызова, которую можно использовать для генерации текста в конце каждой эпохи обучения.

Пример 5.9. Функция обратного вызова TextGenerator

```

class TextGenerator(callbacks.Callback):
    def __init__(self, index_to_word, top_k=10):
        self.index_to_word = index_to_word
        self.word_to_index = {
            word: index for index, word in enumerate(index_to_word)
        } ❶

    def sample_from(self, probs, temperature): ❷
        probs = probs ** (1 / temperature)
        probs = probs / np.sum(probs)
        return np.random.choice(len(probs), p=probs), probs

    def generate(self, start_prompt, max_tokens, temperature):
        start_tokens = [
            self.word_to_index.get(x, 1) for x in start_prompt.split()
        ] ❸
        sample_token = None
        info = []
        while len(start_tokens) < max_tokens and sample_token != 0: ❹
            x = np.array([start_tokens])
            y = self.model.predict(x) ❺
            sample_token, probs = self.sample_from(y[0][-1], temperature) ❻
            info.append({'prompt': start_prompt, 'word_probs': probs})
            start_tokens.append(sample_token) ❼
            start_prompt = start_prompt + ' ' + self.index_to_word[sample_token]
        print(f"\ngenerated text:\n{start_prompt}\n")
        return info

    def on_epoch_end(self, epoch, logs=None):
        self.generate("recipe for", max_tokens = 100, temperature = 1.0)

```

❶ Создается словарь обратного отображения (слова в индекс лексемы).

❷ Эта функция обновляет вероятности с коэффициентом *temperature*.

❸ Начальный текст — это последовательность слов, которую требуется передать модели, чтобы запустить процесс генерации, например, *recipe for* («рецепт для»). Слова сначала преобразуются в список лексем.

❹ Последовательность генерируется до тех пор, пока ее длина не достигнет *max_tokens* или не будет создана стоп-лексема (0).

❺ Модель возвращает для каждого слова в словаре его вероятность стать следующим словом в последовательности.

❻ Вероятности передаются функции *sample_from*, которая делает выбор с учетом параметра *temperature*.

❼ Новое слово добавляется в исходный текст для следующей итерации.

Давайте посмотрим, что генерирует эта сеть с двумя разными значениями `temperature` (рис. 5.9).

`temperature = 1.0`

generated text:

recipe for sour japanese potatoes julienne | in a bowl stir together the yeast mixture with the milk and the peanut butter crumbs , the sour cream , and the butter mixture with a fork , gently fold in the prunes gently or until incorporated . lightly stir the oil and yeast until it just holds soft peaks , but not runny , on bo ttom of a 7 - sided sheet of aluminum foil , top it with a round , and a pinch of each brownies into a goblet , or with the baking dish . serve each with sorbet

`temperature = 0.2`

generated text:

recipe for grilled chicken with mustard - herb sauce | combine first 6 ingredients in medium bowl . add chick en to pot . add chicken and turn to coat . cover and refrigerate at least 1 hour and up to 1 day . preheat ov en to 450°f . place turkey on rock in roasting pan . roast until thermometer inserted into thickest part of t high registers 175°f , about 1 hour longer . transfer to rack in center of oven and preheat to 450°f . brush chicken with oil . sprinkle with salt and pepper . roast until thermometer inserted into

Рис. 5.9. Два фрагмента текста, сгенерированных с параметрами `temperature = 1.0` и `temperature = 0.2`

В этих двух отрывках есть несколько моментов, которые стоит отметить. Во-первых, оба они стилистически похожи на рецепты из обучающего набора. Оба открываются названиями рецептов и содержат в целом грамматически правильные конструкции. Разница лишь в том, что текст, сгенерированный с параметром `temperature = 1.0`, более смелый и поэтому менее точный, чем пример с параметром `temperature = 0.2`. Таким образом, создание нескольких выборок с параметром `temperature = 1.0` приведет к большему разнообразию, поскольку модель выполняет выборку из распределения вероятностей с большей дисперсией.

Для демонстрации на рис. 5.10 показаны пять слов с наибольшей вероятностью следования за различными последовательностями для обоих значений параметра `temperature`.

Модель может генерировать список следующих наиболее вероятных слов в пределах контекста. Например, хотя модель ничего не знает о частях речи, таких как существительные, глаголы, прилагательные и предлоги, она обычно способна разделить слова на эти классы и использовать их грамматически правильно.

temperature = 1.0	temperature = 0.2
PROMPT: recipe for roast turkey: 22.81% chicken: 19.41% beef: 10.24% pork: 9.96% leg: 4.06% ----- PROMPT: recipe for roasted vegetables preheat: 69.63% prepare: 3.68% heat: 3.45% put: 2.12% combine: 1.96% ----- PROMPT: recipe for chocolate ice cream in: 27.31% combine: 11.21% stir: 6.66% whisk: 5.64% mix: 3.68% ----- PROMPT: recipe for roasted vegetables chop 1 / 2: 53.51% 4: 29.83% 3: 13.11% 8: 0.78% 1: 0.56% -----	PROMPT: recipe for roast turkey: 67.54% chicken: 30.15% beef: 1.23% pork: 1.07% leg: 0.01% ----- PROMPT: recipe for roasted vegetables preheat: 100.0% prepare: 0.0% heat: 0.0% put: 0.0% combine: 0.0% ----- PROMPT: recipe for chocolate ice cream in: 98.71% combine: 1.15% stir: 0.09% whisk: 0.04% mix: 0.0% ----- PROMPT: recipe for roasted vegetables chop 1 / 2: 94.81% 4: 5.11% 3: 0.08% 8: 0.0% 1: 0.0% -----

Рис. 5.10. Распределение вероятностей слов, которые могли бы продолжить разные последовательности, для значений 1.0 и 0.2 параметра *temperature*

Более того, модель способна подобрать подходящий глагол для начала рецепта, исходя из предшествующего заголовка. Для жареных овощей (*roasted vegetables*) в качестве наиболее вероятных вариантов она выбирает *preheat* (предварительный нагрев сковороды), *prepare* (подготовка), *heat* (обжарка), *combine* (объединение), тогда как для мороженого (*ice cream*) — *combine* (объединение), *stir* (помешивание), *whisk* (взбивание) и *mix* (перемешивание). Это показывает, что модель имеет некоторое контекстуальное понимание различий между рецептами в зависимости от ингредиентов.

Обратите также внимание на то, что вероятность первой лексемы в примере с *temperature = 0.2* взвешивается сильнее. По этой причине при более низком значении температурного параметра разнообразие сгенерированных рецептов обычно меньше.

Наша простая модель LSTM отлично справилась с созданием реалистичного текста, но ей все еще трудно уловить семантические значения генерируемых слов. В своих рецептах она перечисляет ингредиенты, которые вряд ли будут хорошо сочетаться друг с другом, например, кислый японский картофель (*sour Japanese potatoes*), крошки пекана (*pecan crumbs*) и шербет (*sorbet*)! В некоторых

случаях такое поведение может быть желательно, например, если мы хотим, чтобы модель LSTM генерировала необычные и уникальные шаблоны слов, но в других случаях хотелось бы, чтобы она имела более глубокое понимание способов группировки слов и более долгую память для запоминания идей, появляющихся ранее в тексте.

В следующем разделе мы рассмотрим некоторые способы улучшения нашей сети LSTM, а в главе 9 — новый тип модели авторегрессии — трансформер, который выводит моделирование текстов на новый уровень.

Расширения RNN

В предыдущем разделе был представлен простой пример обучения сети LSTM генерированию текста в заданном стиле. Здесь же рассмотрим несколько расширений этой идеи.

Многослойные рекуррентные сети

Сеть, которую мы только что рассмотрели, содержала всего один слой LSTM, но аналогичным образом можно обучать сети с множеством слоев LSTM, чтобы извлечь из текста более глубокие особенности. Для этого нужно просто добавить еще один слой LSTM после первого. В этом случае второй слой LSTM сможет использовать скрытые состояния первого слоя в качестве входных данных. Структура такой сети показана на рис. 5.11, а общая архитектура модели — в табл. 5.2.

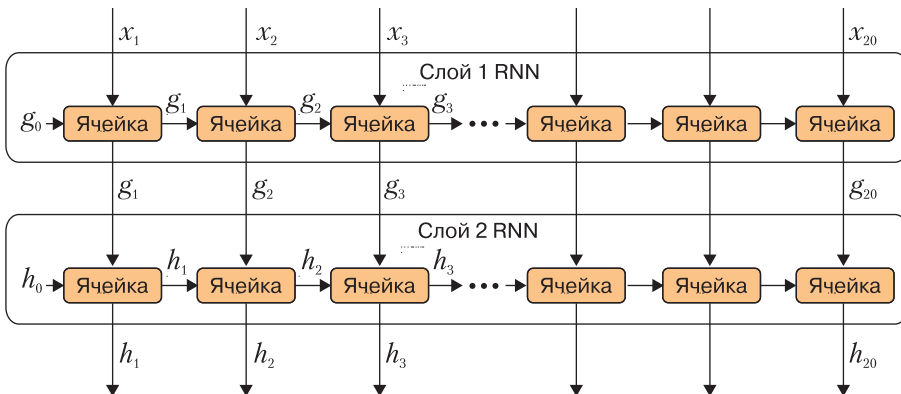


Рис. 5.11. Схема многослойной рекуррентной сети: g_t обозначают скрытые состояния в первом слое, а h_t — во втором слое

Таблица 5.2. Сводная информация о дискриминаторе

Слой (тип)	Выходная форма	Число параметров
Input	(None, None)	0
Embedding	(None, None, 100)	1 000 000
LSTM	(None, None, 128)	117 248
LSTM	(None, None, 128)	131 584
Dense	(None, None, 10000)	1 290 000

Всего параметров 2 538 832

Обучаемых параметров 2 538 832

Необучаемых параметров 0

Код, конструирующий многослойную сеть LSTM, показан в примере 5.10.

Пример 5.10. Конструирование многослойной сети LSTM

```
text_in = layers.Input(shape = (None,))
embedding = layers.Embedding(total_words, embedding_size)(text_in)
x = layers.LSTM(n_units, return_sequences = True)(x)
x = layers.LSTM(n_units, return_sequences = True)(x)
probabilites = layers.Dense(total_words, activation = 'softmax')(x)
model = models.Model(text_in, probabilites)
```

Управляемые рекуррентные блоки

Другой тип слоев, широко используемых в RNN, — *управляемые рекуррентные блоки* (Gated Recurrent Unit, GRU) [Cho et al., 2014]. Вот основные его отличия от LSTM.

1. *Входной* фильтр и *фильтр забывания* замещаются фильтрами *сброса* и *обновления*.
2. Отсутствуют *состояние ячейки* и *выходной* фильтр, имеется только *скрытое состояние*, выводимое ячейкой.

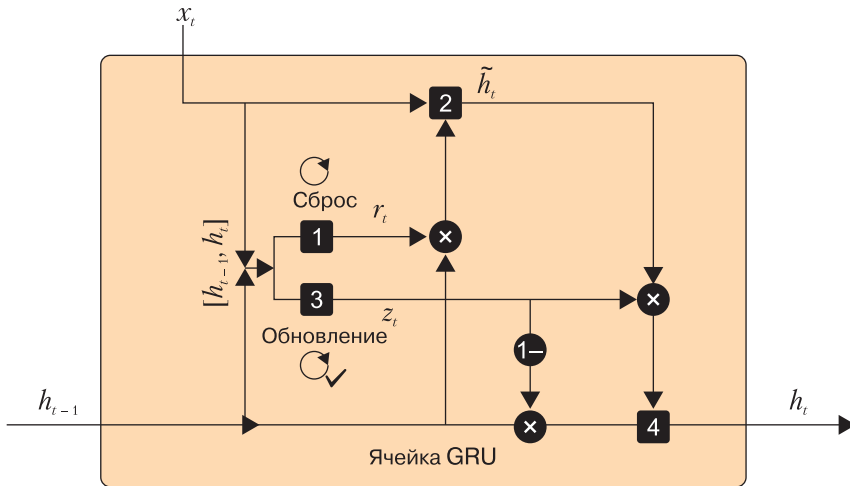
Скрытое состояние обновляется в четыре приема (рис. 5.12).

Вот как это происходит.

1. Скрытое состояние предыдущего временного шага h_{t-1} и векторное представление текущего слова x_t объединяются и используются для создания фильтра *сброса*. Он организован как полносвязный слой с матрицей весов W_r и функцией активации sigmoid. Вектор результата r_t имеет длину, равную

количеству узлов в ячейке, и хранит значения в диапазоне от 0 до 1, которые определяют, какая доля предыдущего скрытого состояния h_{t-1} должна применяться для вычисления новых убеждений ячейки.

2. Фильтр сброса применяется к скрытому состоянию h_{t-1} , и результат объединяется с векторным представлением текущего слова x_t . Затем объединенный вектор передается в полносвязный слой с матрицей весов W и функцией активации $\tanh$, а на выходе получается вектор $\tilde{h}_t$, в котором хранятся новые убеждения ячейки. Он имеет длину, равную количеству узлов в ячейке, и хранит значения от -1 до 1 .
3. Результат объединения скрытого состояния предыдущего временного шага h_{t-1} и векторного представления текущего слова x_t используется также для создания фильтра обновления. Он организован как полносвязный слой с матрицей весов W_z и функцией активации sigmoid . Вектор результата z_t имеет длину, равную количеству узлов в ячейке, и хранит значения в диапазоне от 0 до 1, которые определяют, какая доля новых убеждений $\tilde{h}_t$ будет смешана с текущим скрытым состоянием h_{t-1} .
4. Новые убеждения ячейки $\tilde{h}_t$ и текущее скрытое состояние h_{t-1} смешиваются в пропорции, определяемой фильтром обновления z_t , в результате чего получается обновленное скрытое состояние h_t , выводимое из ячейки.



- 1 $r_t = \sigma(W_r[h_{t-1}, x_t])$
- 2 $\tilde{h}_t = \tanh(W[\tilde{r}h_{t-1}, x_t])$
- 3 $z_t = \sigma(W_z[h_{t-1}, x_t])$
- 4 $h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t$

Рис. 5.12. Единственная ячейка GRU

Двунаправленные ячейки

Для задач прогнозирования, когда модели доступен весь текст для анализа, нет оснований обрабатывать последовательность только в прямом направлении — это можно сделать и в обратном направлении. Воспользоваться этим преимуществом позволяет слой `Bidirectional`. Он хранит два набора скрытых состояний: одно является результатом обработки последовательности в прямом направлении, другое — в обратном. Таким образом, слой может обучаться на информации, и предшествующей заданному временному шагу, и следующей за ним. В Keras это реализуется как обертка вокруг рекуррентного слоя (пример 5.11).

Пример 5.11. Создание двунаправленного слоя GRU

```
layer = layers.Bidirectional(layers.GRU(100))
```



Скрытое состояние

Скрытые состояния в получившемся слое — это векторы, длина которых равна удвоенному количеству узлов в обернутой ячейке (объединение прямого и обратного скрытого состояния). То есть скрытые состояния слоя в примере — это векторы длиной 200.

До сих пор мы применяли модели авторегрессии (LSTM) только к текстовым данным. В следующем разделе увидим, что модели авторегрессии можно использовать и для создания изображений.

PixelCNN

В 2016 году ван ден Оорд (van den Oord) с коллегами [Oord et al., 2016] представили модель, которая генерирует изображения попиксельно, прогнозируя вероятность появления следующего пиксела по предшествующим. Модель называется *PixelCNN*, и ее можно обучить генерировать изображения авторегрессионным способом.

Чтобы разобраться с PixelCNN, нам нужно познакомиться с двумя новыми концепциями: *маскированными сверточными слоями* и *остаточными блоками*.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/05_autoregressive/02_pixelcnn/pixelcnn.ipynb` в репозитории книги. Он был заимствован (и адаптирован) из превосходного руководства по PixelCNN (<https://keras.io/examples/generative/pixelcnn>), написанного Адриеном Моро (Adrien Moreau) и доступного на сайте Keras.

Маскированные сверточные слои

Как мы видели в главе 2, сверточный слой можно использовать для извлечения признаков из изображения путем применения последовательности фильтров. Результатом слоя для конкретного пиксела является взвешенная сумма весов фильтра, умноженная на значения из предыдущего слоя для небольшой квадратной окрестности с центром в пикселе. Этот метод позволяет обнаруживать края и текстуры, а в более глубоких слоях — формы и объекты высокого уровня.

Сверточные слои очень эффективно выявляют признаки, но их нельзя использовать напрямую в авторегрессионном смысле, поскольку пикселы не упорядочены. Они полагаются на тот факт, что все пикселы обрабатываются одинаково — ни один из них не рассматривается как *начало* или *конец* изображения. Это отличает их от текстовых данных, которые мы уже видели в этой главе, где лексемы четко упорядочены, благодаря чему есть возможность применять рекуррентные модели, такие как LSTM.

Чтобы получить возможность задействовать сверточные слои для генерации изображений в авторегрессионном смысле, нужно сначала упорядочить пикселы и гарантировать, что фильтры увидят только те из них, которые предшествуют рассматриваемому. Затем мы можем генерировать изображения пиксел за пикселем, применяя к текущему изображению сверточные фильтры, которые будут предсказывать значение следующего пиксела на основе предшествующих.

Сначала выберем порядок расположения пикселов, например, слева вправо сверху вниз, перемещаясь сначала по горизонтали, а затем по вертикали. После этого наложим маску на сверточные фильтры, чтобы на выходные данные слоя в каждом пикселе влияли только значения предшествующих ему пикселов. Этого можно добиться умножением маски из единиц и нулей на матрицу весов фильтра, чтобы обеспечить обнуление значений любых пикселов, следующих за целевым.

На самом деле в PixelCNN используются маски двух типов (рис. 5.13):

- тип А маскирует значение центрального пиксела;
- тип В не маскирует значение центрального пиксела.

Исходный маскированный сверточный слой (то есть непосредственно применяющийся к входному изображению) не может использовать центральный пиксел, потому что именно его сеть должна угадать! Однако последующие слои могут задействовать центральный пиксел, поскольку его значение будет ими вычислено только на основе информации из предыдущих пикселов исходного входного изображения.

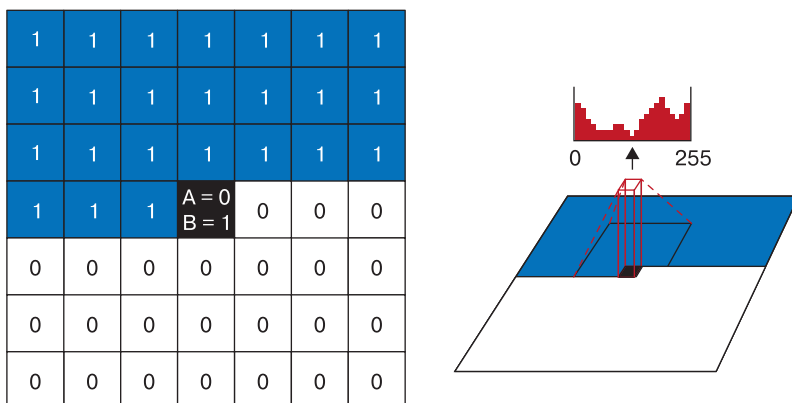


Рис. 5.13. Слева: маска сверточного фильтра; справа: маска, применяемая к набору пикселей для предсказания распределения значения центрального пиксела (источник: [Oord et al., 2016], <https://arxiv.org/pdf/1606.05328>)

В примере 5.12 показано, как создать `MaskedConvLayer` с помощью Keras.

Пример 5.12. `MaskedConvLayer` в Keras

```
class MaskedConvLayer(layers.Layer):
    def __init__(self, mask_type, **kwargs):
        super(MaskedConvLayer, self).__init__()
        self.mask_type = mask_type
        self.conv = layers.Conv2D(**kwargs) ❶

    def build(self, input_shape):
        self.conv.build(input_shape)
        kernel_shape = self.conv.kernel.get_shape()
        self.mask = np.zeros(shape=kernel_shape) ❷
        self.mask[: kernel_shape[0] // 2, ...] = 1.0 ❸
        self.mask[kernel_shape[0] // 2, : kernel_shape[1] // 2, ...] = 1.0 ❹
        if self.mask_type == "B":
            self.mask[kernel_shape[0] // 2, kernel_shape[1] // 2, ...] = 1.0 ❺

    def call(self, inputs):
        self.conv.kernel.assign(self.conv.kernel * self.mask) ❻
        return self.conv(inputs)
```

- ❶ Слой `MaskedConvLayer` основан на обычном слое `Conv2D`.
- ❷ Маска инициализируется нулями.
- ❸ Пиксели в предыдущих строках демаскируются единицами.
- ❹ Пиксели в предыдущих столбцах, находящиеся в одной строке, демаскируются единицами.

- 5 Если маска имеет тип В, центральный пиксел демаскируется единицей.
- 6 Веса фильтра умножаются на маску.

Обратите внимание: в этом упрощенном примере предполагается, что изображение черно-белое, то есть имеет один канал. Для обработки цветных изображений потребуется три цветовых канала, по которым тоже можно организовать упорядочение так, чтобы, например, красный канал предшествовал синему, а тот, в свою очередь, — зеленому.

Остаточные блоки

Теперь, увидев, как замаскировать сверточный слой, приступим к созданию PixelCNN. Основным строительным блоком будет остаточный блок.

Остаточный блок (residual block) — это набор слоев, которые добавляют выходные данные к входным перед передачей в остальную часть сети. Другими словами, входные данные могут передаваться по *короткому пути* на выход без прохождения через промежуточные слои — это называется *пропускающим соединением* (skip connection). Суть приема заключается в следующем: если оптимальное преобразование состоит в том, чтобы сохранить входные данные неизменными, этого можно достичь, просто обнулив веса промежуточных слоев. Без пропускающего соединения сети пришлось бы отыскивать в промежуточных слоях отображения, сохраняющие идентичность, что гораздо сложнее.

На рис. 5.14 показано, как устроен остаточный блок в нашей сети PixelCNN.

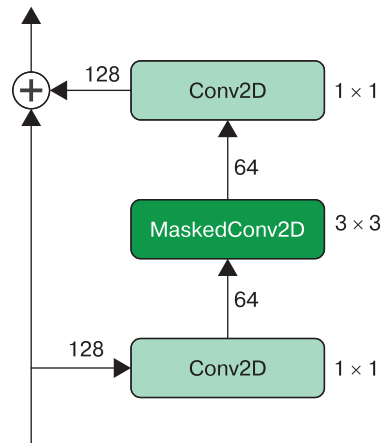


Рис. 5.14. Остаточный блок в PixelCNN (числа у стрелок показывают количество фильтров, а числа возле блоков — размеры фильтров)

В примере 5.13 показано, как создать ResidualBlock.

Пример 5.13. ResidualBlock

```
class ResidualBlock(layers.Layer):
    def __init__(self, filters, **kwargs):
        super(ResidualBlock, self).__init__(**kwargs)
        self.conv1 = layers.Conv2D(
            filters=filters // 2, kernel_size=1, activation="relu"
        ) ❶
```

```

self.pixel_conv = MaskedConv2D(
    mask_type="B",
    filters=filters // 2,
    kernel_size=3,
    activation="relu",
    padding="same",
) ❷
self.conv2 = layers.Conv2D(
    filters=filters, kernel_size=1, activation="relu"
) ❸

def call(self, inputs):
    x = self.conv1(inputs)
    x = self.pixel_conv(x)
    x = self.conv2(x)
    return layers.add([inputs, x]) ❹

```

❶ Начальный слой Conv2D уменьшает количество каналов вдвое.

❷ Слой MaskedConv2D типа B с размером ядра 3 использует информацию только из пяти пикселей — трех пикселей в строке над целевым пикселем, одного слева и самого целевого пикселя.

❸ Последний слой Conv2D удваивает количество каналов, чтобы снова соответствовать входной форме.

❹ Выходные данные сверточных слоев добавляются к входным — это пропускающее соединение.

Обучение PixelCNN

В примере 5.14 мы собрали всю сеть PixelCNN, примерно следуя структуре, изложенной в оригинальной статье. В той статье выходным является слой Conv2D с 256 фильтрами и функцией активации softmax. Другими словами, сеть пытается воссоздать входные данные, предсказывая значения пикселей подобно автокодировщику. Разница заключается в том, что PixelCNN ограничена так, что никакая информация из более ранних пикселей не может пройти и повлиять на прогноз для каждого пикселя из-за особенностей архитектуры сети — наличия слоев MaskedConv2D.

Проблема этого подхода в том, что сеть не может понять, что значение пикселя, скажем, 200 очень близко к значению 201. Она должна изучить выходные значения всех пикселей независимо, а это значит, что обучение может протекать очень медленно даже для самых простых наборов данных. Поэтому в своей реализации мы пошли на упрощение, согласно которому каждый пиксел может принимать только одно из четырех значений. Благодаря этому можем использовать выходной слой Conv2D с четырьмя фильтрами вместо 256.

Пример 5.14. Архитектура PixelCNN

```

inputs = layers.Input(shape=(16, 16, 1)) ❶
x = MaskedConv2D(mask_type="A"
                  , filters=128
                  , kernel_size=7
                  , activation="relu"
                  , padding="same")(inputs) ❷

for _ in range(5):
    x = ResidualBlock(filters=128)(x) ❸

for _ in range(2):
    x = MaskedConv2D(
        mask_type="B",
        filters=128,
        kernel_size=1,
        strides=1,
        activation="relu",
        padding="valid",
    )(x) ❹

out = layers.Conv2D(
    filters=4, kernel_size=1, strides=1, activation="softmax", padding="valid"
)(x) ❺

pixel_cnn = models.Model(inputs, out) ❻

adam = optimizers.Adam(learning_rate=0.0005)
pixel_cnn.compile(optimizer=adam, loss="sparse_categorical_crossentropy")

pixel_cnn.fit(
    input_data
    , output_data
    , batch_size=128
    , epochs=150
) ❼

```

❶ Входные данные модели — это черно-белое изображение размером $16 \times 16 \times 1$ с входными значениями, масштабированными в диапазон от 0 до 1.

❷ Первый слой `MaskedConv2D` типа А с размером ядра 7 использует информацию из 24 пикселей — 21 пикселя в трех рядах над целевым пикселем и трех слева от него (сам целевой пиксел не применяется).

❸ Далее друг за другом следуют пять групп слоев `ResidualBlock`.

❹ Два слоя `MaskedConv2D` типа В с размером ядра 1 действуют как слой `Dense` по количеству каналов в каждом пикселе.

❺ Последний слой `Conv2D` уменьшает количество каналов до четырех — количества значений пикселей в этом примере.

- ❶ Создается модель, принимающая и возвращающая изображения тех же размеров.
- ❷ Обучение модели — входные данные `input_data` масштабируются в диапазон $[0, 1]$ (числа с плавающей точкой), выходные данные `output_data` масштабируются в диапазон $[0, 3]$ (целые числа).

Анализ PixelCNN

Попробуем обучить свою модель PixelCNN на изображениях из набора Fashion-MNIST, с которым мы встречались в главе 3. Чтобы сгенерировать новые изображения, нужно попросить модель предсказать каждый следующий пиксел на основе всех предыдущих. Это очень медленный процесс по сравнению с такой моделью, как вариационный автокодировщик! Для черно-белых изображений 32×32 нужно последовательно получить 1024 прогноза. Сравните это с получением одного прогноза при использовании VAE. Это один из основных недостатков таких моделей авторегрессии, как PixelCNN, — они работают очень медленно из-за последовательного характера процесса выборки.

Чтобы ускорить создание новых изображений, мы применяем размер изображения 16×16 , а не 32×32 . Класс обратного вызова, генерирующий изображения, показан в примере 5.15.

Пример 5.15. Генерация новых изображений с помощью PixelCNN

```
class ImageGenerator(callbacks.Callback):
    def __init__(self, num_img):
        self.num_img = num_img

    def sample_from(self, probs, temperature):
        probs = probs ** (1 / temperature)
        probs = probs / np.sum(probs)
        return np.random.choice(len(probs), p=probs)

    def generate(self, temperature):
        generated_images = np.zeros(
            shape=(self.num_img,) + (pixel_cnn.input_shape)[1:]
        ) ❶
        batch, rows, cols, channels = generated_images.shape

        for row in range(rows):
            for col in range(cols):
                for channel in range(channels):
                    probs = self.model.predict(generated_images)[
                        :, row, col, :
                    ] ❷
                    generated_images[:, row, col, channel] = [
                        self.sample_from(x, temperature) for x in probs
                    ] ❸
```



```

        generated_images[:, row, col, channel] /= 4 ④
    return generated_images

def on_epoch_end(self, epoch, logs=None):
    generated_images = self.generate(temperature = 1.0)
    display(
        generated_images,
        save_to = "./output/generated_img_%03d.png" % (epoch)
    )

img_generator_callback = ImageGenerator(num_img=10)

```

- ❶ Процесс начинается с партии пустых изображений (значения всех пикселей равны нулю).
- ❷ Цикл по строкам, столбцам и каналам текущего изображения с прогнозированием распределения значения следующего пикселя.
- ❸ Получение значений пикселей из прогнозируемого распределения (в нашем примере — значений из диапазона $[0, 3]$).
- ❹ Преобразование значений пикселей в диапазон $[0, 1]$ и подстановка значения пикселя в текущем изображении для подготовки к следующей итерации цикла.

На рис. 5.15 показаны несколько изображений из исходного обучающего набора, а также сгенерированные моделью PixelCNN.



Рис. 5.15. Примеры изображений из обучающего набора и сгенерированных моделью PixelCNN

Модель прекрасно воссоздает общую форму и стиль исходных изображений! Но самое удивительное, что теперь мы можем рассматривать изображения как серию лексем (значений пикселей) и применять модели авторегрессии, такие как PixelCNN, для создания реалистичных изображений.

Как упоминалось ранее, один из недостатков авторегрессионных моделей — низкая скорость выборки, поэтому в книге представлен простой пример их применения. Однако, как будет показано в главе 10, можно создавать более сложные формы авторегрессионных моделей и применять их к изображениям для получения более впечатляющих результатов. В таких случаях низкая скорость генерации является неизбежной платой за результаты исключительно высокого качества.

С момента публикации оригинальной статьи в архитектуру и процесс обучения PixelCNN было внесено несколько улучшений. В следующем разделе представлено одно из этих изменений — использование смешанных распределений — и показано, как обучить модель PixelCNN с помощью этого улучшения, задействуя TensorFlow.

Смесь распределений

В предыдущем примере мы уменьшили разнообразие выходных данных PixelCNN до четырех значений в пикселях, чтобы гарантировать, что модели не придется изучать распределение 256 независимых значений и это не замедлит процесс ее обучения. Однако полученный результат далек от идеала: для цветных изображений было бы желательно, чтобы холст был ограничен лишь несколькими возможными цветами.

Для решения этой проблемы выходные данные сети можно представить как *смесь распределений* 256 дискретных значений пикселей вместо softmax, в соответствии с идеями, представленными Салимансом (Salimans) и его коллегами [Salimans et al., 2017]. Смесь распределений — это просто смесь двух или более распределений вероятностей. Например, смесь распределений может состоять из пяти логистических распределений, каждое из которых имеет разные параметры. Смесь распределений также требует дискретного многоуровневого распределения, обозначающего вероятность выбора из каждого распределения, включенного в смесь. Пример показан на рис. 5.16.

Чтобы получить выборку из смеси распределений, сначала нужно выполнить выборку из многоуровневого распределения и узнать, какое конкретное под-распределение использовать, а затем сделать выборку из него обычным спосо-

бом. Так можно создавать сложные распределения с относительно небольшим количеством параметров.

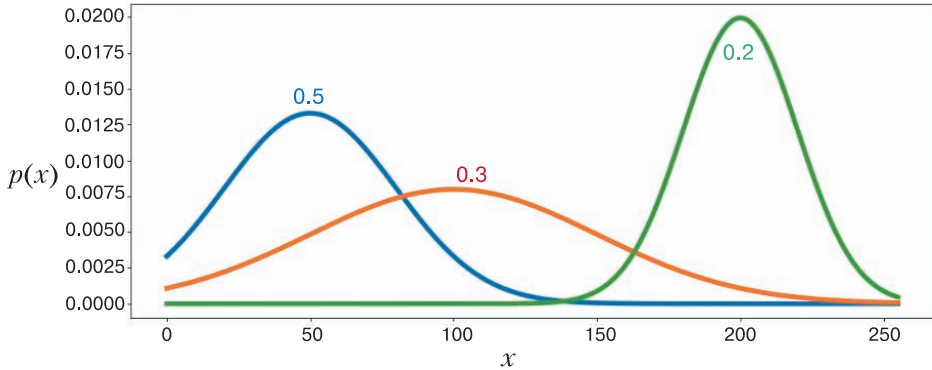


Рис. 5.16. Смесь трех нормальных распределений с разными параметрами — многоуровневое распределение для этой смеси имеет вид [0.5, 0.3, 0.2]

Например, для описания смеси распределений, представленной на рис. 5.16, требуется всего восемь параметров — два для многоуровневого распределения, а также среднее значение и дисперсия для каждого из трех нормальных распределений. Сравните это с 255 параметрами, которые определяют многоуровневое распределение по всему диапазону значений пикселей.

Самое замечательное то, что библиотека TensorFlow Probability предоставляет функцию, позволяющую одной строкой кода создать PixelCNN со смесью распределений на выходе. Пример 5.16 демонстрирует, как построить PixelCNN с помощью этой функции.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу notebooks/05_autoregressive/03_pixelcnn_md/pixelcnn_md.ipynb в репозитории книги.

Пример 5.16. Создание PixelCNN с помощью функции из библиотеки TensorFlow

```
import tensorflow_probability as tfp

dist = tfp.distributions.PixelCNN(
    image_shape=(32, 32, 1),
    num_resnet=1,
    num_hierarchies=2,
```

```

    num_filters=32,
    num_logistic_mix=5,
    dropout_p=.3,
) ❶

image_input = layers.Input(shape=(32, 32, 1)) ❷

log_prob = dist.log_prob(image_input)

model = models.Model(inputs=image_input, outputs=log_prob) ❸
model.add_loss(-tf.reduce_mean(log_prob)) ❹

```

❶ Определение PixelCNN как распределения, то есть выходной слой этой сети является смесью из пяти логистических распределений.

❷ Входные данные — это черно-белые изображения размером $32 \times 32 \times 1$.

❸ Модель принимает черно-белое изображение и выводит логарифм правдоподобия изображения в смеси распределений, вычисленной PixelCNN.

❹ Функция потерь — это отрицательное среднее логарифма правдоподобия для группы входных изображений.

Модель обучается так же, как и раньше, но на этот раз принимает на входе целочисленные значения пикселей в диапазоне $[0, 255]$. Выходные данные можно сгенерировать из распределения с использованием функции `sample`, как показано в примере 5.17.

Пример 5.17. Выборка из смеси распределений PixelCNN

```
dist.sample(10).numpy()
```

На рис. 5.17 приводятся примеры сгенерированных изображений. В отличие от предыдущих примеров теперь используется весь диапазон значений пикселей.



Рис. 5.17. Изображения, сгенерированные PixelCNN из полученной смеси распределений

Резюме

В этой главе мы выяснили, как применять авторегрессионные модели, такие как рекуррентные нейронные сети, для генерации текстовых последовательностей, имитирующих определенный стиль письма, а также как с помощью PixelCNN генерировать изображения последовательным способом, подавая на вход по одному пикселу за раз.

Мы исследовали два разных типа рекуррентных слоев: долгой краткосрочной памяти (LSTM) и управляемых рекуррентных блоков (GRU) — и поговорили о том, как эти ячейки можно компоновать или делать двунаправленными для формирования сложных сетевых архитектур. С помощью Keras мы сконструировали LSTM для создания реалистичных описаний рецептов и увидели, как манипулировать температурой процесса выборки, чтобы увеличить или уменьшить случайность выходных данных.

Вы также увидели, как авторегрессионным способом генерировать изображения, используя PixelCNN. Создали свою сеть PixelCNN с нуля, включив в нее маскированные сверточные слои и остаточные блоки, чтобы информация могла проходить через сеть и помогала генерировать очередной пиксел только на основе предыдущих. Наконец, вы познакомились с функцией PixelCNN из библиотеки TensorFlow Probability, реализующей смесь распределений в выходном слое, что позволяет еще больше усовершенствовать процесс обучения.

В следующей главе рассмотрим еще одно семейство генеративных моделей, которое явно моделирует распределение, генерирующее данные, — модели нормализующих потоков.

ГЛАВА 6

Модели нормализующих потоков

В этой главе вы:

- узнаете, как модели нормализующих потоков используют уравнение замены переменных;
- увидите, почему определитель якобиана играет жизненно важную роль в вычислении явной функции плотности;
- узнаете, как ограничить форму якобиана, используя слои связи;
- познакомитесь с устройством обратимой нейронной сети;
- создадите модель RealNVP — конкретный пример процесса нормализации для создания точек в двумерном пространстве;
- используете модель RealNVP для создания новых точек, которые выглядят полученными из исходного распределения;
- познакомитесь с двумя ключевыми расширениями модели RealNVP: GLOW и FFJORD.

К настоящему моменту мы обсудили три семейства генеративных моделей: вариационные автокодировщики, генеративно-состязательные сети и авторегрессионные модели. Каждое из них представляет определенный способ решения задачи моделирования распределения $p(x)$ либо путем введения скрытой переменной, которую легко выбрать (и преобразовать с помощью декодировщика в VAE или генератора в GAN), либо путем моделирования распределения как функции значений предыдущих элементов (авторегрессионные модели).

В этой главе мы рассмотрим новое семейство генеративных моделей — модели нормализующих потоков. Как вы увидите далее, нормализующие потоки имеют сходство как с авторегрессионными моделями, так и с вариационными автокодировщиками. Подобно моделям авторегрессии, нормализующие потоки способны явно и понятно моделировать распределение, генерирующее данные $p(x)$. Как и VAE, нормализующие потоки пытаются отобразить данные в более простое распределение, например распределение Гаусса. Ключевое отличие состоит

в том, что нормализующие потоки накладывают ограничения на форму функции отображения: в частности, она должна быть обратимой и, следовательно, может использоваться для создания новых точек данных.

В первом разделе этой главы мы подробно исследуем теоретические основы нормализующих потоков, а затем перейдем к реализации модели под названием RealNVP с использованием Keras. Мы также увидим, как можно расширить идею нормализующих потоков для создания более мощных моделей, таких как GLOW и FFJORD.

Введение

Начнем с короткой истории, чтобы проиллюстрировать ключевые понятия нормализующих потоков.

ЯКОБ И МАШИНА F.L.O.W.

Приехав в небольшую деревню, вы замечаете загадочный магазин с вывеской «Якоб». Заинтригованные, вы осторожно входите и спрашиваете стоящего за прилавком старика, что он продает (рис. 6.1).



Рис. 6.1. Внутри антикварного магазина с большим металлическим колоколом (создано с помощью Midjourney, <https://midjourney.com/>)

Он отвечает, что предлагает услуги по неточной оцифровке картин. Быстро осмотрев стеллаж за своей спиной, старик достает серебряную коробку с тиснением в виде букв F.L.O.W. Он говорит вам, что эта аббревиатура расшифровывается как Finding Likenesses Of Watercolors («поиск тождественности картин»)¹, что приблизительно описывает то, что делает машина. Вы решаете опробовать ее.

На следующий день вы приходите в магазин и передаете его владельцу набор своих любимых картин, а он закладывает их в машину. Машина F.L.O.W. начинает гудеть и свистеть и через какое-то время выдает набор чисел, кажущихся случайными. Владелец магазина вручает вам список и идет к кассе, чтобы подсчитать, сколько вы ему должны за процесс оцифровки и коробку F.L.O.W. Совершенно не впечатленные, вы спрашиваете продавца, что вам делать с этим длинным списком цифр и как вернуть свои картины. Владелец магазина закатывает глаза, будто вы спросили несусветную глупость. Он возвращается к автомату и вводит длинный список цифр, на этот раз с противоположной стороны. Вы снова слышите жужжание машины, и спустя какое-то время оригинальные картины появляются в приемном лотке.

Получив свои картины обратно, вы с облегчением решаете, что, возможно, лучше просто хранить их на чердаке. Однако прежде, чем вы успеваете шагнуть за порог, продавец приглашает вас в другой конец магазина, где со стропил свисает гигантский колокол. Он ударяет в него, вызывая вибрацию по всему магазину.

В то же мгновение машина F.L.O.W. у вас под мышкой начинает шипеть и жужжать, будто только что был введен новый набор цифр. Через несколько мгновений из нее начинают выпадать удивительные акварельные картины, но не те, которые вы изначально оцифровали. Новые картины по стилю и форме напоминают ваши, но каждая из них совершенно уникальна!

Вы спрашиваете продавца, как работает это невероятное устройство. Он объясняет: волшебство заключается в специально разработанном процессе, который гарантирует, что преобразование будет выполняться чрезвычайно быстро, но при этом расчеты достаточно сложны для того, чтобы преобразовать вибрации, производимые колоколом, в сложные узоры и формы, напоминающие ранее оцифрованные картины.

Осознав потенциал этого изобретения, вы поспешно платите за устройство и выходите из магазина, радуясь, что теперь у вас есть возможность создавать новые картины в любимом стиле, для чего нужно лишь зайти в магазин, ударить в колокол и ждать, когда машина F.L.O.W. сотворит чудо!

История Якоба и машины F.L.O.W. представляет модели нормализующих потоков. Давайте подробнее рассмотрим теорию, лежащую в основе процесса нормализации, прежде чем перейти к практическому примеру с использованием Keras.

¹ Здесь игра слов: flow в английском языке означает «поток» (напомню, что в этой главе речь идет о потоковых моделях), поэтому расшифровку Finding Likenesses Of Watercolors можно перевести как «Поиск Тождественности Картин» — «поток»). — *Примеч. пер.*

Нормализующие потоки

Идея моделей нормализующих потоков аналогична идее вариационных автокодировщиков, которые мы исследовали в главе 3. Напомню, что модель вариационного автокодировщика обучает *кодировщик* отображать сложное распределение в гораздо более простое, из которого можно осуществлять выборку, а затем обучает *декодировщик* выполнять обратное отображение из более простого распределения в сложное, чтобы дать возможность выбрать точку в более простом распределении и применить выявленное преобразование для создания новой точки в исходном сложном распределении. Говоря языком вероятностей, декодировщик моделирует $p(x|z)$, а кодировщик лишь аппроксимирует $q(z|x)$ в истинное распределение $p(z|x)$, причем кодировщик и декодировщик — это две совершенно разные нейронные сети.

В модели нормализующего потока функция декодирования должна быть точной обратной версией функции кодирования и быстро вычисляться, что обеспечивает управляемость нормализующих потоков. Однако нейронные сети по умолчанию не являются обратимыми функциями. В результате возникает вопрос: как, используя гибкость и широкие возможности глубокого обучения, создать обратимый процесс, который преобразует сложное распределение (например, распределение данных набора акварельных картин) в гораздо более простое (например, колоколообразное распределение Гаусса)?

Чтобы ответить на этот вопрос, сначала нужно разобраться с приемом, известным как *замена переменных*. В этом разделе исследуем простой двумерный пример, чтобы вы могли в мельчайших деталях увидеть, как именно работают нормализующие потоки. Более сложные примеры — это лишь расширение представленных здесь основных методов.

Замена переменных

Допустим, у нас есть распределение вероятностей $p_X(x)$, определенное на прямоугольной области X в двух измерениях, $x = (x_1, x_2)$, как показано на рис. 6.2.

Эта функция интегрируется до 1 по области распределения (то есть x_1 в диапазоне $[1, 4]$ и x_2 в диапазоне $[0, 2]$), поэтому она представляет четко определенное распределение вероятностей. Мы можем записать это следующим образом:

$$\int_0^2 \int_1^4 p_X(x) dx_1 dx_2 = 1.$$

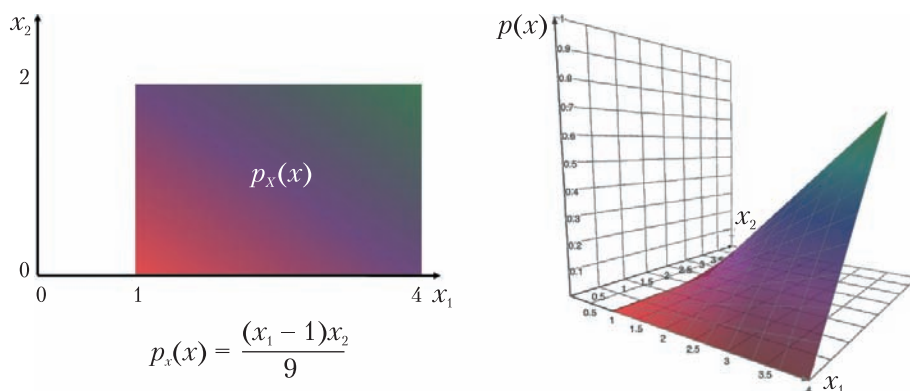


Рис. 6.2. Распределение вероятностей $p_x(x)$, определенное в двух измерениях: *слева* — в двумерном пространстве; *справа* — в трехмерном

Допустим, мы хотим сдвинуть и масштабировать это распределение так, чтобы оно определялось на единичном квадрате Z . Добиться этого можно, определив новую переменную $z = (z_1, z_2)$ и функцию f , отображающую каждую точку в X ровно в одну точку в Z :

$$z = f(x); z_1 = \frac{x_1 - 1}{3}; z_2 = \frac{x_2}{2}.$$

Обратите внимание на то, что эта функция *обратима*, то есть существует функция g , которая отображает каждую точку z обратно в соответствующую ей точку x . Это важно для замены переменных, поскольку в противном случае мы не сможем отображать точки между пространствами туда и обратно. Мы можем найти g , просто переставив уравнения, определяющие f (рис. 6.3).

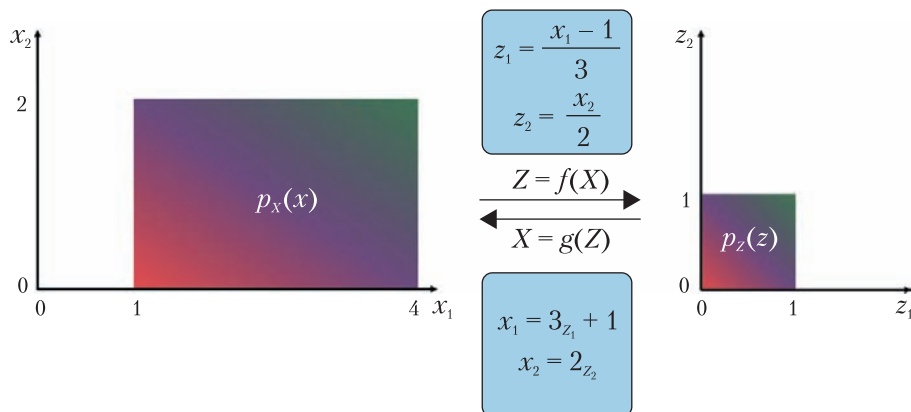


Рис. 6.3. Замена переменных между X и Z

Теперь посмотрим, как замена переменных с X на Z влияет на распределение вероятностей $pX(x)$. Для этого подставим уравнения, определяющие g , в $pX(x)$, чтобы получить функцию $pZ(z)$, которая определяется через z :

$$p_Z(z) = \frac{((3z_1 + 1) - 1)(2z_2)}{9} = \frac{2z_1 z_2}{3}.$$

Однако, если теперь проинтегрировать $p_Z(z)$ на единичном квадрате, мы видим, что у нас появилась проблема:

$$\int_0^1 \int_0^1 \frac{2z_1 z_2}{3} dz_1 dz_2 = \frac{1}{6}.$$

Преобразованная функция $p_Z(z)$ больше не является действительным распределением вероятностей, потому что интегрируется только в $1/6$. Чтобы получить преобразование из сложного распределения вероятностей в более простое, откуда мы сможем осуществлять выборку, необходимо гарантировать, что оно интегрируется в 1.

Недостающий коэффициент 6 обусловлен тем, что область преобразованного распределения вероятностей в шесть раз меньше исходной области — исходный прямоугольник X имел площадь 6 и был сжат в единичный квадрат Z , площадь которого равна 1. Следовательно, нужно умножить новое распределение вероятностей на коэффициент нормализации, равный отношению изменению площади (или объема в более высоких измерениях).

К счастью, изменение объема для данного преобразования можно вычислить — это абсолютное значение определителя якобиана матрицы преобразования. Давайте посмотрим, что это такое.

Определитель якобиана

Якобиан функции $z = f(x)$ — это матрица ее частных производных первого порядка:

$$J = \frac{\partial z}{\partial x} = \begin{bmatrix} \frac{\partial z_1}{\partial x_1} & \cdots & \frac{\partial z_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial z_m}{\partial x_1} & \cdots & \frac{\partial z_m}{\partial x_n} \end{bmatrix}.$$

Наш пример — лучший способ объяснить все это. Если взять частную производную от z_1 по x_1 , получим $1/3$. Если взять частную производную z_1 по x_2 , получим 0. Аналогично, если взять частную производную z_2 по x_1 , получим 0.

Наконец, если взять частную производную z_2 по x_2 , получим $1/2$. Следовательно, матрица якобиана нашей функции $f(x)$ имеет вид:

$$J = \begin{pmatrix} \frac{1}{3} & 0 \\ 0 & \frac{1}{2} \end{pmatrix}.$$

Определитель имеют только квадратные матрицы, и он равен объему параллелепипеда со знаком, созданного применением представленного матрицей преобразования в единичный (гипер)куб. Таким образом, в двух измерениях это просто площадь параллелограмма со знаком, созданного применением представленного матрицей преобразования в единичный квадрат.

Существует общая формула (<https://oreil.ly/FuDCf>) для вычисления определителя матрицы с n измерениями, которая имеет сложность $\mathcal{O}(n^3)$. Для нашего примера нужна формула для двух измерений, которая выглядит так:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc.$$

Соответственно, в нашем примере определитель якобиана равен $1/3 \times 1/2 - 0 \times 0 = 1/6$. Это тот самый масштабный коэффициент $1/6$, который нам нужен, чтобы гарантировать, что распределение вероятностей после преобразования по-прежнему интегрируется в 1!



По определению определитель имеет знак, то есть может быть отрицательным. Поэтому, чтобы получить относительное изменение объема, нужно взять абсолютное значение определителя якобиана.

Уравнение замены переменных

Теперь мы можем записать уравнение, описывающее процесс замены переменных между X и Z . Оно известно как *уравнение замены переменных*:

$$p_X(x) = p_Z(z) \left| \det \left(\frac{\partial z}{\partial x} \right) \right|. \quad (6.1)$$

Как это поможет нам построить генеративную модель? Главное — понять, что если $p_Z(z)$ — это простое распределение, из которого легко сделать выборку (например, гауссово распределение), то теоретически нам нужно лишь найти подходящую обратимую функцию $f(x)$, способную отображать дан-

ные из X в Z , и соответствующую обратную функцию $g(z)$, которую можно использовать для отображения выбранной точки z обратно в точку x в исходной области. Чтобы найти точную и понятную формулу распределения данных $p(x)$, мы можем задействовать предыдущее уравнение, включающее определитель якобиана.

Однако практическое применение этого подхода сопряжено с двумя основными проблемами, которые необходимо решить.

Во-первых, вычисление определителя многомерной матрицы — процедура, чрезвычайно затратная в вычислительном отношении. Она имеет сложность $\mathcal{O}(n^3)$. Ее реализация лишена практического смысла, так как даже небольшие черные изображения размером 32×32 пиксела имеют 1024 измерения.

Во-вторых, не совсем понятно, как вычислять обратимую функцию $f(x)$. Мы могли бы использовать нейронную сеть для поиска некоторой функции $f(x)$, но не сможем обратить эту сеть — нейронные сети работают только в одном направлении!

Чтобы решить эти две проблемы, необходимо задействовать нейронную сеть с особой архитектурой, которая гарантирует обратимость функции замены переменных f и наличие легко вычисляемого определителя.

В следующем разделе мы увидим, как это сделать, используя метод, называемый *вещественнозначным преобразованием без сохранения объема* (realvalued non-volume preserving transformation, RealNVP).

RealNVP

Модель RealNVP была впервые представлена Динхом (Dinh) с коллегами в 2017 году [Dinh et al., 2016]. В своей статье авторы показывают, как построить нейронную сеть, способную преобразовать сложное распределение данных в простое гауссово распределение и обладающую желаемыми свойствами обратимости и наличием якобиана, который легко вычислить.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/06_normflow/01_realnvp/realnvp.ipynb` в репозитории книги. Этот код был заимствован (и адаптирован) из превосходного руководства по RealNVP (<https://oreil.ly/ZjjwP>), написанного Джорджо Марией Мандолини (Giorgio Maria Mandolini) с коллегами и доступного на сайте Keras.

Набор данных Two Moons

В этом примере мы используем набор данных, созданный функцией `make_moons` из библиотеки `sklearn` для Python. Функция создает зашумленный набор точек в двумерном пространстве, напоминающий два полумесяца (рис. 6.4).

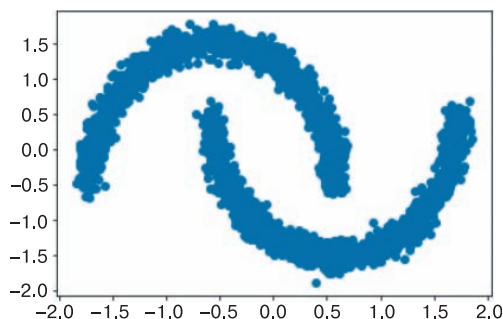


Рис. 6.4. Набор двумерных данных Two Moons

Код, создающий этот набор данных, показан в примере 6.1.

Пример 6.1. Создание набора данных Two Moons

```
data = datasets.make_moons(3000, noise=0.05)[0].astype("float32") ❶  
norm = layers.Normalization()  
norm.adapt(data)  
normalized_data = norm(data) ❷
```

❶ Создает зашумленный ненормализованный набор данных, который включает 3000 точек.

❷ Нормализация набора данных, чтобы привести его к распределению со средним значением 0 и стандартным отклонением 1.

Мы построим модель RealNVP, способную генерировать точки в двумерном пространстве, которые соответствуют распределению точек в наборе данных Two Moons. Это очень простой пример, но он поможет понять, как фактически работает модель нормализующего потока.

Однако сначала введем новый тип слоя, называемый слоем связи.

Слой связи

Слой связи генерирует коэффициенты масштабирования и переноса для каждого элемента входных данных. Иначе говоря, он генерирует два тензора того же размера, что и входные данные: один для масштабного коэффициента, другой для коэффициента переноса (рис. 6.5).

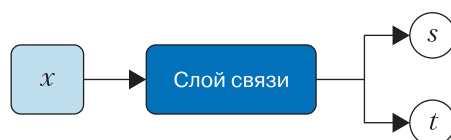


Рис. 6.5. Слой связи генерирует два тензора той же формы, что и входные данные: коэффициент масштабирования (s) и коэффициент переноса (t)

Чтобы создать слой **Coupling** для нашего простого примера, можно построить два набора слоев **Dense** — для создания выходных коэффициентов масштабирования и выходных коэффициентов переноса, как показано в примере 6.2.



Для работы с изображениями блоки слоя **Coupling** используют слои **Conv2D** вместо слоев **Dense**.

Пример 6.2. Слой **Coupling** в Keras

```
def Coupling():
    input_layer = layers.Input(shape=2) ❶

    s_layer_1 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(input_layer) ❷
    s_layer_2 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(s_layer_1)
    s_layer_3 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(s_layer_2)
    s_layer_4 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(s_layer_3)
    s_layer_5 = layers.Dense(
        2, activation="tanh", kernel_regularizer=regularizers.l2(0.01)
    )(s_layer_4) ❸

    t_layer_1 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(input_layer) ❹
    t_layer_2 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(t_layer_1)
    t_layer_3 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(t_layer_2)
    t_layer_4 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(t_layer_3)
```

```
t_layer_5 = layers.Dense(
    2, activation="linear", kernel_regularizer=regularizers.l2(0.01)
)(t_layer_4) ❸

return models.Model(inputs=input_layer, outputs=[s_layer_5, t_layer_5]) ❹
```

- ❶ Слой Coupling в примере принимает двумерные данные.
- ❷ Масштабирующий поток — это стопка слоев Dense с 256 узлами.
- ❸ Последний масштабирующий слой имеет два узла и функцию активации `tanh`.
- ❹ Поток переноса — это стопка слоев Dense с 256 узлами.
- ❺ Последний слой переноса имеет два узла и функцию активации `linear`.
- ❻ Слой Coupling конструируется как экземпляр Model с двумя выходами (коэффициентами масштабирования и переноса).

Обратите внимание на то, что количество каналов временно увеличивается, чтобы можно было изучить более сложное представление, прежде чем снова свернуть его до количества каналов, имеющихся во входных данных. В оригинальной статье авторы дополнительно добавляют регуляризаторы к каждому слою, чтобы штрафовать их за большие веса.

Передача данных через слой связи

Архитектура слоя связи не представляет особого интереса — его уникальность в том, как входные данные маскируются и преобразуются при прохождении через слой (рис. 6.6).

Обратите внимание на то, что в первый слой связи передаются только первые d измерений данных, а остальные $D - d$ полностью маскируются, то есть устанавливаются равными нулю. В нашем простом примере с $D = 2$ выбор $d = 1$ означает, что вместо двух значений (x_1, x_2) слой связи увидит $(x_1, 0)$.

На выходе слой возвращает коэффициенты масштабирования и переноса. Они снова маскируются, но на этот раз используется маска, *обратная* предыдущей, соответственно, пропускаются только вторые половинки, то есть в нашем примере получаем $(0, s_2)$ и $(0, t_2)$. Затем они поэлементно применяются ко второй половине входных данных x_2 , а первая половина входных дан-

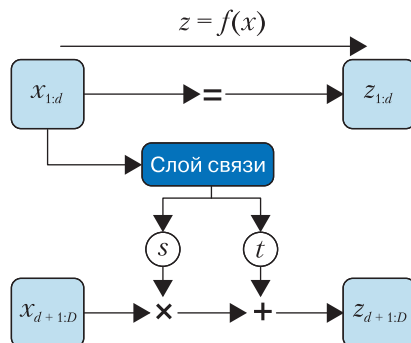


Рис. 6.6. Процесс преобразования x при прохождении через слой связи

ных x_1 передается напрямую, вообще не обновляясь. В результате для вектора с размерностью D , где $d < D$, уравнения обновления выглядят так:

$$\begin{aligned} z_{1:d} &= x_{1:d}; \\ z_{d+1:D} &= x_{d+1:D} \odot \exp(s(x_{1:d})) + t(x_{1:d}). \end{aligned}$$

Возможно, вам покажется странным, что мы утруждаем себя созданием слоя, который маскирует так много информации. Однако причина становится понятной, если исследовать структуру матрицы якобиана этой функции:

$$\frac{\partial z}{\partial x} = \begin{bmatrix} \mathbf{I} & 0 \\ \frac{\partial z_{d+1:D}}{\partial x_{1:d}} & \text{diag}(\exp[s(x_{1:d})]) \end{bmatrix}.$$

Верхняя левая подматрица $d \times d$ — это обычная единичная матрица, поскольку $z_{1:d} = x_{1:d}$. Эти элементы передаются напрямую без обновления. Соответственно, верхняя правая подматрица равна 0, потому что $z_{1:d}$ не зависит от $x_{d+1:D}$.

Нижняя левая подматрица довольно сложная, и мы не стремимся ее упростить. Нижняя правая подматрица — это просто диагональная матрица, заполненная элементами $\exp(s(x_{1:d}))$, потому что $z_{d+1:D}$ линейно зависит от $x_{d+1:D}$, а градиент зависит только от масштабного коэффициента и не зависит от коэффициента переноса. На рис. 6.7 показана диаграмма этой матричной формы, где ненулевые элементы выделены цветом.

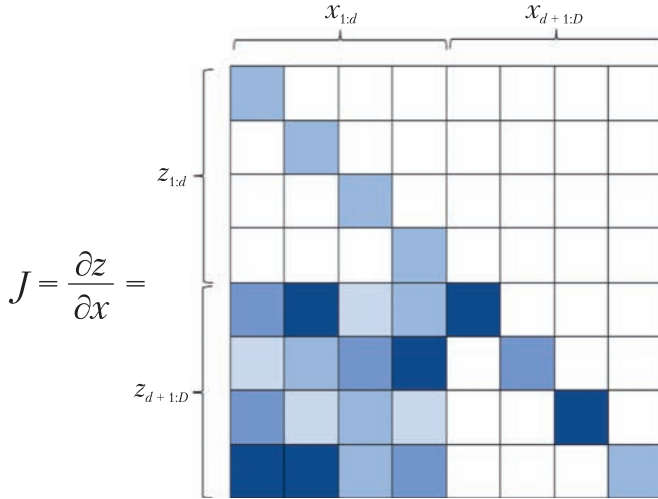


Рис. 6.7. Матрица якобиана преобразования — нижнетреугольная матрица, определитель которой равен произведению диагональных элементов

Обратите внимание на то, что выше диагонали нет ненулевых элементов — по этой причине такая матричная форма называется *нижнетреугольной*. Теперь очевидно преимущество данного структурирования матрицы: определитель нижнетреугольной матрицы равен простому произведению диагональных элементов. Другими словами, определитель не зависит ни от одной из комплексных производных в нижней левой подматрице!

Формулу нахождения определителя этой матрицы можно записать следующим образом:

$$\det(J) = \exp \left[\sum_j s(x_{1:d})_j \right].$$

Ее легко вычислить, что было одной из двух первоначальных целей построения модели нормализующего потока.

Другая цель заключалась в обратимости функции. Мы видим, что и это условие выполняется, поскольку обратную функцию можно записать, просто переупорядочив уравнения в прямой функции:

$$\begin{aligned} x_{1:d} &= z_{1:d}; \\ x_{d+1:D} &= (z_{d+1:D} - t(x_{1:d})) \odot \exp(-s(x_{1:d})). \end{aligned}$$

Эквивалентная диаграмма показана на рис. 6.8.

Теперь у нас есть почти все необходимое для построения модели RealNVP. Однако остается одна проблема: как обновить первые d элементов во входных данных? На текущий момент модель оставляет их без изменения!

Добавление слоев связи

Для решения этой проблемы можно использовать простой трюк. Если расположить слои связи друг за другом с чередованием шаблона маскировки, то данные, оставленные без изменения в одном слое, будут обновлены в следующем. Эта архитектура дает дополнительное преимущество — возможность изучения более сложных представлений данных, так как она образует более глубокую нейронную сеть.

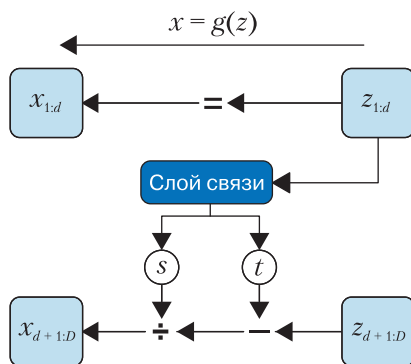


Рис. 6.8. Обратная функция $x = g(z)$

Якобиан такого сочетания слоев связи по-прежнему легко вычислить, поскольку линейная алгебра говорит нам, что определитель матричного произведения является произведением определителей. Точно так же обратное сочетание двух функций — это просто сочетание обратных функций, как показано в следующих уравнениях:

$$\det(A \cdot B) = \det(A) \det(B);$$

$$(f_b \circ f_a)^{-1} = f_a^{-1} \circ f_b^{-1}.$$

Следовательно, если добавить дополнительные слои связи, каждый раз перевернув маску, то можно построить нейронную сеть, способную преобразовать весь входной тензор и сохраняющую основные свойства: простоту вычисления определителя якобиана и обратимость. На рис. 6.9 показана общая структура.

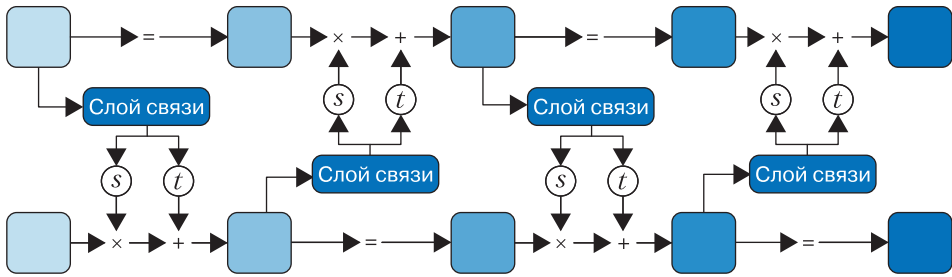


Рис. 6.9. Добавление дополнительных слоев связи

Обучение модели RealNVP

Теперь, создав модель RealNVP, можем обучить ее на наборе данных Two Moons. Не забывайте, что мы должны минимизировать отрицательную логарифмическую вероятность данных $-\log p_X(x)$. Опираясь на уравнение (6.1), можем записать следующее:

$$-\log p_X(x) = -\log p_Z(z) - \log \left| \det \left(\frac{\partial z}{\partial x} \right) \right|.$$

На роль целевого распределения $p_Z(z)$, получаемого на выходе прямого процесса f , выбрано стандартное гауссово распределение, потому что из него легко сделать выборку. Затем выбранную точку можно преобразовать обратно в пространство исходных изображений, применив обратный процесс g (рис. 6.10).

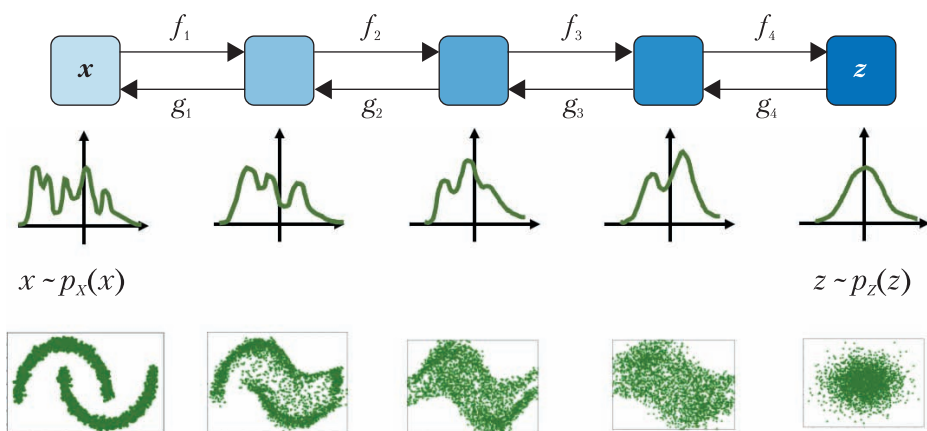


Рис. 6.10. Преобразование сложного распределения $p_X(x)$ и простого гауссова распределения $p_Z(z)$ в одномерное (средний ряд) и двумерное (нижний ряд)

В примере 6.3 показано, как построить сеть RealNVP в форме модели `Model` с помощью Keras.

Пример 6.3. Построение модели RealNVP в Keras

```
class RealNVP(models.Model):
    def __init__(self, input_dim, coupling_layers, coupling_dim, regularization):
        super(RealNVP, self).__init__()
        self.coupling_layers = coupling_layers
        self.distribution = tfp.distributions.MultivariateNormalDiag(
            loc=[0.0, 0.0], scale_diag=[1.0, 1.0]
        ) ①
        self.masks = np.array(
            [[0, 1], [1, 0]] * (coupling_layers // 2), dtype="float32"
        ) ②
        self.loss_tracker = metrics.Mean(name="loss")
        self.layers_list = [
            Coupling(input_dim, coupling_dim, regularization)
            for i in range(coupling_layers)
        ] ③

    @property
    def metrics(self):
        return [self.loss_tracker]

    def call(self, x, training=True):
        log_det_inv = 0
        direction = 1
        if training:
            direction = -1
        for i in range(self.coupling_layers)[::-direction]: ④
```

```

        x_masked = x * self.masks[i]
        reversed_mask = 1 - self.masks[i]
        s, t = self.layers_list[i](x_masked)
        s *= reversed_mask
        t *= reversed_mask
        gate = (direction - 1) / 2
        x = (
            reversed_mask
            * (x * tf.exp(direction * s) + direction * t * tf.exp(gate * s))
            + x_masked
        ) ❸
        log_det_inv += gate * tf.reduce_sum(s, axis = 1) ❹
    return x, log_det_inv

def log_loss(self, x):
    y, logdet = self(x)
    log_likelihood = self.distribution.log_prob(y) + logdet ❺
    return -tf.reduce_mean(log_likelihood)

def train_step(self, data):
    with tf.GradientTape() as tape:
        loss = self.log_loss(data)
        g = tape.gradient(loss, self.trainable_variables)
        self.optimizer.apply_gradients(zip(g, self.trainable_variables))
        self.loss_tracker.update_state(loss)
    return {"loss": self.loss_tracker.result()}

def test_step(self, data):
    loss = self.log_loss(data)
    self.loss_tracker.update_state(loss)
    return {"loss": self.loss_tracker.result()}

model = RealNVP(
    input_dim = 2
    , coupling_layers= 6
    , coupling_dim = 256
    , regularization = 0.01
)

model.compile(optimizer=optimizers.Adam(learning_rate=0.0001))

model.fit(
    normalized_data
    , batch_size=256
    , epochs=300
)

```

❶ Целевое распределение — это стандартное двумерное гауссово распределение.

❷ Здесь создается чередующийся шаблон маски.

❸ Список слоев `Coupling`, определяющих сеть `RealNVP`.

❹ Основная функция `call` сети. Здесь мы перебираем слои `Coupling`. Если `training=True`, значит, мы движемся вперед по слоям, то есть от данных к скрытому пространству. Если `training=False`, значит, движемся в обратном направлении, то есть от скрытого пространства к данным.

❺ Эта строка описывает прямое или обратное уравнения в зависимости от направления (попробуйте подставить `direction = -1` и `direction = 1`, чтобы убедиться в этом!).

❻ Логарифм определителя якобиана, необходимый для вычисления функции потерь, представляет собой простую сумму коэффициентов масштабирования.

❼ С учетом целевого гауссова распределения и логарифма определителя якобиана функция потерь вычисляется как отрицательная сумма логарифмов вероятностей преобразованных данных.

Анализ модели RealNVP

После обучения модель можно применять для преобразования обучающего набора в скрытое пространство (используя прямое направление f) и, что более важно, для преобразования выбранной точки в скрытом пространстве в точку, которая выглядит так, будто она могла быть выбрана из исходного распределения данных (применяя обратное направление g).

На рис. 6.11 показаны выходные данные сети до обучения: в обоих направлениях информация передается без каких-либо преобразований.

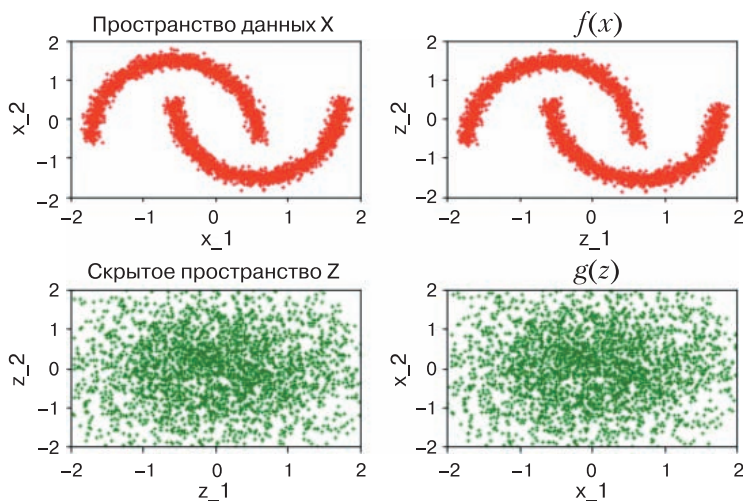


Рис. 6.11. Вход (слева) и выход (справа) модели RealNVP перед обучением для прямого (вверху) и обратного (внизу) процесса

После обучения (рис. 6.12) прямой процесс преобразует точки из обучающего набора в распределение, напоминающее гауссово. Аналогично обратный процесс преобразует точки, выбранные из гауссова распределения, обратно в пространство с распределением, напоминающим распределение исходных данных.

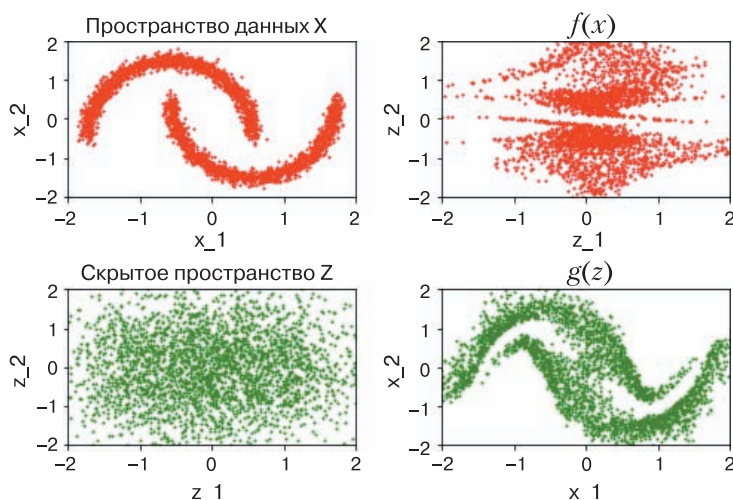


Рис. 6.12. Вход (слева) и выход (справа) модели RealNVP после обучения для прямого (вверху) и обратного (внизу) процесса

Кривая изменения потерь в процессе обучения показана на рис. 6.13.

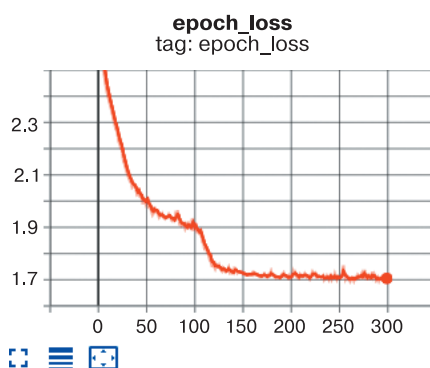


Рис. 6.13. Кривая изменения потерь в процессе обучения RealNVP

На этом мы завершаем обсуждение RealNVP — частного случая генеративной модели нормализующего потока. В следующем разделе рассмотрим некоторые современные модели, расширяющие идеи, представленные в статье о RealNVP.

Другие модели нормализующего потока

Двумя другими успешными и важными моделями нормализующего потока являются *GLOW* и *FFJORD*. В следующих разделах описаны их ключевые достижения.

GLOW

Представленная на конференции NeurIPS в 2018 году модель GLOW была одной из первых, продемонстрировавших способность нормализующих потоков генерировать высококачественные образцы и создавать значимое скрытое пространство, через которое можно манипулировать образцами. Ее ключевой особенностью была замена настройки обратной маскировки инвертируемыми сверточными слоями 1×1 . Например, при применении RealNVP к изображениям порядок каналов меняется после каждого шага, чтобы гарантировать, что сеть получит возможность преобразовать все входные данные. В GLOW, напротив, задействуется свертка 1×1 , которая действует как универсальный метод для создания любой перестановки каналов (рис. 6.14). Авторы показали, что даже с этим дополнением распределение в целом остается управляемым, с определителями и обратными им величинами, которые легко вычислить.



Рис. 6.14. Случайные образцы, сгенерированные моделью GLOW
(источник: [Kingma, Dhariwal, 2018], <https://arxiv.org/abs/1807.03039>)

FFJORD

RealNVP и GLOW — это дискретные нормализующие потоки, они преобразуют входные данные через дискретный набор слоев связи. Модель FFJORD (Free-Form Continuous Dynamics for Scalable Reversible Generative Models), представленная на конференции ICLR в 2019 году, показывает, как можно смоделировать преобразование в виде непрерывного во времени процесса (конечно же, установив ограничения, поскольку количество шагов в потоке стремится к бесконечности, а размер шага — к нулю). В этом случае динамика моделируется с помощью обыкновенного дифференциального уравнения (ОДУ), параметры которого определяются нейронной сетью (f_θ). Для решения ОДУ в момент времени t_1 , то есть для нахождения z_1 по некоторой начальной точке z_0 , выбранной из гауссовой функции в момент t_0 , используется решатель типа «черный ящик», как описано следующими уравнениями:

$$z_0 \sim p(z_0); \quad \frac{\partial z(t)}{\partial t} = f_\theta(x(t), t); \quad x = z_1.$$

Схема процесса преобразования показана на рис. 6.15.

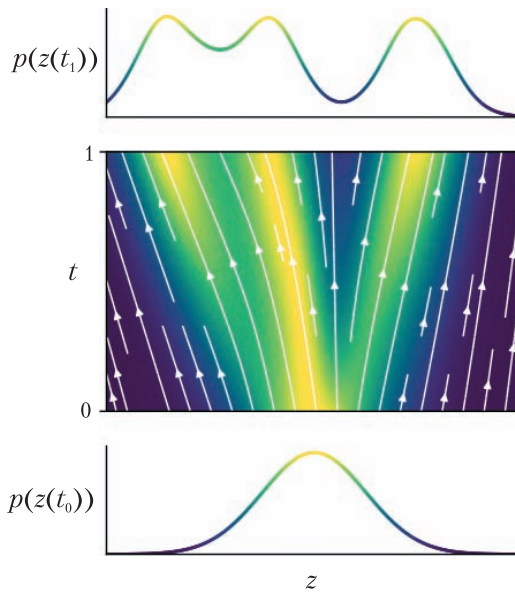


Рис. 6.15. Модели FFJORD выполняют преобразование между распределением исходных данных и стандартным гауссовым распределением с помощью обыкновенного дифференциального уравнения, параметризуемого нейронной сетью (источник: [Grathwohl et al., 2018], <https://arxiv.org/abs/1810.01367>)

Резюме

В этой главе мы исследовали модели нормализующего потока RealNVP, GLOW и FFJORD.

Модель нормализующего потока — это обратимая функция, определяемая нейронной сетью, которая позволяет напрямую моделировать плотность распределения данных посредством замены переменных. В общем случае уравнение замены переменных требует вычисления очень сложного определителя якобиана, что непрактично для большинства случаев, кроме самых простых.

Чтобы обойти эту проблему, модель RealNVP ограничивает форму нейронной сети так, чтобы она соответствовала двум основным критериям: была обратима и имела легко вычисляемый определитель якобиана. Это достигается за счет комбинации слоев связи, создающих на каждом этапе коэффициенты масштабирования и переноса. Важно отметить, что слой связи маскирует проходящие через сеть данные так, что якобиан имеет форму нижнетреугольной матрицы и, следовательно, легко вычисляемый определитель. Полная видимость входных данных достигается за счет инвертирования масок в каждом слое.

Операции масштабирования и переноса легко инвертируются, поэтому после обучения модели можно пропускать данные через сеть в обратном направлении, а это означает, что можно нацелить процесс прямого преобразования на стандартное гауссово распределение, из которого легко выполнить выборку. Затем можно пропустить выбранные точки в обратном направлении через сеть, чтобы сгенерировать новые наблюдения.

В статье, описывающей RealNVP, также показано, как применить эту модель к изображениям, используя свертки внутри слоев связи вместо полносвязных слоев. В статье, описывающей модель GLOW, эта идея была расширена, а также устранена необходимость в каких-либо жестко запрограммированных перестановках масок. Модель FFJORD представила концепцию непрерывных нормализующих потоков, моделирующих процесс преобразования как ОДУ, определяемое нейронной сетью.

В целом вы увидели, что нормализующие потоки — это мощное семейство моделей генеративного моделирования, способных создавать высококачественные образцы, сохраняя при этом возможность понятного описания функции плотности вероятности данных.

Модели на основе энергии

В этой главе вы:

- узнаете, как сформулировать глубокую модель на основе энергии (Energy-Based Model, EBM);
- увидите, как выполнить выборку из EBM с использованием динамики Ланжевена;
- обучите свою EBM с применением контрастивной дивергенции;
- проанализируете EBM, включая просмотр моментальных снимков процесса выборки с помощью динамики Ланжевена;
- познакомитесь с другими типами EBM, такими как ограниченные машины Больцмана.

Модели на основе энергии — это широкий класс генеративных моделей, заимствующих ключевую идею моделирования из физических систем, а именно возможность выражения вероятности события с помощью распределения Больцмана — специальной функции, нормализующей вещественнозначную энергетическую функцию в диапазон между 0 и 1. Это распределение было сформулировано в 1868 году Людвигом Больцманом (Ludwig Boltzmann), который применял его для описания газов, находящихся в тепловом равновесии.

В этой главе вы увидите, как использовать эту идею для того, чтобы обучить генеративную модель создавать изображения рукописных цифр. Вы познакомитесь с несколькими новыми концепциями, в том числе с контрастивной дивергенцией для обучения EBM и динамикой Ланжевена для выборки.

Введение

Начнем с короткого рассказа, иллюстрирующего ключевые концепции, лежащие в основе моделей на основе энергии.

СПОРТИВНЫЙ КЛУБ ПО ЛЕГКОЙ АТЛЕТИКЕ В ЛОНЖ-Е-ВЕНЕ

Дайана Микс — главный тренер команды по бегу на длинные дистанции в вымышленном французском городе Лонж-е-Вене. Она хорошо известна как отличный тренер и приобрела репутацию человека, способного превратить даже самых посредственных спортсменов в бегунов мирового класса (рис. 7.1).



Рис. 7.1. Тренер по бегу, воспитавшая несколько выдающихся атлетов
(создано с помощью Midjourney, <https://midjourney.com/>)

Ее методы основаны на оценке уровня энергии каждого спортсмена. За годы работы с самыми разными атлетами она выработала невероятную способность одним взглядом точно определять, сколько энергии осталось у конкретного бегуна после преодоления дистанции. Чем ниже уровень остаточной энергии, тем лучше: элитные спортсмены всегда отдавали все силы во время забега!

Чтобы поддерживать свои навыки на должном уровне, Дайана регулярно тренировалась, измеряя разницу в воспринимаемой ею энергии у знаменитых спортсменов и лучших спортсменов своего клуба. Она позаботилась о том, чтобы расхождение между ее прогнозами для этих двух групп было как можно больше, чтобы люди восприняли ее слова всерьез, если бы она сказала, что нашла в своем клубе настоящего элитного спортсмена.

Настоящим волшебством была ее способность превратить посредственного бегуна в стайера мирового класса. Процесс был прост: она оценивала текущий уровень энергии спортсмена и разрабатывала оптимальный набор корректировок, которые он должен был внести в свою манеру преодоления дистанции, чтобы в следующий раз

улучшить результаты. После внесения корректировок она снова оценивала уровень энергии спортсмена, убеждаясь, что он немного ниже, чем раньше, что объясняло улучшение результатов на дистанции. Процесс выбора оптимальных корректировок и выполнения небольшого шага в правильном направлении продолжался до тех пор, пока в конечном итоге спортсмен не переставал отличаться от бегуна мирового класса.

История Дайаны Микс и спортивного клуба в Лонж-е-Вене отражает ключевые идеи, лежащие в основе моделирования на основе энергии. Давайте сейчас подробно рассмотрим теорию, а затем реализуем практический пример с использованием Keras.

Модели на основе энергии

Модели на основе энергии пытаются моделировать истинное распределение, генерирующее данные, используя распределение Больцмана:

$$p(\mathbf{x}) = \frac{e^{-E(\mathbf{x})}}{\int_{\tilde{\mathbf{x}} \in \mathbf{X}} e^{-E(\tilde{\mathbf{x}})}}, \quad (7.1)$$

где $E(\mathbf{x})$ — это *функция энергии* (или *оценка*) наблюдения x .

На практике все сводится к обучению нейронной сети $E(\mathbf{x})$ выводить низкие оценки для вероятных наблюдений (значения $p(\mathbf{X})$ близко к 1) и высокие оценки — для маловероятных (значение $p(\mathbf{X})$ близко к 0).

Такое моделирование данных сопряжено с двумя проблемами. Во-первых, неясно, как использовать модель для выборки новых наблюдений, — мы можем применять ее для получения оценки по данному наблюдению, но как сгенерировать наблюдение с низкой оценкой, то есть правдоподобное?

Во-вторых, нормализующий знаменатель в уравнении (7.1) содержит интеграл, который разрешим только для самых простых задач. Не имея возможности вычислить его, мы не сможем использовать оценку максимального правдоподобия для обучения модели, поскольку для этого требуется правильное распределение вероятностей.

Ключевая идея моделей на основе энергии заключается в возможности применять методы аппроксимации, избавляющие от необходимости вычислять трудноразрешимый знаменатель. Это ключевое отличие от нормализующих потоков, где мы прилагаем все усилия, чтобы гарантировать, что преобразования, применяемые к стандартному гауссовому распределению, не изменяют

тот факт, что выходные данные по-прежнему являются действительным распределением вероятностей.

Сложную проблему неразрешимого знаменателя можно обойти, используя прием, называемый контрастивной дивергенцией (для обучения), и прием, называемый динамикой Ланжевена (для выборки), как описано в статье Ду (Du) и Мордатча (Mordatch) *Implicit Generation and Modeling with Energy-Based Models*, опубликованной в 2019 году [Du, Mordatch, 2019]. Далее в этой главе мы подробно изучим эти методы при создании собственной ЕВМ. Но сначала подготовим набор данных и спроектируем простую нейронную сеть, которая будет представлять нашу вещественнозначную энергетическую функцию $E(x)$.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/07_ebm/01_ebm/ebm.ipynb` в репозитории книги. Он был заимствован (и адаптирован) из превосходного руководства по глубоким генеративным моделям на основе энергии (<https://oreil.ly/kyO9B>), написанного Филлипом Липпе (Phillip Lippe).

Набор данных MNIST

В этой главе используем стандартный набор данных MNIST (<https://oreil.ly/mSvhc>), состоящий из черно-белых изображений рукописных цифр. Некоторые примеры изображений из этого набора данных показаны на рис. 7.2.

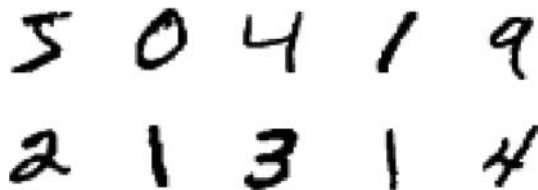


Рис. 7.2. Примеры изображений из набора данных MNIST

Набор данных поставляется в виде архива вместе с TensorFlow, поэтому его можно загрузить, как показано в примере 7.1.

Пример 7.1. Загрузка набора данных MNIST

```
from tensorflow.keras import datasets
(x_train, _), (x_test, _) = datasets.mnist.load_data()
```

Как обычно, масштабируем значения пикселей в диапазон $[-1, 1]$ и добавим поля, чтобы получить изображения размерами 32×32 пиксела. Преобразуем их в набор данных TensorFlow, как показано в примере 7.2.

Пример 7.2. Предварительная обработка набора данных MNIST

```
def preprocess(imgs):
    imgs = (imgs.astype("float32") - 127.5) / 127.5
    imgs = np.pad(imgs, ((0,0), (2,2), (2,2)), constant_values= -1.0)
    imgs = np.expand_dims(imgs, -1)
    return imgs

x_train = preprocess(x_train)
x_test = preprocess(x_test)
x_train = tf.data.Dataset.from_tensor_slices(x_train).batch(128)
x_test = tf.data.Dataset.from_tensor_slices(x_test).batch(128)
```

Теперь, получив набор данных, можно построить нейронную сеть, которая будет представлять функцию энергии $E(x)$.

Функция энергии

Функция энергии $E_\theta(x)$ — это нейронная сеть с параметрами θ , которая может трансформировать входное изображение x в скалярное значение. В этой сети мы используем функцию активации swish, которая описывается во врезке далее.

ФУНКЦИЯ АКТИВАЦИИ SWISH

Swish — это функция активации, альтернативная функции ReLU. Она представлена компанией Google в 2017 году [Ramachandran et al., 2017] и определяется следующим образом:

$$\text{swish}(x) = x \cdot \text{sigmoid}(x) = \frac{x}{e^{-x} + 1}.$$

График функции swish визуально похож на график ReLU, но в отличие от последнего он гладкий, что помогает решить проблему затухания градиента. Это особенно важно для моделей, основанных на энергии. График функции swish показан на рис. 7.3.

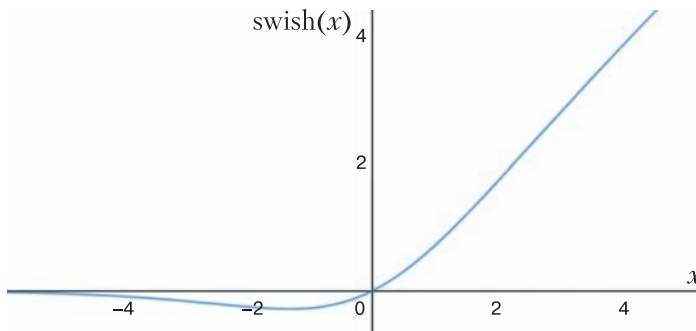


Рис. 7.3. График функции активации swish

Сеть образована стопкой слоев Conv2D, которые постепенно уменьшают размер изображения, одновременно увеличивая количество каналов. Последний слой представлен единственным полносвязным узлом с линейной активацией, поэтому сеть может выводить значения в диапазоне $(-\infty, \infty)$. Код, конструирующий сеть, показан в примере 7.3.

Пример 7.3. Конструирование нейронной сети функции энергии $E(\cdot)$

```
ebm_input = layers.Input(shape=(32, 32, 1))
x = layers.Conv2D(
    16, kernel_size=5, strides=2, padding="same", activation = activations.swish
)(ebm_input) ❶
x = layers.Conv2D(
    32, kernel_size=3, strides=2, padding="same", activation = activations.swish
)(x)
x = layers.Conv2D(
    64, kernel_size=3, strides=2, padding="same", activation = activations.swish
)(x)
x = layers.Conv2D(
    64, kernel_size=3, strides=2, padding="same", activation = activations.swish
)(x)
x = layers.Flatten()(x)
x = layers.Dense(64, activation = activations.swish)(x)
ebm_output = layers.Dense(1)(x) ❷
model = models.Model(ebm_input, ebm_output) ❸
```

❶ Функция энергии реализована как стопка слоев Conv2D с функцией активации swish.

❷ Последний слой представлен единственным полносвязным узлом с линейной активацией.

❸ Модель Keras, преобразующая входное изображение в скалярное значение энергии.

Выборка с использованием динамики Ланжевена

Функция энергии выводит оценку только для заданных входных данных. Так как же с помощью этой функции создавать новые образцы с низкой оценкой энергии?

Для этой цели мы будем применять метод, называемый *динамикой Ланжевена*, основанный на возможности вычисления градиента функции энергии по отношению к ее входным данным. Начав со случайной точки в пространстве выборки и делая короткие шаги в направлении, противоположном вычисленному градиенту, мы будем постепенно уменьшать функцию энергии. Если нейронная сеть обучена правильно, то случайный шум должен на наших глазах превратиться в изображение, напоминающее наблюдение из обучающей выборки!



Стохастическая градиентная динамика Ланжевена

Важно отметить, что по мере перемещения по пространству выборки необходимо добавлять во входные данные немного случайного шума, иначе есть риск попасть в локальный минимум. Эта техника известна как стохастическая градиентная динамика Ланжевена [Welling, Teh, 2011].

Этот градиентный спуск для двумерного пространства со значением функции энергии в третьем измерении можно визуализировать так, как показано на рис. 7.4. Ломаная линия представляет спуск вниз по зашумленному отрицательному градиенту функции энергии $E(x)$ относительно входного параметра x . Изображения в наборе данных MNIST состоят из 1024 пикселей, поэтому фактически мы перемещаемся в 1024-мерном пространстве, но принципы остаются неизменными!

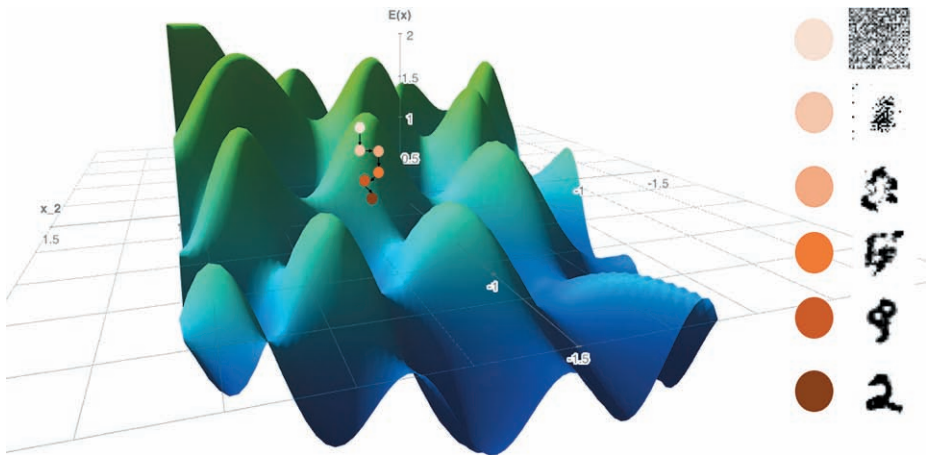


Рис. 7.4. Градиентный спуск с использованием динамики Ланжевена

Стоит отметить разницу между этим и другим видом градиентного спуска, который обычно используется для обучения нейронной сети.

При обучении нейронной сети мы вычисляем градиент *функции потерь* относительно *параметров* сети (то есть весов) на этапе обратного распространения ошибки. Затем немного корректируем параметры в отрицательном направлении градиента, чтобы за множество итераций постепенно минимизировать потери.

При использовании динамики Ланжевена мы сохраняем веса нейронной сети *фиксированными* и вычисляем градиент *выхода* по отношению к *входу*. Затем немного корректируем входные данные в направлении отрицательного градиента, благодаря чему на протяжении многих итераций постепенно минимизируем

выходные данные (оценку энергии). Оба процесса основаны на одном и том же принципе — градиентном спуске, но применяются к разным функциям и по отношению к разным сущностям.

Формально динамику Ланжевена можно описать следующим уравнением:

$$x^k = x^{k-1} - \eta \nabla_x E_\theta(x^{k-1}) + \omega,$$

где $\omega \sim \mathcal{N}(0, \sigma)$ и $x^0 \sim \mathcal{U}(-1, 1)$. η — это гиперпараметр, определяющий величину шага, который необходимо настроить: если сделать шаг слишком большим, то алгоритм будет перешагивать через минимумы, если слишком маленьким, то алгоритм будет сходиться слишком медленно.



$x^0 \sim \mathcal{U}(-1, 1)$ — это равномерное распределение в диапазоне $[-1, 1]$.

В примере 7.4 показано, как можно реализовать функцию выборки Ланжевена.

Пример 7.4. Функция выборки Ланжевена

```
def generate_samples(model, inp_imgs, steps, step_size, noise):
    imgs_per_step = []
    for _ in range(steps): ❶
        inp_imgs += tf.random.normal(inp_imgs.shape, mean = 0, stddev = noise) ❷
        inp_imgs = tf.clip_by_value(inp_imgs, -1.0, 1.0)
        with tf.GradientTape() as tape:
            tape.watch(inp_imgs)
            out_score = -model(inp_imgs) ❸
        grads = tape.gradient(out_score, inp_imgs) ❹
        grads = tf.clip_by_value(grads, -0.03, 0.03)
        inp_imgs += -step_size * grads ❺
        inp_imgs = tf.clip_by_value(inp_imgs, -1.0, 1.0)
    return inp_imgs
```

- ❶ Цикл по заданному количеству шагов.
- ❷ Добавление небольшого количества шума в изображение.
- ❸ Передача изображения в модель для получения оценки энергии.
- ❹ Вычисление градиента выхода по отношению ко входу.
- ❺ Добавление небольшой доли градиента во входное изображение.

Обучение с контрастивной дивергенцией

Теперь, узнав, как из пространства выборки получить новую точку с низкой энергией, рассмотрим процесс обучения модели.

Мы не можем применить оценку максимального правдоподобия, потому что функция энергии выводит не вероятность, а оценку, которая не интегрируется в 1 по всему выборочному пространству. Вместо этого мы применим метод, впервые предложенный в 2002 году Джеффри Хинтоном (Geoffrey Hinton), называемый *контрастивной дивергенцией* и предназначенный для обучения ненормализованных оценочных моделей [Hinton, 2002].

Значение, которое нужно минимизировать (как обычно), — отрицательное логарифмическое правдоподобие данных:

$$\mathcal{L} = -\mathbb{E}_{\mathbf{x} \sim \text{data}} [\log p_{\theta}(\mathbf{x})].$$

Когда $p_{\theta}(\mathbf{x})$ имеет вид распределения Больцмана с функцией энергии $E_{\theta}(\mathbf{x})$, можно показать, что градиент этой величины записывается следующим образом (полный вывод приводится в статье Оливера Вудфорда (Oliver Woodford) *Notes on Contrastive Divergence* [Woodford, 2006]):

$$\nabla_{\theta} \mathcal{L} = \mathbb{E}_{\mathbf{x} \sim \text{data}} [\nabla_{\theta} E_{\theta}(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim \text{model}} [\nabla_{\theta} E_{\theta}(\mathbf{x})].$$

Суть здесь в следующем: обучить модель выводить большие отрицательные оценки энергии для реальных наблюдений и большие положительные оценки энергии для сгенерированных наблюдений, чтобы разность между этими двумя крайностями была как можно большей.

Иначе говоря, мы можем вычислить разность между оценками энергии реальных и сгенерированных образцов и использовать ее в качестве функции потерь.

Чтобы вычислить оценки энергии сгенерированных образцов, нужно иметь возможность делать выборку точно из распределения $p_{\theta}(\mathbf{x})$, что невозможно из-за чрезмерно сложного знаменателя. Но мы можем задействовать свою процедуру выборки Ланжевена и с ее помощью создать набор наблюдений с низкими оценками энергии. Чтобы получить идеальную выборку, процесс должен выполнить бесконечно много шагов (что, очевидно, непрактично), поэтому мы просто выполняем некоторое конечное количество шагов, которого, как предполагается, достаточно, чтобы получить значимую функцию потерь.

Мы храним буфер с образцами, полученными на предыдущих итерациях, чтобы использовать его в качестве отправной точки для следующего пакета вместо чистого случайного шума. Код создания буфера показан в примере 7.5.

Пример 7.5. Буфер

```
class Buffer:
    def __init__(self, model):
        super().__init__()
        self.model = model
        self.examples = [
            tf.random.uniform(shape = (1, 32, 32, 1)) * 2 - 1
            for _ in range(128)
        ] ❶

    def sample_new_exmps(self, steps, step_size, noise):
        n_new = np.random.binomial(128, 0.05) ❷
        rand_imgs = (
            tf.random.uniform((n_new, 32, 32, 1)) * 2 - 1
        )
        old_imgs = tf.concat(
            random.choices(self.examples, k=128-n_new), axis=0
        ) ❸
        inp_imgs = tf.concat([rand_imgs, old_imgs], axis=0)
        inp_imgs = generate_samples(
            self.model, inp_imgs, steps=steps, step_size=step_size, noise = noise
        ) ❹
        self.examples = tf.split(inp_imgs, 128, axis = 0) + self.examples ❺
        self.examples = self.examples[:8192]
        return inp_imgs
```

❶ Изначально буфер образцов инициализируется пакетом случайного шума.

❷ Каждый раз с нуля генерируется в среднем 5 % наблюдений, то есть случайный шум.

❸ Остальные наблюдения извлекаются случайным образом из существующего буфера.

❹ Наблюдения объединяются и пропускаются через семплер Ланжевена.

❺ Полученная выборка добавляется в буфер, который обрезается до максимальной длины 8192 наблюдения.

На рис. 7.5 показан один шаг обучения методом контрастивной дивергенции. Алгоритм понижает оценки реальных наблюдений и повышает оценки сгенерированных наблюдений, не заботясь о нормализации этих оценок после каждого шага.

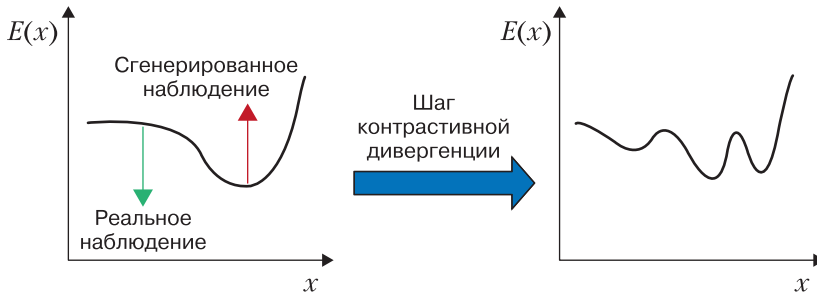


Рис. 7.5. Один шаг алгоритма контрастивной дивергенции

В примере 7.6 показано, как реализовать шаг обучения методом контрастивной дивергенции с использованием собственной модели Keras.

Пример 7.6. Обучение EBM методом контрастивной дивергенции

```
class EBM(models.Model):
    def __init__(self):
        super(EBM, self).__init__()
        self.model = model
        self.buffer = Buffer(self.model)
        self.alpha = 0.1
        self.loss_metric = metrics.Mean(name="loss")
        self.reg_loss_metric = metrics.Mean(name="reg")
        self.cdiv_loss_metric = metrics.Mean(name="cdiv")
        self.real_out_metric = metrics.Mean(name="real")
        self.fake_out_metric = metrics.Mean(name="fake")

    @property
    def metrics(self):
        return [
            self.loss_metric,
            self.reg_loss_metric,
            self.cdiv_loss_metric,
            self.real_out_metric,
            self.fake_out_metric
        ]

    def train_step(self, real_imgs):
        real_imgs += tf.random.normal(
            shape=tf.shape(real_imgs), mean = 0, stddev = 0.005
        ) ❶
        real_imgs = tf.clip_by_value(real_imgs, -1.0, 1.0)
        fake_imgs = self.buffer.sample_new_exmps(
            steps=60, step_size=10, noise = 0.005
        ) ❷
```

```

inp_imgs = tf.concat([real_imgs, fake_imgs], axis=0)
with tf.GradientTape() as training_tape:
    real_out, fake_out = tf.split(self.model(inp_imgs), 2, axis=0) ❸
    cdiv_loss = tf.reduce_mean(fake_out, axis = 0) - tf.reduce_mean(
        real_out, axis = 0
    ) ❹
    reg_loss = self.alpha * tf.reduce_mean(
        real_out ** 2 + fake_out ** 2, axis = 0
    ) ❺
    loss = reg_loss + cdiv_loss
grads = training_tape.gradient(loss, self.model.trainable_variables) ❻
self.optimizer.apply_gradients(
    zip(grads, self.model.trainable_variables)
)
self.loss_metric.update_state(loss)
self.reg_loss_metric.update_state(reg_loss)
self.cdiv_loss_metric.update_state(cdiv_loss)
self.real_out_metric.update_state(tf.reduce_mean(real_out, axis = 0))
self.fake_out_metric.update_state(tf.reduce_mean(fake_out, axis = 0))
return {m.name: m.result() for m in self.metrics}

def test_step(self, real_imgs): ❼
    batch_size = real_imgs.shape[0]
    fake_imgs = tf.random.uniform((batch_size, 32, 32, 1)) * 2 - 1
    inp_imgs = tf.concat([real_imgs, fake_imgs], axis=0)
    real_out, fake_out = tf.split(self.model(inp_imgs), 2, axis=0)
    cdiv = tf.reduce_mean(fake_out, axis = 0) - tf.reduce_mean(
        real_out, axis = 0
    )
    self.cdiv_loss_metric.update_state(cdiv)
    self.real_out_metric.update_state(tf.reduce_mean(real_out, axis = 0))
    self.fake_out_metric.update_state(tf.reduce_mean(fake_out, axis = 0))
    return {m.name: m.result() for m in self.metrics[2:]}

```

```

ebm = EBM()
ebm.compile(optimizer=optimizers.Adam(learning_rate=0.0001), run_eagerly=True)
ebm.fit(x_train, epochs=60, validation_data = x_test,)

```

❶ В реальные изображения добавляется небольшое количество случайного шума, чтобы избежать переобучения модели.

❷ Из буфера выбирается набор сгенерированных изображений.

❸ Реальные и сгенерированные изображения пропускаются через модель для получения оценок.

❹ Потери контрастивной дивергенции — это просто разность между оценками реальных и сгенерированных наблюдений.

- 5 Чтобы избежать слишком больших оценок, добавляется регуляризация потерь.
- 6 На этапе обратного распространения ошибки вычисляются градиенты функции потерь относительно весов сети.
- 7 Метод `test_step` используется во время проверки и вычисляет контрастивную дивергенцию между оценками для набора случайного шума и данными из обучающего набора. Его можно применять как меру обучения модели (см. следующий раздел).

Анализ модели на основе энергии

На рис. 7.6 показаны кривые функции потерь и вспомогательных показателей процесса обучения.

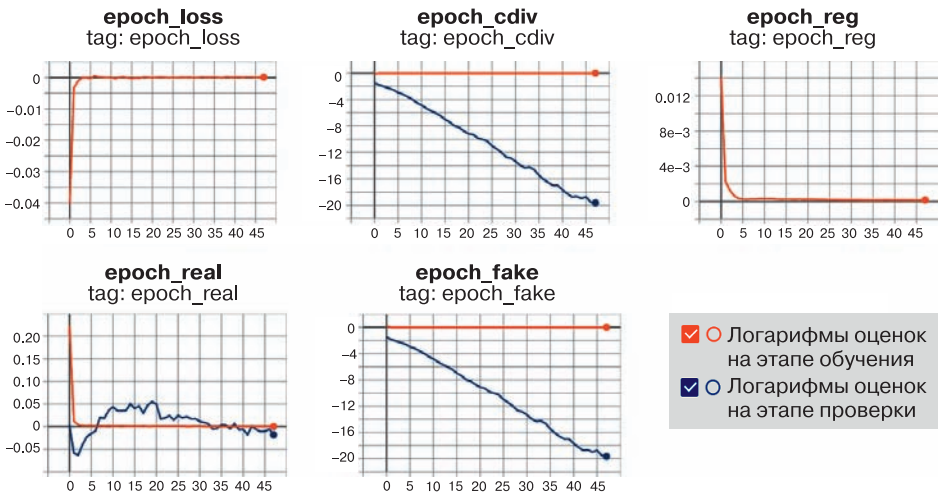


Рис. 7.6. Кривые функции потерь и вспомогательных показателей процесса обучения EBM

Прежде всего обратите внимание на то, что значения потерь, вычисленные на этапе обучения, остаются примерно постоянными и малыми. Несмотря на то что модель постоянно улучшается, качество сгенерированных изображений в буфере, которые сравниваются с реальными изображениями из обучающего набора, тоже улучшается, поэтому не следует ожидать значительного снижения потерь при обучении.

Соответственно, чтобы оценить качество модели, мы реализовали процесс проверки, который не выполняет выборку из буфера, а оценивает образцы случайного шума и сравнивает полученные оценки с оценками образцов из обучающего набора. Если модель улучшается, то мы должны увидеть падение контрастной дивергенции с течением эпох (то есть модель все лучше отличает случайный шум от реальных изображений), как видно на рис. 7.6.

Генерация новых образцов из ЕВМ — это просто запуск семплера Ланжевена для выполнения большого количества шагов с нуля (случайный шум), как показано в примере 7.7. Наблюдение *спускается вниз*, следуя градиентам оценок функции по отношению к входным данным, в результате чего из шума появляется правдоподобное наблюдение.

Пример 7.7. Генерация новых наблюдений с использованием ЕВМ

```
start_imgs = np.random.uniform(size = (10, 32, 32, 1)) * 2 - 1
gen_img = generate_samples(
    ebm.model,
    start_imgs,
    steps=1000,
    step_size=10,
    noise = 0.005,
    return_img_per_step=True,
)
```

На рис. 7.7 показаны некоторые примеры наблюдений, сгенерированных семплером после 50 эпох обучения.



Рис. 7.7. Примеры, полученные семплером Ланжевена с использованием модели ЕВМ для управления градиентным спуском

Мы можем даже показать динамику создания отдельного наблюдения, сохраняя снимки текущих наблюдений во время процесса выборки Ланжевена (рис. 7.8).

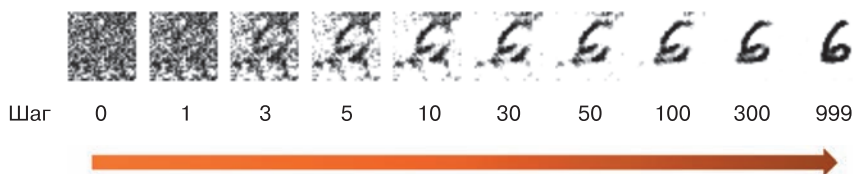


Рис. 7.8. Снимки наблюдения, сделанные в разные моменты процесса выборки Ланжевена

Другие модели на основе энергии

В предыдущем примере мы использовали глубокую ЕВМ, обученную с помощью метода контрастивной дивергенции, и динамический семплер Ланжевена. Однако ранние модели ЕВМ не применяли семплер Ланжевена, а полагались на другие методы и архитектуры.

Одним из самых ранних примеров ЕВМ была *машина Больцмана* [Ackley et al., 1985]. Это полносвязная однонаправленная нейронная сеть, в которой двоичные узлы либо *видимы* (*visible*, v), либо *скрыты* (*hidden*, h). Энергия сети с такой конфигурацией определяется следующим образом:

$$E_{\theta}(v, h) = -\frac{1}{2}(v^T L v + h^T J h + v^T W h),$$

где W, L, J — матрицы весов, изучаемые моделью. Обучение достигается за счет контрастной дивергенции, но с использованием выборки Гиббса для чередования видимых и скрытых слоев до тех пор, пока не будет найдено равновесие. Это очень медленный и немасштабируемый процесс при большом количестве скрытых узлов.



Интересный и простой пример выборки Гиббса можно найти в статье Gibbs Sampling in Python (<https://oreil.ly/tXmOq>), опубликованной в блоге Джессики Стрингхэм (Jessica Stringham).

Расширение этой модели, *ограниченная машина Больцмана* (Restricted Boltzmann Machine, RBM), удаляет связи между узлами одного типа, создавая двухслойный двудольный граф. Это позволяет объединять машины RBM в *глубокие сети убеждений* (deep belief networks) для моделирования более сложных распределений. Однако моделирование многомерных данных с помощью RBM остается непрактичным из-за необходимости использовать выборку Гиббса, отличающуюся большим временем смешивания.

Лишь в конце 2000-х годов было показано, что ЕВМ обладают потенциалом для моделирования весьма объемных наборов данных, и была разработана основа для создания глубоких ЕВМ [Lecun et al., 2006]. Динамика Ланжевена стала предпочтительным методом выборки для ЕВМ, которая позже превратилась в методику обучения, известную как *сопоставление оценок*. Дальнейшее развитие этой идеи вылилось в класс моделей, известный как *вероятностные диффузионные модели удаления шума* (Denoising Diffusion Probabilistic Models), которые применяются в современных генеративных моделях, таких как DALL·E 2 и ImageGen. Более подробно диффузионные модели мы рассмотрим в главе 8.

Резюме

Модели на основе энергии — это класс генеративных моделей, в которых используется функция оценки энергии — нейронная сеть, обученная выдавать низкие оценки для реальных наблюдений и высокие — для сгенерированных. Расчет распределения вероятностей, заданного этой функцией, требует нормализации с помощью трудноразрешимого знаменателя. ЕВМ решают эту проблему с помощью двух приемов: контрастивной дивергенции для обучения сети и динамики Ланжевена для выборки новых наблюдений.

Функция на основе энергии обучается путем минимизации разности между оценками для сгенерированной выборки и обучающих данных. Этот метод известен как контрастивная дивергенция. Можно показать, что он эквивалентен минимизации отрицательного логарифмического правдоподобия, как того требует оценка максимального правдоподобия, но для него не нужно вычислять трудноразрешимый нормализующий знаменатель. На практике процесс выборки генерированных образцов аппроксимируется, чтобы гарантировать эффективность алгоритма.

Выборка из глубоких ЕВМ осуществляется с помощью динамики Ланжевена — метода, использующего градиент оценки по отношению к входному изображению для постепенного преобразования случайного шума в правдоподобное наблюдение путем обновления входных данных небольшими шагами. Это более совершенный метод по сравнению с более ранними, такими как выборка Гиббса, которая применяется ограниченными машинами Больцмана.

Модели диффузии

В этой главе вы:

- познакомитесь с основными принципами и компонентами, определяющими модель диффузии;
- посмотрите, как используется прямой процесс для добавления шума в обучающий набор изображений;
- узнаете, что такое перепараметризация и почему этот прием так важен;
- исследуете разные формы планирования прямого распространения;
- изучите процесс обратной диффузии и узнаете, как он связан с прямым процессом добавления шума;
- познакомитесь с архитектурой U-Net, которая используется для параметризации процесса обратной диффузии;
- с помощью Keras создадите свою диффузионную модель удаления шума (Denoising Diffusion Model, DDM) для изображений цветов;
- попытаетесь извлечь новые изображения цветов из своей модели;
- исследуете влияние количества шагов диффузии на качество изображения и интерполируете два изображения в скрытом пространстве.

Наряду с GAN диффузионные модели являются одним из наиболее важных и эффективных методов генеративного моделирования для создания изображений среди появившихся за последнее десятилетие. Во многих тестах диффузионные модели превосходят генеративно-состязательные сети, ранее считавшиеся непревзойденными, и быстро становятся предпочтительным выбором для практиков генеративного моделирования, особенно в области визуальных изображений (например, DALL·E 2 от OpenAI и ImageGen от Google для преобразования текста в изображения). В последнее время произошел взрывной рост разнообразия моделей диффузии, применяемых для решения широкого круга задач, что напоминает распространение GAN в 2017–2020 годах.

Многие идеи, лежащие в основе диффузионных моделей, имеют сходство с идеями, на которых базировались более ранние типы генеративных моделей,

уже рассмотренные в этой книге, например автокодировщики с удалением шума и модели на основе энергии. Название «*диффузионные*» заимствовано из хорошо изученного свойства термодинамической диффузии: в 2015 году была установлена важная связь между этой чисто физической областью и глубоким обучением [Sohl-Dickstein et al., 2015].

Важный прогресс был достигнут также в области генеративных моделей на основе оценок [Song, Ermon, 2019], [Song, Ermon, 2020] — ветви моделирования энергии, — напрямую вычисляющих градиент распределения логарифмов (также известный как функция оценки) для обучения модели и используемых как альтернатива контрастивной дивергенции. В частности, Ян Сонг (Yang Song) и Стефано Эрмон (Stefano Ermon), чтобы гарантировать устойчивую работу модели, пробовали применить к исходным данным несколько шумовых возмущений разного масштаба — *сети условной оценки шума* (Noise Conditional Score Network, NCSN) в областях с низкой плотностью данных.

Летом 2020 года вышла прорывная статья о моделях диффузии [Ho et al., 2020]. Основываясь на более ранних работах, авторы раскрыли глубокую связь между моделями диффузии и генеративными моделями на основе оценок и использовали этот факт для обучения на нескольких наборах данных модели диффузии, способной конкурировать с GAN и получившей название *вероятностной диффузионной модели удаления шума* (Denoising Diffusion Probabilistic Model, DDPM).

В этой главе рассмотрим теоретические положения, необходимые для понимания особенностей работы диффузионной модели удаления шума. Затем вы узнаете, как построить свою диффузионную модель с помощью Keras.

Введение

Чтобы лучше понять ключевые идеи, лежащие в основе моделей диффузии, начнем с небольшой истории.

DIFFUSE TV

Вы стоите в магазине электроники, где продаются телевизоры. Однако этот магазин явно отличается от тех, которые вы посещали раньше. Вместо большого разнообразия марок на стеллажах, уходящих вглубь помещения, стоят сотни одинаковых экземпляров одного и того же телевизора, соединенных последовательно. Кроме того, первые несколько телевизоров показывают лишь случайный статический шум (рис. 8.1).

Продавец подходит и спрашивает, нужна ли вам помощь. В замешательстве вы спрашиваете ее о странных стеллажах с телевизорами. Она объясняет, что это новая модель DiffuseTV, которая призвана произвести революцию в индустрии развлечений, и начинает рассказывать, как она работает, постепенно увлекая вас вдоль стеллажа.



Рис. 8.1. Длинный стеллаж, уходящий вглубь магазина, с телевизорами, соединенными последовательно (создано с помощью Midjourney, <https://midjourney.com/>)

Она объясняет, что в процессе производства DiffuseTV подвергается воздействию тысяч изображений выпущенных ранее телешоу, но каждое из этих изображений постепенно искажается случайными статическими помехами, пока не становится неотличимым от чистого случайного шума. Затем телевизоры проектируются так, чтобы устранять случайный шум небольшими шагами, по сути пытаясь предсказать, как выглядело изображение до добавления шума. И в самом деле, двигаясь вдоль стеллажа, вы замечаете, что изображение на каждом следующем телевизоре становится немного четче, чем на предыдущем.

В конце концов вы достигаете конца длинной череды телевизоров и на экране последнего видите идеальное изображение. Это, безусловно, умная технология, но вам интересно понять, чем она полезна для зрителя. Продавец продолжает объяснение.

Вместо выбора канала для просмотра зритель выбирает случайную начальную статистику. Разные настройки дают разные выходные изображения, а некоторыми моделями можно даже управлять текстовой подсказкой, которую вы вводите. В отличие от обычного телевизора с ограниченным набором каналов DiffuseTV дает зрителю неограниченный выбор и свободу создавать на экране все, что он хочет!

Вы сразу покупаете DiffuseTV, услышав, что длинная череда телевизоров в магазине предназначена только для демонстрационных целей и вам не придется дополнительно покупать склад для хранения нового устройства.

История DiffuseTV описывает общую идею диффузионной модели. Теперь давайте углубимся в технические детали конструирования такой модели с помощью Keras.

Модели удаления шума

Основная идея диффузионной модели удаления шума проста: мы обучаем модель глубокого обучения подавлять шум на изображении, сделав серию очень маленьких шагов. Начав с чистого случайного шума, теоретически можно продолжать применять модель, пока не получится изображение, которое выглядит так, будто было взято из обучающего набора. Удивительно, насколько хорошо работает на практике эта простая концепция!

Давайте сначала настроим набор данных, а затем пройдемся по процессу прямой (зашумление) и обратной (удаление шума) диффузии.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу notebooks/08_diffusion/01_ddm/ddm.ipynb в репозитории книги. Он был заимствован (и адаптирован) из превосходного руководства по диффузионным моделям удаления шума (<https://oreil.ly/srPCe>), написанного Андрашем Берешем (András Béres).

Набор данных Flowers

Мы используем набор данных Oxford 102 Flower (<https://oreil.ly/HfrKV>), доступный на сайте Kaggle. Он содержит более 8000 изображений самых разных цветов.

Загрузить этот набор данных можно с помощью сценария из репозитория книг, как показано в примере 8.1. Изображения будут сохранены в папке `/data`.

Пример 8.1. Загрузка набора данных Oxford 102 Flower

```
bash scripts/download_kaggle_data.sh nunenuh pytorch-challenge-flower-dataset
```

Как обычно, загрузим изображения с помощью функции `image_dataset_from_directory` из библиотеки Keras, изменим размер изображений до 64×64 пиксела и масштабируем значения пикселей в диапазон $[0, 1]$. Повторим набор данных пять раз, чтобы увеличить длину эпохи, и сгруппируем данные в пакеты по 64 изображения, как показано в примере 8.2.

Пример 8.2. Загрузка набора данных Oxford 102 Flower

```
train_data = utils.image_dataset_from_directory(
    "/app/data/pytorch-challenge-flower-dataset/dataset",
    labels=None,
    image_size=(64, 64),
    batch_size=None,
    shuffle=True,
    seed=42,
    interpolation="bilinear",
) ❶

def preprocess(img):
    img = tf.cast(img, "float32") / 255.0
    return img

train = train_data.map(lambda x: preprocess(x)) ❷
train = train.repeat(5) ❸
train = train.batch(64, drop_remainder=True) ❹
```

❶ Загружает данные (по мере необходимости — в процессе обучения) с помощью функции `image_dataset_from_directory` из библиотеки Keras.

❷ Масштабирование значений пикселей в диапазон $[0, 1]$.

❸ Повторение набора данных пять раз.

❹ Группировка набора данных в пакеты по 64 изображения.

На рис. 8.2 показаны некоторые примеры изображений.

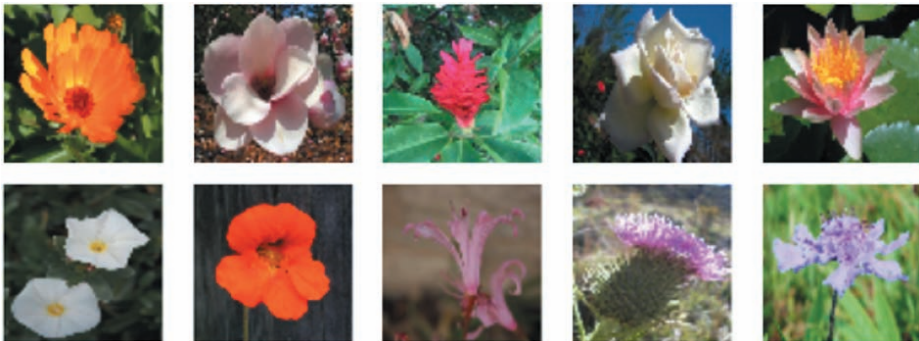


Рис. 8.2. Примеры изображений из набора данных Oxford 102 Flower

Теперь, подготовив набор данных, посмотрим, как добавить шум в изображения, используя процесс прямой диффузии.

Процесс прямой диффузии

Пусть у нас есть изображение $\mathbf{x}_0$, которое мы хотим постепенно испортить, выполнив большое количество шагов (скажем, $T = 1000$), чтобы в конечном итоге оно стало неотличимым от стандартного гауссова шума (то есть $\mathbf{x}_T$ должно иметь нулевое среднее значение и единичную дисперсию). Как это сделать?

Мы можем определить функцию q , добавляющую небольшое количество гауссова шума с дисперсией β_t к изображению $\mathbf{x}_{t-1}$ для создания нового изображения $\mathbf{x}_t$. Если продолжить применять эту функцию, можно сгенерировать последовательность все более зашумленных изображений ($\mathbf{x}_0 \dots \mathbf{x}_T$), как показано на рис. 8.3.

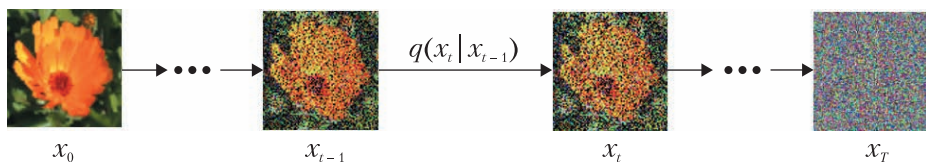


Рис. 8.3. Процесс прямой диффузии q

Этот процесс можно записать математически следующим образом (здесь ε_{t-1} — стандартное гауссово распределение с нулевым средним значением и единичной дисперсией):

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \varepsilon_{t-1}.$$

Обратите внимание на то, что мы также масштабируем входное изображение $\mathbf{x}_{t-1}$, чтобы гарантировать неизменность во времени дисперсии выходного изображения $\mathbf{x}_t$. Таким образом, если нормализовать исходное изображение $\mathbf{x}_0$ так, чтобы оно имело нулевое среднее значение и единичную дисперсию, то $\mathbf{x}_T$ будет аппроксимировать стандартное гауссово распределение для достаточно большого T по индукции, как описывается далее.

Если предположить, что $\mathbf{x}_{t-1}$ имеет нулевое среднее и единичную дисперсию, тогда $\sqrt{1 - \beta_t} \mathbf{x}_{t-1}$ будет иметь дисперсию $1 - \beta_t$, а $\sqrt{\beta_t} \varepsilon_{t-1}$ — дисперсию β_t согласно правилу $\text{Var}(aX) = a^2 \text{Var}(X)$. Сложив их, получаем новое распределение $\mathbf{x}_t$ с нулевым средним значением и дисперсией $1 - \beta_t + \beta_t = 1$ согласно правилу $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$ для независимых X и Y . Следовательно, нормализация $\mathbf{x}_0$ до нулевого среднего и единичной дисперсии гарантирует, что это правило будет выполняться для всех $\mathbf{x}_t$, включая окончательное изображение $\mathbf{x}_T$, которое будет аппроксимировать стандартное гауссово распределение. Это именно то, что нам нужно, чтобы иметь возможность легко выбрать $\mathbf{x}_T$, а затем применить процесс обратной диффузии с помощью обученной модели нейронной сети!

Другими словами, прямой процесс зашумления q можно записать следующим образом:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}).$$

Трюк с перепараметризацией

Также было бы полезно иметь возможность напрямую переходить от изображения $\mathbf{x}_0$ к любой зашумленной версии изображения $\mathbf{x}_t$, не прибегая к t применениям функции q . К счастью, для этой цели можно использовать трюк с перепараметризацией.

Если определить $\alpha_t = 1 - \beta_t$ и $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, то мы сможем записать следующее:

$$\begin{aligned} \mathbf{x}_t &= \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\varepsilon}_{t-1} = \\ &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \boldsymbol{\varepsilon} = \\ &= \dots = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}. \end{aligned}$$

Обратите внимание на то, что во второй строке используется тот факт, что сумма двух гауссовых функций дает новую гауссову функцию. Таким образом, у нас есть возможность перейти от исходного изображения $\mathbf{x}_0$ к любому шагу процесса прямой диффузии $\mathbf{x}_t$. Более того, мы можем определить режим диффузии, применяя значения $\bar{\alpha}_t$ вместо исходных значений β_t , с той интерпретацией, что $\bar{\alpha}_t$ — это дисперсия, обусловленная сигналом (исходным изображением $\mathbf{x}_0$), а $1 - \bar{\alpha}_t$ — дисперсия, обусловленная шумом ($\boldsymbol{\varepsilon}$). Поэтому процесс прямой диффузии q можно записать так:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}).$$

Режимы диффузии

Обратите внимание: на каждом временном шаге мы также можем выбирать разные значения β_t — они не обязательно должны быть одинаковыми. Закономерность изменения значения β_t (или $\bar{\alpha}_t$) в зависимости от t называется *режимом диффузии* (diffusion schedule).

В оригинальной статье [Ho et al., 2020] авторы выбрали *линейный режим диффузии* для β_t , когда β_t увеличивается линейно относительно t , от $\beta_1 = 0,0001$ до $\beta_T = 0,02$. Это гарантирует, что на ранних стадиях процесса шума добавляется меньше, чем на более поздних, когда изображение уже очень зашумлено.

В примере 8.3 показано, как реализовать линейный режим диффузии.

Пример 8.3. Линейный режим диффузии

```
def linear_diffusion_schedule(diffusion_times):
    min_rate = 0.0001
    max_rate = 0.02
    betas = min_rate + tf.convert_to_tensor(diffusion_times) * (max_rate - min_rate)
    alphas = 1 - betas
    alpha_bars = tf.math.cumprod(alphas)
    signal_rates = alpha_bars
    noise_rates = 1 - alpha_bars
    return noise_rates, signal_rates

T = 1000
diffusion_times = [x/T for x in range(T)] ❶
linear_noise_rates, linear_signal_rates = linear_diffusion_schedule(
    diffusion_times
) ❷
```

❶ Шаги диффузии, равноотстоящие друг от друга на интервале от 0 до 1.

❷ Линейный режим диффузии применяется к шагам диффузии для получения соотношения шума и сигнала.

В более поздней статье было показано, что *косинусный режим диффузии* превосходит линейный, предложенный в исходной статье [Nichol, Dhariwal, 2021]. Косинусный режим определяет следующие значения $\bar{\alpha}_t$:

$$\bar{\alpha}_t = \cos^2\left(\frac{t \pi}{T 2}\right).$$

Таким образом, обновленное уравнение выглядит следующим образом (с использованием тригонометрического тождества $\cos^2(x) + \sin^2(x) = 1$):

$$\mathbf{x}_t = \cos\left(\frac{t \pi}{T 2}\right) \mathbf{x}_0 + \sin\left(\frac{t \pi}{T 2}\right) \varepsilon.$$

Это уравнение — упрощенная версия фактического косинусного режима диффузии, используемого в статье. Авторы также добавляют член смещения и масштабирование, чтобы шаги зашумления не оказались слишком маленькими в начале процесса. В примере 8.4 показано, как реализовать косинусный и смещенный косинусный режимы диффузии.

Пример 8.4. Косинусный и смещенный косинусный режимы диффузии

```
def cosine_diffusion_schedule(diffusion_times): ❶
    signal_rates = tf.cos(diffusion_times * math.pi / 2)
    noise_rates = tf.sin(diffusion_times * math.pi / 2)
    return noise_rates, signal_rates
```

```
def offset_cosine_diffusion_schedule(diffusion_times): ❷
    min_signal_rate = 0.02
    max_signal_rate = 0.95
    start_angle = tf.acos(max_signal_rate)
    end_angle = tf.acos(min_signal_rate)

    diffusion_angles = start_angle + diffusion_times * (end_angle - start_angle)

    signal_rates = tf.cos(diffusion_angles)
    noise_rates = tf.sin(diffusion_angles)
    return noise_rates, signal_rates
```

❶ Чисто косинусный режим диффузии (без смещения или масштабирования).

❷ Смещенный косинусный режим диффузии, который мы будем использовать. Он корректирует режим так, чтобы шаги зашумления не оказались слишком маленькими в начале процесса.

Мы можем вычислить значения $\bar{\alpha}_t$ для каждого t , чтобы показать, какую долю составляют сигнал ($\bar{\alpha}_t$) и шум ($1 - \bar{\alpha}_t$) на каждом этапе процесса линейного, косинусного и смещенного косинусного режимов диффузии (рис. 8.4).

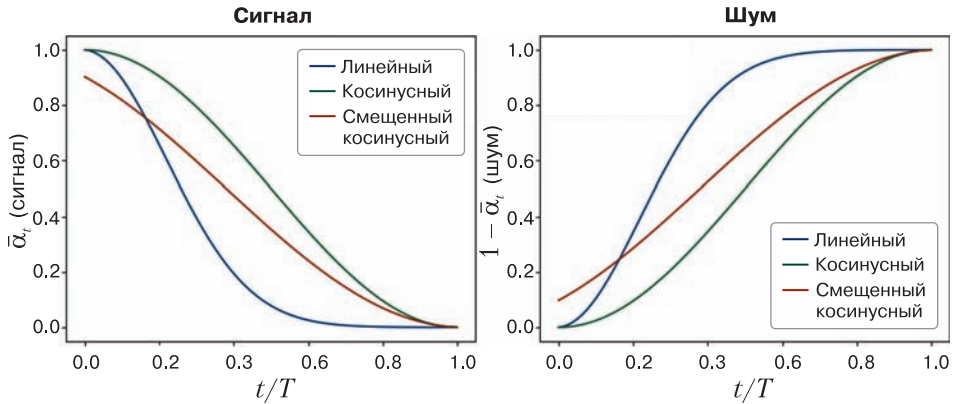


Рис. 8.4. Величина сигнала и шума на каждом этапе процесса зашумления при использовании линейного, косинусного и смещенного косинусного режимов диффузии

Обратите внимание на то, что при использовании косинусного режима диффузии уровень шума нарастает медленнее. Режим косинусной диффузии добавляет шум в изображение постепенно, что повышает эффективность обучения и качество генерируемых изображений. Это можно увидеть также на изображениях, полученных на разных этапах зашумления с помощью линейного и косинусного режимов (рис. 8.5).



Рис. 8.5. Изображение, зашумленное с помощью линейного (вверху) и косинусного (внизу) режимов диффузии при значениях t , равноотстоящих друг от друга в диапазоне от 0 до T (источник: [Ho et al., 2020], <https://arxiv.org/abs/2006.11239>)

Процесс обратной диффузии

Теперь рассмотрим процесс обратной диффузии. Как вы наверняка помните, нам нужно построить нейронную сеть $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$, способную обратить процесс зашумления, то есть аппроксимировать обратное распределение $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$. Создав ее, мы сможем выбирать случайный шум из $\mathcal{N}(0, \mathbf{I})$ и затем многократно применять процесс обратной диффузии, чтобы сгенерировать новое изображение (рис. 8.6).

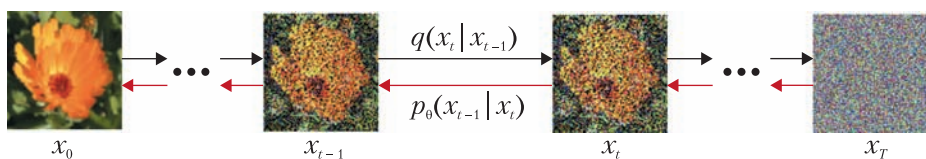


Рис. 8.6. Процесс обратной диффузии $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ устраняет шум, добавленный процессом прямой диффузии

Процесс обратной диффузии и процесс декодирования в вариационном автокодировщике имеют много общего. В обоих случаях целью является преобразование случайного шума в значимый результат с помощью нейронной сети. Разница между диффузионными моделями и VAE заключается в том, что в VAE прямой процесс (преобразование изображений в шум) является частью модели (то есть он изучается), тогда как в диффузионной модели он не параметризуется.

Поэтому имеет смысл применить ту же функцию потерь, что и в вариационном автокодировщике. Оригинальная статья, описывающая DDPM, выводит точную форму этой функции потерь и показывает, что ее можно оптимизировать путем обучения сети ϵ_θ , чтобы предсказать шум ϵ , который был добавлен в данное изображение $\mathbf{x}_0$ на временном шаге t .

Другими словами, мы выбираем изображение $\mathbf{x}_0$ и преобразуем его, выполняя t шагов зашумления, чтобы получить изображение $\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon$. Затем

передаем это новое изображение и степень зашумленности $\bar{\alpha}_t$ нейронной сети и просим ее дать прогноз, выполнив шаг градиента по квадрату ошибки между предсказанием $\varepsilon_\theta(\mathbf{x}_t)$ и истиной ε .

Структуру этой нейронной сети мы рассмотрим в следующем разделе, а пока отметим, что модель диффузии фактически поддерживает две копии сети: одну, которая активно обучается с использованием градиентного спуска, и другую (сеть ЕМА), которая представляет собой экспоненциальное скользящее среднее (exponential moving average, ЕМА) весов активно обучаемой сети за предыдущие этапы обучения. Сеть ЕМА не так восприимчива к кратковременным колебаниям и скачкам в процессе обучения, что делает ее более устойчивой к генерации по сравнению с активно обучаемой сетью. Поэтому для генерирования мы задействуем сеть ЕМА.

Процесс обучения модели показан на рис. 8.7.

Алгоритм 1. Обучение

- 1) **повторять**
 - 2) $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3) $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4) $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5) выполнить шаг градиентного спуска по формуле

$$\nabla_{\theta} \|\varepsilon - \varepsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, t)\|^2$$
 - 6) **пока** не сойдется
-

Рис. 8.7. Обучение диффузионной модели удаления шума
(источник: [Ho et al., 2020], <https://arxiv.org/abs/2006.11239>)

В примере 8.5 показано, как можно запрограммировать этот этап обучения с помощью Keras.

Пример 8.5. Функция `train_step` диффузионной модели

```
class DiffusionModel(models.Model):
    def __init__(self):
        super().__init__()
        self.normalizer = layers.Normalization()
        self.network = unet
        self.ema_network = models.clone_model(self.network)
        self.diffusion_schedule = cosine_diffusion_schedule

    ...
```

```

def denoise(self, noisy_images, noise_rates, signal_rates, training):
    if training:
        network = self.network
    else:
        network = self.ema_network
    pred_noises = network(
        [noisy_images, noise_rates**2], training=training
    )
    pred_images = (noisy_images - noise_rates * pred_noises) / signal_rates

    return pred_noises, pred_images

def train_step(self, images):
    images = self.normalizer(images, training=True) ❶
    noises = tf.random.normal(shape=tf.shape(images)) ❷
    batch_size = tf.shape(images)[0]
    diffusion_times = tf.random.uniform(
        shape=(batch_size, 1, 1, 1), minval=0.0, maxval=1.0
    ) ❸
    noise_rates, signal_rates = self.cosine_diffusion_schedule(
        diffusion_times
    ) ❹
    noisy_images = signal_rates * images + noise_rates * noises ❺
    with tf.GradientTape() as tape:
        pred_noises, pred_images = self.denoise(
            noisy_images, noise_rates, signal_rates, training=True
        ) ❻
        noise_loss = self.loss(noises, pred_noises) ❼
        gradients = tape.gradient(noise_loss, self.network.trainable_weights)
        self.optimizer.apply_gradients(
            zip(gradients, self.network.trainable_weights)
        )
        self.noise_loss_tracker.update_state(noise_loss)

    for weight, ema_weight in zip(
        self.network.weights, self.ema_network.weights
    ):
        ema_weight.assign(0.999 * ema_weight + (1 - 0.999) * weight)

    return {m.name: m.result() for m in self.metrics}

...

```

❶ Сначала нормализуем пакет изображений, чтобы получить нулевое среднее значение и единичную дисперсию.

❷ Затем генерируем тензор с шумом и формой, соответствующей форме входных изображений.

❸ Также выбираем случайные времена диффузии...

- ④ ...И используем их для получения соотношения шума и сигнала в соответствии с режимом косинусной диффузии.
- ⑤ Затем применяем взвешивание сигнала и шума к входным изображениям для создания зашумленных изображений.
- ⑥ Далее удаляем шум из зашумленных изображений, предлагая сети спрогнозировать шум, а затем отменяя операцию зашумления с помощью предоставленных значений `noise_rates` и `signal_rates`.
- ⑦ Затем можно вычислить потерю (среднюю абсолютную ошибку) между прогнозируемым и истинным шумом...

...И сделать шаг градиентного спуска в направлении уменьшения потерь.

Веса сети ЕМА обновляются до взвешенного среднего значения существующих весов ЕМА и весов обученной сети после шага градиентного спуска.

Модель удаления шума U-Net

Теперь, познакомившись с типом нейронной сети, которую нужно построить (предсказывающей шум, добавляемый в данное изображение), можно взглянуть на архитектуру, которая делает это возможным.

Авторы статьи о DDPM использовали тип архитектуры, известный как *U-Net*. Структура этой сети показана на рис. 8.8, где явно видно, как меняется форма тензора при его прохождении через сеть.

Подобно вариационному автокодировщику, U-Net состоит из двух половин: понижающей разрешение, где размеры входных изображений сжимаются, но увеличивается число каналов, и повышающей разрешение, где размеры представлений увеличиваются, а количество каналов уменьшается. Однако, в отличие от VAE, здесь между слоями одинаковой размерности в половинах, понижающих и повышающих разрешение, имеются дополнительные *пропускающие связи*. VAE действует линейно: данные передаются через всю сеть от входа к выходу, от одного слоя к другому. В U-Net имеются дополнительные связи, которые пропускают часть информации по более короткому пути между слоями одинаковой формы.

Архитектура U-Net особенно полезна, когда необходимо, чтобы выходные данные имели ту же форму, что и входные. В своем примере диффузионной модели мы хотим предсказать шум, добавляемый в изображение и имеющий точно такую же форму, как и само изображение, поэтому U-Net является естественным выбором для архитектуры сети.

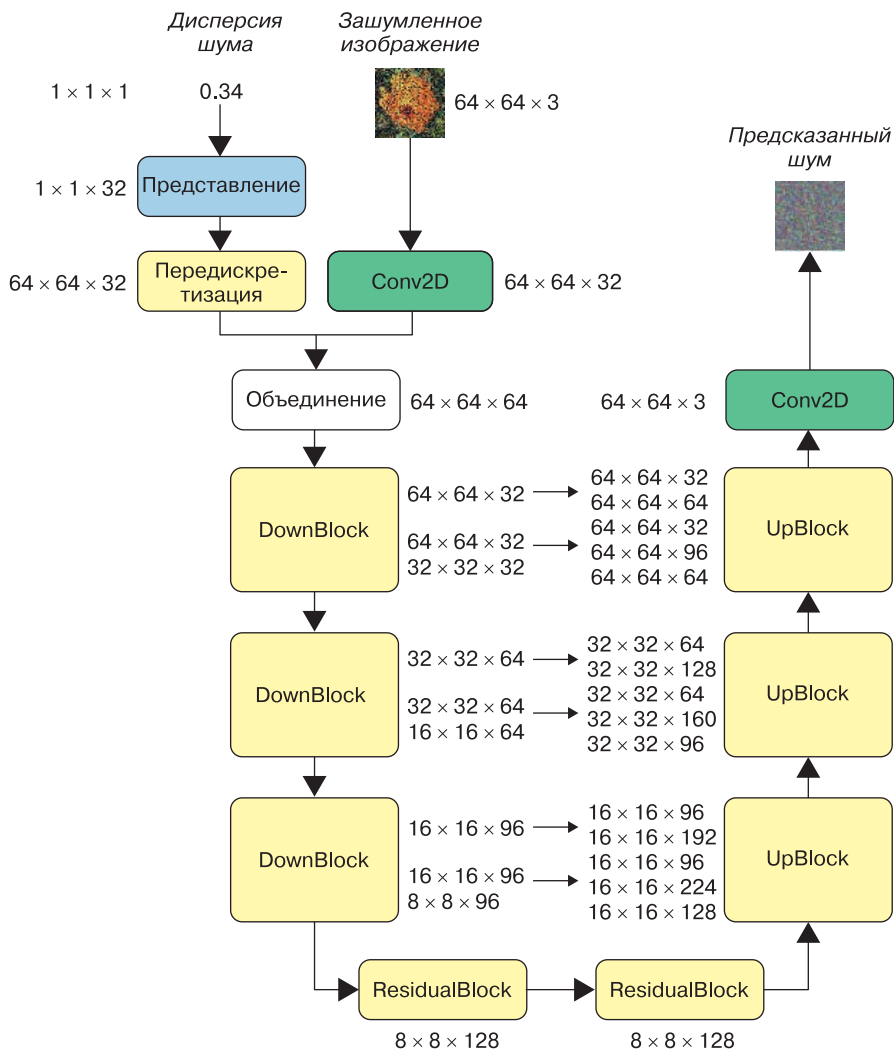


Рис. 8.8. Архитектура U-Net

Сначала взглянем на код, который создает U-Net с помощью Keras (пример 8.6).

Пример 8.6. Реализация модели U-Net с помощью Keras

```
noisy_images = layers.Input(shape=(64, 64, 3)) ❶
x = layers.Conv2D(32, kernel_size=1)(noisy_images) ❷

noise_variances = layers.Input(shape=(1, 1, 1)) ❸
noise_embedding = layers.Lambda(sinusoidal_embedding)(noise_variances) ❹
noise_embedding = layers.UpSampling2D(size=64, interpolation="nearest")()
```



```

    noise_embedding
) ⑤

x = layers.Concatenate()([x, noise_embedding]) ⑥

skips = [] ⑦

x = DownBlock(32, block_depth = 2)([x, skips])
x = DownBlock(64, block_depth = 2)([x, skips])
x = DownBlock(96, block_depth = 2)([x, skips])

x = ResidualBlock(128)(x)
x = ResidualBlock(128)(x)

x = UpBlock(96, block_depth = 2)([x, skips]) ⑩
x = UpBlock(64, block_depth = 2)([x, skips])
x = UpBlock(32, block_depth = 2)([x, skips])

x = layers.Conv2D(3, kernel_size=1, kernel_initializer="zeros")(x) ⑪

unet = models.Model([noisy_images, noise_variances], x, name="unet") ⑫

```

- ① Первый вход в U-Net — это изображение, из которого нужно удалить шум.
- ② Это изображение передается через слой `Conv2D` для увеличения количества каналов.
- ③ Второй вход в U-Net — это дисперсия шума (скаляр).
- ④ Она кодируется с использованием синусоидального представления.
- ⑤ Это представление копируется по пространственным измерениям, чтобы соответствовать размеру входного изображения.
- ⑥ Два входных потока объединяются по каналам.
- ⑦ Список `skips` будет содержать выходные данные слоев `DownBlock`, которые мы передадим последующим слоям `UpBlock`.

Тензор пропускается через серию слоев `DownBlock`, которые уменьшают размер изображения, одновременно увеличивая количество каналов.

Затем тензор передается через два слоя `ResidualBlock`, которые сохраняют размер изображения и количество каналов постоянными.

⑩ Далее тензор пропускается через серию слоев `UpBlock`, которые увеличивают размер изображения и уменьшают количество каналов. Пропускающие связи добавляют выходные данные из более ранних слоев `DownBlock`.

⑪ Последний слой `Conv2D` уменьшает количество каналов до трех (RGB).

⑫ U-Net — это модель, которая принимает зашумленные изображения и дисперсию шума и выводит прогнозируемую карту шума.

Чтобы до конца понять, как работает U-Net, нужно изучить еще четыре концепции: синусоидальное представление дисперсии шума, `ResidualBlock`, `DownBlock` и `UpBlock`.

Синусоидальное представление

Синусоидальное представление было впервые определено в статье Васвани (Vaswani) и др. [Vaswani et al., 2017]. Мы будем применять адаптацию этой оригинальной идеи, использованную в статье Милденхолла (Mildenhall) и др. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis* [Mildenhall et al., 2020].

Суть концепции состоит в том, чтобы получить возможность преобразовывать скалярное значение (дисперсию шума) в отдельный вектор более высокой размерности, который сможет обеспечить более сложное представление для дальнейшего использования в сети. В оригинальной статье она применялась для преобразования в векторы местоположений слов в предложении, статья о NeRF распространяет ее на непрерывные значения.

В частности, скалярное значение x кодируется, как показано в следующем уравнении:

$$\gamma(x) = \left(\sin(2\pi e^{0f} x) \dots \sin(2\pi e^{(L-1)f} x), \cos(2\pi e^{0f} x) \dots \cos(2\pi e^{(L-1)f} x) \right),$$

где мы выбираем значение $L = 16$, чтобы оно было вдвое меньше желаемой длины векторного представления шума, и $f = \frac{\ln(1000)}{L-1}$ — максимальный масштабный коэффициент масштабирования.

В результате получается шаблон представления (рис. 8.9).

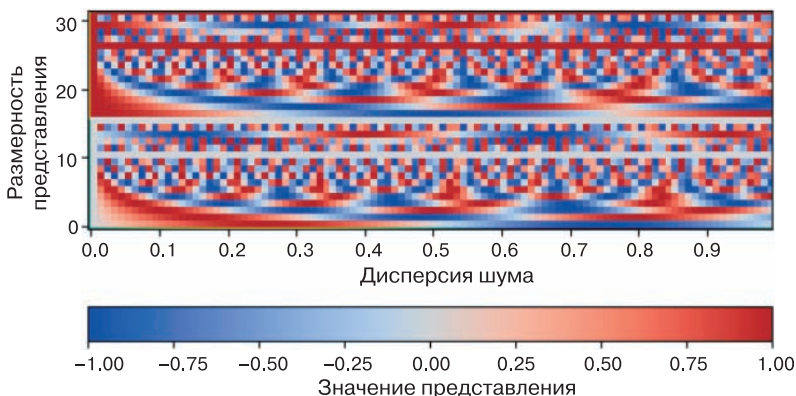


Рис. 8.9. Шаблон синусоидального представления для дисперсии шума от 0 до 1

В примере 8.7 показано, как реализовать эту функцию синусоидального представления. Она преобразует одно скалярное значение дисперсии шума в вектор длиной 32.

Пример 8.7. Функция `sinusoidal_embedding`, преобразующая дисперсию шума в векторное представление

```
def sinusoidal_embedding(x):
    frequencies = tf.exp(
        tf.linspace(
            tf.math.log(1.0),
            tf.math.log(1000.0),
            16,
        )
    )
    angular_speeds = 2.0 * math.pi * frequencies
    embeddings = tf.concat(
        [tf.sin(angular_speeds * x), tf.cos(angular_speeds * x)], axis=3
    )
    return embeddings
```

ResidualBlock

Слои `DownBlock` и `UpBlock` содержат слои `ResidualBlock`, поэтому начнем с последних. Мы уже исследовали остаточные блоки в главе 5, когда создавали `PixelCNN`, но для полноты картины повторим основные выводы.

Остаточный блок — это группа слоев, содержащая пропускающую связь, которая добавляет входные данные в выходные. Остаточные блоки помогают строить более глубокие сети, способные изучать более сложные закономерности, не страдая при этом проблемами затухания градиента и деградации. Проблема затухания градиента заключается в следующем: в глубоких сетях значения распространяемого градиента становятся все меньше, поэтому обучение протекает очень медленно. Проблема деградации заключается в том, что с увеличением глубины нейронных сетей они могут терять точность. В какой-то момент на определенной глубине точность достигает предела, а затем быстро ухудшается.



Деградация

Проблема деградации выглядит противоречащей здравому смыслу, но она наблюдается на практике, потому что более глубокие слои должны по крайней мере изучить тождественное отображение, что очень непросто, особенно если учесть другие проблемы, с которыми сталкиваются глубокие сети, такие как проблема затухания градиента.

Простое решение было представлено в статье с описанием `ResNet`, созданной Хе (He) с коллегами в 2015 году [He et al., 2015]. Включив *магистраль*

пропускающей связи между основными взвешивающими слоями, блок получает возможность обойти сложные обновления веса и просто передать тождественное отображение. Это позволяет обучать сеть на большую глубину, не жертвуя ее точностью или размером градиента.

На рис. 8.10 показана схема слоя `ResidualBlock`. Обратите внимание на то, что в некоторые остаточные блоки может включаться дополнительный слой `Conv2D` с размером ядра 1 для пропускающей связи, чтобы количество каналов соответствовало остальной части блока.

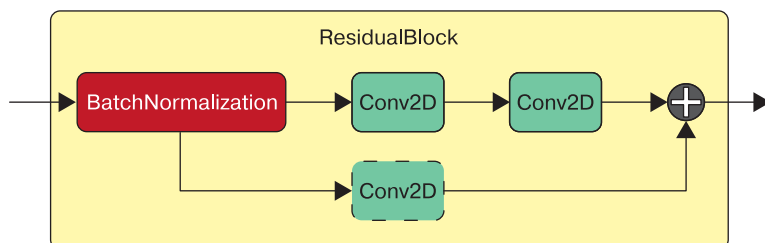


Рис. 8.10. Слой `ResidualBlock` в сети U-Net

В примере 8.8 показано, как можно реализовать `ResidualBlock` с помощью Keras.

Пример 8.8. Реализация `ResidualBlock` в U-Net

```
def ResidualBlock(width):
    def apply(x):
        input_width = x.shape[3]
        if input_width == width: ❶
            residual = x
        else:
            residual = layers.Conv2D(width, kernel_size=1)(x)
        x = layers.BatchNormalization(center=False, scale=False)(x) ❷
        x = layers.Conv2D(
            width, kernel_size=3, padding="same", activation=activations.swish
        )(x) ❸
        x = layers.Conv2D(width, kernel_size=3, padding="same")(x)
        x = layers.Add()([x, residual]) ❹
        return x
    return apply
```

❶ Проверка соответствия количества каналов на входе и выходе блока. Если количества не соответствуют, то в пропускающую связь добавляется дополнительный слой `Conv2D`, чтобы привести количество каналов в соответствие с остальной частью блока.

- ❷ Добавление слоя `BatchNormalization`.
- ❸ Добавление двух слоев `Conv2D`.
- ❹ Добавление входных данных в выходные для получения окончательного результата блока.

Слои `DownBlock` и `UpBlock`

Каждый последующий слой `DownBlock` увеличивает количество каналов в слоях `ResidualBlock` через `block_depth` ($= 2$ в нашем примере), а также применяет последний слой `AveragePooling2D`, чтобы уменьшить размер изображения вдвое. Каждый `ResidualBlock` добавляется в список для использования последующими слоями `UpBlock` в качестве пропускающих связей.

Слой `UpBlock` сначала применяет слой `UpSampling2D`, который удваивает размер изображения посредством билинейной интерполяции. Каждый последующий слой `UpBlock` уменьшает количество каналов в `ResidualBlock` через `block_depth` ($= 2$), а также объединяет выходные данные из `DownBlock` посредством пропускающих связей в U-Net. На рис. 8.11 показана схема этого процесса.

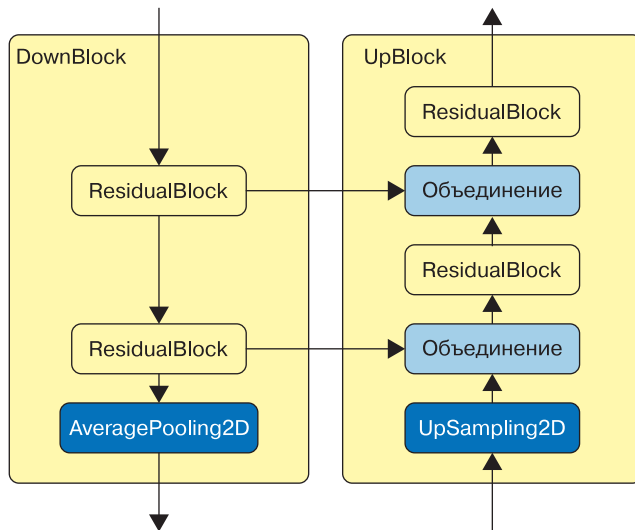


Рис. 8.11. Слой `DownBlock` и соответствующий ему слой `UpBlock` в сети U-Net

В примере 8.9 показано, как реализовать слои `DownBlock` и `UpBlock` с помощью Keras.

Пример 8.9. Реализация слоев DownBlock и UpBlock в модели U-Net

```
def DownBlock(width, block_depth):
    def apply(x):
        x, skips = x
        for _ in range(block_depth):
            x = ResidualBlock(width)(x) ❶
            skips.append(x) ❷
        x = layers.AveragePooling2D(pool_size=2)(x) ❸
        return x

    return apply

def UpBlock(width, block_depth):
    def apply(x):
        x, skips = x
        x = layers.UpSampling2D(size=2, interpolation="bilinear")(x) ❹
        for _ in range(block_depth):
            x = layers.Concatenate()([x, skips.pop()]) ❺
            x = ResidualBlock(width)(x) ❻
        return x

    return apply
```

❶ DownBlock увеличивает количество каналов в изображении, используя ResidualBlock с заданным значением width...

❷ ...Каждый из которых сохраняется в списке (skips) для последующего использования в слоях UpBlock.

❸ Последний слой AveragePooling2D уменьшает размерность изображения вдвое.

❹ UpBlock начинается со слоя UpSampling2D, который удваивает размер изображения.

❺ Выходные данные слоя DownBlock добавляются в текущий вывод с помощью слоя Concatenate.

❻ Слой ResidualBlock используется для уменьшения количества каналов в изображении при прохождении через UpBlock.

Обучение диффузионной модели

Теперь у нас есть все компоненты, можно приступать к обучению диффузионной модели удаления шума! Код примера 8.10 создает, компилирует и обучает диффузионную модель.

Пример 8.10. Код для обучения DiffusionModel

```
model = DiffusionModel() ❶
model.compile(
    optimizer=optimizers.experimental.AdamW(learning_rate=1e-3, weight_decay=1e-4),
    loss=losses.mean_absolute_error,
) ❷

model.normalizer.adapt(train) ❸

model.fit(
    train,
    epochs=50,
) ❹
```

❶ Создание экземпляра модели.

❷ Компиляция модели с использованием оптимизатора AdamW (напоминает оптимизатор Adam, но уменьшает веса, что помогает стабилизировать процесс обучения) и функции потерь — средней абсолютной ошибки.

❸ Вычисление статистик нормализации с помощью обучающего набора.

❹ Обучение модели на протяжении более чем 50 эпох.

На рис. 8.12 показан график изменения потерь (средняя абсолютная ошибка (mean absolute error, MAE) шума).

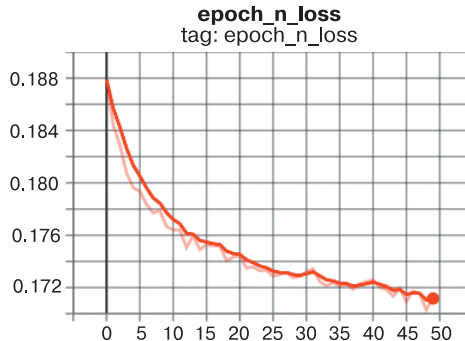


Рис. 8.12. График изменения потерь — средней абсолютной ошибки шума

Выборка из диффузионной модели удаления шума

Чтобы получить образцы изображений из обученной модели, нужно применить процесс обратной диффузии, то есть начать со случайного шума и использовать модель для постепенного удаления шума, пока не останется узнаваемое изображение цветка.

При этом следует иметь в виду, что наша модель обучена прогнозировать общее количество шума, добавленного в данное зашумленное изображение из обучающего набора, а не только шум, добавленный на последнем шаге процесса зашумления. Однако мы не будем устранять весь шум за один раз, потому что предсказать изображение по чисто случайному шуму за один раз точно не получится! Предпочтительнее по аналогии с процессом прямой диффузии постепенно удалять прогнозируемый шум, выполняя множество небольших шагов, чтобы позволить модели адаптироваться к собственным прогнозам.

С этой целью мы можем организовать переход от x_t к x_{t-1} за два шага — сначала с помощью модели прогнозирования шума вычислить оценки исходного изображения x_0 , а затем повторно применить прогнозируемый шум к этому изображению в течение $t-1$ временных шагов, чтобы произвести x_{t-1} (рис. 8.13).

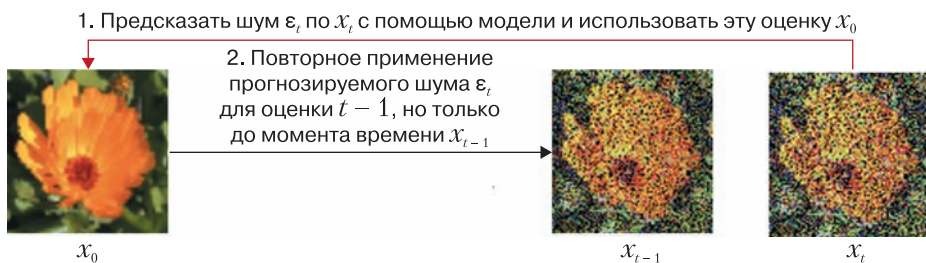


Рис. 8.13. Один шаг процесса выборки из нашей диффузионной модели

Если повторить этот процесс в несколько этапов, то, последовательно выполняя множество маленьких шагов, мы в конечном итоге вернемся к оценке для x_0 . Фактически мы вольны выбрать любое количество шагов, и, что особенно важно, оно не обязательно должно совпадать с количеством шагов, в течение которых добавлялся шум, то есть 1000. Оно может быть намного меньше — в этом примере мы выбрали 20.

Следующее уравнение [Song et al., 2020] описывает этот процесс математически:

$$\mathbf{x}_{t-1} = \underbrace{\sqrt{\bar{\alpha}_{t-1}} \left(\frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}_\theta^{(t)}(\mathbf{x}_t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{Предсказание } x_0} + \underbrace{\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \boldsymbol{\varepsilon}_\theta^{(t)}(\mathbf{x}_t)}_{\text{Направление, указывающее на } x_t} + \underbrace{\sigma_t \boldsymbol{\varepsilon}_t}_{\text{Случайный шум}}$$

Давайте разберем эту формулу. Первый член в скобках в правой части уравнения — это предполагаемое изображение x_0 , вычисленное на основе шума, предсказанного нашей сетью, $\boldsymbol{\varepsilon}_\theta^{(t)}$. Затем выполняется масштабирование по уровню сигнала $\sqrt{\bar{\alpha}_{t-1}}$ на шаге $t-1$ и повторно применяется предсказанный шум, но на

этот раз масштабированный по уровню шума $\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2}$ на шаге $t - 1$. Добавляется дополнительный гауссов случайный шум $\sigma_t \epsilon_t$, при этом коэффициенты σ_t определяют, насколько случайным должен быть процесс генерации.

Особый случай $\sigma_t = 0$ для всех t соответствует типу модели, известной как *неявная диффузионная модель удаления шума* (Denoising Diffusion Implicit Model, DDIM), представленной Сонгом (Song) и его коллегами в 2020 году [Song et al., 2020]. При использовании DDIM процесс генерации протекает полностью детерминированно, то есть один и тот же случайный шум на входе всегда будет давать один и тот же результат на выходе. Это желательно, поскольку в таком случае мы получаем четкое соответствие между выборками из скрытого пространства и сгенерированными выходными данными в пространстве пикселей.

В примере мы реализуем DDIM, что сделает процесс генерации детерминированным. Реализация выборки DDIM (обратная диффузия) показана в примере 8.11.

Пример 8.11. Выборка из модели

```
class DiffusionModel(models.Model):
    ...

    def reverse_diffusion(self, initial_noise, diffusion_steps):
        num_images = initial_noise.shape[0]
        step_size = 1.0 / diffusion_steps
        current_images = initial_noise
        for step in range(diffusion_steps): ❶
            diffusion_times = tf.ones((num_images, 1, 1, 1)) - step * step_size ❷
            noise_rates, signal_rates = self.diffusion_schedule(diffusion_times) ❸
            pred_noises, pred_images = self.denoise(
                current_images, noise_rates, signal_rates, training=False
            ) ❹
            next_diffusion_times = diffusion_times - step_size ❺
            next_noise_rates, next_signal_rates = self.diffusion_schedule(
                next_diffusion_times
            ) ❻
            current_images = (
                next_signal_rates * pred_images + next_noise_rates * pred_noises
            ) ❼
        return pred_images
```

❶ Выполнение фиксированного количества шагов, например 20.

❷ Все времена диффузии устанавливаются равными 1, то есть в начало процесса обратной диффузии.

- ③ Уровни сигнала и шума вычисляются в соответствии с режимом диффузии.
- ④ Для прогнозирования шума используется U-Net, что позволяет рассчитать оценку изображения без шума.
- ⑤ Время диффузии уменьшается на один шаг.
- ⑥ Рассчитываются новые уровни шума и сигнала.
- ⑦ Изображения $t - 1$ рассчитываются путем повторного применения прогнозируемого шума к прогнозируемому изображению в соответствии с режимом диффузии на шаге $t - 1$.

После 20 шагов возвращается окончательное предсказанное изображение x_0 .

Анализ модели

Теперь рассмотрим три способа использования обученной модели: для генерации новых изображений, для проверки влияния количества шагов обратной диффузии на качество и для интерполяции между двумя изображениями в скрытом пространстве.

Генерирование изображений

Чтобы извлечь образцы из обученной модели, можно просто запустить процесс обратной диффузии и денормализовать выходные данные в конце, то есть вернуть значения пикселей обратно в диапазон $[0, 1]$. В примере 8.12 показано, как это реализовать внутри класса `DiffusionModel`.

Пример 8.12. Генерация изображений с использованием модели

```
class DiffusionModel(models.Model):
    ...

    def denormalize(self, images):
        images = self.normalizer.mean + images * self.normalizer.variance**0.5
        return tf.clip_by_value(images, 0.0, 1.0)

    def generate(self, num_images, diffusion_steps):
        initial_noise = tf.random.normal(shape=(num_images, 64, 64, 3)) ①
        generated_images = self.reverse_diffusion(initial_noise, diffusion_steps) ②
        generated_images = self.denormalize(generated_images) ③
        return generated_images
```

- ❶ Создание нескольких начальных карт шума.
- ❷ Применение процесса обратной диффузии.
- ❸ Изображения, генерируемые сетью, будут иметь нулевое среднее значение и единичную дисперсию, поэтому необходимо выполнить денормализацию, повторно применив среднее значение и дисперсию, вычисленные на основе обучающих данных.

На рис. 8.14 показаны некоторые образцы, сгенерированные моделью диффузии в разные эпохи процесса обучения.



Рис. 8.14. Образцы, сгенерированные моделью диффузии в разные эпохи процесса обучения

Корректировка количества шагов

Далее посмотрим, как изменение количества шагов в процессе обратной диффузии влияет на качество изображения. Интуитивно понятно, что чем больше шагов выполняется, тем выше качество сгенерированного изображения.

Как показано на рис. 8.15, качество изображений действительно улучшается с ростом количества шагов диффузии. Сделав один гигантский скачок по сравнению с исходным выбранным шумом, модель может предсказать только неясное цветное пятно. Благодаря значительному количеству шагов модель может все больше и больше совершенствовать последующие поколения. Однако время,

необходимое для создания изображений, линейно зависит от количества шагов диффузии, поэтому важно найти компромисс между качеством и скоростью. Между 20-м и 100-м шагами изображения улучшаются незначительно, поэтому здесь мы выбрали 20 шагов как разумный компромисс.

Шагов диффузии

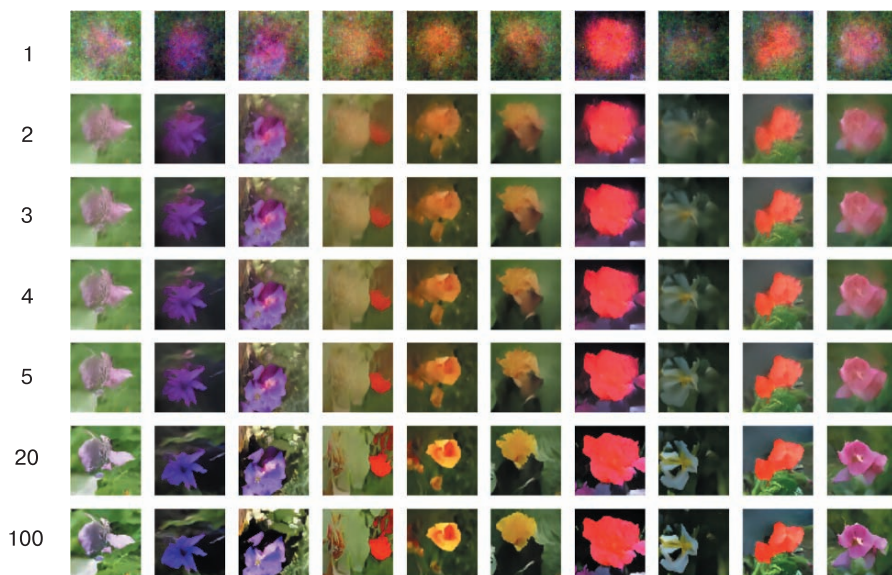


Рис. 8.15. Качество изображений улучшается с увеличением количества шагов диффузии

Интерполяция между изображениями

Наконец, как было показано ранее в примере вариационного автокодировщика, можно интерполировать между точками в гауссовом скрытом пространстве, чтобы плавно перейти от одного изображения к другому в пространстве пикселей. Здесь мы решили использовать форму сферической интерполяции, гарантирующей постоянство дисперсии при смешивании двух карт гауссова шума. В частности, начальная карта шума на каждом этапе определяется выражением $a \sin\left(\frac{\pi}{2}t\right) + b \cos\left(\frac{\pi}{2}t\right)$, где t плавно меняется в диапазоне от 0 до 1, а коэффициенты a и b — это два тензора случайного гауссова шума, между которыми производится интерполяция.

Полученные изображения показаны на рис. 8.16.

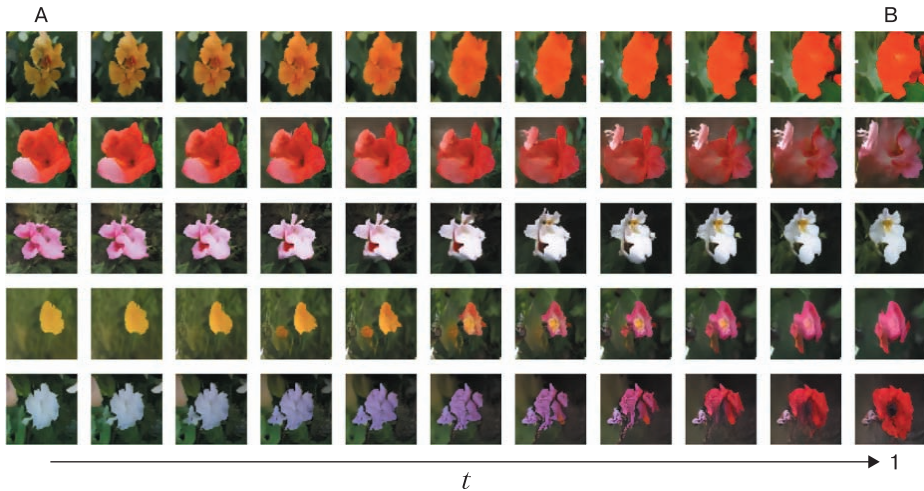


Рис. 8.16. Интерполяция между изображениями с использованием диффузионной модели удаления шума

Резюме

В этой главе мы исследовали одну из самых интересных и многообещающих областей генеративного моделирования, появившихся в последнее время, — диффузионные модели. В частности, реализовали ключевые концепции статьи, описывающей генеративные модели диффузии [Ho et al., 2020], в которой была представлена оригинальная вероятностная диффузионная модель удаления шума (Denoising Diffusion Probabilistic Model, DDPM). Затем расширили реализацию идеями из статьи, описывающей неявную диффузионную модель удаления шума (Denoising Diffusion Implicit Model, DDIM), чтобы сделать процесс генерации полностью детерминированным.

Мы увидели, как в диффузионных моделях протекают процессы прямой и обратной диффузии. В ходе прямой диффузии модель добавляет шум в обучающие данные, выполняя последовательность небольших шагов, а при обратной диффузии пытается предсказать добавленный шум.

Нами применен прием перепараметризации, чтобы вычислить зашумленные изображения на любом этапе прямого процесса, не выполняя несколько шагов

добавления шума. Мы видели, насколько важную роль в общем успехе играет набор параметров, используемый для добавления шума.

Процесс обратной диффузии параметризуется с помощью сети U-Net, которая пытается предсказать шум на каждом временном шаге, получая зашумленное изображение и уровень шума на этом шаге. U-Net состоит из блоков `DownBlock`, которые увеличивают количество каналов и уменьшают размеры изображений, и `UpBlock`, которые уменьшают количество каналов и увеличивают размеры. Уровень шума определяется с помощью синусоидального векторного представления.

Выборка из модели диффузии выполняется в несколько этапов. Сначала с помощью сети U-Net прогнозируется шум, добавляемый к заданному зашумленному изображению, и вычисляется оценка исходного изображения. Затем прогнозируемый шум (меньший уровень) применяется повторно. Этот процесс повторяется на протяжении нескольких шагов (их количество может быть значительно меньше количества шагов, выполняемых во время обучения), начиная со случайной точки, выбранной из стандартного гауссова распределения шума, для получения окончательной генерации.

Вы увидели, что увеличение количества шагов обратной диффузии улучшает качество генерируемых изображений за счет скорости. А также увидели возможность выполнения арифметики скрытого пространства для интерполяции между двумя изображениями.

Часть III

Практическое применение

В части III мы рассмотрим некоторые ключевые примеры практического применения методов генеративного моделирования, которые видели до сих пор, к изображениям, тексту, музыке и играм. Также поговорим о том, как можно пересечь эти области, используя современные мультимодальные модели.

В **главе 9** мы обратим внимание на трансформеры — новейшую архитектуру, лежащую в основе большинства современных моделей генерации текста. В частности, изучим внутреннюю работу GPT и создадим свою версию с помощью Keras, а также посмотрим, как она закладывает основы для таких инструментов, как ChatGPT.

В **главе 10** мы рассмотрим некоторые наиболее важные архитектуры GAN, существенно повлиявшие на создание изображений, включая ProGAN, StyleGAN, StyleGAN2, SAGAN, BigGAN, VQ-GAN и ViT VQ-GAN. Поговорим о том, какой вклад внесла каждая из них, и постараемся понять, как они развивались с течением времени.

В **главе 11** рассматривается область генерации музыки, которая ставит дополнительные задачи, такие как моделирование высоты музыкальных нот и ритма. Вы узнаете, что многие методы, используемые для генерации текста, например трансформеры, с успехом можно применить и в данной области. Кроме этого, вы познакомитесь с архитектурой глубокого обучения, известной как MuseGAN, которая применяет подход на основе GAN для создания музыки.

В **главе 12** показано, как можно задействовать генеративные модели в других областях машинного обучения, например, в обучении с подкреплением. Мы сосредоточимся на статье *World Models*, показывающей, что генеративную модель можно применять в качестве среды для обучения агента, позволяющей ему обучаться в вымышленной версии среды, а не в реальной.

В **главе 13** мы рассмотрим современные мультимодальные модели, объединяющие такие области, как изображения и текст. К ним относятся модели преобразования текста в изображение DALL.E 2, Imagen и Stable Diffusion, а также модели визуального языка, например Flamingo.

Наконец, в **главе 14** подводится итог развития генеративного ИИ, дается оценка текущей ситуации в области генеративного ИИ и обсуждаются возможные пути развития в будущем. Здесь мы посмотрим, как генеративный ИИ может изменить наши образ жизни и работу, а также оценим потенциал открытия более глубоких форм искусственного интеллекта в ближайшие годы.

Трансформеры

В этой главе вы:

- познакомитесь с историей происхождения GPT — мощной модели трансформера-декодировщика для генерации текста;
- узнаете, как механизм внимания имитирует человеческую способность придавать большее значение одним словам в предложении и меньшее — другим;
- исследуете основные принципы работы механизма внимания, в том числе способы создания запросов, ключей и значений и управления ими;
- узнаете о важности причинно-следственной маскировки в задачах генерации текста;
- увидите, как модули внимания можно сгруппировать в слой многоголового внимания;
- узнаете, что слои внимания с несколькими модулями образуют одну часть блока трансформера, включающего также послонную нормализацию и пропускающие связи;
- реализуете позиционное представление, фиксирующее положение каждой лексемы, а также векторные представления лексем слов;
- с помощью Keras создадите модель GPT, генерирующую текст для обзоров вин;
- проанализируете выходные данные модели GPT, в том числе проведете опрос оценок внимания, чтобы определить, куда смотрит модель;
- познакомитесь с различными типами трансформеров и примерами задач, которые может решать каждый из них;
- узнаете, как работают архитектуры кодировщика-декодировщика, например модель Google T5;
- исследуете процесс обучения, лежащий в основе ChatGPT OpenAI.

В главе 5 вы видели, как строить генеративные модели для создания текстовых данных на основе рекуррентных нейронных сетей (Recurrent Neural Networks, RNN), таких как LSTM и GRU. Эти авторегрессионные модели обрабатывают последовательные данные по одной лексеме за раз, постоянно обновляя скрытый вектор, фиксирующий текущее скрытое представление входных данных. Сеть RNN можно спроектировать так, что она будет предсказывать следующее слово

в последовательности, применяя полносвязный слой и функцию активации softmax к скрытому вектору. Этот подход считался самым сложным способом генеративного создания текста до 2017 года, когда выход одной статьи навсегда изменил сферу генерации текста.

Введение

Статья, вышедшая в Google Brain и озаглавленная *Attention is All You Need* [Vaswani et al., 2017], известна популяризацией концепции *внимания* — механизма, который в настоящее время положен в основу большинства современных моделей генерации текста.

Авторы показали, как создавать мощные нейронные сети, называемые *трансформерами* (transformers), для последовательного моделирования, не требующие сложных рекуррентных или сверточных архитектур и полагающиеся только на механизмы внимания. Этот подход преодолевает ключевой недостаток подхода на основе RNN, заключающийся в сложности распараллеливания обработки исходных данных из-за необходимости обрабатывать последовательности по одной лексеме за раз. Трансформеры обладают возможностью распараллеливания, что позволяет обучать их на огромных наборах данных.

В этой главе углубимся в современные модели генерации текста, использующие архитектуру трансформера для достижения высочайшей производительности при решении задач генерации текста. В частности, рассмотрим модель авторегрессионного типа, известную как *предварительно обученный генеративный трансформер* (Generative Pre-trained Transformer, GPT), лежащую в основе модели OpenAI GPT-4, которая считается одной из самых успешных моделей генерации текста.

GPT

В июне 2018 года в статье *Improving Language Understanding by Generative Pre-Training* [Radford et al., 2018], вышедшей почти ровно через год после появления оригинальной статьи о трансформерах, OpenAI представила модель GPT. Ее авторы показали, что можно обучить архитектуру трансформера на огромном объеме текстовых данных предсказывать следующее слово в последовательности, а затем точно настроить ее для решения конкретных задач.

Процесс предварительного обучения GPT включает обучение модели на большом корпусе текста BookCorpus (4,5 Гбайт текста из 7000 неопубликованных

книг разных жанров). В ходе предварительного обучения модель учится предсказывать следующее слово в последовательности с учетом предыдущих слов. Этот процесс известен как *языковое моделирование* и используется для того, чтобы обучить модель понимать структуру и закономерности естественного языка.

После предварительного обучения модель GPT можно точно настроить для решения конкретной задачи, предоставив ей меньший набор специализированных данных. Точная настройка включает корректировку параметров модели, чтобы она лучше соответствовала поставленной задаче. Например, модель можно точно настроить для таких задач, как классификация, оценка сходства или ответы на вопросы.

С выпуском последующих моделей GPT-2, GPT-3, GPT-3.5 и GPT-4 архитектура GPT постоянно улучшалась и расширялась. Эти модели обучаются на больших наборах данных и имеют значительную емкость, поэтому могут генерировать сложный и связный текст. Модели GPT получили широкое распространение среди исследователей и практиков в разных отраслях и способствовали значительному прогрессу при решении задач обработки естественного языка.

В этой главе мы создадим свою версию исходной модели GPT, обученную на меньшем количестве данных, но использующую те же компоненты и основные принципы.



Запуск кода этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу notebooks/09_transformer/01_gpt/gpt.ipynb в репозитории книги. Он был заимствован (и адаптирован) из превосходного руководства по GPT (<https://oreil.ly/J86pg>), написанного Апурвом Нанданом (Apoorv Nandan).

Набор данных Wine Reviews

Далее мы используем набор данных Wine Reviews, доступный на сайте Kaggle (<https://oreil.ly/DC9EG>). Он содержит более 130 000 обзоров вин с сопутствующими метаданными, такими как описание и цена.

Загрузить набор данных можно с помощью сценария, находящегося в репозитории книги, как показано в примере 9.1. Он сохранит обзоры вин и сопутствующие метаданные в локальной папке `/data`.

Пример 9.1. Загрузка набора данных Wine Reviews

```
bash scripts/download_kaggle_data.sh zynicide wine-reviews
```

Шаги подготовки данных идентичны шагам, использованным в главе 5, где мы подготавливали данные для передачи в LSTM, поэтому не будем подробно описывать их здесь. Вот эти шаги (см. также рис. 9.1).

1. Загрузить данные и создать список строк с описаниями вин.
2. Добавить пробелы вокруг знаков препинания, чтобы каждый знак воспринимался как отдельное слово.
3. Передать строки в слой `TextVectorization`, который лексемизирует данные и дополнит/обрежет каждую строку до фиксированной длины.
4. Создать обучающий набор с текстовыми строками, состоящими из лексем. Прогнозируемыми выходными данными будут те же строки, смещенные на одну лексему.



Рис. 9.1. Обработка данных для передачи в трансформер

Внимание

Первый шаг к пониманию работы GPT — знакомство с особенностями работы *механизма внимания*. Именно он делает архитектуру трансформера уникальной и отличной от традиционных подходов к языковому моделированию. Разобравшись с механизмом внимания, мы увидим, как он используется в архитектурах трансформера, таких как GPT.

Когда вы пишете предложение, на выбор следующего слова в нем влияют слова, которые вы уже написали. Предположим, что вы начинаете писать следующее предложение: «Этот розовый слон решил попытаться залезть в этот автомобиль, но он был слишком...»

Очевидно, что следующее слово должно быть синонимом слова «большой». Как мы это узнали?

Некоторые слова в предложении особенно важны и помогают нам принять решение. Например, тот факт, что это слон, а не ленивец, означает, что мы предпочитаем

больших, а не *медленных*. Если бы вместо автомобиля был бассейн, то мы могли бы выбрать «*напуган*» как возможную альтернативу слову «*большой*». Наконец, действие по посадке в автомобиль подразумевает, что проблема заключается в размере: если бы слон пытался раздавить машину, то мы могли бы выбрать последнее слово «*быстрым*», и теперь оно относилось бы к автомобилю.

Остальные слова в предложении совершенно неважны. Например, розовый цвет слона не влияет на выбор последнего слова. Точно так же другие второстепенные слова («*этот*», «*но*», «*он*» и т. д.) лишь придают предложению грамматическую форму и неважны для определения нужного прилагательного. Другими словами, мы *обращаем внимание* на определенные слова в предложении и почти полностью игнорируем другие. Было бы здорово, если бы наша модель могла поступать так же!

Механизм внимания, также известный как *модуль внимания*, в трансформере предназначен именно для этого. Он решает, на какие входные данные обратить особое внимание, чтобы эффективно извлечь полезную информацию, не путаясь в ненужных деталях. Этот механизм делает трансформер легко адаптируемым к различным обстоятельствам, поскольку может решать, где искать информацию во время вывода.

Рекуррентный слой, напротив, пытается создать общее скрытое состояние, фиксирующее общее представление входных данных на каждом временном шаге. Слабость этого подхода заключается в том, что многие слова, включенные в скрытый вектор, не будут иметь прямого отношения к текущей задаче, например к предсказанию следующего слова, как мы только что видели. Модули внимания не страдают этой проблемой, потому что могут выбирать, как объединять информацию из соседних слов, в зависимости от контекста.

Запросы, ключи и значения

Как же модуль внимания решает, где искать информацию? Прежде чем углубиться в детали, рассмотрим работу этого механизма с высоты птичьего полета на примере *розового слона*.

Представьте, что мы хотим предсказать слово, следующее за «*слишком*». Чтобы помочь решить эту задачу, мы учитываем другие слова, но их вклад оцениваем с учетом наших знаний об их влиянии на предсказание последующих слов. Например, по наличию слова «*слон*» можно с уверенностью утверждать, что оно, скорее всего, связано с размером или громкостью, тогда как слово «*был*» почти ничего не может предложить, чтобы сузить выбор.

Другими словами, мы можем думать о модуле внимания как о своего рода системе поиска информации, в которой запрос («Какое слово следует за «*слишком*»?»)

преобразуется в хранилище *ключей/значений* (другие слова в предложении), а результирующий вывод представляет собой сумму значений, взвешенных по *резонансу* между запросом и каждым ключом.

Теперь подробно рассмотрим этот процесс (рис. 9.2), снова со ссылкой на предложение о розовом слоне.

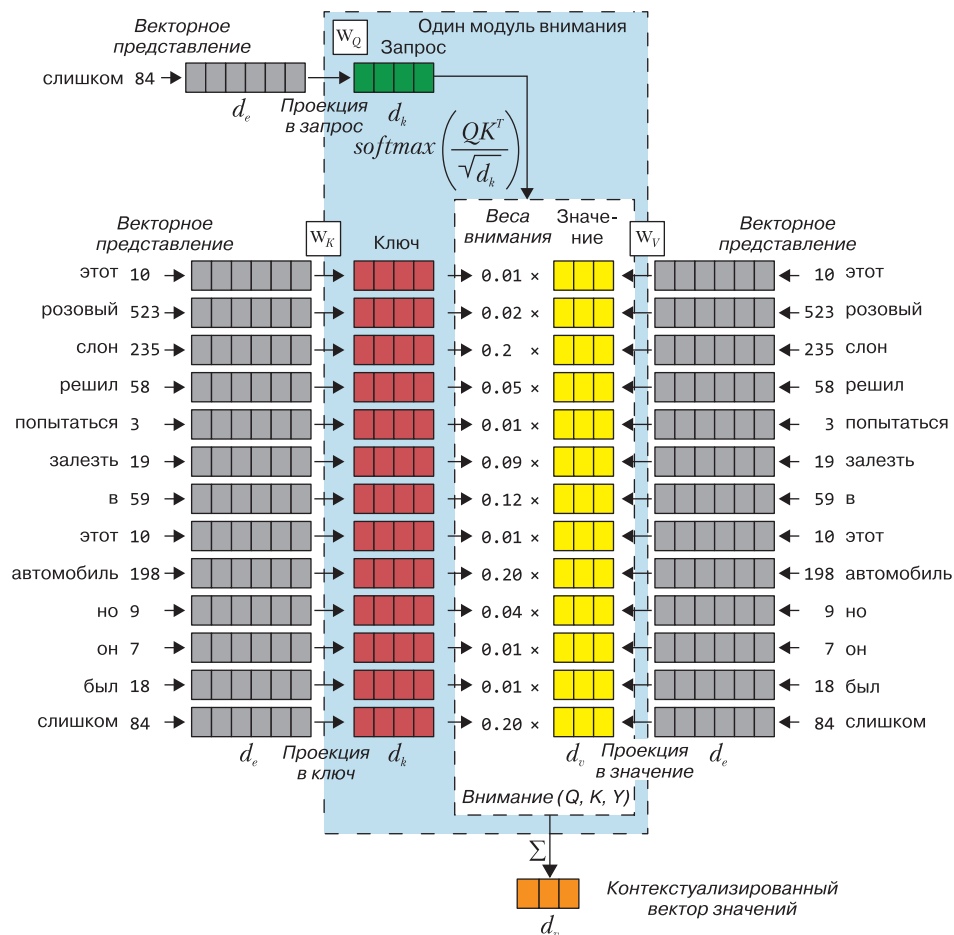


Рис. 9.2. Механика работы модуля внимания

Запрос (Q) можно рассматривать как представление текущей задачи (например, «Какое слово следует за «слишком»?»). В этом примере оно получается из векторного представления слова «слишком» путем его передачи через матрицу весов W_Q для изменения размерности вектора с d_e на d_k .

Векторы *ключей* (K) — это представления слов в предложении. Их можно рассматривать как описания видов задач прогнозирования, в решении которых может помочь каждое слово. Они выводятся аналогично запросу передачей каждого векторного представления через матрицу весов W_K , чтобы изменить размерность каждого вектора с d_e на d_k . Обратите внимание на то, что ключи и запрос имеют одинаковую длину (d_k).

Внутри модуля внимания каждый ключ сравнивается с запросом с использованием скалярного произведения каждой пары векторов (QK^T). Вот почему ключи и запрос должны быть одинаковой длины. Чем больше это число для конкретной пары «ключ/запрос», тем больше ключ резонирует с запросом и тем больший вклад ему разрешено вносить в выходные данные модуля внимания. Результирующий вектор масштабируется по $\sqrt{d_k}$, чтобы дисперсия векторной суммы оставалась стабильной (приблизительно равной 1), иприменяется функция активации softmax, чтобы сумма вкладов была равна 1. Это вектор *весов внимания*.

Векторы *значений* (V) тоже являются представлениями слов в предложении — о них можно думать как о невзвешенном вкладе каждого слова. Они получаются путем передачи каждого векторного представления через матрицу весов W_V для изменения размерности каждого вектора с d_e на d_v . Обратите внимание на то, что векторы значений не обязательно должны иметь ту же длину, что и векторы ключей и запроса (но это часто делают для простоты).

Векторы значений умножаются на веса внимания, чтобы получить *внимание* для заданных Q , K и V , как показано в следующем уравнении:

$$\text{внимание}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Чтобы получить окончательный вектор на выходе из модуля внимания, значения внимания суммируют, чтобы получить вектор длины d_v . Этот *контекстный вектор* отражает смешение мнений слов в предложении для предсказания слова, следующего за «слишком».

Многоголовое внимание

Нет причин останавливаться на одном модуле внимания! В Keras можно создать слой `MultiHeadAttention`, объединяющий выходы нескольких модулей внимания, позволяя каждому обучить свой механизм внимания, чтобы слой в целом мог изучать более сложные отношения.

Объединенные выходные данные передаются через общую конечную матрицу весов W_O для проецирования вектора в желаемое выходное пространство,

которое в нашем случае совпадает с входным пространством запроса (d_e), благодаря чему слои можно размещать последовательно друг за другом.

На рис. 9.3 показано, как формируются выходные данные слоя `MultiHeadAttention`. В Keras для создания такого слоя можно просто написать строку, показанную в примере 9.2.

Пример 9.2. Создание слоя `MultiHeadAttention` с помощью Keras

```
layers.MultiHeadAttention(  
    num_heads = 4, ❶  
    key_dim = 128, ❷  
    value_dim = 64, ❸  
    output_shape = 256 ❹  
)
```

- ❶ Этот слой многоголового внимания имеет четыре модуля.
- ❷ Ключи (и запрос) — это векторы длиной 128.
- ❸ Значения (и, соответственно, выходные данные каждого модуля) — это векторы длиной 64.
- ❹ Выходной вектор имеет длину 256.

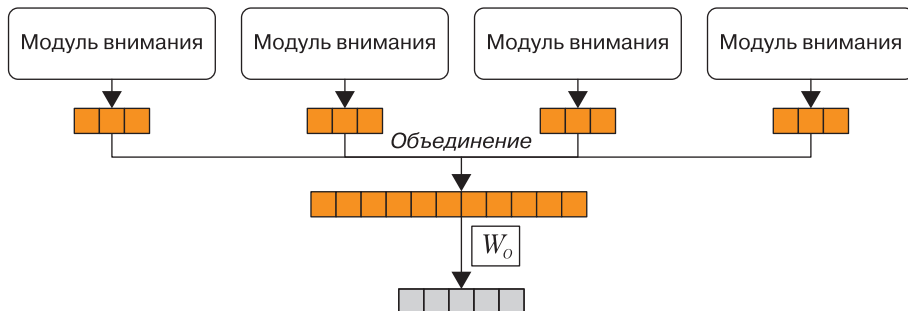


Рис. 9.3. Слой многоголового внимания с четырьмя модулями внимания

Причинно-следственная маскировка

До сих пор мы предполагали, что входной запрос, поступающий в модуль внимания, имеет вид единственного вектора. Однако для увеличения эффективности желательно, чтобы во время обучения слой внимания мог работать со всеми входными словами одновременно, предсказывая для каждого, каким будет следующее слово. Другими словами, желательно, чтобы модель GPT могла параллельно обрабатывать группу векторов запросов, то есть матрицу.

Можно подумать, что достаточно объединить векторы в матрицу и позволить линейной алгебре сделать все остальное. Это правда, но нам нужно сделать еще один шаг — применить маску к скалярному произведению запроса/ключа, чтобы избежать утечки информации из будущих слов. Этот шаг известен как *причинно-следственная маскировка* и показан на рис. 9.4.

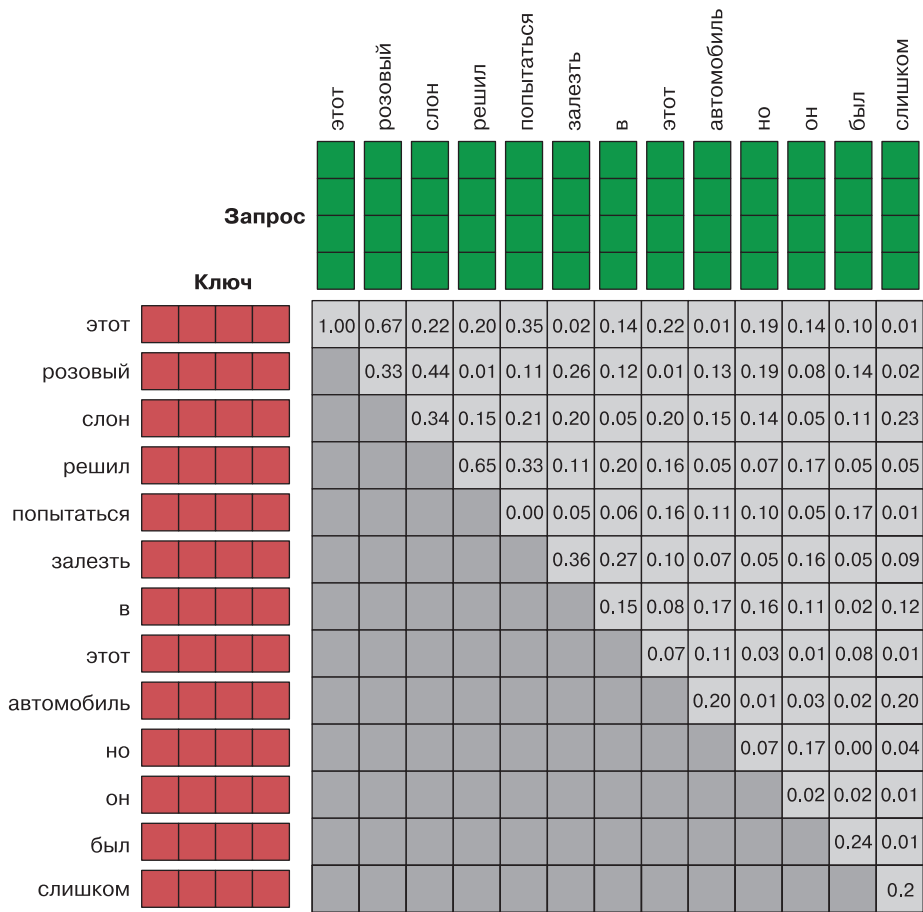


Рис. 9.4. Матрица вычисления оценок внимания для пакета входных запросов с применением причинно-следственной маски для сокрытия ключей, отсутствующих в запросе (потому что они находятся далее в предложении)

Без этой маски наша модель GPT смогла бы идеально угадать следующее слово в предложении, поскольку в качестве признака использовала бы ключ самого слова! Код создания причинно-следственной маски показан в примере 9.3, а полученный массив `numpy` (транспонированный в соответствии со схемой) — на рис. 9.5.

Пример 9.3. Функция создания причинно-следственной маски

```
def causal_attention_mask(batch_size, n_dest, n_src, dtype):
    i = tf.range(n_dest)[: , None]
    j = tf.range(n_src)
    m = i >= j - n_src + n_dest
    mask = tf.cast(m, dtype)
    mask = tf.reshape(mask, [1, n_dest, n_src])
    mult = tf.concat(
        [tf.expand_dims(batch_size, -1), tf.constant([1, 1], dtype=tf.int32)], 0
    )
    return tf.tile(mask, mult)

np.transpose(causal_attention_mask(1, 10, 10, dtype = tf.int32)[0])
```

```
array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]], dtype=int32)
```

Рис. 9.5. Причинно-следственная маска в виде массива numpy: 1 означает «не замаскирован», а 0 — «замаскирован»



Причинное маскирование требуется только в *трансформерах-декодировщиках*, таких как GPT, перед которыми стоит задача последовательно генерировать лексемы с учетом предыдущих лексем. Поэтому очень важно маскировать будущие лексемы во время обучения.

Другие разновидности трансформеров (например, *трансформеры-кодировщики*) не нуждаются в причинно-следственной маскировке, поскольку обучаются не для предсказания следующей лексемы. Например, модель BERT, разработанная в Google, предсказывает замаскированные слова в данном предложении, поэтому может использовать как предыдущий, так и последующий контекст слова в запросе [Devlin et al., 2018].

В конце главы мы рассмотрим различные типы трансформеров более подробно.

На этом мы завершаем описание механизма многоголового внимания, который присутствует во всех трансформерах. Примечательно, что обучаемые параметры такого влиятельного слоя состоят не более чем из трех плотно связанных матриц весов для каждого модуля внимания (W_Q , W_K , W_V) и одной матрицы весов для

изменения формы выхода (W_o). В слое многоголового внимания вообще нет сверточных или рекуррентных механизмов!

Далее отступим на шаг назад и посмотрим, как слой многоголового внимания образует лишь одну часть более крупного компонента, известного как *блок трансформера*.

Блок трансформера

Блок трансформера — это единый компонент внутри модели трансформера, добавляющий к слою многоголового внимания дополнительные пропускающие связи, слои прямого распространения (полносвязные) и нормализацию. Схема блока трансформера показана на рис. 9.6.

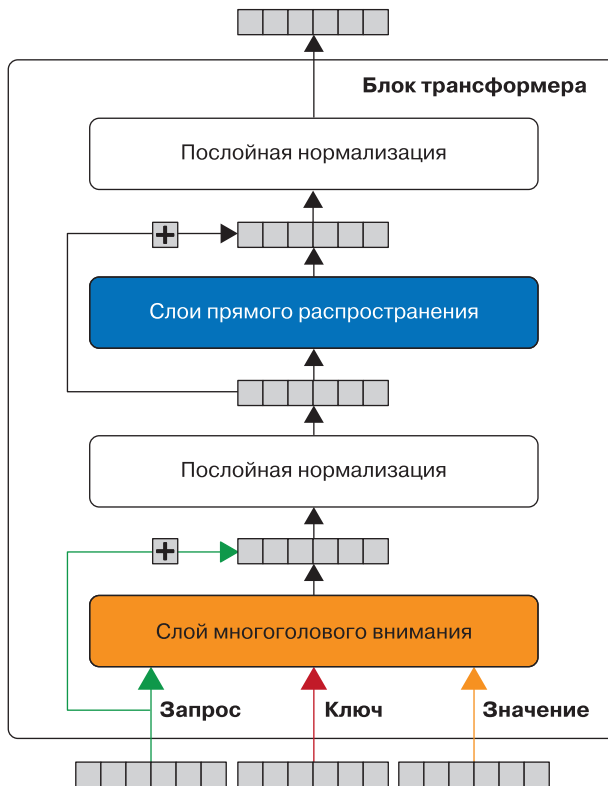


Рис. 9.6. Блок трансформера

Во-первых, обратите внимание на то, что запрос одновременно передается в многоголовый слой внимания и добавляется в выходные данные — это

пропускающая связь, часто встречающаяся в современных архитектурах глубокого обучения. Ее наличие позволяет строить очень глубокие нейронные сети, которые не так сильно страдают от проблемы затухания градиента, поскольку пропускающая связь обеспечивает безградиентную *магистраль*, позволяющую сети беспрепятственно передавать информацию в прямом направлении.

Во-вторых, в блоке трансформера выполняется *послойная нормализация*, обеспечивающая стабильность процесса обучения. В этой книге мы уже видели, как действует слой пакетной нормализации, нормализующий выходные данные каждого канала так, чтобы среднее значение было равно 0, а стандартное отклонение — 1. Статистика нормализации рассчитывается для пакетов и пространственных измерений.

Послойная нормализация в блоке трансформера, напротив, нормализует каждую позицию каждой последовательности в пакете путем вычисления статистики нормализации по каналам. Это полная противоположность пакетной нормализации с точки зрения расчета статистики нормализации. Схема, иллюстрирующая разницу между пакетной и послойной нормализацией, показана на рис. 9.7.

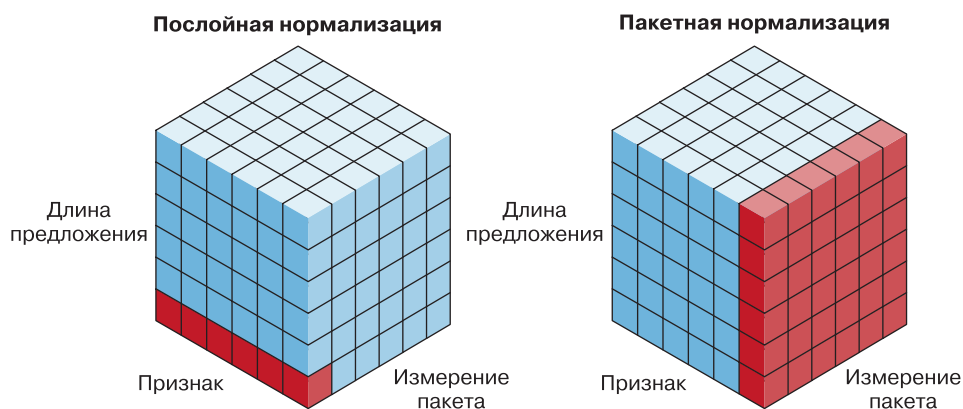


Рис. 9.7. Послойная и пакетная нормализация — статистика нормализации вычисляется по синим ячейкам [Shen et al., 2020]



Послойная и пакетная нормализация

Послойная нормализация использовалась в оригинальной статье с описанием GPT и часто задействуется в задачах обработки текста, чтобы избежать создания зависимостей нормализации между последовательностями в пакете. Однако некоторые исследователи, такие как Шен (Shen) и др., в своих работах оспаривают это предположение, показывая, что некоторые формы пакетной нормализации все еще можно использовать в трансформерах и получать более качественные результаты, чем при помощи более традиционной послойной нормализации.

Наконец, в блок трансформера включены несколько слоев прямого распространения (то есть полносвязных), чтобы компонент мог извлекать признаки более высокого уровня по мере углубления сети.

Реализация блока трансформера с помощью Keras показана в примере 9.4.

Пример 9.4. Слой TransformerBlock, полученный с помощью Keras

```
class TransformerBlock(layers.Layer):
    def __init__(self, num_heads, key_dim, embed_dim, ff_dim, dropout_rate=0.1): ❶
        super(TransformerBlock, self).__init__()
        self.num_heads = num_heads
        self.key_dim = key_dim
        self.embed_dim = embed_dim
        self.ff_dim = ff_dim
        self.dropout_rate = dropout_rate
        self.attn = layers.MultiHeadAttention(
            num_heads, key_dim, output_shape = embed_dim
        )
        self.dropout_1 = layers.Dropout(self.dropout_rate)
        self.ln_1 = layers.LayerNormalization(epsilon=1e-6)
        self.ffn_1 = layers.Dense(self.ff_dim, activation="relu")
        self.ffn_2 = layers.Dense(self.embed_dim)
        self.dropout_2 = layers.Dropout(self.dropout_rate)
        self.ln_2 = layers.LayerNormalization(epsilon=1e-6)

    def call(self, inputs):
        input_shape = tf.shape(inputs)
        batch_size = input_shape[0]
        seq_len = input_shape[1]
        causal_mask = causal_attention_mask(
            batch_size, seq_len, seq_len, tf.bool
        ) ❷
        attention_output, attention_scores = self.attn(
            inputs,
            inputs,
            attention_mask=causal_mask,
            return_attention_scores=True
        ) ❸
        attention_output = self.dropout_1(attention_output)
        out1 = self.ln_1(inputs + attention_output) ❹
        ffn_1 = self.ffn_1(out1) ❺
        ffn_2 = self.ffn_2(ffn_1)
        ffn_output = self.dropout_2(ffn_2)
        return (self.ln_2(out1 + ffn_output), attention_scores) ❻
```

❶ Подслои, составляющие слой TransformerBlock, определяются в функции инициализации.

❷ Создается причинно-следственная маска, чтобы скрыть будущие ключи от запроса.

- 3 Создается многоголовый слой внимания с заданными масками внимания.
- 4 Первый слой *сложения и нормализации*.
- 5 Слои прямого распространения.
- 6 Второй слой *сложения и нормализации*.

Позиционное представление

Остался еще один, последний шаг, который нужно сделать, прежде чем мы сможем собрать все воедино для обучения своей модели GPT. Возможно, вы заметили, что в слое многоголового внимания нет ничего, что заботилось бы об упорядочении ключей. Скалярное произведение между каждым ключом и запросом вычисляется параллельно, а не последовательно, как в рекуррентной нейронной сети. Это преимущество, обеспечивающее возможность распараллеливания, но одновременно и проблема, поскольку нам явно нужен слой внимания, чтобы иметь возможность спрогнозировать разные выходные данные для следующих двух предложений:

- «Собака посмотрела на мальчика и... (залаяла?)».
- «Мальчик посмотрел на собаку и... (улыбнулся?)».

Для решения этой проблемы используем метод, называемый *позиционным представлением*, для создания входных данных для начального блока трансформера. Вместо того чтобы кодировать каждую лексему в виде *векторного представления*, мы закодируем также позицию лексемы, применяя *позиционное представление*.

Векторное представление лексемы создается с помощью стандартного слоя **Embedding**, преобразующего каждую лексему в изучаемый вектор. Точно так же мы можем создать позиционное представление, используя стандартный слой **Embedding** для преобразования каждой целочисленной позиции в изучаемый вектор.



В GPT для формирования представления позиции используется слой **Embedding**, тогда как в оригинальной статье о трансформерах применялись тригонометрические функции. Мы рассмотрим эту альтернативу в главе 11, когда будем изучать генерирование музыки.

Чтобы построить объединенное представление лексемы и позиции, к позиционному представлению добавляется векторное представление лексемы (рис. 9.8). В результате значение и положение каждого слова в последовательности фиксируются в одном векторе.

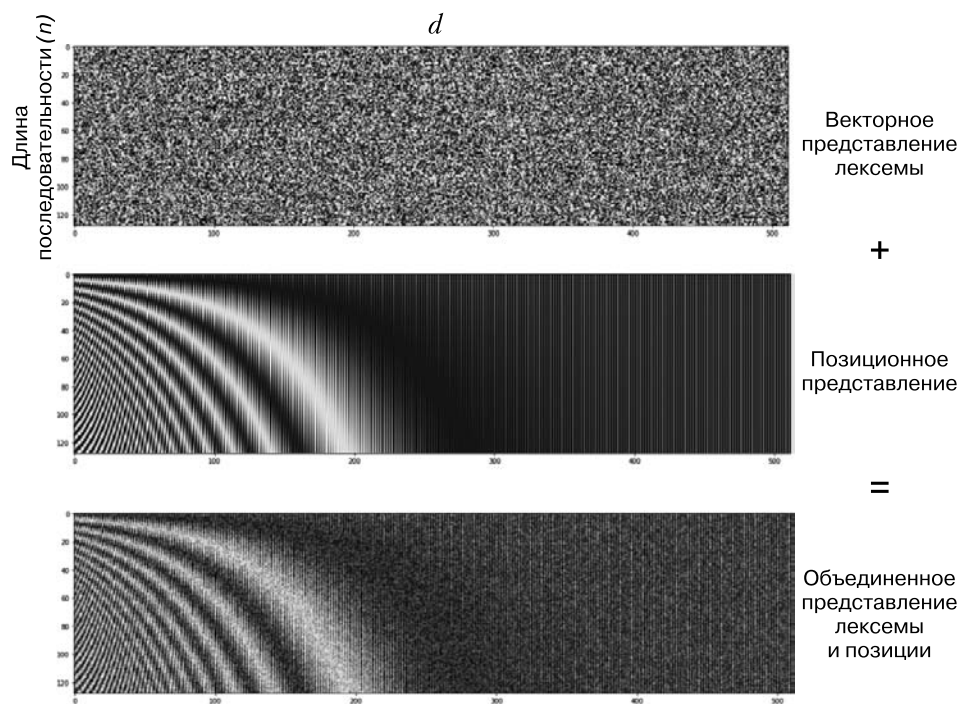


Рис. 9.8. Векторные представления лексем складываются с позиционными представлениями, чтобы получить объединенное представление лексемы и позиции

Код, определяющий слой `TokenAndPositionEmbedding`, показан в примере 9.5.

Пример 9.5. Слой `TokenAndPositionEmbedding`

```
class TokenAndPositionEmbedding(layers.Layer):
    def __init__(self, maxlen, vocab_size, embed_dim):
        super(TokenAndPositionEmbedding, self).__init__()
        self.maxlen = maxlen
        self.vocab_size = vocab_size
        self.embed_dim = embed_dim
        self.token_emb = layers.Embedding(
            input_dim=vocab_size, output_dim=embed_dim
        ) ❶
        self.pos_emb = layers.Embedding(input_dim=maxlen, output_dim=embed_dim) ❷

    def call(self, x):
        maxlen = tf.shape(x)[-1]
        positions = tf.range(start=0, limit=maxlen, delta=1)
        positions = self.pos_emb(positions)
        x = self.token_emb(x)
        return x + positions ❸
```

- ❶ С помощью слоя `Embedding` создаются векторные представления лексем.
- ❷ Также с помощью слоя `Embedding` создаются представления позиций лексем.
- ❸ Выходные данные слоя объединяются для получения общего представления лексем и их позиций.

Обучение GPT

Теперь мы готовы создать и обучить модель GPT! Чтобы собрать все воедино, нужно передать входной текст в слой создания объединенного представления лексемы и позиции, а затем в блок трансформера. На выходе сети будет находиться простой слой `Dense` с функцией активации `softmax` по количеству слов в словаре.



Для простоты мы будем использовать только один блок трансформера, а не 12, как в статье.

Общая архитектура сети показана на рис. 9.9, а конструирующий ее код — в примере 9.6.

Пример 9.6. Создание модели GPT с помощью Keras

```
MAX_LEN = 80
VOCAB_SIZE = 10000
EMBEDDING_DIM = 256
N_HEADS = 2
KEY_DIM = 256
FEED_FORWARD_DIM = 256

inputs = layers.Input(shape=(None,), dtype=tf.int32) ❶
x = TokenAndPositionEmbedding(MAX_LEN, VOCAB_SIZE, EMBEDDING_DIM)(inputs) ❷
x, attention_scores = TransformerBlock(
    N_HEADS, KEY_DIM, EMBEDDING_DIM, FEED_FORWARD_DIM
)(x) ❸
outputs = layers.Dense(VOCAB_SIZE, activation = 'softmax')(x) ❹
gpt = models.Model(inputs=inputs, outputs=[outputs, attention]) ❺
gpt.compile("adam", loss=[losses.SparseCategoricalCrossentropy(), None]) ❻
gpt.fit(train_ds, epochs=5)
```

- ❶ Дополняет входные данные нулями.
- ❷ Создается векторное представление текста с помощью слоя `TokenAndPositionEmbedding`.

- 3 Векторное представление передается в `TransformerBlock`.
- 4 Результат преобразования передается через полносвязный слой с функцией активации `softmax` для прогнозирования распределения последующего слова.
- 5 Модель принимает последовательность лексем на входе и выводит прогнозируемое распределение следующего слова. Также возвращаются выходные данные блока трансформера, чтобы мы могли проверить, куда направляет свое внимание модель.
- 6 Модель компилируется с функцией потерь `SparseCategoricalCrossentropy` в прогнозируемом распределении слов.

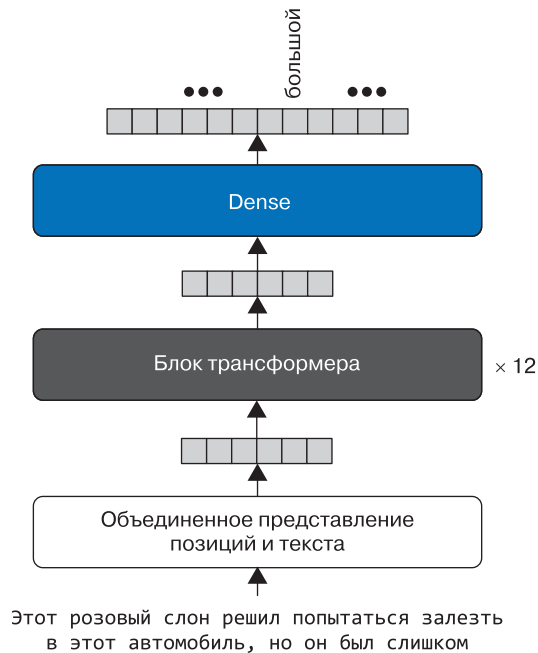


Рис. 9.9. Упрощенная архитектура модели GPT

Анализ GPT

Теперь, скомпилировав и обучив модель GPT, мы можем начать использовать ее для генерации длинных строк текста. Также можем вывести веса внимания из `TransformerBlock`, чтобы понять, где трансформер ищет информацию на разных этапах процесса генерации.

Генерация текста

Сгенерировать новый текст можно с помощью следующего процесса.

1. Передать в сеть существующую последовательность слов и запустить предсказание следующего слова.
2. Добавить это слово в конец существующей последовательности и повторить.

Сеть будет выводить набор вероятностей для каждого слова, из которого можно сделать выборку, поэтому мы можем сделать генерацию текста стохастической, а не детерминированной.

Используем тот же класс `TextGenerator`, который применялся в главе 5 для генерации текста с помощью LSTM, включая параметр `temperature`, определяющий, насколько детерминированным должен быть процесс выборки. Посмотрим на результаты, полученные с двумя разными значениями температуры (рис. 9.10).

`temperature = 1.0`

Generated text:

wine review : us : washington : chenin blanc : a light , medium - bodied wine , this light - bodied expressi on is not a lot of enjoyment . it ' s simple with butter and vanilla flavors that frame mixed with expressiv e fruit . it ' s juicy and tangy with a lemon lingers on the finish .

`temperature = 0.5`

Generated text:

wine review : italy : piedmont : nebbiolo : this opens with aromas of french oak , menthol and a whiff of to ast . the straightforward palate offers red cherry , black raspberry jam and a hint of star anise alongside firm but rather fleeting tannins , drink through 2016 .

Рис. 9.10. Сгенерированный текст со значениями `temperature = 1.0` и `temperature = 0.5`

В отношении этих двух отрывков следует отметить, что, во-первых, оба стилистически похожи на обзоры вин из обучающего набора. Оба начинаются с указания региона и типа вина, и этот тип остается неизменным на протяжении всего фрагмента (например, цвет вина не меняется в середине). Как мы видели в главе 5, текст, сгенерированный с параметром `temperature = 1.0`, более смелый и поэтому менее точный, чем пример с параметром `temperature = 0.5`. Таким образом, создание нескольких выборок с параметром `temperature = 1.0` приведет к большему разнообразию, поскольку модель выполняет выборку из распределения вероятностей с большей дисперсией.

Просмотр оценок внимания

Мы также можем попросить модель сообщить, какое внимание уделяется каждому слову при выборе следующего слова в предложении. `TransformerBlock` выводит веса внимания для каждого модуля, которые представляют собой распределение `softmax`, полученное по предыдущим словам в предложении.

Для демонстрации на рис. 9.11 показаны пять главных лексем с наибольшей вероятностью трех разных входных запросов, а также среднее внимание обоих модулей по отношению к каждому предыдущему слову. Предшествующие слова окрашены в соответствии с их оценками внимания, усредненными по двум модулям внимания. Чем темнее цвет, тем больше внимания уделяется слову.

```
wine review : germany :
pfalz:      51.53%
mosel:      41.21%
rheingau:   4.27%
rheinessen: 2.16%
franken:    0.44%
-----

wine review : germany : rheingau : riesling : this is a ripe , full - bodied
riesling:    46.56%
,:          27.78%
wine:        16.88%
and:         4.58%
yet:         1.33%
-----

wine review : germany : rheingau : riesling : this is a ripe , full - bodied riesling
with a touch of residual sugar : it ' s a slightly
sweet:       94.23%
oily:        1.25%
viscous:     1.09%
bitter:      0.88%
honeyed:     0.66%
-----
```

Рис. 9.11. Распределение вероятностей следующих слов в последовательностях

В первом примере при выборе слова, относящегося к региону, внимание модели направлено на страну (`germany` — Германия). В этом определенно есть смысл! Чтобы выбрать регион, необходимо получить много информации из слов, относящихся к стране. Первым двум словам (`wine review` — обзор вина) уделяется меньше внимания, поскольку они не содержат никакой полезной информации о регионе.

Во втором примере модель должна сослаться на сорт винограда (`riesling` — рислинг), поэтому она обращает на него внимание, когда он упоминается в первый раз. Модель может получить эту информацию, обращая внимание непосредственно на слово независимо от того, как далеко оно находится в предложении (в пределах верхнего предела в 80 слов). Обратите внимание на то, что этим

наша модель сильно отличается от рекуррентной нейронной сети, которая полагается на скрытое состояние для хранения всей интересной информации на протяжении всей последовательности, чтобы ее можно было использовать в случае необходимости, — это гораздо менее эффективный подход.

Последний пример показывает, как модель GPT выбирает подходящее прилагательное на основе комбинации информации. Здесь внимание снова обращено на сорт винограда (*riesling*), а также на то, что он содержит *residual sugar* (остаточный сахар). Поскольку из сорта рислинг обычно производят сладкие вина, а сахар уже упоминался, имеет смысл охарактеризовать его, например, как *slightly sweet* (слегка сладкое), а не *slightly earthy* (слегка землистое).

Невероятно интересно иметь возможность извлекать такую информацию из сети, чтобы понять, откуда именно она черпает информацию, принимая решение о выборе каждого последующего слова. Я настоятельно рекомендую поиграть с входными запросами и посмотреть, сможете ли вы заставить модель уделять внимание словам, находящимся в самом конце предложения, а заодно убедиться в превосходстве моделей, основанных на внимании, над более традиционными рекуррентными моделями.

Другие трансформеры

Наша модель GPT является *трансформером-декодировщиком*: она генерирует текстовую строку по одной лексеме за раз и использует причинно-следственную маскировку, чтобы обрабатывать только предыдущие слова во входной строке. Существуют также *трансформеры-кодировщики*, которые не задействуют причинно-следственную маскировку — они обрабатывают входную строку целиком, чтобы извлечь значимое контекстное представление входных данных. Для других задач, таких как перевод с одного языка на другой, тоже существуют трансформеры кодировщики-декодировщики, способные переводить текстовую строку с одного языка на другой, — этот тип модели содержит как блоки трансформера-кодировщика, так и блоки трансформера-декодировщика.

В табл. 9.1 представлены три типа трансформеров с наиболее удачными примерами реализации каждой архитектуры и типичными вариантами применения.

Таблица 9.1. Три архитектуры трансформера

Тип	Примеры	Случаи использования
Кодировщик	BERT (Google)	Классификация предложений, распознавание именованных сущностей, выборочные ответы на вопросы
Кодировщик-декодировщик	T5 (Google)	Обобщение, перевод, ответы на вопросы
Декодировщик	GPT-3 (OpenAI)	Генерирование текста

Хорошо известным примером трансформера-кодировщика является модель *двунаправленных представлений кодировщика на основе трансформера* (Bidirectional Encoder Representations from Transformers, BERT), разработанная в Google [Девлин (Devlin) и др., 2018], которая предсказывает пропущенные слова в предложении с учетом контекста как до, так и после пропущенного слова.



Трансформеры-кодировщики

Трансформеры-кодировщики обычно используются в задачах, требующих понимания входных данных в целом, таких как классификация предложений, распознавание именованных сущностей и выборочные ответы на вопросы. Они не применяются для генерации текста, поэтому мы не будем их рассматривать в этой книге. За дополнительной информацией обращайтесь к книге Льюиса Танстола (Lewis Tunstall) и др. *Natural Language Processing with Transformers* (O'Reilly).

В следующих разделах рассмотрим особенности работы трансформера кодировщика-декодировщика и обсудим расширения исходной архитектуры модели GPT, созданные в OpenAI, включая модель ChatGPT, специально разработанную для диалоговых приложений.

T5

Примером современного трансформера, использующего структуру кодировщика-декодировщика, может служить модель T5, созданная в Google [Raffel et al., 2019]. Эта модель сводит некоторые задачи к преобразованию текста в текст, включая перевод на разные языки, оценку лингвистической приемлемости, определение сходства предложений и обобщение документов (рис. 9.12).

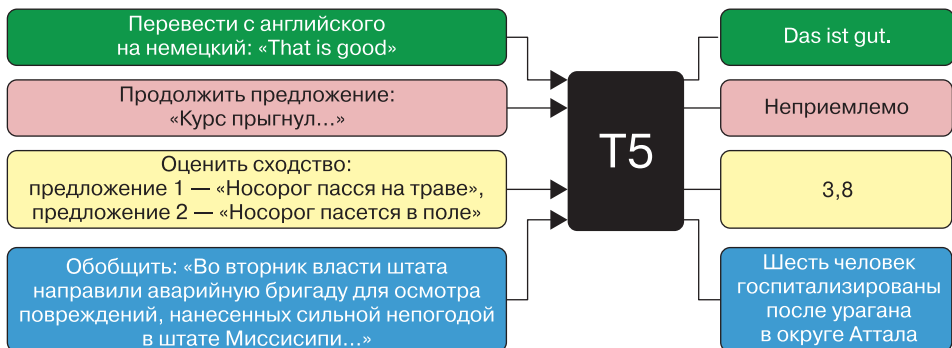


Рис. 9.12. Примеры того, как T5 сводит ряд задач к преобразованию текста в текст, включая перевод на разные языки, оценку лингвистической приемлемости, определение сходства предложений и обобщение документов (источник: [Raffel et al., 2019], <https://ai.googleblog.com/2020/02/exploring-transfer-learning-with-t5.html>)

Архитектура модели T5 очень близка к архитектуре кодировщика-декодировщика, использованной в оригинальной статье о трансформерах и показанной на рис. 9.13. Основное отличие: T5 обучается на огромном корпусе текста объемом 750 Гбайт (Colossal Clean Crawled Corpus, или C4), тогда как модель в оригинальной статье была сосредоточена на переводе с одного языка на другой и обучалась на корпусе объемом 1,4 Гбайт, состоящем из пар предложений на английском и немецком языках.

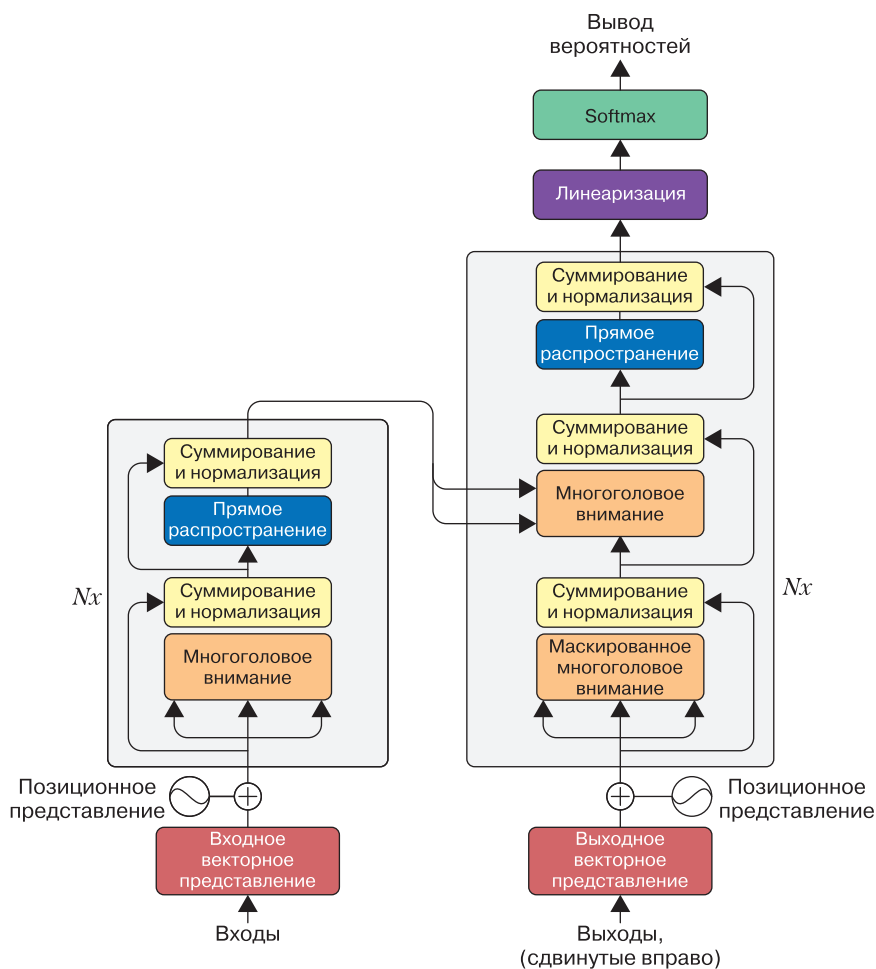


Рис. 9.13. Модель трансформера кодировщика-декодировщика: серые прямоугольники — это блоки трансформеров (источник: [Vaswani et al., 2017], <https://arxiv.org/abs/1706.03762>)

Большая часть этой диаграммы вам уже знакома — мы видим здесь повторяющиеся блоки трансформера и использование позиционного представления для фиксации порядка входных последовательностей лексем. Однако эта модель имеет два ключевых отличия от модели GPT, построенной ранее в этой главе.

- Набор блоков трансформеров-кодировщиков слева кодирует последовательность, которую нужно перевести. В слое внимания не выполняется причинно-следственная маскировка, потому что кодировщик не генерирует дополнительный текст, расширяющий переводимую последовательность, а просто должен получить хорошее общее представление последовательности, которое можно передать в декодировщик. Соответственно, из слоев внимания в кодировщике можно полностью убрать причинно-следственное маскирование, чтобы он мог уловить все перекрестные зависимости между словами независимо от их порядка.
- Набор блоков трансформеров-декодировщиков справа генерирует текст перевода. Начальный слой внимания является *автореферентным* (то есть ключ, значение и запрос поступают с одного и того же входа), а чтобы информация из будущих лексем не просочилась в текущее слово, которое нужно предсказать, используется причинно-следственное маскирование. Однако следующий уровень внимания извлекает ключ и значение из кодировщика, оставляя только запрос, переданный самим декодировщиком. Это называется *перекрестным вниманием* и означает, что декодировщик может обращать внимание на представление, созданное кодировщиком из входной последовательности, которую нужно перевести. Именно так декодировщик узнает, какой смысл должен передать перевод.

На рис. 9.14 показан пример действия перекрестного внимания. Два модуля внимания в слое декодировщика могут работать вместе, чтобы дать правильный перевод на немецкий артикля *the*, когда он используется в словосочетании *the street*. В немецком языке есть три определенных артикля — *der*, *die* и *das*, которые применяются в зависимости от рода существительного, но трансформер знает, что должен выбрать артикль *die*, потому что один модуль внимания следит за словом *street* (в немецком языке имеет женский род), а другой — за переводимым словом (*the*).



Этот пример взят из репозитория Tensor2Tensor на GitHub (<https://oreil.ly/84IIA>), где имеется блокнот Colab, в котором можно поэкспериментировать с обученной моделью трансформера и посмотреть, как механизмы внимания кодировщика и декодировщика влияют на перевод заданного предложения на немецкий язык.

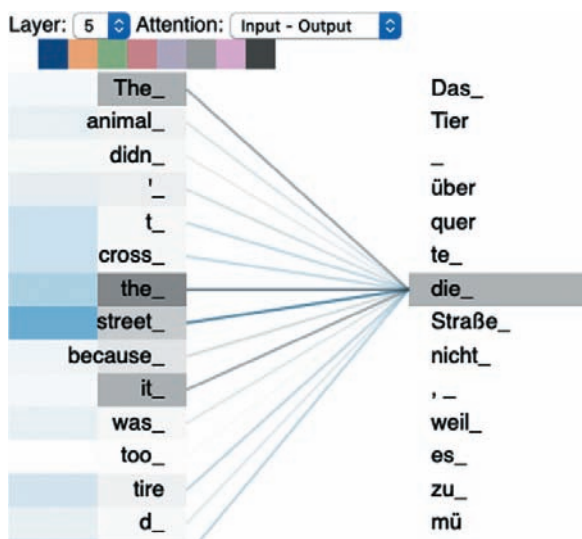


Рис. 9.14. Пример того, как один модуль внимания следит за словом *the*, а другой — за словом *street*, чтобы правильно перевести английский артикль *the* в немецкий *die* — определенный артикль женского рода для слова *Straße*

GPT-3 и GPT-4

С момента публикации статьи о GPT в 2018 году OpenAI выпустила несколько обновленных версий исходной модели. Они перечислены в табл. 9.2.

Архитектура модели GPT-3 очень похожа на архитектуру исходной модели GPT, только она намного больше и обучена на гораздо большем количестве данных. На момент написания этой книги близилась к концу разработка модели GPT-4, но OpenAI еще не опубликовала информацию о ее структуре и размерах, хотя мы знаем, что она может принимать изображения, поэтому является первой мультимодальной моделью. Веса моделей GPT-3 и GPT-4 закрыты, хотя сами они доступны для использования с помощью коммерческого инструмента и API (<https://platform.openai.com/>).

Кроме того, GPT-3 можно дообучить на ваших собственных обучающих данных, благодаря этому вы сможете получить несколько примеров того, как она будет реагировать на заданный стиль запроса, физически обновляя веса сети. Во многих случаях в этом нет необходимости, поскольку реакцию GPT-3 на заданный стиль запроса можно настроить, предоставив несколько примеров в самом запросе (это называется *обучением за несколько шагов*). Главное преимущество дообучения — вам не придется передавать примеры в каждом отдельном запросе на ввод данных, что в итоге позволит сократить затраты.

Таблица 9.2. Эволюция коллекции моделей OpenAI GPT

Модель	Дата	Слоев	Модулей внимания	Размер векторно- го представления слова	Контекст- ное окно	Количество параметров	Обучающие данные
GPT (https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf)	Июнь 2018 года	12	12	768	512	120 000 000	BookCorpus: 4,5 Гбайт текста из неопубликованных книг
GPT-2 (https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf)	Февраль 2019 года	48	48	1600	1024	1 500 000 000	WebText: 40 Гбайт текста из исходящих ссылок на Reddit
GPT-3 (https://arxiv.org/abs/2005.14165)	Май 2020 года	96	96	12 888	2048	175 000 000 000	CommonCrawl, WebText, англоязычная Википедия, собрание книг и др.: 570 Гбайт
GPT-4 (https://arxiv.org/abs/2303.08774)	Март 2023 года	—	—	—	—	—	—

Пример вывода GPT-3 в ответ на ввод текстового запроса показан на рис. 9.15.

Write me a short story about a unicorn who defeats an evil dragon.

Luna was out for a leisurely stroll through the forest when she heard a loud, ferocious roar. She immediately knew it was a dragon and ran as fast as she could in the opposite direction.

As she ran, she could feel the dragon's hot breath on her back and knew she wouldn't be able to outrun it for long. She had to think of a plan, and fast.

Just as the dragon was about to snatch her up in its teeth, Luna turned around and stabbed it in the eye with her horn. The dragon screeched in pain and flew into a rage, but Luna was ready for it.

She dodged its fire breath and claws until she finally had an opportunity to strike. She impaled the dragon with her horn and it fell to the ground with a thud, dead.

Luna breathed a sigh of relief and trotted away, knowing that she had saved the forest from the evil dragon.

Рис. 9.15. Пример реакции GPT-3 в ответ на ввод текстового запроса

Языковые модели, такие как GPT, значительно выигрывают от масштабирования с точки зрения как количества весов, так и размера набора данных. Потолок возможностей больших языковых моделей еще не достигнут, и исследователи продолжают расширять границы с помощью все более крупных моделей и наборов данных.

ChatGPT

За несколько месяцев до выхода бета-версии GPT-4 OpenAI анонсировала создание ChatGPT (<https://chat.openai.com/>) — инструмента, позволяющего пользователям взаимодействовать с их набором больших языковых моделей через диалоговый интерфейс. Первая версия, выпущенная в ноябре 2022 года, была основана на GPT-3.5 — более мощной версии модели, чем GPT-3, и точно настроенной на генерирование ответов в диалоговом режиме.

Пример диалога показан на рис. 9.16. Обратите внимание на то, что агент может поддерживать контекст между входными данными и понимает, что слово **attention** (внимание) во втором вопросе связано с понятием внимания в контексте трансформеров, а не со способностью человека концентрироваться.

На момент написания этой книги еще не вышел официальный документ, подробно описывающий работу ChatGPT, но из официального сообщения в блоге мы знаем, что этот инструмент использует метод *обучения с подкреплением на основе обратной связи с человеком* (Reinforcement Learning from Human Feedback, RLHF) для точной настройки модели GPT-3.5. Этот метод упоминался также

в более ранней статье [Ouyang et al., 2022] группы ChatGPT, в которой была представлена модель *InstructGPT* — доработанная модель GPT-3, специально созданная для более точного следования письменным инструкциям.

DA

What is a Transformer model and how is it used in generative deep learning? Can you give a short ~100 word summary?



A Transformer model is a type of deep learning model used in natural language processing tasks such as language translation and text generation. It utilizes an attention mechanism to enable the model to focus on different parts of the input sequence during processing, allowing it to capture long-range dependencies and better model context.

In generative deep learning, a Transformer model can be used as a language model to generate new text that is similar to the input data it was trained on. This can be achieved by using the model's output at each time step as input to the next time step, and sampling from the output probability distribution to generate the next word in the sequence. With sufficient training data and appropriate tuning of hyperparameters, Transformer models can produce high-quality, coherent text that is indistinguishable from text written by humans.



DA

What's attention? Reply in the form of a limerick.



Attention's a powerful tool
For deep learning models to rule
It lets them decide
Which parts to abide
And focus on features that are cool



DA

If a Transformer were an animal, what would it be and why?



If a Transformer were an animal, it would likely be a chameleon.

Like a Transformer, a chameleon has the ability to adapt and change its appearance to blend in with its environment, allowing it to be more effective in different situations. Similarly, a Transformer model can adapt its attention to focus on different parts of input sequences based on the specific task it is performing, allowing it to excel at a wide range of natural language processing tasks.

In addition, both chameleons and Transformers have a reputation for being able to "blend in" seamlessly with their surroundings, making them difficult to detect. In the case of Transformers, this is often because the generated text is so coherent and similar to human-written text that it can be difficult to distinguish between the two.



Рис. 9.16. Пример ответов ChatGPT на вопросы о трансформерах

Процесс обучения ChatGPT выглядит следующим образом.

1. *Тонкая настройка с учителем* (supervised fine-tuning): собирается демонстрационный набор диалоговых входных данных (запросов) и желаемых результатов, написанных людьми. Эти данные используются для точной настройки базовой языковой модели (GPT-3.5) методом обучения с учителем.
2. *Моделирование вознаграждения* (reward modeling): специалисту по маркировке предоставляются примеры запросов и ответов модели и предлагается расположить ответы от лучшего к худшему. На полученных данных обучается модель вознаграждения, которая прогнозирует оценку, присвоенную каждому результату, с учетом истории диалога.
3. *Обучение с подкреплением* (reinforcement learning). Диалог рассматривается как среда обучения с подкреплением, где *политикой* является базовая языковая модель, инициализированная моделью, полученной в результате тонкой настройки на шаге 1. Учитывая текущее *состояние* (историю диалога), политика выводит *действие* (последовательность лексем), которое оценивается с помощью модели вознаграждения, обученной на шаге 2. Алгоритм обучения с подкреплением — оптимизация проксимальной политики (Proximal Policy Optimization, PPO) — затем может быть обучен для максимизации вознаграждения путем корректировки весов языковой модели.



Обучение с подкреплением

В главе 12 поближе познакомимся с обучением с подкреплением. Там мы увидим, как использовать генеративные модели в условиях обучения с подкреплением.

Процесс RLHF показан на рис. 9.17.

Модель ChatGPT все еще имеет множество ограничений (например, иногда «выдумывает» фактически неверную информацию), но она наглядно демонстрирует возможность использования трансформеров для создания генеративных моделей, способных производить сложные, согласованные и новые выходные данные, которые часто неотличимы от текста, написанного человеком. Прогресс, достигнутый на данный момент такими моделями, как ChatGPT, служит свидетельством потенциала ИИ и его преобразующего влияния на мир.

Более того, очевидно, что взаимодействия на основе ИИ продолжают быстро развиваться в будущем. Такие проекты, как Visual ChatGPT7, теперь сочетают лингвистические возможности ChatGPT с моделями анализа изображений, такими как Stable Diffusion, что позволяет пользователям взаимодействовать с ChatGPT посредством не только текста, но и изображений. Слияние лингвистических и визуальных возможностей в таких проектах, как Visual ChatGPT и GPT-4, может означать начало новой эры взаимодействия человека и компьютера.

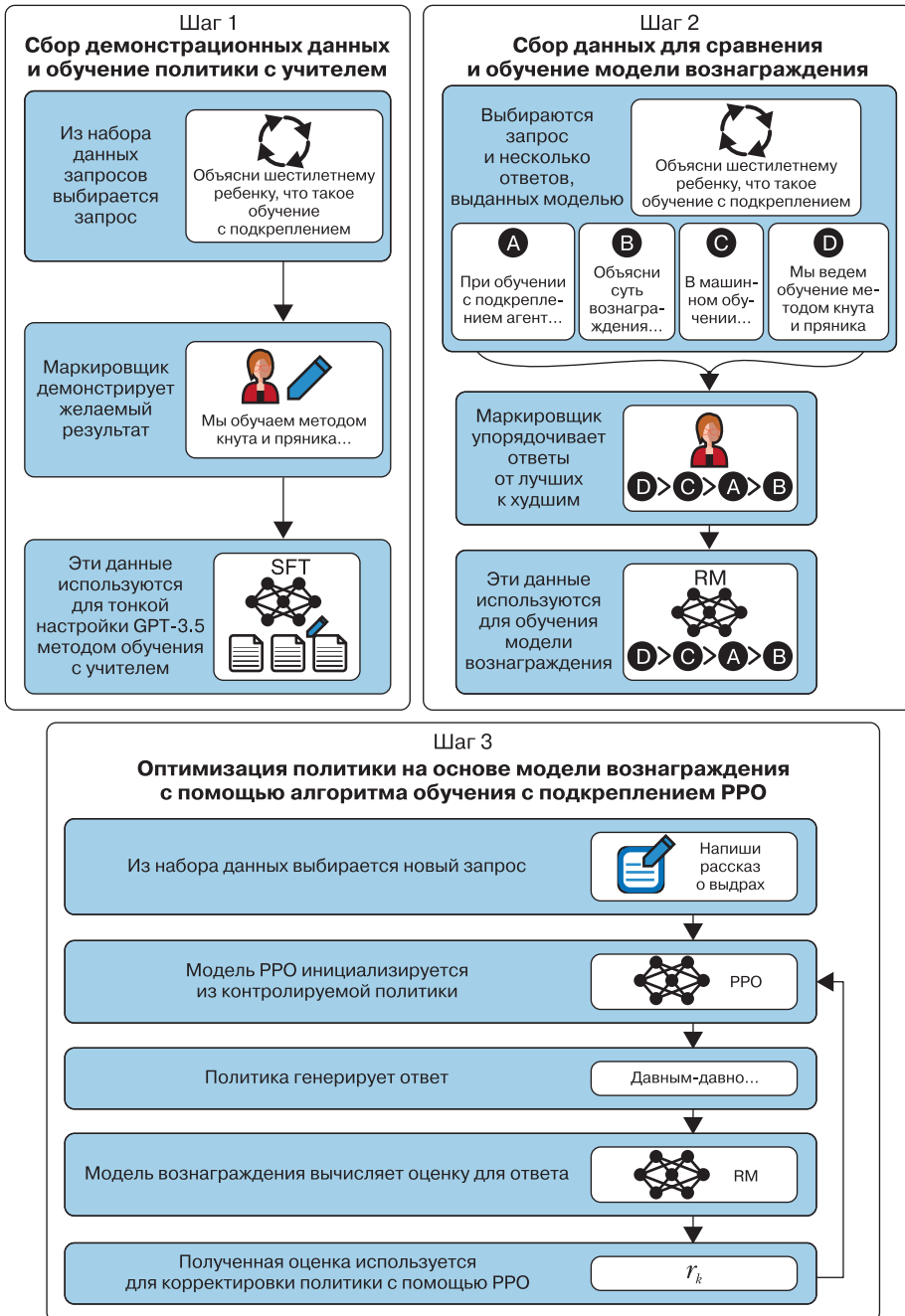


Рис. 9.17. Процесс обучения с подкреплением на основе обратной связи с человеком, используемый в ChatGPT (источник: OpenAI, <https://openai.com/blog/chatgpt>)

Резюме

В этой главе мы рассмотрели архитектуру модели трансформера и создали версию GPT — современную модель генерации текста.

GPT использует механизм внимания, который устраняет необходимость применять рекуррентные слои, такие как LSTM. Эта модель действует как система поиска информации, задействуя запросы, ключи и значения, чтобы оценить информативность каждой входной лексемы.

Модули внимания можно сгруппировать в так называемый слой многоголового внимания, а затем включить этот слой в блок трансформера, содержащий также послойную нормализацию и пропускающие связи. Можно соединять блоки трансформеров друг с другом и так создавать очень глубокие нейронные сети.

Чтобы гарантировать невозможность просачивания информации из последующих лексем в текущий прогноз, в GPT используется причинно-следственная маскировка. Кроме того, чтобы гарантировать учет порядка лексем, применяется метод, известный как позиционное представление, который внедряет позиции лексем во входные данные наряду с традиционными векторными представлениями слов.

Анализируя вывод модели GPT, мы увидели, что она способна не только генерировать новые текстовые отрывки, но и опрашивать слой внимания, чтобы понять, в каком месте предложения следует искать информацию для улучшения прогноза. Модель GPT может получать информацию, отстоящую далеко от текущей позиции, без потери сигнала, поскольку оценки внимания рассчитываются параллельно и не зависят от скрытого состояния, которое передается по сети последовательно, что отличает ее от рекуррентных нейронных сетей.

Вы увидели, что существует три семейства трансформеров (кодировщики, декодировщики и кодировщики-декодировщики) и различные задачи, которые можно решать с помощью каждого из них. Наконец, мы исследовали структуру и процесс обучения других крупных языковых моделей, таких как T5 от компании Google и ChatGPT от компании OpenAI.

Продвинутые GAN

В этой главе вы:

- посмотрите, как модель ProGAN постепенно обучает GAN созданию изображений с высоким разрешением;
- узнаете, как на основе ProGAN была создана StyleGAN — высокопроизводительная GAN для синтеза изображений;
- узнаете, как на основе StyleGAN была разработана StyleGAN2 — современная модель, еще больше улучшающая свою предшественницу;
- познакомитесь с ключевыми преимуществами этих моделей, включая прогрессивное обучение, адаптивную нормализацию экземпляров, модуляцию и демодуляцию веса, а также регуляризацию длины пути;
- познакомитесь с архитектурой генеративно-сопоставительных сетей SAGAN, включающей механизм внимания в структуру GAN;
- увидите, как BigGAN развивает концепции SAGAN для создания высококачественных изображений;
- узнаете, как VQ-GAN использует таблицы кодирования для преобразования изображений в дискретные последовательности лексем, которые можно смоделировать с помощью трансформера;
- увидите, как ViT VQ-GAN адаптирует архитектуру VQ-GAN для использования трансформеров вместо сверточных слоев в кодировщике и декодировщике.

В главе 4 вашему вниманию были представлены генеративно-сопоставительные сети (Generative Adversarial Networks, GAN) — класс генеративных моделей, позволяющих получить потрясающие результаты в широком спектре задач генерации изображений. Гибкость архитектуры и процесса обучения побудила ученых и специалистов по глубокому обучению искать новые способы проектирования и обучения GAN, что привело к появлению множества продвинутых архитектур, которые мы рассмотрим в этой главе.

Введение

Подробное объяснение всех разработок GAN и их последствий легко могло бы занять целую книгу. Репозиторий GAN Zoo на GitHub (<https://oreil.ly/Oy6bR>) содержит более 500 примеров GAN с описаниями, от ABC-GAN до ZipNet-GAN!

В этой главе мы поговорим об основных GAN, оказавших существенное влияние на развитие этой области, и подробно исследуем архитектуры и процессы обучения каждой модели. Сначала рассмотрим три важные модели, созданные в компании NVIDIA, которые раздвинули горизонты возможностей создания изображений: ProGAN, StyleGAN и StyleGAN2. Мы проанализируем их все достаточно подробно для того, чтобы понять фундаментальные концепции, лежащие в их основе, и увидеть, что каждая из них построена на идеях из более ранних разработок.

Рассмотрим также еще две важные архитектуры, использующие механизм внимания: GAN с самовниманием (Self-Attention GAN, SAGAN) и BigGAN, которые основаны на принципах, изложенных в статье о SAGAN. В главе 9 вы уже видели, насколько широкими возможностями обладает механизм внимания в контексте трансформеров.

Наконец, мы рассмотрим VQ-GAN и ViT VQ-GAN, использующие сочетание идей, на которых базируются вариационные автокодировщики, трансформеры и генеративно-сопоставительные сети. VQ-GAN является ключевым компонентом современной модели Muse преобразования текста в изображение, созданной в Google [Chang et al., 2023]. Кроме того, подробнее поговорим о так называемых мультимодальных моделях.



Обучение собственных моделей

Для краткости я решил не включать код построения этих моделей в репозиторий с примерами для этой книги. Вместо этого там, где возможно, буду приводить ссылки на общедоступные реализации, чтобы вы могли при желании обучить собственные версии.

ProGAN

ProGAN — это метод, разработанный в NVIDIA Labs в 2017 году [Karras et al., 2017] для повышения скорости и стабильности обучения GAN. В статье о ProGAN предлагается вместо обучения GAN непосредственно на изображениях с полным разрешением сначала обучить генератор и дискриминатор на изображениях с низким разрешением, скажем 4×4 пиксела, а затем на протяжении всего процесса обучения постепенно добавлять слои и увеличивать разрешение.

Рассмотрим концепцию прогрессивного обучения более подробно.



Обучение собственной модели ProGAN

В блоге Paperspace (<https://oreil.ly/b2CJm>) вы найдете отличное руководство Бхарата К. (Bharath K.) по обучению собственной модели ProGAN с использованием Keras. Но имейте в виду, что для обучения ProGAN, способной показать результаты, описанные в статье, требуется значительная вычислительная мощность.

Прогрессивное обучение

Типичная генеративно-сопоставительная сеть состоит из двух независимых сетей — генератора и дискриминатора, которые состязаются между собой в процессе обучения. В обычной GAN генератор всегда выводит изображения в полном разрешении, даже на ранних этапах обучения. Разумно предположить, что эта стратегия может быть неоптимальной — генератор может медленно изучать структуры высокого уровня на ранних этапах обучения, поскольку он сразу же работает со сложными изображениями с высоким разрешением. Не лучше ли сначала обучить облегченную GAN генерировать точные изображения с низким разрешением, а затем использовать ее для постепенного увеличения разрешения?

Эта простая идея ведет нас к *прогрессивному обучению* — одной из ключевых идей в статье о ProGAN. Обучение модели ProGAN протекает поэтапно и начинается с сокращенного обучающего набора, содержащего изображения размером 4×4 пиксела, полученные путем интерполяции (рис. 10.1).

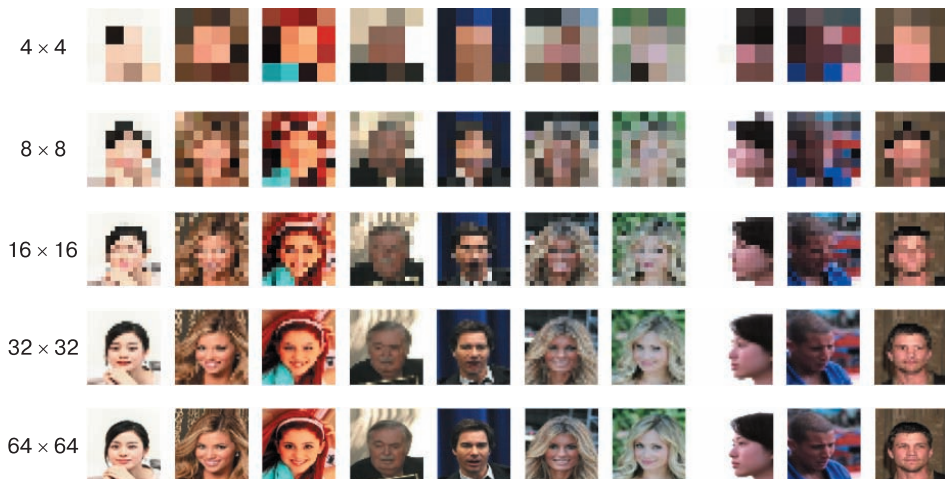


Рис. 10.1. Изображения в наборе данных можно сжать методом интерполяции до более низкого разрешения

Затем можно сначала обучить генератор преобразовывать вектор скрытого входного шума z (скажем, длиной 512) в изображение формой $4 \times 4 \times 3$, а парный ему дискриминатор — преобразовывать входное изображение размером $4 \times 4 \times 3$ в один скалярный прогноз. Сетевые архитектуры для первого шага показаны на рис. 10.2. Синий прямоугольник в генераторе — это сверточный слой, преобразующий набор признаков в изображение RGB (`toRGB`), а синий прямоугольник в дискриминаторе — сверточный слой, который преобразует изображения RGB в набор признаков (`fromRGB`).

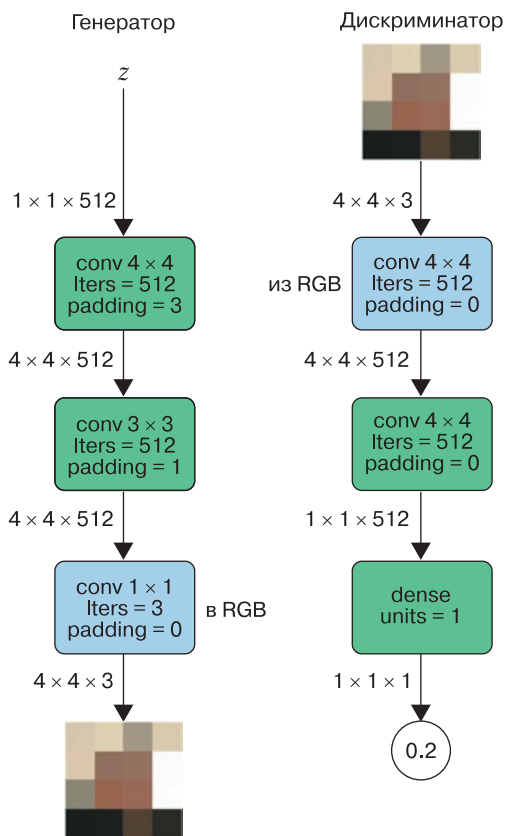


Рис. 10.2. Архитектуры генератора и дискриминатора для первого этапа обучения ProGAN

В статье авторы обучают эту пару сетей до тех пор, пока дискриминатор не увидит 800 000 реальных изображений. Давайте разберемся, как расширить генератор и дискриминатор для работы с изображениями 8×8 пикселей. Чтобы

сделать это, нужно добавить дополнительные слои. Добавление выполняется в два этапа — переноса и стабилизации (рис. 10.3).

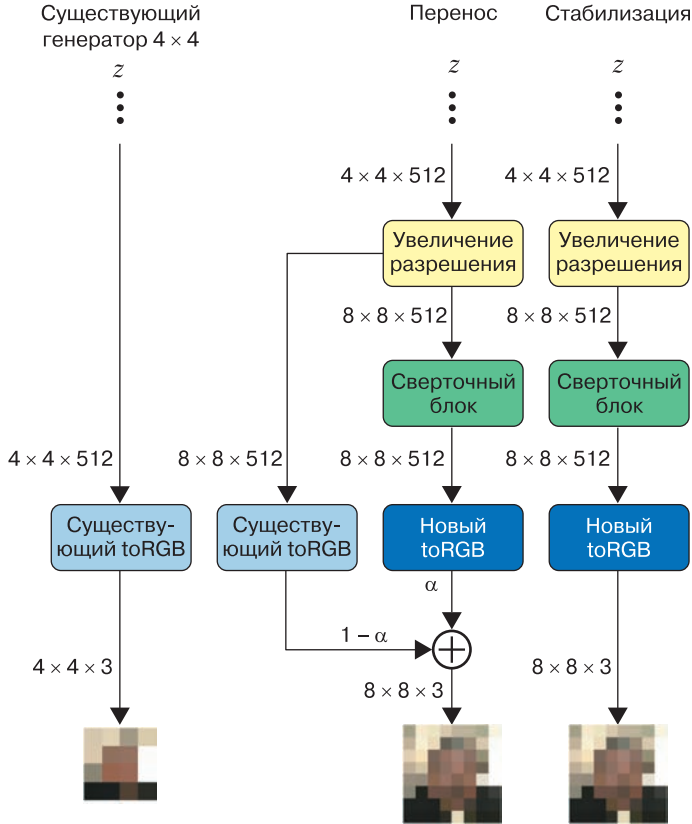


Рис. 10.3. Процесс обучения генератора ProGAN с увеличением размеров изображений с 4×4 до 8×8 (многоточия обозначают остальную часть сети, здесь не показанную)

Рассмотрим сначала генератор. На *этапе переноса* в существующую сеть добавляются новые слои увеличения разрешения и свертки, а также остаточная связь для поддержки выходных данных существующего обученного слоя toRGB. Важно отметить, что новые слои изначально маскируются с помощью параметра α , значение которого постепенно увеличивается от 0 до 1 на протяжении этапа переноса для пропуска все большего количества вывода из нового и меньшего — из существующего слоя toRGB. Это сделано для того, чтобы избежать *потрясений* в сети при вступлении в игру новых слоев.

В конце этапа поток через старый слой `toRGB` прекращается и начинается *этап стабилизации* — следующий этап обучения, в ходе которого сеть сможет точно настроить выходные данные без участия потока через старый слой `toRGB`.

Обучение дискриминатора протекает аналогично (рис. 10.4).

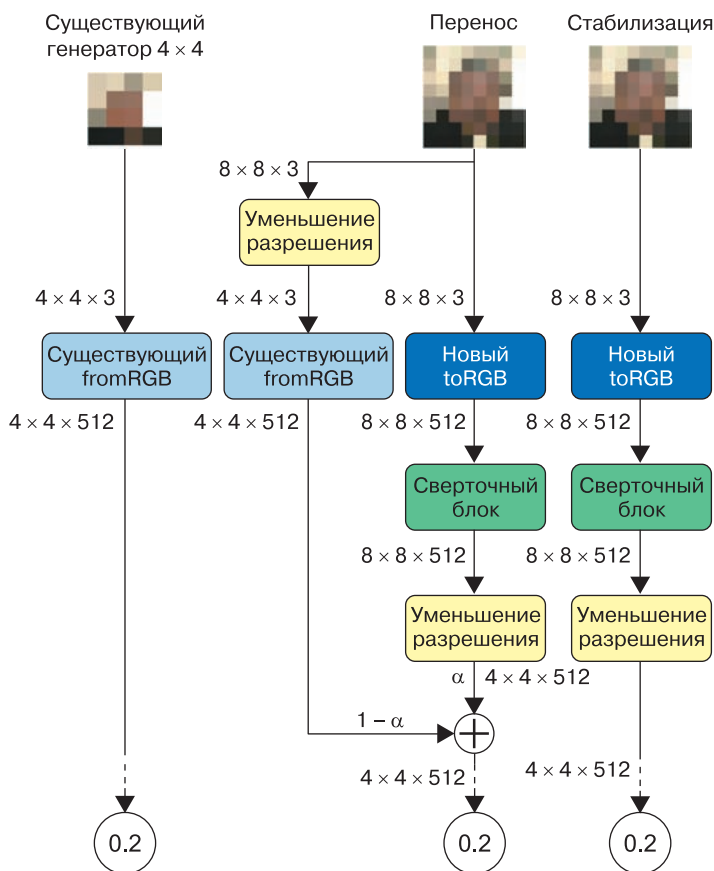


Рис. 10.4. Процесс обучения дискриминатора ProGAN с увеличением размеров изображений с 4×4 до 8×8 (пунктирные линии обозначают остальную часть сети, здесь не показанную)

В этом случае нам нужно добавить дополнительные слои уменьшения разрешения и свертки, но на этот раз они добавляются в начало сети, сразу после ввода изображения. На этапе переноса существующий слой `fromRGB` подключается через остаточную связь и постепенно выводится из обучения по мере включения

в работу новых слоев. Этап стабилизации позволяет дискриминатору точно настроиться на использование новых слоев.

Этапы переноса и стабилизации длятся до тех пор, пока дискриминатор не увидит 800 000 реальных изображений. Обратите внимание на то, что при прогрессивном обучении сети ни один ее слой *незамораживается*. На протяжении всего процесса обучения все слои продолжают обучаться.

После увеличения разрешения изображений с 4×4 до 8×8 процесс продолжается и разрешение увеличивается до 16×16 , затем до 32×32 и т. д., пока не будет достигнуто полное разрешение 1024×1024 (рис. 10.5).

Генератор

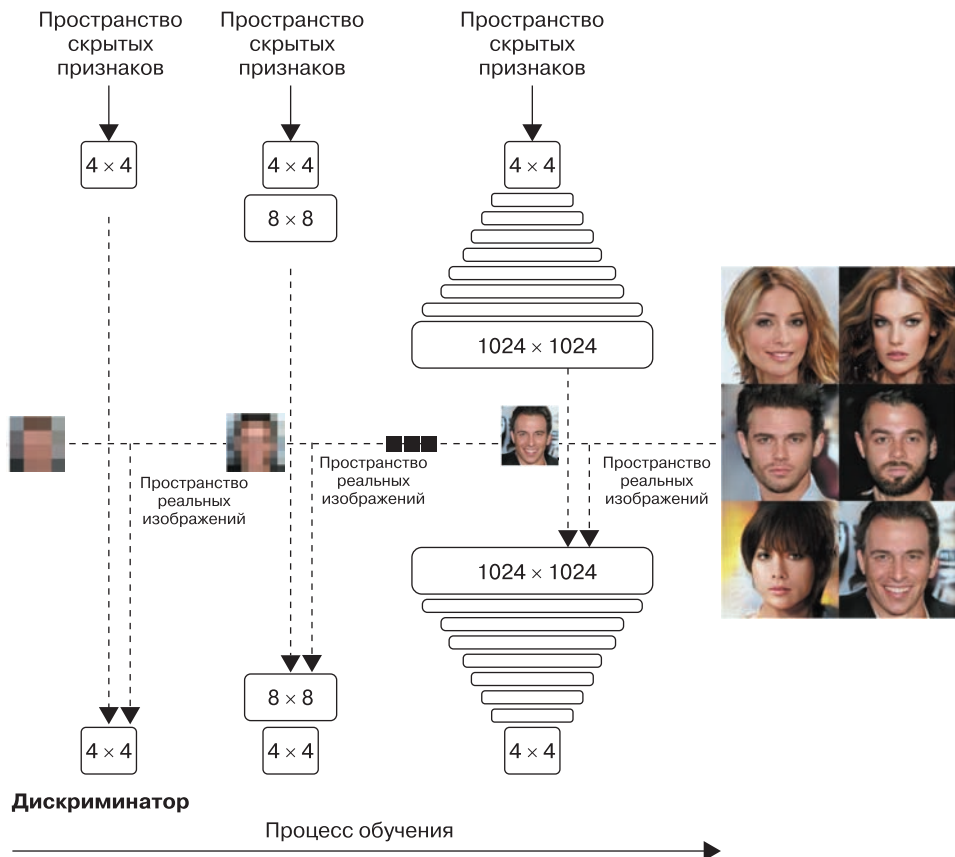


Рис. 10.5. Механика обучения ProGAN и несколько примеров сгенерированных изображений лиц (источник: [Karras et al., 2017], <https://arxiv.org/abs/1710.10196>)

Общая структура генератора и дискриминатора после завершения процесса прогрессивного обучения показана на рис. 10.6.

Generator	Act.	Output shape	Params
Latent vector	—	$512 \times 1 \times 1$	—
Conv 4×4	LReLU	$512 \times 4 \times 4$	4.2M
Conv 3×3	LReLU	$512 \times 4 \times 4$	2.4M
Upsample	—	$512 \times 8 \times 8$	—
Conv 3×3	LReLU	$512 \times 8 \times 8$	2.4M
Conv 3×3	LReLU	$512 \times 8 \times 8$	2.4M
Upsample	—	$512 \times 16 \times 16$	—
Conv 3×3	LReLU	$512 \times 16 \times 16$	2.4M
Conv 3×3	LReLU	$512 \times 16 \times 16$	2.4M
Upsample	—	$512 \times 32 \times 32$	—
Conv 3×3	LReLU	$512 \times 32 \times 32$	2.4M
Conv 3×3	LReLU	$512 \times 32 \times 32$	2.4M
Upsample	—	$512 \times 64 \times 64$	—
Conv 3×3	LReLU	$256 \times 64 \times 64$	1.2M
Conv 3×3	LReLU	$256 \times 64 \times 64$	590k
Upsample	—	$256 \times 128 \times 128$	—
Conv 3×3	LReLU	$128 \times 128 \times 128$	295k
Conv 3×3	LReLU	$128 \times 128 \times 128$	148k
Upsample	—	$128 \times 256 \times 256$	—
Conv 3×3	LReLU	$64 \times 256 \times 256$	74k
Conv 3×3	LReLU	$64 \times 256 \times 256$	37k
Upsample	—	$64 \times 512 \times 512$	—
Conv 3×3	LReLU	$32 \times 512 \times 512$	18k
Conv 3×3	LReLU	$32 \times 512 \times 512$	9.2k
Upsample	—	$32 \times 1024 \times 1024$	—
Conv 3×3	LReLU	$16 \times 1024 \times 1024$	4.6k
Conv 3×3	LReLU	$16 \times 1024 \times 1024$	2.3k
Conv 1×1	linear	$3 \times 1024 \times 1024$	51
Total trainable parameters			23.1M

Discriminator	Act.	Output shape	Params
Input image	—	$3 \times 1024 \times 1024$	—
Conv 1×1	LReLU	$16 \times 1024 \times 1024$	64
Conv 3×3	LReLU	$16 \times 1024 \times 1024$	2.3k
Conv 3×3	LReLU	$32 \times 1024 \times 1024$	4.6k
Downsample	—	$32 \times 512 \times 512$	—
Conv 3×3	LReLU	$32 \times 512 \times 512$	9.2k
Conv 3×3	LReLU	$64 \times 512 \times 512$	18k
Downsample	—	$64 \times 256 \times 256$	—
Conv 3×3	LReLU	$64 \times 256 \times 256$	37k
Conv 3×3	LReLU	$128 \times 256 \times 256$	74k
Downsample	—	$128 \times 128 \times 128$	—
Conv 3×3	LReLU	$128 \times 128 \times 128$	148k
Conv 3×3	LReLU	$256 \times 128 \times 128$	295k
Downsample	—	$256 \times 64 \times 64$	—
Conv 3×3	LReLU	$256 \times 64 \times 64$	590k
Conv 3×3	LReLU	$512 \times 64 \times 64$	1.2M
Downsample	—	$512 \times 32 \times 32$	—
Conv 3×3	LReLU	$512 \times 32 \times 32$	2.4M
Conv 3×3	LReLU	$512 \times 32 \times 32$	2.4M
Downsample	—	$512 \times 16 \times 16$	—
Conv 3×3	LReLU	$512 \times 16 \times 16$	2.4M
Conv 3×3	LReLU	$512 \times 16 \times 16$	2.4M
Downsample	—	$512 \times 8 \times 8$	—
Conv 3×3	LReLU	$512 \times 8 \times 8$	2.4M
Conv 3×3	LReLU	$512 \times 8 \times 8$	2.4M
Downsample	—	$512 \times 4 \times 4$	—
Minibatch stddev	—	$513 \times 4 \times 4$	—
Conv 3×3	LReLU	$512 \times 4 \times 4$	2.4M
Conv 4×4	LReLU	$512 \times 1 \times 1$	4.2M
Fully-connected	linear	$1 \times 1 \times 1$	513
Total trainable parameters			23.1M

Рис. 10.6. Генератор и дискриминатор ProGAN, используемые для генерирования изображений лиц размером 1024×1024 и обученные на наборе данных CelebA (источник: [Karras et al., 2018], <https://arxiv.org/abs/1812.04948>)

В статье описываются еще несколько важных дополнений к архитектуре: учет стандартного отклонения мини-пакетов, выравнивание скорости обучения и по-пиксельная нормализация, которые кратко рассмотрены далее.

Стандартное отклонение мини-пакетов

Слой *стандартного отклонения мини-пакетов* (minibatch standard deviation) — это дополнительный слой в дискриминаторе, который добавляет стандартное отклонение значений признаков, усредненное по всем пикселям и мини-пакету, как дополнительный (постоянный) признак. Он помогает генератору создавать более разнообразные выходные изображения — если разнообразие в мини-пакете низкое, то стандартное отклонение будет небольшим и дискриминатор сможет использовать эту функцию, чтобы отличать поддельные пакеты от настоящих. Соответственно, генератор будет стремиться создавать изображения с той же степенью разнообразия, которой отличаются реальные обучающие данные.

Выравнивание скорости обучения

Все полносвязные и сверточные слои в ProGAN используют *выровненные скорости обучения*. Обычно веса в нейронной сети инициализируются с помощью такого метода, как *инициализация Хе* — гауссово распределение, в котором стандартное отклонение масштабируется так, чтобы оно было обратно пропорционально квадратному корню от количества входов слоя. Соответственно, слои с большим количеством входов будут инициализированы весами, имеющими меньшее отклонение от нуля, что в целом повышает стабильность процесса обучения.

Авторы статьи о ProGAN обнаружили, что это решение может вызывать проблемы при использовании в сочетании с современными оптимизаторами, такими как Adam или RMSProp. Эти методы нормализуют обновление градиента для каждого веса, так что величина обновления не зависит от масштаба (величины) веса. Однако это означает, что для корректировки весов с более широким динамическим диапазоном (то есть слоев с меньшим количеством входов) потребуется больше времени, чем для весов с более узким динамическим диапазоном (то есть слоев с большим количеством входов). Это вызывает дисбаланс между скоростью обучения разных слоев генератора и дискриминатора в ProGAN, поэтому, чтобы решить проблему, авторы использовали выравнивание скоростей обучения.

В ProGAN веса инициализируются с помощью простой стандартной гауссовой функции независимо от количества входов слоя. Нормализация применяется динамически при каждом вызове слоя, а не только при инициализации. Соответственно, оптимизатор считает, что каждый вес имеет примерно одинаковый динамический диапазон, поэтому использует ту же скорость обучения. Только при вызове слоя вес масштабируется на коэффициент из инициализатора Хе.

Попиксельная нормализация

Наконец, в генераторе ProGAN применяется *попиксельная нормализация*, а не пакетная. Она нормализует вектор признаков в каждом пикселе до единичной длины и помогает предотвратить выход сигнала из-под контроля при его распространении по сети. Слой попиксельной нормализации не имеет обучаемых весов.

Выходы

Авторы применили ProGAN не только к набору данных CelebA, но и к изображениям из набора данных Large-scale Scene Understanding (LSUN) и получили отличные результаты (рис. 10.7). Это продемонстрировало преимущество ProGAN над более ранними архитектурами GAN и послужило толчком

к созданию следующих поколений, таких как StyleGAN и StyleGAN2, которые мы рассмотрим далее.



Рис. 10.7. Изображения, сгенерированные сетью ProGAN, прогрессивно обучавшейся на изображениях размерами 256×256 из набора данных LSUN (источник: [Karras et al., 2017], <https://arxiv.org/abs/1710.10196>)

StyleGAN

StyleGAN [Karras et al., 2018] — это архитектура GAN, разработанная в 2018 году на основе идей, изложенных в статье о ProGAN. Дискриминатор в этой архитектуре практически не претерпел изменений, изменился только генератор.

Нередко при обучении GAN трудно выделить векторы в скрытом пространстве, соответствующие атрибутам высокого уровня, — они часто *спутаны*. Например, при попытке изменить изображение, чтобы добавить на лицо веснушек, может неожиданно измениться и цвет фона. Сеть ProGAN генерирует фантастически реалистичные изображения, но она не исключение из этого общего правила. В идеале хотелось бы иметь полный контроль над стилем изображения, а для этого требуется четкое разделение высокоуровневых признаков в скрытом пространстве.

StyleGAN достигает желаемого путем явного внедрения векторов стиля в сеть в разных точках: некоторые из них отвечают за высокоуровневые признаки (например, ориентацию лица), а некоторые — за низкоуровневые детали (например, как волосы падают на лоб).

На рис. 10.8 показана обобщенная архитектура генератора StyleGAN. Рассмотрим эту архитектуру поближе, начав с сети отображения.

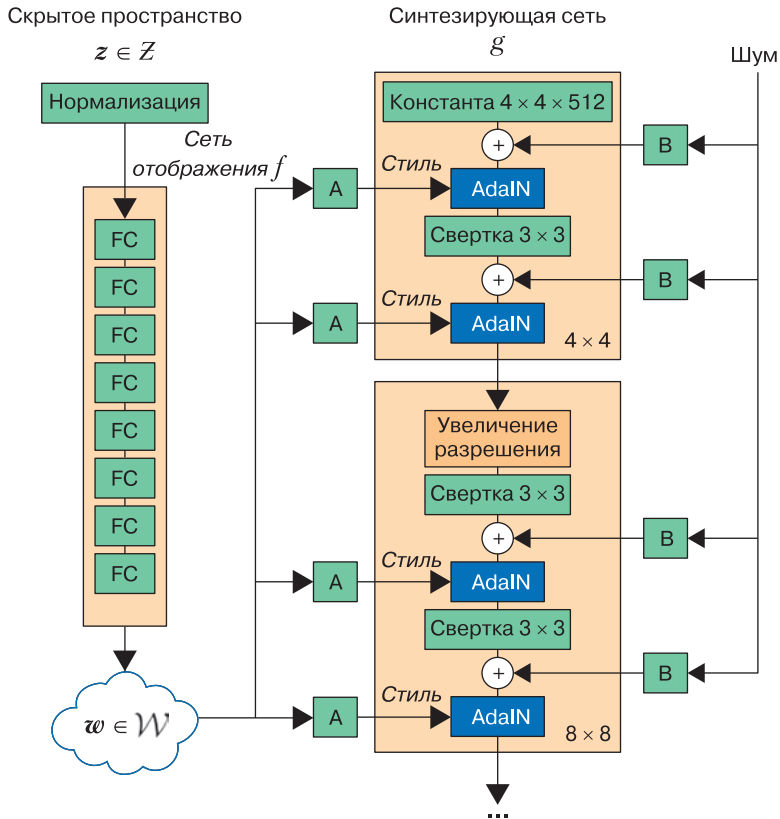


Рис. 10.8. Архитектура генератора StyleGAN
(источник: [Karras et al., 2018], <https://arxiv.org/abs/1812.04948>)



Обучение собственной модели StyleGAN

На сайте Keras есть отличное руководство Сун-Яу Чонга (Soon-Yau Cheong) по обучению собственной модели StyleGAN с использованием Keras (<https://oreil.ly/MooSe>). Но имейте в виду, что для обучения StyleGAN, способной показать результаты, описанные в статье, требуется значительная вычислительная мощность.

Сеть отображения

Сеть отображения f — это простая сеть прямого распространения, которая преобразует входной шум $z \in \mathcal{Z}$ в другое скрытое пространство $w \in \mathcal{W}$. Это позволяет генератору распутать зашумленный входной вектор на отдельные факторы дисперсии, которые могут быть учтены последующими слоями, генерирующими стили.

Смысл этих преобразований состоит в том, чтобы отделить процесс выбора стиля (сеть отображения) от синтеза изображения с заданным стилем (синтезирующая сеть).

Синтезирующая сеть

Синтезирующая сеть — это генератор изображения с заданным стилем, выделенным сетью отображения. Как видно на рис. 10.8, вектор стиля вводится в синтезирующую сеть в разных ее точках через отдельные полносвязные слои A_i , которые генерируют два вектора: вектор смещения $y_{b,i}$ и масштабирующий вектор $y_{s,i}$. Эти векторы определяют конкретный стиль, который должен быть внедрен в сеть в этой точке, то есть они сообщают синтезирующей сети, как настроить наборы признаков, чтобы сместить сгенерированное изображение в направлении выбранного стиля.

Эта корректировка осуществляется за счет слоев *адаптивной нормализации экземпляров* (Adaptive Instance Normalization, AdaIN).

Адаптивная нормализация экземпляров

AdaIN — это слой нейронной сети, регулирующий среднее значение и дисперсию каждого набора признаков x_i в соответствии со смещением эталонного стиля $y_{b,i}$ и масштабом $y_{s,i}$ [Huang, Belongie, 2017]. Оба вектора имеют длину, равную количеству каналов на выходе из сверточного слоя в синтезирующей сети. Вот как выглядит уравнение адаптивной нормализации:

$$\text{AdaIN}(x_i, y) = y_{s,i} \frac{x_i - \mu(x_i)}{\sigma(x_i)} + y_{b,i}.$$

Слои адаптивной нормализации экземпляра гарантируют влияние векторов стилей, внедряемых в каждый слой, только на признаки в этом слое и отсутствие утечек любой информации о стиле между слоями. Авторы показывают, что

в результате скрытые векторы $\mathbf{w}$ получаются значительно менее спутанными, чем исходные векторы $\mathbf{z}$.

Поскольку синтезирующая сеть основана на архитектуре ProGAN, она обучается прогрессивно. Векторы стилей в самых первых ее слоях (когда разрешение изображения самое низкое — 4×4 , 8×8) влияют на более грубые признаки, чем следующие за ними (с разрешением от 64×64 до 1024×1024). При таком подходе мы не только получаем полный контроль над сгенерированным изображением через скрытый вектор $\mathbf{w}$, но и можем переключать вектор $\mathbf{w}$ в разных точках синтезирующей сети, чтобы изменить стиль на разных уровнях детализации.

Смешивание стилей

Чтобы гарантировать, что генератор не сможет использовать корреляции между соседними стилями во время обучения (то есть стили, внедряемые в каждый слой, максимально распутываются), авторы применяют прием, известный как *смешивание стилей*. Вместо выборки единственного скрытого вектора $\mathbf{z}$ они выбирают два ($\mathbf{z}_1$, $\mathbf{z}_2$), соответствующих двум векторам стилей ($\mathbf{w}_1$, $\mathbf{w}_2$). Затем в каждом слое случайно выбирается один из них ($\mathbf{w}_1$ или $\mathbf{w}_2$), чтобы разрушить любую возможную корреляцию между векторами.

Стохастическое разнообразие

Сеть синтезатора добавляет шум (проходящий через обученный слой B) после каждой свертки для добавления некоторой доли случайности в детали, такие как расположение отдельных волос или фон позади лица. Опять же глубина, на которой вводится шум, влияет на точность воздействия.

Это также означает, что первоначальными входными данными для синтезирующей сети может служить не дополнительный шум, а изученная константа. Во входных стиле и шуме уже присутствует достаточная стохастичность, чтобы генерировать довольно разнообразные изображения.

Вывод сети StyleGAN

На рис. 10.9 можно видеть результаты работы StyleGAN.

Здесь имеются два исходных изображения, A и B, сгенерированных из двух разных векторов $\mathbf{w}$. Чтобы получить объединенное изображение, вектор $\mathbf{w}$ исходного изображения A пропускается через синтезирующую сеть и в некоторый

момент замещается вектором $\mathbf{w}$ исходного изображения В. Если это замещение происходит раньше (на уровне разрешения 4×4 или 8×8), то из исходного изображения В в исходное изображение А переносятся грубые стили: поза, форма лица и очки. Если замещение происходит позже, то из исходного изображения В в исходное изображение А передаются только мелкие детали, такие как цвет и микроструктура лица, тогда как грубые элементы из исходного изображения А сохраняются.

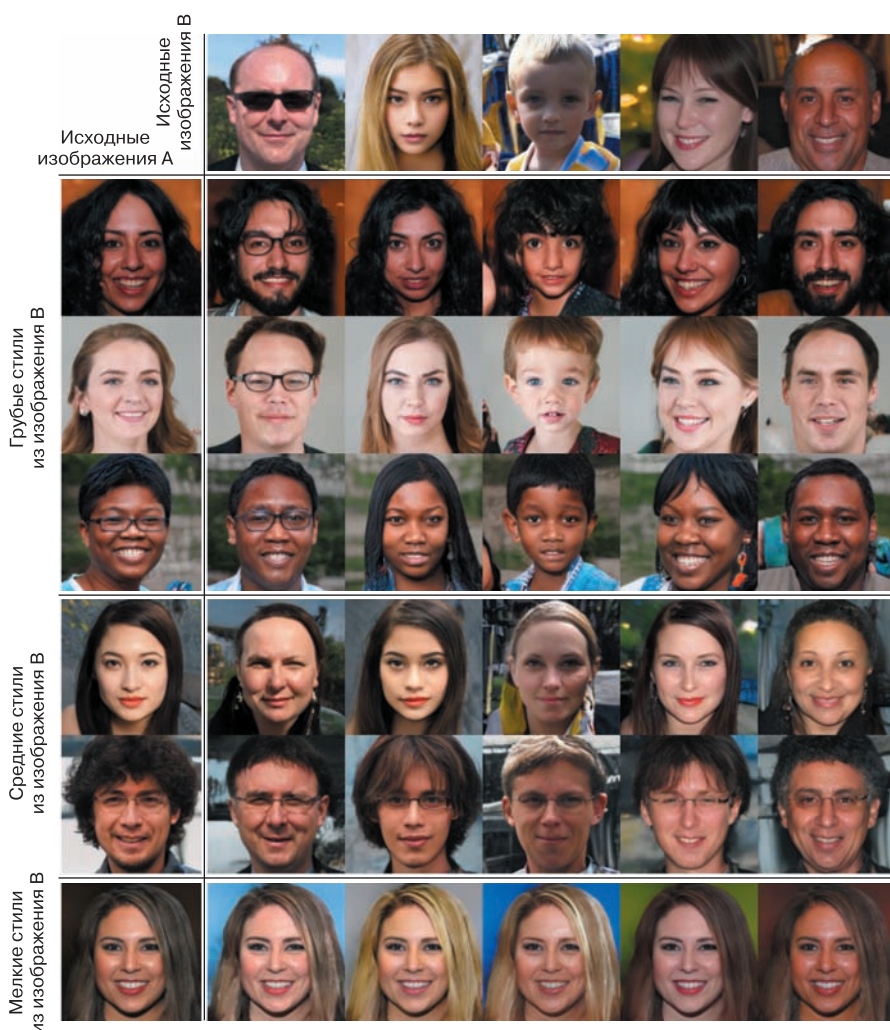


Рис. 10.9. Объединение стилей двух изображений на разных уровнях детализации (источник: [Karras et al., 2018], <https://arxiv.org/abs/1812.04948>)

StyleGAN2

Последний важный вклад в эту цепочку — сеть StyleGAN2 [Karras et al., 2019]. Эта архитектура является дальнейшим развитием архитектуры StyleGAN с некоторыми ключевыми изменениями, улучшающими качество генерируемых изображений. В частности, изображения, полученные с помощью StyleGAN2, не так сильно страдают от *артефактов* — областей изображения, похожих на капли воды, которые, как выяснилось, обусловлены воздействием слоев адаптивной нормализации экземпляров в StyleGAN (рис. 10.10).



Рис. 10.10. Артефакт на изображении лица, сгенерированном сетью StyleGAN (источник: [Karras et al., 2019], <https://arxiv.org/abs/1912.04958>)

И генератор, и дискриминатор в StyleGAN2 отличаются от генератора и дискриминатора в StyleGAN. В следующих разделах мы рассмотрим ключевые различия между этими архитектурами.



Обучение собственной модели StyleGAN2

Официальный код, реализующий обучение StyleGAN2 с помощью TensorFlow, можно найти на GitHub (<https://oreil.ly/alB6w>). Но имейте в виду, что для обучения StyleGAN2, способной показать результаты, описанные в статье, требуется значительная вычислительная мощность.

Модуляция и демодуляция весов

Проблема артефактов решается путем удаления слоев AdaIN из генератора с заменой их этапами модуляции и демодуляции весов (рис. 10.11). Вектор $\mathbf{w}$ представляет веса сверточного слоя, которые напрямую обновляются этапами модуляции и демодуляции в StyleGAN2 во время выполнения. Для сравнения: слои AdaIN в StyleGAN работают с тензором изображения при его прохождении через сеть.

Слой AdaIN в StyleGAN просто выполняет нормализацию экземпляра, за которой следует модуляция стиля (масштабирование и смещение). Принцип работы StyleGAN2 состоит в том, чтобы применить модуляцию стиля и нормализацию (демодуляцию) непосредственно к весам сверточных слоев во время выполнения, а не к их выходным данным (см. рис. 10.11). Авторы показывают, что это решение устраняет проблему артефактов и позволяет сохранить контроль над стилем изображения.

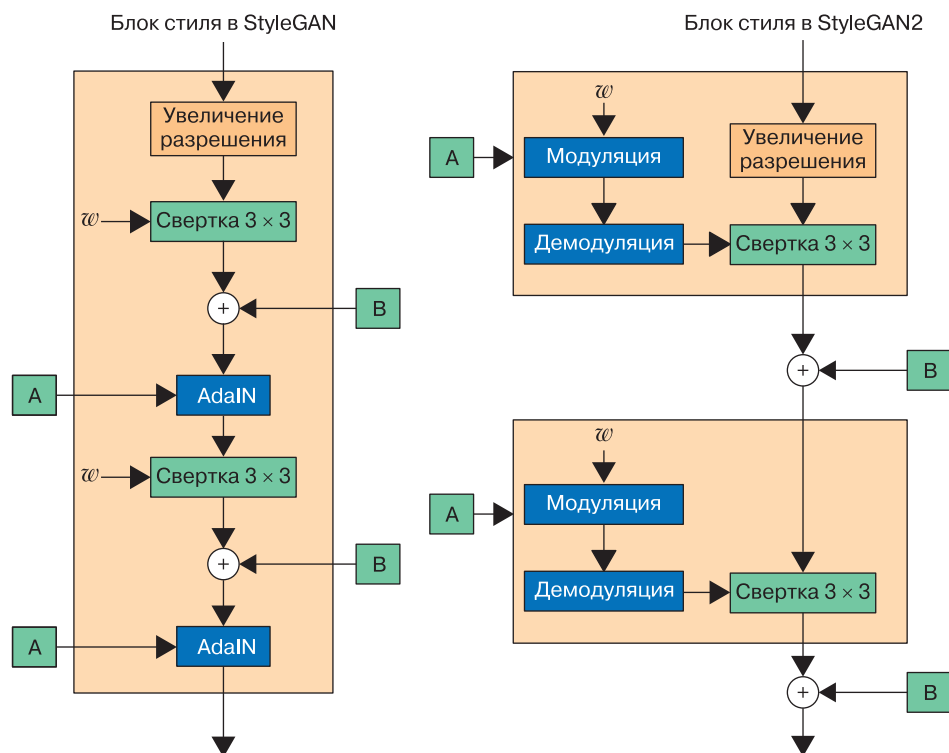


Рис. 10.11. Сравнение блоков стиля в StyleGAN и StyleGAN2

В StyleGAN2 каждый полносвязный слой A выводит один вектор стиля $\mathbf{s}_i$, где i индексирует количество входных каналов в соответствующем сверточном слое. Затем этот вектор стиля применяется к весам сверточного слоя:

$$w'_{i,j,k} = s_i w_{i,j,k}.$$

Здесь j индексирует каналы на выходе из слоя, а k — пространственные измерения. Это этап *модуляции* процесса.

Затем нужно нормализовать веса так, чтобы они снова получили единичное стандартное отклонение, и тем самым обеспечить стабильность процесса обучения. Вот шаг *демодуляции*:

$$w''_{i,j,k} = \frac{w'_{i,j,k}}{\sqrt{\sum_{i,k} w'^2_{i,j,k} + \epsilon}},$$

где ϵ — небольшая постоянная величина, предотвращающая деление на ноль.

В статье авторы показывают, что этого простого изменения достаточно, чтобы предотвратить появление артефактов в виде капель воды, сохранив при этом контроль над изображениями, сгенерированными с помощью векторов стилей, и обеспечить высокое качество результатов.

Регуляризация длины пути

Еще одно изменение в архитектуре StyleGAN — включение в функцию потерь дополнительного штрафного члена, известного как *регуляризация длины пути* (path length regularization).

Для нас желательны гладкость и однородность скрытого пространства, чтобы шаг фиксированного размера в любом направлении в скрытом пространстве приводил к изменению изображения на фиксированную величину. Для обеспечения этого свойство StyleGAN2 стремится минимизировать следующий член наряду с обычными потерями Вассерштейна со штрафом за градиент:

$$\mathbb{E}_{w,y} \left(\left\| \mathbf{J}_w^T y \right\|_2 - a \right)^2.$$

Здесь w — это набор векторов стилей, созданных сетью отображения, y — набор зашумленных изображений, извлеченных из $\mathcal{N}(0, \mathbf{I})$, $\mathbf{J}_w = \frac{\partial g}{\partial w}$ — якобиан сети генератора относительно векторов стилей.

Член $\left\| \mathbf{J}_w^T y \right\|_2$ измеряет величину изображений y после преобразования по градиентам, заданным в якобиане. Нам желательно, чтобы она была близка к константе a ,

которая рассчитывается динамически как экспоненциальное скользящее среднее $\|\mathbf{J}_w^T \mathbf{y}\|_2$ в процессе обучения.

Авторы считают, что этот дополнительный член делает исследование скрытого пространства более надежным и последовательным. Кроме того, для большей эффективности члены регуляризации в функции потерь применяются только один раз на каждые 16 мини-пакетов. Этот метод, называемый *ленивой регуляризацией*, не вызывает заметного снижения производительности.

Отсутствие прогрессивного роста

Еще одно важное обновление затрагивает обучение StyleGAN2. Вместо обычного механизма прогрессивного обучения в StyleGAN2 используются пропускающие связи в генераторе и остаточные связи в дискриминаторе, чтобы сеть обучалась как одно целое. Больше не требуется независимое обучение различными разрешениями и их смешивание.

На рис. 10.12 показаны блоки генератора и дискриминатора в сети StyleGAN2.

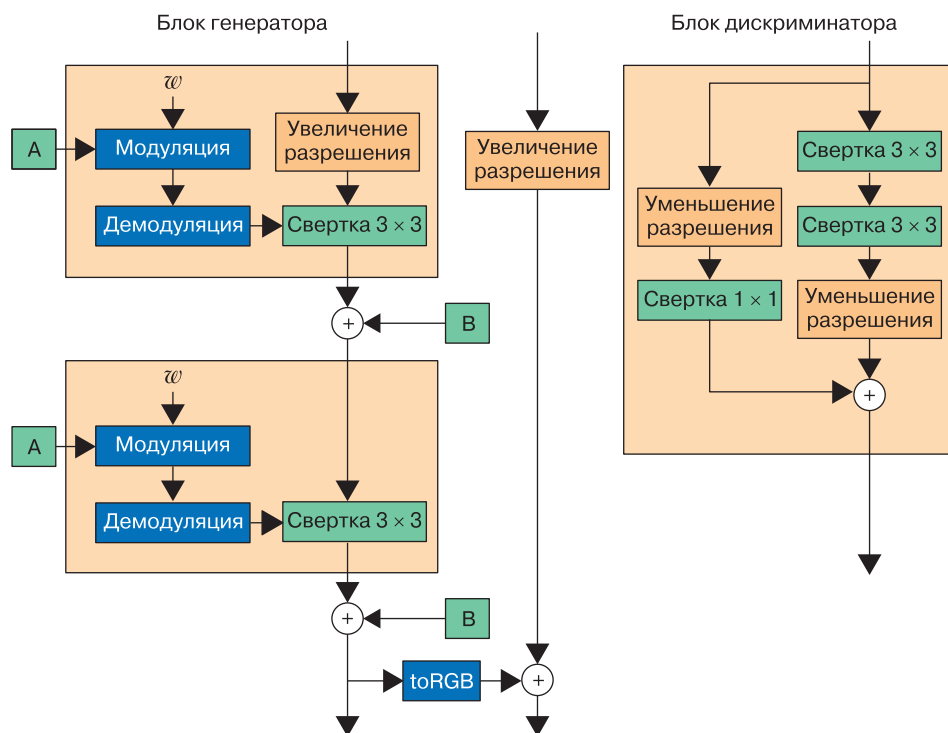


Рис. 10.12. Блоки генератора и дискриминатора в сети StyleGAN2

Важнейшее свойство, которое хотелось бы сохранить, — начало обучения StyleGAN2 с изучения признаков в изображениях низкого разрешения и постепенное уточнение результатов по мере обучения. Авторы показывают, что это свойство действительно сохраняется при использовании такой архитектуры. Каждая сеть получает выгоду от уточнения сверточных весов в слоях с более низким разрешением, при этом пропускающие и остаточные связи, применяемые для передачи выходных данных через слои с более высоким разрешением, практически не затрагиваются. По мере обучения слои с более высоким разрешением начинают доминировать, так как генератор обнаруживает более сложные способы улучшить реалистичность изображений и обмануть дискриминатор. Этот процесс продемонстрирован на рис. 10.13.

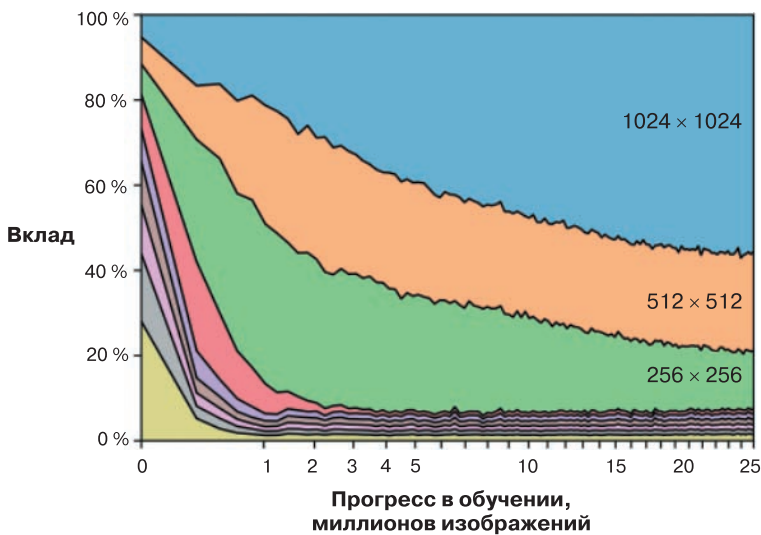


Рис. 10.13. Вклад слоев для каждого разрешения в результат генератора в зависимости от времени обучения (заимствовано из [Karras et al., 2019], <https://arxiv.org/pdf/1912.04958.pdf>)

Вывод сети StyleGAN2

На рис. 10.14 показаны некоторые примеры изображений, сгенерированных сетью StyleGAN2. По данным сайта сравнительного анализа Papers with Code (<https://oreil.ly/VwH2r>), в настоящее время архитектура StyleGAN2 (и масштабированные варианты, такие как StyleGAN-XL [Sauer et al., 2022]) остается самой совершенной в сфере создания изображений после обучения на таких наборах данных, как Flickr-Faces-HQ (FFHQ) и CIFAR-10.



Рис. 10.14. Изображения, сгенерированные сетью StyleGAN2 после обучения на наборах данных FFHQ face и LSUN car (источник: [Karras et al., 2019], <https://arxiv.org/pdf/1912.04958.pdf>)

Другие важные генеративно-сопоставительные сети

В этом разделе мы рассмотрим еще две архитектуры, которые внесли значительный вклад в развитие GAN, — SAGAN и BigGAN.

Self-Attention GAN

Генеративно-сопоставительные сети с механизмом самовнимания (Self-Attention GAN, SAGAN) [Zhang et al., 2018] являются важным достижением, так как демонстрируют возможность использования механизма внимания для генерирования изображений, который первоначально задумывался для поддержки последовательных моделей, таких как трансформер. На рис. 10.15 показан механизм самовнимания из статьи, описывающей эту архитектуру.

Проблема моделей на основе GAN без механизма внимания заключается в том, что сверточные карты признаков могут обрабатывать только локальную информацию. Для передачи информации о пикселах с одной стороны изображения на другую требуется несколько сверточных слоев, которые уменьшают пространственные измерения изображения и увеличивают количество каналов. Точная информация о местоположении постепенно сворачивается на протяжении всего этого процесса, и сеть захватывает объекты все более высокого уровня, что делает такой подход неэффективным в вычислительном отношении для изучения за-

зависимостей между значительно удаленными друг от друга пикселями. Модель SAGAN решает эту проблему включением механизма внимания, который мы исследовали ранее в этой главе (рис. 10.16).

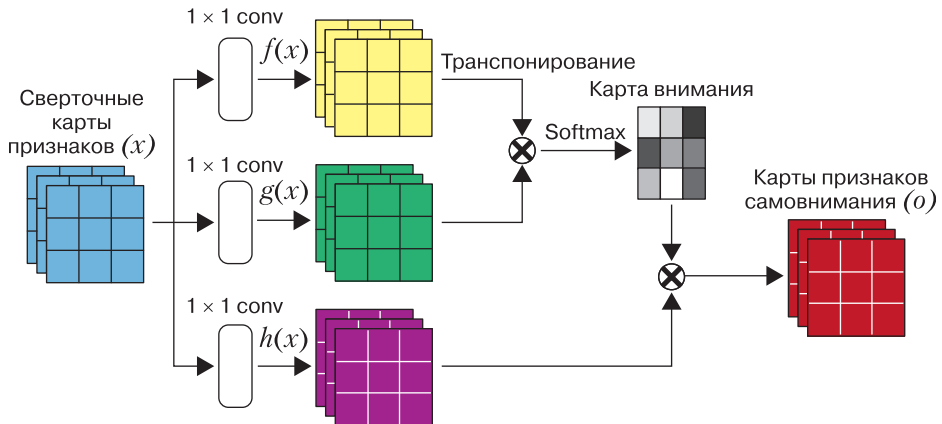


Рис. 10.15. Механизм самовнимания в модели SAGAN
(источник: [Zhang et al., 2018], <https://arxiv.org/abs/1805.08318>)



Рис. 10.16. Изображение птицы, сгенерированное моделью SAGAN (крайнее слева), и карты внимания конечного слоя генератора с механизмом внимания для пикселей, отмеченных тремя цветными точками (остальные три изображения)
(источник: [Zhang et al., 2018], <https://arxiv.org/abs/1805.08318>)

Красная точка отмечает пиксел на теле птицы, поэтому внимание, естественно, будет привлечено к окружающим ячейкам на теле. Зеленая точка отмечает пиксел фона, и здесь взгляд фактически падает на другую сторону от головы птицы, на другие пиксели фона. Синяя точка отмечает пиксел на длинном хвосте птицы, поэтому внимание обращено на другие пиксели хвоста, иногда отстоящие далеко от синей точки. Было бы трудно поддерживать эту действующую на большом расстоянии зависимость пикселей без механизма внимания, особенно в отношении длинных и тонких структур на изображении, например хвост в данном случае.



Обучение собственной модели SAGAN

Официальный код, реализующий обучение SAGAN с помощью TensorFlow, можно найти на GitHub (<https://oreil.ly/gvej0>). Но имейте в виду, что для обучения SAGAN, способной показать результаты, описанные в статье, требуется значительная вычислительная мощность.

BigGAN

Модель BigGAN [Brock et al., 2018], разработанная в DeepMind, расширяет концепции статьи с описанием SAGAN и показывает выдающиеся результаты. На рис. 10.17 представлены некоторые изображения, сгенерированные моделью BigGAN, обученной на наборе данных ImageNet с изображениями размером 128×128 .



Рис. 10.17. Примеры изображений, сгенерированных моделью BigGAN
(источник: [Brock et al., 2018], <https://arxiv.org/abs/1809.11096>)

Наряду с небольшими изменениями базовой модели SAGAN в статье представлены и нововведения, которые выводят модель на совершенно новый уровень сложности. Одно из них — так называемый *трюк усечения* (truncation trick), когда скрытое распределение, используемое для выборки, отличается от распределения $z \sim \mathcal{N}(0, \mathbf{I})$, применяемого во время обучения. В частности, распределение, применяемое для выборки, является *усеченным нормальным распределением* (при получении величины, превышающей определенный порог, выполняется повторная выборка из z). Чем меньше порог усечения, тем выше достоверность генерируемых образцов за счет уменьшения изменчивости (рис. 10.18).

Кроме того, как следует из названия, BigGAN превосходит SAGAN отчасти просто за счет *увеличения* размеров. В BigGAN используются пакеты размером 2048, то есть в восемь раз больше, чем в SAGAN, где пакет имеет размер 256, а число каналов в каждом слое увеличено на 50 %. BigGAN показывает также, что SAGAN можно улучшить структурно за счет включения общего векторного представления, путем ортогональной регуляризации и включения скрытого вектора z во все слои генератора, а не только в начальный.



Рис. 10.18. Трюк усечения: слева направо используется порог 2, 1, 0,5 и 0,04 (источник: [Brock et al., 2018], <https://arxiv.org/abs/1809.11096>)

Полное описание нововведений, предложенных в BigGAN, вы найдете в оригинальной статье и сопроводительной презентации (<https://oreil.ly/vPn8T>).



Использование BigGAN

Учебное пособие по созданию изображений с помощью предварительно обученной сети BigGAN доступно на сайте TensorFlow (<https://oreil.ly/YLbLb>).

VQ-GAN

Еще один важный тип GAN — векторная квантованная генеративно-сопоставительная сеть (Vector Quantized GAN, VQ-GAN), представленная в 2020 году [Esser et al., 2020]. Эта архитектура основана на концепции, изложенной в 2017 году в статье *Neural Discrete Representation Learning* [Oord et al., 2017], согласно которой представления, изученные с помощью VAE, могут быть дискретными, а не непрерывными. Авторы статьи показали, что модель нового типа, Vector Quantized VAE (VQ-VAE), генерирует высококачественные изображения, избегая при этом некоторых проблем, часто наблюдаемых в традиционных VAE с непрерывным скрытым пространством, таких как *апостериорный коллапс* (когда изученное скрытое пространство становится неинформативным из-за слишком мощного декодера).



Первая версия DALL·E — модели преобразования текста в изображение, созданная в OpenAI в 2021 году (см. главу 13), использовала VAE с дискретным скрытым пространством подобно VQ-VAE.

Под *дискретным скрытым пространством* подразумевается список скрытых векторов (таблица кодирования), каждый из которых связан с соответствующим индексом. Задача кодировщика в VQ-VAE — свернуть входное изображение

верхний левый квадрат сетки, скорее всего, будет закодирован как *фон*, тогда как квадраты сетки, расположенные ближе к центру изображения, вряд ли будут закодированы как таковые. Чтобы решить эту проблему, авторы использовали другую модель — авторегрессионную модель PixelCNN (см. главу 5) — для предсказания следующего вектора кода в сетке по заданным предыдущим векторам. Другими словами, распределение изучается моделью, а не задается статически, как в случае с обычным VAE.



Обучение собственной модели VQ-VAE

На сайте Keras вы найдете отличное руководство Саяка Пола (Sayak Paul) по обучению собственной модели VQ-VAE с использованием Keras (<https://oreil.ly/dmcb4>).

В статье с описанием VQ-GAN подробно рассмотрены несколько ключевых изменений в архитектуре VQ-VAE, которые можно видеть на рис. 10.20.

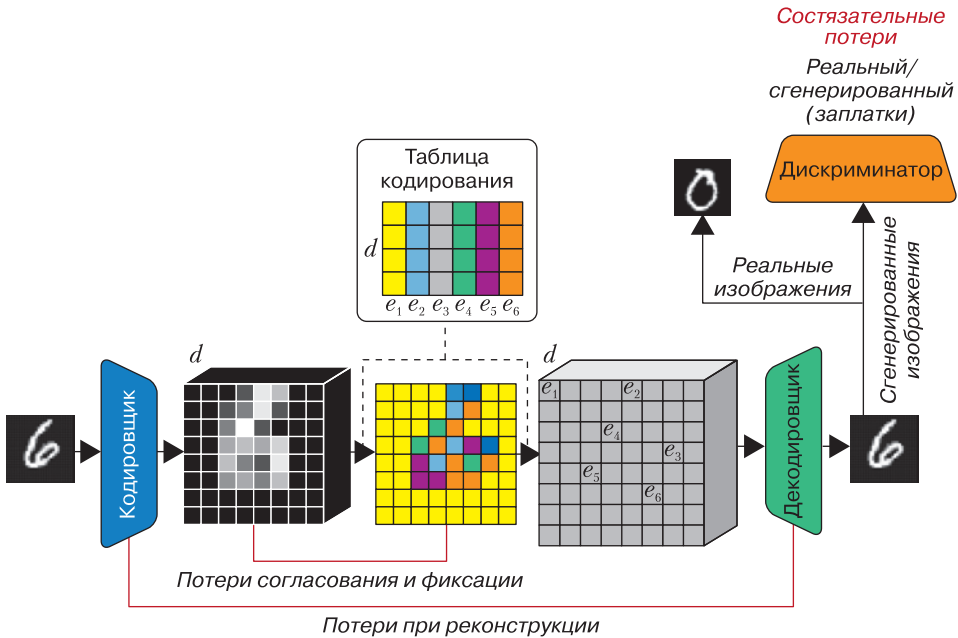


Рис. 10.20. Схема VQ-VAE: дискриминатор GAN помогает стимулировать VAE генерировать более четкие изображения за счет дополнительного члена состязательных потерь

Во-первых, как следует из названия, авторы включили в нее дискриминатор GAN, который пытается отличить сгенерированные изображения, созданные

декодировщиком VAE, от реальных, используя дополнительный член в функции потерь. Как мы знаем, GAN создают более четкие изображения, чем VAE, поэтому это дополнение улучшает общее качество изображений. Обратите внимание на то, что, несмотря на название, VAE все еще сохраняется в модели VQ-GAN — дискриминатор GAN дополняет, а не заменяет VAE. Идея объединения VAE с дискриминатором GAN (VAE-GAN) была впервые предложена Ларсеном (Larsen) и его коллегами в статье 2015 года [Larsen et al., 2015].

Во-вторых, дискриминатор GAN предсказывает, реальными или сгенерированными являются небольшие участки изображений (заплатки — *patches*), а не все изображение сразу. Эта идея (*PatchGAN*) была применена в успешной модели *pix2pix* «изображение — изображение», представленной в 2016 году Исолой (Isola) с коллегами [Isola et al., 2016], а также успешно использовалась в *CycleGAN* [Zhu et al., 2017] — еще одной модели переноса стилей «изображение — изображение». Дискриминатор PatchGAN выводит вектор прогнозов (для каждой заплатки), а не один прогноз для всего изображения. Преимущество использования дискриминатора PatchGAN в том, что функция потерь может измерить, насколько хорошо дискриминатор различает *стиль* изображений, а не их *содержимое*. Так как каждый отдельный элемент в предсказании дискриминатора основан на небольшом фрагменте изображения, он вынужден оценивать стиль фрагмента, а не его содержимое. Это полезно, поскольку мы знаем, что VAE создают изображения, более размытые, чем реальные, поэтому дискриминатор PatchGAN может стимулировать декодировщик VAE генерировать более четкие изображения, чем он мог бы создать при иных условиях.

В-третьих, вместо применения при реконструкции одной функции потерь MSE, которая сравнивает пиксели входного изображения с пикселями на выходе декодировщика VAE, в VQ-GAN используется член потерь восприятия, который вычисляет разницу между картами признаков в промежуточных слоях кодировщика и соответствующих слоях декодировщика. Эта идея взята из статьи Хоу (Hou) и др., опубликованной в 2016 году [Hou et al., 2016], где авторы показывают, что такое изменение функции потерь приводит к созданию более реалистичных изображений.

Наконец, в качестве авторегрессионной части модели вместо PixelCNN используется трансформер, обученный генерировать последовательности кодов. Обучение трансформера проводится отдельно после полного обучения VQ-GAN. Вместо задействования всех предыдущих лексем исключительно авторегрессионным способом авторы предпочитают использовать только лексемы, которые попадают в скользящее окно вокруг прогнозируемой лексемы. Это гарантирует возможность масштабирования модели до более крупных изображений, что требует большего размера скрытой сетки и, следовательно, создания большего количества лексем с помощью трансформера.

ViT VQ-GAN

Последнее расширение VQ-GAN сделали Юй (Yu) с коллегами и описали в статье *Vector-Quantized Image Modeling with Improved VQGAN*, опубликованной в 2021 году [Yu et al., 2021]. Авторы показывают, что сверточный кодировщик и декодировщик в VQ-GAN можно заменить трансформерами (рис. 10.21).

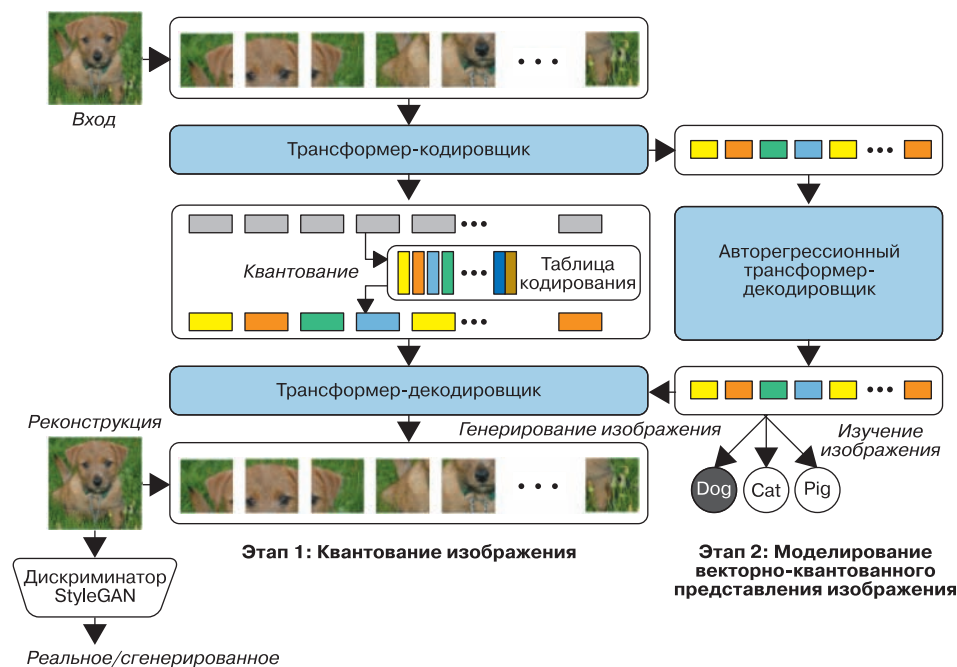


Рис. 10.21. Схема ViT VQ-GAN: дискриминатор GAN помогает стимулировать VAE генерировать более четкие изображения за счет дополнительного члена состязательных потерь (источник: [Yu, Koh, 2022], <https://ai.googleblog.com/2022/05/vector-quantized-image-modeling-with.html>)

В качестве кодировщика авторы применяют *Vision Transformer* (ViT) [Dosovitskiy et al., 2020]. ViT — это архитектура нейронной сети, применяющая модель трансформера, изначально разработанную для обработки текстов на естественном языке, к изображениям. Для извлечения признаков вместо использования сверточных слоев ViT делит изображение на последовательность фрагментов (заплаток), которые маркируются и передаются на вход кодировщика трансформера.

В частности, в ViT VQ-GAN непересекающиеся входные фрагменты (каждый размером 8×8) сначала сглаживаются, затем проецируются в скрытое пространство меньшей размерности, куда добавляются позиционные представления.

Затем эта последовательность подается на вход стандартного кодировщика трансформера, и полученные представления квантуются в соответствии с полученной таблицей кодирования. Эти целочисленные коды затем обрабатываются моделью декодировщика трансформера, и в результате получается последовательность фрагментов, которые можно объединить, сформировав исходное изображение. Общая модель кодировщика-декодировщика обучается как автокодировщик.

Как и в исходной модели VQ-GAN, второй этап обучения включает использование авторегрессионного декодировщика трансформера для генерирования последовательностей кодов. Таким образом, в ViT VQ-GAN, помимо дискриминатора GAN и получаемой таблицы кодирования, имеется также три трансформера. Примеры изображений, созданных моделью ViT VQGAN, из статьи показаны на рис. 10.22.



Рис. 10.22. Примеры изображений, созданных моделью ViT VQGAN, которая была обучена на наборе данных ImageNet (источник: [Yu et al., 2021], <https://arxiv.org/pdf/2110.04627.pdf>)

Резюме

В этой главе я представил обзор некоторых наиболее важных статей с описаниями моделей GAN, появившихся после 2017 года. В частности, мы исследовали ProGAN, StyleGAN, StyleGAN2, SAGAN, BigGAN, VQ-GAN и ViT VQ-GAN.

Мы начали с концепции прогрессивного обучения, впервые представленной в статье о ProGAN 2017 года. В статье о StyleGAN 2018 года были рассмотрены несколько ключевых изменений, расширивших возможности управления генерированием изображений, такие как сеть отображения для создания определенного вектора стиля и синтезирующая сеть, позволяющая внедрять стиль в разных разрешениях. Наконец, StyleGAN2 заменила адаптивную нормализацию экземпляра в StyleGAN шагами модуляции и демодуляции весов, а также

дополнительными улучшениями, такими как регуляризация пути. В документе также показано, как сохранить желательное свойство постепенного улучшения разрешения без необходимости прогрессивного обучения сети.

Появление SAGAN в 2018 году показало, как можно встроить идею внимания в GAN. Механизм внимания позволяет сети фиксировать долгосрочные зависимости, такие как сходство цвета фона на противоположных сторонах изображения, без использования глубоких сверточных карт для передачи информации по пространственным измерениям изображения. Дальнейшим развитием этой идеи стала модель BigGAN, где были внесены несколько ключевых изменений в обучение более крупной сети для улучшения качества генерируемых изображений.

В статье о VQ-GAN авторы показали, что можно весьма эффективно объединять генеративные модели разных типов. Основываясь на оригинальной статье о VQ-VAE, где была представлена концепция VAE с дискретным скрытым пространством, создатели модели VQ-GAN добавили дополнительный дискриминатор, стимулирующий VAE генерировать более четкие изображения за счет дополнительного члена состязательных потерь. В ней используется авторегрессионный трансформер, создающий новую последовательность кодов, которая может декодироваться декодировщиком VAE для создания новых изображений. Статья о ViT VQ-GAN расширяет эту идею, заменяя сверточные кодировщики и декодировщики VQ-GAN трансформерами.

Генерирование музыки

В этой главе вы:

- узнаете, что задачу генерирования музыки можно рассматривать как задачу прогнозирования последовательности и для ее решения применять модели авторегрессии, такие как трансформеры;
- узнаете, как анализировать и кодировать MIDI-файлы с помощью пакета `music21`, чтобы создать обучающий набор;
- узнаете, как использовать синусоидальное позиционное кодирование;
- обучите генерированию музыки трансформер, имеющий несколько входов и выходов для обработки высоты и длительности нот;
- узнаете, как обращаться с полифонической музыкой, включая кодирование сеткой и событийное кодирование;
- обучите модель MuseGAN для создания полифонической музыки;
- используете MuseGAN для настройки различных свойств создаваемых тактов.

Сочинение музыки — сложный творческий процесс комбинирования различных музыкальных элементов, таких как мелодия, гармония, ритм и тембр. Традиционно оно считалось исключительно человеческим видом деятельности, но благодаря последним достижениям нейронные сети научились генерировать музыку, которая одновременно приятна для слуха и имеет долговременную структуру.

Трансформеры — один из самых популярных инструментов создания музыки, потому что ее генерирование можно рассматривать как задачу предсказания последовательности. Эти модели, адаптированные для создания музыки, считают ноты подобными последовательностям лексем в предложении. Модель трансформера учится предсказывать следующую ноту в последовательности на основе предыдущих, и в результате получается музыкальное произведение.

Модель MuseGAN использует совершенно иной подход. В отличие от трансформеров, генерирующих музыку нота за нотой, MuseGAN создает целые музыкальные партии одновременно, рассматривая музыку как изображение,

имеющее ось высоты тона и ось времени. Более того, MuseGAN разделяет различные музыкальные компоненты, такие как аккорды, стиль, мелодия и ритм, чтобы ими можно было управлять независимо.

В этой главе мы будем учиться обрабатывать музыкальные данные и применять модели трансформера и MuseGAN для создания музыки, стилистически похожей на музыку в обучающем наборе.

Введение

Чтобы сочинять гармоничную, приятную для человеческого слуха музыку, компьютер должен уметь преодолевать технические проблемы, подобные рассмотренным в главе 9, выявлять и воссоздавать последовательную музыкальную структуру, а также выбирать последующие ноты из дискретного набора. Более того, модель, генерирующая музыку, сталкивается с дополнительными проблемами, которых нет при генерировании текста, а именно: определением высоты тональности и ритма создаваемой мелодии. Кроме того, музыка часто полифонична, то есть одновременно звучат несколько потоков нот, извлекаемых на разных инструментах, которые объединяются и создают диссонантные (конфликтующие) или консонантные (согласующиеся) гармонии, поэтому при генерировании музыки, в отличие от текста, требуется обрабатывать не единственный поток слов, а параллельные потоки аккордов.

Текст можно обрабатывать по словам. В отличие от текстовых данных музыка представляет собой многослойное переплетение звуков, которые не обязательно воспроизводятся одновременно: наибольший эмоциональный настрой вызывает прослушивание музыки, получающейся объединением различных ритмов в ансамбле. Например, гитарист может играть быстро сменяющиеся ноты, тогда как пианист способен воспроизводить более длинные протяжные аккорды. Проще говоря, создание музыки нота за нотой — сложное дело, потому что часто бывает нежелательно, чтобы все инструменты меняли ноту одновременно.

Мы начнем эту главу с того, что упростим задачу и сосредоточимся на генерировании музыки для одной (монофонической) музыкальной линии. Как вы увидите сами, многие методы генерирования текста, описанные в главе 9, могут использоваться и для генерирования музыки, поскольку эти две задачи имеют много общего. Сначала мы обучим трансформер генерировать музыку в стиле сюит И. С. Баха для виолончели и посмотрим, как механизм внимания позволяет модели сосредотачиваться на предыдущих нотах, чтобы определить наиболее естественную следующую. Затем займемся созданием полифонической музыки и узнаем, как развернуть архитектуру, основанную на GAN, с целью создания музыки для нескольких голосов.

Генерирование музыки с помощью модели трансформера

В этом разделе мы построим трансформер-декодировщик, взяв в качестве прообраза модель трансформера MuseNet (<https://oreil.ly/OaCDY>), созданную в OpenAI, где тоже используется трансформер-декодировщик (похожий на GPT-3), обученный предсказывать следующую ноту с учетом последовательности предыдущих нот.

В задачах генерирования музыки длина последовательности N увеличивается по мере продвижения вперед, то есть вычислять и хранить матрицу $N \times N$ для каждого модуля внимания становится очень дорого. Но мы не можем просто взять и обрезать входную последовательность, потому что желательно, чтобы модель строила фрагменты, опираясь на долгосрочную структуру, и повторяла мотив и фразы, сыгранные несколько минут назад, как часто делают живые композиторы.

Для преодоления этой сложности в MuseNet используется *Sparse Transformer* (<https://oreil.ly/euQiL>), — разреженный трансформер. Весовые коэффициенты в каждой выходной позиции в матрице внимания вычисляют только для ограниченного поднабора входных позиций, тем самым уменьшаются сложность вычислений и объем памяти, необходимый для обучения модели. Поэтому MuseNet может работать с полным вниманием к 4096 элементам и изучать долговременную мелодическую структуру различных стилей. В качестве примеров можно привести фрагменты, имитирующие Шопена (<https://oreil.ly/cmwsO>) и Моцарта (<https://oreil.ly/-T-Je>), в облаке OpenAI на SoundCloud.

Чтобы увидеть, как на продолжение музыкальной фразы часто влияют ноты, записанные несколько тактов назад, взгляните на вступительный отрывок из прелюдии к Сюите № 1 Баха для виолончели (рис. 11.1).



Рис. 11.1. Вступительный отрывок из прелюдии к Сюите № 1 Баха для виолончели



Такты

Такт (или цифра) — это единичный фрагмент мелодии, содержащий небольшое фиксированное количество долей. Например, если, слушая музыку, вы можете сосчитать 1, 2, 1, 2, это значит, что каждый такт состоит из двух долей и, скорее всего, это марш. Если вы можете сосчитать 1, 2, 3, 1, 2, 3, значит, каждый такт состоит из трех долей и вы слушаете вальс.

Какая нота должна идти дальше? Даже если у вас нет музыкального образования, вы все равно сможете догадаться. Сказав «соль» (первая нота в этом произведении), вы будете совершенно правы — возможно, заметив, что каждый такт и половина такта начинаются с одной и той же ноты, и используя этот факт для обоснования своего решения. Хотелось бы, чтобы наша модель смогла проделать тот же трюк, в частности, чтобы она обращала внимание на конкретную ноту из предыдущего полутакта, когда была встречена предыдущая нота G (соль) нижней октавы. Модель, основанная на внимании, такая как трансформер, способна на это и не требует поддерживать скрытое состояние многих тактов, в отличие от рекуррентной нейронной сети.

Любой решивший заняться генерированием музыки должен быть знаком с базовыми положениями музыкальной теории. В следующем разделе мы рассмотрим основные понятия, необходимые для чтения музыки, а также узнаем, как представить ее в числовом виде и преобразовать в набор данных для обучения трансформера.



Запуск кода для этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/11_music/01_transformer/transformer.ipynb` в репозитории книги.

Набор данных JS Bach Cello Suite

В качестве исходных данных в этой главе мы используем набор MIDI-файлов с сочинениями И. С. Баха для виолончели. Загрузить его можно с помощью сценария, находящегося в репозитории книги, как показано в примере 11.1. Этот сценарий сохранит MIDI-файлы в локальной папке `/data`.

Пример 11.1. Загрузка набора данных JS Bach Cello Suites

```
bash scripts/download_music_data.sh
```

Для прослушивания музыки, сгенерированной моделью, понадобится программное обеспечение, способное создавать нотную запись, — например, MuseScore (<https://musescore.org/>), доступный бесплатно.

Парсинг MIDI-файлов

Для загрузки и обработки MIDI-файлов используем библиотеку для Python `music21`. Пример 11.2 демонстрирует, как загрузить MIDI-файл и изобразить его (рис. 11.2) в виде партитуры и структурированных данных.

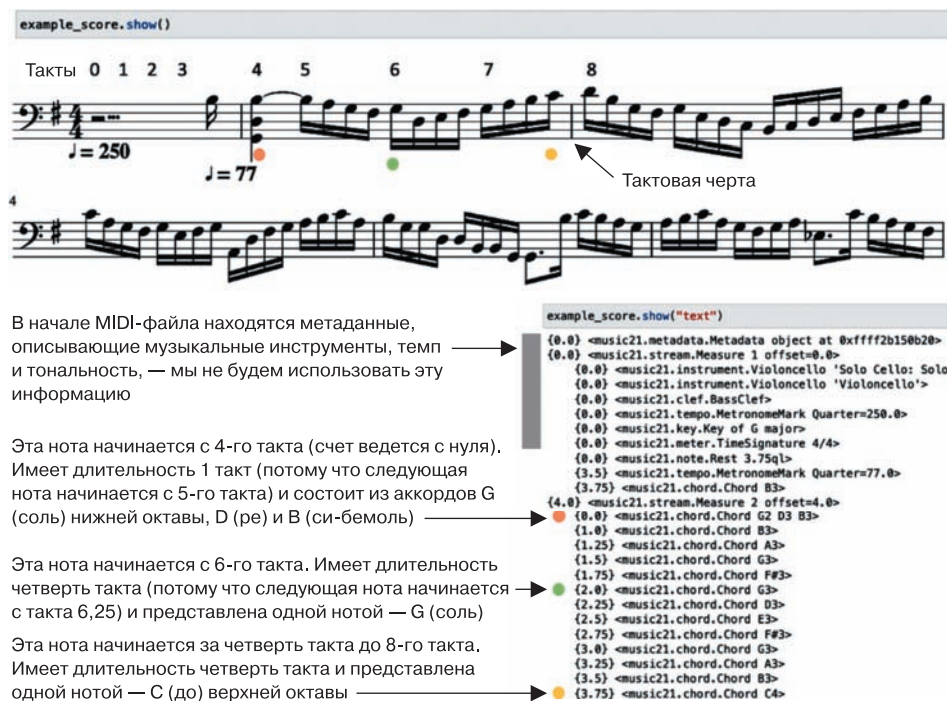


Рис. 11.2. Нотная запись

Пример 11.2. Импортинрование MIDI-файла

```
import music21
```

```
file = "/app/data/bach-cello/cs1-2all.mid"
example_score = music21.converter.parse(file).chordify()
```



Октавы

Число после каждого названия ноты указывает ее октаву — поскольку имена нот (от А до G) ¹ повторяются в каждой октаве, оно помогает однозначно определить высоту ноты. Например, G2 на октаву ниже, чем G3.

¹ В английском языке используются названия нот C, D, E, F, G, A, B, H — до, ре, ми, фа, соль, ля, си-бемоль, си. — *Примеч. пер.*

Теперь полученные результаты нужно преобразовать во что-то похожее на текст. Начнем с перебора каждой партитуры и извлечения каждой ноты и ее длительности в две отдельные текстовые строки, элементы которых разделены пробелами. Тональность и размер произведения закодируем как специальные символы с нулевой длительностью.



Монофоническая и полифоническая музыка

В первом примере будем рассматривать музыку как монофоническую, извлекая из каждого аккорда только верхнюю ноту. Иногда может потребоваться разделить аккорды на части, чтобы сгенерировать музыку, полифоническую по своей природе. Это создает дополнительные проблемы, которые мы рассмотрим далее в этой главе.

Результат показан на рис. 11.3. Сравните его с рис. 11.2, чтобы понять, как исходные музыкальные данные были преобразованы в две текстовые строки.

Notes string

```
START G:major 4/4TS rest B3 B3 B3 A3 G3 F#3 G3 D3 E3 F#3 G
3 A3 B3 C4 D4 B3 G3 F#3 G3 E3 D3 C3 B2 C3 D3 E3 F#3 G3 A3 B
3 C4 A3 G3 F#3 G3 E3 F#3 G3 A2 D3 F#3 G3 A3 B3 C4 A3 B3 ...
```

Duration string

```
0.0 0.0 0.0 3.75 0.25 1.0 0.25 0.25 0.25 0.25 0.25 0.25 0.
25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.25 0.25 0.25 ...
```

Рис. 11.3. Примеры текстовых строк с нотами и их длительностями, соответствующими партитуре, приведенной на рис. 11.2

Это представление больше похоже на текстовые данные, с которыми мы имели дело ранее. Слова представляют комбинации нот и их длительностей, и мы должны попытаться построить модель, предсказывающую следующую ноту и ее длительность, исходя из последовательности предыдущих нот и их длительностей. Ключевое отличие задачи генерирования музыки от генерирования текста — модель должна прогнозировать два потока информации: ноты и их длительности, что отличает ее от моделей, которые мы видели в главе 9.

Кодирование

Чтобы создать набор данных для обучения модели, сначала нужно закодировать каждую ноту и ее длительность, как мы поступали со словами в текстовом корпусе. Для этого можно применить слой `TextVectorization` к нотам и их длительностям по отдельности, как показано в примере 11.3.

Пример 11.3. Кодирование высоты и длительности нот

```
def create_dataset(elements):
    ds = (
        tf.data.Dataset.from_tensor_slices(elements)
        .batch(BATCH_SIZE, drop_remainder = True)
        .shuffle(1000)
    )
    vectorize_layer = layers.TextVectorization(
        standardize = None, output_mode="int"
    )
    vectorize_layer.adapt(ds)
    vocab = vectorize_layer.get_vocabulary()
    return ds, vectorize_layer, vocab

notes_seq_ds, notes_vectorize_layer, notes_vocab = create_dataset(notes)
durations_seq_ds, durations_vectorize_layer, durations_vocab = create_dataset(
    durations
)
seq_ds = tf.data.Dataset.zip((notes_seq_ds, durations_seq_ds))
```

Полный процесс парсинга и кодирования показан на рис. 11.4.

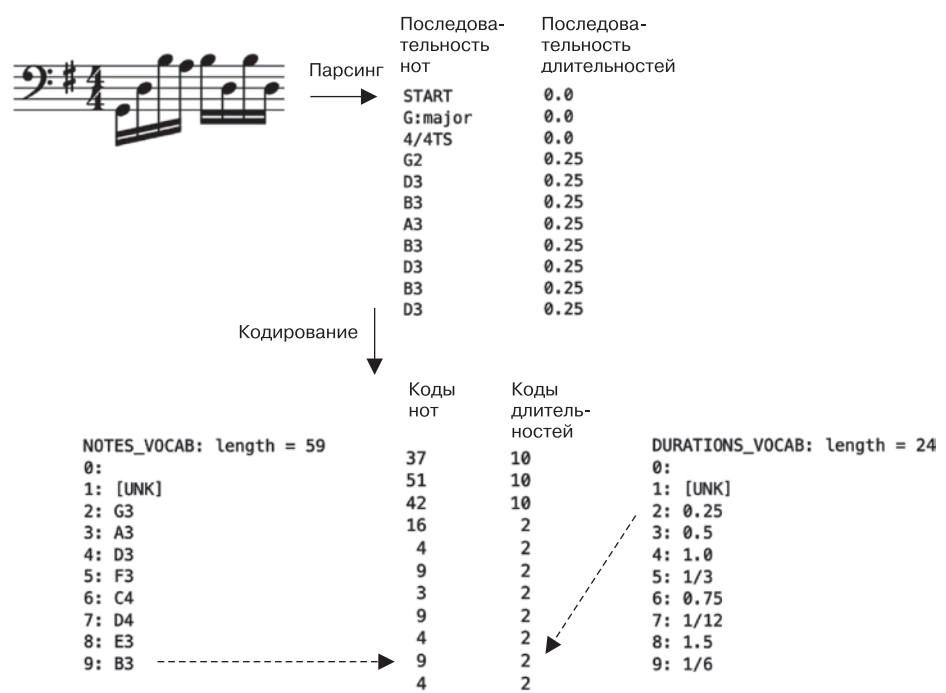


Рис. 11.4. Парсинг MIDI-файлов и кодирование высоты и длительности нот

Создание обучающего набора

Последний шаг предварительной обработки — создание обучающего набора для передачи трансформеру.

Для этого разобьем строки нот и их длительностей на фрагменты по 50 элементов, используя прием скользящего окна. Результатом будет входное окно, сдвинутое на одну ноту, соответственно, трансформер будет обучаться прогнозировать ноту и ее длительность на один временной шаг в будущее с учетом предыдущих элементов в окне. Пример с применением скользящего окна, включающего всего четыре элемента, исключительно ради демонстрации показан на рис. 11.5.

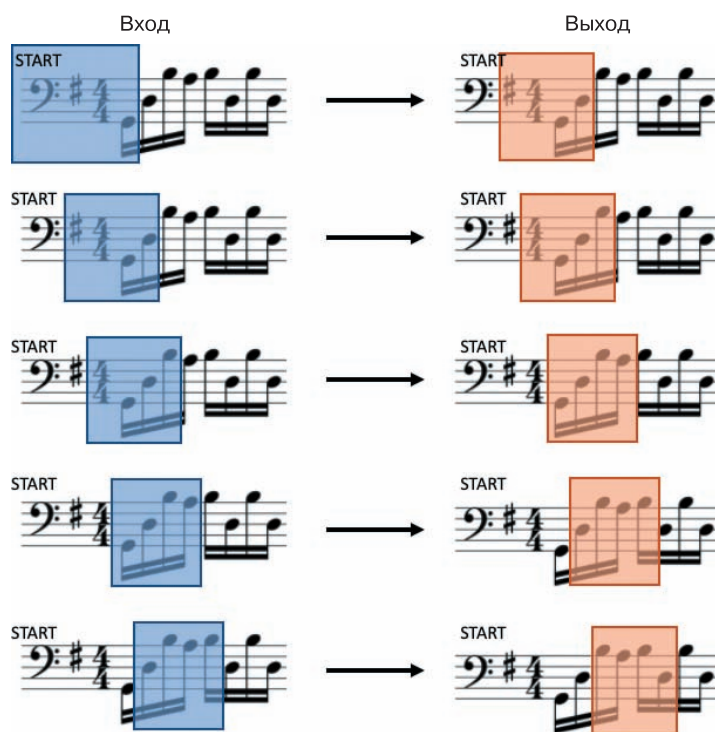


Рис. 11.5. Входные и выходные данные модели музыкального трансформера — в этом примере используется скользящее окно шириной 4 для создания входных фрагментов, которые затем сдвигаются на один элемент для создания целевого результата

Для своего трансформера мы задействуем ту же архитектуру, которую применяли в главе 9 для генерации текста, но с некоторыми важными изменениями.

Синусоидальное позиционное кодирование

Прежде всего рассмотрим еще один вид кодирования позиций. В главе 9 мы использовали простой слой **Embedding** для представления местоположений лексем, отображая каждую целочисленную позицию в отдельный скрытый вектор. Поэтому нам нужно было определить максимальную длину (N) последовательности и обучать модель на последовательностях этой длины. Недостаток этого подхода — невозможность экстраполяции на последовательности, длина которых превышает выбранную максимальную длину. Мы могли бы обрезать входные данные до последних N лексем, но это решение плохо подходит для генерирования протяженного содержимого.

Для решения этой проблемы можно выбрать другой способ представления, называемый *синусоидальным позиционным кодированием* (sine position encoding). Этот способ представления похож на представление, использованное в главе 8 для кодирования дисперсии шума в диффузионной модели. В частности, следующая функция применяется для преобразования позиции слова (pos) во входной последовательности в уникальный вектор длиной d :

$$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10\,000^{2i/d}}\right);$$

$$\text{PE}_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10\,000^{(2i+1)/d}}\right).$$

При малых i длина волны этой функции невелика, поэтому значение функции быстро меняется вдоль оси, определяющей позицию. Большие значения i создают более длинные волны. Таким образом, каждая позиция имеет уникальное представление, определяемое комбинацией волн различной длины.



Обратите внимание на то, что это представление определено для всех возможных позиций. Это детерминированная функция (то есть она не изучается моделью), использующая тригонометрические функции для определения уникального представления каждой возможной позиции.

Модуль *NLP* в библиотеке Keras определяет встроенный слой, как раз реализующий это представление. Благодаря этому мы можем определить свой слой `TokenAndPositionEmbedding`, как показано в примере 11.4.

Пример 11.4. Кодирование высоты и длительности нот

```
class TokenAndPositionEmbedding(layers.Layer):
    def __init__(self, vocab_size, embed_dim):
        super(TokenAndPositionEmbedding, self).__init__()
        self.vocab_size = vocab_size
```

```

self.embed_dim = embed_dim
self.token_emb = layers.Embedding(input_dim=vocab_size, output_dim=embed_dim)
self.pos_emb = keras_nlp.layers.SinePositionEncoding()

def call(self, x):
    embedding = self.token_emb(x)
    positions = self.pos_emb(embedding)
    return embedding + positions

```

На рис. 11.6 показано, что два представления (код ноты и ее позиция) объединяются для создания общего представления последовательности.

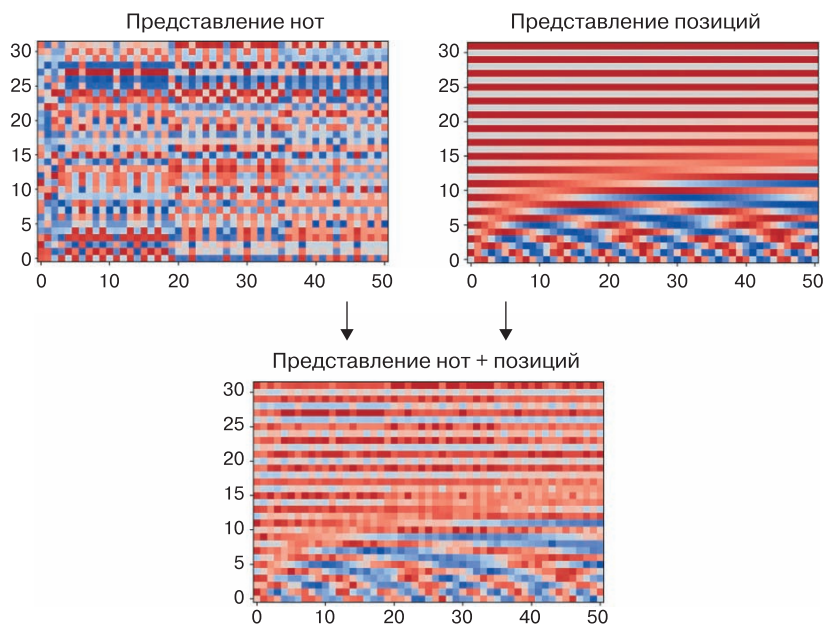


Рис. 11.6. Слой `TokenAndPositionEmbedding` объединяет представления нот и синусоидальные позиции для получения общего представления последовательности

Несколько входов и выходов

Теперь у нас есть два входных потока (ноты и их длительности) и два выходных потока (прогнозируемые ноты и их длительности). Соответственно, нужно адаптировать архитектуру трансформера, чтобы учесть это обстоятельство.

Существует множество способов обработки двойных потоков входных данных. Мы могли бы создать коды, представляющие каждую пару «нота/длительность», а затем рассматривать последовательность как один поток кодов. Однако у этого способа есть обратная сторона — невозможность представить

пары «нота/длительность», отсутствующие в обучающем наборе. Например, в наборе могли бы быть нота G#2 и ноты с длительностью 1/3 независимо друг от друга, но никогда вместе, поэтому код, представляющий G#2:1/3, не будет создан.

Так что мы выбрали способ отдельного кодирования нот и их длительностей с последующим объединением для создания единого общего представления, которое может использоваться последующим блоком трансформера. Аналогично данные на выходе из блока трансформера будут передаваться в два отдельных полносвязных слоя, выдающих прогнозируемые вероятности ноты и ее длительности. Общая архитектура модели показана на рис. 11.7. Формы выходных слоев показаны с размером пакета b и длиной последовательности l .

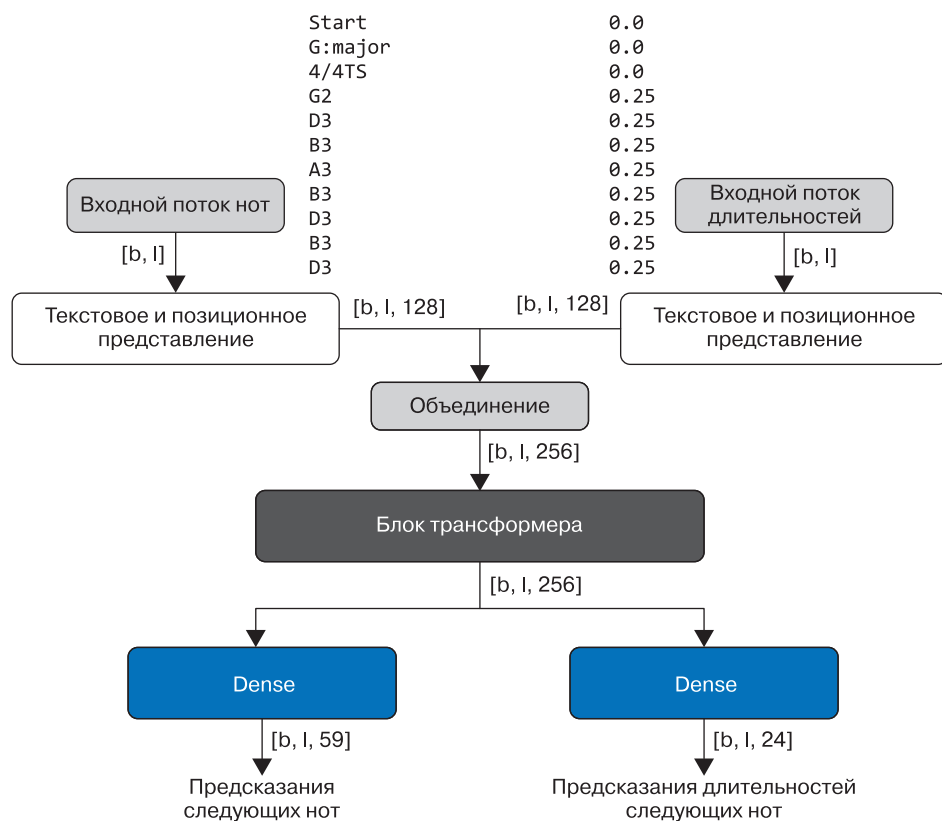


Рис. 11.7. Архитектура модели трансформера, генерирующей музыку

Альтернативное решение — чередовать коды нот и их длительности в одном входном потоке и позволить модели научиться генерировать один выходной поток с чередующимися кодами нот и их длительностей. Однако это решение

влечет за собой дополнительную сложность — необходимость анализа выходных данных в случае, если модель не научится правильно чередовать коды и длительности.



Не существует правильной или неправильной архитектуры модели — эксперименты с различными настройками и выбор наилучших, подходящих именно вам, — это незабываемое удовольствие!

Анализ трансформера генерирования музыки

Чтобы сгенерировать музыкальный фрагмент с нуля, передадим сети последовательность с кодом `START` и длительностью `0.0`, то есть сообщим модели, что она должна начать с самого начала. Затем сгенерируем музыкальный фрагмент, используя тот же способ, с помощью которого в главе 9 создавались текстовые последовательности.

1. На основе текущей последовательности (с нотами и их длительностями) модель спрогнозирует два распределения вероятностей для выбора следующей ноты и ее продолжительности.
2. Выберем значения из обоих распределений, используя параметр `temperature`, который управляет долей случайности при выборе.
3. Выбранную ноту и продолжительность добавим в соответствующие последовательности.
4. Повторим процесс с новыми входными последовательностями до получения фрагмента желаемой длины.

Примеры мелодий, сгенерированных моделью с нуля при разных количествах эпох обучения, представлены на рис. 11.8. Мы использовали параметр `temperature = 0.5` и для нот, и для их длительностей.

Основное внимание в этом разделе мы уделим предсказаниям тональности нот, потому что сюиты Баха для виолончели имеют очень сложный гармонический строй, ритмику которого трудно выявить — для этого требуется гораздо более тщательное обучение сети. Тем не менее приемы анализа, которые описываются далее (например, партитуру ударных инструментов), можно применить и к ритмическим прогнозам модели, что может оказаться особенно актуальным для других музыкальных стилей, которые могут быть использованы для обучения модели.

Следует также отметить несколько моментов, касающихся сгенерированных фрагментов, приведенных на рис. 11.8. Во-первых, с каждой эпохой обучения музыка все больше усложняется. В самом начале модель стремится воспроизводить

все те же группы нот и ритмов. К эпохе 10 модель начала генерировать небольшие серии нот, а к эпохе 20 — производить интересные ритмы, прочно утвердившись в выборе тональности (ми-бемоль мажор).

Эпоха 2

Эпоха 10

Эпоха 20

Рис. 11.8. Некоторые примеры музыкальных фрагментов, сгенерированных моделью, на вход которой поданы единственный код *START*, представляющий ноту, и длительность 0,0

Во-вторых, мы можем проанализировать распределение высот нот во времени, построив тепловую карту прогнозируемого распределения на каждом временном шаге. Пример тепловой карты для эпохи 20, изображенной на рис. 11.8, представлен на рис. 1.19.

Любопытно, что модель четко определила, какие ноты принадлежат конкретным *тональностям*. Об этом свидетельствуют пробелы в распределениях, соответствующих нотам, не принадлежащим тональности. Например, в ряду для ноты 54 (соль-бемоль/фа мажор) имеется серый разрыв. Эта нота вряд ли появится в музыкальном произведении, имеющем тональность ми-бемоль мажор. На ранней стадии процесса (левая часть диаграммы) тональность еще не определена, поэтому существует большая неопределенность при выборе следующей ноты. По мере продвижения модель выбирает определенную тональность, поэтому некоторые ноты почти не имеют шанса быть выбранными. Особенно примечательно то, что изначально модель не выбирала тональность для генерируемой

музыки, а буквально придумала ее, пытаясь подобрать ноту, которая лучше всего соответствует выбранным ранее.

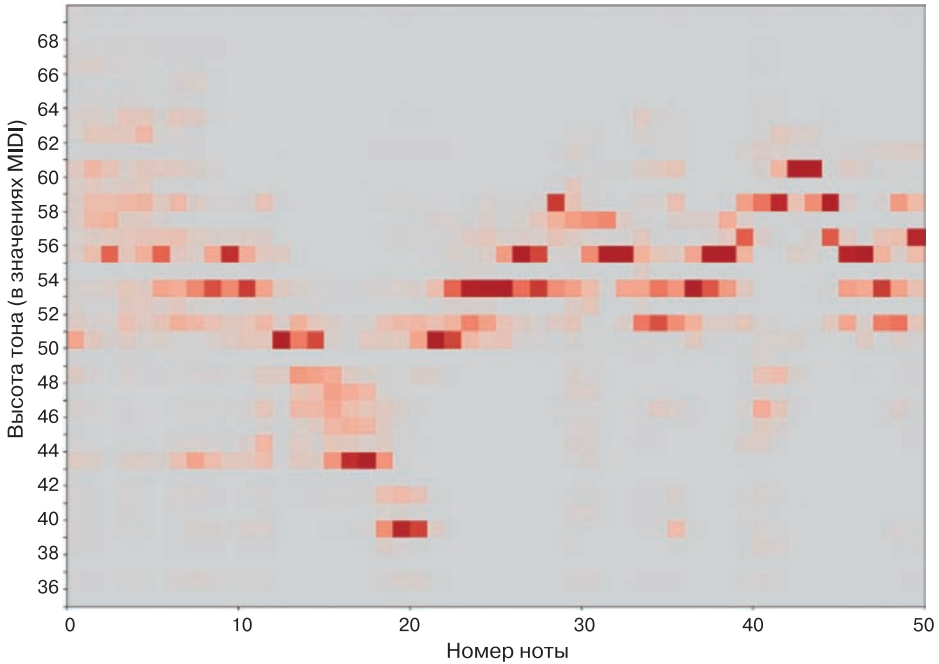


Рис. 11.9. Распределение вероятностей возможных следующих нот в эпоху 20: чем темнее квадратик, тем выше вероятность того, что следующей модель выберет эту ноту

Стоит также отметить, что модель выучила характерный для Баха стиль понижения тона в конце фразы с последующим резким переходом на высокую ноту. Посмотрите на окружение ноты 20. Эта фраза заканчивается низким ми-бемоль — в своих сюитах для виолончели Бах в начале следующей фразы обычно возвращается к более высокому, более звучному диапазону инструмента, что очень точно предсказывает модель. Между низким ми-бемоль (высота тона 39) и следующей предсказанной нотой (с высотой тона 50) имеется большой серый промежуток, что говорит об отсутствии переборов в низкой тональности.

Наконец, проверим, как работает наш механизм внимания. На рис. 11.10 изображена сгенерированная последовательность нот: вертикальная ось показывает, куда было направлено внимание сети при прогнозировании каждой ноты на горизонтальной оси. Чем темнее квадратик, тем больше внимания уделяется скрытому состоянию, соответствующему этой точке последовательности. Для простоты на этой диаграмме показаны только ноты, но сеть учитывает и их продолжительности.

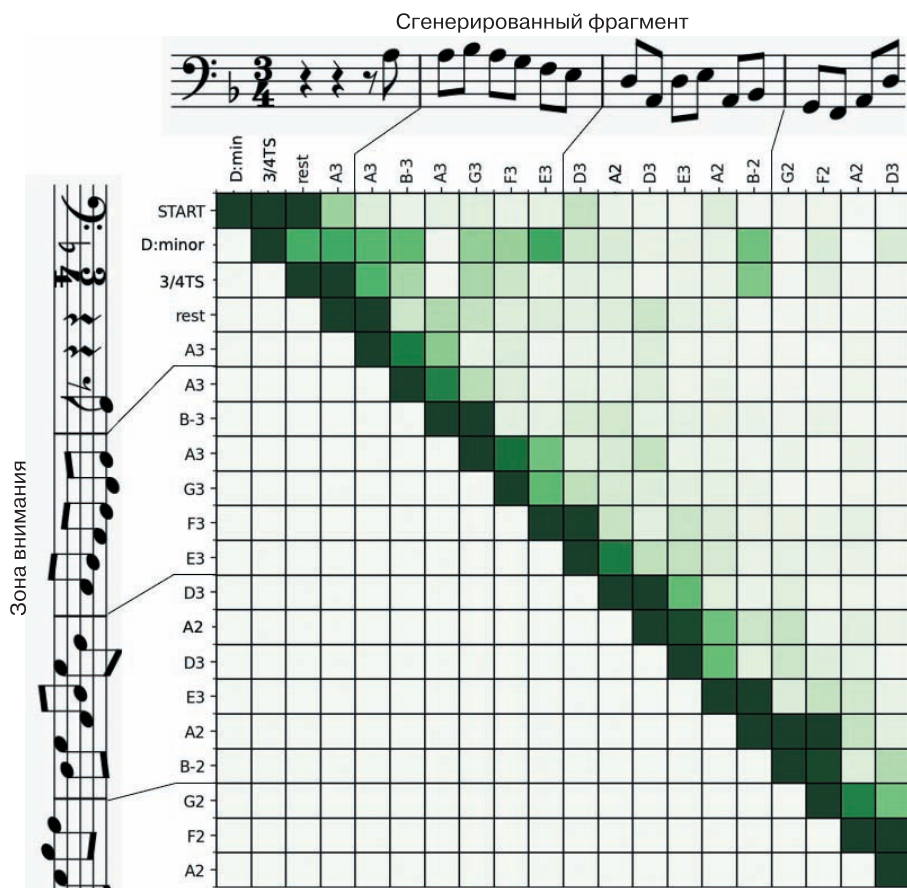


Рис. 11.10. Цвет каждого квадратика отражает количество внимания, уделявшегося скрытому состоянию сети, которое соответствует ноте на вертикальной оси в точке прогнозирования на горизонтальной оси

Как видите, выбирая первую ноту, размер такта и все остальное, сеть решила сосредоточить почти все свое внимание на коде **START**. В этом есть определенный смысл, поскольку этот артефакт всегда присутствует в начале музыкального произведения. Но как только появляются первые ноты, сеть практически перестает обращать внимание на код **START**.

Далее сеть примерно одинаково распределяет свое внимание между предыдущими 2–4 нотами и редко выделяет ноты, находящиеся более чем на четыре ноты позади. И в этом тоже есть определенный смысл: скорее всего, в предыдущих четырех нотах содержится достаточно информации, чтобы понять, как эта музыкальная фраза должна продолжаться. Кроме того, некоторые ноты

наиболее соответствуют тональности ре минор, например, Е3 (ми третьей октавы, седьмая нота пьесы) и В-2 (си-бемоль второй октавы, 14-я нота). Это обстоятельство особенно интересно, потому что именно на этих нотах основана тональность ре минор. Сеть должна *заглянуть назад*, чтобы определить, что в определении тональности присутствует си-бемоль минор, а не обычная си-бемоль, но в определении тональности нет ноты ми-бемоль — вместо нее следует использовать обычную ноту ми.

Есть также примеры, когда сеть решила игнорировать определенную близко расположенную ноту, потому что та не добавляет информации для понимания фразы. Например, предпоследняя нота ля (A2) не особенно внимательна к си-бемоль (В-2) тремя нотами назад, но более внимательна к ля (A2) четырьмя нотами назад. Нота A2, попадающая в такт, показалась модели интереснее, чем В-2 вне такта, которая является просто проходной нотой.

Напомню: мы ничего не сообщали модели о том, как связаны ноты через октавы, — она сама обнаружила это, просто изучив музыку И. С. Баха, что особенно примечательно.

Генерирование полифонической музыки

Модель трансформера, которую мы исследовали в этом разделе, прекрасно справляется с задачей генерирования монофонической музыки. А можно ли адаптировать ее для генерирования полифонии? Сложность заключается в том, как представить несколько партий в одной последовательности кодов. В предыдущем разделе мы решили разделить ноты и их длительности на две отдельные входные и выходные последовательности, а также увидели, что могли бы чередовать ноты и длительности в одном потоке. Эту идею можно применить для обработки полифонической музыки. Здесь будут представлены два разных подхода: *кодирование сеткой* и *событийное кодирование*, как описано в статье 2018 года *Music Transformer: Generating Music with Long-Term Structure* [Huang et al., 2018].

Кодирование сеткой

Рассмотрим два такта хорала И. С. Баха (рис. 11.11). Здесь мы имеем четыре отдельные партии: сопрано (S), альт (A), тенор (T), бас (B), написанные на разных нотных станах.

Представим, что мы рисуем эту музыку на сетке, где ось Y представляет высоту ноты, а ось X — количество шестнадцатых нот, исполненных с момента начала пьесы. Заполненный квадрат сетки соответствует звучащей в этот момент ноте. Все четыре партии нарисованы на одной сетке. Она известна как

перфорированная лента для механического пианино, потому что напоминает рулон бумаги с пробитыми в нем отверстиями, который использовался в качестве носителя информации до изобретения цифровых систем.



Рис. 11.11. Первые два такта хорала И. С. Баха

Мы можем преобразовать сетку в поток кодов нот, перемещаясь сначала по четырем партиям, а затем по временным шагам. В результате получается последовательность $S_1, A_1, T_1, B_1, S_2, A_2, T_2, B_2$ и т. д., где индекс обозначает временной шаг (рис. 11.12).

Такую последовательность кодов можно использовать для обучения трансформера предсказанию следующей ноты на основе предыдущих. Затем сгенерированную последовательность можно декодировать обратно в сеточную структуру, разворачивая ее в группы по четыре ноты — по одной для каждой партии. Этот метод дает на удивление хорошие результаты, несмотря на то что часто разным партиям назначается одна и та же нота.

Однако этот подход имеет некоторые недостатки. Во-первых, обратите внимание на то, что модель не может отличить одну длинную ноту от двух более коротких одинаковой высоты, следующих друг за другом. Это связано с тем, что выбранный способ кодирования не позволяет определить продолжительность ноты — только ее присутствие на каждом временном шаге.

Во-вторых, этот метод требует, чтобы музыка имела регулярный ритм, позволяющий разделить мелодию на фрагменты разумного размера. Например, используя данную систему, не получится закодировать триоли — группы из трех нот, играемых за одну долю. Мы могли бы разделить мелодию на фрагменты по 12 шагов размером $1/4$ вместо четырех, что утроило бы количество кодов, необходимых для представления одного и того же музыкального отрывка, увеличило накладные расходы на обучение и повлияло на возможность ретроспективного анализа модели.

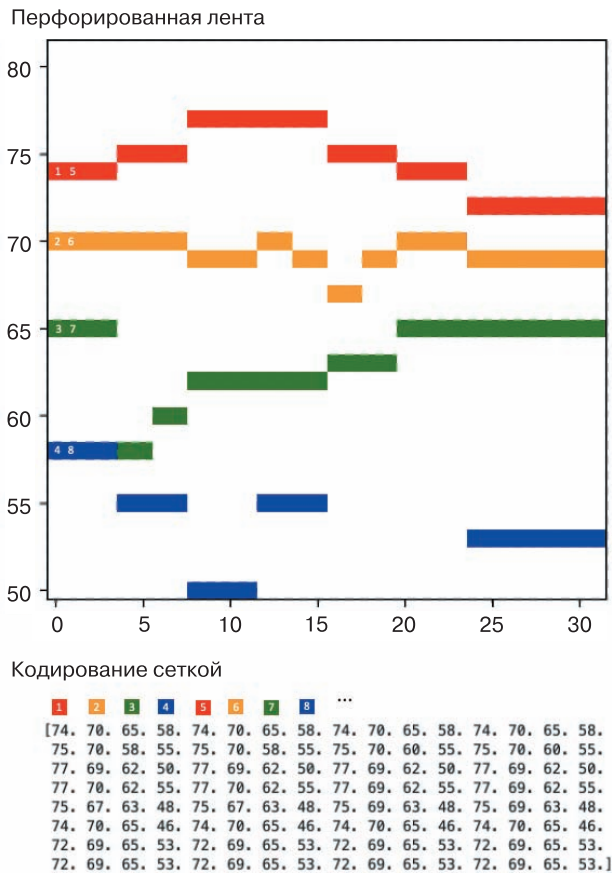


Рис. 11.12. Представление первых двух тактов хора Баха

И в-третьих, не совсем понятно, как закодировать другие компоненты, такие как динамика (уровень громкости каждой партии) или изменения темпа.

Мы заперты в двумерной сетке, которая обеспечивает удобный способ представления высоты тона и ритма, но не предлагает простого способа включения других компонентов, делающих музыку приятной на слух.

Событийное кодирование

Более гибкий подход — использовать событийное кодирование. В этом случае набор кодов можно рассматривать как последовательность, которая буквально описывает звучание музыки в форме последовательности событий с помощью богатого набора кодов.

Например, на рис. 11.13 показано использование трех видов кодов:

- `NOTE_ON<высота>` — начало звучания ноты заданной высоты;
- `NOTE_OFF<высота>` — прекращение звучания ноты заданной высоты;
- `TIME_SHIFT<шаг>` — сдвиг во времени вперед на заданный шаг.



Событийное кодирование

```
[NOTE_ON<74>, NOTE_ON<70>, NOTE_ON<65>, NOTE_ON<58>
, TIME_SHIFT<1.0>
, NOTE_OFF<74>, NOTE_OFF<65>, NOTE_OFF<58>
, NOTE_ON<75>, NOTE_ON<58>, NOTE_ON<55>
, TIME_SHIFT<0.5>
, NOTE_OFF<58>, NOTE_ON<60>
, TIME_SHIFT<0.5>
, NOTE_OFF<75>, NOTE_OFF<70>, NOTE_OFF<60>, NOTE_OFF<55>
, NOTE_ON<77>, NOTE_ON<69>, NOTE_ON<62>, NOTE_ON<50>
, TIME_SHIFT<1.0>
, NOTE_OFF<69>, NOTE_OFF<50>, NOTE_ON<70>, NOTE_ON<55>
, TIME_SHIFT<0.5>
, NOTE_OFF<70>, NOTE_ON<69>
, TIME_SHIFT<0.5>
, NOTE_OFF<77>, NOTE_OFF<69>, NOTE_OFF<62>, NOTE_OFF<55>
, ...]
```

Рис. 11.13. Событийное кодирование первого такта хорала Баха

Эти коды можно использовать для создания последовательности, описывающей мелодию как набор инструкций.

Мы могли бы добавить другие типы кодов, отражающих изменение динамики и темпа последующих нот. Этот метод также позволяет генерировать триоли на фоне четвертных нот, разделяя ноты в триолях кодами `TIME_SHIFT<0.33>`. В целом это более выразительный способ кодирования, но он усложняет трансформеру изучение закономерностей мелодий в обучающем наборе, поскольку он по определению менее структурирован, чем метод сетки.



Я советую попробовать реализовать эти методы генерирования полифонической музыки и обучить трансформер на новом закодированном наборе данных, используя все знания, которые вы накопили к данному моменту. Я бы также рекомендовал познакомиться с руководством доктора Тристана Беренса (Dr. Tristan Behrens) по исследованиям в области генерирования музыки, доступным на GitHub (<https://oreil.ly/YfaiJ>), где представлен всесторонний обзор различных статей по теме генерирования музыки с применением глубокого обучения.

В следующем разделе мы воспользуемся совершенно другим подходом к созданию музыки, применив GAN.

MuseGAN

Возможно, вы заметили некоторое сходство между перфорированной лентой на рис. 11.12 и современными произведениями художников-абстракционистов. В связи с этим возникает вопрос: можно ли рассматривать эту ленту как *изображение* и использовать методы генерирования изображений вместо методов генерирования последовательностей?

Фактически, как мы увидим далее, задачу генерирования музыки действительно можно считать прямым аналогом задачи генерирования изображений. Это означает, что вместо трансформера вполне можно задействовать те же методы на основе свертки, которые так хорошо зарекомендовали себя в задачах генерирования изображений, в музыке, в частности в сетях GAN.

Сеть MuseGAN была представлена в статье *MuseGAN: Multi-Track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment* [Dong et al., 2017]. Ее авторы показали, как с помощью новейшего фреймворка GAN обучить модель генерировать полифоническую музыку и, разделяя обязанности между векторами шума, которые питают генератор, обеспечить полный контроль над высокоуровневыми признаками музыки.

Начнем с представления набора данных Bach Chorale.



Запуск кода для этого примера

Код для этого примера можно найти в блокноте Jupyter, доступном по адресу `notebooks/11_music/02_musegan/musegan.ipynb` в репозитории книги.

Набор данных Bach Chorale

Чтобы опробовать этот пример, сначала нужно скачать MIDI-файлы, на которых будет обучаться сеть MuseGAN. Используем набор данных из 229 хоралов И. С. Баха для четырех голосов.

Загрузить набор данных можно с помощью сценария, находящегося в репозитории книги, как показано в примере 11.5. Этот сценарий сохранит MIDI-файлы в локальной папке `/data`.

Пример 11.5. Загрузка набора данных Bach Chorale

```
bash scripts/download_bach_chorale_data.sh
```

Набор данных состоит из массива, содержащего по четыре числа для каждого временного шага — высоту ноты MIDI для каждого из четырех голосов. Временной шаг в этом наборе равен шестнадцатой ноте. Так, например, в одном такте с четырьмя четвертями будет 16 временных шагов. Кроме того, набор данных автоматически делится на *обучающую*, *проверочную* и *контрольную* части. Для обучения MuseGAN используем *обучающий* набор.

Сначала данные нужно привести к форме, пригодной для передачи в GAN. Сгенерируем два такта музыки и для этого сначала извлечем из каждого хораля только первые два такта. Каждый такт делится на 16 временных шагов, то есть всего для четырех голосов — 84 шага.



В дальнейшем мы будем называть голоса (партии) дорожками, чтобы сохранить соответствие терминологии с оригинальной статьей.

Соответственно, преобразованные данные будут иметь следующую форму:

```
[BATCH_SIZE, N_BARS, N_STEPS_PER_BAR, N_PITCHES, N_TRACKS]
```

где

```
BATCH_SIZE = 64
N_BARS = 2
N_STEPS_PER_BAR = 16
N_PITCHES = 84
N_TRACKS = 4
```

Чтобы привести данные к этой форме, закодируем числовые значения нот в вектор длиной 84 и разобьем каждую последовательность нот на два такта по 16 временных шагов. Предполагается, что каждый хораль имеет четыре доли в каждом такте, что вполне разумно, и даже если бы это было не так, то не повлияло бы отрицательно на обучение модели.

На рис. 11.14 показано, как два столбца исходных данных преобразуются в набор данных, представляющий перфорированную ленту, который мы будем использовать для обучения GAN.

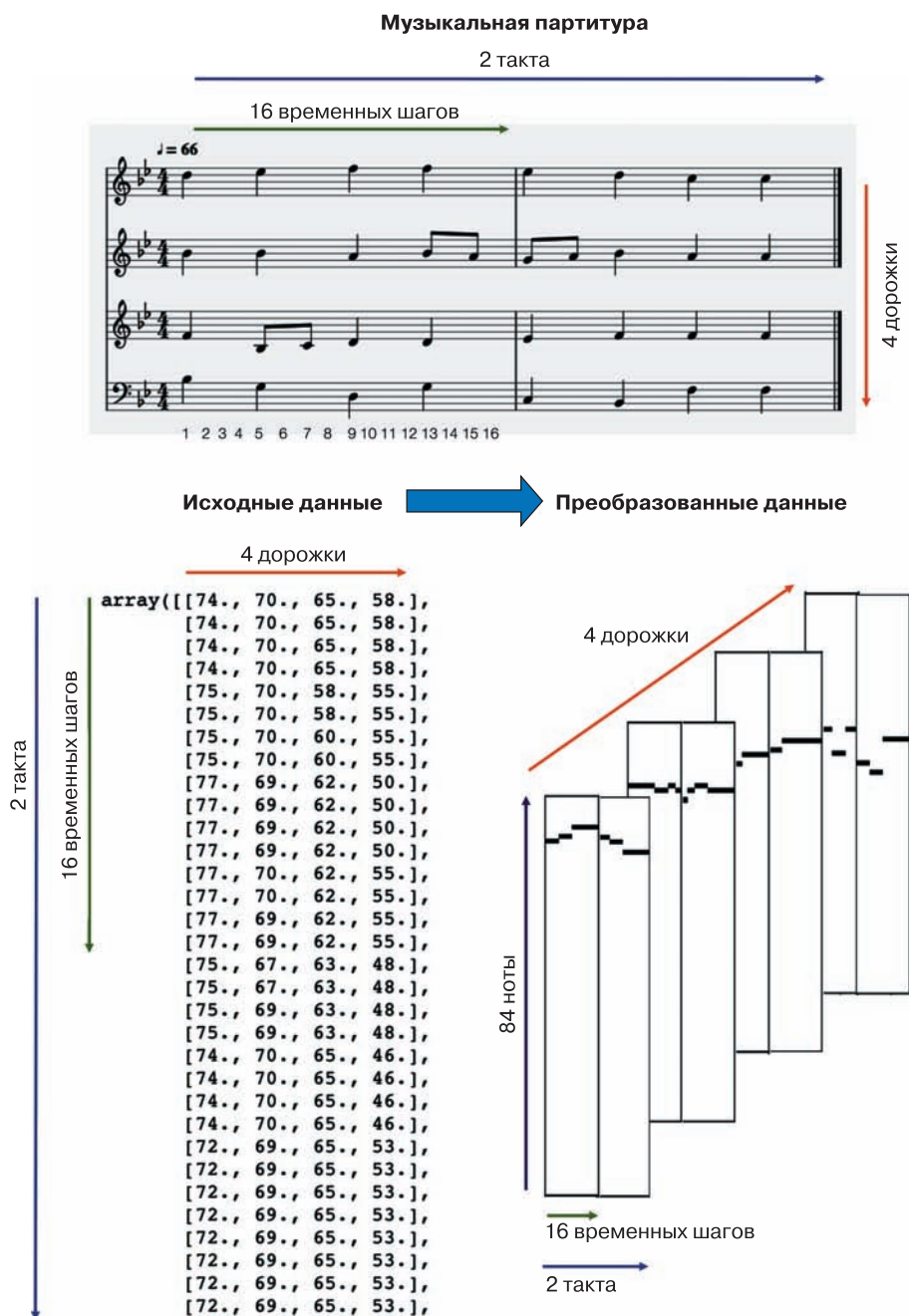


Рис. 11.14. Обработка двух тактов исходных данных в представление перфорированной ленты, которое будет применяться для обучения GAN

Генератор MuseGAN

Подобно всем генеративно-сопоставительным сетям (GAN), MuseGAN состоит из генератора и критика. Генератор пытается одурачить критика своими музыкальными творениями, а критик старается предотвратить это, пытаясь отличить поддельные хоралы Баха, созданные генератором, от настоящих.

Особенность сети MuseGAN в том, что ее генератор принимает на входе не один вектор шума, а целых четыре, которые соответствуют четырем составляющим — аккордам, стилю, мелодии и дорожке. Управляя каждым из этих входов, можно менять высокоуровневые свойства сгенерированной музыки. Обобщенное представление генератора показано на рис. 11.15.

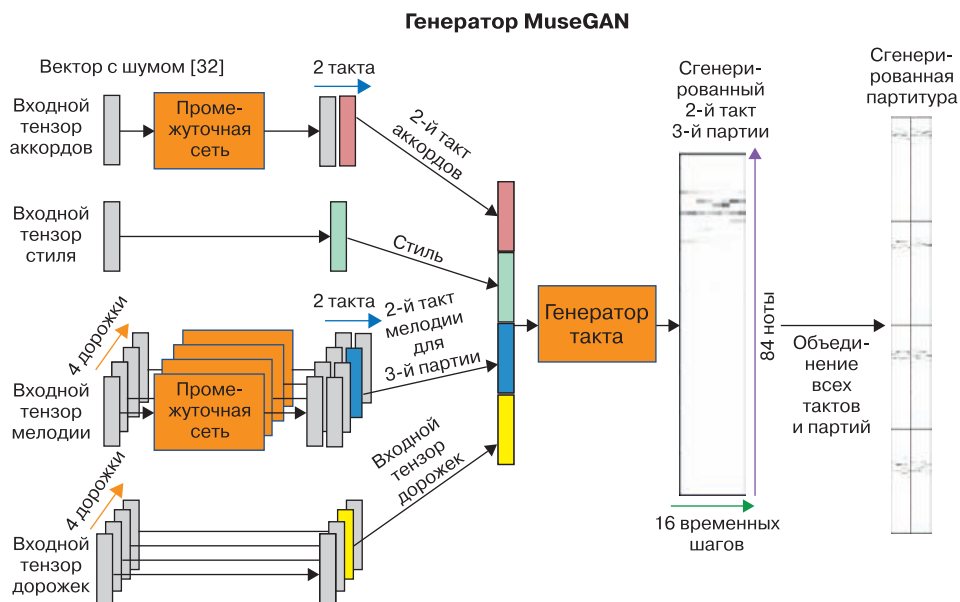


Рис. 11.15. Обобщенная схема генератора MuseGAN

На диаграмме видно, как входные тензоры аккордов и мелодии сначала передаются в *промежуточную сеть*, выводящую тензор, одно из измерений которого равно количеству сгенерированных тактов. Благодаря этому нет необходимости растягивать во времени входные тензоры стилей и дорожек, так как они остаются постоянными на протяжении всего произведения.

Затем, чтобы сгенерировать конкретный такт для конкретной партии, объединяются соответствующие векторы аккордов, стиля, мелодии и дорожек. Они формируют более длинный вектор, передаваемый в генератор тактов, который

в конечном итоге выводит указанный такт для указанной партии. Объединением сгенерированных тактов для всех партий создается партитура, которую критик может сравнить с настоящими.

Сначала посмотрим, как построить промежуточную сеть.

Промежуточная сеть

Задача промежуточной сети (нейронной сети, состоящей из слоев обратной свертки) — преобразовать один входной вектор шума длиной $Z_DIM = 32$ в другой вектор шума для каждого такта (также длиной 32). Код создания этой сети показан в примере 11.6.

Пример 11.6. Конструирование промежуточной сети

```
def conv_t(x, f, k, s, a, p, bn):
    x = layers.Conv2DTranspose(
        filters = f
        , kernel_size = k
        , padding = p
        , strides = s
        , kernel_initializer = self.weight_init
    )(x)

    if bn:
        x = layers.BatchNormalization(momentum = 0.9)(x)

    x = layers.Activation(a)(x)
    return x

def TemporalNetwork(self):
    input_layer = layers.Input(shape=(Z_DIM,), name='temporal_input') ❶
    x = layers.Reshape([1,1,Z_DIM])(input_layer) ❷
    x = conv_t(
        x, f=1024, k=(2,1), s=(1,1), a= 'relu', p = 'valid', bn = True
    ) ❸
    x = conv_t(
        x, f=Z_DIM, k=(N_BARS - 1,1), s=(1,1), a = 'relu', p = 'valid', bn = True
    )
    output_layer = layers.Reshape([N_BARS, Z_DIM])(x) ❹
    return models.Model(input_layer, output_layer)
```

- ❶ На входе промежуточная сеть принимает вектор длиной 32 (Z_DIM).
- ❷ Этот вектор преобразуется в тензор с формой $1 \times 1 \times 32$ каналами, чтобы к нему можно было применить операцию обратной двумерной свертки.
- ❸ Слои `Conv2DTranspose` увеличивают размер тензора вдоль одной из осей, чтобы он получил длину, равную N_BARS .
- ❹ С помощью слоя `Reshape` удаляются ставшие ненужными дополнительные измерения.

Мы используем сверточные операции вместо передачи двух независимых векторов тактов с тем, чтобы сеть знала, что такты должны следовать друг за другом. Применение нейронной сети для расширения входного вектора вдоль оси времени позволяет модели узнать, что музыка исполняется тактами и каждый следующий такт не является полностью независимым от предыдущего.

Аккорды, стиль, мелодия и дорожки

Теперь рассмотрим поближе четыре входных вектора, которые передаются в генератор.

Аккорды

Входной тензор аккордов имеет длину 32 (`Z_DIM`). Задача этого вектора — обеспечить развитие музыки во времени, общее для всех дорожек, поэтому мы используем `TemporalNetwork` для преобразования этого единственного вектора в другой скрытый вектор для каждого такта. Обратите внимание: несмотря на свое имя `chords_input`, этот тензор может контролировать все, что касается изменения музыки с началом нового такта, например общий ритмический стиль, без привязки к какой-либо конкретной партии.

Стиль

Входной тензор стиля тоже имеет длину `Z_DIM`. Он передается в генератор тактов без каких-либо изменений, потому что для него не существует понятий партий и тактов. Другими словами, генератор тактов должен использовать этот тензор для согласования тактов и партий.

Мелодия

Входной тензор мелодии — это массив с формой `[N_TRACKS, Z_DIM]`, и для каждой партии модели передается случайный вектор шума длиной `Z_DIM`. Каждый из этих векторов передается через свою копию промежуточной сети, описанной ранее. Обратите внимание на то, что веса в этих копиях разные, поэтому на выходе для каждой партии в каждом такте получается вектор длиной `Z_DIM`. При подобной организации генератор тактов может использовать этот вектор для точной настройки содержимого каждого отдельного такта в каждой партии.

Дорожки

Входной тензор дорожек тоже является массивом с формой `[N_TRACKS, Z_DIM]`, то есть массивом, содержащим вектор со случайным шумом и длиной `Z_DIM` для каждой партии. В отличие от тензора мелодии, он не преобразуется промежуточной сетью, а передается непосредственно в генератор тактов, как и тензор стиля. Но в отличие от тензора стиля, каждой партии соответствует отдельная входная дорожка. Таким образом, эти векторы можно использовать для независимой настройки каждой партии.

Мы можем обобщить назначение каждого компонента генератора MuseGAN, как показано в табл. 11.1.

Таблица 11.1. Компоненты генератора MuseGAN

Параметр	Результат разный для каждого такта?	Результат разный для каждой дорожки?
Стиль	X	X
Дорожки	X	✓
Аккорды	✓	X
Мелодия	✓	✓

Последняя часть генератора MuseGAN — генератор тактов. Давайте посмотрим, как можно использовать его для объединения результатов, выдаваемых в виде векторов аккордов, стиля, мелодии и дорожек.

Генератор тактов

Генератор тактов получает четыре скрытых вектора — аккордов, стиля, мелодии и дорожек. Они объединяются для получения входного вектора длиной $4 * Z_DIM$. Выходные данные имеют вид перфорированной ленты для одного такта и одной дорожки, то есть представлены тензором с формой $[1, N_STEPS_PER_BAR, N_PITCHES, 1]$.

Генератор тактов — это нейронная сеть, которая использует слои обратной свертки для увеличения размерностей, соответствующих времени и нотам. Мы создадим для каждой партии свой генератор тактов, и все генераторы будут иметь свои веса. Код в примере 11.7 показывает, как с помощью Keras построить генератор тактов.

Пример 11.7. Конструирование генератора тактов

```
def BarGenerator(self):
```

```
    input_layer = layers.Input(shape=(Z_DIM * 4,), name='bar_generator_input') ❶

    x = layers.Dense(1024)(input_layer) ❷
    x = layers.BatchNormalization(momentum = 0.9)(x)
    x = layers.Activation('relu')(x)
    x = layers.Reshape([2,1,512])(x)

    x = conv_t(x, f=512, k=(2,1), s=(2,1), a= 'relu', p = 'same', bn = True) ❸
    x = conv_t(x, f=256, k=(2,1), s=(2,1), a= 'relu', p = 'same', bn = True)
    x = conv_t(x, f=256, k=(2,1), s=(2,1), a= 'relu', p = 'same', bn = True)
    x = conv_t(x, f=256, k=(1,7), s=(1,7), a= 'relu', p = 'same', bn = True) ❹
    x = conv_t(x, f=1, k=(1,12), s=(1,12), a= 'tanh', p = 'same', bn = False) ❺
```

```
output_layer = layers.Reshape([1, N_STEPS_PER_BAR, N_PITCHES, 1])(x) ❸

return models.Model(input_layer, output_layer)
```

- ❶ На вход генератора тактов подается вектор длиной $4 * Z_DIM$.
- ❷ Пропустив тензор через слой `Dense`, мы подготавливаем его к операции обратной свертки, изменяя форму.
- ❸ Сначала тензор расширяется по оси временных шагов...
- ❹ ...А затем по оси нот.
- ❺ В последнем слое применяется функция активации `tanh`, потому что далее для обучения сети используется механизм WGAN-GP, требующий активации `tanh`.
- ❻ Добавлением двух измерений размером 1 изменяется форма тензора, чтобы он был готов к объединению с другими тактами и партиями.

Объединяем все вместе

В целом генератор MuseGAN принимает четыре входных тензора (аккорды, стиль, мелодия и дорожки) и преобразует их в партитуру из нескольких тактов и партий. В примере 11.8 показан код, создающий генератор MuseGAN.

Пример 11.8. Конструирование генератора MuseGAN

```
def Generator():
    chords_input = layers.Input(shape=(Z_DIM,), name='chords_input') ❶
    style_input = layers.Input(shape=(Z_DIM,), name='style_input')
    melody_input = layers.Input(shape=(N_TRACKS, Z_DIM), name='melody_input')
    groove_input = layers.Input(shape=(N_TRACKS, Z_DIM), name='groove_input')

    chords_tempNetwork = TemporalNetwork() ❷
    chords_over_time = chords_tempNetwork(chords_input)

    melody_over_time = [None] * N_TRACKS
    melody_tempNetwork = [None] * N_TRACKS
    for track in range(N_TRACKS):
        melody_tempNetwork[track] = TemporalNetwork() ❸
        melody_track = layers.Lambda(lambda x, track = track: x[:,track,:])(
            melody_input
        )
        melody_over_time[track] = melody_tempNetwork[track](melody_track)

    barGen = [None] * N_TRACKS
    for track in range(N_TRACKS):
        barGen[track] = BarGenerator() ❹
```

```

bars_output = [None] * N_BARS
c = [None] * N_BARS
for bar in range(N_BARS): ❸
    track_output = [None] * N_TRACKS

    c[bar] = layers.Lambda(lambda x, bar = bar: x[:,bar,:])(chords_over_time)
    s = style_input

    for track in range(N_TRACKS):

        m = layers.Lambda(lambda x, bar = bar: x[:,bar,:])(
            melody_over_time[track]
        )
        g = layers.Lambda(lambda x, track = track: x[:,track,:])(
            groove_input
        )

        z_input = layers.Concatenate(
            axis = 1, name = 'total_input_bar_{}_track_{}'.format(bar, track)
        )([c[bar],s,m,g])

        track_output[track] = barGen[track](z_input)

    bars_output[bar] = layers.Concatenate(axis = -1)(track_output)

generator_output = layers.Concatenate(axis = 1, name = 'concat_bars')(
    bars_output
) ❹

return models.Model(
    [chords_input, style_input, melody_input, groove_input], generator_output
) ❺

generator = Generator()

```

- ❶ Определение входных слоев генератора.
- ❷ Передать входной тензор аккордов через промежуточную сеть.
- ❸ Передать входной тензор мелодии через промежуточную сеть.
- ❹ Создать независимую сеть генератора тактов для каждой партии.
- ❺ Выполнить обход партий и тактов и создать генерируемый такт для каждой комбинации.
- ❻ Объединить все вместе и сформировать общий выходной тензор.
- ❼ Модель MuseGAN принимает четыре тензора с шумом и выводит сгенерированную партитуру.

Критик MuseGAN

По сравнению с генератором критик имеет более простую архитектуру, что довольно характерно для сетей GAN. Он пытается отличить партитуры, созданные генератором, от фрагментов настоящих хоралов Баха. Это сверточная нейронная сеть, состоящая в основном из слоев Conv3D, которые свертывают партитуру в один выходной прогноз.



Слой Conv3D

До сих пор мы работали только со слоями Conv2D, применяя их к изображениям, имеющим три измерения: ширину, высоту и каналы цвета. Здесь мы должны использовать слой Conv3D. Они подобны слоям Conv2D, но принимают четырехмерные тензоры (N_BARS, N_STEPS_PER_BAR, N_PITCHES, N_TRACKS).

Кроме того, мы не применяем слой пакетной нормализации в критике, потому что для обучения GAN будем задействовать инфраструктуру WGAN-GP, которая запрещает нормализацию.

В примере 11.9 показан код, конструирующий критика с помощью Keras.

Пример 11.9. Конструирование критика MuseGAN

```
def conv(x, f, k, s, p):
    x = layers.Conv3D(filters = f
                      , kernel_size = k
                      , padding = p
                      , strides = s
                      , kernel_initializer = initializer
                      )(x)
    x = layers.LeakyReLU()(x)
    return x

def Critic():
    critic_input = layers.Input(
        shape=(N_BARS, N_STEPS_PER_BAR, N_PITCHES, N_TRACKS),
        name='critic_input'
    ) ❶

    x = critic_input
    x = conv(x, f=128, k = (2,1,1), s = (1,1,1), p = 'valid') ❷
    x = conv(x, f=128, k = (N_BARS - 1,1,1), s = (1,1,1), p = 'valid')

    x = conv(x, f=128, k = (1,1,12), s = (1,1,12), p = 'same') ❸
    x = conv(x, f=128, k = (1,1,7), s = (1,1,7), p = 'same')
```



```

x = conv(x, f=128, k = (1,2,1), s = (1,2,1), p = 'same') ❹
x = conv(x, f=128, k = (1,2,1), s = (1,2,1), p = 'same')
x = conv(x, f=256, k = (1,4,1), s = (1,2,1), p = 'same')
x = conv(x, f=512, k = (1,3,1), s = (1,2,1), p = 'same')

x = layers.Flatten()(x)

x = layers.Dense(1024, kernel_initializer = initializer)(x)
x = layers.LeakyReLU()(x)

critic_output = layers.Dense(
    1, activation=None, kernel_initializer = initializer
)(x) ❺

return models.Model(critic_input, critic_output)

critic = Critic()

```

❶ На вход критика подается массив партитуры с формой `[N_BARS, N_STEPS_PER_BAR, N_PITCHES, N_TRACKS]`.

❷ Сначала выполняется свертка тензора вдоль оси тактов. Для работы с четырехмерными тензорами используем слой `Conv3D`.

❸ Далее выполняется свертка тензора вдоль оси нот.

❹ Наконец, выполняется свертка тензора вдоль оси временных шагов.

❺ Результат формируется слоем `Dense` с единственным узлом без функции активации, как того требует механизм WGAN-GP.

Анализ сети MuseGAN

Можете немного поэкспериментировать со своей сетью MuseGAN, генерируя партитуры и затем настраивая некоторые параметры входного шума, чтобы увидеть, как они влияют на результат.

Результатом работы генератора является массив значений в диапазоне $[-1, 1]$ (из-за функции активации $\tanh$ в последнем слое). Чтобы преобразовать его в единственную ноту в каждой партии, для каждого временного шага выбирается нота с максимальным значением из всех 84 нот. В оригинальной статье с описанием MuseGAN авторы используют пороговое значение 0, поскольку каждая партия может содержать несколько нот, но в случае с хорами Баха можно просто взять максимальное значение, чтобы гарантировать ровно одну ноту на каждом шаге в каждой партии.

На рис. 11.16 показана партитура, сгенерированная моделью из векторов со случайным нормально распределенным шумом (*вверху слева*). Мы можем найти ближайшую (по евклидову расстоянию) партитуру в наборе данных и убедиться в том, что сгенерированная партитура не является копией музыкальной пьесы, существующей в наборе данных, — ближайшая партитура показана чуть ниже, и, как нетрудно заметить, она не похожа на сгенерированную партитуру.

Сгенерированная партитура 	Изменение входного тензора стиля 
Ближайшая реальная партитура 	Изменение входного тензора с мелодией, только для первой (верхней) партии 
Изменение входного тензора аккордов 	Изменение входного тензора дорожек, только для последней (нижней) партии 

Рис. 11.16. Партитура, сгенерированная сетью MuseGAN, ближайшая к ней реальная партитура из обучающих данных и примеры влияния попыток изменить входной шум на сгенерированную партитуру

Теперь поэкспериментируем с входным шумом, чтобы улучшить сгенерированную партитуру. Для начала изменим вектор шума (см. рис. 11.16, *внизу*). Все партии изменились, как и ожидалось, и теперь такты имеют разные свойства. Во втором такте базовая партия более динамична, а верхняя имеет более высокую тональность, чем в первом такте. Это связано с тем, что скрытые векторы, влияющие на два такта, различны, поскольку входной вектор аккордов прошел через промежуточную сеть.

При изменении стиля (*вверху справа*) оба такта меняются одинаково. Они не сильно различаются по стилю, но весь фрагмент изменился по сравнению с первоначально сгенерированной партитурой (то есть для настройки всех дорожек и тактов используется один и тот же скрытый вектор).

Мы можем также изменять отдельные партии через входные тензоры мелодии и дорожек. На рис. 11.16 можно видеть эффект изменения входного шума в начальном тензоре мелодии только для верхней музыкальной партии. Она изменилась довольно существенно, а все прочие остались неизменными. Можно заметить и изменение ритма между тактами в верхней партии: второй такт получился более динамичным — он содержит более короткие ноты, чем первый.

Наконец, *внизу справа* на рис. 11.16 показана партитура, сгенерированная после изменения входного тензора дорожек только для нижней партии. И снова изменилась только она, все остальные части неизменны. Более того, в целом такты в нижней партии сохранили сходство между собой, как и следовало ожидать.

Этот пример показывает, как, изменяя входные параметры, можно влиять непосредственно на высокоуровневые признаки сгенерированной музыкальной последовательности. Все происходит почти так же, как в предыдущих главах, где, регулируя скрытые векторы в вариационных автокодировщиках и генеративно-состязательных сетях, мы изменяли внешний вид сгенерированных образов. Один из недостатков модели — необходимость заранее определить количество генерируемых тактов. Чтобы избавиться от него, авторы предложили расширение, реализующее передачу предыдущих тактов на вход, что позволяет модели генерировать длинные партитуры за счет непрерывной передачи самых последних сгенерированных тактов обратно в модель в качестве дополнительных входных данных.

Резюме

Итак, мы рассмотрели два типа моделей для генерирования музыки: трансформер и MuseGAN. Своим дизайном трансформер напоминает сети для генерирования текста (см. главу 9). Музыка и текст имеют много общего, и часто для их генерирования можно использовать одинаковые методы. Мы расширили модель трансформера, добавив в нее входные и выходные потоки нот и их длительностей, и увидели, как модель смогла выявить такие понятия, как октавы и тональности, научившись точно имитировать музыку Баха.

Затем мы посмотрели, как адаптировать процесс кодирования для создания полифонической музыки. Кодирование сеткой создает представление партитуры

в виде перфорированной ленты для механического пианино, позволяя обучать трансформер на одной последовательности кодов, описывающих каждую ноту в каждой партии через дискретные, равноотстоящие друг от друга интервалы времени. Кодирование на основе событий позволяет описать сразу несколько музыкальных партий с помощью единого потока инструкций. Оба метода имеют свои преимущества и недостатки: успех или неудача подхода к созданию музыки на основе модели трансформера во многом зависит от выбора метода кодирования.

Вы также узнали, что для генерирования последовательных данных не обязательно применять последовательный подход — сеть MuseGAN создает полифонические музыкальные партитуры с несколькими партиями, используя сверточные слои и обрабатывая партитуру как своеобразное изображение, в котором партии играют роль каналов изображения. Новизна MuseGAN заключается в том, что четыре входных тензора с шумом (аккорды, стиль, мелодия и дорожки) организованы так, чтобы с их помощью можно было управлять высокоуровневыми свойствами музыки. Да, базовая гармония все еще не так совершенна или разнообразна, как в произведениях Баха, но эта сеть является хорошей попыткой выполнить чрезвычайно сложную задачу и подчеркивает способность генеративно-сопоставительных сетей решать широкий спектр задач.

Модели мира

В этой главе вы:

- познакомитесь с основами обучения с подкреплением (Reinforcement Learning, RL);
- узнаете, как можно использовать генеративное моделирование в рамках подхода *модели мира* к обучению с подкреплением;
- увидите, как обучить вариационный автокодировщик (Variational AutoEncoder, VAE) для сбора данных об окружающей среде в низкоразмерном скрытом пространстве;
- исследуете процесс обучения рекуррентной нейронной сети с полносвязной сетью смеси распределений (MDN-RNN), предсказывающей скрытую переменную;
- познакомитесь с эволюционной стратегией с адаптацией ковариационной матрицы (CMA-ES) для обучения контроллера, способного выполнять осмысленные действия в окружающей среде;
- узнаете, что обученная сеть MDN-RNN сама может использоваться в качестве окружающей среды, позволяющей агенту обучать контроллер на собственных галлюцинациях, а не в реальной среде.

В этой главе представлено одно из наиболее интересных применений генеративных моделей, а именно их использование в так называемых моделях мира.

Введение

В марте 2018 года Дэвид Ха (David Ha) и Юрген Шмидхубер (Jürgen Schmidhuber) опубликовали статью *World Models* [Ha, Schmidhuber, 2018]. В ней они показали, как обучить модель, способную выполнять определенную задачу, посредством экспериментов в своих генеративных представлениях, а не в самом окружении. Это отличный пример использования генеративного моделирования для решения практических задач, когда это происходит в дополнение к другим методам машинного обучения, в том числе таким, как обучение с подкреплением.

Ключевым компонентом архитектуры является генеративная модель, способная вычислить распределение вероятностей для следующего возможного состояния

с учетом текущего состояния и предполагаемого действия. Получив представление о физике окружающей среды с помощью случайных движений, модель может затем научиться решать совершенно новые задачи, действуя исключительно в рамках своего внутреннего представления об окружающей среде. Этот подход позволил получить лучшие результаты в обеих задачах, на которых он был протестирован.

В этой главе мы подробно исследуем модель, представленную в статье, уделив особое внимание задаче обучения агента управлять машиной на виртуальной гоночной трассе. В качестве окружающей среды используем двумерную компьютерную модель, но это не значит, что данный метод нельзя применить к сценариям реального мира, когда тестирование в реальной обстановке стоит слишком дорого или в принципе невозможно.



В этой главе я буду ссылаться на реализацию модели на основе TensorFlow из статьи *World Models*, доступной на GitHub (https://oreil.ly/_OIJX), которую я советую клонировать и запустить самостоятельно!

Прежде чем приступить к созданию модели, рассмотрим более подробно идею обучения с подкреплением.

Обучение с подкреплением

Обучение с подкреплением можно определить так:

Обучение с подкреплением (Reinforcement Learning, RL) — область машинного обучения, целью которой является обучение агента для оптимального решения конкретных задач в конкретной среде.

В отличие от дискриминативного и генеративного моделирования, направленных на минимизацию функции потерь по набору наблюдений, обучение с подкреплением стремится максимизировать долгосрочное вознаграждение агента в данном окружении. Этот подход часто описывают как одну из трех основных ветвей машинного обучения наряду с *обучением с учителем* (прогнозирование с использованием маркированных данных) и *обучением без учителя* (обучение по немаркированным данным).

Для начала познакомимся с некоторыми ключевыми терминами, относящимися к обучению с подкреплением.

Среда (окружение)

Мир, в котором действует агент. Характеристики мира определяются набором правил, которые управляют процессом обновления состояния игры и распределением вознаграждений с учетом предыдущих действий агента и текущего состояния игры. Например, для алгоритма обучения

с подкреплением, изучающего игру в шахматы, среда будет характеризоваться правилами, регламентирующими, как данное действие (например, ход е2-е4) влияет на следующее состояние игры (новое расположение фигур на доске), как определить, является ли данная позиция матом, и как назначить победившему игроку вознаграждение 1 после победного хода.

Агент

Сущность, предпринимающая действия в реально складывающейся или смоделированной обстановке.

Состояние игры

Данные, описывающие конкретную ситуацию, с которой может столкнуться агент (иногда называется просто *состоянием*), например конкретное расположение фигур на шахматной доске с дополнительной игровой информацией — в частности, о том, кому принадлежит право следующего хода.

Действие

Ход, который может сделать агент.

Вознаграждение

Ценность, возвращаемая агенту средой после выполнения действия. Агент стремится максимизировать долгосрочную сумму своих вознаграждений. Например, в шахматах мат королю противника позволяет получить вознаграждение 1, а за каждый следующий ход вознаграждение равно 0. В других играх вознаграждения присуждаются постоянно на протяжении всего эпизода (например, очки в игре «Космические захватчики»).

Эпизод

Один сеанс действий агента в окружении, иногда называемый также *эпизодом*.

Временной шаг

В дискретном окружении все состояния, действия и вознаграждения относятся к конкретному временному шагу t .

Взаимосвязи между этими понятиями показаны на рис. 12.1.

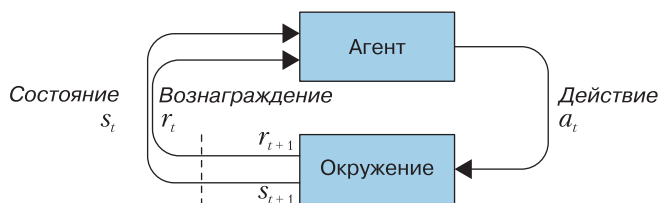


Рис. 12.1. Диаграмма работы алгоритма обучения с подкреплением

Среда инициализируется с текущим состоянием игры s_0 . На временном шаге t агент получает текущее состояние игры s_t и использует его для выбора следующего оптимального действия, которое затем выполняет. После действия агента среда вычисляет следующее состояние s_{t+1} и величину вознаграждения r_{t+1} , возвращая их агенту, чтобы тот мог начать новый цикл. Так продолжается до тех пор, пока не будет достигнут критерий окончания эпизода (например, заданное число временных шагов или победа/поражение агента).

Как заставить агента максимизировать сумму вознаграждений в данном окружении? Можно определить набор правил, руководствуясь которыми агент должен реагировать на то или иное состояние игры. Однако этот подход быстро становится неосуществимым по мере усложнения среды — мы не сможем создать агента, обладающего сверхчеловеческими способностями в конкретной задаче, жестко задавая правила. Обучение с подкреплением предусматривает создание агента, который может самостоятельно выявлять оптимальные стратегии в сложных условиях, многократно повторяя игру.

А теперь познакомимся со средой CarRacing, которую мы используем для имитации движения по гоночной трассе.

CarRacing

CarRacing — это окружение (среда), входящее в пакет Gymnasium (<https://gymnasium.farama.org/>). Gymnasium — это набор инструментов для разработки алгоритмов обучения с подкреплением, доступный в виде библиотеки для Python. В состав библиотеки входит несколько классических окружений для обучения с подкреплением, например CartPole и Pong, а также окружения, представляющие более сложные задачи, такие как обучение агента ходьбе по неровной местности или выигрышу в игре Atari.



Gymnasium

Gymnasium — это версия библиотеки OpenAI Gym. С 2021 года развитие библиотеки Gym взял на себя проект Gymnasium, поэтому в книге мы будем называть окружения Gymnasium окружениями Gym.

Все окружения предлагают метод `step`, вызовом которого можно сообщить о выполненном действии, в ответ окружение вернет следующее состояние и вознаграждение. Многократно вызывая метод `step` с действиями, выбранными агентом, можно воспроизвести эпизод в окружении. Существует также метод `reset` для возврата среды в исходное состояние и метод `render`, позволяющий наблюдать за работой агента в данном окружении. Это может пригодиться при отладке и поиске направлений для дальнейшего совершенствования агента.

Посмотрим, как определяются состояние игры, действия, вознаграждение и эпизод в окружении CarRacing.

Состояние игры

Изображение 64×64 пиксела в формате RGB, показывающее вид сверху на трассу и автомобиль.

Действие

Набор из трех значений: направление (угол поворота руля, от -1 до 1), ускорение (от 0 до 1) и торможение (от 0 до 1). На каждом временном шаге агент должен установить все три значения.

Вознаграждение

Отрицательный штраф $-0,1$ за каждый временной шаг и положительное вознаграждение $1000/N$ за пересечение границы нового фрагмента трека, где N — общее количество фрагментов, составляющих трек.

Эпизод

Эпизод завершается при пересечении автомобилем финишной черты или по прошествии 3000 временных шагов.

На рис. 12.2 изображено графическое представление состояния игры.

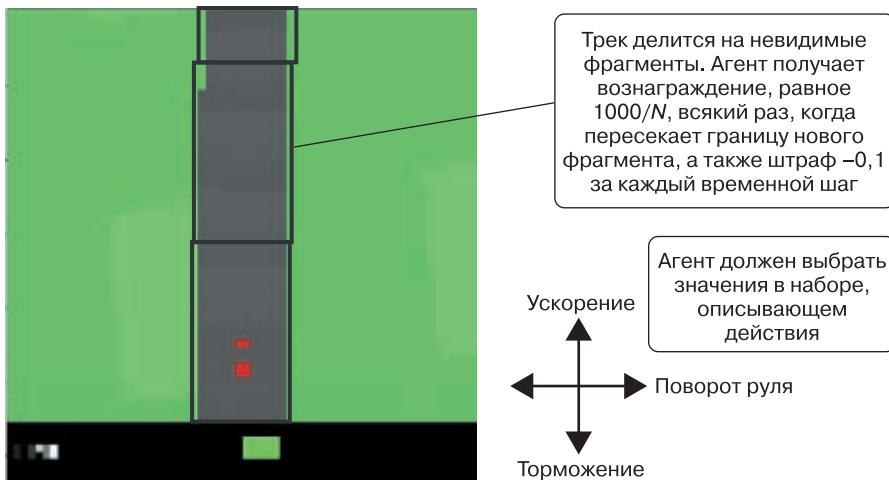


Рис. 12.2. Графическое представление одного состояния игры в окружении CarRacing



Перспектива

Представляйте, что агент парит над трассой и управляет машиной с высоты птичьего полета, а не наблюдает за трассой с водительского места.

Обзор модели мира

Теперь в общих чертах рассмотрим архитектуру модели мира и процесс обучения, а потом перейдем к подробному описанию этапов создания каждого компонента.

Архитектура

Решение состоит из трех основных компонентов (рис. 12.3), обучаемых независимо:

- V — вариационный автокодировщик (VAE);
- M — рекуррентная нейронная сеть с полносвязной сетью смеси распределений (MDN-RNN);
- C — контроллер.

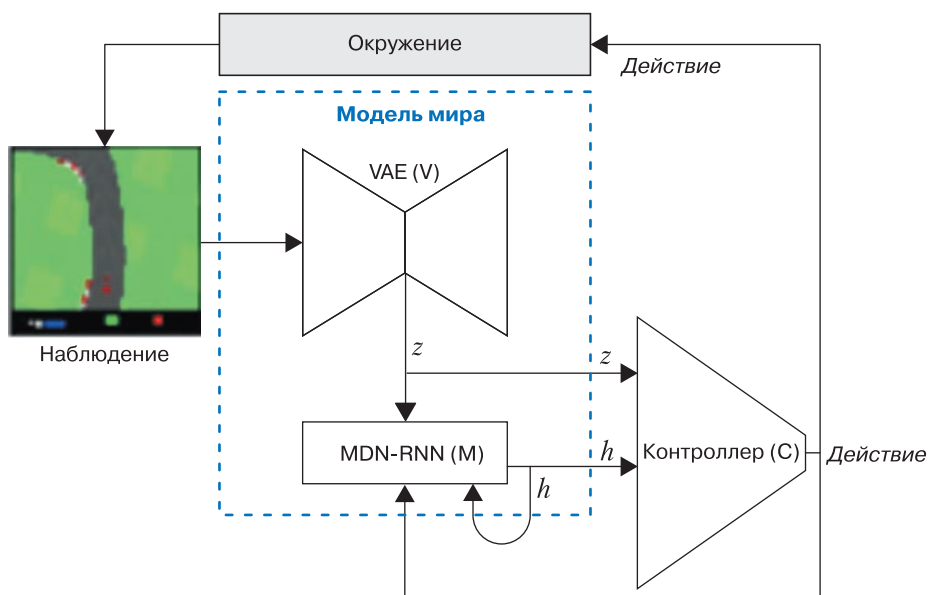


Рис. 12.3. Архитектура модели мира

Вариационный автокодировщик

Принимая решения во время вождения, никто из нас не анализирует каждый видимый пиксел — вместо этого мы уплотняем визуальную информацию по нескольким параметрам: прямолинейность дороги, предстоящие повороты и положение автомобиля на дороге, что помогает выбрать следующее действие.

В главе 3 мы видели, как вариационный автокодировщик принимает на входе изображение большой размерности и путем минимизации ошибки восстановления и расстояния Кульбака — Лейблера (Kullback — Leibler, KL) сжимает его в некоторую случайную переменную в скрытом пространстве, приблизительно соответствующую стандартному многомерному нормальному распределению. Это гарантирует непрерывность скрытого пространства и возможность легко делать выборки из него, чтобы генерировать новые значимые наблюдения.

В примере автомобильных гонок вариационный автокодировщик сжимает входное изображение $64 \times 64 \times 3$ (RGB) в трехмерную нормально распределенную случайную величину, параметризованную двумя переменными, μ и $\log \text{var}$, где $\log \text{var}$ — это логарифм дисперсии распределения. Мы можем сделать выборку из распределения, чтобы получить скрытый вектор z , представляющий текущее состояние. Этот вектор передается следующему компоненту сети MDN-RNN.

Сеть MDN-RNN

Во время вождения каждое следующее наблюдение не является для нас полной неожиданностью. Если текущее наблюдение показывает впереди левый поворот, то, поворачивая руль влево, мы ожидаем, что к моменту следующего наблюдения все еще будем находиться на дороге. Если бы это было не так, мы ездили бы по дорогам, виляя из стороны в сторону, так как не замечали бы небольшого отклонения от центра на следующем временном шаге, и затем оказываясь в положении, требующем немедленных и активных действий.

Эта предусмотрительность — зона ответственности сети MDN-RNN, которая пытается предсказать распределение следующего скрытого состояния на основе предыдущего скрытого состояния и предыдущего действия. В частности, MDN-RNN — это слой LSTM с 256 скрытыми узлами, за которым следует выходной слой сети смеси распределений (Mixture Density Network, MDN), который учитывает тот факт, что следующее скрытое состояние фактически можно получить из любого из нескольких нормальных распределений.

Тот же метод применялся одним из авторов статьи *World Models*, Дэвидом Ха (David Ha), к задаче генерации рукописного текста (<https://oreil.ly/WmPGp>), чтобы описать, что далее перо может двигаться в сторону любой из красных областей (рис. 12.4).

В примере с гоночным автомобилем мы допускаем, что каждый элемент следующего наблюдаемого скрытого состояния может быть получен из любого из пяти нормальных распределений.

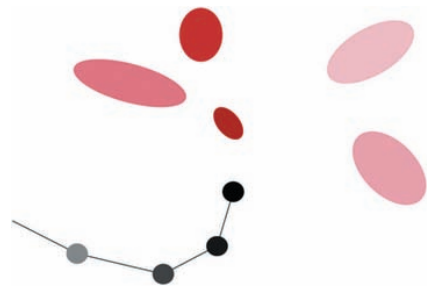


Рис. 12.4. Генерация рукописного текста с помощью MDN

Контроллер

До этого момента мы вообще не упоминали выбор действия. Эта ответственность лежит на контроллере, реализованном как полносвязная нейронная сеть, на вход которой подается конкатенация вектора z (текущего скрытого состояния, выбранного из распределения, закодированного вариационным автокодировщиком) и скрытого состояния сети RNN. Три выходных нейрона соответствуют трем действиям — повороту, ускорению, торможению — и масштабируются, чтобы попасть в соответствующие диапазоны.

Мы должны обучить контроллер, используя метод обучения с подкреплением, потому что у нас нет обучающего набора данных, который помогал бы понять, какое действие *хорошее*, а какое *плохое*. Агент должен сам определить это путем повторных экспериментов.

Как вы увидите далее, суть статьи *World Models* заключается в демонстрации того, как обучение с подкреплением может происходить внутри генеративной модели окружения в агенте, а не в окружении Gym. Иными словами, это происходит не в реальном окружении, а в его версии, *вымышленной* агентом. Чтобы понять, какова роль каждого из этих трех компонентов и как они работают вместе, вообразим себе следующий диалог:

VAE (просматривая последнее наблюдение $64 \times 64 \times 3$): Выглядит как прямая дорога с пологим левым поворотом, к которому приближается автомобиль, следующий в направлении дороги (z).

RNN: Исходя из этого описания (z) и того факта, что на предыдущем шаге контроллер решил сильно ускориться ($action$), я обновлю свое скрытое состояние, и в следующем наблюдении дорога по-прежнему останется прямой, но с чуть большим левым поворотом впереди.

Контроллер: Исходя из описания, полученного от VAE (z) и текущего скрытого состояния RNN (h), моя нейронная сеть предсказывает действие $[0.34, 0.8, 0]$.

Затем действие, сгенерированное контроллером, передается в окружение, возвращающее обновленное наблюдение, и цикл начинается снова.

Обучение

Процесс обучения включает пять этапов.

1. Сбор данных в ходе случайных прогонов. Здесь перед агентом не стоит цель решить задачу — он просто исследует окружение случайным образом. На этом этапе имитируются несколько эпизодов и на каждом временном шаге сохраняются наблюдаемое состояние, действие и вознаграждение. Смысл в том,

чтобы получить данные о работе окружения, которые затем VAE сможет изучить и научиться представлять состояния в виде векторов в скрытом пространстве. Затем MDN-RNN сможет узнать, как скрытые векторы развиваются с течением времени.

2. Обучение VAE. Используя данные, собранные в ходе случайных прогонов, обучим VAE на наблюдаемых изображениях.
3. Сбор данных для обучения MDN-RNN. После обучения VAE применяем его для кодирования каждого из собранных наблюдений в векторы `mu` и `logvar`, сохранив их вместе с текущим действием и вознаграждением.
4. Обучение MDN-RNN. Возьмем пакеты эпизодов и на каждом временном шаге загрузим переменные `mu`, `logvar`, `action` и `reward`, которые были сгенерированы на шаге 3. Затем выберем вектор `z` из векторов `mu` и `logvar`. Далее на текущем векторе `z`, `action` и `reward` обучим MDN-RNN предсказывать следующий вектор `z` и `reward`.
5. Обучение контроллера. Используя подготовленные VAE и RNN, обучим контроллер предсказывать действие для текущего `z` и скрытого `h` состояний сети RNN. В качестве оптимизатора контроллер применяет эволюционный алгоритм CMA-ES (Covariance Matrix Adaptation Evolution Strategy — эволюционная стратегия с адаптацией матрицы ковариаций). Он поощряет веса матрицы, которые генерируют действия, приводящие к более высокому общему вознаграждению, поэтому такое желаемое поведение почти наверняка будет унаследовано будущими поколениями.

А теперь подробно рассмотрим каждый этап.

Сбор данных в ходе случайных прогонов

Первый шаг — сбор данных с помощью агента, выполняющего случайные действия. Это может показаться странным, учитывая, что наша конечная цель — научить агента совершать разумные действия, но этот шаг обеспечит данные, опираясь на которые агент будет изучать устройство мира и узнавать, как его действия (пусть поначалу случайные) влияют на последующие наблюдения.

Мы можем исследовать сразу несколько эпизодов, запуская несколько процессов Python, каждый из которых исследует свой экземпляр среды. Каждый процесс будет выполняться на отдельном ядре, поэтому чем больше ядер имеет процессор вашего компьютера, тем быстрее вы сможете собирать данные.

На этом шаге используются следующие гиперпараметры:

- `parallel_processes` — число одновременно запускаемых процессов для обработки (например, восемь для компьютера с восьмиядерным процессором);

- `max_trials` — количество эпизодов, которое должен запустить каждый процесс (например, 125, чтобы восемь процессов в общей сложности создали 1000 эпизодов);
- `max_frames` — максимальное число временных шагов в одном эпизоде (например, 300).

На рис. 12.5 показаны кадры 40–59 из одного эпизода, когда машина приближается к повороту, а также случайно выбранные действия и вознаграждения. Обратите внимание на то, что вознаграждение увеличивается до 3,22, когда машина пересекает границу нового фрагмента трека, а в остальных случаях взывается штраф $-0,1$.

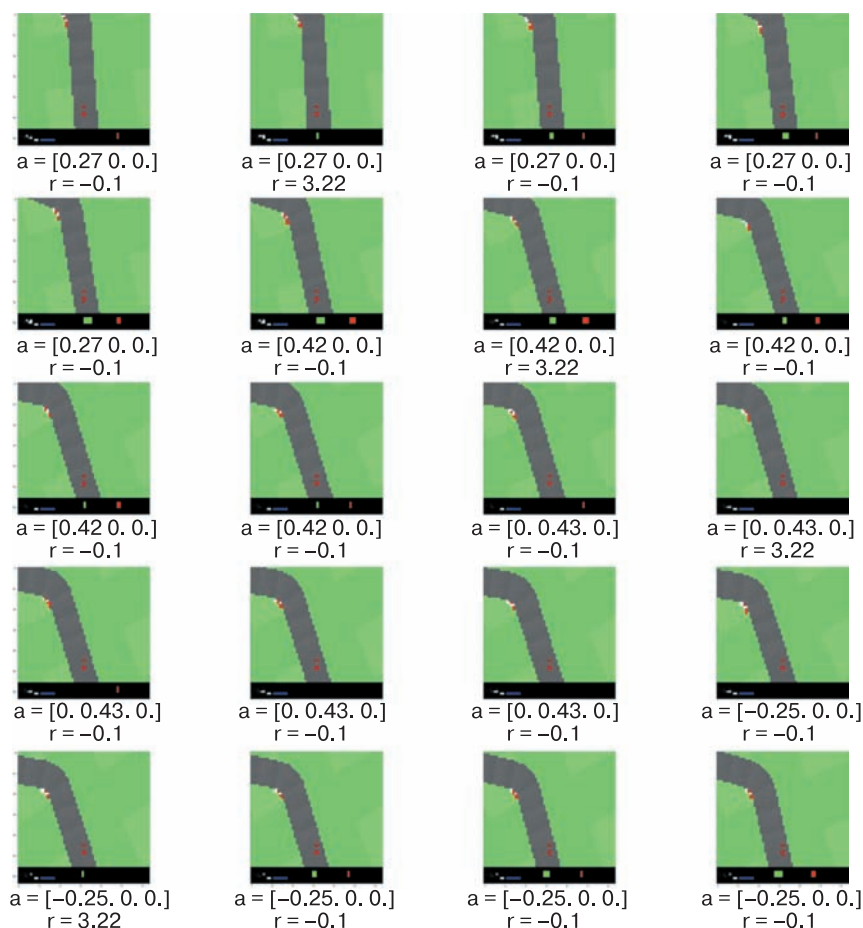


Рис. 12.5. Кадры 40–59 из одного эпизода

Обучение VAE

Теперь на основе собранных данных можно построить генеративную модель (вариационный автокодировщик VAE). Напомню: цель VAE — дать возможность свернуть одно изображение размером $64 \times 64 \times 3$ в нормально распределенную случайную переменную z , распределение которой параметризовано двумя векторами, μ и $\log\text{var}$. Каждый из них имеет длину 32. На этом шаге используются следующие гиперпараметры:

- `vae_batch_size` — размер пакетов в обучающем наборе для VAE (количество наблюдений на пакет, например, 100);
- `z_size` — длина скрытого вектора z (и, следовательно, переменных μ и $\log\text{var}$, например, 32);
- `vae_num_epoch` — число эпох обучения (например, 10).

Архитектура VAE

Как мы уже видели, библиотека Keras позволяет определить не только полную модель VAE для обучения, но и составляющие ее подмодели кодировщика и декодировщика по отдельности. Это может пригодиться, если вдруг появится желание закодировать конкретное изображение или декодировать некоторый вектор z . В этом примере мы определим полную модель VAE и три подмодели внутри нее:

- `vae` — полная модель для обучения. Она будет принимать изображение $64 \times 64 \times 3$ и выводить реконструированное изображение $64 \times 64 \times 3$;
- `encode` — принимает наблюдение $64 \times 64 \times 3$ и выводит вектор z . Если снова и снова передавать этой модели одно и то же наблюдение, она всякий раз будет создавать разные векторы z , используя вычисленные значения μ и $\log\text{var}$;
- `encode_mu_log_var` — принимает наблюдение $64 \times 64 \times 3$ и выводит соответствующие ему векторы μ и $\log\text{var}$. Если снова и снова передавать этой модели одно и то же наблюдение, она каждый раз будет возвращать одни и те же векторы μ и $\log\text{var}$;
- `decode` — принимает вектор z и возвращает восстановленное наблюдение $64 \times 64 \times 3$.

На рис. 12.6 представлена диаграмма, отражающая архитектуру модели VAE и ее подмоделей.

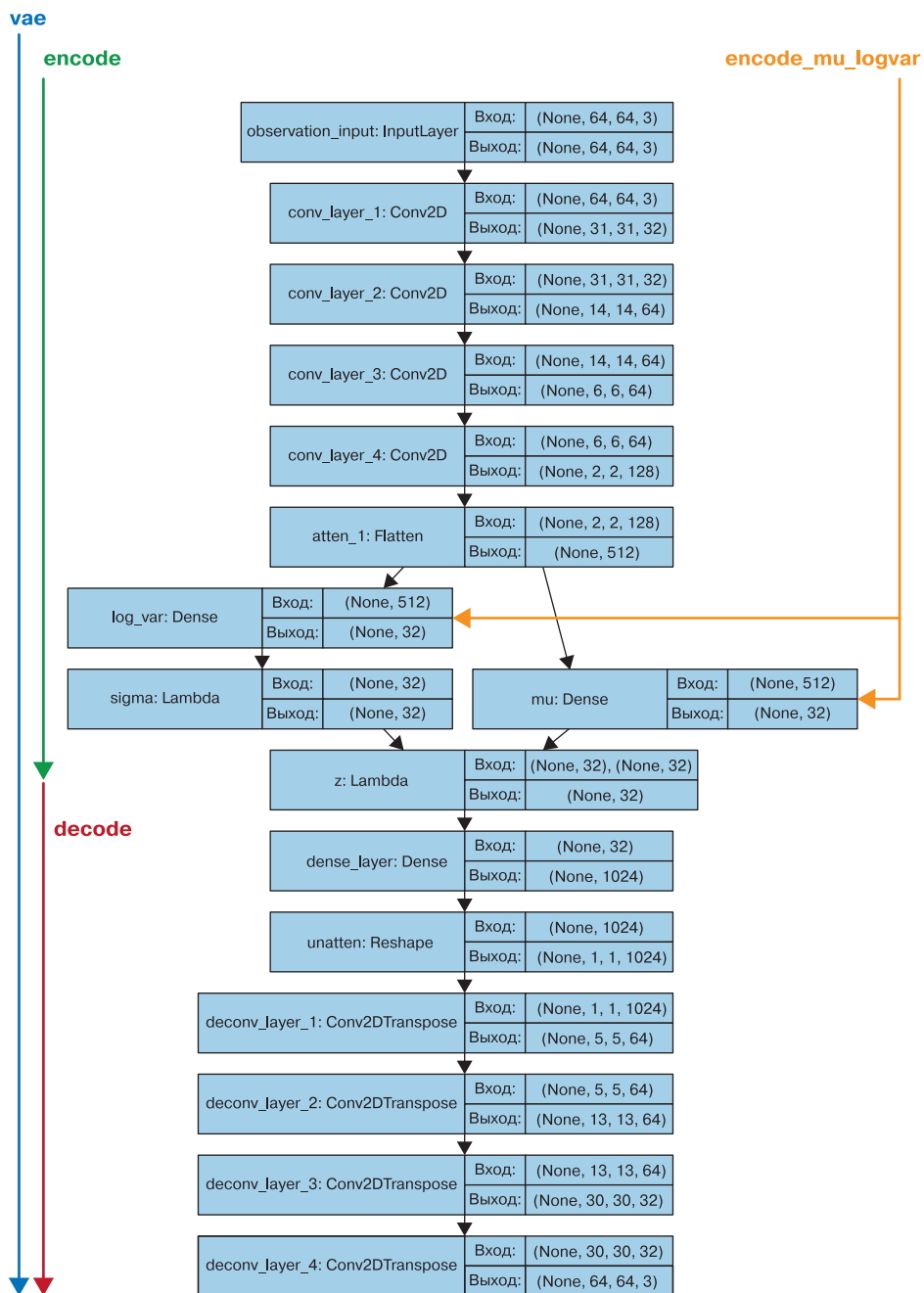


Рис. 12.6. Архитектура вариационного автокодировщика из статьи World Models

Анализ VAE

Теперь посмотрим, что возвращают сама модель VAE и ее подмодели, а затем — как можно использовать VAE для генерирования совершенно новых наблюдений трека.

Полная модель

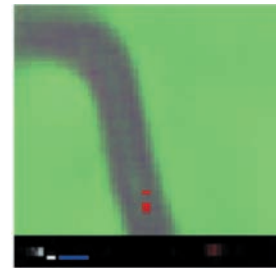
После обучения модели VAE на наблюдениях она сможет восстановить точное изображение трека (рис. 12.7). Это может пригодиться для визуальной проверки правильности ее работы.

Модели кодировщиков

После обучения модели `encode_mu_logvar` на наблюдениях она будет возвращать сгенерированные векторы `mu` и `logvar`, описывающие многомерное нормальное распределение. Модель `encode` делает еще шаг вперед и выбирает конкретный вектор `z` из этого распределения. Результаты, возвращаемые этими двумя моделями кодировщиков, показаны на рис. 12.8.



Вход



Выход

Рис. 12.7. Входные данные полной модели VAE и полученный ею результат

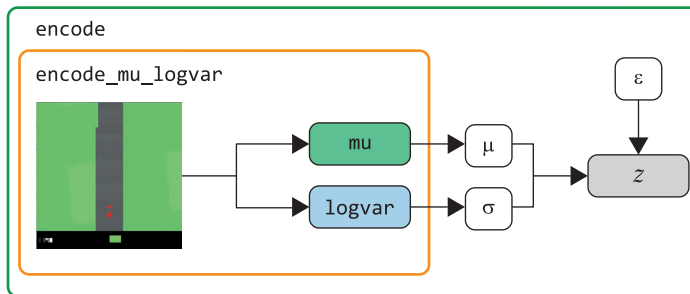


Рис. 12.8. Результаты, возвращаемые моделями кодировщиков

Скрытая переменная `z` извлекается из гауссова распределения с параметрами `mu` и `logvar` путем выборки из стандартного нормального распределения с последующим масштабированием и сдвигом выборочного вектора (пример 12.1).

Пример 12.1. Выборка z из многомерного нормального распределения с параметрами μ и $\logvar$

```
eps = tf.random_normal(shape=tf.shape(mu))
sigma = tf.exp(logvar * 0.5)
z = mu + eps * sigma
```

Модель декодировщика

Модель `decode` принимает вектор z и реконструирует исходное изображение. На рис. 12.9 показан результат линейной интерполяции двух измерений в z , позволяющий увидеть, как каждое измерение кодирует определенный аспект трека. Например, $z[4]$ определяет направление трека влево/вправо относительно автомобиля, а $z[7]$ — крутизну приближающегося левого поворота.

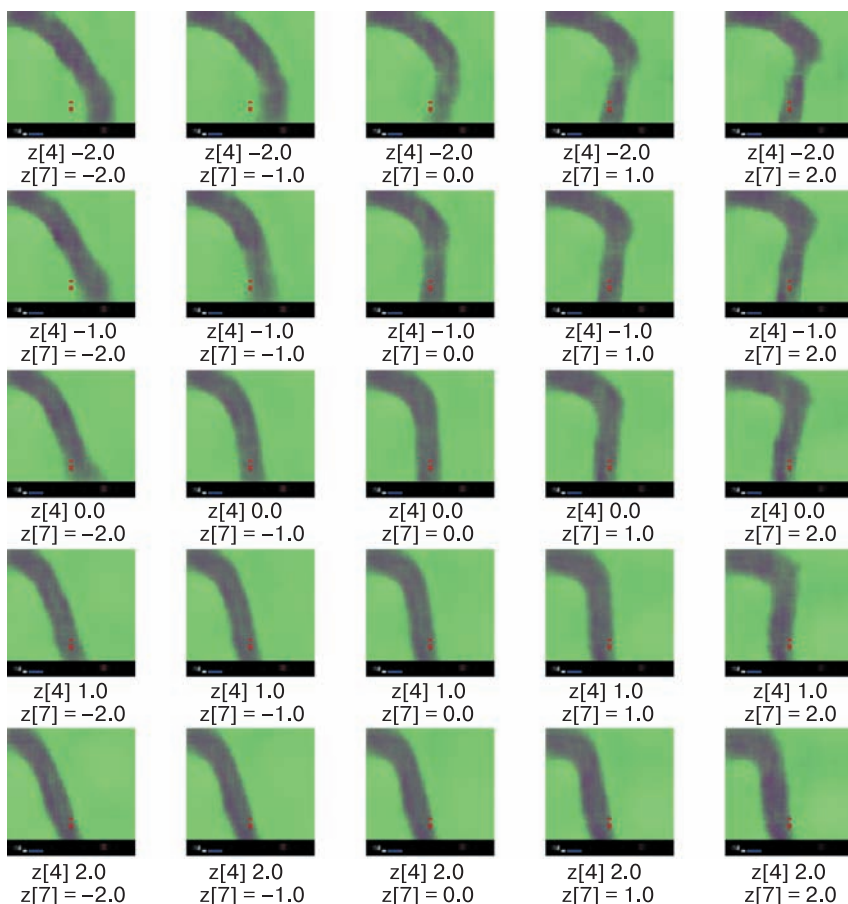


Рис. 12.9. Линейная интерполяция двух измерений в z

Это доказывает, что скрытое пространство, изученное вариационным автокодировщиком, непрерывно и может использоваться для генерирования новых фрагментов трека, которые раньше не наблюдались агентом.

Сбор данных для обучения RNN

Обученный VAE теперь можно применять для генерирования обучающих данных для сети MDN-RNN.

На этом этапе пропустим все данные случайного прогона через модель `encode_mu_logvar` и сохраним векторы `mu` и `logvar`, соответствующие каждому наблюдению. Эти закодированные данные вместе с собранными данными о действиях и вознаграждениях будут использоваться для обучения MDN-RNN. Этот процесс показан на рис. 12.10.

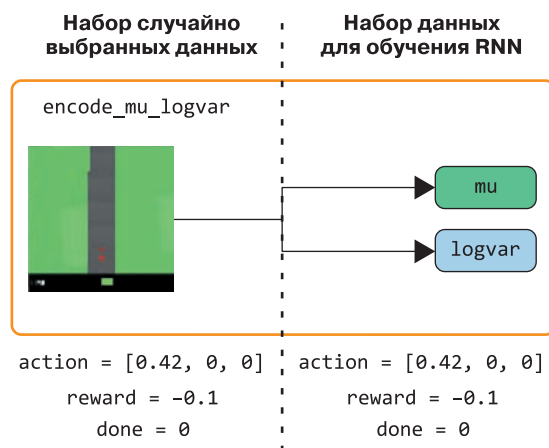


Рис. 12.10. Создание набора данных для обучения MDN-RNN

Обучение сети MDN-RNN

Теперь обучим сеть MDN-RNN предсказывать распределение следующего вектора `z` и вознаграждения, исходя из текущего значения `z`, текущего действия и предыдущего вознаграждения. После этого мы сможем использовать внутреннее скрытое состояние RNN (его можно рассматривать как текущее представление окружающей среды, сложившееся в модели) для передачи на вход контроллера, который и примет решение о том, какое следующее действие лучше всего предпринять.

Гиперпараметры этого процесса:

- `rnn_batch_size` — размер пакета, используемого при обучении MDN-RNN (количество последовательностей в пакете);
- `rnn_num_steps` — общее число итераций обучения (например, 4000).

Архитектура сети MDN-RNN

На рис. 12.11 представлена диаграмма, отражающая архитектуру сети MDN-RNN.

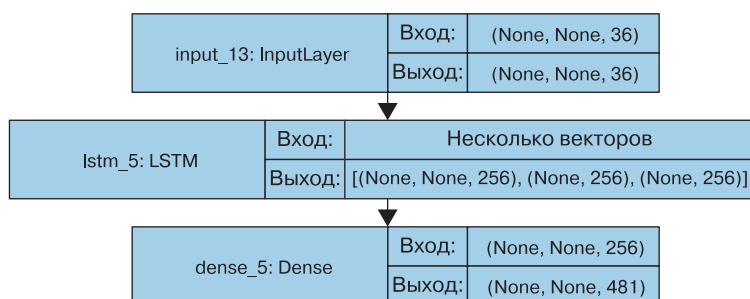


Рис. 12.11. Архитектура сети MDN-RNN

Сеть состоит из слоя LSTM (RNN), за которым следует полносвязный слой (MDN), преобразующий скрытое состояние LSTM в параметры смеси распределений. Давайте рассмотрим эту архитектуру детальнее.

На вход слоя LSTM подается вектор длиной 36 — конкатенация векторов z (длиной 32), выполненная вариационным автокодировщиком текущего действия (длиной 3) и предыдущего вознаграждения (длиной 1). На выходе слой LSTM возвращает вектор длиной 256 — по одному значению для каждой ячейки в слое LSTM. Этот вектор передается в MDN — обычный полносвязный слой, который преобразует вектор длиной 256 в вектор длиной 481.

Почему 481? На рис. 12.12 показано, как устроен вектор, получаемый на выходе из MDN-RNN. Напомню: цель сети смеси распределений — смоделировать возможность получения следующего вектора состояния z из нескольких возможных распределений с определенной вероятностью. В примере с гоночным автомобилем мы выбрали пять нормальных распределений. Сколько параметров нужно, чтобы определить эти распределения? Для каждой из пяти смесей нам нужны μ и $\log\text{var}$ (чтобы определить распределение) и вероятность выбора этой

смеси ($\log p_i$) для каждого из 32 измерений в z . То есть $5 \times 3 \times 32 = 480$ параметров. Еще один дополнительный параметр предназначен для прогнозирования вознаграждения, точнее, логарифмических коэффициентов вознаграждения на следующем временном шаге.

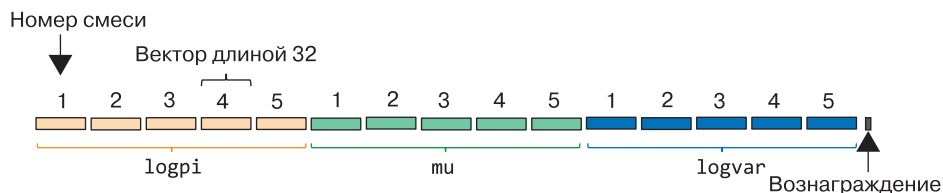


Рис. 12.12. Вектор на выходе из MDN-RNN

Выборка следующего состояния и вознаграждения из MDN-RNN

Мы можем сделать выборку из выходных данных MDN, чтобы сгенерировать прогноз для z и вознаграждения на следующем временном шаге, как описано далее.

1. Разделить 481-мерный выходной вектор на три переменные ($\log p_i$, μ , $\log var$) и значение вознаграждения.
2. Масштабировать вектор $\log p_i$ так, чтобы его можно было интерпретировать как 32 распределения вероятностей пяти индексов смесей.
3. Для каждого из 32 измерений в z сделать выборку из распределений, созданных из $\log p_i$ (то есть выбрать одно из пяти распределений для каждого измерения в z).
4. Получить значения μ и $\log var$ для этого распределения.
5. Получить выборку значений для каждого измерения z из нормального распределения, параметризованного значениями μ и $\log var$, соответствующими этим измерениям.

Функция потерь для MDN-RNN — это сумма потерь при реконструкции вектора z и вознаграждения. Потери при реконструкции вектора z — это отрицательное логарифмическое правдоподобие распределения, предсказанного MDN-RNN, с учетом истинного значения z , а потеря за вознаграждение — это средняя квадратическая ошибка между предсказанным и истинным вознаграждениями.

Обучение контроллера

Последний этап — обучение контроллера (сети, которая выбирает действие) с использованием эволюционного алгоритма СМА-ES (Covariance Matrix Adaptation Evolution Strategy — эволюционная стратегия с адаптацией матрицы ковариаций).

Гиперпараметры этого этапа:

- `controller_num_worker` — количество рабочих процессов, которые будут параллельно тестировать решения;
- `controller_num_worker_trial` — количество решений, которые каждый рабочий процесс получит для тестирования в каждом поколении;
- `controller_num_episode` — количество эпизодов, на которых будет проверяться каждое решение для вычисления среднего вознаграждения;
- `controller_eval_steps` — количество поколений между оценками текущего наилучшего набора параметров.

Архитектура контроллера

Контроллер имеет очень простую архитектуру. Это полносвязная нейронная сеть без скрытого слоя, она соединяет входной вектор непосредственно с вектором действия.

Входной вектор — это конкатенация текущего вектора φ (длиной 32) и текущего скрытого состояния LSTM (длиной 256), которая является вектором длиной 288. Поскольку каждый входной узел подключается непосредственно к трем выходным узлам, общее количество весов равно $288 \times 3 = 864 + 3$ веса смещения, то есть всего 867 весов.

Как же обучить эту сеть? Эта задача не относится к категории обучения с учителем — мы не пытаемся *предсказать* правильное действие. У нас нет обучающего набора с *правильными* действиями, так как мы не знаем, какое действие является оптимальным при данном состоянии окружения. Это отличительная черта задач обучения с подкреплением. Нам нужно, чтобы агент сам нашел оптимальные значения весов, экспериментируя и обновляя веса и опираясь на обратную связь.

Эволюционные стратегии пользуются большой популярностью в задачах обучения с подкреплением благодаря их простоте, эффективности и масштабируемости. Мы будем задействовать конкретную стратегию, известную как СМА-ES.

СМА-ES

Эволюционные стратегии обычно придерживаются такой последовательности действий.

1. Создается *популяция* агентов, и каждый инициализируется случайными значениями параметров.
2. В цикле:
 - каждый агент действует в окружении и возвращает среднее вознаграждение за несколько эпизодов;
 - производится селекция агентов с лучшими результатами для создания новых членов популяции;
 - в параметры новых членов добавляются случайные искажения;
 - пул популяции агентов обновляется — в него добавляются новые члены, а старые, показавшие худшие результаты, удаляются.

Все это напоминает процесс эволюции в животном мире. Именно поэтому данные стратегии и называются *эволюционными*. Под селекцией здесь подразумевается простое объединение существующих агентов с лучшими результатами, чтобы повысить вероятность получения высоких результатов в следующем поколении. Как и во всех решениях обучения с подкреплением, необходимо найти баланс между жадным поиском локально-оптимальных решений и исследованием неизвестных областей в пространстве параметров, где могут скрываться лучшие решения. Вот почему важно добавить в популяцию элемент случайности, чтобы не сужать поле поиска.

СМА-ES — это лишь одна из множества эволюционных стратегий. Она основана на использовании нормального распределения, из которого выбирает параметры для новых агентов. В каждом поколении алгоритм стратегии обновляет среднее значение распределения, чтобы максимизировать вероятность выбора агентов с высокими показателями из предыдущего временного шага. Одновременно он обновляет ковариационную матрицу распределения, чтобы максимизировать вероятность выбора агентов с высокой оценкой, опираясь на предыдущее среднее значение. Его можно рассматривать как разновидность градиентного спуска, обладающую дополнительным преимуществом — в ней не применяются производные, а это означает, что не нужно выполнять дорогостоящие вычисления для оценки градиентов.

На рис. 12.13 представлено одно поколение алгоритма. Здесь мы пытаемся найти минимум сильно нелинейной двумерной функции — в красной и черной областях она имеет большие значения, чем в желтой и белой.

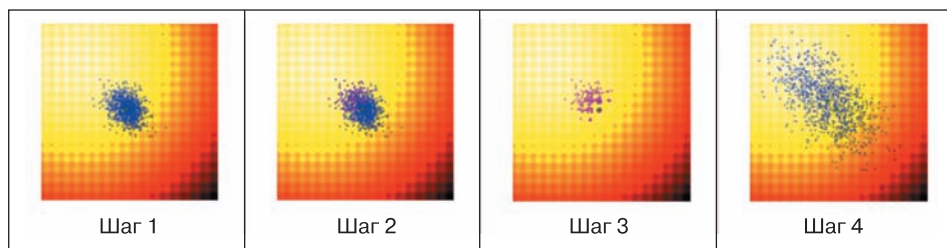


Рис. 12.13. Один шаг обновления в алгоритме CMA-ES [На, 2017]

Вот эти шаги.

1. Сначала генерируется случайное двумерное нормальное распределение и из него выбирается популяция кандидатов, изображенных как синие точки.
2. Затем для каждого кандидата вычисляется значение функции и выбираются лучшие 25 %, изображенные как фиолетовые точки, — назовем этот набор точек P .
3. По точкам из набора P вычисляется среднее значение для нового нормального распределения. Этот шаг можно рассматривать как стадию селекции, в которой выбираются лучшие кандидаты для вычисления среднего нового распределения. По точкам из набора P вычисляется ковариационная матрица нового нормального распределения, но с использованием прежнего среднего значения, а не текущего, полученного по набору P . Чем больше разница между прежним и новым средним в наборе точек P , тем больше дисперсия следующего нормального распределения. Это создает эффект импульса при поиске оптимальных параметров.
4. Затем из нового нормального распределения с новыми средним значением и ковариационной матрицей выбирается новая популяция кандидатов.

На рис. 12.14 показаны несколько поколений процесса. Посмотрите, как ковариация расширяется, когда среднее значение движется большими шагами к минимуму, и сужается, когда оно достигает истинного минимума.

В задаче с гоночным автомобилем вместо четко определенной функции, которую можно было бы максимизировать, у нас имеется окружение с 867 параметрами, определяющими, насколько хорошо действует агент. Первоначально некоторые наборы параметров случайно будут давать высокие оценки, и алгоритм будет постепенно смещать нормальное распределение в направлении параметров с самыми высокими оценками в окружении.

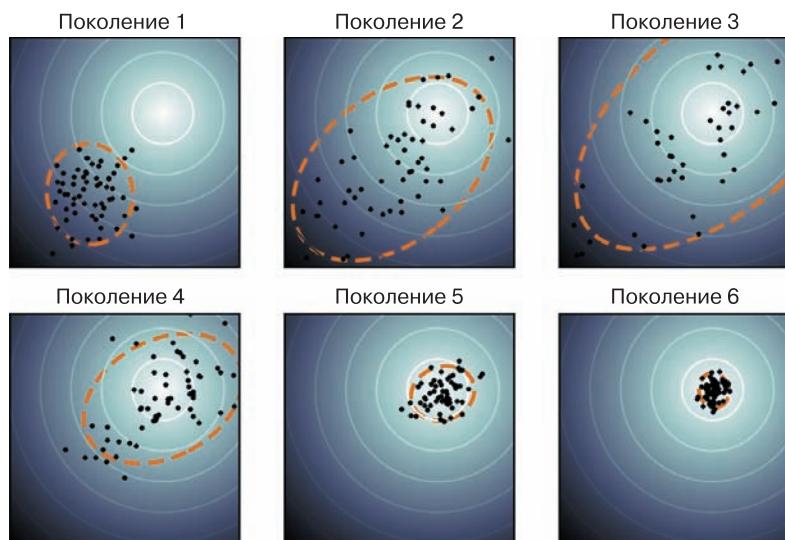


Рис. 12.14. CMA-ES (источник: Википедия, <https://oreil.ly/F0bGZ>)

Параллельное выполнение алгоритма CMA-ES

Одним из больших преимуществ алгоритма CMA-ES является простота его распараллеливания. Наиболее трудоемкая часть алгоритма — вычисление оценки для заданного набора параметров, поскольку для этого необходимо смоделировать поведение агента с этими параметрами в окружении. Данный процесс можно распараллелить, так как между отдельными экземплярами агента нет никаких зависимостей. Для этого используем архитектуру «ведущий/ведомый», где есть главный (ведущий) процесс, который отправляет наборы параметров для параллельного тестирования множеству подчиненных (ведомых) процессов. Ведомые процессы возвращают результаты ведущему, который накапливает их и затем передает общий результат в объект CMA-ES. Этот объект обновляет среднее значение и ковариационную матрицу нормального распределения (см. рис. 12.13) и передает ведущему процессу новую популяцию для тестирования. Затем цикл повторяется. Диаграмма на рис. 12.15 поясняет происходящее.

- 1 Ведущий процесс запрашивает у объекта CMA-ES (es) набор параметров для опробования.
- 2 Ведущий процесс распределяет параметры между ведомыми процессами. Здесь каждый из четырех ведомых процессов получает два набора параметров.

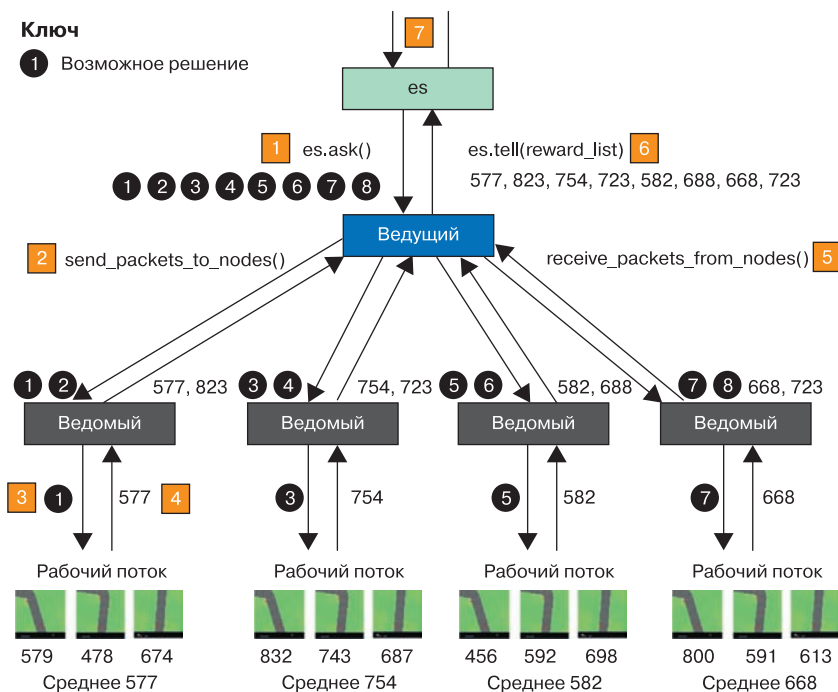


Рис. 12.15. Параллельное выполнение алгоритма CMA-ES — здесь имеется восемь популяций и четыре ведомых процесса, поэтому каждый ведомый процесс выполняет $t = 2$ испытания

- 3 Ведомые процессы запускают рабочие потоки, которые в цикле перебирают набор параметров и для каждого выполняют несколько эпизодов. Каждый набор параметров опробуется на трех эпизодах.
- 4 Для вознаграждений во всех эпизодах вычисляется среднее, которое служит оценкой для каждого набора параметров.
- 5 Ведомый процесс возвращает список оценок ведущему.
- 6 Ведущий процесс группирует оценки и посылает полученный список объекту es.
- 7 Объект es вычисляет по списку оценок новое нормальное распределение (см. рис. 12.13).

Примерно через 200 временных шагов процесс обучения в задаче управления гоночным автомобилем достиг средней величины вознаграждения 840 (рис. 12.16).

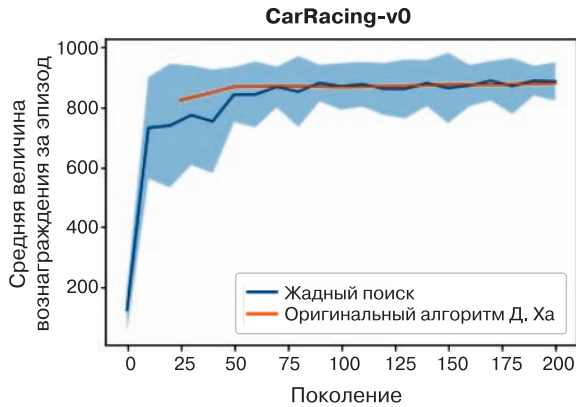


Рис. 12.16. Средняя величина вознаграждения за эпизод, полученная контроллером в процессе обучения

Обучение в мнимом окружении

До настоящего времени обучение контроллера проводилось с использованием метода `step` окружения CarRacing, который реализует переход модели из одного состояния в другое. Эта функция вычисляет следующее состояние и вознаграждение, исходя из текущего состояния окружения и выбранного действия.

Отмечу, что функционально метод `step` очень похож на сеть MDN-RNN в нашей модели. MDN-RNN выводит прогноз z и вознаграждения, исходя из текущего z и выбранного действия.

Фактически MDN-RNN можно рассматривать как самостоятельное окружение, работающее в пространстве z , а не в пространстве исходных изображений. Невежливо, но это значит, что мы можем заменить реальное окружение копией сети MDN-RNN и обучить контроллер, используя исключительно сеть MDN-RNN, которая *представляет*, как должно вести себя окружение.

Иными словами, сеть MDN-RNN довольно хорошо изучила физику реального окружения на исходном наборе данных с информацией о случайных перемещениях, и теперь ее можно использовать в качестве заменителя реального окружения для обучения контроллера. Это означает, что агент может научиться решать новые задачи, *стремясь* максимизировать вознаграждение в мнимом окружении, не проверяя стратегии в реальном мире. После этого он сможет хорошо выполнить задачу с первого раза, никогда прежде не выполнив ее в реальности.

Далее приводится сравнение архитектур для обучения в реальном и мнимом окружениях: архитектура с реальным окружением показана на рис. 12.17, а с мнимым — на рис. 12.18.

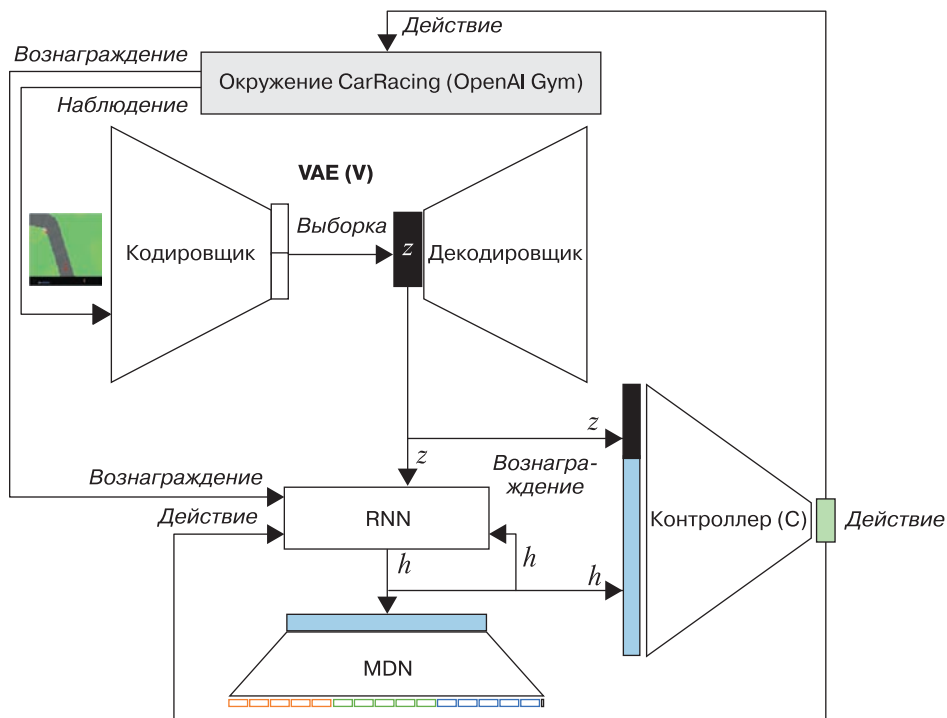


Рис. 12.17. Обучение контроллера в окружении Gym

Обратите внимание на то, что в архитектуре с мнимым окружением обучение контроллера полностью происходит в пространстве z и нет необходимости декодировать векторы z обратно в изображения трека. Конечно, мы могли бы сделать это, чтобы визуально проверить работу агента, но для обучения этого не требуется.

Одним из недостатков обучения агентов полностью в мнимом окружении MDN-RNN является переобучение. Это происходит, когда агент обнаруживает беспроеигрышную стратегию в мнимом окружении, но не обобщает ее на реальное окружение из-за того, что MDN-RNN не смогла изучить поведение истинного окружения в определенных условиях.

Авторы статьи *World Models* подчеркивают эту проблему и показывают, как включение в модель параметра *temperature* для управления неопределенностью

никогда не пытался выполнить эту задачу в реальной среде. Он лишь предпринимал в ней случайные шаги для обучения VAE и модели *мнимой среды* MDN-RNN, а затем использовал мнимую среду для обучения.

TEMPERATURE τ	VIRTUAL SCORE	ACTUAL SCORE
0.10	2086 $\pm$ 140	193 $\pm$ 58
0.50	2060 $\pm$ 277	196 $\pm$ 50
1.00	1145 $\pm$ 690	868 $\pm$ 511
1.15	918 $\pm$ 546	1092 $\pm$ 556
1.30	732 $\pm$ 269	753 $\pm$ 139
RANDOM POLICY	N/A	210 $\pm$ 108
GYM LEADER	N/A	820 $\pm$ 58

Рис. 12.19. Использование параметра *temperature* для управления изменчивостью мнимого окружения (источник: [Ha, Schmidhuber, 2018], <https://arxiv.org/abs/1803.10122>)

Ключевое преимущество применения генеративных моделей мира как подхода к обучению с подкреплением заключается в том, что обучение каждого следующего поколения в мнимой среде происходит намного быстрее, чем обучение в реальной среде, потому что прогнозирование z и вознаграждения с помощью MDN-RNN выполняется быстрее, чем вычисление z и вознаграждения в среде Gym.

Резюме

В этой главе вы увидели, как использовать генеративную модель (VAE) в условиях обучения с подкреплением, чтобы позволить агенту найти эффективную стратегию путем проверки правил в пределах собственного сгенерированного мнимого, а не реального окружения.

Вариационный автокодировщик (VAE) учится изучать скрытое представление окружения, которое затем передается на вход рекуррентной нейронной сети, прогнозирующей будущие траектории в скрытом пространстве. Удивительно, но агент может затем применять эту генеративную модель в качестве мнимого окружения для итеративной проверки правил, используя эволюционную методологию, которая хорошо обобщает реальное окружение.

Дополнительную информацию о модели можно найти в Интернете, где есть отличное интерактивное объяснение (<https://worldmodels.github.io/>), написанное авторами оригинальной статьи.

Мультимодальные модели

В этой главе вы:

- узнаете, что такое мультимодальные модели;
- исследуете особенности работы DALL.E 2 — крупномасштабной модели преобразования текста в изображение, созданной в OpenAI;
- узнаете, какую важную роль играют модели CLIP и модели диффузии, такие как GLIDE, в общей архитектуре DALL.E 2;
- познакомитесь с ограничениями модели DALL.E 2, отмеченными ее авторами;
- изучите архитектуру Imagen — крупномасштабной модели преобразования текста в изображение, созданной в Google Brain;
- узнаете о скрытом процессе распространения, используемом в Stable Diffusion — модели преобразования текста в изображение с открытым исходным кодом;
- познакомитесь со сходством DALL.E 2, Imagen и Stable Diffusion и различиями между ними;
- изучите DrawBench — пакет сравнительного анализа для оценки моделей преобразования текста в изображение;
- изучите архитектуру Flamingo — новейшей модели визуального языка, созданной в DeepMind;
- исследуете различные компоненты Flamingo и узнаете, какой вклад каждый из них вносит в модель в целом;
- изучите некоторые возможности Flamingo, включая диалоговые подсказки.

До настоящего момента мы рассматривали задачи генеративного обучения, фокусирующиеся исключительно на одной модальности данных — тексте, изображениях или музыке. Мы увидели, что GAN и диффузионные модели способны генерировать высококачественные изображения, а трансформеры открывают новые возможности в области генерирования текста и изображений. Будучи людьми, мы не испытываем трудностей с пересечением модальностей, например, без особого труда описываем словами, что изображено на фотографии, представляем себе, как выглядит вымышленный мир, описанный в фантастическом романе, и связываем музыку в фильме с эмоциональной окраской данной сцены. Но можно ли научить машины делать то же самое?

Введение

Мультимодальное обучение предполагает обучение генеративных моделей преобразованию двух или более различных видов данных. Некоторые из наиболее впечатляющих генеративных моделей, представленных за последние 2 года, имели мультимодальный характер. В этой главе мы подробно рассмотрим, как они работают, вы познакомитесь с большими мультимодальными моделями, за которыми будущее генеративного моделирования.

Мы исследуем четыре модели преобразования текста в изображение: DALL·E 2, созданную в OpenAI; Imagen, разработанную в Google Brain; Stable Diffusion, созданную в Stability AI, CompVis и Runway; и Flamingo — продукт DeepMind.



Цель этой главы — кратко объяснить, как работает каждая модель, не вдаваясь в мелкие детали каждого решения. За дополнительной информацией обращайтесь к соответствующим статьям, описывающим модели, где подробно объясняются все варианты реализации и архитектурные решения.

Цель преобразования текста в изображение — получить высококачественное изображение по заданному текстовому описанию. Например, мы хотели бы, чтобы по описанию «слепленный из глины кочан брокколи, улыбающийся солнцу» модель сгенерировала изображение, показанное на рис. 13.1.

Слепленный из глины кочан
брокколи, улыбающийся солнцу



Рис. 13.1. Пример изображения, сгенерированного моделью DALL·E 2 по текстовому описанию

Очевидно, что это очень сложная задача. Анализ смыслового содержания текста и генерирование изображений сложны сами по себе, как мы видели в предыдущих главах этой книги. Мультимодальное моделирование, подобное

этому, сопряжено с дополнительными сложностями, поскольку модель должна научиться также выстраивать связи между двумя областями и изучить общее представление, которое позволит ей точно преобразовывать блок текста в качественное изображение без потери информации.

Более того, чтобы добиться успеха, модель должна уметь сочетать концепции и стили, которых она, возможно, никогда раньше не видела. Например, вы нигде не найдете фресок Микеланджело, на которых есть люди в гарнитурах виртуальной реальности, но нам бы хотелось, чтобы наша модель могла создать такое изображение, если ее об этом попросят. Точно так же желательно, чтобы модель по текстовому запросу точно определяла, как объекты в сгенерированном изображении связаны друг с другом. Например, изображение астронавта, летящего на пончике в космосе, должно сильно отличаться от изображения астронавта, поедающего пончик в многолюдном месте. Модель должна научиться понимать, как словам придается значение через контекст и как преобразовывать явные связи между сущностями в тексте в изображения, подразумевающие то же значение.

DALL.E 2

Первой мы рассмотрим модель DALL.E 2, разработанную в OpenAI, для преобразования текста в изображение. Первая версия этой модели, DALL.E [Ramesh et al., 2021], вышла в феврале 2021 года и вызвала новую волну интереса к генеративным мультимодальным моделям. В этом разделе мы рассмотрим работу второй версии модели, DALL.E 2 [Ramesh et al., 2022], вышедшей чуть более года спустя, в апреле 2022-го.

DALL.E 2 — впечатляющая модель, наглядно показывающая, что искусственный интеллект способен решать подобные мультимодальные задачи. Это достижение вызывает не только академический интерес, но и заставляет нас задаваться серьезными вопросами, касающимися роли ИИ в творческих процессах, которые раньше считались присущими только людям. Для начала рассмотрим некоторые особенности работы DALL.E 2, опираясь на фундаментальные идеи, которые мы уже рассмотрели в этой книге.

Архитектура

Чтобы понять, как работает модель DALL.E 2, рассмотрим сначала ее общую архитектуру (рис. 13.2).

Здесь мы видим три отдельные части: *кодировщик*, *модель выборки* и *декодировщик*. Текст сначала передается в кодировщик, который создает векторное представление текста. Затем этот вектор преобразуется в векторное представление

изображения, после чего оно передается в декодировщик вместе с исходным текстом для создания изображения. Мы рассмотрим все эти компоненты по очереди, чтобы получить полное представление о том, как работает DALL.E 2.

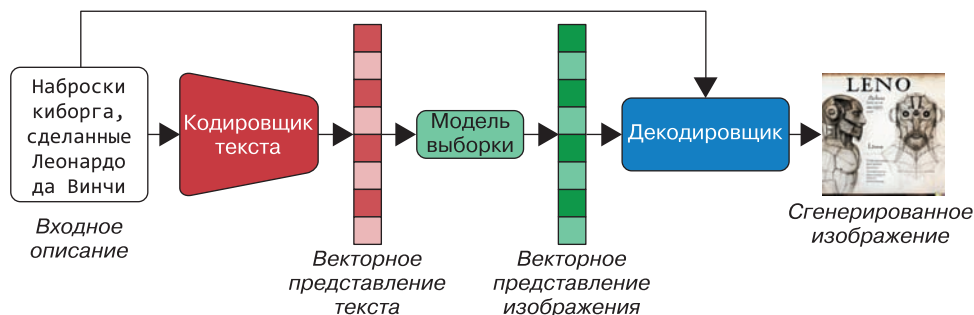


Рис. 13.2. Архитектура DALL.E 2

Кодировщик

Цель кодировщика — преобразование текстового описания в векторное представление, задающее концептуальное значение текстового описания в скрытом пространстве. Как было показано в предыдущих главах, преобразование дискретного текста в непрерывное скрытое пространство играет важную роль для всех последующих задач, поскольку дает возможность манипулировать векторным представлением в зависимости от конкретной цели.

В DALL.E 2 авторы не обучают кодировщик текста с нуля, а используют существующую модель *Contrastive Language-Image Pre-training* (CLIP), также созданную в OpenAI. Поэтому, чтобы понять работу кодировщика текста, сначала нужно разобраться с тем, как работает CLIP.

CLIP

Модель CLIP [Radford et al., 2021] (<https://openai.com/blog/clip>) была представлена в статье, опубликованной OpenAI в феврале 2021 года (всего через несколько дней после первой статьи с описанием DALL.E), где о ней сказано: «Нейронная сеть, эффективно изучающая визуальные концепции на основе описания на естественном языке».

Она использует прием, называемый *контрастивным обучением*, для изучения соответствий между изображениями и текстовыми описаниями. Модель обуча-

ется на наборе данных, содержащем 400 млн пар «текст — изображение», взятых из Интернета, некоторые примеры пар показаны на рис. 13.3. Для сравнения: в ImageNet имеется 14 млн изображений, описанных вручную. Когда есть изображение и список возможных текстовых описаний, задача сети — найти то описание, которое действительно соответствует изображению.

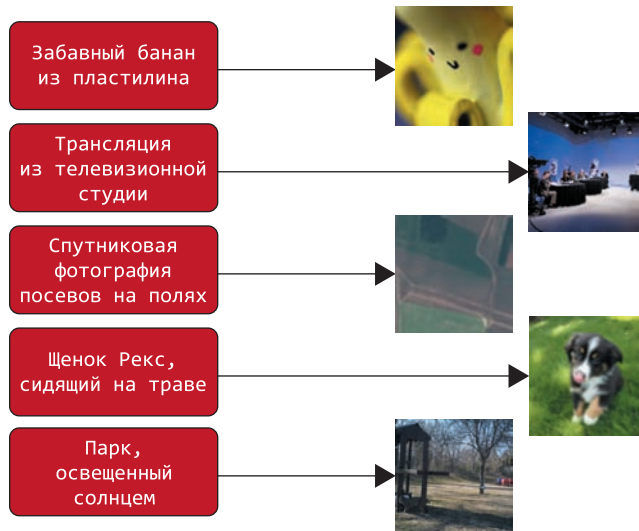


Рис. 13.3. Примеры пар «текст — изображение»

Основная идея контрастивного обучения проста. Мы обучаем две нейронные сети: кодировщик, преобразующий текст в векторное представление, и кодировщик изображений, преобразующий изображение в векторное представление. Затем, взяв пакет пар «текст — изображение», сравниваем все комбинации векторных представлений текста и изображений, используя косинусное сходство, и обучаем сети максимизировать оценку для совпадающих пар и минимизировать для несовпадающих. Этот процесс показан на рис. 13.4.



CLIP — это не генеративная модель

Обратите внимание на то, что модель CLIP не является генеративной моделью — она не может создавать изображения или текст. Скорее, она ближе к дискриминативным моделям, потому что окончательным результатом является определение того, какое текстовое описание из набора наиболее точно соответствует данному изображению или, наоборот, какое изображение наиболее точно соответствует данному текстовому описанию.

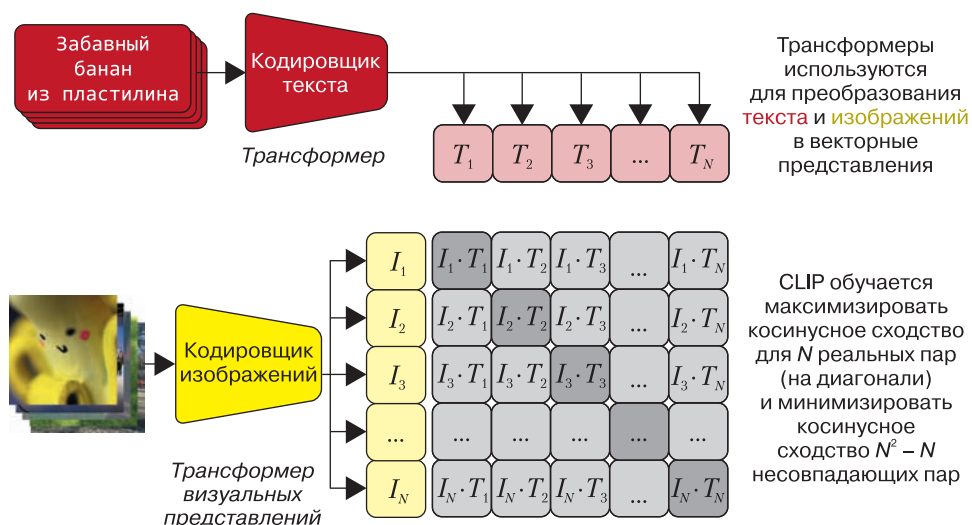


Рис. 13.4. Процесс обучения модели CLIP

Кодировщики текста и изображений являются трансформерами. Кодер изображений — это визуальный трансформер (Vision Transformer, ViT), представленный в разделе «ViT VQ-GAN» главы 10, использующий тот же принцип внимания применительно к изображениям. Авторы протестировали и другие архитектуры моделей, но обнаружили, что эта комбинация дает наилучшие результаты.

Наиболее интересная особенность модели CLIP — возможность применять ее для *предсказания без ознакомления* (zero-shot prediction). Предположим, мы хотим использовать CLIP для предсказания метки определенного изображения в наборе данных ImageNet. Сначала мы можем преобразовать метки ImageNet в предложения с помощью шаблона, например, «фотография <объект>» (рис. 13.5).

Чтобы предсказать метку данного изображения, можно передать ее кодировщику изображений CLIP, вычислить косинусное сходство между векторным представлением изображения и всеми возможными векторными представлениями текста и найти метку с максимальной оценкой (рис. 13.6).

Обратите внимание на то, что нам не требуется обучать ни одну из нейронных сетей в модели CLIP, чтобы ее можно было использовать в новых задачах. Она применяет язык как обобщенное средство, посредством которого может быть выражен любой набор меток.

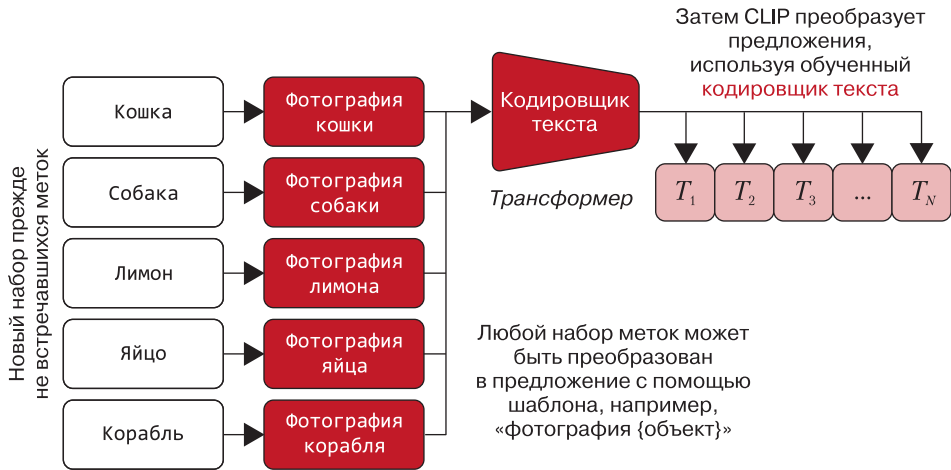


Рис. 13.5. Преобразование меток в новый набор подписей для создания векторных представлений в CLIP

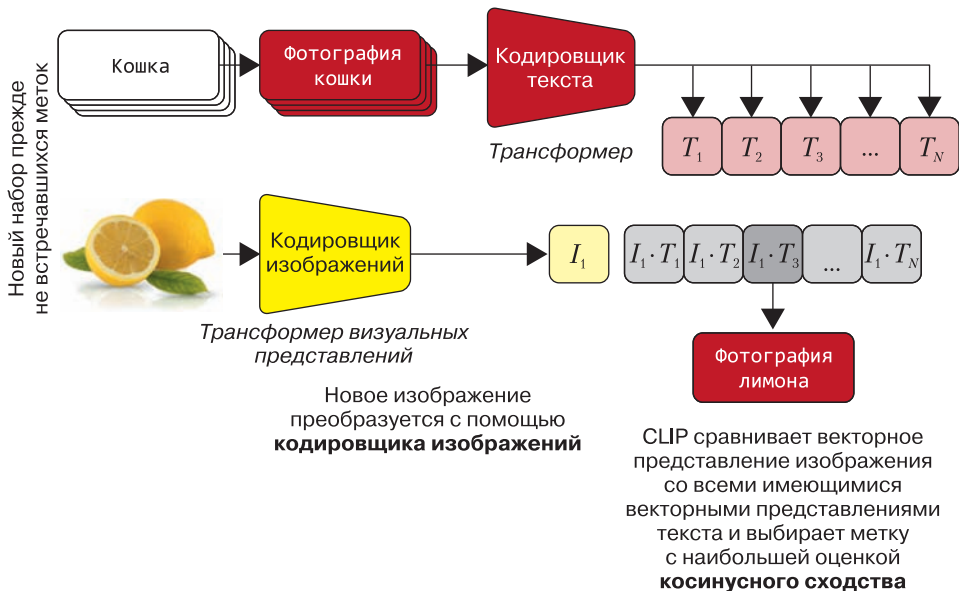


Рис. 13.6. Использование CLIP для предсказания содержимого изображения

С помощью этого подхода можно показать, что CLIP хорошо справляется с широким спектром задач маркировки наборов данных изображений (рис. 13.7).

Другие модели, обученные на конкретном наборе данных для предсказания некоторого набора меток, часто терпят неудачу, когда их применяют к разным наборам данных с одинаковыми метками, потому что они оптимизированы для наборов данных, на которых обучались. CLIP — гораздо более надежная модель, потому что она обучалась глубокому концептуальному пониманию полнотекстовых описаний и изображений, а не решению узкой задачи назначения одной метки определенному изображению в наборе данных.

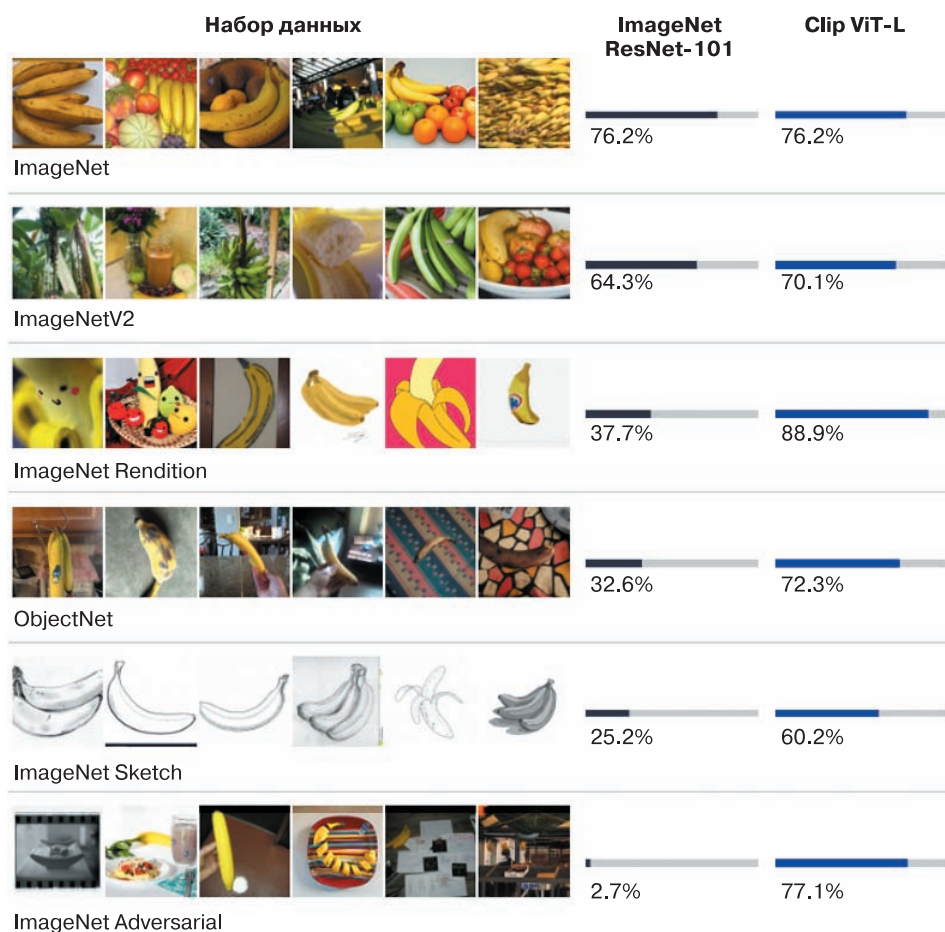


Рис. 13.7. CLIP хорошо справляется с широким спектром задач маркировки наборов данных изображений (источник: [Radford et al., 2021], <https://arxiv.org/abs/2103.00020>)

Уже упоминалось, что модель CLIP оценивается по ее способности различать, как она может помочь создавать генеративные модели, например DALL·E 2.

Все просто. Мы можем взять обученный кодировщик текста и использовать его как часть более крупной модели, такой как DALL.E 2, с фиксированными весами. Обученный кодировщик — это просто обобщенная модель преобразования текста в векторное представление, которое можно применять в последующих задачах, таких как генерирование изображений. Кодировщик текста способен выявить богатое концептуальное содержание текста, поскольку обучен обеспечивать максимальное сходство с соответствующим аналогом векторного представления изображения, которое создается только из парного изображения. Образно говоря, кодировщик текста — это первая часть моста, по которому можно перейти из текстовой области в область изображений.

Модель выборки

Следующий этап — преобразование векторного представления текста в векторное представление изображения. Авторы DALL.E 2 попробовали применить два вида моделей:

- авторегрессионную;
- диффузионную.

Они обнаружили, что диффузионная модель превосходит авторегрессионную и более эффективна в вычислительном отношении. В этом разделе мы рассмотрим оба варианта, чтобы понять, чем они отличаются друг от друга.

Авторегрессионная модель выборки

Авторегрессионная модель генерирует выходные данные последовательно, упорядочивая выходные единицы, например слова или пиксели, и выбирая следующую единицу на основе значений предыдущих. В предшествующих главах я приводил примеры использования такого подхода в рекуррентных нейронных сетях (например, LSTM), трансформерах и PixelCNN.

Авторегрессионная модель выборки в DALL.E 2 — это трансформер типа «кодировщик — декодировщик». Она обучена воспроизводить векторное представление изображения CLIP, принимая на входе векторное представление текста CLIP (рис. 13.8). Обратите внимание на то, что в авторегрессионной модели, упомянутой в оригинальной статье, есть некоторые дополнительные компоненты, которые мы здесь для краткости опустили.

Модель обучается на наборе данных с парами «текст — изображение CLIP». Ее можно представить себе как вторую часть моста, который нужен для перехода из текстовой области в область изображения: она преобразует вектор из скрытого пространства с представлением текста в вектор скрытого пространства с представлением изображения.

Векторное представление входного текста обрабатывается кодировщиком трансформера для создания другого представления, которое передается в декодировщик вместе с текущим сгенерированным векторным представлением изображения. Выходное изображение генерируется по одному элементу за раз с использованием учителя, заставляющего сравнивать следующий предсказанный элемент с фактическим векторным представлением изображения CLIP.

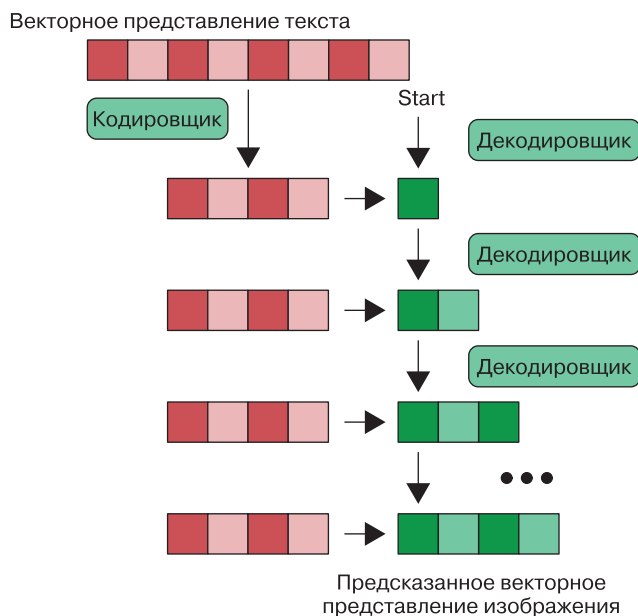


Рис. 13.8. Упрощенная схема авторегрессионной модели выборки в DALL.E 2

Последовательный характер работы объясняет меньшую вычислительную эффективность авторегрессионной модели по сравнению с другим методом, опробованным авторами, который мы рассмотрим далее.

Диффузионная модель выборки

Как мы видели в главе 8, диффузионные модели быстро завоевывают популярность у практиков генеративного моделирования наряду с трансформерами. В DALL.E 2 в качестве модели выборки используется трансформер, содержащий только декодировщик, обученный с применением процесса диффузии.

Процесс обучения и генерирования изображения показан на рис. 13.9. И снова здесь упрощенная версия — полную информацию об организации диффузионной модели вы найдете в оригинальной статье.

Во время обучения каждая пара векторных представлений и изображений CLIP сначала объединяется в один вектор. Затем в течение 1000 временных шагов в векторное представление изображения добавляется шум, пока оно не станет неотличимым от случайного шума. Затем диффузионную модель выборки обучают прогнозированию векторного представления изображения с удалением шума, добавленного на предыдущем временном шаге. Модель выборки имеет доступ к векторному представлению текста на всем протяжении обучения, поэтому может корректировать свои прогнозы с учетом этой информации, постепенно преобразуя случайный шум в предсказанное векторное представление изображения CLIP. В роли функции потерь используется среднее средней квадратической ошибки на всех этапах удаления шума.

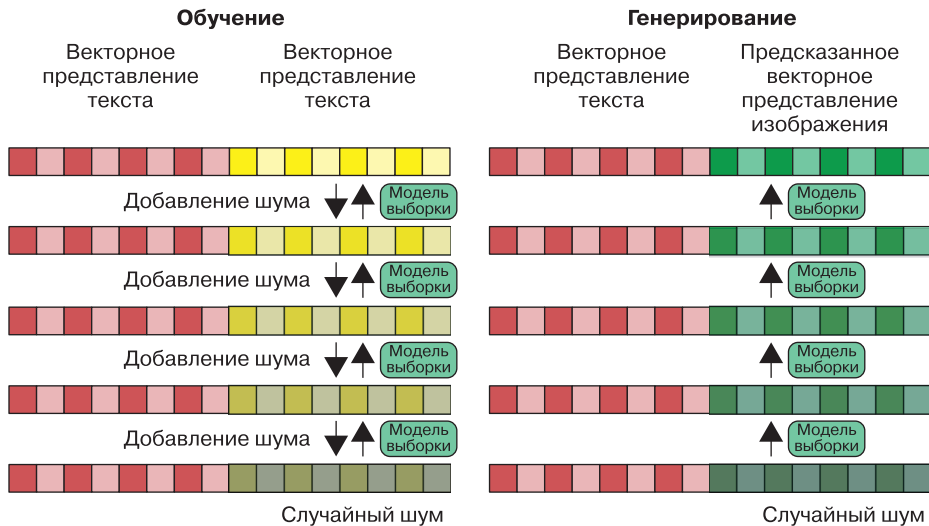


Рис. 13.9. Упрощенная схема процессов обучения и использования диффузионной модели выборки в DALL.E 2

Чтобы сгенерировать новые векторные представления изображений, выбирается случайный вектор, в начало добавляется соответствующее векторное представление текста и результат несколько раз пропускается через обученную модель выборки.

Декодировщик

Последняя часть DALL.E 2 — декодировщик. Эта часть модели генерирует окончательное изображение на основе текстового описания и предсказанного векторного представления полученного ранее изображения.

Архитектура и процесс обучения декодировщика заимствованы из более ранней статьи OpenAI, опубликованной в декабре 2021 года, где была представлена генеративная модель Guided Language to Image Diffusion for Generation and Editing (GLIDE) [Nichol et al., 2021]. Эта модель способна генерировать реалистичные изображения из текстовых описаний почти так же, как это делает DALL.E 2. Разница лишь в том, что GLIDE не использует векторных представлений CLIP, а обучается с нуля непосредственно на исходных текстовых описаниях (рис. 13.10).

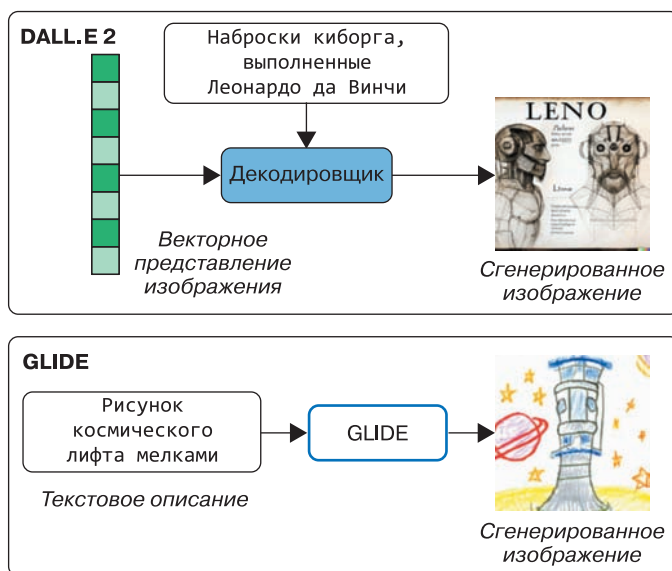


Рис. 13.10. Сравнение DALL.E 2 и GLIDE: GLIDE обучается с нуля, тогда как DALL.E 2 использует векторные представления CLIP для переноса информации из исходного текста

Посмотрим сначала, как работает GLIDE.

GLIDE

Модель GLIDE обучена как диффузионная модель с архитектурой U-Net для удаления шума и архитектурой трансформера для кодировщика текста. Она учится удалять шум, добавленный в изображение, руководствуясь текстовым описанием. Наконец, *Upsampler* обучается масштабировать сгенерированное изображение до размера 1024×1024 пиксела.

GLIDE обучает модель с 3,5 млрд параметров с нуля — 2,3 млрд параметров для визуальной части модели (U-Net и Upsampler) и 1,2 млрд для трансформера. Обучение производится на 250 млн пар «текст — изображение».

Процесс диффузии показан на рис. 13.11. Трансформер используется для создания векторного представления входного текстового описания, которое затем задействуется для управления U-Net на протяжении всего процесса удаления шума. Мы исследовали архитектуру U-Net в главе 8 — это идеальный выбор, когда общий размер изображения должен оставаться неизменным, например, для передачи стиля, удаления шума и т. д.

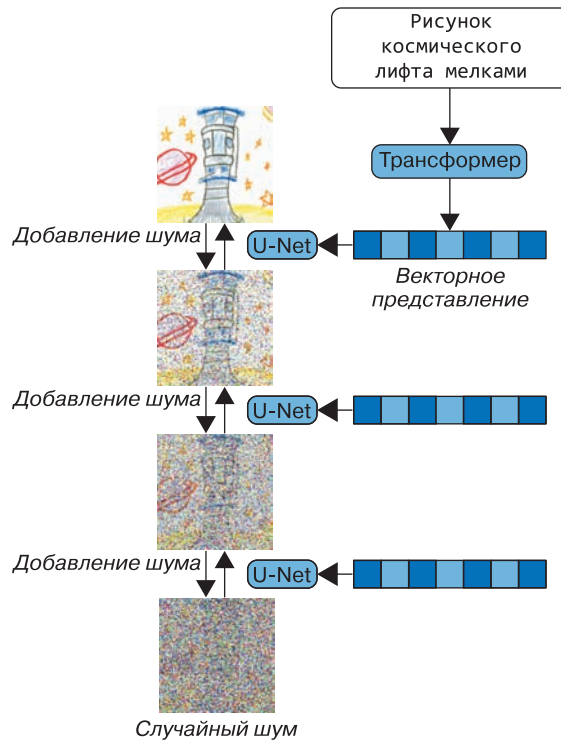


Рис. 13.11. Процесс диффузии в модели GLIDE

Декодировщик DALL·E 2 тоже использует модель U-Net для удаления шума и модель трансформера для кодирования текста, но дополнительно обусловлен моделью CLIP с векторными представлениями изображений. Это ключевое отличие DALL·E 2 от GLIDE, как показано на рис. 13.12.

Как и во всех моделях диффузии, чтобы сгенерировать новое изображение, мы просто выбираем некоторый случайный шум и несколько раз пропускаем его через сеть удаления шума U-Net вместе с векторными представлениями текста описания и изображения, созданными трансформером. На выходе получается изображение 64×64 пиксела.

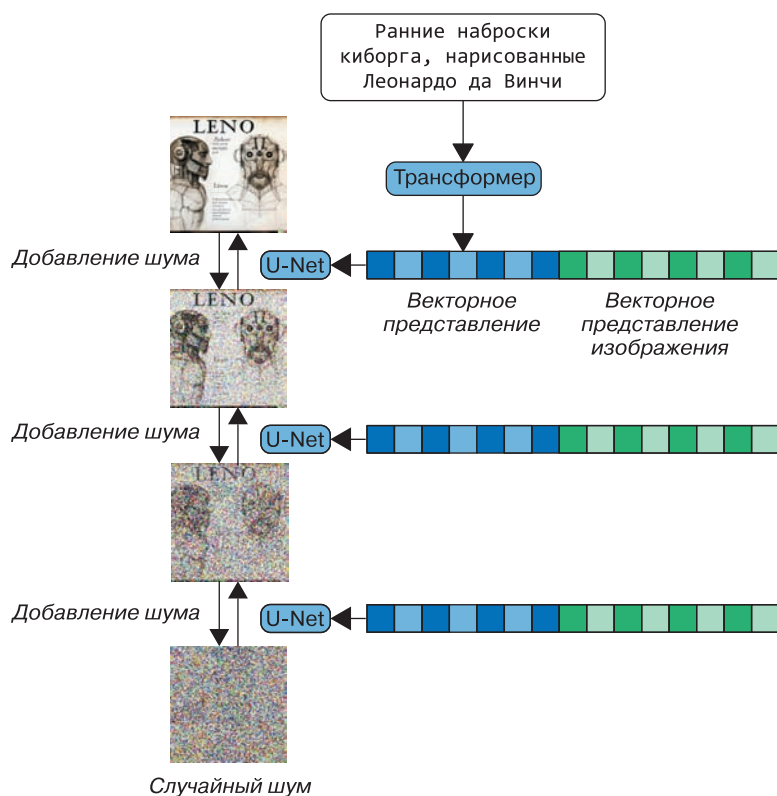


Рис. 13.12. Декодировщик DALL.E 2 дополнительно обусловлен предварительно созданным векторным представлением изображения

Upsampler

Последняя часть декодера — Upsampler (две отдельные диффузионные модели). Первая преобразует изображение размером 64×64 в изображение размером 256×256 пикселей. Вторая преобразует изображение 256×256 в изображение 1024×1024 пикселей (рис. 13.13).

Добавление операции увеличения размеров означает, что нам не нужно создавать большие модели для обработки изображений больших размеров. Мы можем работать с небольшими изображениями до заключительных этапов процесса, где применяется операция увеличения размеров. Это дает экономию на параметрах модели и повышает эффективность процесса обучения.

На этом мы завершаем описание модели DALL.E 2! Итак, DALL.E 2 использует предварительно обученную модель CLIP для создания векторного представле-

ния текстового описания. Затем с применением модели диффузии, называемой моделью выборки, преобразует его в векторное представление изображения. И наконец, с помощью модели диффузии в стиле GLIDE генерирует выходное изображение, учитывая прогнозируемое векторное представление изображения и векторное представление текста, сгенерированного трансформером.

Примеры изображений, сгенерированных моделью DALL.E 2

Примеры изображений, сгенерированных моделью DALL.E 2, можно найти на официальном сайте (<https://openai.com/dall-e-2>). Способность модели воплощать сложные и разрозненные понятия в реалистичные изображения поражает. Безусловно, это значительный шаг вперед для искусственного интеллекта и генеративного моделирования.

В статье авторы показывают, как можно использовать модель для других целей, помимо преобразования текста в изображение. Одно из таких применений — создание вариантов заданного изображения, которое мы рассмотрим в следующем разделе.

Создание вариантов изображений

Как обсуждалось ранее, чтобы сгенерировать изображение с помощью декодировщика DALL.E 2, нужно выбрать изображение, состоящее из чистого случайного шума, а затем постепенно удалять шум с помощью диффузионной модели, обусловленной предоставленным векторным представлением изображения. Выбор разных вариантов случайного шума приведет к получению разных изображений.

Чтобы получить варианты заданного изображения, нужно лишь подать на вход декодировщика векторное представление требуемого изображения. Получить такое векторное представление можно с помощью оригинального кодировщика изображений CLIP, специально предназначенного для преобразования изображения в его векторное представление CLIP. Этот процесс показан на рис. 13.14.

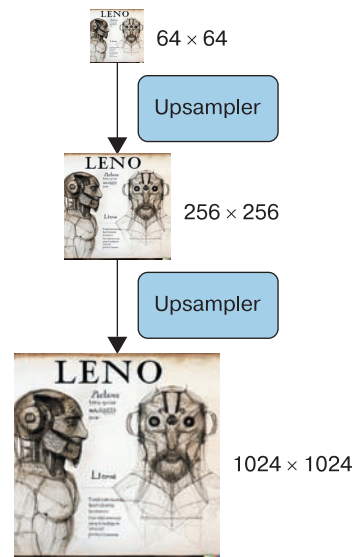


Рис. 13.13. Первая диффузионная модель Upsampler увеличивает размеры изображения с 64×64 до 256×256 пикселей, а вторая — с 256×256 до 1024×1024 пиксела

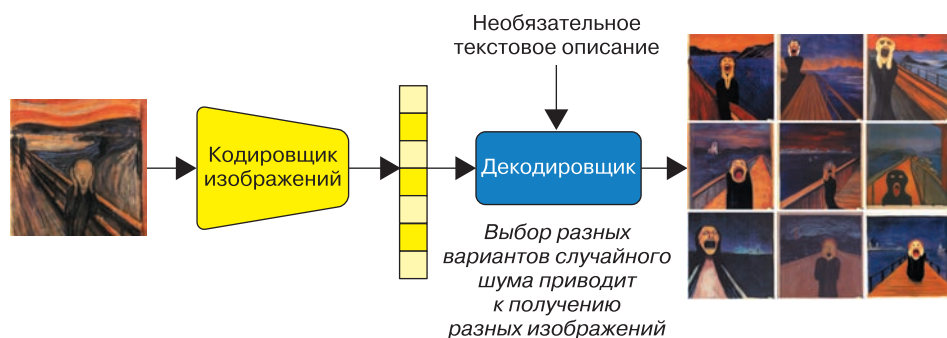


Рис. 13.14. Модель DALL.E 2 можно использовать для создания вариантов заданного изображения

Важность модели выборки

Еще один путь, исследованный авторами, — ограничение использования модели выборки. Задача этой модели — подготовить для декодировщика полезное векторное представление генерируемого изображения с помощью предварительно обученной модели CLIP. Но так ли нужен этот шаг? Может быть, можно, например, вместо векторного представления изображения передать в декодировщик векторное представление текста или вообще игнорировать векторные представления CLIP и обусловить модель одним текстовым описанием? Повлияет ли это на качество сгенерированных изображений?

Чтобы ответить на эти вопросы, авторы попробовали три разных подхода.

1. Передать в декодировщик только текст описания и вектор нулей, представляющий изображение.
2. Передать в декодировщик текст описания и его векторное представление взамен векторного представления изображения.
3. Передать в декодировщик текст описания и векторное представление изображения, то есть использовать полную модель.

Примеры полученных результатов показаны на рис. 13.15. Как видите, когда декодировщик не получает векторного представления изображения, он может вывести только грубое приближение текстового описания, упуская ключевую информацию, такую как калькулятор. Использование векторного представления текста вместо векторного представления изображения немного улучшает результат, но модель все равно оказывается не в состоянии отразить взаимосвязь между ежом и калькулятором. И только применение полной модели выборки приводит к созданию изображения, точно отражающего всю информацию, содержащуюся в описании.

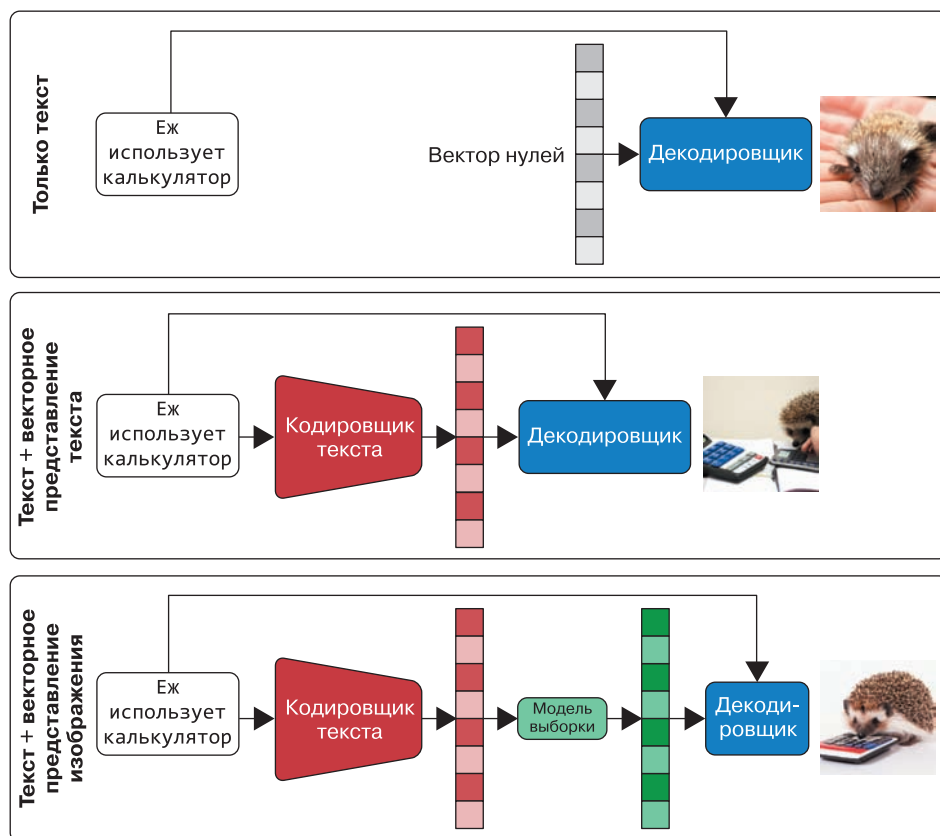


Рис. 13.15. Модель выборки поставляет дополнительный контекст и помогает декодировщику создавать более точные изображения (источник: [Ramesh et al., 2022], <https://arxiv.org/pdf/2204.06125.pdf>)

Ограничения

В статье с описанием DALL.E 2 авторы также выделяют несколько известных ограничений модели. Два из них — привязка атрибутов и генерирование текста — показаны на рис. 13.16.

Привязка атрибутов — это способность модели понимать взаимосвязь между словами в заданном текстовом описании, в частности связь между атрибутами и объектами. Например, изображение, сгенерированное для описания «красный куб на синем кубе», должно отличаться от изображения, сгенерированного для описания «синий куб на красном кубе». DALL.E испытывает некоторые сложности с этим по сравнению с более ранними моделями, такими как GLIDE, хотя общее качество сгенерированных изображений лучше и они разнообразнее.



Рис. 13.16. Два ограничения DALL.E 2 касаются возможностей привязки атрибутов с объектами и воспроизведения текстовой информации: *вверху* — изображения, полученные по описанию «красный куб на синем кубе», *внизу* — по описанию «табличка с надписью deep learning» (источник: [Ramesh et al., 2022], <https://arxiv.org/pdf/2204.06125.pdf>)

Кроме того, модель DALL.E 2 не способна точно воспроизводить текст — вероятно, это связано с тем, что векторные представления CLIP не фиксируют правописание, а содержат только высокоуровневое представление текста. Эти представления могут декодироваться в текст с частичным успехом (например, отдельные буквы в основном верны), но без достаточного понимания правил формирования полных слов.

Imagen

Через месяц с небольшим после появления DALL.E 2 команда Google Brain выпустила свою модель преобразования текста в изображение, названную Imagen [Saharia et al., 2022]. Многие из того, что мы уже рассмотрели в этой главе, относятся и к Imagen: например, эта модель использует текстовый кодировщик и диффузионную модель в роли декодировщика.

В следующем разделе рассмотрим общую архитектуру Imagen и сравним ее с DALL.E 2.

Архитектура

Архитектура Imagen изображена на рис. 13.17.

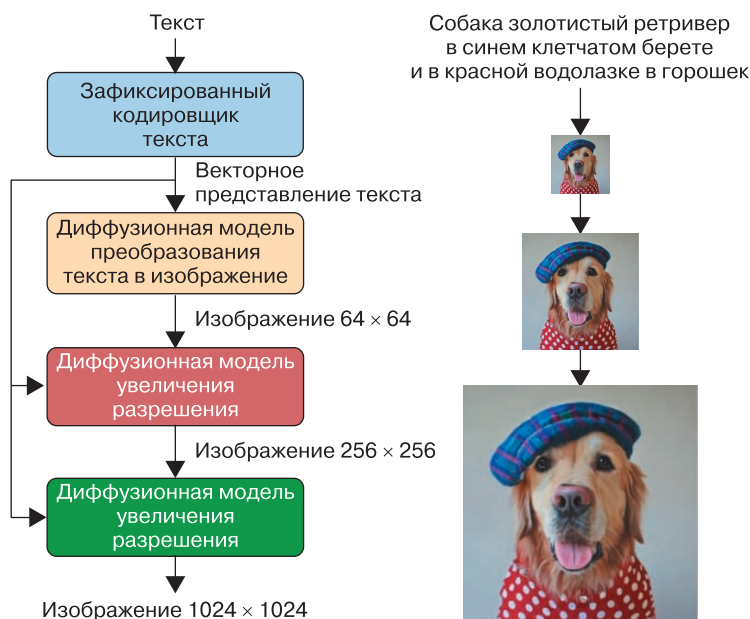


Рис. 13.17. Архитектура Imagen (источник: [Saharia et al., 2022], <https://arxiv.org/abs/2205.11487>)

Зафиксированный кодировщик текста — это предварительно обученная модель T5-XXL, большой трансформер типа «кодировщик-декодировщик». В отличие от CLIP, обучение кодировщика проводилось только на текстовых описаниях, без изображений, так что это не мультимодальная модель. Однако авторы обнаружили, что она очень хорошо подходит на роль кодировщика текста для Imagen и масштабирование этой модели сильнее влияет на качество результата, чем масштабирование декодировщика диффузионной модели.

Как и в DALL·E 2, в Imagen диффузионная модель декодировщика основана на архитектуре U-Net, обусловленной векторным представлением текста. В стандартную архитектуру U-Net внесены несколько изменений, позволяющих создать то, что авторы называют *эффективной U-Net*. Эта модель использует меньше памяти, быстрее сходится и имеет лучшее качество выборки, чем предыдущие модели U-Net.

Модели Upsampler, увеличивающие разрешение сгенерированного изображения с 64×64 до 1024×1024 пиксела, тоже являются диффузионными моделями и тоже используют векторное представление текста для управления процессом увеличения разрешения.

DrawBench

Дополнительным вкладом в Imagen является *DrawBench* — набор из 200 текстовых описаний для преобразования в изображение. Текстовые описания охватывают 11 категорий, таких как «Счет» (возможность создания заданного количества объектов), «Описание» (возможность создания сложных и длинных текстовых описаний объектов) и «Текст» (возможность генерирования текста в кавычках). Для сравнения двух моделей текстовые описания DrawBench пропускаются через каждую модель, а результаты передаются группе людей-экспертов для оценки по двум показателям.

Соответствие

Какое изображение точнее соответствует описанию?

Реалистичность

Какое изображение более фотореалистично (выглядит более реальным)?

Результаты оценки на тестах DrawBench показаны на рис. 13.18.

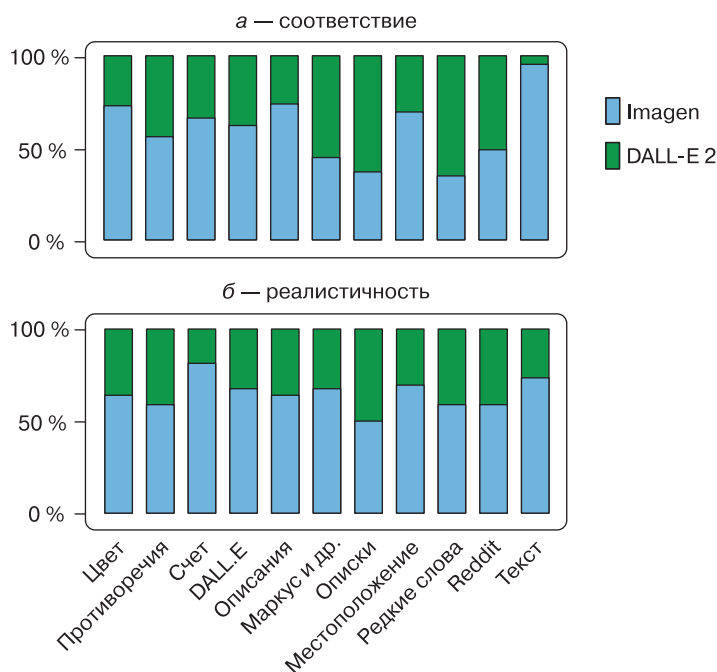


Рис. 13.18. Сравнение Imagen и DALL-E 2 на тестах DrawBench по таким параметрам, как соответствие и реалистичность (источник: [Saharia et al., 2022], <https://arxiv.org/abs/2205.11487>)

Обе модели, DALL.E 2 и Imagen, внесли значительный вклад в область преобразования текста в изображение. Несмотря на то что Imagen превосходит DALL.E 2 по многим тестам DrawBench, DALL.E 2 предоставляет дополнительные возможности, отсутствующие в Imagen. Например, поскольку DALL.E 2 использует CLIP (мультимодальную модель преобразования текста в изображение), она может принимать изображения для создания векторных представлений изображений, то есть DALL.E 2 позволяет редактировать и изменять изображения. Это невозможно в Imagen: текстовый кодировщик — исключительно текстовая модель, поэтому нет никакой возможности ввести в нее изображение.

Примеры изображений, созданных Imagen

На рис. 13.19 показаны примеры изображений, созданных моделью Imagen.



Три стеклянных шара падают в океан в брызгах воды на фоне заходящего солнца

Выходящие стебли растений в форме текста Imagen, с цветами, вылетающими из старого телевизора, и вылетающими бабочками

Клубника с плеском падает в кружку с кофе на фоне звездного неба

Рис. 13.19. Примеры изображений, созданных моделью Imagen (источник: [Saharia et al., 2022], <https://arxiv.org/abs/2205.11487>)

Stable Diffusion

Последняя диффузионная модель преобразования текста в изображение, которую мы рассмотрим, — *Stable Diffusion*, выпущенная в августе 2022 года компанией Stability AI (<https://stability.ai/>) в сотрудничестве с исследовательской группой компьютерного зрения и обучения Мюнхенского университета Людвига-Максимилиана (<https://ommer-lab.com/>) и Runway (<https://runwayml.com/>). Она отличается от DALL.E 2 и Imagen тем, что код и веса модели опубликованы для общего доступа на Hugging Face (<https://oreil.ly/BTrWI>). Это означает, что любой желающий может использовать модель на своем компьютере, не применяя проприетарные API.

Архитектура

Основное архитектурное отличие Stable Diffusion от моделей преобразования текста в изображение, обсуждавшихся ранее, заключается в том, что в качестве базовой генеративной модели она использует модель *скрытой диффузии*. Модели скрытой диффузии (Latent Diffusion Models, LDM) были представлены Ромбахом (Rombach) с коллегами в декабре 2021 года в статье *High-Resolution Image Synthesis with Latent Diffusion Models* [Rombach et al., 2021]. Ключевая концепция состоит в обертывании модели диффузии автокодировщиком, чтобы процесс диффузии применялся к представлению изображения в скрытом пространстве, а не к самому изображению (рис. 13.20).

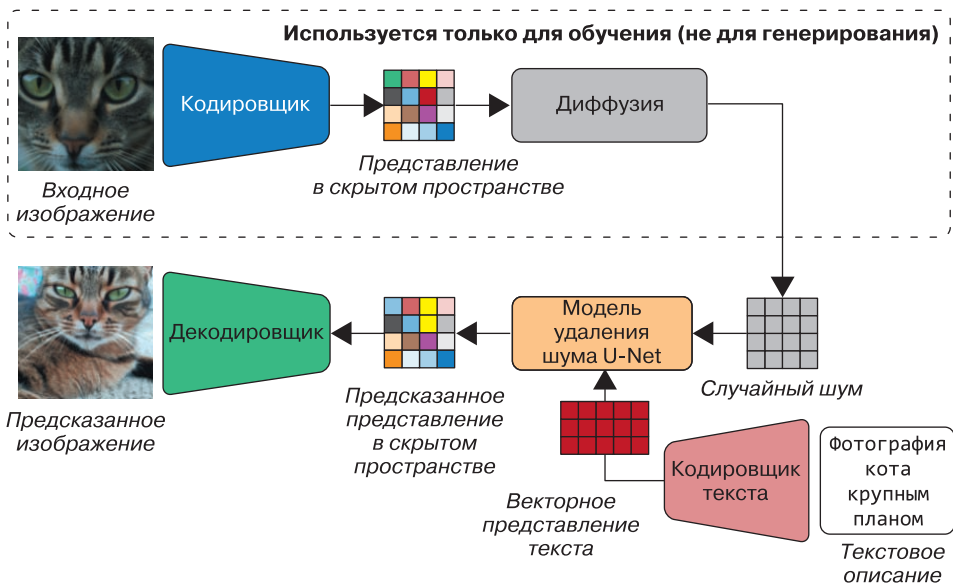


Рис. 13.20. Архитектура Stable Diffusion

Этот прорыв означает, что модель удаления шума U-Net может оставаться относительно легковесной по сравнению с моделями U-Net, обрабатывающими полные изображения. Автокодировщик берет на себя тяжелую работу по кодированию деталей изображения в скрытое пространство и декодированию скрытого пространства обратно в изображение с высоким разрешением, позволяя диффузионной модели работать исключительно в скрытом концептуальном пространстве. Это дает существенный рост скорости обучения.

Процесс удаления шума также может управляться текстовым описанием, что передается в кодировщик текста. Первая версия Stable Diffusion использовала

предварительно обученную модель CLIP, созданную в OpenAI (как и DALL·E 2), но уже в Stable Diffusion 2 ей на смену пришла OpenCLIP (<https://oreil.ly/RaCbu>) — специально обученная с нуля модель CLIP.

Примеры изображений, созданных Stable Diffusion

На рис. 13.21 показаны примеры изображений, сгенерированных моделью Stable Diffusion 2.1, — можете попробовать передать ей свои описания, воспользовавшись сайтом Hugging Face (<https://oreil.ly/LpGW4>).



Многорукий робот готовит вкусную еду



Высокотехнологичное устройство на солнечных батареях



Маленькая избушка на заснеженном склоне горы в стиле мультфильмов Диснея

Рис. 13.21. Примеры изображений, созданных моделью Stable Diffusion 2.1



Исследование скрытого пространства

Если у вас появится желание исследовать скрытое пространство модели Stable Diffusion, я рекомендую изучить пошаговое руководство, имеющееся на веб-сайте Keras (<https://oreil.ly/4sNe5>).

Flamingo

К настоящему моменту мы рассмотрели три модели преобразования текста в изображение. В этом разделе исследуем еще одну мультимодальную модель, которая генерирует текст на основе потока текстовых и визуальных данных. Модель Flamingo, представленная в статье DeepMind в апреле 2022 года [Alayrac et al., 2022], — это семейство визуально-языковых моделей (Visual Language Models, VLM), которые действуют как мост между предварительно обученными моделями компьютерного зрения и языковыми моделями.

Далее мы рассмотрим архитектуру моделей Flamingo и сравним ее с моделями преобразования текста в изображение, обсуждавшимися до сих пор.

Архитектура

Общая архитектура Flamingo проиллюстрирована на рис. 13.22. Более или менее подробно мы рассмотрим только основные ее компоненты — Vision Encoder, Perceiver Resampler и Language Mode, чтобы уяснить ключевые идеи, делающие Flamingo уникальной. Более тщательный анализ каждой части модели вы найдете в оригинальной статье.

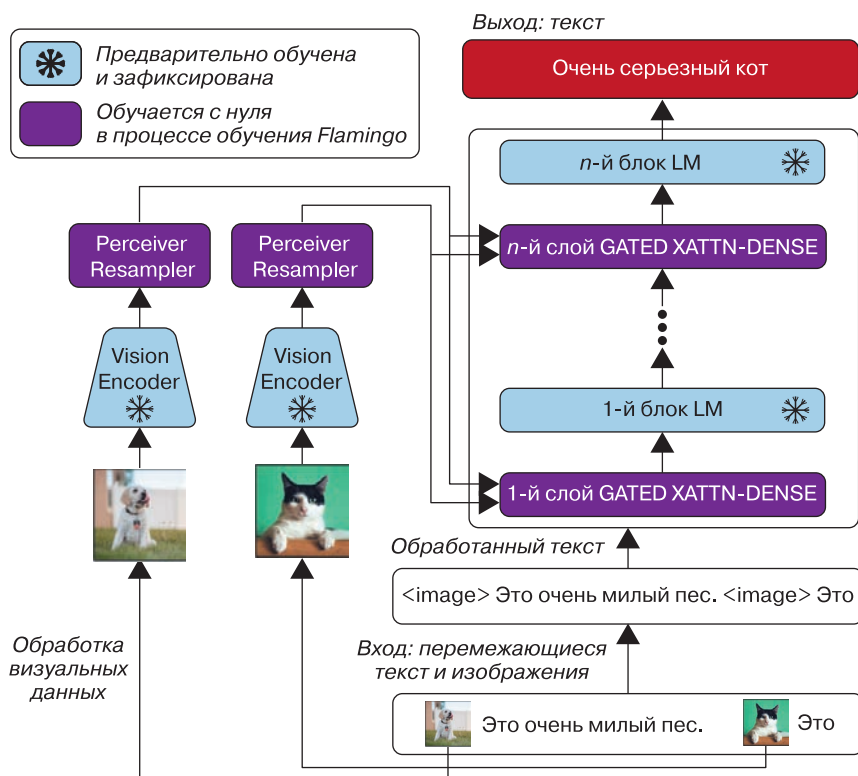


Рис. 13.22. Архитектура Flamingo
(источник: [Alayrac et al., 2022], <https://arxiv.org/pdf/2204.14198.pdf>)

Vision Encoder

Первое отличие модели Flamingo от моделей преобразования текста в изображение, таких как DALL·E 2 и Imagen, заключается в том, что Flamingo может принимать комбинацию перемежающихся текстовых и визуальных данных. В роли визуальных данных могут выступать как видео, так и изображения.

Задача Vision Encoder заключается в преобразовании входных визуальных данных в векторные представления, подобно тому как это делает кодировщик изображений в CLIP. Vision Encoder в Flamingo — это предварительно обученная сеть ResNet без нормализаторов (Normalizer-Free ResNet, NFNet), представленная Брокком (Brock) и его коллегами в 2021 году [Brock et al., 2021], — в частности, NFNet-F6 (модели NFNet варьируются в диапазоне от F0 до F6, увеличиваясь в размерах и мощности). Это одно из ключевых различий между кодировщиком изображений CLIP и Flamingo Vision Encoder: первый использует архитектуру ViT, второй — архитектуру ResNet.

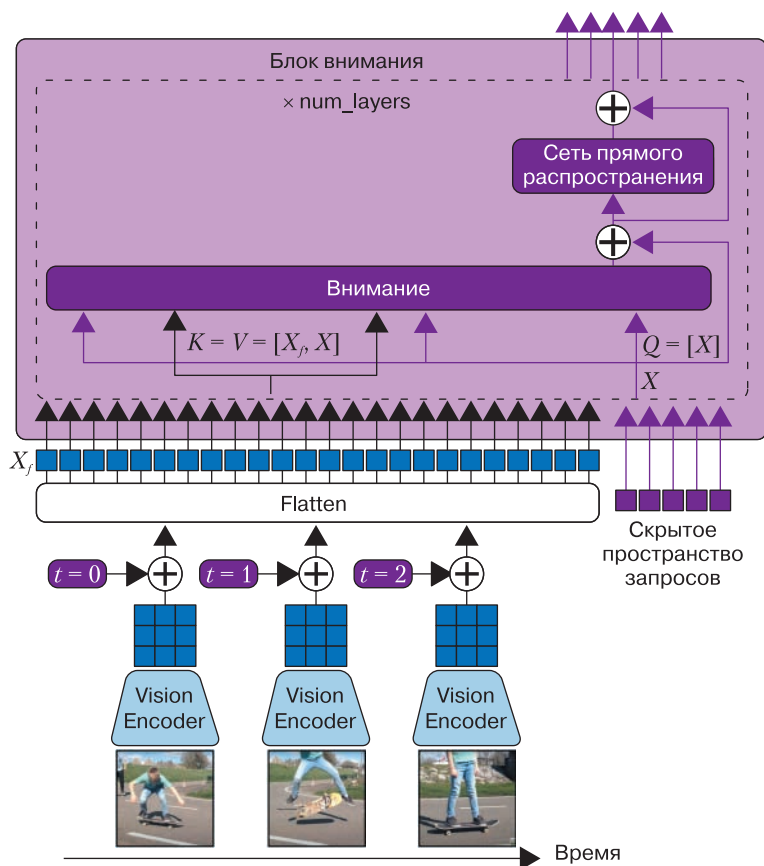
Vision Encoder обучается на парах «изображение — текст» с той же целью выявления различий, которая представлена в статье с описанием CLIP. После обучения модели ее веса фиксируются, чтобы дальнейшее обучение модели Flamingo не влияло на веса Vision Encoder.

На выходе Vision Encoder выводит двумерную сетку объектов, которая перед передачей в Perceiver Resampler преобразуется в одномерный вектор. Видео обрабатывается путем выборки изображений со скоростью 1 кадр в секунду с последующей передачей каждого снимка через Vision Encoder для создания нескольких сеток признаков: полученные векторные представления добавляются в сетки, те преобразуются в одномерные векторы и объединяются в один вектор.

Perceiver Resampler

Потребление памяти в традиционном кодировщике трансформера, таком как BERT, находится в квадратичной зависимости от длины входной последовательности, поэтому входные последовательности обычно ограничиваются определенным количеством значений, например, 512 в BERT. Однако из-за разных разрешений входных изображений и разного количества кадров в видеороликах Vision Encoder выводит вектор переменной длины, поэтому они могут получаться очень длинными.

Архитектура модели Perceiver специально разработана для эффективной обработки длинных входных последовательностей. Она работает не с полной входной последовательностью, а со скрытым вектором фиксированной длины и использует входную последовательность только в механизме перекрестного внимания. В частности, в Flamingo Resampler Perceiver *ключ* и *значение* — это конкатенация входной последовательности и скрытого вектора, *запрос* — только скрытый вектор. Схема обработки видеоданных в Vision Encoder и Perceiver Resampler показана на рис. 13.23.



```
def perceiver_resampler(
    x_f, # Визуальные признаки [T, S, d] (T=время, S=место)
    time_embeddings, # Позиционно-временные представления [T, 1, d]
    x, # R изучаемых скрытых пространств формы [R, d]
    num_layers, # Количество слоев
):
    """Модель Perceiver Resampler"""

    # Добавить и сгладить позиционно-временные представления
    x_f = x_f + time_embeddings
    x_f = flatten(x_f) # [T, S, d] -> [T * S, d]
    # Применить слои Perceiver Resampler
    for i in range(num_layers)
        # Блок внимания.
        x = x + attention_i(q=x, kv=concat([x_f, x]))
        # Прямое распространение.
        x = x + ffw_i(x)
    return x
```

Рис. 13.23. Обработка видеоданных в Perceiver Resampler
(источник: [Alayrac et al., 2022], <https://arxiv.org/pdf/2204.14198.pdf>)

На выход Perceiver Resampler выводит скрытый вектор фиксированной длины, который передается в языковую модель.

Языковая модель

Языковая модель Language Model состоит из нескольких следующих друг за другом блоков в стиле трансформера-декодировщика, которые выводят предсказанное продолжение текста. Фактически большая часть этой модели заимствована из предварительно обученной модели DeepMind под названием *Chinchilla*. В статье с описанием Chinchilla, опубликованной в марте 2022 года [Hoffmann et al., 2022], демонстрируется языковая модель, которая значительно меньше своих аналогов (сравните 70 млрд параметров в Chinchilla со 170 млрд в GPT-3), но обучена на значительно большем количестве лексем. Авторы показывают, что в ряде задач их модель превосходит более крупные модели, подчеркивая тем самым важность компромисса между обучением крупной модели и использованием большего количества лексем для обучения.

Важный вклад, внесенный статьей о Flamingo, — демонстрация возможности адаптировать Chinchilla для обработки дополнительных визуальных данных (X), перемежающихся языковыми данными (Y). Сначала посмотрим, как входные текстовые и визуальные данные объединяются с целью получения входных данных для языковой модели Language Model (рис. 13.24).

Сначала обрабатывается только текст, для чего визуальные данные, например изображения, заменяются тегами <image>, а текст разбивается на *фрагменты* с помощью тега <EOS> (end of chunk — «конец фрагмента»). Каждый фрагмент содержит не более одного изображения, которое всегда находится в его начале, то есть предполагается, что последующий текст относится только к этому изображению. Начало последовательности отмечается тегом <BOS> (beginning of sentence — «начало предложения»).

Затем последовательность лексемизируется, и каждой лексеме присваивается индекс (phi), соответствующий индексу предыдущего изображения (или 0, если во фрагменте нет начального изображения). Как следствие, путем маскировки можно принудительно создать перекрестные связи между текстовыми лексемами (Y) и соответствующими им изображениями (X). Например, на рис. 13.24 первый фрагмент не содержит изображения, поэтому все изображения, получаемые из Perceiver Resampler, оказываются замаскированными. Второй фрагмент содержит изображение 1, поэтому лексемы с этим индексом могут быть связаны с элементами изображения 1. Аналогично последний фрагмент содержит изображение 2, поэтому лексемы с этим индексом могут быть связаны с элементами изображения 2.

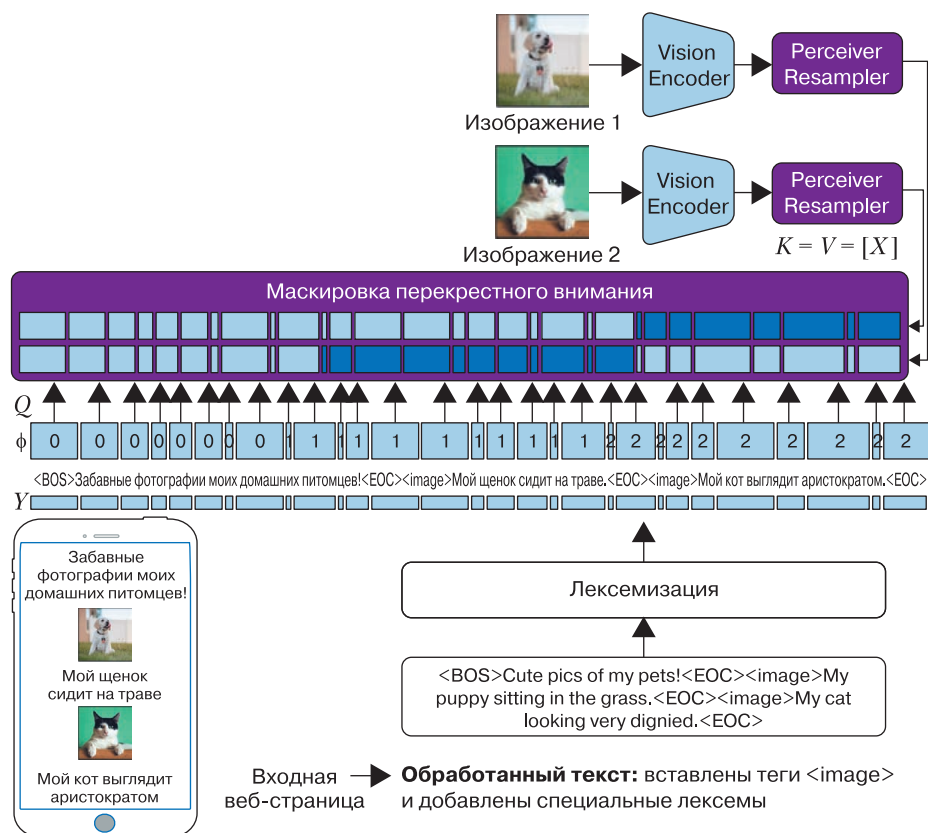
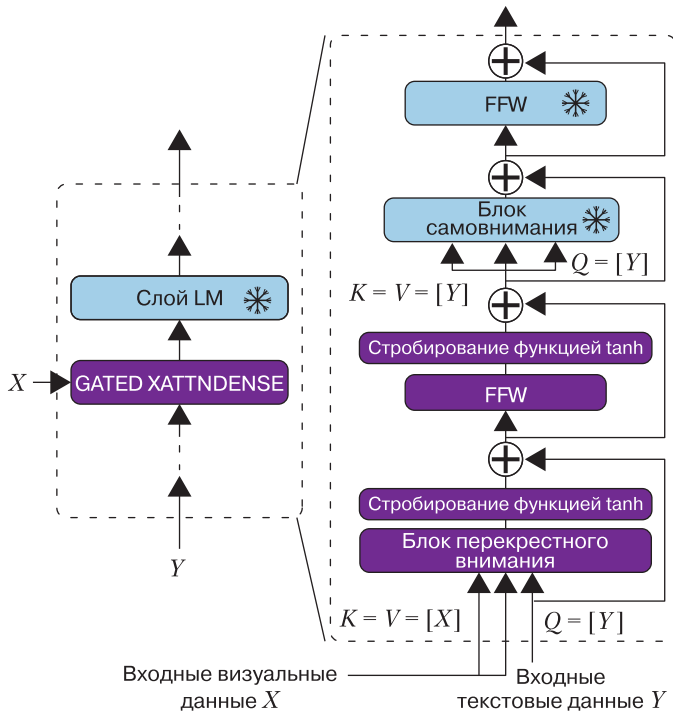


Рис. 13.24. Маскировка перекрестного внимания (XATTN), объединение визуальных и текстовых данных — замаскированные элементы обозначены светло-синим цветом, незамаскированные — темно-синим (источник: [Alayrac et al., 2022], <https://arxiv.org/pdf/2204.14198.pdf>)

Теперь посмотрим, как этот компонент маскировки перекрестного внимания вписывается в общую архитектуру Language Model (рис. 13.25).

Синие компоненты слоя LM — это зафиксированные слои, заимствованные из Chinchilla. Они не обновляются в процессе обучения. Фиолетовые слои GATED XATTN-DENSE обучаются как часть Flamingo и включают компоненты маскировки перекрестного внимания, смешивающие текстовую и визуальную информацию. Далее следуют слои прямого распространения (полносвязные).

Слой GATED XATTN-DENSE называется *gated* — стробирующим или коммутирующим, потому что пропускает выходные данные из компонентов перекрестного внимания и прямого распространения через два отдельных входа $\tanh$, инициализируемых нулевыми значениями. По этой причине сразу после инициализации слои GATED XATTN-DENSE не действуют и текстовая информация передается напрямую.



```
def gated_xattn_dense(
    y, # входные текстовые данные
    x, # входные визуальные данные
    alpha_xattn, # параметр стробирования перекрестного внимания – инициализируется 0
    alpha_dense, # параметр стробирования прямого распространения – инициализируется 0
):
    """Применяет слой GATED XATTN-DENSE"""

    # 1. Стробирование слоя перекрестного внимания
    y = y + tanh(alpha_xattn) * attention(q=y, kv=x)
    # 2. Стробирование слоя прямого распространения
    y = y + tanh(alpha_dense) * ffw(y)

    # Применение блоков самовнимания и прямого распространения к текстовым данным
    y = y + frozen_attention(q=y, kv=y)
    x = x + frozen_ffw(y)
    return x # вывод визуального представления текстовой информации
```

Рис. 13.25. Блок Language Model в модели Flamingo включает слой языковой модели с зафиксированными весами из модели Chinchilla и слой GATED XATTN-DENSE (источник: [Alayrac et al., 2022], <https://arxiv.org/pdf/2204.14198.pdf>)

Параметры стробирования α изучаются сетью в процессе обучения, чтобы обеспечить эффективное смешивание текстовой и визуальной информации.

Примеры использования Flamingo

Модель Flamingo можно применять для разных целей, включая изучение изображений и видео и диалоговое общение. На рис. 13.26 показано несколько примеров того, на что способна Flamingo.

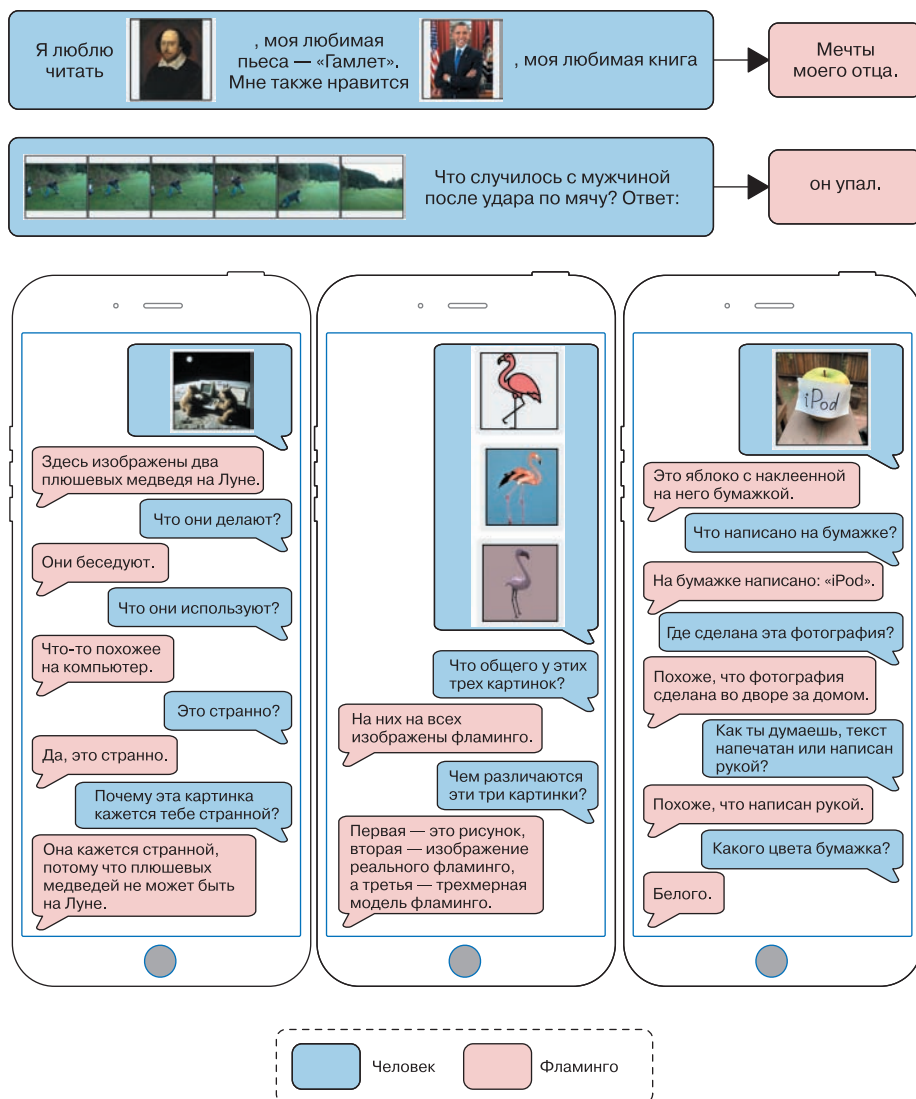


Рис. 13.26. Примеры входных и выходных данных модели Flamingo с 80 млрд параметров (источник: [Alayrac et al., 2022], <https://arxiv.org/pdf/2204.14198.pdf>)

Обратите внимание на то, что в каждом примере Flamingo смешивает текстовую и визуальную информацию в настоящем мультимодальном стиле. В первом примере вместо некоторых слов используются изображения, и в продолжение диалога модель подсказывает подходящую книгу¹. Во втором примере показаны кадры из видео, и Flamingo правильно определяет последствия действий мужчины. Последние три примера демонстрируют, что Flamingo можно применить в интерактивном режиме и получить от нее дополнительную информацию путем диалога в форме «вопрос/ответ».

Удивительно видеть способность машины отвечать на сложные вопросы в таком широком диапазоне модальностей и входных данных. В статье авторы количественно оценивают возможности Flamingo на ряде тестовых задач и отмечают, что во многих тестах она способна превзойти модели, специально разработанные для решения одной конкретной задачи. Это наглядно демонстрирует, что большие мультимодальные модели могут быстро адаптироваться для решения широкого круга задач и тем самым открывается путь для разработки агентов ИИ, которые не просто специализированы для решения одной задачи, а являются по-настоящему универсальными агентами, выводом которых можно управлять.

Резюме

В этой главе мы исследовали четыре современные мультимодальные модели: DALL.E 2, Imagen, Stable Diffusion и Flamingo.

DALL.E 2 — это созданная в OpenAI крупномасштабная модель преобразования текста в изображение, способная генерировать реалистичные изображения в разных стилях на основе текстового описания. Она объединяет предварительно обученные модели, такие как CLIP, с созданными ранее архитектурами диффузионных моделей (GLIDE). Ее дополнительные возможности — редактирование изображений с помощью текстовых подсказок и создание вариантов данного изображения. DALL.E 2 имеет некоторые ограничения, например непоследовательное отображение текста и привязку атрибутов, тем не менее это невероятно мощная модель искусственного интеллекта, которая помогла приблизить наступление новой эры генеративного моделирования.

Модель, превзошедшая предыдущую, — это Imagen, созданная в Google Brain. Она имеет много общего с DALL.E 2, например текстовый кодировщик и диффузионную модель декодировщика. Одно из ключевых различий этих двух моделей — текстовый кодировщик в Imagen обучается исключительно на текстовых

¹ «Мечты моего отца» (Dreams from My Father) — первая книга Барака Обамы, вышедшая в 1995 году. — *Примеч. пер.*

данных, а обучение текстового кодировщика в DALL·E 2 проводилось также на изображениях (в виде контрастивной цели обучения CLIP). С помощью своего тестового пакета DrawBench авторы показали, что этот подход дает результаты высочайшего качества при выполнении целого ряда задач.

Stable Diffusion — это модель с открытым исходным кодом, созданная в Stability AI, CompVis и Runway. Код и веса этой модели преобразования текста в изображение находятся в свободном доступе, поэтому вы можете запустить ее на своем компьютере. Stable Diffusion получилась быстрой и легковесной благодаря использованию модели скрытой диффузии, которая работает со скрытым пространством автокодировщика, а не с самими изображениями.

Наконец, Flamingo, созданная в DeepMind, — это визуально-языковая модель. Она принимает поток перемежающихся текстовых и визуальных данных (изображений и видео) и способна поддерживать диалог, получая дополнительные текстовые вопросы. Эта модель показала, что визуальная информация может передаваться в трансформер через модели Visual Encoder и Perceiver Resampler, которые преобразуют признаки из визуальных данных в небольшое количество визуальных лексем. Сама языковая модель Language Model является расширением более ранней модели Chinchilla, также созданной в DeepMind и адаптированной для смешивания визуальной и текстовой информации.

Все четыре модели наглядно демонстрируют широту возможностей мультимодальных моделей. Весьма вероятно, что в будущем генеративное моделирование станет более мультимодальным, а модели ИИ смогут совмещать модальности и задачи посредством интерактивных языковых подсказок.

Заключение

В этой главе вы:

- узнаете историю генеративного искусственного интеллекта с 2014 года по сегодняшний день, включая хронологию ключевых моделей и разработок;
- узнаете текущее состояние генеративного искусственного интеллекта, включая темы, доминирующие в этой сфере;
- познакомитесь с моими прогнозами о будущем генеративного искусственного интеллекта и о том, как он повлияет на повседневную жизнь, работу и образование;
- узнаете о важных этических и практических проблемах, с которыми столкнется генеративный искусственный интеллект в будущем;
- прочитаете мои соображения о значимости генеративного ИИ и его способности произвести революцию в поисках общего искусственного интеллекта.

В мае 2018 года я приступил к работе над первым изданием этой книги. Спустя пять лет я еще больше воодушевлен безграничными возможностями и потенциальным влиянием генеративного ИИ.

За это время мы увидели невероятный прогресс в этой области, сулящий безграничный потенциал реальных приложений. Меня переполняют трепет и удивление достигнутым, и я с нетерпением жду возможности увидеть, как повлияет генеративный ИИ на мир в ближайшие годы. Генеративное глубокое обучение способно формировать будущее так, как мы даже не можем себе представить.

Более того, по мере подбора материала для книги я все яснее понимал, что эта область охватывает не только создание изображений, текста или музыки. Я считаю, что дальнейшее развитие генеративного глубокого обучения поможет нам открыть секрет самого интеллекта.

В первом разделе этой главы мы увидим, как был достигнут текущий уровень развития генеративного ИИ. Мы последовательно пройдемся по хронологии развития генеративного ИИ, начав с 2014 года, чтобы вы могли увидеть, какое

место в его истории занимает каждый метод, известный на сегодняшний день. Во втором разделе поговорим о том, где находимся в настоящее время с точки зрения современного генеративного ИИ. Обсудим нынешние тенденции в подходах к генеративному глубокому обучению и современные общедоступные готовые модели. Затем побеседуем о будущем генеративного искусственного интеллекта, а также о возможностях и проблемах, которые ждут нас впереди. Мы представим, как будет выглядеть генеративный ИИ через пять лет и как станет потенциально влиять на общество и бизнес, а также рассмотрим некоторые основные этические и практические проблемы.

Хронология генеративного ИИ

На рис. 14.1 показана история ключевых разработок в области генеративного моделирования, которые мы вместе исследовали в этой книге. Различные цвета представляют разные типы моделей.

Область генеративного ИИ опирается на более ранние разработки в области глубокого обучения, такие как обратное распространение ошибки и сверточные нейронные сети, давшие моделям возможность изучать сложные взаимосвязи в больших наборах данных в любом масштабе. В этом разделе мы исследуем современную историю генеративного ИИ, с 2014 года развивавшуюся с головокружительной скоростью.

Чтобы лучше понять историю, мы условно разобьем ее на три основные эпохи:

- 1) 2014–2017 годы — эпоха VAE и GAN;
- 2) 2018–2019 годы — эпоха трансформеров;
- 3) 2020–2022 годы — эпоха больших моделей.

2014–2017 годы — эпоха VAE и GAN

Изобретение VAE в декабре 2013 года послужило искрой, которая зажгла фитиль генеративного ИИ. В своей статье Дидерик П. Кингма и Макс Веллинг [Kingma, Max Welling, 2013] показали, что в скрытом пространстве, позволяющем плавно перемещаться по нему, можно генерировать не только простые изображения, например цифры MNIST, но и более сложные, такие как лица. В 2014 году вслед за VAE появились GAN — совершенно новый вид состязательных структур для решения задач генеративного моделирования.

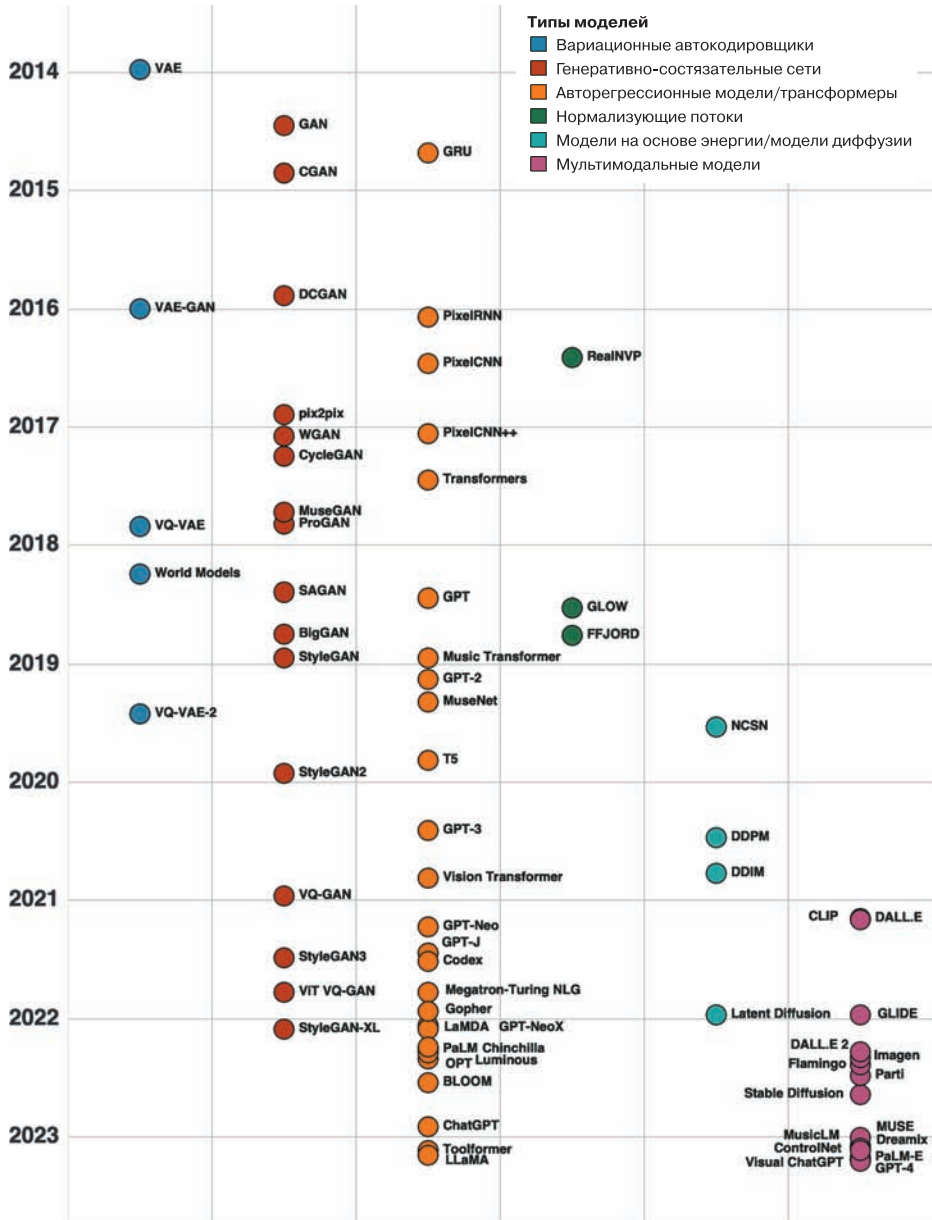


Рис. 14.1. Хронология развития генеративного ИИ с 2014 по 2023 годы (некоторые важные разработки, такие как LSTM и ранние модели на основе энергии, например машины Больцмана, появились раньше, до 2014 года)

Следующие три года были отмечены еще более впечатляющим расширением портфолио GAN. Произошли фундаментальные изменения в архитектуре модели GAN (DCGAN, 2015), в функции потерь (Wasserstein GAN, 2017) и процессе обучения (ProGAN, 2017), также генеративно-состязательные сети проникли в новые области, такие как преобразование изображения в изображение (pix2pix, 2016 и CycleGAN, 2017) и создание музыки (MuseGAN, 2017).

В эту эпоху были предложены важные усовершенствования VAE, такие как VAE-GAN (2015) и VQ-VAE (2017), а в статье *World Models* (2018) рассмотрены варианты практического применения обучения с подкреплением.

Устоявшиеся модели авторегрессии, такие как LSTM и GRU, продолжали доминировать в генерировании текста. Идеи авторегрессии использовались и в моделях создания изображений: PixelRNN (2016) и PixelCNN (2016) были представлены как новые подходы к созданию изображений. Исследовались и другие подходы к генерированию изображений, в результате чего появилась модель RealNVP (2016), проложившая путь для более поздних моделей нормализующих потоков.

В июне 2017 года была опубликована новаторская статья *Attention Is All You Need*, обозначившая начало следующей эпохи генеративного ИИ — эпохи трансформеров.

2018–2019 годы — эпоха трансформеров

В основе трансформера лежит механизм внимания, устраняющий необходимость в рекуррентных слоях, присутствующих в старых авторегрессионных моделях, таких как LSTM. Трансформеры быстро приобрели известность после появления в 2018 году моделей GPT (трансформер-декодировщик) и BERT (трансформер-кодировщик). В следующем году были созданы еще более крупные языковые модели, которые превосходно справлялись с широким кругом задач, рассматривая их как задачи преобразования текста в текст. Яркими примерами этих моделей могут служить GPT-2 (2018, 1,5 млрд параметров) и T5 (2019, 11 млрд параметров).

Также появились первые успешные модели трансформера для создания музыки: Music Transformer (2018) и MuseNet (2019).

За эти два года были выпущены также несколько впечатляющих GAN, закрепивших эту технологию на передовых позициях в области генерирования изображений. В частности, SAGAN (2018) и более крупная BigGAN (2018) включили механизм внимания в структуру GAN и показали потрясающие результаты, а StyleGAN (2018) и StyleGAN2 (2019) продемонстрировали, что можно генерировать изображения, удивительно точно управляя стилем и содержанием конкретного изображения.

Еще одной областью генеративного ИИ, набравшей обороты, были модели на основе оценок (NCSN, 2019), которые в конечном итоге проложили путь к следующему революционному сдвигу в ландшафте генеративного ИИ — появлению диффузионных моделей.

2020–2022 годы — эпоха больших моделей

В эту эпоху появились несколько моделей, объединивших концепции из разных семейств генеративных моделей и усовершенствовавших существующие архитектуры. Например, модель VQ-GAN (2020) включила дискриминатор GAN в архитектуру VQ-VAE, а Vision Transformer (2020) показала, что можно обучить трансформер работать с изображениями. В 2022 году была выпущена модель StyleGAN-XL — обновление архитектуры StyleGAN, — позволяющая генерировать изображения размером 1024×1024 пиксела.

В 2020 году были представлены две модели, заложившие основу для всех будущих моделей генерирования больших изображений: DDPM и DDIM. Совершенно неожиданно диффузионные модели оказались серьезными конкурентами генеративно-состязательных сетей с точки зрения качества генерируемых изображений, о чем прямо сказано в названии опубликованной в 2021 году статьи *Diffusion Models Beat GANs on Image Synthesis*. Диффузионные модели генерируют изображения невероятно высокого качества, и для их обучения требуется только одна сеть U-Net вместо комбинации двух сетей в GAN, что делает процесс обучения намного более стабильным.

Примерно в то же время была выпущена GPT-3 (2020) — огромная модель трансформера с 175 млрд параметров, способная генерировать текст практически на любую тему, что кажется почти невозможным. Доступ к модели был реализован в виде веб-приложения и API, что позволило компаниям создавать на ее основе свои продукты и услуги. ChatGPT (2022) — это веб-приложение и API-обертка для последней версии GPT, позволяющие пользователям вступать в естественный диалог с ИИ на любую тему.

В 2021 и 2022 годах было выпущено множество других крупных языковых моделей, конкурирующих с GPT-3, в том числе Megatron-Turing NLG (2021) от Microsoft и NVIDIA, Gopher (2021) и Chinchilla от DeepMind (2022), LaMDA (2022) и PaLM (2022) от Google и Luminous (2022) от Aleph Alpha. Также появились модели с открытым исходным кодом, такие как GPT-Neo (2021), GPT-J (2021) и GPT-NeoX (2022) от EleutherAI, модель OPT от Meta с 66 млрд параметров (2022), доработанная модель Flan-T5 (2022) от Google, BLOOM (2022) от Hugging Face и др. Все они используют некоторую разновидность трансформера, обученного на огромном массиве данных.

Быстрый рост популярности мощных трансформеров в сфере генерирования текста и современных диффузионных моделей в области генерирования изображений привел к тому, что основное внимание при разработке генеративного ИИ в течение последних двух лет сосредоточилось на мультимодальных моделях, то есть моделях, работающих более чем в одной области, например, преобразующих текст в изображение.

Эта тенденция оформилась в 2021 году, когда OpenAI выпустила DALL·E — модель преобразования текста в изображение, основанную на дискретном VAE (подобном VQ-VAE) и CLIP (модель трансформера, предсказывающая пары «изображение/текст»). За ней последовали GLIDE (2021) и DALL·E 2 (2022) с обновленной генеративной частью, использующей диффузионную модель вместо дискретного VAE и показавшей впечатляющие результаты. В эту эпоху также появились три модели преобразования текста в изображение от Google: Imagen (2022, на основе трансформера и диффузионных моделей), Parti (2022, на основе трансформера и модели ViT-VQGAN), а затем MUSE (2023, на основе трансформера и VQ-GAN). DeepMind также выпустила визуально-языковую модель Flamingo (2022), основанную на большой языковой модели Chinchilla, которая позволяет передавать изображения как часть запроса.

Еще одним важным достижением развития диффузионных моделей, представленным в 2021 году, стала скрытая диффузия, когда диффузионная модель обучается в скрытом пространстве автокодировщика. Этот метод был положен в основу модели Stable Diffusion, выпущенной в сотрудничестве компаниями Stability AI, CompVis и Runway в 2022 году. В отличие от DALL·E 2, Imagen и Flamingo, веса и код модели Stable Diffusion открыты для всех желающих, то есть любой сможет запустить модель на своем компьютере.

Текущее состояние генеративного ИИ

Подойдя к концу путешествия по истории генеративного ИИ, важно задуматься о том, где мы находимся сейчас с точки зрения приложений и моделей. Давайте воспользуемся моментом и оценим прогресс и ключевые достижения в этой области на сегодняшний день.

Большие языковые модели

В настоящее время при разработке генеративного ИИ для текста почти все усилия сосредоточены на построении больших языковых моделей (Large Language Models, LLM), единственная цель которых — напрямую моделировать язык на основе огромного корпуса текста, то есть их обучают предсказывать следующее слово в стиле трансформера-декодировщика.

Подход с применением большой языковой модели получил широкое распространение благодаря своей гибкости и способности решать широкий круг задач. Одну и ту же модель можно задействовать и для ответов на вопросы, и для обобщения текста, и для создания нового текста, и для решения множества других задач, поскольку каждый вариант использования можно представить как задачу преобразования текста в текст, когда конкретные инструкции, описывающие задачу (подсказка), передаются в модель как часть входных данных.

В качестве примера возьмем GPT-3. На рис. 14.2 показано, как одну и ту же модель можно задействовать и для обобщения текста, и для создания нового текста.

Напиши вступительный абзац для учебника о генеративном искусственном интеллекте, заканчивающийся захватывающим моментом:

«Представьте себе мир, где искусственный интеллект не просто следует заранее запрограммированным алгоритмам, а обладает способностью к творчеству, подобно человеку. Этот учебник — ваш проводник в мир генеративного искусственного интеллекта, где границы между человеком и машиной становятся все более размытыми. Здесь вы узнаете, как разрабатывать интеллектуальные системы, способные к автономному обучению, анализу и созданию данных, текстов, изображений, музыки и даже видео.

Но это не просто книга о теории и практике генеративного ИИ. Это учебник с изюминкой. На его страницах мы вместе с вами создадим с нуля интеллектуальную систему, которая будет обучаться на огромных объемах данных и генерировать оригинальные и захватывающие идеи. Мы пройдем через каждый этап этого процесса, от сбора и обработки данных до разработки и тестирования нашего генератора идей. И вот когда вы перевернете последнюю страницу, вы окажетесь на пороге удивительного мира генеративного искусственного интеллекта — мира, где машины творят наравне с человеком, а возможно, и лучше».

Напиши одну строфу рифмованного стихотворения к следующим абзацам:

«До недавнего времени основные успехи в машинном обучении были обусловлены достижениями в дискриминативном моделировании. Это связано с тем, что для любой дискриминативной задачи решить соответствующую задачу генеративного моделирования намного сложнее. Например, гораздо проще научить модель предсказывать, написана ли картина Ван Гогом, чем научить ее создавать совершенно новые картины в стиле Ван Гога.

Точно так же гораздо проще обучить модель прогнозировать, была ли страница текста написана Чарльзом Диккенсом, чем построить модель, генерирующую последовательность абзацев в стиле Диккенса. До недавнего времени большинство генеративных задач были просто недостижимы, и многие сомневались, что их вообще удастся решить. Творчество считалось чисто человеческой способностью, с которой не мог конкурировать ИИ».

До недавнего прошлого знания глубоки,
Дискриминативные модели лишь расцвели.
Генерация? Сложно, спору нет,
В сравнении с Ван Гогом Диккенс — легкий свет.

Рис. 14.2. Вывод GPT-3: невыделенный текст — это запрос, а выделенный — ответ GPT-3¹

¹ Думаю, что нашим читателям будет интереснее видеть результаты на русском языке. Для подготовки рис. 14.2 я вводил запросы на русском языке в ChatGPT и получал такие ответы. — *Примеч. пер.*

Обратите внимание на то, что в обоих случаях запрос содержит соответствующие инструкции. Задача модели GPT-3 — просто продолжить содержимое запроса, добавляя лексему за лексемой. У нее нет базы данных фактов, где она могла бы искать информацию, или фрагментов текста, которые могла бы копировать в свои ответы. Она должна лишь предсказать, какая лексема с наибольшей вероятностью будет следовать за существующими лексемами, а затем добавить ее в конец запроса и перейти к созданию следующей лексемы.

Невероятно, но этого простого дизайна достаточно, чтобы языковая модель справилась с рядом задач (см. рис. 14.2). Более того, он придает языковой модели потрясающую гибкость в генерировании реалистичного текста в ответ на любой запрос, какой только можно вообразить!

На рис. 14.3 показано, как увеличились размеры больших языковых моделей с момента публикации оригинальной статьи с описанием модели GPT в 2018 году. Количество параметров росло экспоненциально до конца 2021 года, когда появилась модель Megatron-Turing NLG с 530 млрд параметров. В последнее время больше внимания уделяется созданию более эффективных языковых

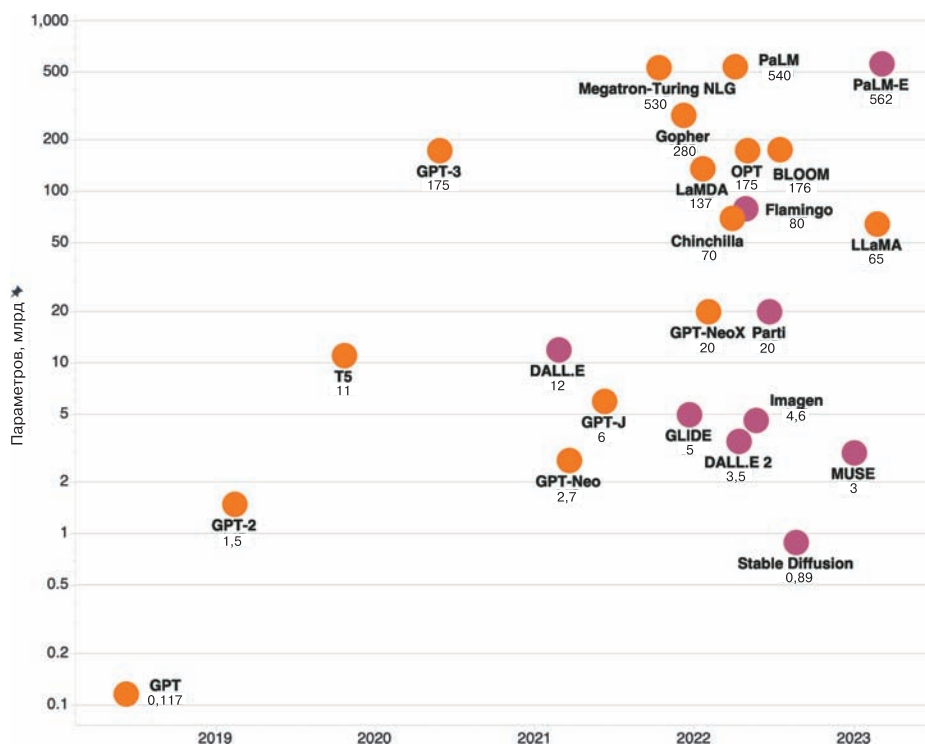


Рис. 14.3. Размеры больших языковых (оранжевые) и мультимодальных (розовые) моделей по количеству параметров

моделей, использующих меньше параметров, потому что крупные модели обходятся дороже и сложнее в обслуживании в промышленной среде.

Коллекция моделей GPT (GPT-3, GPT-3.5, GPT-4 и т. д.) по-прежнему считается набором самых мощных и современных языковых моделей из доступных для личного и коммерческого использования с помощью веб-приложения и API.

Еще одно недавнее пополнение в большом семействе языковых моделей — *Large Language Model Meta AI* (LLaMA), созданная в Meta [Touvron et al., 2023]. Это целый набор моделей с количеством параметров от 7 до 65 млрд, которые обучаются исключительно на общедоступных наборах данных.

В табл. 14.1 приводится краткое описание некоторых из наиболее мощных LLM. Часть из них, например LLaMA, на самом деле являются семействами моделей разных размеров — в таких случаях в таблице приводится размер самой большой модели. Код и веса некоторых моделей опубликованы и доступны всем желающим, а это означает, что любой может свободно их использовать и развивать.

Таблица 14.1. Большие языковые модели

Модель	Дата	Разработчик	Число параметров, млрд	Свободно-доступна
GPT-3	Май 2020	OpenAI	175	Нет
GPT-Neo	Март 2021	EleutherAI	2 700 000 000	Да
GPT-J	Июнь 2021	EleutherAI	6	Да
Megatron-Turing NLG	Октябрь 2021	Microsoft & NVIDIA	530	Нет
Gopher	Декабрь 2021	DeepMind	280	Нет
LaMDA	Январь 2022	Google	137	Нет
GPT-NeoX	Февраль 2022	EleutherAI	20	Да
Chinchilla	Март 2022	DeepMind	70	Нет
PaLM	Апрель 2022	Google	540	Нет
Luminous	Апрель 2022	Aleph Alpha	70	Нет
OPT	Май 2022	Meta	175	Да (66 млрд)
BLOOM	Июль 2022	Hugging Face collaboration	175	Да
Flan-T5	Октябрь 2022	Google	11	Да
GPT-3.5	Ноябрь 2022	OpenAI	Неизвестно	Нет
LLaMA	Февраль 2023	Meta	65	Нет
GPT-4	Март 2023	OpenAI	Неизвестно	Нет

Несмотря на впечатляющие возможности, большие языковые модели все еще страдают серьезными проблемами, которые необходимо решить. В частности, они склонны выдумывать факты и не способны на логические мыслительные процессы (рис. 14.4).

Каков рекорд скорости пересечения Атлантического океана пешком?

Рекорд пересечения Атлантического океана пешком был установлен Эндрю Тауном в 2020 году и составил 42 дня 5 часов и 12 минут.

Когда мне исполнилось 8 лет, я был вдвое старше сестры. Сколько лет мне сейчас, если сестре 70?

Сейчас вам $70 \times 2 = 140$ лет.

Рис. 14.4. Большие языковые модели прекрасно справляются с некоторыми задачами, но они также склонны к ошибкам в фактических или логических рассуждениях (здесь показан вывод GPT-3)

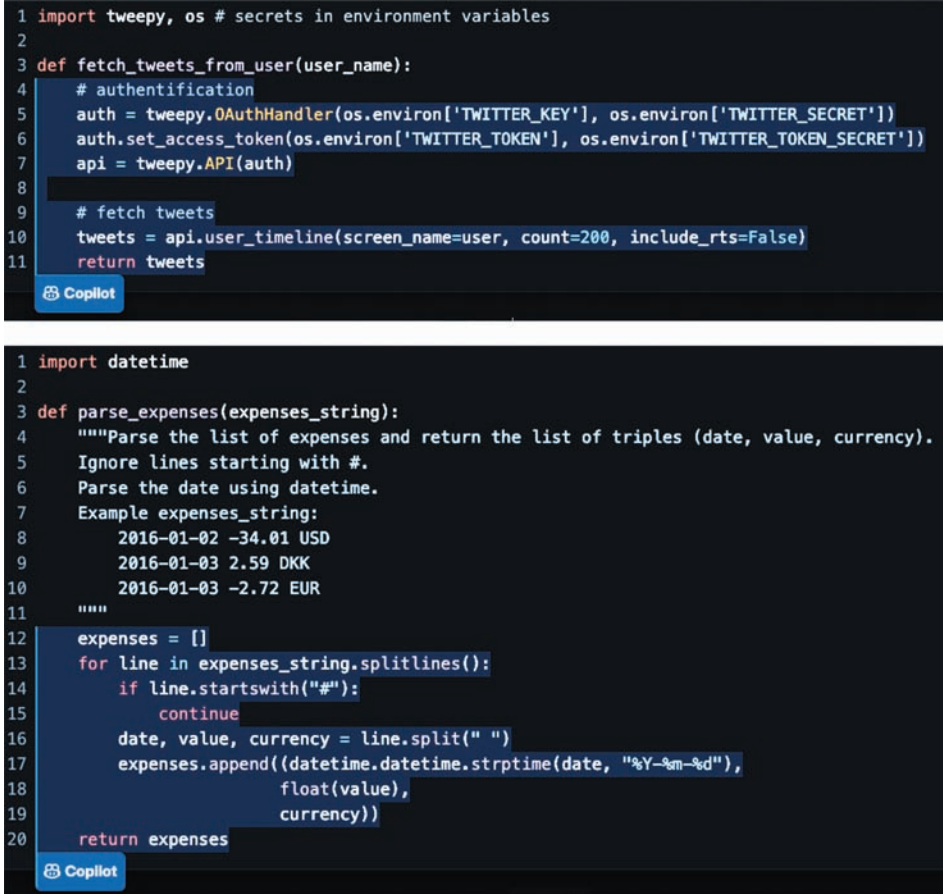
Важно помнить, что LLM обучены предсказывать лишь следующее слово. Они не имеют никакой связи с реальностью, которая позволила бы им достоверно выявлять фактические или логические заблуждения. Поэтому следует быть крайне осторожными, рассматривая вопрос о применении этих мощных моделей прогнозирования текста в промышленном окружении — они пока недостаточно надежны в областях, где важна точность рассуждений.

Модели преобразования текста в программный код

Еще одна сфера применения больших языковых моделей — генерирование программного кода. В июле 2021 года OpenAI представила модель *Codex* — языковую модель GPT, доработанную на основе кода с GitHub [Chen et al., 2021]. Эта модель успешно справляется с реализацией решений ряда задач в программном коде, требуя только описания задачи или имени функции. Современная технология основана на GitHub Copilot (<https://oreil.ly/P5WXo>) — реализации искусственного интеллекта, которую можно использовать для разработки программного кода в диалоговом режиме. Copilot — это платный сервис с бесплатным пробным периодом.

На рис. 14.5 показаны два примера автоматически сгенерированных вариантов завершения заданного фрагмента программного кода. Первый пример — это

функция, извлекающая твиты с помощью Twitter API. Опираясь на имя функции и ее параметры, Copilot может автоматически закончить определение функции. Во втором примере модели Copilot предложено проанализировать содержимое списка `expenses` по текстовому комментарию, объясняющему формат входного параметра и саму задачу. Copilot может автоматически реализовать функцию, используя только описание.



```

1 import tweepy, os # secrets in environment variables
2
3 def fetch_tweets_from_user(user_name):
4     # authentication
5     auth = tweepy.OAuthHandler(os.environ['TWITTER_KEY'], os.environ['TWITTER_SECRET'])
6     auth.set_access_token(os.environ['TWITTER_TOKEN'], os.environ['TWITTER_TOKEN_SECRET'])
7     api = tweepy.API(auth)
8
9     # fetch tweets
10    tweets = api.user_timeline(screen_name=user, count=200, include_rts=False)
11    return tweets

```

```

1 import datetime
2
3 def parse_expenses(expenses_string):
4     """Parse the list of expenses and return the list of triples (date, value, currency).
5     Ignore lines starting with #.
6     Parse the date using datetime.
7     Example expenses_string:
8         2016-01-02 -34.01 USD
9         2016-01-03 2.59 DKK
10        2016-01-03 -2.72 EUR
11    """
12    expenses = []
13    for line in expenses_string.splitlines():
14        if line.startswith("#"):
15            continue
16        date, value, currency = line.split(" ")
17        expenses.append((datetime.datetime.strptime(date, "%Y-%m-%d"),
18                        float(value),
19                        currency))
20    return expenses

```

Рис. 14.5. Два примера, демонстрирующие возможности GitHub Copilot
(источник: GitHub Copilot)

Эта замечательная технология уже начинает менять используемые программистами подходы к решению конкретных задач. Значительную часть рабочего времени программист обычно тратит на поиск примеров существующих решений, чтение форумов сообщества, таких как Stack Overflow, и поиск подходящего

синтаксиса в документации пакета. Это означает выход из интерактивной среды разработки (IDE), в которой пишется код, переключение на веб-браузер, копирование и вставку фрагментов кода из Интернета и проверку работоспособности найденного решения. Copilot во многих случаях избавляет от необходимости делать все это, и вы можете просто просмотреть возможные решения, созданные ИИ в среде IDE, написав лишь краткое описание желаемого результата.

Модели преобразования текста в изображение

В современном мире генерирования изображений доминируют большие мультимодальные модели, преобразующие текстовое описание в изображение. Модели преобразования текста в изображение очень полезны, поскольку позволяют пользователям легко манипулировать создаваемыми изображениями с помощью естественного языка. В этом их главное отличие от таких моделей, как StyleGAN, показывающих впечатляющие результаты, но не имеющих текстового интерфейса, с помощью которого можно описать изображение, которое нужно сгенерировать.

Три наиболее примечательные модели преобразования текста в изображение, которые в настоящее время доступны для коммерческого и личного использования, — это DALL·E 2, Midjourney и Stable Diffusion.

DALL·E 2, созданная в OpenAI, доступна как платная услуга с оплатой через веб-приложение и API (<https://labs.openai.com/>). Midjourney (<https://midjourney.com/>) предоставляет услугу преобразования текста в изображение на основе подписки через свой канал Discord. Обе модели, DALL·E 2 и Midjourney, имеют бесплатные подписки для тех, кто присоединяется к платформе в роли тестировщиков ранних выпусков.



Midjourney

Сервис Midjourney использовался для создания иллюстраций к рассказам в части II этой книги!

Отличительная особенность модели Stable Diffusion — она полностью открыта. Веса модели и код для ее обучения доступны на GitHub (<https://oreil.ly/C47vN>), поэтому каждый желающий сможет запустить модель на своем компьютере. Набор данных, используемый для обучения Stable Diffusion, тоже распространяется бесплатно. Этот набор данных, получивший название LAION-5B (<https://oreil.ly/20758>), содержит 5,85 млрд пар «изображение — текст» и в настоящее время является крупнейшим в мире общедоступным набором таких данных.

Одна из важных особенностей Stable Diffusion — возможность построить базовую модель и адаптировать ее к различным вариантам использования. Прекрасной демонстрацией такого подхода может служить ControlNet — структура нейронной сети, позволяющая управлять выходными данными Stable Diffusion с применением дополнительных условий [Zhang, Agrawala, 2023]. Например, выходные изображения можно обусловить картой границ Canny (<https://oreil.ly/8v9Ym>) данного входного изображения (рис. 14.6).

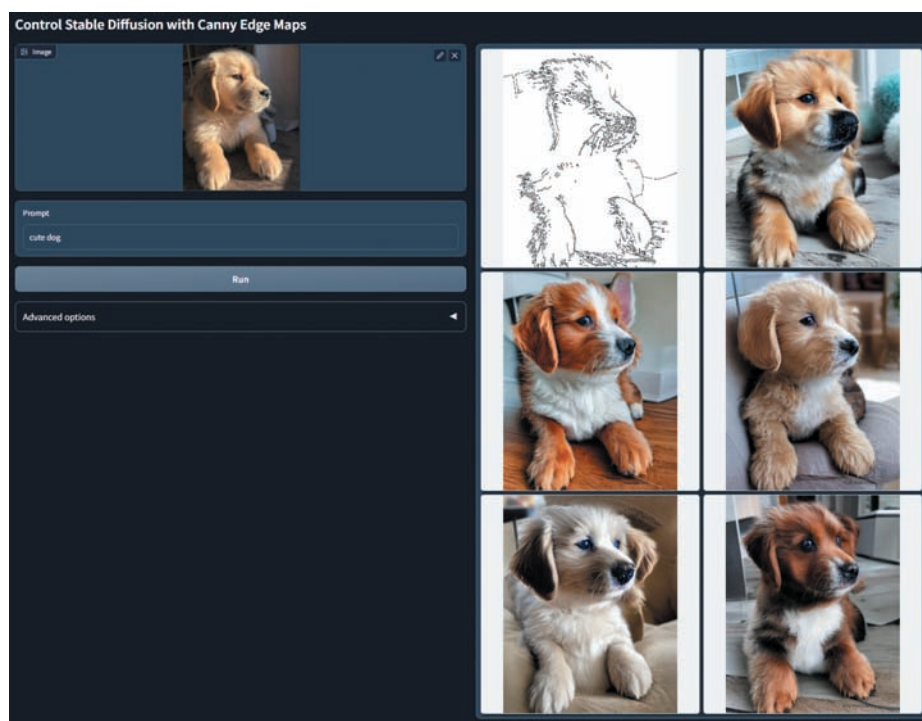


Рис. 14.6. Управление выводом модели Stable Diffusion с применением карты границ Canny и ControlNet (источник: Lvmin Zhang, ControlNet, <https://github.com/lllyasviel/ControlNet>)

ControlNet содержит обучаемую копию кодировщика Stable Diffusion, а также зафиксированную копию полной модели Stable Diffusion (рис. 14.7). Задача обучаемого — научиться обрабатывать входные условия (например, карту границ Canny), при этом зафиксированная копия сохраняет мощь исходной модели. Проще говоря, Stable Diffusion можно настроить для решения конкретных задач, используя небольшое количество пар изображений. Нулевые свертки — это обычные свертки 1×1 , в которых веса и смещения равны нулю, поэтому на начальном этапе перед обучением ControlNet они не дают никакого эффекта.

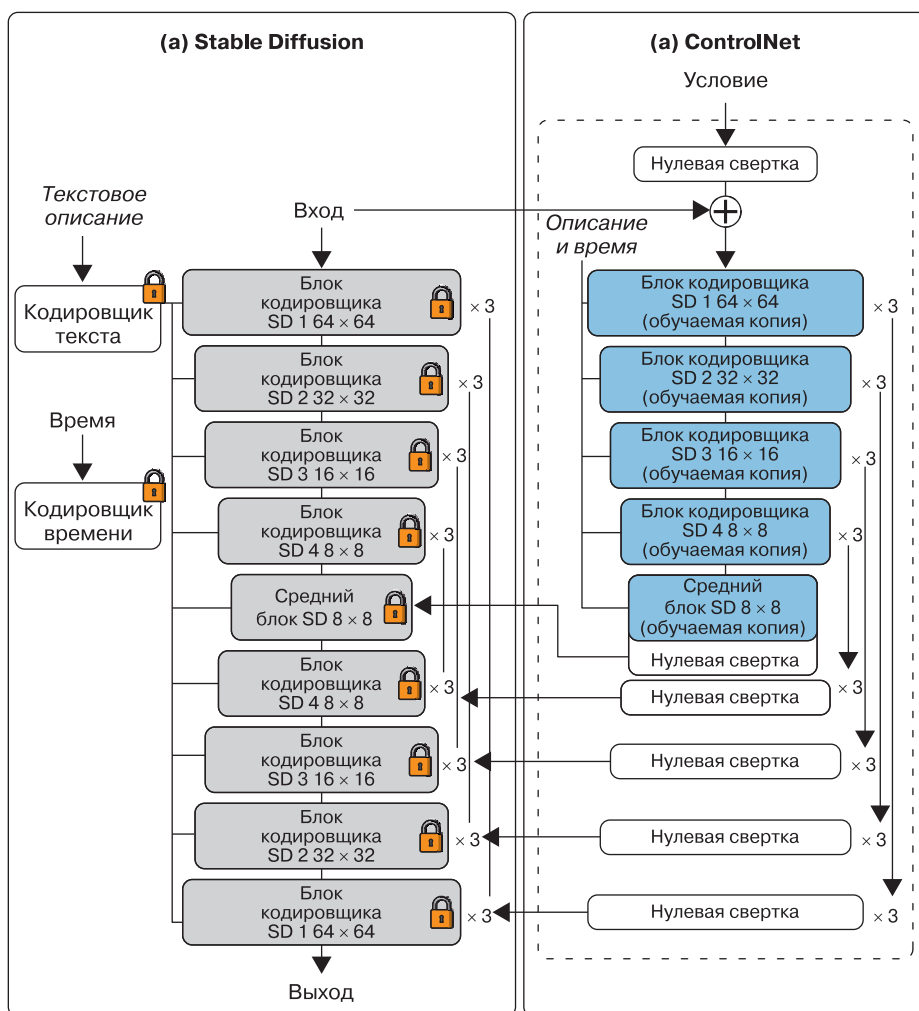


Рис. 14.7. Архитектура ControlNet с обучаемыми копиями кодировщика Stable Diffusion, выделенными синим цветом
(источник: Lvmin Zhang, ControlNet, <https://github.com/lllyasviel/ControlNet>)

Еще одно преимущество Stable Diffusion — ее способность работать на одном графическом процессоре небольшой вычислительной мощности всего с 8 Гбайт видеопамяти. Это позволяет работать на локальном оборудовании, не используя облачные сервисы. Поскольку услуги преобразования текста в изображение включены в последующие продукты, скорость создания становится все более важной. Это одна из причин тенденции к уменьшению размеров мультимодальных моделей (см. рис. 14.3).

На рис. 14.8 можно увидеть примеры вывода всех трех моделей. Все они генерируют потрясающие результаты и способны передать содержимое и стиль рисунка на основе описания.

Единорог, перепрыгивающий через величественную реку, красивая картина маслом, потрясающие природные пейзажи, отмеченное наградами искусство, золотая молния, тонкие детали пейзажа



Рис. 14.8. Изображения, сгенерированные моделями Stable Diffusion v2.1, Midjourney и DALL.E 2 по одному и тому же текстовому описанию

В табл. 14.2 приводится краткое описание некоторых наиболее мощных из современных моделей преобразования текста в изображение.

Таблица 14.2. Модели преобразования текста в изображение

Модель	Дата	Разработчик	Количество параметров, млрд	Свободнодоступна
DALL.E 2	Апрель 2022	OpenAI	3,5	Нет
Imagen	Май 2022	Google	4,6	Нет
Parti	Июнь 2022	Google	20	Нет
Stable Diffusion	Август 2022	Stability AI, CompVis и Runway	0,89	Да
MUSE	Январь 2023	Google	3	Нет

Важным навыком работы с моделями преобразования текста в изображение является умение создать описание, одновременно описывающее изображение, которое нужно сгенерировать, и содержащее ключевые слова, побуждающие модель создать изображение в определенном стиле. Например, такие прилагательные, как «потрясающий» или «отмеченный наградами», часто можно

использовать для улучшения качества получаемого изображения. Однако не всегда одно и то же описание будет давать одинаково хорошие результаты в разных моделях — многое зависит от содержимого конкретного набора данных «текст — изображение», используемого для обучения модели. Искусство составления описаний, дающих хорошие результаты при работе с конкретной моделью, известно как *разработка описаний*.

Другие приложения

Генеративный ИИ быстро находит применение в самых разных областях, от обучения с подкреплением до создания других видов мультимодальных моделей *преобразования текста в X*.

Например, в ноябре 2022 года компания Meta опубликовала статью о *CICERO* (<https://oreil.ly/kBQvY>) — агенте ИИ, обученном играть в настольную игру «Дипломатия». В ней игроки представляют разные страны Европы до Первой мировой войны. Они должны вести переговоры и обманывать друг друга, чтобы получить контроль над континентом. Эта игра очень сложна для ИИ не в последнюю очередь потому, что в ней есть коммуникативный элемент, когда игроки должны обсуждать свои планы друг с другом, чтобы найти союзников, координировать свои действия и предлагать стратегические цели. Для этого в *CICERO* есть языковая модель, способная инициировать диалог и отвечать на сообщения других игроков. Крайне важно, чтобы диалог соответствовал стратегическим планам агента, которые генерируются другой частью модели и адаптируются к постоянно развивающемуся сценарию. Агент способен блефовать во время переговоров с другими игроками, стремясь убедить их сотрудничать с ним, только для того, чтобы потом совершить агрессивный маневр против игрока. Примечательно, что в анонимной онлайн-лиге «Дипломатия», включающей 40 игр, *CICERO* показал результат, более чем вдвое превышающий средний показатель игроков-людей, и вошел в число 10 % лучших участников, сыгравших несколько игр. Это отличный пример того, как генеративный ИИ можно успешно сочетать с обучением с подкреплением.

Разработка больших языковых моделей — интересная область исследований. Одним из примеров таких моделей может служить PaLM-E (<https://palm-e.github.io/>), созданная в Google. Эта модель сочетает мощную языковую модель PaLM и Vision Transformer для преобразования визуальных и сенсорных данных в лексемы, которые можно чередовать с текстовыми инструкциями, что позволяет роботам выполнять задачи на основе текстовых описаний и непрерывной обратной связи от других сенсорных модальностей. На сайте PaLM-E демонстрируются некоторые возможности модели, в том числе управление роботом для расстановки блоков и извлечения объектов с использованием текстовых описаний.

Модели *преобразования текста в видео* создают видеоролики на основе текстового описания. Эта область, основанная на преобразовании текста в изображение, сопряжена с дополнительной проблемой, связанной с включением измерения времени. Например, в сентябре 2022 года компания Meta объявила о создании *Make-A-Video* (<https://makeavideo.studio/>) — генеративной модели, позволяющей изготавливать короткие видеоролики исключительно на основе текстового описания. Модель также способна добавлять движение между двумя статическими изображениями и создавать различные варианты заданного исходного видео. Интересно отметить, что она обучается только на парных данных «текст — изображение» и видеоматериалах без описания, то есть в обучении не участвуют пары «текст — видео». Видеороликов без описаний вполне достаточно, чтобы модель могла узнать, как происходят движения в реальном мире, затем она использует пары «текст — изображение», чтобы научиться сопоставлять текст и изображения с последующим добавлением анимации. Модель *Dreamix* (<https://oreil.ly/F9wdw>) способна редактировать видео, преобразуя входное видео, как описано в заданной текстовой подсказке, сохраняя при этом другие стилистические атрибуты. Например, видео, где молоко наливается в стакан, можно преобразовать в видео, где в чашку наливается кофе, сохранив при этом ракурс камеры, фон и элементы освещения из исходного видео.

Точно так же модели преобразования текста в трехмерные изображения расширяют традиционные подходы преобразования текста в изображение, добавляя третье измерение. В сентябре 2022 года компания Google объявила о создании *DreamFusion* (<https://dreamfusion3d.github.io/>) — диффузионной модели, которая генерирует трехмерные ресурсы на основе текстового описания. Важно отметить, что для ее обучения не требуются маркированные трехмерные ресурсы. В качестве основы авторы используют предварительно обученную двумерную модель преобразования текста в изображение (Imagen), а затем обучают трехмерное нейронное поле излучения (Neural Radiance Field, NeRF) созданию качественных изображений, видимых под случайными углами. Другой пример — модель *Point-E* (<https://openai.com/research/point-e>), созданная в OpenAI в декабре 2022 года. Point-E — это система, основанная исключительно на моделях диффузии, способная генерировать трехмерное облако точек на основе заданного текстового описания. Качество получаемых изображений не такое высокое, как у DreamFusion, но у этой модели имеется преимущество — она намного быстрее модели на основе NeRF и может создавать изображения всего за одну-две минуты на одном графическом процессоре, тогда как NeRF требует для этого нескольких часов работы графического процессора.

Учитывая сходство текста и музыки, неудивительно, что неоднократно предпринимались попытки создания моделей преобразования *текста в музыку*. В январе 2023 года в Google была создана *MusicLM* (<https://oreil.ly/qb7II>) — языковая

модель, способная преобразовывать текстовое описание (например, «успокаивающая мелодия скрипки, сопровождаемая искаженным гитарным рифом») в аудиофрагмент продолжительностью звучания несколько минут, точно соответствующий описанию. Она основана на более ранней модели *AudioLM* (<https://oreil.ly/0EDRY>), в которую была добавлена возможность управления с помощью текстового описания. Примеры, которые можно послушать, доступны на сайте Google Research.

Будущее генеративного ИИ

В этом последнем разделе мы исследуем потенциальное влияние мощных систем генеративного ИИ на мир, в котором живем, — на нашу повседневную жизнь, работу и образование. Мы также перечислим ключевые практические и этические проблемы, с которыми придется столкнуться генеративному ИИ на пути превращения в повсеместно используемый инструмент, вносящий значительный положительный вклад в жизнь общества.

Генеративный ИИ в повседневной жизни

Вне всяких сомнений, в будущем генеративный ИИ станет играть все более важную роль в повседневной жизни людей, особенно это относится к большим языковым моделям. С помощью OpenAI ChatGPT (<https://chat.openai.com/chat>) уже можно создать идеальное заявление о приеме на работу, написать профессиональный ответ по электронной почте коллеге или опубликовать забавное сообщение на заданную тему в социальных сетях. Эта технология действительно интерактивна: она способна конкретизировать детали, отвечать на отзывы и задавать собственные вопросы, если что-то неясно. Такая ее роль *личного помощника* кажется чем-то из области научной фантастики, но это не так — она уже существует здесь и сейчас и любой желающий может ее использовать.

Что нас ждет, когда такого рода приложения войдут в обыденную жизнь? Вполне вероятно, что самым непосредственным эффектом будет повышение качества письменного общения. Доступ к большим языковым моделям с удобным интерфейсом позволит людям за секунды преобразовать набросок идеи в связанные предложения. Написание электронных писем, публикации в социальных сетях и даже обмен короткими мгновенными сообщениями в корне изменятся благодаря этой технологии. Она не только устранил общие барьеры, связанные с правописанием, грамматикой и удобочитаемостью, но и напрямую свяжет наши мыслительные процессы с полезными результатами и избавит от необходимости участвовать в формулировании предложений.

Создание грамотно построенного текста — это лишь один из вариантов применения больших языковых моделей. Люди начнут использовать их для генерации идей, советов и поиска информации. Я считаю, что мы можем рассматривать это как четвертый этап способности нашего вида приобретать, извлекать и синтезировать информацию, а также делиться ею. Сначала мы получали информацию от окружающих или совершали физические поездки в новые места для передачи своих знаний. Изобретение печатного станка позволило создать книгу, ставшую основным средством обмена идеями. Наконец, появление Интернета дало нам возможность мгновенно отыскать и получить информацию одним нажатием кнопки. Генеративный ИИ открывает новую эру синтеза информации, которая, я уверен, заменит многие из нынешних поисковых систем.

Например, набор моделей OpenAI GPT может предоставить индивидуальные рекомендации по выбору места отдыха (рис. 14.9), дать совет, как реагировать на сложную ситуацию, или представить подробное объяснение непонятной идеи. Использование этой технологии больше похоже на обращение к другу, чем на ввод запроса в поисковую систему, и по этой причине люди начинают быстро осваивать ее. ChatGPT — самая быстрорастущая технологическая платформа за всю историю: за пять дней после запуска она приобрела 1 млн пользователей. Для сравнения: Instagram потребовалось 2,5 месяца, чтобы охватить такое же количество пользователей, а Facebook — 10 месяцев.

Посоветуй мне три места для отдыха в Европе в глуши, где предлагаются как активные, так и спокойные развлечения для семьи с детьми-подростками. Приведи примеры, почему каждое из мест можно считать идеальным.

1. Рига, Латвия. Рига — отличное место отдыха для семьи с детьми-подростками. Она находится в глуши, и каждый тут найдет что-то для себя. Здесь предлагается широкий спектр развлечений от необычных видов спорта, таких как кайтсерфинг, каякинг и скалолазание, до более спокойных занятий, таких как экскурсии по достопримечательностям и историческим местам города. Это также отличная отправная точка для знакомства с сельской местностью Латвии, где возможны различные виды активного отдыха, такие как пешие походы и проживание в кемпинге.

2. Национальный парк «Плитвицкие озера», Хорватия. Национальный парк «Плитвицкие озера» — идеальное место отдыха для семьи с детьми-подростками. Здесь предлагается уникальное сочетание развлечений: от расслабляющих прогулок на лодке и круизов по озеру до захватывающего спуска на тросе и пеших прогулок. В парке также находятся одни из самых потрясающих каскадов водопадов в Европе, что делает его прекрасным местом для того, чтобы наслаждаться невероятными видами.

3. Лофотенские острова, Норвегия. Лофотенские острова в Норвегии — отличное неизведанное место отдыха для семьи с детьми-подростками. На этом отдаленном архипелаге с его красивыми фьордами, потрясающими морскими видами и богатой дикой природой есть что исследовать. Здесь также есть чем заняться: от рыбалки и катания на каяках до пеших прогулок по побережью. Это отличное сочетание приключений и отдыха.

Рис. 14.9. GPT-3 дает индивидуальные рекомендации по выбору места отдыха

Генеративный ИИ на рабочем месте

Генеративный ИИ найдет применение не только в повседневной жизни, но и в конкретных профессиях, требующих творческого подхода. Далее перечислены несколько примеров профессий, где генеративный ИИ может принести пользу.

Реклама

Генеративный ИИ можно применять для разработки персонализированных рекламных кампаний, ориентированных на определенную демографическую группу на основе истории посещений и покупок.

Сочинение музыки

Генеративный ИИ можно задействовать для создания оригинальных музыкальных треков, что открывает безграничные возможности.

Архитектура

Генеративный ИИ можно применять для проектирования зданий и сооружений с учетом таких факторов, как стиль и ограничения, связанные с планировкой.

Моделирование одежды

Генеративный ИИ можно использовать для создания уникального и разнообразного дизайна одежды с учетом тенденций и предпочтений пользователей.

Автомобильный дизайн

Генеративный ИИ можно применять для проектирования и разработки новых моделей автомобилей и поиска интересных вариаций конкретной конструкции.

Производство фильмов и видео

Генеративный ИИ можно использовать для создания спецэффектов и анимации, а также диалогов, сцен и целых сюжетных линий.

Фармацевтические исследования

Генеративный ИИ можно задействовать в создании новых лекарственных компонентов при разработке новых методов лечения.

Письменное творчество

Генеративный ИИ можно использовать для создания произведений письменного творчества, например художественных рассказов, стихов, новостных статей и т. д.

Игровой дизайн

Генеративный ИИ можно применять для проектирования и разработки новых игровых уровней и сцен, создавая бесконечное разнообразие игровых впечатлений.

Цифровой дизайн

Генеративный ИИ можно задействовать для создания оригинальных произведений цифрового искусства и анимации, а также для проектирования и разработки новых пользовательских интерфейсов и веб-дизайна.

Часто говорят, что ИИ может оставить не у дел работающих в подобных областях, но я не верю, что такое возможно. Для меня ИИ — это просто еще один инструмент в арсенале творческих личностей (хотя и очень мощный), а не их замена. Те, кто решит использовать эту новую технологию, обнаружат, что они могут гораздо быстрее исследовать новые и перебирать имеющиеся идеи, что было недоступно в прошлом.

Генеративный ИИ в образовании

Еще одна сфера повседневной жизни, на которую может быть оказано существенное влияние, — образование. Генеративный ИИ бросает вызов фундаментальным аксиомам образования. Нечто подобное происходило только во времена становления Интернета. Сеть дала студентам возможность мгновенно получать однозначную информацию, в результате чего экзамены, проверявшие исключительно способность к запоминанию, стали казаться старомодными и неактуальными. Это вызвало изменение подхода, и теперь проверяют способность учащихся синтезировать идеи новым способом, а не только наличие фактических знаний.

Я считаю, что генеративный ИИ вызовет еще один важный сдвиг в сфере образования и потребует переоценки и корректировки существующих методов обучения и критериев оценки. Если каждый студент получит доступ к машине, способной писать эссе и генерировать новые ответы на вопросы, то какой будет цель курсовой работы на основе эссе?

Многие предлагают запретить использование таких инструментов ИИ, приравняв результаты его деятельности к плагиату. Однако все не так просто: выяснить, что текст сгенерирован искусственным интеллектом, гораздо сложнее, чем обнаружить плагиат, и еще труднее доказать это. Более того, студенты могут с помощью инструментов ИИ создать эскизный проект эссе, а затем добавить некоторые детали или исправить неверные факты. Что в данном случае получится — оригинальная работа ученика или работа ИИ?

Очевидно, что это очень важный вопрос, который необходимо решить, чтобы образование и сертификация сохранили свою ценность. На мой взгляд, нет смысла сопротивляться распространению инструментов ИИ в сфере образования — это обречено на провал, потому что ИИ получит настолько широкое распространение в повседневной жизни, что попытки ограничить его применение будут бесполезны. Вместо этого нужно найти способы освоить эту технологию и задаться вопросом, как можно разрабатывать курсовые проекты с помощью ИИ, подобно тому как мы разрешаем писать курсовые с использованием книг и поощряем студентов исследовать материал, задействуя Интернет и инструменты ИИ.

Потенциал генеративного ИИ при оказании помощи в самом процессе обучения тоже огромен. Репетитор на базе искусственного интеллекта может помочь студенту изучить новую тему (рис. 14.10), преодолеть недопонимание или составить персонализированный план обучения. Задача отделения правды от вымысла ничем не отличается от той, которую мы сейчас решаем в отношении информации, доступной в Интернете, и представляет собой ценный навык, требующий дальнейшего осмысления во всей учебной программе.

Объясни старшекласснику разницу между генеративным и дискриминативным ИИ.

Генеративный ИИ использует алгоритмы для формирования новых данных на основе существующих. В основном применяется для создания новых идей или объектов, таких как музыка, текст, изображения и т. д. Он пытается уловить основную структуру и закономерности в данных и с помощью них сгенерировать новые данные из того же распределения.

Дискриминативный ИИ использует алгоритмы для классификации или различения двух разных предметов. В основном применяется для таких задач, как распознавание изображений, обработка естественного языка и распознавание речи. Он пытается выявить различия между элементами, а затем использовать эти различия для их классификации.

Рис. 14.10. Вывод GPT-3 — пример использования большой языковой модели для обучения

Генеративный ИИ может стать невероятно мощным инструментом, позволяющим уравнивать правила игры для тех, у кого есть доступ к отличным преподавателям и лучшим учебным материалам, и тех, у кого ничего этого нет. Я рад видеть прогресс в этой области, поскольку считаю, что он может раскрыть огромный потенциал многих талантливых людей по всему миру.

Практические и этические проблемы генеративного ИИ

Несмотря на невероятный прогресс, достигнутый в области генеративного ИИ, остается еще много проблем, которые предстоит преодолеть. Некоторые из них носят практический характер, другие — этический.

Например, основной критикуемый недостаток больших языковых моделей — их склонность генерировать неверную информацию в ответ на вопрос по незнакомой или противоречивой теме (см. рис. 14.4). Опасность заключается в том, что трудно проверить точность информации, содержащейся в сгенерированном ответе. Даже если попросить LLM объяснить свои доводы или процитировать источники, она может привести ссылки или высказать ряд утверждений, которые логически не связаны друг с другом. Эту проблему нелегко решить, потому что LLM — это всего лишь набор весов, фиксирующих наиболее вероятное следующее слово с учетом набора входных слов. У нее нет *банкастинной* информации, которую она могла бы использовать для справки.

Потенциальное решение этой проблемы — предоставление большим языковым моделям возможности задействовать структурированные инструменты, такие как калькуляторы, компиляторы кода и онлайн-источники информации, для задач, требующих предоставления точных фактов. Например, на рис. 14.11 показаны выходные данные модели *Toolformer*, выпущенной компанией Meta в феврале 2023 года [Schick et al., 2023].



Рис. 14.11. Пример того, как Toolformer может вызывать разные API, чтобы получить точную информацию для ответа
(источник: [Schick et al., 2023], <https://arxiv.org/pdf/2302.04761.pdf>)

Модель Toolformer может явно вызывать API для получения информации в процессе генерирования ответа, например использовать API Википедии для получения информации о конкретном человеке, а не полагаться на возможное ее встраивание в веса модели. Этот подход полезен также при выполнении точных математических операций, для вычисления которых Toolformer может обратиться к API калькулятора вместо попытки сгенерировать ответ методом авторегрессии.

Еще одна заметная этическая проблема генеративного ИИ связана с тем фактом, что крупные компании применяют для обучения своих моделей огромные объемы данных, извлеченных из Интернета, хотя создатели этих данных не давали на это явного согласия. Часто эти данные даже не выкладываются в публичный доступ, поэтому невозможно узнать, используются ли ваши данные для обучения больших языковых или мультимодальных моделей. Очевидно, что это серьезная проблема, особенно для художников, которые могут утверждать, что создатели моделей задействовали их произведения, не заплатив никаких гонораров или комиссий. Более того, имя художника может использоваться в качестве подсказки для создания большого количества произведений искусства, схожих по стилю с оригиналами, тем самым умаляя уникальность картин и превращая стиль в товар.

Пионером на пути решения этой проблемы стала компания Stability AI, чья мультимодальная модель Stable Diffusion обучается на подмножестве набора общедоступных данных LAION-5B. Они также запустили сайт *Have I Been Trained?* (<https://haveibeentrained.com/>), где любой может выполнить поиск конкретного изображения или текстового фрагмента в наборе обучающих данных и запретить использовать его данные для обучения модели в дальнейшем. Это возвращает контроль над данными первоначальным создателям и гарантирует прозрачность данных, которые применяются для создания подобных мощных инструментов. Однако эта практика пока не получила широкого распространения, и многие разработчики коммерческих моделей генеративного ИИ не публикуют свои наборы данных или веса и не дают никакой возможности запретить использовать те или иные данные в процессе обучения.

В заключение хочу отметить, что генеративный ИИ является мощным инструментом для общения в повседневной жизни и на рабочем месте, повышения продуктивности обучения в сфере образования, но его широкое использование имеет как преимущества, так и недостатки. Важно осознавать потенциальные риски применения результатов деятельности генеративной модели ИИ и подходить к этому ответственно. Тем не менее я по-прежнему с оптимизмом смотрю на будущее генеративного ИИ и с нетерпением жду возможности увидеть, как предприятия и люди адаптируются к этой новой интересной технологии.

Заключительные комментарии

В этой книге мы совершили путешествие по последнему десятилетию исследований в области генеративного моделирования, начав с основных идей, положенных в основу вариационных автокодировщиков, генеративно-состязательных сетей, авторегрессионных моделей, моделей нормализующих потоков, моделей на основе энергии и диффузионных моделей, и, опираясь на эти основы, постарались понять, как современные модели (VQ-GAN, трансформеры, модели мира и мультимодальные модели) раздвигают границы способности генеративных моделей решать множество задач.

Я уверен, что в будущем генеративное моделирование сможет стать ключом к более глубокой форме искусственного интеллекта, позволяющей машинам выйти за рамки любой конкретной задачи и формулировать собственные стратегии, вознаграждения и в конечном счете постигать свое окружение. Моя убежденность тесно связана с принципом *активного вывода*, впервые предложенным Карлом Фристоном (Karl Friston). Обсуждение теории, лежащей в основе концепции активного вывода, могло бы легко занять целый том, что и подтверждает замечательная книга Томаса Парра (Thomas Parr) и др. *Active Inference: The Free Energy Principle in Mind, Brain, and Behavior* (MIT Press), которую я настоятельно рекомендую и поэтому попытаюсь дать здесь краткую выжимку.

С самого младенчества мы постоянно исследуем свое окружение, выстраивая ментальную модель возможных вариантов будущего без какой-либо очевидной цели — просто чтобы лучше понимать мир. На данных, которые мы получаем, нет никаких меток — только кажущиеся случайными потоки световых и звуковых волн, бомбардирующие наши органы чувств с момента рождения. Даже когда мама или папа показывает нам яблоко и говорит: «Яблоко», наш юный мозг не получает никакой подсказки, помогающей понять, что свет, попадающий в наш глаз в этот конкретный момент, каким-то образом связан со звуковыми волнами, достигшими ушей. У нас нет обучающих наборов звуков и образов, запахов и вкусов, действий и вознаграждений — только бесконечный поток чрезвычайно искаженных данных.

И все же сейчас, читая эти строки, вы, возможно, наслаждаетесь вкусом кофе в шумном кафе. Вы не обращаете внимания на фоновый шум, потому что концентрируетесь на преобразовании неосвещенных участков на крошечной части вашей сетчатки в последовательность абстрактных понятий, которые почти не передают смысла по отдельности, но при объединении вызывают волну одновременно возникающих представлений в вашем разуме — образы, эмоции, идеи, убеждения и потенциальные действия заполняют сознание и ожидают, когда вы узнаете их. Поток данных, почти бессмысленный для детского мозга,

с возрастом становится уже не таким бессмысленным. Все обретает для вас свой смысл. Вы видите структуру везде. Вы уже не удивляетесь физике повседневной жизни. Вы воспринимаете мир таким, какой он есть, потому что ваш мозг решил, что так и должно быть. В этом смысле мозг представляет собой чрезвычайно сложную генеративную модель, оснащенную способностью уделять внимание определенной части входных данных, формировать представления в скрытом пространстве нейронных путей и постепенно обрабатывать последовательные данные.

Активный вывод — это структура, базирующаяся на данной концепции и объясняющая, как мозг обрабатывает и использует сенсорную информацию для принятия решений и выполнения действий. Она утверждает, что у организма есть генеративная модель мира, в котором он обитает, и он применяет эту модель для прогнозирования будущих событий. Чтобы уменьшить замешательство, вызванное несоответствием модели и реальности, организм определенным образом корректирует свои действия и убеждения. Ключевая идея Фристона заключается в том, что оптимизацию действий и восприятия можно рассматривать как две стороны одной медали, причем обе стремятся минимизировать величину, известную как *свободная энергия*.

В основе этой структуры лежит генеративная модель окружающей среды (запечатленная в мозгу), которая постоянно сравнивается с реальностью. Важно отметить, что мозг не является пассивным наблюдателем событий. Он прикреплен к шее и набору конечностей, которые могут перемещать основные входные датчики в любое местоположение относительно источника входных данных. Последовательность возможных вариантов будущего зависит не только от понимания физики окружающей среды, но и от понимания *самого себя* и возможных способов действий. Эта петля обратной связи действия и восприятия чрезвычайно интересна для меня, и я считаю, что мы затронули лишь самый верхний слой возможностей генеративных моделей, способных предпринимать действия в данной среде в соответствии с принципами активного вывода.

Это простая концепция, которая, как я полагаю, в ближайшие десять лет будет привлекать внимание к генеративному моделированию как к одному из ключей к искусственному интеллекту в целом. Учитывая это, я призываю вас продолжать узнавать больше о генеративных моделях из замечательных материалов, доступных в Интернете и в других книгах. Благодарю вас за то, что нашли время дочитать эту книгу до конца. Надеюсь, вам было так же приятно читать, как мне — генерировать ее!

Ссылки

Ackley D. H. et al. A Learning Algorithm for Boltzmann Machines. 1985, Cognitive Science 9 (1), 147–165.

Alayrac J.-B. et al. Flamingo: A Visual Language Model for Few-Shot Learning. April 29, 2022 // <https://arxiv.org/abs/2204.14198>.

Arjovsky M. et al. Wasserstein GAN. January 26, 2017 // <https://arxiv.org/abs/1701.07875>.

Brock A. et al. High-Performance Large-Scale Image Recognition Without Normalization. February 11, 2021 // <https://arxiv.org/abs/2102.06171>.

Brock A. et al. Large Scale GAN Training for High Fidelity Natural Image Synthesis. September 28, 2018 // <https://arxiv.org/abs/1809.11096>.

Brundage M. et al. The Malicious Use of Artificial Intelligence: Forecasting, Prevention, and Mitigation. February 20, 2018 // https://www.eff.org/files/2018/02/20/malicious_ai_report_final.pdf.

Chang H. et al. Muse: Text-to-Image Generation via Masked Generative Transformers. January 2, 2023 // <https://arxiv.org/abs/2301.00704>.

Chen M. et al. Evaluating Large Language Models Trained on Code. July 7, 2021 // <https://arxiv.org/abs/2107.03374>.

Cho K. et al. Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation. June 3, 2014 // <https://arxiv.org/abs/1406.1078>.

Devlin J. et al. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. October 11, 2018 // <https://arxiv.org/abs/1810.04805>.

Dinh L. et al. Density Estimation Using Real NVP. May 27, 2016 // <https://arxiv.org/abs/1605.08803v3>.

Dong H.-W. et al. MuseGAN: Multi-Track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment. September 19, 2017 // <https://arxiv.org/abs/1709.06298>.

Dosovitskiy A. et al. An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale. October 22, 2020 // <https://arxiv.org/abs/2010.11929v2>.

- Du Y., Mordatch I.* Implicit Generation and Modeling with Energy-Based Models. March 20, 2019 // <https://arxiv.org/abs/1903.08689>.
- Dumoulin V., Visin F.* A Guide to Convolution Arithmetic for Deep Learning. January 12, 2018 // <https://arxiv.org/abs/1603.07285>.
- Esser P. et al.* Taming Transformers for High-Resolution Image Synthesis. December 17, 2020 // <https://arxiv.org/abs/2012.09841>.
- Goodfellow I.J. et al.* Generative Adversarial Nets. June 10, 2014 // <https://arxiv.org/abs/1406.2661>.
- Grathwohl W. et al.* FFJORD: Free-Form Continuous Dynamics for Scalable Reversible Generative Models. October 22, 2018 // <https://arxiv.org/abs/1810.01367>.
- Gulrajani I. et al.* Improved Training of Wasserstein GANs. March 31, 2017 // <https://arxiv.org/abs/1704.00028>.
- Ha D.* A Visual Guide to Evolution Strategies. October 29, 2017 // <https://blog.otoro.net/2017/10/29/visual-evolution-strategies>.
- Ha D., Schmidhuber J.* World Models. March 27, 2018 // <https://arxiv.org/abs/1803.10122>.
- He K. et al.* Deep Residual Learning for Image Recognition. December 10, 2015 // <https://arxiv.org/abs/1512.03385>.
- Hinton G. E.* Training Products of Experts by Minimizing Contrastive Divergence. 2002 // <https://www.cs.toronto.edu/~hinton/absps/tr00-004.pdf>.
- Hinton et al.* Networks by Preventing Co-Adaptation of Feature Detectors. July 3, 2012 // <https://arxiv.org/abs/1207.0580>.
- Ho J. et al.* Denoising Diffusion Probabilistic Models. June 19, 2020 // <https://arxiv.org/abs/2006.11239>.
- Hochreiter S., Schmidhuber J.* Long Short-Term Memory, Neural Computation 9 (1997): 1735–1780 // <https://www.bioinf.jku.at/publications/older/2604.pdf>.
- Hoffmann J. et al.* Training Compute-Optimal Large Language Models. March 29, 2022 // <https://arxiv.org/abs/2203.15556v1>.
- Hou X. et al.* Deep Feature Consistent Variational Autoencoder. October 2, 2016 // <https://arxiv.org/abs/1610.00291>.
- Huang C.-Z. A. et al.* Music Transformer: Generating Music with Long-Term Structure. September 12, 2018 // <https://arxiv.org/abs/1809.04281>.

Huang X., Belongie S. Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization. March 20, 2017 // <https://arxiv.org/abs/1703.06868>.

Ioffe S., Szegedy C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. February 11, 2015 // <https://arxiv.org/abs/1502.03167>.

Isola P. et al. Image-to-Image Translation with Conditional Adversarial Networks. November 21, 2016 // <https://arxiv.org/abs/1611.07004v3>.

Karras T. et al. A Style-Based Generator Architecture for Generative Adversarial Networks. December 12, 2018 // <https://arxiv.org/abs/1812.04948>.

Karras T. et al. Analyzing and Improving the Image Quality of StyleGAN. December 3, 2019 // <https://arxiv.org/abs/1912.04958>.

Karras T. et al. Progressive Growing of GANs for Improved Quality, Stability, and Variation. October 27, 2017 // <https://arxiv.org/abs/1710.10196>.

Kingma D., Ba J. Adam: A Method for Stochastic Optimization. December 22, 2014 // <https://arxiv.org/abs/1412.6980v8>.

Kingma D. P., Welling M. Auto-Encoding Variational Bayes. December 20, 2013 // <https://arxiv.org/abs/1312.6114>.

Kingma D. P., Dhariwal P. Glow: Generative Flow with Invertible 1x1 Convolutions. July 10, 2018 // <https://arxiv.org/abs/1807.03039>.

Krizhevsky A. Learning Multiple Layers of Features from Tiny Images. April 8, 2009 // <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.

Larsen A. B. L. et al Autoencoding Beyond Pixels Using a Learned Similarity Metric. December 31, 2015 // <https://arxiv.org/abs/1512.09300>.

Lecun Y. et al A Tutorial on Energy-Based Learning. 2006 // https://www.researchgate.net/publication/200744586_A_tutorial_on_energy-based_learning.

Liu Z. et al. Large-Scale CelebFaces Attributes (CelebA) Dataset. 2015 // <http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html>.

Mildenhall B. et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. March 1, 2020 // <https://arxiv.org/abs/2003.08934>.

Mirza M., Osindero S. Conditional Generative Adversarial Nets. November 6, 2014 // <https://arxiv.org/abs/1411.1784>.

Nichol A., Dhariwal P. Improved Denoising Diffusion Probabilistic Models. February 18, 2021 // <https://arxiv.org/abs/2102.09672>.

Nichol A. et al. GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models. December 20, 2021 // <https://arxiv.org/abs/2112.10741>.

Odena A. et al. Deconvolution and Checkerboard Artifacts. October 17, 2016 // <https://distill.pub/2016/deconv-checkerboard>.

Oord van den A. et al. Neural Discrete Representation Learning. November 2, 2017 // <https://arxiv.org/abs/1711.00937v2>.

Oord van den A. et al. Pixel Recurrent Neural Networks. August 19, 2016 // <https://arxiv.org/abs/1601.06759>.

Ouyang L. et al. Training Language Models to Follow Instructions with Human Feedback. March 4, 2022 // <https://arxiv.org/abs/2203.02155>.

Radford A. et al. Improving Language Understanding by Generative PreTraining. June 11, 2018 // <https://openai.com/research/language-unsupervised>.

Radford A. et al. Learning Transferable Visual Models From Natural Language Supervision. February 26, 2021 // <https://arxiv.org/abs/2103.00020>.

Radford A. et al. Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks. January 7, 2016 // <https://arxiv.org/abs/1511.06434>.

Raffel C. et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. October 23, 2019 // <https://arxiv.org/abs/1910.10683>.

Ramachandran P. et al. Searching for Activation Functions. October 16, 2017 // <https://arxiv.org/abs/1710.05941v2>.

Ramesh A. et al. Hierarchical Text-Conditional Image Generation with CLIP Latents. April 13, 2022 // <https://arxiv.org/abs/2204.06125>.

Ramesh A. et al. Zero-Shot Text-to-Image Generation. February 24, 2021 // <https://arxiv.org/abs/2102.12092>.

Rombach R. et al. High Resolution Image Synthesis with Latent Diffusion Models. December 20, 2021 // <https://arxiv.org/abs/2112.10752>.

Saharia C. et al. Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding. May 23, 2022 // <https://arxiv.org/abs/2205.11487>.

Salimans T. et al. PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood and Other Modifications. January 19, 2017 // <http://arxiv.org/abs/1701.05517>.

- Sauer A. et al.* StyleGAN-XL: Scaling StyleGAN to Large Diverse Datasets. February 1, 2022 // <https://arxiv.org/abs/2202.00273v2>.
- Schick T. et al.* Toolformer: Language Models Can Teach Themselves to Use Tools. February 9, 2023 // <https://arxiv.org/abs/2302.04761>.
- Shen S. et al.* PowerNorm: Rethinking Batch Normalization in Transformers. June 28, 2020 // <https://arxiv.org/abs/2003.07845>.
- Smith S. L. et al.* Don't Decay the Learning Rate, Increase the Batch Size. November 1, 2017 // <https://arxiv.org/abs/1711.00489>.
- Sohl-Dickstein J. et al.* Deep Unsupervised Learning Using Nonequilibrium Thermodynamics. March 12, 2015 // <https://arxiv.org/abs/1503.03585>.
- Song Y., Ermon S.* Generative Modeling by Estimating Gradients of the Data Distribution. July 12, 2019 // <https://arxiv.org/abs/1907.05600>.
- Song Y., Ermon S.* Improved Techniques for Training Score-Based Generative Models. June 16, 2020 // <https://arxiv.org/abs/2006.09011>.
- Song J. et al.* Denoising Diffusion Implicit Models. October 6, 2020 // <https://arxiv.org/abs/2010.02502>.
- Srivastava N. et al.* Dropout: A Simple Way to Prevent Neural Networks from Overfitting // Journal of Machine Learning Research 15 (2014): 1929–1958 // <http://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>.
- Touvron H. et al.* LLaMA: Open and Efficient Foundation Language Models. February 27, 2023 // <https://arxiv.org/abs/2302.13971>.
- Vaswani A. et al.* Attention Is All You Need. June 12, 2017 // <https://arxiv.org/abs/1706.03762>.
- Welling M., Teh Y. W.* Bayesian Learning via Stochastic Gradient Langevin Dynamics. 2011 // <https://www.stats.ox.ac.uk/~teh/research/compstats/WelTeh2011a.pdf>.
- Woodford O.* Notes on Contrastive Divergence. 2006 // <https://www.robots.ox.ac.uk/~ojw/files/NotesOnCD.pdf>.
- Wu C. et al.* Visual ChatGPT: Talking, Drawing and Editing with Visual Foundation Models. March 8, 2023 // <https://arxiv.org/abs/2303.04671>.
- Yu J., Koh J. Y.* Vector-Quantized Image Modeling with Improved VQGAN. May 18, 2022 // <https://ai.googleblog.com/2022/05/vector-quantized-image-modeling-with.html>.
- Yu J. et al.* Vector-Quantized Image Modeling with Improved VQGAN. October 9, 2021 // <https://arxiv.org/abs/2110.04627>.

Zhang L., Agrawala M. Adding Conditional Control to Text-to-Image Diffusion Models. February 10, 2023 // <https://arxiv.org/abs/2302.05543>.

Zhang H. et al. Self-Attention Generative Adversarial Networks. May 21, 2018 // <https://arxiv.org/abs/1805.08318>.

Zhu J.-Y. et al. Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. March 30, 2017 // <https://arxiv.org/abs/1703.10593>.

Об авторе

Дэвид Фостер (David Foster) — специалист по данным, предприниматель и преподаватель, специализирующийся на применении искусственного интеллекта в области творчества. Как соучредитель Applied Data Science Partners (ADSP) помогает организациям использовать преобразующую силу данных и искусственного интеллекта. Имеет степень магистра математики, полученную в Тринити-колледже (Кембридж), степень магистра операционного анализа, полученную в Уорикском университете, и преподает в институте машинного обучения, специализируясь на практическом применении искусственного интеллекта и решении реальных задач. В круг его исследовательских интересов входит повышение прозрачности и интерпретируемости алгоритмов искусственного интеллекта. Он также опубликовал несколько статей по объяснимому машинному обучению в здравоохранении.

Иллюстрация на обложке

На обложке книги изображен краснохвостый попугай (*Pyrhura picta*). Род *Pyrhura* относится к семейству *Psittacidae* — одному из трех семейств попугаев. К его подсемейству *Arinae* принадлежат несколько видов ара и попугаев из Западного полушария. Краснохвостый попугай обитает в прибрежных лесах и горах северо-восточной части Южной Америки.

Большая часть тела краснохвостого попугая покрыта ярко-зелеными перьями, но над клювом перья окрашены в синий цвет, на верхней части головы — в коричневый, а на груди и хвосте — в красный. Самое интересное, что перья на шее краснохвостого попугая похожи на чешую: коричневые в центре с белой окантовкой. Такая комбинация цветов помогает птице маскироваться.

Краснохвостые попугаи обычно обитают в тропическом лесу, где зеленое оперение маскирует их лучше всего. Собираясь в стаи от 5 до 12 птиц, они кормятся разнообразными фруктами, семенами и цветами. Иногда краснохвостые попугаи употребляют в пищу водоросли из лесных водоемов. Взрослые особи вырастают до 23 сантиметров в длину и живут от 13 до 15 лет. Обычно краснохвостый попугай откладывает до пяти яиц.

Многие животные, изображаемые на обложках книг издательства O'Reilly, находятся под угрозой вымирания, но все они очень важны для нашего мира. Чтобы узнать, как вы можете помочь сохранить их, посетите сайт animals.oreilly.com.

Иллюстрацию нарисовала Карен Монтгомери (Karen Montgomery), взяв за основу черно-белую гравюру из книги *Shaw's Zoology*.

Дэвид Фостер

**Генеративное глубокое обучение.
Как не мы рисуем картины,
пишем романы и музыку
2-е международное издание**

Перевела с английского Л. Киселева

Изготовлено в России. Изготовитель: ТОО «Спринт Бук».

Место нахождения и фактический адрес: 010000, Казахстан, город Астана, район Алматы,
Проспект Рахымжан Кошкарбаев, д. 10/1, н. п. 18.

Дата изготовления: 03.2024. Наименование: книжная продукция. Срок годности: не ограничен.

Подписано в печать 09.02.24. Формат 70×100/16. Бумага офсетная. Усл. п. л. 36,120. Тираж 1000. Заказ 0000.



Евгений Брикман

TERRAFORM: ИНФРАСТРУКТУРА НА УРОВНЕ КОДА

3-е международное издание

Terraform — настоящая звезда в мире DevOps. Эта технология позволяет управлять облачной инфраструктурой как кодом (IaC) на облачных платформах и платформах виртуализации, включая AWS, Google Cloud, Azure и др. Третье издание было полностью переработано и дополнено, чтобы вы могли быстро начать работу с Terraform.

Евгений (Джим) Брикман знакомит вас с примерами кода на простом декларативном языке программирования Terraform, иллюстрирующими возможность развертывания инфраструктуры и управления ею с помощью команд. Умудренные опытом системные администраторы, инженеры DevOps и начинающие разработчики быстро перейдут от основ Terraform к использованию полного стека, способного поддерживать трафик огромного объема и большую команду разработчиков.

Джейк Вандер Плас

PYTHON ДЛЯ СЛОЖНЫХ ЗАДАЧ: НАУКА О ДАННЫХ

2-е международное издание



Python — первоклассный инструмент, и в первую очередь благодаря наличию множества библиотек для хранения, анализа и обработки данных. Отдельные части стека Python описываются во многих источниках, но только в новом издании «Python для сложных задач» вы найдете подробное описание: IPython, NumPy, pandas, Matplotlib, Scikit-Learn и др.

Специалисты по обработке данных, знакомые с языком Python, найдут во втором издании решения таких повседневных задач, как обработка, преобразование и подготовка данных, визуализация различных типов данных, использование данных для построения статистических моделей и моделей машинного обучения. Проще говоря, эта книга является идеальным справочником по научным вычислениям в Python.



Евгений Брикман

ЭВОЛЮЦИОННАЯ АРХИТЕКТУРА. АВТОМАТИЗИРОВАННОЕ УПРАВЛЕНИЕ ПРОГРАММНЫМ ОБЕСПЕЧЕНИЕМ

2-е международное издание

Новые инструменты, фреймворки методики и парадигмы вновь и вновь меняют экосистему разработки программного обеспечения. Непрерывный прогресс основных практик разработки на протяжении последних пяти лет заставил искать новые пути и подходы к архитектуре, чтобы соответствовать постоянно меняющимся требованиям пользователей. В обновленном издании авторы Нил Форд, Ребекка Парсонс, Патрик Куа и Прамод Садаладж приводят реальные примеры, соответствующие потребностям современной разработки ПО.

«Эта книга знаменует собой важную веху, обозначающую нынешний уровень понимания проблемы. По мере того как люди начинают осознавать роль ПО в XXI веке, информация о том, как реагировать на изменения, сохраняя достигнутое, становится важнейшим навыком в области создания программного обеспечения». — *Мартин Фаулер*